Robustness and Typed Hypernet Contexts

A mathematical commentary on the Lean development

Companion to `danghica/hypernets-lean`

27 October 2026

Source snapshot: `dbb4d4c`; Lean `4.35.0`

Abstract

This document explains every definition and proved statement in the Lean development, including its regression examples, in mathematical English. Each declaration links to its complete source fragment at a fixed Git revision. The abstract counting-simulation soundness theorem is proved outright; the forward and converse robustness arguments are proved abstractly, and their specimen locality interface now has concrete named-state and state-class instances. Literal class determinism and both concrete counting simulations are proved; concrete observations, initial lifting, and both contextual clauses are now proved for the fixed-depth machine. The unbounded full-machine theorem is also proved in the concrete encoding. Section 1 explains its correspondence with the paper's Theorem 8.11 and the remaining context-closure and substitution-stability audit questions.

The concrete development supplies finite typed graphs, well-formed substitution, operation-path uniqueness, finite boxed hierarchies, exact hole inventories and freshness criteria, recursive isomorphism, transported sites, and nested substitution laws. Globally focus-free and focussed hypernets, actual states, focus insertion and erasure, both round trips, and recursive renaming invariance are formalized. The five coordinate search rules induce deterministic search on recursive state classes. Projection from literal-label classes preserves and lifts executions with exact lengths and reflects search termination. Concrete rootedness and refocusing are defined; search preserves rootedness. Rooted focus sources have operation paths from the unique input, proving the concrete Lemma 23 and the non-operation-path obstruction of Lemma 26(1).

One-way hypernets have their term-to-term path definition, renaming invariance, and filling closure. Path extraction proves the single-hole obstruction of Lemma 26(2), including self-looping holes. Simultaneous shallow substitution now has proved incidence, finiteness, and typing invariants. A concrete pre-template and shallow-specimen representation supplies actual filled states. Output-closure yields the multi-hole obstruction of Lemma 26(3) for these shallow specimens, allowing the one-way side to vary at different holes. The recursive deep-specimen extension now derives global focus freedom through filled box bodies and proves the same Lemma 26(3), boundary, and jointly rooted context-path conclusions for actual deep-filled states.

Context search transports through recursive fillings with exact lengths and reflects away from shallow hole boundaries. Erasure now commutes with expansion, identifying filled-coordinate realizations with the original specimen states. Concrete entering/exiting classification and the non-exiting conclusion of Lemma 39(2) are proved. Settled rooted specimens admit synchronized actual search steps to rooted endpoints at a remote coordinate. Repackaging this successor as an allowed specimen and proving its full-UAM context-step law are now complete.

Independent original-graph adjacency and incidence equations now characterize erasure-based state search in both directions. The equivalence also identifies recursive endpoint renaming. Finite two-edge interaction fragments now factor through explicit focus-free simple hole contexts in both directions, proving search adequacy. The proof covers arbitrary simple contexts and self-looping attachments.

Concrete substitution now applies the explicit Fig. 9(f) fragments in one simple context. Admissible compute behaviour completes the full UAM transition relation, with proved mode discipline, disjointness, and final normality. Actual executions test variable removal and compute behaviour.

Every finite simultaneous shallow expansion is now realized by ordinary addressed substitutions, with a proved finite length bound and order independence up to graph isomorphism. Independent substitutions now commute at arbitrary depth with constructed residual addresses. Hosts with selected sites are quotiented by structural renaming, and substitution and exact freshness descend to these classes. The dependency map distinguishes these checked results from the remaining obligations. Recursive specimens now have complete finite original-hole tuples, with distinct names and closed related replacements. Concrete quasi-specimens, input-safety, and robustness retain the full paper clauses. Full-UAM determinism and refocusing are now reduced to the corresponding behaviour properties, with substitution rootedness and uniqueness proved. The named-state specimen interface and its safety and robustness bridges are now constructed. The full class machine is literally deterministic under the behaviour hypotheses and has concrete forward and converse counting simulations. Concrete initial observation lifting and both contextual clauses are now proved. The operational transfer to the union of all finite depths is proved. This establishes the formal result, subject to the paper-correspondence qualifications in Section 1. Outer set-presentation normalization and depth coherence are proved. A checked freshening translation now resolves the box-label correspondence: its labels are actual graphs, it satisfies the paper's equal-label/equal-signature condition, and encoding and decoding are inverse on recursive graph classes. The earlier counterexample to naive binder erasure is repaired without restricting the original typed graphs. The copying/store extension for Proposition 12.2 now has scoped syntax, capture-avoiding substitution algebra, a separate typed graph hierarchy, canonical contraction trees, recursive renaming, and nested replacement laws. The full Figure 16 translation and intrinsic copying search laws are now checked. Reference allocation now has a concrete local rule, fresh-store proofs, and a checked execution of the unchanged reference translation. Dereferencing now has a general local rule with preserved store identity and an exact execution to copy request for arbitrary atom trees. The literal copy rule now preserves residual trees, nested boxes and shared stores, with proved representation laws. Stable assignment now has complete contextual successor-state coherence, retaining store identity, the new value and weakened old contents. Its successful cost with arbitrary attached contents is exactly seven, including unchanged translated-data regressions and all finite depth padding. Both atom-equality cases now have complete successor-state coherence for independently chosen contextual matches, and exactly seven successful steps with arbitrary stored contents, including unchanged term translations and finite depth padding. Allocation also has complete successor coherence. Dereference now completes successor coherence for all store operations, including arbitrary contextual matches. Its full-machine successful-cost law retains five transitions before the actual copying request, followed by that request's successful continuation. The unchanged translated-data regressions respect this law at every finite padding. Arithmetic now has complete contextual successor coherence as well, proved by source-relative survivor certificates derived from actual rewrites. Its full successful costs are exactly five for unary and seven for binary redexes, at every finite padding. An exhaustive theorem reduces the remaining successor comparisons first to application and copying. Application now also has complete contextual successor coherence, proved through the actual box-opening rule and all context constructors. Copying now closes the final case, through complete source-relative certificates for independently chosen tree and payload matches. The full finite and unbounded machines have proved successor determinism up to recursive representation. Every successful run has a unique cost; constants, instances and lambdas take exactly three transitions from the selected copying input. Contextual copying and allocation now preserve search rootedness. Exact search history and prefix replay reduce full refocusing to the remaining operation families. Specimen locality and the template argument remain open. Application's exact six-step handoff preserves the whole opened body and host; its stable premise, the terms and Figure 16's translation are unchanged. Exact padding reflection now discharges concrete execution-padding invariance

and full term-context congruence. Full-machine laws and the template argument remain pending; Section 53 records the dependency map and proved prerequisites.

Contents

1 The paper and the formal theorem: correspondence and differences **16**
1.1 The statement and its quantitative conclusion . . . 18
1.2 Hypernets, labels, and equality . . . 18
1.3 States and genuinely unbounded execution . . . 19
1.4 Templates, specimens, and robustness . . . 19
1.5 Observations and the context-closure discrepancy . . . 20
1.6 Substitution and the stability discrepancy . . . 20
1.7 Proof organization and the boundary of the claim . . . 21

2 Scope, notation, and the status of the formalization **22**
2.1 Paper-to-Lean proof dependency map . . . 24
2.2 Conventions . . . 26
2.3 Dependency order . . . 26

3 Machines and finite executions **27**

4 Cost relations and state refinement **29**

5 Counting simulation and its soundness **32**

6 Typed observations and contextual lifting **34**

7 The abstract specimen interface **35**

8 The forward robustness argument **38**

9 Opposite specimens and the converse argument **39**

10 State and contextual consequences **41**

11 Semantic examples and regression proofs **43**
11.1 A countdown machine . . . 43
11.2 A crash and an infinite loop . . . 44
11.3 The zero-step converse case . . . 45

12 Finite carriers and concrete interfaced graphs **47**
12.1 Enumerations and bijections . . . 47
12.2 Ports, signatures, and graph invariants . . . 48
12.3 Elementary graph constructors . . . 50

13 Shallow plugging, including self-loop holes **51**
13.1 Carriers and boundary identification . . . 51
13.2 Preservation of incidence invariants . . . 52
13.3 Labels, typing, and the certified result . . . 53

14 Paths, renaming, and isomorphism **54**
14.1 Paths and their behaviour under plugging . . . 54
14.2 Transport along carrier bijections . . . 56
14.3 Graph isomorphism and shallow context congruence . . . 57

15 Concrete uniqueness of operation paths **58**

16 Typed boxes, deep occurrences, and contexts **60**
16.1 Relabelling without changing incidence . . . 60
16.2 The linear wire and label discipline . . . 60
16.3 Box construction and promotion of the depth bound . . . 62
16.4 Finite occurrence addresses at arbitrary depth . . . 62
16.5 Hole lookup and the context condition . . . 63

17 Addressed deep replacement and concrete locality **64**
17.1 Changing one box body . . . 64
17.2 Typed sites and replacement at a site . . . 64

18 Concrete graph examples and regression proofs **65**
18.1 Self-loop gluing and the necessary reflection restriction . . . 65
18.2 Values, boxes, and distinguished occurrence addresses . . . 66
18.3 An input-rooted operation path . . . 67
18.4 Filling a hole inside a certified context . . . 68

19 Occurrence fibres and transport of hole observations **69**
19.1 Coordinates local to an outer edge . . . 69
19.2 Transport along equality and edge maps . . . 70

20 Hole inventories and sharp shallow context closure **71**
20.1 Hole inventories, absence, and isomorphism . . . 71
20.2 Provenance and the abstract collision argument . . . 72
20.3 Concrete shallow provenance and a first context operation . . . 73
20.4 Exact inventory and the sharp freshness condition . . . 73
20.5 Consumption of a selected hole and certified composition . . . 74

21 Provenance and context closure at arbitrary box depth **75**
21.1 Local provenance for a changed label . . . 75
21.2 Lifting a body's provenance through one box . . . 76
21.3 The deep context closure theorem . . . 77

22 Context composition: absence, reuse, and a collision **78**
22.1 Hole-free filling inside a box . . . 78
22.2 Two distinct shallow holes in series . . . 79
22.3 A checked failure without freshness . . . 80

23 Exact inventories and sharp context closure at every depth **80**
23.1 Separating one outer edge from the rest of a graph . . . 80
23.2 A syntactic inventory of retained holes . . . 82
23.3 Necessary and sufficient context closure . . . 83
23.4 Consumption of the selected label . . . 84
23.5 Concrete regressions inside boxes . . . 85

24 Recursive isomorphism and substitution modulo renaming **86**
24.1 Isomorphisms with related labels . . . 86
24.2 Recursive comparison of box contents . . . 87
24.3 Operation traces and paths . . . 89
24.4 Transporting occurrences and holes through all boxes . . . 89
24.5 Congruence of shallow and deep replacement . . . 91
24.6 Graphs and replacement modulo recursive renaming . . . 92
24.7 A strict extension of literal-label isomorphism . . . 93

25 Transporting replacement sites through isomorphisms **94**
25.1 Transport of boundary indices and shallow hosts . . . 94
25.2 Structural correspondence of addresses . . . 96
25.3 Substitution and freshness under transport . . . 97
25.4 A moved address and a wrong-address counterexample . . . 98

26 Associativity, units, and exact context composition **99**
26.1 Shallow associativity with explicit carrier bijections . . . 99
26.2 Composition at arbitrary box depth . . . 100
26.3 Exact inventories and context validity of composites . . . 102
26.4 Corolla units . . . 103
26.5 Restoration and unit equations at deep sites . . . 105
26.6 Regression proofs: self-loops, mixed depth, and repaired collisions . . . 105

27 Concrete focuses and the boundary argument of Lemma 39(1) **107**
27.1 Global focus freedom and a unique shallow focus . . . 107
27.2 Preservation through one filling . . . 108
27.3 Finite fillings and the state space . . . 109
27.4 Nonvacuous checks . . . 110

28 The five search rules on underlying graph coordinates **111**
28.1 Operation kinds and certified incidence views . . . 111
28.2 Local outcomes and the search relation . . . 112
28.3 Determinism and initial/final boundaries . . . 113
28.4 Preservation under context filling . . . 114
28.5 Concrete checks of rule selection . . . 115

29 Explicit focus insertion and realization of typed search positions **116**
29.1 Splitting a vertex . . . 116
29.2 Focus certification, contraction, and holes . . . 118
29.3 Search preserves term typing . . . 119
29.4 Actual states from typed coordinates . . . 119
29.5 Boundary and interior regression checks . . . 121

30 Focus insertion, erasure, and recovery **122**
30.1 Contraction maps and the retained carriers . . . 122
30.2 A valid focus-free underlying graph . . . 124
30.3 Reinsertion recovers an arbitrary focussed graph . . . 124
30.4 Focus-preserving isomorphisms and state recovery . . . 126
30.5 Insertion followed by erasure . . . 127

31 Search and state isomorphism classes **128**
31.1 Search between actual states . . . 128
31.2 Concrete execution and interior recovery . . . 129
31.3 Search coordinates respect renaming . . . 130
31.4 Insertion respects renaming . . . 131
31.5 Exact characterization of search from realized states . . . 131
31.6 Erasure and state search respect renaming . . . 132
31.7 Search on isomorphism classes of states . . . 134
31.8 Concrete quotient-search regression . . . 135
31.9 Recursive renaming of focussed states . . . 135
31.10 Search and insertion under recursive renaming . . . 136
31.11 Erasure under recursive renaming . . . 138
31.12 State search modulo recursive endpoint renaming . . . 139
31.13 Recursive state classes and their search relation . . . 140
31.14 Projection and lifting between the two state quotients . . . 141
31.15 Exact-length executions across the quotients . . . 142
31.16 Recursive-renaming and execution regression proofs . . . 143

32 Rootedness and operation paths **143**
32.1 Canonical roots of concrete states . . . 144
32.2 Concrete rootedness and its search invariant . . . 145
32.3 Operation reachability under coordinate search . . . 146
32.4 Incoming paths and insertion of the focus . . . 147
32.5 Rooted states have actual operation paths: Lemma 23 . . . 148
32.6 The non-operation-path obstruction: Lemma 26(1) . . . 149
32.7 Concrete refocusing and reduction to rewriting . . . 150
32.8 Rooted and non-rooted concrete examples . . . 151

33 One-way hypernets and the single-hole obstruction **151**
33.1 One-way hypernets and the term-boundary restriction . . . 151
33.2 Extracting an operation path from a filling . . . 152
33.3 Reflecting host reachability through a one-way filling . . . 153
33.4 The concrete one-way obstruction: Lemma 26(2) . . . 154
33.5 One-way examples and a filled-state obstruction . . . 155

34 Simultaneous substitution and shallow specimens **156**
34.1 Simultaneous shallow substitution: carriers and incidence . . . 156
34.2 Well-formedness of simultaneous expansion . . . 157
34.3 Retained corollas, identity expansion, and hole freedom . . . 158
34.4 Paired path reflection for many fillings . . . 159
34.5 Actual focussed expansions and their obstruction . . . 160
34.6 Concrete pre-templates and shallow specimens . . . 161
34.7 Lemma 26(3) and the boundary bridge for shallow specimens . . . 162
34.8 A constructed two-hole focussed specimen . . . 163
34.9 Alternating one-way choices: a necessary regression . . . 164

35 Sequential realization of simultaneous substitution **165**
35.1 Peeling one filling . . . 165
35.2 Stopping at arbitrary identity copies . . . 166
35.3 Certified executions and order independence . . . 167
35.4 Existence and a finite length bound . . . 168
35.5 Two-hole examples and scope . . . 169

36 Independent deep substitution and pointed host classes **169**
36.1 Interchange at distinct shallow edges . . . 169
36.2 Changing a box beside another substitution . . . 170
36.3 Structural independence and residual addresses . . . 171
36.4 A quotient of hosts with selected occurrences . . . 173
36.5 Examples that distinguish occurrences . . . 175

37 Recursive deep specimens and their boundary and rooted-path laws **176**
37.1 Copies with an updated box label . . . 176
37.2 Recursive paired filling certificates . . . 177
37.3 Actual deep specimen states . . . 178
37.4 A genuinely deep closed-state example . . . 180

38 Exact search transport and reflection through recursive fillings **182**
38.1 Transporting individual search rules . . . 182
38.2 Exact executions and successful search costs . . . 183
38.3 Realizing executions as actual states . . . 184
38.4 Reading a step back from the expanded graph . . . 185
38.5 Reflection of entire executions . . . 186
38.6 Paired actual executions and rooted endpoints . . . 187
38.7 A deep example and the necessary boundary restriction . . . 187

39 Erasure commutation and traversal of actual deep specimens **188**
39.1 Erasing a copied focus . . . 189
39.2 Commutation of incidences and the join coordinate . . . 190
39.3 Recovering expansion of an already focussed context . . . 191
39.4 Actual-state search and exact executions . . . 192
39.5 Entering, exiting, and remote coordinates . . . 193
39.6 Specialization to recursive deep specimens . . . 194
39.7 Output closure and settled traversal . . . 195
39.8 Traversal and rootedness regressions . . . 196

40 Independent local incidence semantics for search **197**
40.1 Interactions in the original graph . . . 197
40.2 Independent equations for the target graph . . . 198
40.3 Canonical construction and uniqueness . . . 199
40.4 State-level equivalence and regression examples . . . 200

41 Finite interaction patterns and simple-context search **201**
41.1 Subdividing at an output port . . . 201
41.2 Finite fragments and their plugging equations . . . 202
41.3 Reconciling the two subdivision orientations . . . 204
41.4 Factoring the search rules through one corolla . . . 204
41.5 Actual state endpoints . . . 205
41.6 Opening and closing an explicit simple context . . . 207
41.7 The explicit-hole correspondence . . . 209
41.8 Regression examples and scope . . . 210

42 Concrete substitution, compute behaviour, and the full UAM **211**
42.1 Constructing the substitution fragments . . . 211
42.2 Context application and its consequences . . . 212
42.3 Extrinsic behaviour and assembly . . . 214
42.4 Actual executions and non-vacuity . . . 215

43 Original-hole tuples and concrete safety predicates **216**
43.1 Finite tuples and their application . . . 217
43.2 Exact original-hole inventory . . . 218
43.3 The full quasi-specimen predicate . . . 221
43.4 Input-safety and robustness with the paper's clauses . . . 222
43.5 Regression examples and scope . . . 223

44 Set presentations, depth bounds, and a box-label obstruction **224**
44.1 Arbitrary vertex sets and ordered boundaries . . . 224
44.2 Outer round trips and transport of isomorphisms . . . 226
44.3 Recursive annotated boxes . . . 227
44.4 Coherence of depth allowances . . . 229
44.5 The obstruction to naive binder erasure . . . 230

45 Actual graph labels by fresh vertex naming **232**
45.1 Finite sets of actual vertex names . . . 232
45.2 Isomorphism infrastructure for transported boundaries . . . 233
45.3 Boxes labelled solely by actual contents . . . 235
45.4 Encoding, decoding, and recursive round trips . . . 236
45.5 Faithfulness and equivalence of graph classes . . . 238
45.6 Compatibility with larger depth allowances . . . 239
45.7 Relations on the finite union of depth bounds . . . 240
45.8 The repaired counterexample and the remaining theorem . . . 242

46 Search history, stationary rewriting, and the full UAM **243**
46.1 Chronological search visits and the operation stack . . . 244
46.2 Actual states, restart, and coordinate reflection . . . 245
46.3 Stationary replay for simultaneous expansion . . . 247
46.4 Moving an inserted focus through ordinary plugging . . . 249
46.5 Reflecting search through one shallow filling . . . 250
46.6 The stationary criterion for ordinary contexts . . . 251
46.7 Rootedness of the concrete substitution rule . . . 252
46.8 Determinism despite different context decompositions . . . 253
46.9 The full UAM structural theorem . . . 255
46.10 Regression executions and the remaining work . . . 256

47 Relocating specimens and constructing the concrete interface **257**
47.1 Exact hole inventories under focus movement . . . 257
47.2 Insertion commutes with simultaneous filling . . . 259
47.3 The successor specimen and full-machine traversal . . . 260
47.4 The concrete abstract-interface instance and its bridges . . . 262
47.5 Nested-hole regression and remaining obligations . . . 263

48 The full class machine and concrete forward simulation **264**
48.1 Comparing steps from isomorphic sources . . . 264
48.2 The full UAM on recursive state classes . . . 265
48.3 The class-valued specimen interface . . . 266
48.4 Class safety, robustness, and forward sufficiency . . . 267
48.5 Behaviour closure and non-vacuous regressions . . . 269

49 Concrete converse simulation and two-sided refinement **270**

50 Concrete observations and contextual robustness **273**
50.1 Independent recursive substitution . . . 273
50.2 Initial witnesses without a pre-existing specimen . . . 275
50.3 The concrete contextual theorem . . . 276
50.4 Single fragments and a nested regression . . . 277

51 Depth padding and finite executions in the unbounded carrier **279**
51.1 Padding graphs, inventories and state classes . . . 279

51.2 Exact compatibility of concrete search . . . 282
51.3 The union of all finite state spaces . . . 283
51.4 Finite runs and their exact costs . . . 285

52 The full unbounded UAM and contextual sufficiency 288
52.1 Substitution and extrinsic computation . . . 288
52.2 Determinism, rootedness and finite-source locality . . . 292
52.3 Concrete specimen systems and semantic hypotheses . . . 295
52.4 Independent observations and the final theorem . . . 297

53 Copying and store: foundations for Proposition 12.2 300
53.1 Scope, dependency map, and semantic boundary . . . 300
53.2 Scoped terms and the two programs . . . 302
53.3 Substitution algebra . . . 304
53.4 Copying signatures and finite boxed graphs . . . 305
53.5 Recursive isomorphism and addressed replacement . . . 307
53.6 Global predicates and closure . . . 308
53.7 The ordered canonical contraction tree . . . 309
53.8 Regression examples and what they establish . . . 310

54 Ordered distributors and graph assembly 312
54.1 Parallel assembly with an ordered boundary . . . 312
54.2 The finite input permutation and distributor forest . . . 313
54.3 Serial composition through a shared interface . . . 315
54.4 Concrete checks and remaining obligations . . . 316

55 Arbitrary contraction trees and deterministic store tracing 317
55.1 Exact traces and finite termination . . . 317
55.2 Arbitrary contraction trees on concrete graphs . . . 318
55.3 Atom tracing in arbitrary hosts . . . 319
55.4 Regressions: no weakening and a cyclic store . . . 320

56 Literal clauses of the paper's term translation 322
56.1 Fixed target and correspondence requirement . . . 322
56.2 Ordered binary juxtaposition . . . 322
56.3 Interface equalities and environment distributors . . . 323
56.4 The five implemented Figure 16 clauses . . . 323
56.5 Global alphabet and absence of focus and holes . . . 324
56.6 Checks that preserve the paper's graph structure . . . 325

57 Complete recursive translation and the original proposition graphs 327
57.1 The middle-block permutation . . . 327
57.2 Application and binary-operation clauses . . . 328
57.3 Common depth allowance without graph simplification . . . 328
57.4 The full term translation . . . 329
57.5 Permutation, sharing, and full-program regressions . . . 330

58 Stable graphs and copyable payloads 332
58.1 The complete stability predicate . . . 332
58.2 Copyability specialised to the full example alphabet . . . 333
58.3 Distinguishing the predicates and using the original program . . . 334

59 Binding-free paths and the boundary of plugging closure 336
59.1 The literal path restriction . . . 336
59.2 Plugging closure at sourced holes . . . 337
59.3 Term-hole checks and a zero-input counterexample . . . 338

60 Annotated term contexts and concrete closing observations 340
60.1 Exact single-hole closing contexts . . . 340
60.2 The annotated grammar and literal graph interpretation . . . 340
60.3 Termination observations with an explicit execution interface . . . 341

61 Binding-freedom of every translated lambda-context 344
61.1 Adjacency through finite expansion . . . 344
61.2 Depth and ordered parallel assembly . . . 344
61.3 Directed serial assembly . . . 345

61.4 The complete context-translation certificate . . . 346

62 Certified context occurrences and graph-context congruence 348
62.1 Exact occurrence inventory . . . 348
62.2 Preservation by the original assembly operations . . . 348
62.3 Composition of actual addressed contexts . . . 350
62.4 The semantic congruence theorem and its hypothesis . . . 351

63 Actual filling through expansion and unary term contexts 353
63.1 The replacement–expansion isomorphism . . . 353
63.2 Component isomorphisms and serial assembly . . . 355
63.3 The actual unary term-context filling law . . . 356

64 Certified filling under arbitrarily nested binders 358
64.1 Deep replacement through serial assembly . . . 358
64.2 Boxes and the full abstraction clause . . . 359
64.3 Induction over the original nested contexts . . . 360

65 Parallel argument filling and depth padding 363
65.1 Replacement in an ordered parallel family . . . 363
65.2 Deep sites in either parallel component . . . 365
65.3 The unchanged pair, application and binary clauses . . . 366
65.4 Padding the address and the replacement together . . . 367
65.5 Combined regressions and completion boundary . . . 369

66 Unrestricted term-context filling 370
66.1 Exact composition of depth padding . . . 370
66.2 Congruence of the constructor interpretation . . . 371
66.3 The unrestricted addressed filling theorem . . . 372
66.4 Mixed-context regression and semantic boundary . . . 373

67 Common allowances and observation transfer 375
67.1 Exact padding of programs and closing contexts . . . 375
67.2 The execution obligation and observation reflection . . . 376
67.3 A cofinal criterion for the unchanged term equivalence . . . 377
67.4 A negative regression for the missing machine law . . . 377

68 Observational context congruence at equal input depths 379
68.1 Congruence through padded open graph contexts . . . 379
68.2 From actual filling to term-context congruence . . . 380
68.3 Exact context depths and the remaining unequal-depth case . . . 381

69 Unrestricted observational context congruence 382
69.1 One-step padding through the literal graph constructors . . . 382
69.2 Padding to any specified adequate allowance . . . 383
69.3 Structural comparison at a sufficient common allowance . . . 384
69.4 The unrestricted semantic congruence theorem . . . 386
69.5 Original-program regression and completion boundary . . . 387

70 Padding stability and copyable payloads 388
70.1 Stability is independent of unused allowance . . . 388
70.2 Copyable shapes survive padding . . . 388
70.3 Remaining operational boundary . . . 389

71 Actual composition of translated contexts 390
71.1 Certified open graph contexts compose . . . 390
71.2 Composing the unchanged translated hosts . . . 390
71.3 Successive filling of terms . . . 391

72 Resource permutation of literal distributors 392
72.1 Permuting resources while retaining use order . . . 392
72.2 An isomorphism of the actual finite graphs . . . 392
72.3 Separate variable and atom environments . . . 393

73 Typed boundary isomorphisms for renaming 394
73.1 The interface and its elementary laws . . . 394

73.2 An automorphism of the original distributor . . . 395

74 Renaming the literal atomic translations 396
74.1 Coordinates for the selected-resource graph . . . 396
74.2 Renaming the original selected resource . . . 397
74.3 Transport through ordered parallel assembly . . . 398
74.4 Retaining the atom's instance edge . . . 398
74.5 The atomic-case endpoint and remaining work . . . 399

75 Compatible serial renaming and structural safety 401
75.1 Extracting the three carrier actions . . . 401
75.2 The exact gluing condition . . . 402
75.3 Paths, nested binding restrictions, and stability . . . 403
75.4 Recovering the boundary-fixing interface . . . 404
75.5 Concrete distributor compositions and mismatch rejection . . . 404

76 Renaming the complete unary fragment 406
76.1 Decomposing a typed environment permutation . . . 406
76.2 Exact actions through the original distributors . . . 407
76.3 The complete unary constructor theorem . . . 407
76.4 Induction over the original syntax . . . 408

77 Renaming all lambda-free terms and padded binary clauses 410
77.1 Coordinates of the original operand shuffle . . . 410
77.2 Matching the multi-use environment forests . . . 411
77.3 The application and binary constructor proofs . . . 411
77.4 The resource-action induction invariant . . . 412
77.5 Induction on every lambda-free term . . . 413
77.6 Unequal allowances and supplied boxed operands . . . 414
77.7 Concrete instances, rejected mismatch, and remaining scope . . . 415

78 Full translation renaming through box permutation 417
78.1 Typed permutations and the bound-prefix offset . . . 417
78.2 Actual interface permutation and compensation . . . 419
78.3 The concrete BPerm graph pair and its structural safety . . . 420
78.4 The original lambda interface and abstraction shell . . . 421
78.5 Finite structural derivations and their precise scope . . . 422
78.6 Full syntax induction with a resource-action invariant . . . 422
78.7 Invariants, regressions and remaining semantic obligations . . . 423

79 Concrete search for the copying UAM 425
79.1 Incidence rules and determinism . . . 425
79.2 Exact transport across graph representations . . . 426
79.3 Value recognition and wire typing . . . 427
79.4 Padding and shallow context insertion . . . 428
79.5 The phase machine and exact observations . . . 429
79.6 Regressions on the original translated graphs . . . 429

80 Reference allocation with fresh store identity 432
80.1 The literal allocation pair and its stable premise . . . 432
80.2 Fresh allocation and exact store resolution . . . 432
80.3 Renaming and forward padding preservation . . . 433
80.4 Preserved contents and non-aliasing in larger graphs . . . 434
80.5 Safety and the scope of stability . . . 435
80.6 A graph-changing subsystem and the original translated execution . . . 435
80.7 Checking the returned reference and its stored literal . . . 437

81 Dereferencing through arbitrary atom trees 438
81.1 Finite wiring frames with exposed reference inputs . . . 438
81.2 Filling the frames with the actual trees . . . 439
81.3 Preservation of resolved store identity and trace length . . . 440
81.4 The local execution and its pending copy . . . 441
81.5 Carrier renaming, safety, and retained contents . . . 442
81.6 Canonical and noncanonical regression instances . . . 443

82 The literal copying rule and preservation of shared stores 445

82.1 Retaining the matched contraction tree . . . 445
82.2 The two rule graphs and their side condition . . . 446
82.3 Safety throughout nested boxes . . . 447
82.4 Local copy execution . . . 447
82.5 References share their original cell . . . 448
82.6 Renaming and allowance transport . . . 449
82.7 Operational regressions and remaining obligations . . . 450

83 Stable assignment with preserved store identity 451
83.1 Interfaces and finite assignment frames . . . 451
83.2 The stable compute rule . . . 453
83.3 Exact preservation of store resolution . . . 454
83.4 What changes and what remains attached . . . 455
83.5 Search, update and unit return . . . 456
83.6 Renaming, padding and safety . . . 457
83.7 Translated payloads and concrete update regressions . . . 458

84 Atom equality by resolved store identity 459
84.1 The common-store frame . . . 459
84.2 The distinct-store frame . . . 460
84.3 Literal rule graphs and depth allowances . . . 461
84.4 Resolving both references before and after comparison . . . 463
84.5 Store identity determines a unique boolean . . . 464
84.6 The search and equality subsystem . . . 465
84.7 Representation and boundary preservation . . . 466
84.8 Safety and stored-content regressions . . . 467

85 Stable micro-beta application and exact opening costs 469
85.1 Connecting the bound output without changing free outputs . . . 469
85.2 The literal application rule . . . 470
85.3 Boundary and search locality . . . 471
85.4 Recursive renaming and padding . . . 472
85.5 Exact execution costs . . . 473
85.6 Recursive safety and call-by-value restrictions . . . 474
85.7 Unchanged-translation regressions and remaining work . . . 475

86 A combined contextual copying subsystem 476
86.1 Local rules and shallow matching . . . 476
86.2 State isomorphisms and exact costs . . . 477
86.3 Changing graphs inside an active context . . . 478
86.4 An unbounded execution instance and exact representation . . . 479
86.5 Every existing local execution embeds . . . 480
86.6 Term-wire typing throughout execution . . . 481
86.7 Concrete execution regressions . . . 481

87 Arithmetic and exact forward execution padding 483
87.1 Arithmetic meanings and literal local rules . . . 483
87.2 Ordered search and concrete computation . . . 484
87.3 Padding contextual attachments without moving the token . . . 486
87.4 Every combined rewrite and run pads forward . . . 486
87.5 Unbounded programs and padded observations . . . 487
87.6 Arithmetic and representation regressions . . . 488

88 Exact padding reflection and contextual congruence 490
88.1 Typed truncation and the representable fragment . . . 490
88.2 The two round trips and reflection of isomorphism . . . 491
88.3 Assembly, erased slots and search coordinates . . . 492
88.4 Reflection of the operational side conditions . . . 494
88.5 Every local match descends . . . 495
88.6 Contextual matches and exact finite runs . . . 496
88.7 Unbounded executions and observations . . . 497
88.8 Regressions, scope and remaining work . . . 498

89 Canonical rule selection and recursive runtime safety 499
89.1 Occurrences inside arbitrary hosts . . . 499
89.2 Canonical active heads and ordered arguments . . . 500

89.3 Term-sharing resolution . . . 501
89.4 Every actual copy match resolves to its payload . . . 502
89.5 Read and write destinations . . . 502
89.6 Reference-equality branch independence . . . 503
89.7 Arithmetic operand independence . . . 504
89.8 Unique selection for the full contextual relation . . . 504
89.9 Recursive label preservation through computation . . . 506
89.10 Runtime alphabet and reachable-state guarantees . . . 506
89.11 Regression consequences, audit and remaining effort . . . 507

90 Allocation successor coherence and exact costs 509
90.1 States and replacement congruences . . . 509
90.2 Canonical allocation in a source host . . . 510
90.3 Independently chosen contextual matches . . . 511
90.4 Exact costs and the unbounded machine . . . 512
90.5 Regressions on the original translation and arbitrary hosts . . . 513

91 Reference-equality successor coherence and exact costs 515
91.1 A result with two explicit weakenings . . . 515
91.2 The canonical equality successor . . . 516
91.3 The same-store local rule . . . 516
91.4 The distinct-store local rule . . . 517
91.5 Independence of every actual contextual match . . . 518
91.6 Exact costs and arbitrary hosts . . . 519
91.7 Unchanged translated contents, nested boxes and padding . . . 520

92 Assignment successor coherence and exact costs 521
92.1 Target exchange as a typed graph construction . . . 521
92.2 Transport through arbitrary attachments . . . 522
92.3 Unique write data determined by the source . . . 523
92.4 Canonical updates commute with context formation . . . 525
92.5 Explicit comparison with the original local reduct . . . 525
92.6 From local comparison to every actual rewrite . . . 527
92.7 Quantitative consequences and unbounded execution . . . 527
92.8 Unchanged-translation regression scenarios . . . 528

93 Dereference coherence and the complete store interface 530
93.1 A finite graft for the exact distributor . . . 530
93.2 State-level transport of target exchange . . . 531
93.3 Unique source data and the canonical read successor . . . 532
93.4 Commutation with renaming and contexts . . . 533
93.5 The explicit isomorphism to Figure 19(b) . . . 533
93.6 Coherence of every actual contextual read . . . 535
93.7 All store operations under one theorem . . . 536
93.8 Exact cost of the read prefix with arbitrary contents . . . 536
93.9 Unchanged translated data and depth padding . . . 537

94 Arithmetic coherence and the remaining machine cases 538
94.1 Surviving vertices and edges . . . 538
94.2 Renaming the source and successor . . . 539
94.3 Deleting an arithmetic redex inside a plugged graph . . . 540
94.4 Simultaneous expansion and dependent component carriers . . . 541
94.5 Certificates for the original numeric rules . . . 542
94.6 Recognized arithmetic requests and actual contextual rewrites . . . 542
94.7 An exhaustive description of the remaining coherence cases . . . 544
94.8 Exact costs, arbitrary hosts and regressions . . . 545

95 Application coherence and the remaining copying case 547
95.1 Presenting the complete result . . . 547
95.2 The source frame and its actual body . . . 548
95.3 The original beta reduct realizes the presentation . . . 549
95.4 Occurrences and changes of presentation . . . 550
95.5 Complete certificates and independent bodies . . . 551
95.6 Transport through arbitrary plugging . . . 552
95.7 Transport through simultaneous expansion . . . 553
95.8 Derivation from the unchanged transition relation . . . 554

95.9 The historical reduction to the copying case . . . 555
95.10 Exact application costs and host continuations . . . 556
95.11 Regressions with unchanged translated bodies . . . 556

96 Copying coherence and complete successor determinism 558
96.1 Normal presentations of copyable payloads . . . 558
96.2 Rigidity of two payload occurrences . . . 560
96.3 The uniquely determined source data . . . 560
96.4 A presentation retaining the complete source graph . . . 561
96.5 Carrier bijections for the literal reduct . . . 562
96.6 Ports and ordered incidences of the literal construction . . . 563
96.7 The complete local certificate . . . 565
96.8 Transport through occurrences and recursive renaming . . . 566
96.9 Transport through a single plugged context . . . 567
96.10 Comparison of independently described payloads . . . 568
96.11 Transport through simultaneous expansion . . . 568
96.12 From actual rewrites to complete machine determinism . . . 570
96.13 Exact costs of copying and of arbitrary finite prefixes . . . 571
96.14 Semantic regression scenarios and the remaining boundary . . . 572

97 Refocusing for copying and allocation 573
97.1 Rootedness and operation paths . . . 574
97.2 The retained payload is inaccessible to search . . . 575
97.3 Retained occurrences inside the actual reduct . . . 576
97.4 Exact replay of the copying prefix . . . 576
97.5 Discharging the intrinsic machine cases . . . 577
97.6 Exact search history, including backtracking . . . 579
97.7 Single operation replacement and allocation . . . 580
97.8 Representation laws and semantic regressions . . . 581

98 Proof checking 583
98.1 The axiom audits . . . 583
98.2 Reproducing the Lean and document checks . . . 586
98.3 Copying-foundation audit . . . 588

A Technical terms 595

B What remains for the paper's concrete theorem 633
B.1 Remaining work for Proposition 12.2 . . . 634

1 The paper and the formal theorem: correspondence and differences

The principal result formalized here is the general sufficiency-of-robustness argument. Its reference in the version 5 PDF of the paper is *Theorem 8.11*. The HTML rendering numbers the same result Theorem 6. The Lean name `Unbounded.theorem6` and many references in this commentary follow the HTML numbering. They do not identify a different mathematical result. This section uses the PDF numbering when discussing the original statement.

The Lean theorem proves the two corresponding contextual-refinement implications for the definitions constructed in this repository. It is not a literal transcription of the paper. Some changes are representation choices with proved comparison lemmas; others require a further audit of the correspondence with the paper. In particular, the missing plugging-closure premise and the treatment of substitution stability, discussed below, should not be silently described as harmless changes or established strengthenings. A kernel-checked proof establishes the formal proposition; it does not by itself establish that the proposition expresses every condition intended in the paper.

The later copying development now proves full-syntax bijective translation renaming by explicit BPerm derivations (Section 78). Those derivations are syntactic proof trees; their observational interpretation still requires machine and robustness laws. They do not identify box permutation with ordered isomorphism. The same section documents an indexing correction to PDF Definition 12.7(2): the body position of a free output includes the bound-prefix offset. Omitting it is inconsistent already with one bound and one free variable, as a checked counterexample shows. The Proposition 12.2 terms and Figure 16 translation are unchanged.

The copying extension also has a proved intrinsic search phase (Section 79), represented by a token mode and a wire of a fixed graph. Its determinism, term-wire typing, stable-primitive recognition, exact run transport under isomorphism and padding, and shallow plugging preservation are established. A rewrite token suspends this phase; it is not successful completion. Reference allocation now adds actual graph-changing computation with a stable argument, fresh store identity and a five-transition execution of the unchanged `ref` n translation (Section 80). This is a search/allocation subsystem, with forward padding preservation for allocation pairs. Dereferencing now has the general Figure 19(b) local rule and an execution ending in a genuine copy request, with other reference inputs and store identity preserved (Section 81). It does not directly return or replace the stored value. Figure 17 copying now has a literal graph construction, local return executions and preserved shared-store resolution, including unchanged lambda box bodies and proved representation laws (Section 82). Stable assignment now has the literal Figure 19(c) update, with preserved store identity and weakened old contents, checked local executions and representation laws (Section 83). Both atom-equality rules now have local executions and a lookup-outcome uniqueness theorem preserved by context attachment (Section 84). This compares actual store identities, not their contents. Stable micro-beta application now opens the actual lambda box, preserves free interfaces and supplies exact opening and body-search costs (Section 85). A combined subsystem now lifts all these local runs through shallow typed contexts, retains exact costs and proves term-token typing. Its concrete allowance-indexed execution has proved isomorphism invariance and graph-context congruence (Section 86). Arithmetic rules and forward exact-cost padding of all combined runs are proved in Section 87. Exact reflection of arbitrary padded matches and runs now proves the converse, discharging concrete execution-padding invariance and full term-context congruence (Section 88). Canonical source selection is now proved for every contextual rewrite, including ordered numeric operands, reference-equality branches, copy payload heads and read/write stores. Recursive runtime safety holds throughout every finite run (Section 89).

Allocation successor coherence is now proved for independently chosen contextual matches: complete successor graphs and resumed token positions agree up to recursive isomorphism. The original translation of `ref` n has exactly the successful cost five, including under finite padding (Section 90). Both reference-equality branches now also have complete successor-state coherence for arbitrary contextual matches. Comparison has exactly seven successful steps with arbitrary attached contents, including unchanged translated terms and every finite padding (Section 91). Assignment now also has complete contextual successor coherence, proved by typed target exchange and comparison with the original local reduct. Stable writes with arbitrary attached old contents have exactly seven successful steps, with translated-data regressions at every finite padding (Section 92). Dereference now also has complete contextual successor coherence, closing this obligation for all store-operation heads. Its exact cost law retains the five-step read prefix and the subsequent copying continuation, including unchanged translated data at every finite padding (Section 93). Arithmetic now has complete contextual successor coherence, exact five- and seven-step successful costs, and an exact seven-step handoff to an arbitrary host continuation (Section 94). Application has complete contextual successor coherence, including independently represented bodies and arbitrary contexts, with an exact six-step handoff (Section 95). Copying now completes the final successor comparison, including independent tree boundaries, complete recursive payloads, original distributors and retained hosts. The full finite machine has successor determinism up to recursive state isomorphism, and the unbounded machine up to its representation relation. Exact prefix costs and unique successful-run lengths are proved without extra machine-law premises (Section 96).

Section 97 now proves refocusing for search, every actual contextual copying rewrite, and every actual allocation. Copying replays the original search prefix with exactly its final trigger removed. Exact search history, including backtracking, also proves refocusing for replacement of a single operation head. Rootedness is preserved and reflected by recursive representation and finite padding. Full finite and unbounded refocusing is reduced equivalently to dereference, assignment, equality, application and arithmetic; those cases, focused-state correspondence and the specimen interface remain unfinished.

The earlier estimate of 80% completion of Proposition 12.2 is withdrawn. The current cautious assessment is 55–70% of mathematical effort, with low confidence. The template argument and remaining machine laws are substantial proof obligations; declaration counts do not measure their difficulty.

The new recognition predicate records proved facts about the existing source graph. It is not a new reduction rule or a replacement for stability and copyability premises. The runtime alphabet adds stores to the translation alphabet, since allocation introduces stores; preserving the smaller alphabet would be false. Neither hidden focus edges nor holes are introduced, even inside copied or opened boxes.

The new auxiliary truncation is a structural proof device. A recursive representability predicate characterizes exactly the isomorphism-closed image of padding. Truncation recovers that image without changing the graph's meaning; its total definition uses typed placeholders only for excessive boxes outside the certified fragment, including erased context slots. It is not a reduction rule or a replacement for the Figure 16 translation.

The arithmetic domain remains natural numbers, with subtraction truncated at zero. The author's clarification $-x = 0 - x$ therefore gives zero for every unary minus. This convention is explicit in the new rule; the original constant datatype, proposition terms and translation are unchanged.

1.1 The statement and its quantitative conclusion

Write $T(G, H)$ for the pre-template relation and $G \preceq^{\mathcal{C}}_{Q} H$ for contextual refinement with cost comparison Q. The forward conclusion has the form

$$T(G, H) \quad \Longrightarrow \quad G \preceq^{\mathcal{C}}_{Q} H,$$

under input safety and robustness. The second implication reverses G, H; it uses input safety with the converse cost relation and the corresponding converse robustness condition. Both clauses share output closure, deterministic and refocusing behaviour, and a reasonable triple of cost preorders. In the final Lean declaration the paper's (Q, Q', Q'') are named (Q, P, R). These are parameters, not specific numerical inequalities.

Lean: `Unbounded.theorem6`

State refinement compares successful termination and exact numbers of machine steps: a terminating execution on the source must have a terminating matching execution on the target whose lengths satisfy the chosen preorder. The conclusion does not assert equality of final graphs or values. Contextual refinement makes this comparison after observation contexts are filled. Consequently the meaning of the conclusion depends on the definitions of admissible observations, as well as on the transition relation.

1.2 Hypernets, labels, and equality

The paper presents finite typed hypergraphs with interfaces and recursively boxed graphs. Lean separates internal vertices from indexed input and output ports, makes the ordered interface types explicit, and carries finiteness and incidence laws in the graph structure. This replaces informal choices of names and boundary identifications with functions, finite indices, and proof fields. Plugging therefore requires explicit interface transport, even when its graphical meaning is immediate.

A finite allowance for nesting makes the recursive construction of boxed labels well founded. The allowance is not the intrinsic depth of the graph: the same graph can be represented at larger allowances. Global graph classes identify recursive graph isomorphism and such padding. Recursive isomorphism also permits independent renaming inside boxes. The relevant equality is thus structural equality up to these identifications, rather than literal equality of the chosen vertex and edge carriers.

There is a further, substantive label issue. The internal typed representation records box-boundary information that cannot simply be erased: the paper requires equal labels to have equal signatures. The development gives a freshening encoding into actual graph-valued labels, with encoding/decoding round trips on recursive graph classes. These are proved representation results; the earlier counterexample to naive erasure explains why a direct forgetful translation would not have sufficed. See `PaperEncoding.lean` and `PaperRelations.lean`.

1.3 States and genuinely unbounded execution

A state is a suitably focussed closed graph. Focus insertion and erasure let the development reason using a focus-free graph and a distinguished coordinate, then reconstruct the actual focussed graph. Search is first expressed in these coordinates. Separate adequacy proofs compare it in both directions with adjacency equations and with the finite interaction fragments in simple hole contexts. Coordinates are a proof device, not an additional machine operation. The word *corolla* denotes retained-edge bookkeeping used in this construction; it introduces no additional semantic rule from the paper. See `HoleContextSearch.lean`.

The earlier fixed-allowance machines were intermediate results. The final machine uses the union of all finite allowances, quotiented by renaming and padding. A finite run can be represented at a sufficiently large common allowance with exactly the same length. No single allowance bounds all runs, and computation may increase the necessary allowance. Finality and the full transition relation are transferred, not just search. See `UnboundedUAM.lean`.

Rootedness means reachability by search from an initial state; it does not mean arbitrary reachability by the full machine. The global rooted predicate is proved equivalent to this search-reachability formulation. Refocusing of extrinsic behaviour means preservation of rootedness by its compute steps; the full-machine preservation theorem combines this assumption with the intrinsic search and substitution results. See `UnboundedRoots.lean`.

The parameter `Behaviour` supplies the extrinsic compute relation. Determinism and refocusing of that behaviour remain assumptions of the general theorem, as do the semantic conditions on a proposed template. The development proves the required structural results for the assembled machine. Equality of successors in its determinism theorem is equality of state classes, expressing determinism modulo the chosen structural identifications.

1.4 Templates, specimens, and robustness

A global pre-template is a relation between focus-free graph classes with matching interfaces. Its interpretation at every finite allowance gives the layered pre-template used by the concrete constructions. What the paper groups as template conditions is supplied through separate Lean premises: in particular, output closure and input safety are not consequences of an arbitrary pre-template relation.

A specimen consists of an actual context together with two related fillings. The two resulting states share the distinguished context focus. The formal representation uses named, typed slots and recursively fills the original holes. Inserted fragments are not recursively substituted again during the same filling operation. Although the slot environment can declare unused slots, a finite graph has only finitely many actual hole occurrences. Tuple representation lemmas connect these fillings with finite families of original holes; the slot environment itself should not be confused with an infinite graph or an infinite simultaneous rewrite. See `ObservationFillings.lean`.

Quasi-specimens retain an unconditional specimen witness and agreement of focus modes. Only the rootedness and comparison obligations are conditional on the two endpoint states being rooted. Input safety retains its three alternatives. Robustness likewise allows an unrooted case, a stuck continuation, or a matching continuation through a non-rewrite quasi-specimen, with the specified costs and up-to comparisons. In the final definitions, stuckness means stuckness in the unbounded machine. It is not inferred merely from being stuck at one finite allowance.

Lean: `Safety.Quasi, InputSafe, Robust`

1.5 Observations and the context-closure discrepancy

The formal predicate `PaperRefines` quantifies over every pair of finite representatives of the two global graphs, every hole name, and the observation contexts specified independently of the template. The graph interfaces must match, and the observation construction imposes its own well-formedness and closure requirements. A proved common-representatives lemma ensures that the universal quantification over representatives is not vacuous. This lemma does not, on its own, prove that every observation intended by the paper is covered.

Lean: `PaperRefines`

Lean: `common_representatives`

There is an explicit difference in hypotheses. The paper's contextual setup in Section 7.1, and Theorem 8.11 itself, require the context set $\mathcal{C}$ to be closed under plugging: whenever the relevant composite of allowed contexts is defined, it is again allowed. The final Lean theorem instead takes an arbitrary predicate C on global context classes. It induces membership at each finite allowance by interpreting the finite graph in the global class. The resulting membership is invariant under graph isomorphism and padding.

Lean: `globalAllowed, globalAllowed_iso, globalAllowed_padding`

Invariance under renaming or padding is not closure under plugging. The latter is not a premise of `theorem6`. Its absence does not invalidate the proved Lean implication, but it prevents treating the two statements as identical without further analysis. A stronger theorem for arbitrary context sets would imply the instance for plugging-closed sets *if* the observation predicates agree. The remaining audit is to establish that agreement and explain exactly where the paper's closure condition is used, or to adjust the formal setup if its observation quantification differs. This commentary does not claim that this audit has already been completed.

1.6 Substitution and the stability discrepancy

The paper's Lemma 7.6 uses stability of the substituted graph in its argument reducing full-machine refocusing to the extrinsic behaviour. The Lean substitution-rootedness theorem has no explicit stability premise for that graph. Its proof instead verifies a stationary-plugging criterion using the actual variable-prefix geometry: the variable prefix blocks an operation path from the sole input, and the condition concerning other inputs is vacuous for a unary interface.

Lean: `Substitution.prefix_oneWay`

Lean: `State.substitution_rooted`

This is a proved result about the encoded substitution rule. It is not yet a justification for dropping the paper's stability condition. The correspondence audit must compare the admissible instances of Figure 9(f), the definition of rootedness, and the stationary-plugging criterion with the paper's argument. Only after that comparison can the difference be classified as a valid strengthening, a condition supplied elsewhere, or a modelling discrepancy. The final theorem's use of the formal refocusing result makes this comparison material to its relation with Theorem 8.11.

1.7 Proof organization and the boundary of the claim

The proof retains the paper's main route: local specimen analysis yields a counting simulation; soundness of that simulation yields state refinement; initial observation lifting yields contextual refinement. Lean factors the first part through abstract machine and specimen-system interfaces, then constructs concrete instances and proves their laws. The abstraction does not leave those concrete laws as unproved axioms. Additional transport, quotient, freshness, and common-allowance lemmas account for much of the formal proof's length; they make representation choices explicit rather than replace the central simulation argument.

The kernel checks the final propositions and their dependencies. The recorded axiom audits exclude project-specific axioms and unfinished proofs; use of Lean's standard classical and quotient principles is part of the foundation. None of these checks automatically validates the translation of informal mathematics into definitions.

Accordingly, the completed milestone is the unbounded contextual sufficiency theorem *in the repository's concrete encoding*. The two correspondence questions above remain distinct from missing Lean proof terms and are recorded in Appendix B. The copying UAM, translations of specific languages, and application-specific equivalences were outside that general-theorem milestone. The new target is Proposition 12.2: Section 53 begins its separate copying/store development and states what remains. Sections 54–58 construct the distributors, tree/store tracing, full Figure 16 translation, and stable/copyable graph predicates, preserving the proposition's terms and all prescribed distributors. These structural results do not yet supply the copying machine or the observational equivalence of Proposition 12.2. Section 59 proves sourced-hole binding-free plugging closure and exhibits a zero-input-hole counterexample to unrestricted closure under the literal Definition 11.2. Translated term holes always have an input; this qualification does not change the proposition's terms or their translation. Section 60 adds annotated term contexts and a concrete closing-context observation interface parameterised by execution. Exact constructor-interpretation laws are proved. Section 61 proves binding-freedom of all translated lambda-contexts. Section 62 adds actual sole-hole addresses, composition of graph contexts, and graph-context congruence conditional on execution invariance under recursive isomorphism. Section 63 proves actual addressed graph-filling correspondence for unary term contexts. Section 64 extends this to arbitrarily nested lambda/unary contexts, with sole-hole and binding-free certificates. Section 65 proves filling through both parallel arguments, the full application/binary clauses, and arbitrary iterated padding. Section 66 proves unrestricted addressed term-context correspondence, including cumulative padding at common-depth joins and a single certified address valid for all fitting graphs. Section 67 proves observation reflection from larger allowances and the cofinal comparison criterion, with explicit execution-isomorphism and padding hypotheses. A deterministic counterexample shows that the latter is an independent obligation. Section 68 derives observational congruence for all original contexts when the input terms have equal minimal depths, assuming execution isomorphism invariance. The proposition's actual depths are three and two. Section 69 removes the depth restriction and proves general observational context congruence under the explicit execution isomorphism and padding laws. Section 86 now discharges isomorphism invariance for the concrete subsystem; Section 87 now proves forward execution padding as well. Section 88 discharges reverse padding transport. The remaining full-machine laws are listed in Appendix B.1. No equivalence of the proposition programs is inferred. Earlier statements that no work remained for the paper's theorem should be read with this qualification: completion of the formal construction does not settle these external correspondence questions.

Section 70 proves that the actual stability predicate is preserved and reflected by padding, and copyability is preserved. These structural laws do not discharge the copying execution's representation laws; they retain the distinction between stable graphs and copyable payloads.

Section 71 establishes actual graph composition of independently translated contexts. The original hosts, interfaces, term constructors and translations are retained; the correspondence is recursive graph isomorphism at equal outer depth, without an execution premise.

Section 74 proves selected-resource renaming and all atomic translation cases for finite resource permutations, retaining the original weakening/contraction forests and instance prefix. These are actual boundary isomorphisms of the original translations. They do not establish composite-term renaming or the final observational equivalence.

Section 75 proves compatible serial boundary renaming and invariance of stability and recursive binding-freedom. Matching resource permutations is necessary and sufficient for the constructed gluing map, and is proved for the original distributor compositions. Fixed external boundary actions recover the existing recursive isomorphism. These structural results do not establish machine-execution invariance or whole-term renaming.

Section 76 completes bijective renaming for every finite chain of the original unary operations over atomic terms, and proves constructor congruence for arbitrary operands with a typed boundary isomorphism. The original single-use graphs are retained. Allocation and dereference subterms are covered structurally. Their subsequently proved local operational rules are described in Sections 80 and 81; full-machine integration and the proposition remain open.

Section 77 completes bijective translation renaming for all lambda-free original terms and distinct-list scopes. Application and all binary constructors retain their original shuffle and two-use forests; constructor congruence also handles unequal padded allowances. Section 78 subsequently establishes the lambda-box correspondence and the full syntax induction using explicit BPerm derivations. Full-machine execution laws and the final equivalence remain open. The structural representation theorems alone do not establish operational correctness of allocation, dereferencing, assignment, or callback execution.

2 Scope, notation, and the status of the formalization

The abstract part of the development follows the proof in Appendix F of [1]. Its completed mathematical core is the soundness of counting simulation, corresponding to Proposition 37. The remaining argument is expressed through a structure of abstract specimens. Given that structure and its laws, Lean proves both directions of the simulation construction and their contextual consequences. Constructing this structure from the paper's actual hypernets is still a separate task. The concrete graph part now supplies finite interfaced graphs, certified shallow plugging, renaming and path invariance, a typed boxed hierarchy, and addressed deep replacement. These constructions and their regression examples are explained after the abstract development. The context calculus now also proves exact hole inventories, necessary-and-sufficient freshness, and exact consumption of selected hole labels at arbitrary box depth. Recursive isomorphism now identifies independently renamed box contents and respects paths, hole observations, and deep replacement. Sites now transport across isomorphic hosts, and substitution respects simultaneous host and replacement renaming, including the exact freshness condition. Nested substitutions now associate at arbitrary box depth, with corolla unit laws and a sharp final context-validity criterion even when an intermediate collision is later removed.

The concrete development now also defines globally focus-free hypernets, focussed hypernets, and the hole-free state space. The boundary argument of Lemma 39(1) is proved for finite

sequences of addressed focus-free fillings; see Section 27. The five search rules are now defined on underlying graph coordinates, with determinism, boundary laws, and preservation under shallow filling proved (Section 28). Explicit focus insertion now realizes typed coordinates as actual states, preserving holes and boundary observations, and contracting each inserted focus recovers original incidences (Section 29). General erasure and the focus-preserving erasure-then-insertion isomorphism are now proved for arbitrary closed focussed states. Erasure-based search on actual states is deterministic up to focussed isomorphism and has the expected search boundary laws (Section 31). Both focus round trips and source/target renaming invariance are proved; search and boundary observations descend to literal-label state isomorphism classes, with equality-based search determinism. Search also respects recursive renaming inside boxes and descends to recursive state classes. The map from literal to recursive classes preserves and lifts search executions with exact lengths, including successful search termination. Canonical roots and concrete rootedness are now defined; search preserves rootedness, and concrete rooted focus sources have operation paths from the unique input. An incoming path containing a non-operation edge rules out rootedness (Section 32.6). Concrete refocusing is defined and proved for search; Section 46 now proves the complete UAM structural theorem. One-way hypernets and the single-hole obstruction of Lemma 26(2) are now proved, using extraction of actual operation paths through shallow substitution (Section 33.4). Simultaneous shallow expansion now has proved graph invariants. Concrete pre-templates, output-closure, and shallow specimens provide actual filled states and a multi-hole proof of Lemma 26(3) for this representation (Section 34.6). Initial/final agreement now applies directly to these specimen states. Every finite simultaneous shallow filling now has a finite ordinary addressed-substitution realization, and completed schedules give isomorphic results (Section 35). Independent deep substitutions now have constructed residual sites and a recursive interchange isomorphism. A quotient of hosts with selected sites supports substitution and sharp context validity (Section 36). Recursive deep specimens now supply actual filled states, with derived focus freedom and the Lemma 26(3), boundary, and jointly rooted context-path conclusions (Section 37). Recursive fillings now preserve context search with exact execution lengths. Search at host coordinates reflects to context search away from shallow hole boundaries; invariant regions support reflection of entire runs. These coordinate executions also realize paired executions and rooted endpoints on actual state classes (Section 38). General erasure now commutes with expansion, identifying these realizations with the original deep specimen states. Concrete entering/exiting predicates agree with the coordinate boundary test, output closure and joint rootedness imply non-exiting, and settled specimens have synchronized actual search transitions to rooted endpoints at a remote coordinate (Section 39). Section 47 now constructs the named-state interface and safety/robustness bridges. Section 43 proves compatibility with finite original-hole tuples and defines concrete quasi-specimens, input-safety, and robustness. Contextual search adequacy is proved below. Concrete substitution and compute transitions now assemble into the full UAM, with mode discipline, disjoint transition families, and final normality proved (Section 42).

2.1 Paper-to-Lean proof dependency map

The target is the proof of Theorem 6 through Propositions 40, 37, and 38. The following map separates checked arguments from their missing concrete interpretations, rather than treating graph infrastructure as progress on every part of that proof.

1. **Proposition 37: checked.**

   `Machine.CountingSimulation.sound` proves counting-simulation soundness for a deterministic machine and a reasonable cost triple. Its proof follows the quantitative argument, including the eventual-stuck alternative.

2. **Proposition 40: concrete forward simulation checked.**

   `SpecimenSystem.countingSimulation` and its converse construct counting simulations from the abstract specimen laws, safety, and robustness. The abstract laws have concrete named-state and class-valued instances. Section 48 proves literal full-class determinism and the concrete forward counting simulation using class safety and robustness. Section 49 completes the concrete converse counting simulation and two-sided refinement. Section 50 completes those observations and both contextual clauses for the fixed-depth machine. Section 52 completes full-UAM transfer and both clauses on the unbounded carrier.

3. **Lemma 39(1), Proof F.4: boundary argument checked.**

   `Focussed.fillings_boundary_agreement` proves initial/final agreement for two finite filling sequences of one focussed hypernet. The proof preserves the focus mode and input status of its source, as in the paper. Finite original-hole tuples and the recursive representation are now related in both directions in Section 43. Both the shallow and recursive deep specimen representations now have direct proofs of this boundary conclusion for their actual filled states.

4. **Lemma 39(2): checked for recursive specimens; point (3): full-UAM locality checked.**

   Concrete rootedness and output closure yield the non-operation-path obstruction of Lemma 26(3) for actual recursive specimen states. An exiting focus would give a singleton hole path to the focus source, so joint rootedness rules it out, proving the concrete conclusion of point (2).

   General erasure commutes with simultaneous expansion, preserving the joined coordinate. Consequently expansion of an already focussed context is focus-preservingly isomorphic to filling its erasure and then inserting a focus. Entering/exiting adjacency in the original context is equivalent to the checked coordinate boundary test. Under the settled hypotheses of point (3), search enabled on either specimen side yields synchronized actual-state steps to rooted endpoints at a remote coordinate. The proof allows recursive endpoint renaming and uses output closure to exclude exiting backtracking focuses.

   Section 47 now packages that coordinate as an allowed unique-hole specimen with recursive fillings and closed endpoints. Its full-UAM context-step theorem constructs the abstract locality field. Original-hole tuple compatibility is proved in Section 43; contextual search adequacy is established in Section 41.

5. **Paper §6.4: concrete transition definition checked.**

   `State.Substitution` applies the explicit Fig. 9(f) fragments in a shallow simple context. `State.Compute` specifies the required active operation and focus modes; `Behaviour` restricts the extrinsic parameter to this shape. `UAM.machine` assembles all three families, with proved

final normality and transition-family exclusion. This removes the transition definition obligation.

6. **Lemma 24, Lemmas 27–28, and Lemma 3: structural theorem checked.**

   Section 46 proves chronological search revisiting, stationary prefix replay, substitution rootedness and uniqueness, and full-UAM determinism and refocusing exactly under the corresponding extrinsic behaviour conditions. Full executions preserve rootedness.

7. **Paper §§8.1–8.3: tuple compatibility and definitions checked.**

   `DeepSpecimen.substitutionTuple` constructs distinct, complete finite original-hole tuples with related closed entries. `TupleFilling.specimen` reconstructs a recursive specimen with the same filled graphs. Concrete quasi-specimens retain unconditional witness existence and mode equality; input-safety is unrestricted by rootedness, and robustness retains its unrooted alternative. The rooted quasi and robustness consequences are proved. Section 47 proves the safety and robustness bridges for the constructed named-state `SpecimenSystem`.

8. **Proposition 38: logical lifting checked; initial bridge open.**

   `Observations.contextual_of_lifting` proves the logical implication once initial pairs belong to the lifting. Concrete initial focus attachment is now constructed, including initial rooted specimens obtained by relocation. The observation interface and its arbitrary-fragment witnesses must still discharge the `initial_lift` parameter.

9. **Theorem 6: conditional, not complete.**

   The two `contextual_sufficiency` theorems combine these abstract pieces. The paper itself assumes determinism and refocusing of the UAM. These remain the theorem's hypotheses. Section 46 now defines their concrete meaning and proves the full-UAM structural theorem, reducing both properties exactly to the corresponding extrinsic behaviour conditions.

The typed finite-depth hierarchy and explicit renaming bijections are representation choices. Outer set-presentation normalization, recursive annotated round trips, and compatibility across depth bounds are now checked in Section 44. Section 45 resolves the remaining box-label correspondence by fresh naming of actual contents, with inverse class maps and exact depth coherence. The checked counterexample to naive erasure is repaired without restricting the typed graph class. The existing plugging, path, inventory, and isomorphism lemmas support that representation; they do not replace the operational proofs in Appendix F. The contextual-search dependency is now discharged: Section 41 identifies erasure-based state search with finite interaction patterns plugged into explicit simple contexts, in both directions. Opening and closing the selected slot covers arbitrary simple contexts; output fragments and self-looping attachments are included. The transitions of §6.4 are now assembled into a concrete machine in Section 42. Section 46 proves substitution rootedness and uniqueness, stationary replay, and the full-UAM structural laws. Section 47 closes the successor packaging, full-UAM locality, and concrete safety/robustness bridges. The operational connection to the union of all finite depths is proved in Section 52. Section 48 constructs the deterministic full class machine and its concrete specimen system, class safety/robustness bridges, forward counting simulation, and remote-specimen refinement.

Section 49 supplies the concrete converse quasi and robustness bridges, reverse counting simulation, two-sided refinement, and equivalence for constructed initial specimens.

Section 50 constructs template-independent observations, proves initial lifting, and instantiates both contextual sufficiency clauses, including arbitrary single fragments and nested holes. The general-theorem milestone is complete in the concrete encoding. Section 51 constructs the union and exact search-run adequacy. Section 52 proves substitution and full-UAM

compatibility, exact executions and finality, determinism and refocusing, concrete specimen locality and the semantic bridges, independent initial lifting, and both contextual clauses for global pre-templates. Its final `theorem6` uses a depth-independent set of contexts. The earlier 94% planning estimate is superseded: all selected general-theorem proof obligations are discharged, while language-specific applications and the copying UAM are separate work. Appendix B records that the remaining paper-correspondence audit questions, distinct from the completed Lean proof obligations.

There are three distinct kinds of assertion below. A *definition* specifies a mathematical object or predicate. A *proved statement* is established by a Lean proof term. An *interface law* is a hypothesis carried by a structure; the current code uses that hypothesis but does not prove it for concrete hypernets. The concrete instances proved later in this document discharge those interface laws for the fixed-depth universal abstract machine.

All code links point to revision `585baee`, rather than to the moving `main` branch. Each link highlights the declaration together with its proof or defining fields. Namespace prefixes in link labels disambiguate declarations with the same short name. Anonymous Lean `example` declarations are explained alongside the named theorems.

2.2 Conventions

Fix a set of states X and a machine M on X. Write $s \to t$ for its one-step transition relation, $s \longrightarrow^n t$ for exactly n transitions, and $\mathsf{Final}(s)$ for successful finality. Natural numbers include zero. A cost preorder Q is written infix: $n\,Q\,m$. The source count is always the left argument and the target count the right argument. Write $s \preceq_Q t$ for state refinement, and $s \simeq_Q t$ for mutual state refinement.

The paper's three cost parameters (Q, Q', Q'') are named (Q, P, S) in the code and in this commentary. Thus P is the auxiliary relation on the *left* of a simulation candidate, and S is the one on its *right*. We spell out witnesses rather than relying on a convention for composition of relations.

No result assumes that all executions terminate. An arbitrary state can be final, stuck, or capable of further computation. Successful termination is the only qualitative observation in the present state-refinement definition. Neither equality of returned values nor equality of final states across two different source states is required.

2.3 Dependency order

The files are organized as follows. Finite-execution facts in `Transition.lean` support the quantitative relations in `Quantitative.lean` and the soundness argument in `Simulation.lean`. The observation interface in `Contextual.lean` converts state results into contextual ones. The specimen argument in `LocalReasoning.lean` constructs the simulation to which soundness applies. Finally, `Examples.lean` checks substantive edge cases, and `Audit.lean` reports theorem dependencies. The root file `Hypernets.lean` imports the development and its examples; it adds no mathematical declaration.

The concrete branch begins with `Graph/Finite.lean` and `Graph/Basic.lean`. Plugging, paths, and isomorphism support the hierarchy and deep replacement modules. Operation-path uniqueness imports the counted-execution results, providing a concrete use of the abstract machine lemmas. Occurrence fibres and provenance then support shallow and deep context closure in the new context modules. Both branches are imported by the root file.

The exposition follows these dependency orders. Section numbers and theorem numbers here

are local to this document; references to the paper are explicitly identified as such.

3 Machines and finite executions

Definition 3.1 (Machine)**.** A machine on X consists of a relation $\to \subseteq X \times X$, a predicate $\mathsf{Final} \subseteq X$, and the law

$$\mathsf{Final}(s) \implies \neg(s \to t) \qquad (s, t \in X).$$

Thus every successful final state has no outgoing transition. No converse is assumed: an unsuccessful normal form is permitted.

Lean: `Machine`

The fields `step` and `final` supply the relation and predicate. The field `final_normal` supplies the displayed law. A machine may have any state type; finiteness or a graphical representation is not required.

Definition 3.2 (Determinism, normality, and stuckness)**.** The machine is *deterministic* if $s \to t_1$ and $s \to t_2$ imply $t_1 = t_2$. A state s is *normal* if $s \to t$ is false for every t. It is *stuck* if it is normal and is not final:

$$\mathsf{Normal}(s) \iff \forall t\, \neg(s \to t), \qquad \mathsf{Stuck}(s) \iff \mathsf{Normal}(s) \wedge \neg\mathsf{Final}(s).$$

Lean: `Machine.Deterministic`

Lean: `Machine.Normal`

Lean: `Machine.Stuck`

Determinism is a separate hypothesis, not a field of every machine. Stuckness distinguishes failure from successful finality even though both are normal forms.

Definition 3.3 (Counted execution)**.** The relation $s \longrightarrow^n t$ is generated inductively by

$$\frac{}{s \longrightarrow^0 s}, \qquad \frac{s \to u \quad u \longrightarrow^n t}{s \longrightarrow^{n+1} t}.$$

Lean: `Machine.Steps`

These are precisely the constructors `zero` and `cons`. The count includes every transition of the machine. There is no separate cost weight for searching or rewriting. In particular, zero transitions cannot change a state.

Lemma 3.4 (Zero-step equality)**.** *If $s \longrightarrow^0 t$, then $s = t$.*

Lean: `Machine.Steps.zero_eq`

Proof. Only the zero constructor can produce an execution with count zero. Its two endpoints are the same state. The step constructor has a positive count and cannot occur. □

Lemma 3.5 (Concatenation)**.** *If* $s \longrightarrow^m t$ *and* $t \longrightarrow^n v$*, then* $s \longrightarrow^{m+n} v$*.*

Lean: `Machine.Steps.append`

Proof. Induct on the first execution. For a zero-step execution, $s = t$, so the second execution is the required one. Otherwise its first transition is $s \to u$ and its remaining execution has some length k. By induction, that remainder followed by the second execution has length $k + n$. Prepending $s \to u$ gives length $(k + n) + 1 = (k + 1) + n$. No determinism is used. □

Lemma 3.6 (An execution from a normal state)**.** *If* $s \longrightarrow^n t$ *and* $\mathsf{Normal}(s)$*, then* $n = 0$ *and* $s = t$*.*

Lean: `Machine.Steps.normal_eq`

Proof. The zero-step case has both conclusions by definition. A positive execution would begin with a transition out of s, contradicting normality. □

For the following three lemmas, assume determinism.

Lemma 3.7 (Every execution is a prefix of a final execution)**.** *Suppose* $s \longrightarrow^m t$*,* $s \longrightarrow^n z$*, and* $\mathsf{Final}(z)$*. Then there is a natural number* r *such that*

$$n = m + r \quad \textit{and} \quad t \longrightarrow^r z.$$

In particular $m \leq n$*.*

Lean: `Machine.Steps.prefix_of_final`

Proof. Induct on the execution of length m, allowing n to vary. If $m = 0$, then $t = s$; take $r = n$ and use the given final execution.

Otherwise the first execution starts with $s \to u$. The final execution cannot have length zero: that would make $s = z$ final while an outgoing transition exists. It therefore starts with $s \to u'$. Determinism gives $u = u'$. Apply the induction hypothesis to the two remaining executions. If their lengths are $m - 1$ and $n - 1$, the result gives $n - 1 = (m - 1) + r$ and the required execution $t \longrightarrow^r z$. Hence $n = m + r$. The Lean proof uses constructor indices rather than natural-number subtraction, so no subtraction side conditions are needed. □

Lemma 3.8 (Uniqueness of successful terminal executions)**.** *If* $s \longrightarrow^m t$*,* $s \longrightarrow^n z$*,* $\mathsf{Final}(t)$*, and* $\mathsf{Final}(z)$*, then* $m = n$ *and* $t = z$*.*

Lean: `Machine.Steps.final_unique`

Proof. The prefix lemma gives $n = m + r$ and $t \longrightarrow^r z$. Finality of t makes it normal. The normal-state lemma yields $r = 0$ and $t = z$, and therefore $m = n$. □

Lemma 3.9 (A state on a successful execution is not stuck)**.** *If $s \longrightarrow^m t$, $s \longrightarrow^n z$, and $\mathsf{Final}(z)$, then t is not stuck.*

Lean: `Machine.Steps.not_stuck_of_final`

Proof. Suppose t were stuck. The prefix lemma gives a remaining execution $t \longrightarrow^r z$. Normality of t forces $t = z$. Since z is final, t is final, contradicting the unsuccessful part of stuckness. □

Lemma 3.10 (Removing a prescribed first transition)**.** *If $s \to s'$ and $s \longrightarrow^{n+1} t$, then $s' \longrightarrow^n t$.*

Lean: `Machine.Steps.drop_first`

Proof. The positive execution has a first transition $s \to u$ and a remainder $u \longrightarrow^n t$. Determinism identifies u with s', giving the stated remainder. The conclusion concerns the *prescribed* successor s'; without determinism it would only be valid for some successor. □

Lemma 3.11 (A stuck execution continues after an available step)**.** *If $s \to s'$, $s \longrightarrow^n t$, and $\mathsf{Stuck}(t)$, then for some k, $s' \longrightarrow^k t$.*

Lean: `Machine.Steps.stuck_after_step`

Proof. The execution cannot have length zero, since then $s = t$ would be normal despite $s \to s'$. Its first transition is therefore $s \to u$, with a remaining execution to t. Determinism gives $u = s'$. Take the length of that remainder as k. The code does not retain the additional equality $n = k + 1$, because later uses need only the existence of a remainder. □

4 Cost relations and state refinement

Definition 4.1 (Cost preorder)**.** A cost order is a reflexive, transitive relation Q on $\mathbb{N}$.

Lean: `CostOrder`

The relation and its two preorder proofs are stored together. Antisymmetry, decidability, and compatibility with addition are not part of this definition.

Definition 4.2 (Reasonable triple)**.** A triple (Q, P, S) of cost preorders is *reasonable* if all four of the following implications hold:

$$
\begin{aligned}
a\,Q\,b \;\wedge\; c\,Q\,d &\implies (a+c)\,Q\,(b+d), && \text{(addition)}\\
a\,P\,b &\implies b \le a, && \text{(decrease)}\\
a\,P\,b &\implies a\,S\,b, && \text{(inclusion)}\\
a\,P\,b \;\wedge\; b\,Q\,c \;\wedge\; c\,S\,d &\implies a\,Q\,d. && \text{(sandwich)}
\end{aligned}
$$

Lean: `Reasonable`

These four fields express the paper's three conditions: the decrease and inclusion fields are the two parts of its condition (B). The decrease field will justify a strict descent in the soundness proof. Inclusion is used when turning safety's auxiliary equivalences into an asymmetric up-to relation.

Definition 4.3 (Converse and the basic cost orders)**.** The converse preorder is $a\,Q^{-1}\,b$ if $b\,Q\,a$. The four named orders are

$$a\,E\,b \iff a = b, \quad a\,I\,b \iff b \le a,$$
$$a\,H\,b \iff a \le b, \quad a\,U\,b \iff \text{true}.$$

Here E, I, H, and U denote `exact`, `improvement`, `overhead`, and `any`, respectively.

Lean: `CostOrder.converse`

Lean: `CostOrder.exact`

Lean: `CostOrder.improvement`

Lean: `CostOrder.overhead`

Lean: `CostOrder.any`

Verification of the preorder fields. Converse preserves reflexivity. Given $b\,Q\,a$ and $c\,Q\,b$, transitivity of Q gives $c\,Q\,a$, establishing transitivity of Q^{-1}. Equality is reflexive and transitive. The relation I is reflexive by $a \le a$; from $b \le a$ and $c \le b$ one obtains $c \le a$. The corresponding verification for H uses the usual order direction. All requirements for the universal relation U are immediate. □

Lemma 4.4 (Exact costs are reasonable)**.** *The triple (E, E, E) is reasonable.*

Lean: `CostOrder.reasonable_exact`

Proof. Substitute equals for equals in the addition condition. The decrease condition reduces to $a \le a$, inclusion is equality included in itself, and the sandwich condition is transitivity of equality. □

Lemma 4.5 (Improvement costs are reasonable)**.** *The triple (I, I, I) is reasonable.*

Lean: `CostOrder.reasonable_improvement`

Proof. For addition, $b \le a$ and $d \le c$ give $b + d \le a + c$. Decrease is the definition of I, and inclusion is reflexive inclusion. The sandwich premises give $d \le c \le b \le a$, hence $a\,I\,d$. □

Lemma 4.6 (Unconstrained termination costs are reasonable)**.** *The triple (U, I, U) is reasonable.*

Lean: `CostOrder.reasonable_termination`

Proof. Addition, inclusion, and the sandwich condition have universally true conclusions. Decrease is again the definition of I. □

Lemma 4.7 (Overhead with exact auxiliary costs is reasonable)**.** *The triple* (H, E, E) *is reasonable.*

Lean: `CostOrder.reasonable_overhead`

Proof. The addition condition is monotonicity of addition for $\leq$. Equality implies the required weak decrease and includes into itself. In the sandwich condition, substitute $a = b$ and $c = d$ into $b \leq c$ to obtain $a \leq d$. □

Definition 4.8 (State refinement)**.** For a cost preorder Q, define

$$s \preceq_Q t \quad \Longleftrightarrow \quad \begin{array}{c} \text{for every } n, z \text{ with } s \longrightarrow^n z \text{ and } \mathsf{Final}(z), \\ \text{there exist } m, w \text{ with } n\,Q\,m,\ t \longrightarrow^m w, \text{ and } \mathsf{Final}(w). \end{array}$$

State equivalence is mutual refinement: $s \simeq_Q t$ if $s \preceq_Q t$ and $t \preceq_Q s$.

Lean: `Machine.Refines`

Lean: `Machine.Equiv`

If s has no successful execution, its refinement obligation is vacuous. The definition therefore does not distinguish stuckness from divergence unless a surrounding observation supplies additional distinctions. For $Q = I$, the target must terminate in no more steps than the source. For $Q = H$, it must terminate in at least as many steps. For $Q = E$, the counts agree exactly; for $Q = U$, counts are ignored.

Lemma 4.9 (Reflexivity)**.** *For every* s, $s \preceq_Q s$.

Lean: `Machine.Refines.refl`

Proof. Given a successful execution of length n from s, choose that same execution and final state as witnesses. Reflexivity supplies $n\,Q\,n$. □

Lemma 4.10 (Transitivity)**.** *If* $s \preceq_Q t$ *and* $t \preceq_Q v$, *then* $s \preceq_Q v$.

Lean: `Machine.Refines.trans`

Proof. Take a successful execution of length n from s. The first refinement supplies a successful execution of length m from t, with $n\,Q\,m$. Apply the second refinement to this execution to obtain a successful execution of length k from v, with $m\,Q\,k$. Transitivity gives $n\,Q\,k$, and these are the required witnesses. Determinism is unnecessary. □

Lemma 4.11 (Monotonicity in the cost relation)**.** *If* $Q \subseteq P$ *and* $s \preceq_Q t$, *then* $s \preceq_P t$.

Lean: `Machine.Refines.mono`

Proof. Reuse each execution and final-state witness supplied by the original refinement. Replace its cost proof $n\,Q\,m$ by $n\,P\,m$ using the inclusion. □

5 Counting simulation and its soundness

Definition 5.1 (Up-to closure)**.** Given a relation R on states and cost orders P, S, define

$$\mathsf{UpTo}_{P,S}(R)(s,t) \iff \exists u, v\ (s \preceq_P u \ \wedge\ R(u,v)\ \wedge\ v \preceq_S t).$$

Lean: `Machine.UpTo`

The witnesses occur in path order. In particular the right-hand refinement is from v to t, not from t to v. Auxiliary refinement can change the states at which the candidate relation is resumed.

Definition 5.2 (Counting simulation)**.** A relation R is a (Q, P, S)-counting simulation if whenever $R(s,t)$:

(A) If s is final, then t is final.

(B) For every transition $s \to s'$, one of the following holds:

(i) There exist k, z such that $s' \longrightarrow^k z$ and z is stuck.

(ii) There exist k, l, h_1, h_2 such that

$$s' \longrightarrow^k h_1, \qquad t \longrightarrow^l h_2, \qquad (1+k)\,Q\,l, \qquad \mathsf{UpTo}_{P,S}(R)(h_1,h_2).$$

Lean: `Machine.CountingSimulation`

The leading 1 charges for the already prescribed transition $s \to s'$. Neither k nor l is required to be positive. The stuck alternative is a deliberate part of the definition, consistent with observing only successful termination. The finality condition is stronger than simply allowing the target to reach a final state after some extra steps: it requires the target to be final already when the source is final.

Theorem 5.3 (Soundness of counting simulation)**.** *Suppose M is deterministic, (Q, P, S) is reasonable, and R is a (Q, P, S)-counting simulation. Then*

$$R(s,t) \implies s \preceq_Q t.$$

Lean: `Machine.CountingSimulation.sound`

This is the abstract transition-system form of the paper's Proposition 37. Its proof is a strong induction on the length of a successful source execution. Although simulation provides a step-wise, coinductive style of reasoning, the soundness theorem itself is proved by induction on a finite observation; no coinductive datatype is introduced in this file.

Proof. We prove, simultaneously for all states s, t, z, that if $R(s,t)$, $s \longrightarrow^N z$, and $\mathsf{Final}(z)$, then there are m, w such that $N\,Q\,m$, $t \longrightarrow^m w$, and $\mathsf{Final}(w)$. Use strong induction on N; the induction hypothesis applies to every smaller natural number and every pair of related states.

Base case. If $N = 0$, then $s = z$ is final. Condition (A) makes t final. Choose the zero-step execution at t and use $0\,Q\,0$.

Positive case. Write $N = n + 1$ and decompose the successful execution:

$$s \to s' \longrightarrow^n z.$$

Apply condition (B) of simulation. Its stuck alternative would give a finite execution from s' to a stuck state. This is impossible by the lemma that a state on a successful execution cannot be stuck, applied to the successful execution $s' \longrightarrow^n z$.

Consequently there are k, l, h_1, h_2, u, v satisfying

$$\begin{array}{lll} s' \longrightarrow^k h_1, & t \longrightarrow^l h_2, & (1+k)\,Q\,l, \\ h_1 \preceq_P u, & R(u,v), & v \preceq_S h_2. \end{array}$$

The prefix lemma, using determinism and finality of z, gives a remainder length r such that

$$n = k + r, \qquad h_1 \longrightarrow^r z.$$

Apply $h_1 \preceq_P u$ to this remaining successful execution. It supplies a, z_a with

$$r\,P\,a, \qquad u \longrightarrow^a z_a, \qquad \mathsf{Final}(z_a).$$

The decrease condition of reasonableness gives $a \leq r$. Since $N = 1 + k + r$, we have $a < N$. This is the strict inequality needed to invoke the strong induction hypothesis. The proof would not be justified for arbitrary auxiliary cost preorders.

Applying that hypothesis to $R(u, v)$ gives b, z_b such that

$$a\,Q\,b, \qquad v \longrightarrow^b z_b, \qquad \mathsf{Final}(z_b).$$

Now apply $v \preceq_S h_2$. We obtain c, z_c with

$$b\,S\,c, \qquad h_2 \longrightarrow^c z_c, \qquad \mathsf{Final}(z_c).$$

The sandwich condition combines $r\,P\,a$, $a\,Q\,b$, and $b\,S\,c$ to yield $r\,Q\,c$. Additivity of Q then gives

$$(1 + k + r)\,Q\,(l + c), \quad \text{that is,} \quad N\,Q\,(l + c).$$

Concatenate $t \longrightarrow^l h_2$ with $h_2 \longrightarrow^c z_c$. The result is a successful execution of length $l + c$ from t, which supplies the required witnesses. □

The code follows these witnesses exactly: its `count` is $n = k + r$, `smaller` is $a < n + 1$, `rc` is $r\,Q\,c$, and `total` is $(n + 1)\,Q\,(l + c)$. The inclusion $P \subseteq S$ is not needed in this particular proof; it is part of the shared reasonable triple and will be used by the specimen argument.

6 Typed observations and contextual lifting

Definition 6.1 (Observation interface)**.** Let J be a type of interfaces, and for each $i \in J$ let F_i be a type of fragments and C_i a type of contexts. An observation interface consists of a predicate $\mathsf{Allowed}(c)$ for contexts and a relation

$$\mathsf{Obs}(c, a, s), \qquad c \in C_i,\ a \in F_i,\ s \in X.$$

The intended reading is that placing a in c and initializing the result can produce state s.

Lean: `Observations`

The common index i prevents a fragment and a context of incompatible interfaces from being compared. Observation is a relation rather than a total function. It can be undefined, and the structure does not require uniqueness of the produced state. The current code does not yet define it by graph plugging.

Definition 6.2 (Contextual refinement and equivalence)**.** For $a, b \in F_i$, write $a \preceq_Q^{\mathcal{O}} b$ if for every $c \in C_i$ and states s, t,

$$\mathsf{Allowed}(c) \ \wedge\ \mathsf{Obs}(c, a, s) \ \wedge\ \mathsf{Obs}(c, b, t) \implies s \preceq_Q t.$$

Contextual equivalence $a \simeq_Q^{\mathcal{O}} b$ is the conjunction of this refinement and the refinement in the opposite direction.

Lean: `Observations.Refines`

Lean: `Observations.Equiv`

The quantifiers compare every pair of observations of the two fragments in the same allowed context. The definition imposes no existence obligation when either fragment has no observation there. Also, state-refinement reflexivity alone does not establish contextual reflexivity for this fully general interface: two observations of the same fragment might give different states. A concrete, functional initialization relation can supply the missing property, but it is not assumed here.

Proposition 6.3 (Contextual refinement from a lifted relation)**.** *Let T_i be a relation on F_i for each interface i, and let L be a relation on states. Suppose:*

1. *Whenever $T_i(a, b)$ and c is allowed, observations $\mathsf{Obs}(c, a, s)$ and $\mathsf{Obs}(c, b, t)$ imply $L(s, t)$.*
2. *$L(s, t)$ implies $s \preceq_Q t$.*

Then $T_i(a, b)$ implies $a \preceq_Q^{\mathcal{O}} b$.

Lean: `Observations.contextual_of_lifting`

Proof. Fix $T_i(a, b)$, an allowed c, and the two observations. The first hypothesis, named `initial_lift` in Lean, puts their initial states in L. The second hypothesis, named `sound`, then gives the state refinement required by the definition of contextual refinement. □

This isolates the logical step of the paper's Proposition 38. Its concrete proof still has to establish the initial-specimen hypothesis for actual plugging and initialization. The proposition above neither asserts nor requires closure under plugging at this abstract level.

Proposition 6.4 (Contextual equivalence from a lifted relation)**.** *Under the same initial-lifting hypothesis, if $L(s,t)$ implies $s \simeq_Q t$, then $T_i(a,b)$ implies $a \simeq_Q^{\mathcal{O}} b$.*

Lean: `Observations.contextual_equiv_of_lifting`

Proof. For the forward refinement, apply the preceding proposition using the forward component of state equivalence. For the reverse refinement, fix observations of b and a in an allowed context. Put them into `initial_lift` in the order a, b, obtaining L from the state of a to the state of b. Use the reverse component of its state equivalence. This does not require that T_i or L be symmetric. □

7 The abstract specimen interface

Definition 7.1 (Focus modes)**.** The type of modes has exactly three constructors: search, backtrack, and rewrite.

Lean: `Focus`

They represent the paper's searching, backtracking, and triggering focuses. The derived decidable equality allows their distinctness to be checked by finite case analysis. This datatype records only a mode, not a graph position or a focus edge.

Definition 7.2 (Specimen system)**.** A specimen system G over M consists of a type Σ of specimens, maps $\ell, r : \Sigma \to X$, a mode map $\mathsf{mode} : \Sigma \to \mathsf{Focus}$, a predicate Enter on specimens, and a predicate Root on states. Its laws are:

1. *Rootedness preservation:* $\mathsf{Root}(s)$ and $s \to t$ imply $\mathsf{Root}(t)$.
2. *Final-state agreement:* for every $c \in \Sigma$, $\mathsf{Final}(\ell(c)) \iff \mathsf{Final}(r(c))$.
3. *Context-only traversal:* suppose both endpoints of c are rooted and c is either backtracking or searching without entering. If either endpoint has an outgoing transition, there is a specimen d whose focus is remote and such that

$$\ell(c) \to \ell(d), \qquad r(c) \to r(d).$$

The terms "rooted," "entering," and "remote" receive their precise abstract meanings from these data and the definitions immediately below.

Lean: `SpecimenSystem`

The corresponding fields are `Specimen`, `left`, `right`, `mode`, `entering`, `rooted`, `rooted_step`, `same_final`, and `context_step`. The last three are hypotheses needed to construct an instance, not results proved here from a graph model. The intended specimen consists of a multi-hole focussed context and two pointwise related vectors of fragments. That intended representation is not a field of this abstract structure.

In particular, the interface does not identify modes with a focus field on states, define rootedness as reachability from an initial state, or define entering by incidence with a hole. The context-traversal law captures the consequence of graph locality and output-closure that the simulation proof uses. Its availability premise is symmetric between the two endpoints, which will be essential in the converse proof.

Definition 7.3 (Remote, settled, and lifted specimens)**.** A specimen c is remote if

$$\mathsf{Remote}(c) \iff \mathsf{mode}(c) \neq \mathsf{rewrite} \ \vee \ \neg\mathsf{Enter}(c).$$

It is settled if

$$\mathsf{Settled}(c) \iff \mathsf{mode}(c) = \mathsf{backtrack} \ \vee \ (\mathsf{mode}(c) = \mathsf{search} \wedge \neg\mathsf{Enter}(c)).$$

Its lifted state relation is

$$\mathcal{L}_G(s,t) \iff \exists c \in \Sigma \ \mathsf{Remote}(c) \wedge \mathsf{Root}(\ell(c)) \wedge \mathsf{Root}(r(c)) \wedge s = \ell(c) \wedge t = r(c).$$

Lean: `SpecimenSystem.Remote`

Lean: `SpecimenSystem.Settled`

Lean: `SpecimenSystem.Lifting`

Remote excludes only an entering rewrite focus. It allows an entering search focus. Settled is stronger: it excludes every rewrite focus and also excludes an entering search focus. The two equalities in the lifting definition expose a particular specimen as a witness for the endpoints.

Lemma 7.4 (Settled implies remote)**.** *If* $\mathsf{Settled}(c)$*, then* $\mathsf{Remote}(c)$*.*

Lean: `SpecimenSystem.settled_remote`

Proof. A backtracking or searching mode is distinct from the rewrite mode. Therefore the first disjunct in the remote predicate holds in either case. □

Lemma 7.5 (Rootedness along a finite execution)**.** *If* $\mathsf{Root}(s)$ *and* $s \longrightarrow^n t$*, then* $\mathsf{Root}(t)$*.*

Lean: `SpecimenSystem.rooted_steps`

Proof. Induct on the execution. At length zero nothing changes. For a positive execution, the one-step rootedness law makes its first successor rooted; the induction hypothesis propagates this property through the remainder. □

Definition 7.6 (Abstract non-rewrite quasi-specimen)**.** For relations A, B on states, define $\mathsf{Quasi}_G(A,B)(s,t)$ to mean that *if* s and t are rooted, then there is a specimen c with

$$\mathsf{mode}(c) \neq \mathsf{rewrite}, \qquad \mathsf{Root}(\ell(c)), \quad \mathsf{Root}(r(c)),$$
$$A(s,\ell(c)), \qquad B(r(c),t).$$

Lean: `SpecimenSystem.Quasi`

The outer implications matter: no witness is required if either endpoint is unrooted. Subsequent proofs establish both rootedness hypotheses before using a quasi-specimen. Unlike the paper's full definition, this abstract consequence does not compare focus labels of s, t with that of c; there is no state-focus map in the interface. It specifically packages the non-rewrite situation used by safety and robustness.

Lemma 7.7 (A rooted quasi-specimen supplies an up-to witness)**.** *If* $\mathsf{Quasi}_G(\preceq_P, \preceq_S)(s,t)$ *and both* s,t *are rooted, then* $\mathsf{UpTo}_{P,S}(\mathcal{L}_G)(s,t)$*.*

Lean: `SpecimenSystem.quasi_upTo`

Proof. Obtain its witness c. Its non-rewrite mode makes it remote. Together with rootedness of its endpoints, this gives $\mathcal{L}_G(\ell(c), r(c))$. Use $\ell(c)$ and $r(c)$ as the intermediate states in the up-to definition, with the two refinements furnished by the quasi-specimen. □

Definition 7.8 (Input-safety)**.** The predicate $\mathsf{InputSafe}_G(Q,P)$ requires the following for every specimen c whose endpoints are rooted, whose mode is search, and which is entering. At least one of these three cases must hold:

(I) Both endpoints have finite executions to stuck states. Their counts may be unrelated, and either count may be zero.

(II) For some settled specimen d and $k,l \in \mathbb{N}$,

$$\ell(c) \longrightarrow^{1+k} \ell(d), \quad r(c) \longrightarrow^{l} r(d), \quad (1+k)\,Q\,l.$$

(III) For some states a,b and $k,l \in \mathbb{N}$,

$$\ell(c) \longrightarrow^{1+k} a, \quad r(c) \longrightarrow^{1+l} b, \quad (1+k)\,Q\,(1+l),$$

and $\mathsf{Quasi}_G(\simeq_P, \simeq_P)(a,b)$ holds.

Lean: `SpecimenSystem.InputSafe`

Case II requires a positive left execution but permits zero right steps. Case III requires positive executions on both sides and auxiliary *equivalences*. Retaining both directions of those equivalences is necessary for reusing safety in the converse theorem.

Definition 7.9 (Robustness)**.** The predicate $\mathsf{Robust}_G(Q,P,S)$ requires that for every specimen c with rooted endpoints, rewrite mode, and $\neg\mathsf{Enter}(c)$, each transition $\ell(c) \to s'$ satisfies one of:

(i) s' has a finite execution to a stuck state;

(ii) for some k,l,a,b,

$$s' \longrightarrow^{k} a, \quad r(c) \longrightarrow^{l} b, \quad (1+k)\,Q\,l,$$

and $\mathsf{Quasi}_G(\preceq_P, \preceq_S)(a,b)$ holds.

Lean: `SpecimenSystem.Robust`

This is a rooted, remote restriction of the consequences used from the paper's robustness condition. The paper's alternative that an endpoint is unrooted is contradictory under the present premises. The abstract machine does not classify its transition relation into search and rewrite rules; it is the specimen's mode that determines where this predicate is invoked. Establishing the correspondence with concrete rewrite transitions is part of the missing hypernet instance.

8 The forward robustness argument

Throughout this section M is deterministic, (Q, P, S) is reasonable, and G is a specimen system satisfying $\mathsf{InputSafe}_G(Q, P)$ and $\mathsf{Robust}_G(Q, P, S)$. In particular, all the interface laws of Definition 7.2 are available.

Theorem 8.1 (Forward construction of a counting simulation)**.** *The relation $\mathcal{L}_G$ is a (Q, P, S)-counting simulation.*

Lean: `SpecimenSystem.countingSimulation`

This is an abstract form of the forward clause of the paper's Proposition 40. The proof checks the two simulation fields directly.

Proof. Take $\mathcal{L}_G(s, t)$ and choose a witnessing specimen c. Thus $s = \ell(c)$, $t = r(c)$, the specimen is remote, and both endpoints are rooted.

Finality. If s is final, final-state agreement for c makes t final. This verifies simulation condition (A).

A common context-traversal case. Suppose c is settled and there is a prescribed transition $s \to s'$. The context-traversal law gives a remote specimen d and transitions

$$\ell(c) \to \ell(d), \qquad r(c) \to r(d).$$

Determinism identifies s' with $\ell(d)$. Both new endpoints are rooted, so $\mathcal{L}_G(\ell(d), r(d))$. The simulation matching case can use zero further left steps and one right step. Its cost obligation is $(1 + 0)\, Q\, 1$, which is reflexivity of Q. To obtain the up-to relation, choose $\ell(d), r(d)$ as intermediate states and insert reflexive refinements for P and S on either side of their lifted relation. These are the local proof witnesses named `context_case` and `settled_case` in Lean.

It remains to cover all possible modes of c.

Backtracking. Such a specimen is settled by definition, so the common context-traversal argument applies.

Searching without entering. This is the other settled case and uses the same argument.

Entering search. Apply input-safety, obtaining one of its three cases.

(I) There is a stuck execution from $\ell(c)$. Since $\ell(c) \to s'$ is available, the stuck-after-step lemma removes that first step and gives a stuck execution from s'. This is the stuck alternative of simulation. Safety also supplies a stuck execution on the right, but the forward simulation condition does not need it.

(II) There are d, k, l with d settled,

$$\ell(c) \longrightarrow^{1+k} \ell(d), \qquad r(c) \longrightarrow^{l} r(d), \qquad (1 + k)\, Q\, l.$$

Remove the prescribed first transition from the left execution using determinism. The resulting execution from s' to $\ell(d)$ has length k. Rootedness propagates to both endpoints of d, and settledness implies remoteness. Hence $\mathcal{L}_G(\ell(d), r(d))$. Add reflexive auxiliary refinements as in the context-traversal case. The matching counts are k, l, with the cost condition already supplied by safety.

(III) There are k, l, a, b with the positive executions and quasi-specimen of safety III. Remove the prescribed first step from the left execution, giving $s' \longrightarrow^{k} a$; retain the right

execution of length $1 + l$. Both a and b are rooted. Unpack the quasi-specimen to get a non-rewrite specimen d with rooted endpoints and

$$a \simeq_P \ell(d), \qquad r(d) \simeq_P b.$$

Use the forward half of the first equivalence to obtain $a \preceq_P \ell(d)$. The forward half of the second gives $r(d) \preceq_P b$, and monotonicity with $P \subseteq S$ changes this to $r(d) \preceq_S b$. Thus the quasi-up-to lemma supplies $\mathsf{UpTo}_{P,S}(\mathcal{L}_G)(a, b)$. The matching counts are $k, 1 + l$, and their required cost relation is exactly $(1 + k)\, Q\, (1 + l)$.

Rewrite mode. Remoteness now implies $\neg\mathsf{Enter}(c)$, since the other remote disjunct contradicts the current mode. Apply robustness to the prescribed transition $\ell(c) \to s'$. If it provides a stuck execution from s', use the stuck alternative of simulation. Otherwise it provides matching executions of lengths k, l, the cost proof $(1+k)\, Q\, l$, and a quasi-specimen up to $\preceq_P, \preceq_S$. Its left endpoint is rooted by the prescribed step followed by the k-step execution; its right endpoint is rooted by its l-step execution. The quasi-up-to lemma therefore supplies the required up-to relation. This completes every mode and every simulation condition. □

Theorem 8.2 (Forward state sufficiency)**.** *Under the hypotheses of this section,*

$$\mathcal{L}_G(s, t) \implies s \preceq_Q t.$$

Lean: `SpecimenSystem.sufficiency`

Proof. The preceding theorem constructs a counting simulation on $\mathcal{L}_G$. Apply counting-simulation soundness, Theorem 5.3, with the same deterministic machine and reasonable triple. □

All graph-specific information in this theorem is supplied through the specimen system and the safety and robustness premises. The theorem's short proof is therefore not evidence that the missing graph-locality verification is automatic.

9 Opposite specimens and the converse argument

Definition 9.1 (Opposite specimen system)**.** The opposite system G^{op} has the same specimens, modes, entering predicate, and rooted-state predicate as G. It swaps the two endpoint maps:

$$\ell_{G^{\mathrm{op}}}(c) = r_G(c), \qquad r_{G^{\mathrm{op}}}(c) = \ell_G(c).$$

Lean: `SpecimenSystem.opposite`

Verification of its interface laws. Rootedness preservation is unchanged. Final-state agreement is the reverse implication pair of the original agreement. For context-only traversal, exchange the two rootedness hypotheses and the two alternatives of the availability premise. Apply the original law to get a remote specimen d and two transitions. Return those transitions in the opposite order. This explains why the interface law permits availability on either side, rather than only on the left. □

Lemma 9.2 (Lifting commutes with taking the opposite)**.** *For all states s, t,*

$$\mathcal{L}_{G^{\mathrm{op}}}(s,t) \iff \mathcal{L}_G(t,s).$$

Lean: `SpecimenSystem.lifting_opposite`

Proof. In either direction retain the witnessing specimen and its remoteness. Exchange the two rootedness proofs and the two endpoint equalities. The resulting tuple is precisely the witness required by the other lifting definition. □

Theorem 9.3 (Converse construction of a counting simulation)**.** *Suppose M is deterministic, (Q, P, S) is reasonable, $\mathsf{InputSafe}_G(Q^{-1}, P)$ holds, and G^{op} is robust for (Q, P, S). Then $\mathcal{L}_{G^{\mathrm{op}}}$ is a (Q, P, S)-counting simulation.*

Lean: `SpecimenSystem.countingSimulation_converse`

The safety premise is a property of the *original* system at the converse cost relation. It is not assumed that the opposite system already satisfies input-safety. This distinction is important when the right execution in safety II has length zero.

Proof. Choose a witness c of $\mathcal{L}_{G^{\mathrm{op}}}(s,t)$, so $s = r(c)$, $t = \ell(c)$, c is remote, and both endpoints are rooted. Finality of $r(c)$ implies finality of $\ell(c)$ by the reverse direction of final-state agreement. This establishes condition (A). For condition (B), fix a transition $r(c) \to s'$.

Settled case. If c is settled, the symmetric availability premise of context-only traversal applies to this right transition. Obtain a remote specimen d with transitions

$$\ell(c) \to \ell(d), \qquad r(c) \to r(d).$$

Determinism gives $s' = r(d)$. Propagate rootedness, and obtain $\mathcal{L}_{G^{\mathrm{op}}}(r(d), \ell(d))$. Choose zero further steps from s' and one step from $\ell(c)$. The cost condition is $1\, Q\, 1$, and reflexive refinements give the up-to witness. This covers backtracking and non-entering search.

Entering search. Input-safety for Q^{-1}, P supplies three cases, which must be read in the original endpoint order.

(I) Use the supplied stuck execution from $r(c)$. The prescribed step $r(c) \to s'$ and the stuck-after-step lemma give a stuck execution from s'. The supplied stuck execution from $\ell(c)$ is not needed in this direction.

(II) There are a settled specimen d and counts k, l with

$$\ell(c) \longrightarrow^{1+k} \ell(d), \qquad r(c) \longrightarrow^{l} r(d), \qquad l\, Q\, (1+k).$$

Both endpoints of d are rooted. We distinguish the two possible forms of l.

If $l = j+1$, remove the prescribed transition from the right execution to get $s' \longrightarrow^{j} r(d)$. Keep the left execution of length $1+k$. The opposite lifted pair is $(r(d), \ell(d))$, and its cost condition is $(1+j)\, Q\, (1+k)$, exactly the supplied relation with $l = j+1$. Reflexive auxiliary refinements finish this subcase.

If $l = 0$, the zero-step equality gives $r(c) = r(d)$. Thus the prescribed transition is an outgoing transition from $r(d)$. Apply context-only traversal to the *settled* specimen d. Obtain a remote specimen e and transitions

$$\ell(d) \to \ell(e), \qquad r(d) \to r(e).$$

Determinism identifies s' with $r(e)$. Propagating rootedness gives the opposite lifted pair $(r(e), \ell(e))$. There are zero further source steps, whereas the target execution is

$$\ell(c) \longrightarrow^{1+k} \ell(d) \to \ell(e),$$

of length $(1+k)+1$. Its cost obligation is $1\,Q\,((1+k)+1)$. The safety relation in this subcase is $0\,Q\,(1+k)$. Add $1\,Q\,1$ using additivity of Q to obtain the required relation. This additional traversal is the essential zero-step correction; the proof never attempts to remove a step from a zero-step execution.

(III) There are k, l, a, b with

$$\ell(c) \longrightarrow^{1+k} a, \qquad r(c) \longrightarrow^{1+l} b, \qquad (1+l)\,Q\,(1+k),$$

and a quasi-specimen for auxiliary equivalences. Remove the prescribed right transition to get $s' \longrightarrow^{l} b$. Both a, b are rooted; unpacking the quasi-specimen yields a non-rewrite specimen d with rooted endpoints and

$$a \simeq_P \ell(d), \qquad r(d) \simeq_P b.$$

This time take the *reverse* halves: $b \preceq_P r(d)$ and $\ell(d) \preceq_P a$. Enlarge the latter auxiliary order using $P \subseteq S$ to obtain $\ell(d) \preceq_S a$. Since $\mathcal{L}_{G^{\mathrm{op}}}(r(d), \ell(d))$, these form

$$\mathsf{UpTo}_{P,S}(\mathcal{L}_{G^{\mathrm{op}}})(b, a).$$

The matching counts are $l, 1+k$, with the already supplied cost relation $(1+l)\,Q\,(1+k)$.

Rewrite mode. Remoteness gives $\neg\mathsf{Enter}(c)$. Apply robustness in G^{op} to the prescribed transition from $r(c)$, which is its left endpoint. The stuck alternative is already the required one. Otherwise the two matching executions and their cost relation are supplied directly. Rootedness propagates along the prescribed transition and both executions. Apply the quasi-up-to lemma *in the opposite system* to obtain the up-to relation on $\mathcal{L}_{G^{\mathrm{op}}}$. This completes the proof. □

10 State and contextual consequences

The following results compose the two simulation constructions with the soundness and observation lemmas. They are short in Lean because their mathematical work has been isolated in those earlier results.

Theorem 10.1 (Converse state sufficiency)**.** *If M is deterministic, (Q, P, S) is reasonable, $\mathsf{InputSafe}_G(Q^{-1}, P)$ holds, and G^{op} is robust for (Q, P, S), then*

$$\mathcal{L}_G(s, t) \implies t \preceq_Q s.$$

Lean: `SpecimenSystem.sufficiency_converse`

Proof. The lifting-opposite lemma changes the premise into $\mathcal{L}_{G^{\mathrm{op}}}(t, s)$. The converse construction supplies a counting simulation on this relation. Apply counting-simulation soundness to obtain the stated refinement. □

Theorem 10.2 (Mutual state refinement)**.** *Suppose M is deterministic, (Q, P, S) is reasonable, G is input-safe for both (Q, P) and (Q^{-1}, P), and both G and G^{op} are robust for (Q, P, S). Then*

$$\mathcal{L}_G(s, t) \implies s \simeq_Q t.$$

Lean: `SpecimenSystem.equivalence`

Proof. Forward state sufficiency supplies $s \preceq_Q t$. Converse state sufficiency supplies $t \preceq_Q s$. These are exactly the two components of state equivalence. □

The main cost relation Q itself need not be symmetric. The two directional refinement statements use the same Q, with source and target counts exchanged when the states are exchanged.

For the contextual results, fix the observation interface of the earlier section and a family T_i of relations on fragments. Assume the following *initial-lifting property*: for $T_i(a, b)$, every allowed context c and observations $\mathsf{Obs}(c, a, s)$, $\mathsf{Obs}(c, b, t)$ give $\mathcal{L}_G(s, t)$. This property is the explicit premise `initial_lift` in the source. It connects the abstract specimen system with the separately supplied observation relation.

Theorem 10.3 (Forward contextual sufficiency)**.** *If M is deterministic, (Q, P, S) is reasonable, $\mathsf{InputSafe}_G(Q, P)$ and $\mathsf{Robust}_G(Q, P, S)$ hold, and the initial-lifting property holds, then*

$$T_i(a, b) \implies a \preceq_Q^{\mathcal{O}} b.$$

Lean: `SpecimenSystem.contextual_sufficiency`

Proof. Use contextual refinement from a lifted relation, taking $L = \mathcal{L}_G$. Its initial-lifting premise is precisely the one assumed here. Its state-soundness premise is forward state sufficiency. Applying that proposition to $T_i(a, b)$ gives the desired contextual refinement. □

Theorem 10.4 (Converse contextual sufficiency)**.** *If M is deterministic, (Q, P, S) is reasonable, $\mathsf{InputSafe}_G(Q^{-1}, P)$ holds, G^{op} is robust for (Q, P, S), and the same initial-lifting property holds, then*

$$T_i(a, b) \implies b \preceq_Q^{\mathcal{O}} a.$$

Lean: `SpecimenSystem.contextual_sufficiency_converse`

Proof. Fix $T_i(a, b)$, an allowed context, an observation of b producing s, and an observation of a producing t. Supply the observations to initial lifting in the original a, b order to get $\mathcal{L}_G(t, s)$. Converse state sufficiency then gives $s \preceq_Q t$, the required contextual comparison. □

These are conditional abstract counterparts of the two clauses of the paper's Theorem 6. Refocusing, output-closure, type-correct plugging, and the paper's concrete specimen definitions have not disappeared from the mathematics: their required consequences are currently represented by the specimen-system laws and initial-lifting premise. Instantiating those hypotheses is the remaining concrete formalization problem.

11 Semantic examples and regression proofs

The examples are small abstract machines used to check the meaning and edge cases of the definitions. They do not formalize the paper's lambda-calculus translation, copying UAM, or program-equivalence examples.

11.1 A countdown machine

Definition 11.1 (Countdown)**.** The countdown machine has state set $\mathbb{N}$, transition $s \to t$ if $s = t + 1$, and final predicate $s = 0$.

Lean: `Examples.countdown`

Verification of the machine law. If $s = 0$, an outgoing transition would imply $0 = t + 1$, impossible for a natural number t. Thus final states are normal as required by the machine structure. □

Lemma 11.2 (Countdown is deterministic)**.** *If $s \to a$ and $s \to b$, then $a = b$.*

Lean: `Examples.countdown_deterministic`

Proof. The premises give $s = a + 1$ and $s = b + 1$. Cancellation of successor gives $a = b$. □

Lemma 11.3 (Canonical countdown execution)**.** *For every $n \in \mathbb{N}$, the countdown machine has $n \longrightarrow^n 0$.*

Lean: `Examples.countdown_run`

Proof. Induct on n. For $n = 0$, use the zero-step constructor. For $n + 1$, take the transition $n + 1 \to n$ and prepend it to the induction hypothesis $n \longrightarrow^n 0$. □

Proposition 11.4 (Countdown characterizes the cost relation)**.** *For every cost preorder Q and states $a, b \in \mathbb{N}$,*

$$a \preceq_Q b \quad \Longleftrightarrow \quad a \, Q \, b.$$

Lean: `Examples.countdown_refines_iff`

Proof. Suppose $a \preceq_Q b$. Apply it to the canonical successful execution $a \longrightarrow^a 0$. It gives a successful execution of some length m from b and $a \, Q \, m$. Compare this execution with $b \longrightarrow^b 0$. Determinism and final-execution uniqueness give $m = b$, so $a \, Q \, b$.

Conversely assume $a \, Q \, b$. For any successful execution of length n from a, compare it with $a \longrightarrow^a 0$. Uniqueness gives $n = a$. Choose $b \longrightarrow^b 0$ as the target execution; its cost condition is $n \, Q \, b$, obtained by replacing n with a. □

The file then proves three anonymous examples, each by the proposition and the indicated numerical fact:

1. $5 \preceq_I 3$, because $3 \leq 5$.
2. $3 \not\preceq_I 5$, because $5 \not\leq 3$.

3. $5 \not\preceq_E 3$, because $5 \neq 3$.

Lean: `Examples: improvement from 5 to 3`

Lean: `Examples: no improvement from 3 to 5`

Lean: `Examples: unequal exact counts`

These verify that improvement uses the target count as the smaller quantity. Reversing the intended cost orientation would invalidate the first two examples.

11.2 A crash and an infinite loop

Definition 11.5 (Fault states and the fault machine)**.** There are four distinct states:

$$\mathsf{done}, \quad \mathsf{crash}, \quad \mathsf{stuck}, \quad \mathsf{loop}.$$

The only transitions are

$$\mathsf{crash} \to \mathsf{stuck}, \qquad \mathsf{loop} \to \mathsf{loop}.$$

Only done is final.

Lean: `Examples.FaultState`

Lean: `Examples.fault`

The four names are constructors of a finite datatype with derived decidable equality. Here the constructor stuck names a particular state; $\mathsf{Stuck}(s)$ remains the general predicate defined earlier.

Verification of the machine law. No transition has source done, because both possible source constructors are distinct from it. Consequently a final state has no outgoing transition. □

Lemma 11.6 (The fault machine is deterministic)**.** *Any two transitions with the same source have the same target.*

Lean: `Examples.fault_deterministic`

Proof. Case-analyze the two transition proofs. If both use the crash rule, both targets are stuck. If both use the loop rule, both targets are loop. Either mixed case would equate crash with loop and is impossible. □

Definition 11.7 (The crash–loop candidate)**.** Let $R(s,t)$ mean $s = \mathsf{crash}$ and $t = \mathsf{loop}$.

Lean: `Examples.crashLoop`

Lemma 11.8 (A simulation using the stuck alternative)**.** *The crash–loop candidate is an* (E,E,E)*-counting simulation.*

Lean: `Examples.crash_loop_simulation`

Proof. For its only related pair, the source is crash, which is not final; hence the finality condition is vacuous. Its only successor is stuck. Use the zero-step execution from that successor to itself in the stuck alternative of simulation. It is normal because neither transition rule starts there, and it is not final because it differs from done. □

The next anonymous example concludes $\mathsf{crash} \preceq_E \mathsf{loop}$.

Lean: `Examples: crash refines loop`

Proof. Apply counting-simulation soundness to the preceding simulation, fault determinism, the reasonable triple (E,E,E), and the only pair of R. □

There is no requirement to match a successful source execution, because none exists. Thus the result is consistent even with exact costs. The example exercises the stuck branch of the implemented proof; it does not prove that every conceivable up-to simulation proof of this same refinement must use that branch.

11.3 The zero-step converse case

Definition 11.9 (Visit specimens and endpoint maps)**.** The type of specimens has three distinct constructors entering, settled, finished, with derived decidable equality. The endpoint maps into the countdown machine are

$$\begin{aligned} \ell(\mathsf{entering}) &= 2, \quad r(\mathsf{entering}) = 1, \\ \ell(\mathsf{settled}) &= 1, \quad r(\mathsf{settled}) = 1, \\ \ell(\mathsf{finished}) &= 0, \quad r(\mathsf{finished}) = 0. \end{aligned}$$

Lean: `Examples.Visit`

Lean: `Examples.visitLeft`

Lean: `Examples.visitRight`

Definition 11.10 (Visit specimen system)**.** Every visit specimen has search mode. The entering predicate holds exactly of the constructor entering. Every state is declared rooted. Together with the endpoint maps above, these data define a specimen system over countdown.

Lean: `Examples.visitSystem`

Verification of the interface laws. Rootedness preservation is immediate since rootedness is the true predicate. Final-state agreement holds because the endpoint pairs are $(2,1)$, $(1,1)$, and $(0,0)$: only in the third pair is either endpoint zero, and then both are.

For context-only traversal, consider the specimen. The constructor entering cannot satisfy the settled premise: its mode is search and its entering predicate holds. At the constructor settled, both endpoints are 1. Choose finished and the countdown transitions $1 \to 0$ on both sides; its search mode is remote. At finished, both endpoints are zero and neither has an outgoing transition, contradicting the availability premise. These cases exhaust the datatype. □

This is a deliberately abstract instance. For example, all its modes are search, including that of the pair of final states. The specimen interface does not enforce a concrete UAM convention for final focus modes.

Lemma 11.11 (Visit input-safety)**.** *The visit system is input-safe for* (H^{-1}, E).

Lean: `Examples.visit_safe`

Proof. The entering premise forces the input specimen to be entering. Use safety II with output specimen settled and $k = l = 0$. The left execution is the single countdown transition $2 \to 1$; the right execution has zero steps at 1. The output specimen is searching without entering and is therefore settled. The cost requirement is $1\, H^{-1}\, 0$, or $0 \le 1$. □

Lemma 11.12 (Visit converse robustness)**.** *The opposite visit system is robust for* (H, E, E).

Lean: `Examples.visit_converse_robust`

Proof. Its mode is always search. The rewrite-mode premise of robustness is therefore impossible, so every required implication holds vacuously. □

Proposition 11.13 (The converse zero-step regression)**.** *In countdown,* $1 \preceq_H 2$.

Lean: `Examples.converse_zero_step_case`

Proof. Use converse state sufficiency for the visit system. Countdown is deterministic, (H, E, E) is reasonable, and the preceding two lemmas supply its safety and converse robustness premises. The entering specimen is remote, both its endpoints are rooted, and it witnesses $\mathcal{L}_G(2,1)$. The theorem consequently gives $1 \preceq_H 2$.

In the underlying converse simulation construction, safety II provides the pair $(1,1)$ after one original left step and zero original right steps. The prescribed right step is $1 \to 0$. Context traversal at $(1,1)$ supplies $(0,0)$ and adds one transition to the original left execution, now $2 \to 1 \to 0$. Thus the source terminates in one step and the target in two, satisfying $1\, H\, 2$. This is precisely the zero-step subcase proved in the converse argument. □

12 Finite carriers and concrete interfaced graphs

We now turn to actual graph objects. The constructions in the following sections prove their own incidence and typing obligations. They do not assume the specimen laws used in the preceding abstract argument. We work with small finite vertex and edge carriers; operation labels may live in a larger universe.

12.1 Enumerations and bijections

Definition 12.1 (Finite enumeration)**.** An enumeration of a type A consists of a finite list l and a proof that every $a \in A$ belongs to l. Repetitions are allowed. This witnesses finiteness without requiring decidable equality or a canonical choice of names.

Lean: `Enumeration`

The empty type is enumerated by the empty list: completeness follows because there is no element to check. The finite ordinal $\{0, \ldots, n-1\}$ is enumerated in increasing order by Lean's finite range; its library membership theorem proves completeness, including when $n = 0$.

Lean: `Enumeration.empty`

Lean: `Enumeration.fin`

For a disjoint union $A + B$, concatenate the tagged enumerations of A and B. An element has exactly one tag, and completeness of the corresponding list places it in the appropriate half of the concatenation.

Lean: `Enumeration.sum`

For a subset $\{a \in A \mid p(a)\}$, attach membership certificates to the enumeration of A and retain just entries satisfying p. Classical decidability of p allows this filtering. Every subset element is retained because its certificate proves the test true. The resulting definition is noncomputable for an arbitrary proposition p; no algorithm deciding p is claimed.

Lean: `Enumeration.subtype`

For a dependent sum $\sum_{a \in A} B_a$, enumerate each fibre, tag it with a, and concatenate over the enumeration of A. A pair (a, b) appears because a occurs in the outer list and b in the enumeration of its fibre.

Lean: `Enumeration.sigma`

Definition 12.2 (Bijection)**.** A bijection $f : A \cong B$ consists of functions $f : A \to B$ and $f^{-1} : B \to A$ with both inverse equations, pointwise.

Lean: `Bijection`

Identity uses the identity function twice. Inversion interchanges the two functions and their inverse laws. Composition uses $g \circ f$ forwards and $f^{-1} \circ g^{-1}$ backwards; cancelling the inner inverse pair and then the outer pair proves each law.

Lean: `Bijection.refl`

Lean: `Bijection.symm`

Lean: `Bijection.trans`

Bijections extend to disjoint unions by applying the appropriate map inside each tag. Case analysis on the tag reduces both inverse laws to the component laws.

Lean: `Bijection.sum`

Every bijection is injective: apply its inverse to an equality of images and cancel. An enumeration transports along a bijection by mapping its forward function over the list. For $b \in B$, the listed preimage $f^{-1}(b)$ maps to b, proving completeness.

Lean: `Bijection.injective`

Lean: `Bijection.enumeration`

Lemma 12.3 (Distinctness under restricted injectivity)**.** *If a list l has no repeated elements and f is injective on elements of l, then* $\mathrm{map}(f, l)$ *has no repeated elements.*

Proof. Distinct positions in l contain distinct elements. If their images were equal, restricted injectivity would identify those elements, a contradiction. Lean expresses this as preservation of the pairwise inequality predicate under mapping. □

Lean: `nodup_map_on`

This restricted version will be essential: replacement can identify an input with an output, while remaining injective on each incidence list separately.

12.2 Ports, signatures, and graph invariants

Definition 12.4 (Signature and ports)**.** A signature σ assigns an ordered input-label list and output-label list to every operation. With internal carrier V and boundary lengths n, m, vertices are the disjoint union

$$\mathrm{Port}(V, n, m) = V + \mathrm{Fin}(n) + \mathrm{Fin}(m).$$

The three constructors are internal, input, and output. In particular the two boundary sets are disjoint even if their wire labels agree.

Lean: `Signature`

Lean: `Port`

The predicates IsInput and IsOutput test the corresponding constructor. A carrier map $f : V \to W$ acts only on internal vertices and fixes each boundary position. It is injective whenever f is: unlike constructors cannot have equal images, and equality within the internal constructor reduces to injectivity of f. The input and output predicates are preserved and reflected by this map, by the same three-case analysis.

Lean: `Port.IsInput`

Lean: `Port.IsOutput`

Lean: `Port.map`

Lean: `Port.map_injective`

Lean: `Port.map_isInput`

Lean: `Port.map_isOutput`

Given boundary-label lists I, O and internal labelling $\lambda : V \to L$, the label of an internal vertex is $\lambda(v)$, of input i is I_i, and of output j is O_j. The finite-index types ensure that these lookups are defined.

Lean: `Port.label`

Definition 12.5 (Concrete typed net)**.** A net $G : I \to O$ has finite internal and edge carriers, labels for both, and ordered source and target vertex lists $s(e), t(e)$ for each edge. It satisfies:

1. Each individual source list and target list has no repetitions.
2. A vertex belongs to the sources of at most one edge, and to the targets of at most one edge.
3. A vertex belongs to some source list exactly when it is not an output; it belongs to some target list exactly when it is not an input.
4. Mapping vertex labels over $s(e)$ and $t(e)$ gives precisely the input and output lists of the signature of the edge label.

Lean: `Net`

These fields are part of a net's certificate of well-formedness. Coverage excludes isolated vertices. It does not exclude cycles or a vertex belonging to both the source and target of the same edge. The full vertex type and its label function are abbreviations for the port type and the labelling just defined.

Lean: `Net.Vertex`

Lean: `Net.vertexLabel`

From forward source coverage, membership in $s(e)$ implies that a vertex is not an output. From target coverage, membership in $t(e)$ implies that it is not an input. Apply the appropriate equivalence to the witness edge e.

Lean: `Net.source_not_output`

Lean: `Net.target_not_input`

Taking lengths in the two typing equations shows that incidence-list lengths are exactly the corresponding signature arities: mapping labels does not change length.

Lean: `Net.source_length`

Lean: `Net.target_length`

The ith source port of e is its source list indexed by a position in the signature's input list; the preceding length equality justifies the lookup. Target ports are defined in the same way with the output list.

Lean: `Net.sourcePort`

Lean: `Net.targetPort`

Each indexed port belongs to its incidence list, by the list lookup membership lemma. Within either list, equality of indexed ports implies equality of indices, because that list has no duplicates; equality of finite ordinals follows from equality of their underlying natural numbers.

Lean: `Net.sourcePort_mem`

Lean: `Net.targetPort_mem`

Lean: `Net.sourcePort_injective`

Lean: `Net.targetPort_injective`

The label of source port i is the ith signature input label, and the label of target port i is the ith signature output label. Apply indexed lookup to the appropriate list typing equation, then commute lookup with mapping.

Lean: `Net.sourcePort_label`

Lean: `Net.targetPort_label`

Finally, membership in a source list is equivalent to being its indexed port for some signature input position. Given membership, choose its list index and transport the bound along the arity equality. Conversely, indexed ports belong to their lists. The target statement uses the output arity and the identical membership argument.

Lean: `Net.mem_source_iff_port`

Lean: `Net.mem_target_iff_port`

12.3 Elementary graph constructors

The ordered finite range has no duplicates: equality of its entries at two positions gives equality of their underlying natural numbers and hence of the positions.

Lean: `finRange_nodup`

Definition 12.6 (Corolla and empty graph)**.** The corolla of operation o has one edge labelled o, no internal vertices, and all of its prescribed inputs and outputs in signature order. Its sources are precisely the input constructors and its targets the output constructors.

Well-formedness. Finite ranges and their constructor maps give distinct incidence lists. There is only one edge, so edge uniqueness is immediate. Every input occurs in its source range and every output in its target range, and the opposite boundary constructors cannot occur there. There are no internal vertices to check. Mapping labels over either ordered range recovers the corresponding signature list, proving typing. □

Lean: `Net.corolla`

The empty graph has empty internal and edge carriers and empty boundaries. Each incidence or label function has empty domain; all obligations follow by eliminating the purported element of an empty type. A bare identity wire is not provided: a single vertex serving as both boundary input and boundary output would violate the disjoint-boundary representation used here.

Lean: `Net.empty`

13 Shallow plugging, including self-loop holes

Fix a host $C : I \to O$, an edge h to remove, and a replacement H whose boundary lists are exactly the input and output signature lists of C's label at h. All constructions in this section are in the namespace Net.Replacement unless explicitly stated otherwise.

13.1 Carriers and boundary identification

Definition 13.1 (Replacement carriers)**.** The new internal vertices are $C_V + H_V$, with disjoint tags providing freshness. The new edges are $(C_E \setminus \{h\}) + H_E$. The outer boundary remains I, O. The surviving-edge subtype carries a proof of inequality to h, so the removed edge cannot accidentally survive as an old edge.

Lean: `Net.Replacement.Vertex`

Lean: `Net.Replacement.Edge`

The host embedding j tags each old internal vertex on the left and fixes the outer boundary. The insertion map k tags a replacement internal vertex on the right, sends replacement input i to $j(s_h(i))$, and sends replacement output i to $j(t_h(i))$. Thus gluing is expressed by a function rather than a quotient of all vertex names.

Lean: `Net.Replacement.host`

Lean: `Net.Replacement.insert`

The host embedding is injective by injectivity of the left sum tag and the port map lemma. Its image is disjoint from freshly tagged replacement internals: case analysis on a host vertex gives either a boundary constructor or an internal left tag, neither equal to an internal right tag.

Lean: `Net.Replacement.host_injective`

Lean: `Net.Replacement.host_ne_internal`

Lemma 13.2 (The two restricted insertion injections)**.** *The insertion map is injective on vertices that are not outputs. It is also injective on vertices that are not inputs.*

Proof. For the first claim, exclude output cases using the hypotheses. Two internal images agree only when their right-tagged names agree. An internal image cannot agree with an input image, by freshness. Two input images agree only when their host source ports agree, and source-port injectivity identifies their indices. For the second claim, exclude input cases instead, use target-port injectivity on two outputs, and use the same freshness argument on mixed cases. □

Lean: `Net.Replacement.insert_source_injective`

Lean: `Net.Replacement.insert_target_injective`

If a non-output replacement vertex has insertion image $j(v)$, it must be some input i, with $s_h(i) = v$. Outputs are excluded by hypothesis, internals by freshness, and in the input case host injectivity gives the equation. Dually, a non-input replacement vertex mapping to $j(v)$

must be output i with $t_h(i) = v$. These inversion lemmas control mixed old/new incidence cases.

Lean: `Net.Replacement.insert_source_host`

Lean: `Net.Replacement.insert_target_host`

13.2 Preservation of incidence invariants

Sources and targets of surviving host edges are their old lists mapped by j. Those of inserted edges are the replacement lists mapped by k.

Lean: `Net.Replacement.source`

Lean: `Net.Replacement.target`

Lemma 13.3 (Distinct incidence entries)**.** *Every new source list and every new target list has no duplicates.*

Proof. A surviving host list uses the injective map j. A replacement source list contains no outputs, so the first restricted injection of k applies to its members. A replacement target list contains no inputs, so the second injection applies. In all cases use preservation of distinctness under restricted injectivity. □

Lean: `Net.Replacement.source_nodup`

Lean: `Net.Replacement.target_nodup`

Lemma 13.4 (Unique source and target incidence)**.** *A vertex belongs to the sources of at most one new edge, and to the targets of at most one new edge.*

Proof. For source incidence, split on the two edge tags. Two host edges reduce, by injectivity of j, to old source uniqueness. Two inserted edges reduce, by restricted source injectivity of k, to source uniqueness in H. In the mixed case the shared vertex is an inserted input, by the source inversion lemma; its corresponding host vertex is therefore a source port of h. Old source uniqueness identifies the surviving host edge with h, contradicting its subtype certificate. The other mixed ordering is the same contradiction with the equality reversed. For target incidence, repeat the four cases with target uniqueness, restricted target injectivity, and the target inversion lemma: the mixed vertex is now an inserted output and a target port of h. □

Lean: `Net.Replacement.source_unique`

Lean: `Net.Replacement.target_unique`

A vertex in a new source list is not an outer output. In the host case use the old source property and preservation of boundary constructors. In the inserted case an internal vertex remains internal, an input maps to a source port of h, and an output is impossible in a replacement source list. For a new target list, exchange source/input with target/output: inserted inputs are impossible and inserted outputs map to host target ports, which cannot be outer inputs.

Lean: `Net.Replacement.source_not_output`

Lean: `Net.Replacement.target_not_input`

Lemma 13.5 (Coverage of surviving host vertices)**.** *For every host vertex v that is not an output, $j(v)$ belongs to a new source list. If v is not an input, $j(v)$ belongs to a new target list.*

Proof. Choose an old source edge by coverage in C. If it differs from h, retain it. If it equals h, express v as source port i; replacement input i belongs to a source edge by coverage in H, and its image is $j(v)$. For target coverage, choose an old target edge. Retain it unless it is h; in that case express v as target port i and use coverage at replacement output i. □

Lean: `Net.Replacement.source_host_covered`

Lean: `Net.Replacement.target_host_covered`

Full source coverage now follows by splitting a new vertex into a host internal, a fresh internal, an outer input, or an outer output. Host internals and inputs use the preceding lemma. Fresh internals use replacement coverage and map the witness edge into the right edge tag. Outer outputs contradict the required non-output hypothesis. The reverse implication is the source exclusion already proved. Full target coverage uses host internals and outer outputs, replacement coverage for fresh internals, and excludes outer inputs; its reverse implication is target exclusion.

Lean: `Net.Replacement.source_coverage`

Lean: `Net.Replacement.target_coverage`

13.3 Labels, typing, and the certified result

New internal labels use the host label on left tags and replacement label on right tags. Surviving edges keep their labels; inserted edges receive their replacement labels.

Lean: `Net.Replacement.internalLabel`

Lean: `Net.Replacement.edgeLabel`

Labelling commutes with j, by the three port cases. It also commutes with k: internal labels agree by definition, while input and output labels agree by the host indexed-port typing lemmas and the matching replacement interface.

Lean: `Net.Replacement.host_label`

Lean: `Net.Replacement.insert_label`

For source typing, split the edge tag and compose the incidence map with the vertex-label map. The commuting equation just proved reduces the result to source typing in C or H, respectively. For target typing the same composition argument reduces to their target typing equations.

Lean: `Net.Replacement.source_typed`

Lean: `Net.Replacement.target_typed`

Theorem 13.6 (Well-formed shallow plugging)**.** *There is a net $C[h := H] : I \to O$ with the carriers, labels, and incidence lists above. It preserves the entire ordered external interface and all graph invariants, even when h has a vertex in both its source and target lists.*

Proof. Enumerate the new internal carrier by the sum enumeration. Enumerate surviving host edges by subset filtering, and concatenate the tagged replacement-edge enumeration. Supply each of the distinctness, uniqueness, coverage, and typing proofs established above as the corresponding net field. This assembles the certified graph. Classical filtering of the surviving-edge subtype accounts for the noncomputable qualifier. □

Lean: `Net.plug`

There is deliberately no claim that k is globally injective in this theorem. If a host source port equals a host target port, its replacement input and output are identified. The separate source and target injection arguments are sufficient for every net invariant.

14 Paths, renaming, and isomorphism

14.1 Paths and their behaviour under plugging

Definition 14.1 (Adjacency and path)**.** Edges e, f are adjacent when some vertex belongs to both $t(e)$ and $s(f)$. A path is a nonempty edge list: a singleton is a path; prepending e to a path starting at f gives a path when e, f are adjacent. Edges may repeat, and this notion does not specify a chosen boundary vertex at either end.

Lean: `Net.Adjacent`

Lean: `Net.Path`

For a list beginning e, f, the path predicate is equivalent to adjacency of e, f together with the path predicate for the tail starting at f. Invert the path constructor for the forward implication and apply it for the reverse one.

Lean: `Net.path_pair_iff`

Any map on edges preserving adjacency maps paths to paths. Induct on the path: singletons remain singletons, and the inductive step transports its first adjacency and then its tail path. If the edge map preserves and reflects adjacency, it preserves and reflects paths. Induct on the list: the empty list is a path on neither side, singletons are paths on both sides, and longer lists reduce to the first adjacency equivalence and the tail induction hypothesis.

Lean: `Net.Path.map`

Lean: `Net.path_map_iff`

Lemma 14.2 (Host and inserted adjacencies)**.** *Two surviving host edges are adjacent after plugging exactly when they were adjacent before plugging. Two inserted edges that were adjacent remain adjacent.*

Proof. For host reflection, pull both incidence witnesses back through their mapped lists. Equality of their host images implies equality of the original vertices by injectivity of j, giving an old shared witness. For host preservation map an old shared witness through j. For inserted preservation map the shared replacement witness through k; injectivity is unnecessary in this direction. □

Lean: `Net.Replacement.adjacent_host_iff`

Lean: `Net.Replacement.adjacent_insert`

Definition 14.3 (Non-self-loop hole)**.** The hole h is non-self-looping if no vertex belongs to both $s(h)$ and $t(h)$. This requires the two lists to be disjoint; it is stronger than distinctness within each individual list, already required of every net.

Lean: `Net.Replacement.NonSelfLoop`

For such a hole, k is globally injective. Equal internal images reduce to equal names; mixed internal/boundary cases contradict freshness. Two input images or two output images reduce to the appropriate port injection. Equality of an input image and an output image would exhibit a host vertex in both $s(h)$ and $t(h)$, contrary to non-self-looping.

Lean: `Net.Replacement.insert_injective`

Consequently inserted adjacency is reflected as well as preserved: pull the two mapped-list witnesses back and use global injectivity of k to identify them. The other implication is the unconditional preservation result above.

Lean: `Net.Replacement.adjacent_insert_iff`

Theorem 14.4 (Path preservation and reflection)**.** *A list of surviving host edges is a path after plugging if and only if its old edge list is a path. Any replacement path gives an inserted path. For a non-self-loop hole the converse holds for inserted paths too.*

Proof. For host paths, induct on the surviving-edge list. Empty and singleton lists are immediate; a longer list reduces to host adjacency equivalence and the tail hypothesis. For inserted preservation, apply path mapping to inserted adjacency preservation. For inserted reflection under the stated restriction, apply the general path equivalence lemma to inserted adjacency equivalence. □

Lean: `Net.Replacement.path_host_iff`

Lean: `Net.Replacement.path_insert`

Lean: `Net.Replacement.path_insert_iff`

14.2 Transport along carrier bijections

A bijection on internal names extends to ports by mapping only internal vertices. Its inverse laws follow by cases on a port: internal cases use the original inverse laws and boundary cases are reflexive. Mapping the identity is pointwise the identity, also by port cases; function extensionality gives the corresponding identity of functions.

Lean: `Port.bijection`

Lean: `Port.map_id`

Lean: `Port.map_id_fun`

Fix bijections $a : G_V \cong V'$ and $b : G_E \cong E'$. Reindexed sources and targets at $e' \in E'$ are the old incidence lists at $b^{-1}(e')$, mapped by the port extension of a.

Lean: `Net.Reindex.source`

Lean: `Net.Reindex.target`

Source uniqueness survives this transport: two shared mapped witnesses are equal by port injectivity, so old source uniqueness identifies their old edge indices; apply b and cancel $b \circ b^{-1}$. The target uniqueness proof uses old target uniqueness and the same cancellation.

Lean: `Net.Reindex.source_unique`

Lean: `Net.Reindex.target_unique`

For source coverage, a new source witness pulls back to an old source vertex, which is not an output; port maps preserve this property. Conversely, pull a non-output vertex backwards through a^{-1}, choose an old source edge by coverage, and map both the edge and vertex forwards. The two inverse laws prove that this is a witness for the original new vertex. Target coverage repeats this argument with non-input vertices and old target coverage.

Lean: `Net.Reindex.source_coverage`

Lean: `Net.Reindex.target_coverage`

Give an internal name w the label of $a^{-1}(w)$. Labelling an old vertex after mapping by a then agrees with its old label: the internal case cancels $a^{-1} \circ a$, and boundary labels are fixed.

Lean: `Net.Reindex.label_map`

Definition 14.5 (Reindexed graph)**.** The reindexed net uses the new carriers, inverse-transported labels, and the incidence lists just defined.

Well-formedness. Transport finite enumerations along the bijections. Distinctness follows from port injectivity, uniqueness and coverage from the preceding lemmas. For typing, compose incidence mapping with the vertex-label map, use label commutation, and apply the original typing equation. The boundary lists are unchanged. □

Lean: `Net.reindex`

14.3 Graph isomorphism and shallow context congruence

Definition 14.6 (Isomorphism)**.** An isomorphism $G \cong H$ between nets with the same signature and boundary lists is a bijection of internal vertices and a bijection of edges preserving internal labels, edge labels, and both ordered incidence lists. Its port action fixes boundary positions.

Lean: `Net.Iso`

Identity bijections and identity list maps yield a reflexive isomorphism. For the inverse isomorphism, invert both bijections. Inverse label preservation follows by applying the original equation at an inverse image and cancelling. To invert an incidence equation, map its two sides by the inverse port map and cancel the forward/inverse maps pointwise. For composition, compose carrier bijections; compose label equalities, and use successive list mapping followed by the two incidence-preservation equations.

Lean: `Net.Iso.refl`

Lean: `Net.Iso.symm`

Lean: `Net.Iso.trans`

Isomorphisms preserve and reflect adjacency: transport a shared witness forwards for preservation; for reflection pull both witnesses back from the incidence maps and identify their preimages by port injectivity. Applying the general path equivalence lemma then preserves and reflects paths, with their edge lists mapped by the edge bijection.

Lean: `Net.Iso.adjacent_iff`

Lean: `Net.Iso.path_iff`

The original graph is isomorphic to its reindexing. Use the chosen carrier bijections; inverse cancellation proves label preservation and each incidence identity directly from the definition of reindexing.

Lean: `Net.reindexIso`

Suppose $f : H \cong K$ and both graphs have the interface of host hole h. The intended isomorphism between $C[h := H]$ and $C[h := K]$ is the identity on host internal names and f on replacement internal names, and likewise the identity on surviving host edges and f on inserted edges. Sum bijections supply their inverse laws.

Lean: `Net.PlugIso.vertices`

Lean: `Net.PlugIso.edges`

The resulting port map commutes with host embedding: check the three port constructors, all of which are reflexive. It commutes with insertion too: internal vertices use the replacement part of the sum map, and boundary vertices reduce to the host-embedding equation at the corresponding hole port.

Lean: `Net.PlugIso.map_host`

Lean: `Net.PlugIso.map_insert`

Theorem 14.7 (Congruence of shallow plugging)**.** *For every typed host and hole, $H \cong K$ implies $C[h := H] \cong C[h := K]$. No non-self-loop hypothesis is required.*

Proof. Use the sum bijections above. For each label equation split the carrier tag: host labels are unchanged and inserted labels use f's corresponding equation. For a host source list, commute the new port map past host embedding pointwise. For an inserted source list, commute it past insertion and then use source preservation by f. Repeat with target lists and target preservation. These arguments prove all isomorphism fields without reflecting through insertion. □

Lean: `Net.plugIso`

Here edge labels are preserved by literal equality. Section 24 defines a recursive relation that also identifies independently renamed box contents and proves the corresponding congruence at arbitrary addressed depth.

15 Concrete uniqueness of operation paths

Choose a predicate P on operation labels in a net G. This section first proves a generic graph theorem; the next section shows that the single-source condition is automatic for the typed operation labels of the linear hierarchy.

Definition 15.1 (Unary operations and traversal)**.** Selected operations are unary when each edge whose label satisfies P has a source list of length one. An operation step $s \rightsquigarrow t$ chooses such an edge e with $s \in s(e)$ and $t \in t(e)$.

Lean: `Net.UnaryOperations`

Lean: `Net.OperationStep`

Reverse traversal is the machine whose transition from t to s is $s \rightsquigarrow t$ and whose successful final states are boundary inputs. Inputs are normal: any reverse step from an input would put that input in a target list, contradicting target exclusion.

Lean: `Net.reverseOperations`

Lemma 15.2 (Determinism derived from incidence)**.** *If selected operations are unary, reverse operation traversal is deterministic.*

Proof. Two reverse steps from the same vertex select edges both targeting that vertex. Target uniqueness identifies the edges. Their source list has length one, so both predecessor vertices equal its only entry and hence equal one another. There is no additional determinism premise. □

Lean: `Net.reverseOperations_deterministic`

Definition 15.3 (Edge-recording trace)**.** A reverse operation trace from v to w records its actual edge list. The empty trace starts and ends at the same vertex. A nonempty trace first chooses an operation edge targeting the current vertex and a vertex in its source, then continues recursively from that source vertex.

Lean: `Net.OperationTrace`

A trace with edge list l yields exactly $|l|$ steps in the reverse machine. Induct on its construction: the empty trace is a zero-step execution and each trace constructor supplies the first step of the counted execution.

Lean: `Net.OperationTrace.steps`

Theorem 15.4 (Exact uniqueness to inputs)**.** *Under the unary condition, two reverse traces from the same vertex to inputs have equal edge lists and equal ending input vertices.*

Proof. Induct on the first trace. If it is empty, the starting vertex is an input; the second trace cannot be nonempty, since no input belongs to a target list. Thus it too is empty and both conclusions follow. If the first trace is nonempty, the second cannot be empty, for the same target-exclusion reason. Their first edges are equal by target uniqueness. Their next vertices agree by the determinism lemma. Apply the induction hypothesis to their tails and prefix the common edge to the resulting list equality. □

Lean: `Net.OperationTrace.unique_to_inputs`

A forward operation path from s to t with edge list l means a reverse trace from t to s with list reverse(l). Empty paths are allowed in this endpoint-sensitive notion, unlike the nonempty edge-only path predicate of the preceding section.

Lean: `Net.OperationPath`

Two forward operation paths from inputs to the same vertex therefore have equal forward edge lists and equal starting input indices. Apply exact trace uniqueness, use injectivity of list reversal, and then injectivity of the input constructor.

Lean: `Net.operationPath_unique_from_inputs`

Theorem 15.5 (No positive cycle on an input-rooted operation path)**.** *If a vertex v is reached by a forward operation path from an input, every reverse operation trace from v back to v has an empty edge list.*

Proof. The input-rooted path gives a reverse execution from v to a successful input. Prefix that execution by the proposed cycle to obtain another successful execution from v to the same input. Deterministic terminal uniqueness identifies their lengths. Thus $|cycle| + |path| = |path|$, so the cycle has length zero and is empty. □

Lean: `Net.no_operation_cycle_on_input_path`

This is a statement about cycles at input-reachable vertices, not a global acyclicity restriction on nets. The self-loop example below is a valid net with a cycle and no boundary inputs.

16 Typed boxes, deep occurrences, and contexts

16.1 Relabelling without changing incidence

A signature-preserving map f of operation alphabets relabels a net by replacing each edge label o with $f(o)$. All carriers, enumerations, vertex labels, and incidence data are retained. Hence their invariant proofs are retained too. For typing, compose the old typing equality with the inverse of the stipulated signature equality for $f(o)$. This yields the new signature list at that edge.

Lean: `Net.relabel`

16.2 The linear wire and label discipline

Definition 16.1 (Wire types, operation arities, and holes)**.** A wire type is either a term wire $*$ or a thunk wire $T^n(*)$, carrying a natural number n of bound variables. An operation specification assigns each operation an argument count $a(o)$ and an ordered list of binder counts $b(o)$. A hole label contains a natural-number name together with arbitrary input and output wire lists. Equality of hole labels includes these lists as well as the name.

Lean: `Linear.Wire`

Lean: `Linear.Operations`

Lean: `Linear.Hole`

Primitive edge labels are operations, variables, focus labels from the abstract three-mode type, or typed holes. This introduces concrete focus *labels*; it does not yet define the UAM transition relation.

Lean: `Linear.Primitive`

Their signature is as follows. Every non-hole primitive has one term input. Operation o has $a(o)$ term outputs followed by the thunk outputs specified in $b(o)$. A variable or focus has one term output. A hole uses precisely the input and output lists stored in its label. Ordering is part of each of these definitions.

Lean: `Linear.primitiveSignature`

Definition 16.2 (Alphabets and typed boxes)**.** An alphabet is a label type and a signature on it. A box over alphabet A stores numbers n, m and a body net over A of type

$$* \longrightarrow *^{n+m}.$$

The corresponding outer box edge will have type $T^n(*) \longrightarrow *^m$. Thus body boundary and outer boundary differ: n body outputs account for binding, and m for exposed free variables.

Lean: `Linear.Alphabet`

Lean: `Linear.Box`

All alphabet label types live in one fixed higher universe, while graph carriers remain small types. The base alphabet contains just primitives, raised into that universe by a universe lift; its signature reads the underlying primitive. The successor alphabet is the disjoint union of

primitives and boxes over the previous alphabet. Its signature uses the primitive signature on the left and the stated external box interface on the right.

Lean: `Linear.baseAlphabet`

Lean: `Linear.nextAlphabet`

Recursively, depth zero uses the base alphabet and depth $d + 1$ uses the successor of the depth-d alphabet. A hypernet at depth bound d is a net over that alphabet. The bound limits possible nesting; it need not be the actual minimum depth of the particular graph.

Lean: `Linear.alphabet`

Lean: `Linear.Hypernet`

The operation predicate selects primitive operation labels only. At depth zero it inspects the universe-lifted primitive; at a successor depth it accepts only the primitive-operation summand, excluding boxes and all other primitive kinds.

Lean: `Linear.IsOperation`

Selected operation edges have exactly one source. Rewrite the incidence length using signature typing, split on the depth and label constructors, discard labels excluded by the predicate, and compute the operation input list $[*]$.

Lean: `Linear.unary_operations`

It follows that two operation paths from inputs to the same vertex have identical edge lists and input indices in every typed hypernet. Instantiate the generic operation-path uniqueness theorem with the preceding derived unary proof. This is the graph-theoretic uniqueness conclusion associated with the paper's Lemma 22 for this representation; neither determinism nor a locality field is assumed.

Lean: `Linear.operation_path_unique`

16.3 Box construction and promotion of the depth bound

The box constructor takes a body with the specified $n + m$ output interface and forms the corolla of its box label in the successor alphabet. Corolla well-formedness proves all outer graph conditions. The result has exactly one outer box edge and the external type $T^n(*) \to *^m$.

Lean: `Linear.box`

Promotion raises the available depth bound by one without changing the outer graph. At depth zero, send a lifted primitive to the primitive summand at depth one. At a successor depth, keep primitives and recursively promote each box body, retaining its bound/free metadata. These maps preserve signatures because neither primitive signatures nor external box arities change. Apply the generic relabel construction at each recursive step.

Lean: `Linear.raise`

Outer sources and targets are unchanged by promotion. The code records these as heterogeneous equalities: the dependent graph types are indexed by different depths, although after unfolding either defining case the incidence functions are definitionally the same. Splitting on depth proves both equalities by reflexivity.

Lean: `Linear.raise_source`

Lean: `Linear.raise_target`

16.4 Finite occurrence addresses at arbitrary depth

Definition 16.3 (All edge occurrences)**.** At depth zero, an occurrence is a shallow edge. At depth $d + 1$, an occurrence is either a shallow edge or a pair (e, a) where e is a box edge and a an occurrence in its body. A primitive edge has the empty type of child occurrences.

Lean: `Linear.AllEdge`

The parent edge is part of every deep address. In particular identical bodies placed at different parent edges do not identify their child occurrences.

Theorem 16.4 (Finiteness of all occurrences)**.** *Every finite-depth hypernet has an explicit finite enumeration of its shallow and deep edge occurrences.*

Proof. Recurse on the depth bound. At zero use the graph's edge enumeration. At a successor bound, enumerate shallow edges in the left summand. For the right summand enumerate the dependent sum over outer edges: a primitive contributes an empty enumeration, and a box contributes the recursively obtained enumeration of its body. The sum and dependent-sum enumeration constructions already prove completeness of this list. Recursion decreases the depth at every box descent. □

Lean: `Linear.allEdgesFinite`

Every given occurrence belongs to this enumeration, by its completeness field. This is the finiteness content of Appendix A, Lemma 19, for the typed hierarchy. The list need not be duplicate-free, since completeness suffices here.

Lean: `Linear.every_edge_enumerated`

16.5 Hole lookup and the context condition

A primitive hole lookup returns its stored label for a hole constructor and no label otherwise. Full occurrence lookup uses that function on a shallow primitive, returns no hole at a shallow box edge, and recursively looks inside a deep box address. A purported child occurrence under a primitive has empty type and is eliminated. Dependent transports in Lean express that the selected label really is the box whose body is being searched.

Lean: `Linear.primitiveHole`

Lean: `Linear.holeAt`

Definition 16.5 (Unique holes and contexts)**.** A hypernet has unique holes if, for each complete typed hole label h, any two occurrences whose lookup returns h are equal. A context is a hypernet together with this property. This permits multiple different hole labels and also permits no holes; it is an at-most-once condition for each label over all depths.

Lean: `Linear.UniqueHoles`

Lean: `Linear.Context`

Lemma 16.6 (Boxing preserves unique holes)**.** *If a body has unique holes, its one-edge box has unique holes.*

Proof. A shallow occurrence in the outer box is not a hole, so either proposed hole occurrence must be deep. The outer graph has only one edge; therefore their parent indices agree. Their lookups are the body lookups, whose occurrences agree by the body's uniqueness assumption. Tagging these equal child addresses with their common parent gives equality of the original occurrences. □

Lean: `Linear.box_unique_holes`

Boxing a context now means boxing its underlying graph and supplying this lemma as the context certificate. Shallow plugging at any fixed hierarchy depth is just the generic typed net plugging construction with that depth's alphabet; its interface constraints are checked by the type of its replacement argument.

Lean: `Linear.boxContext`

Lean: `Linear.plug`

This last operation returns a hypernet. The later context-calculus development now proves that its result is a context under explicit freshness conditions (Section 20), including a sharp necessary-and-sufficient condition for shallow plugging. Section 45 now proves the representation equivalence with actual graph labels by fresh vertex naming.

17 Addressed deep replacement and concrete locality

17.1 Changing one box body

A per-edge signature-preserving relabelling generalizes alphabet relabelling: it assigns a new label directly to each edge, rather than requiring the assignment to factor through its old label. All carriers, incidence, and vertex labels stay unchanged. The two new typing equations follow by composing each old typing proof with the inverse of the specified per-edge signature equality.

Lean: `Net.relabelEdges`

Definition 17.1 (Replacement of a box label)**.** Suppose outer edge e is labelled by a box (n, m, B). Given a new body B' of the same body type $* \to *^{n+m}$ and depth bound, change just the label at e to (n, m, B'), retaining every other edge label.

Well-formedness. Use classical equality testing on outer edges to select e. At e the old and new labels have the same external input $[T^n(*)]$ and output $*^m$; this follows from the given equality identifying the old label as the box. At every other edge the label is unchanged. These two cases prove the per-edge signature conditions for relabelling, which supplies the certified net. □

Lean: `Linear.replaceBox`

Selection uses an edge occurrence, not equality of box contents. Equal bodies at other edges therefore do not cause simultaneous replacement.

17.2 Typed sites and replacement at a site

Definition 17.2 (Site)**.** A site in an outer hypernet G carries a local depth bound and two wire-label lists J, K. It is either a shallow edge of G, in which case J, K are exactly that edge's signature lists, or a box edge with a proof identifying its box label together with a recursively chosen site in its body. In the latter case the site's local depth and interface are inherited from the child site.

Lean: `Linear.Site`

The indices prevent supplying a replacement of the wrong interface or local depth. A site may select an arbitrary edge, not only an edge labelled as a hole; the separate unique-hole context condition is not assumed here.

Definition 17.3 (Deep plugging)**.** To replace at a shallow site use shallow net plugging. To replace at a site inside a box, recursively replace in its body and use box-label replacement to put that modified body back at the chosen outer edge. The recursion is on the site's finite constructor structure, and the result retains the outer depth bound and boundary interface.

Lean: `Linear.plugAt`

At a shallow site this definition agrees with shallow plugging by reflexivity: both sides unfold to the same term.

Lean: `Linear.plugAt_shallow`

Theorem 17.4 (Locality at every enclosing box)**.** *Replacement strictly inside a selected box leaves all outer source lists and all outer target lists unchanged. It also leaves the label of every other outer edge unchanged.*

Proof. The inside branch changes only an edge label using per-edge relabelling. The source and target fields of that construction are definitionally those of the old graph, so their equalities are reflexive, independent of what the recursive replacement did inside. For another outer edge $f \neq e$, the conditional defining box-label replacement chooses the old-label branch, proving the label equation. Repeated descent applies this fact at each enclosing layer. □

Lean: `Linear.plugAt_inside_source`

Lean: `Linear.plugAt_inside_target`

Lean: `Linear.plugAt_inside_other_label`

These are unconditional, concrete graph locality results about the defined replacement operation. They do not yet prove UAM traversal locality, refocusing, or the specimen interface laws: no concrete machine transition relation has been constructed. The subsequent provenance development proves unique-hole context closure for this concrete replacement at any depth. Section 24 proves recursive isomorphism congruence for this replacement. Associativity of context plugging remains a separate task.

18 Concrete graph examples and regression proofs

18.1 Self-loop gluing and the necessary reflection restriction

Use a singleton wire-label set and a singleton operation alphabet, whose only operation has one input and one output of that label. The graph named loop has one internal vertex, one edge, no outer boundary, and that vertex as both the edge's source and target.

Lean: `Examples.unary`

Lean: `Examples.loop`

Its finite carriers are both Fin(1). Singleton incidence lists are distinct; all edges are equal since their indices are bounded by one. The only internal vertex is incident on both sides, providing coverage, while boundary cases have empty finite-index types. Label typing is reflexive. Thus this is a well-formed net despite its cycle. Let wireEdge be the corolla of the same unary operation, with its two distinct boundary vertices.

Lean: `Examples.wireEdge`

The corolla is not adjacent to itself: a shared target/source witness would have to be both its output constructor and its input constructor, an impossible constructor equality.

Lean: `Examples.corolla_not_self_adjacent`

When the corolla replaces the loop's sole edge, its input and output images are equal. Both are, by definition, the same old internal vertex, because the host hole's source and target port coincide. Hence the inserted edge is adjacent to itself: use that old vertex as the witness and compute the mapped singleton source and target lists.

Lean: `Examples.inserted_boundary_identified`

Lean: Examples.inserted_self_adjacent

Consequently the inserted two-entry list $[e, e]$ is a path, while $[e, e]$ is not a path in the original corolla. The positive part uses its new self-adjacency and a singleton tail path. The negative part extracts its first adjacency and contradicts the corolla result. This proves that unconditional reflection of inserted paths would be false; the non-self-loop hypothesis cannot simply be omitted from that theorem.

Lean: Examples.unconditional_path_reflection_fails

The original loop itself has the two-entry path $[e, e]$, using its sole internal vertex as the shared witness. This verifies that the graph representation has not silently imposed global acyclicity.

Lean: Examples.loop_is_cyclic

The finite ordinal with one element is bijective to the unit type: send its index to the sole unit element and send unit back to zero. The inverse laws follow respectively from the ordinal bound and unit case analysis. An anonymous example reindexes the corolla's edge carrier along this bijection and applies shallow plugging congruence. It obtains isomorphic filled loops despite the different replacement-edge carrier types, testing actual renaming rather than reflexivity alone.

Lean: Examples.oneToUnit

Lean: Examples.example (line 89)

18.2 Values, boxes, and distinguished occurrence addresses

For the linear examples, take two operation labels, value and other-value. Give both zero term arguments and no thunk arguments; each therefore has signature $* \to []$. The graph value is the corolla of the first operation at depth zero.

Lean: Linear.Examples.Op

Lean: Linear.Examples.operations

Lean: Linear.Examples.value

Boxing it with zero bound and free variables yields a closed box of type $T^0(*) \to []$ at depth one. A further example constructs the analogous graph at depth bound two by putting a primitive value at depth one inside a box. This is a larger *available bound*, not a claim that the example has two actually nested boxes.

Lean: Linear.Examples.boxedValue

Lean: Linear.Examples.higherBoxedValue

An anonymous proof verifies that any occurrence in the depth-one boxed value belongs to the recursive occurrence enumeration, by the general completeness theorem. The outer box edge is represented by a shallow left-tagged occurrence; the value edge in its body is a right-tagged pair of parent edge and child edge.

Lean: Linear.Examples.example (line 117)

Lean: Linear.Examples.outerOccurrence

Lean: Linear.Examples.innerOccurrence

These two addresses are unequal. If equal, applying the sum-tag test would give true = false,

which is impossible. This checks separation of outer and inner occurrences even though each individual edge carrier is singleton.

Lean: `Linear.Examples.outer_ne_inner`

The other-value corolla has exactly the same interface as the value corolla. A site inside the boxed value descends through its only box edge and selects the value edge in its body. Replacing at this site with other-value defines replacedInside.

Lean: `Linear.Examples.otherValue`

Lean: `Linear.Examples.insideValue`

Lean: `Linear.Examples.replacedInside`

Its outer source and target functions equal those of the original boxed value; instantiate the two deep-locality theorems at the selected site. Its sole outer edge label is the box with body value$[e :=$ otherValue$]$. For this label equation, unfold the site and replacement, simplify the equality test selecting the sole outer edge, and use reflexivity for the resulting body. The outer box is retained rather than replaced by the value operation itself.

Lean: `Linear.Examples.deep_replacement_source_unchanged`

Lean: `Linear.Examples.deep_replacement_target_unchanged`

Lean: `Linear.Examples.deep_replacement_body`

18.3 An input-rooted operation path

Give the single unit operation one term argument and no binders. Its corolla argument has type $* \to *$. The input, its only edge, and the output form a forward operation path: read this as the one-step reverse trace, check membership in the two singleton incidence lists, and end with an empty trace.

Lean: `Linear.Examples.argumentOperations`

Lean: `Linear.Examples.argument`

Lean: `Linear.Examples.argument_path`

Any reverse operation trace from its output back to that output has empty edge list. Apply the no-cycle theorem to the constructed input-rooted path, deriving the required unary condition from the typed signature. This exercises the connection between concrete graph typing and generic machine terminal uniqueness.

Lean: `Linear.Examples.no_cycle_after_argument`

18.4 Filling a hole inside a certified context

Choose the typed hole label with name zero, one term input, and no outputs. Its corolla singleHole is a depth-zero hypernet. It has unique holes because any two edge occurrences are equal: both are elements of Fin(1), so their underlying indices are zero. Pairing this graph with the proof constructs a context.

Lean: `Linear.Examples.holeLabel`

Lean: `Linear.Examples.singleHole`

Lean: `Linear.Examples.singleHole_unique`

Lean: `Linear.Examples.singleContext`

Box the graph with zero bound and free variables to obtain boxedHole, and box the certified context to obtain boxedContext. The unique-hole certificate in the latter is supplied by the general boxing theorem. Looking up the deep address consisting of the only outer edge and the only body edge returns the chosen hole label by definition.

Lean: `Linear.Examples.boxedHole`

Lean: `Linear.Examples.boxedContext`

Lean: `Linear.Examples.hole_is_deep`

The corresponding site descends into that box and chooses its shallow hole edge. Filling this site with the value corolla produces filledBox. Its outer label is the same zero-bound, zero-free box, now containing singleHole$[e := \mathsf{value}]$; unfold deep replacement and simplify the selected-edge equality test to prove this equation.

Lean: `Linear.Examples.boxedHoleSite`

Lean: `Linear.Examples.filledBox`

Lean: `Linear.Examples.filled_box_body`

Finally every edge of this filled body has no primitive hole label. Split on its edge tag. An old surviving edge would be an element of the singleton old carrier different from its only element, contradicting the surviving-edge certificate. An inserted value edge is an operation, whose primitive hole lookup is definitionally empty. Thus this example proves removal of the old hole, not merely the well-typedness of the filled graph.

Lean: `Linear.Examples.filled_body_no_holes`

19 Occurrence fibres and transport of hole observations

The next milestone completes context closure under explicit hole freshness, including replacement at arbitrary box depth. Its proofs need to distinguish individual occurrences, rather than merely sets of labels. This section provides coordinates for those occurrences without changing the existing hypernet type.

19.1 Coordinates local to an outer edge

Definition 19.1 (Children and occurrence fibre)**.** For a label l at depth zero, its child occurrence type is empty. At depth $d+1$, a primitive label again has no children; a box label with body B has child type AllEdge(B). Define the fibre of l to be

$$F_d(l) = \{\bullet\} + \mathrm{Children}_d(l).$$

The left element is the edge itself; a right element is an occurrence below it.

Lean: `Linear.Children`

Lean: `Linear.Fiber`

The shallow hole observation reads the primitive hole label at depth zero or in the primitive summand at positive depth, and returns no hole on a box label. The child observation eliminates empty child types and otherwise uses the body's existing hole lookup. The fibre observation combines these functions: left elements use shallow lookup and right elements use child lookup.

Lean: `Linear.shallowHole`

Lean: `Linear.childHole`

Lean: `Linear.fiberHole`

An occurrence of a graph G in fibre coordinates is a dependent pair

$$(e, x), \qquad e \in G_E, \quad x \in F_d(\lambda_G(e)).$$

The first component is always an outer edge, even when x goes deep into a box. This representation depends on edge labels and occurrence addresses, not on vertex incidence or a particular enumeration of the carriers.

Lean: `Linear.Occurrences`

Lemma 19.2 (Occurrence coordinates)**.** *The existing occurrence type* AllEdge(G) *is bijective to the fibre-coordinate occurrence type.*

Proof. At depth zero, send an edge e to $(e, \mathsf{left}(\bullet))$; the inverse forgets that singleton component. A right component would belong to the empty type and is impossible. At positive depth, send a shallow edge to the same left-tagged pair, and a deep address (e, a) to $(e, \mathsf{right}(a))$. The inverse removes the fibre tag to reconstruct the corresponding shallow or deep address. Both inverse laws follow by cases on the tags and uniqueness of the unit value. □

Lean: `Linear.occurrenceBijection`

Define a graph's occurrence observation at (e, x) by applying the fibre observation to $\lambda_G(e)$ and x.

Lean: `Linear.occurrenceHole`

The bijection preserves hole observations. At depth zero this is definitional. For a positive-depth shallow occurrence, split the label into primitive or box and compute both lookups. For a deep occurrence, both lookups read the same body address: exposing the label cases makes the equality reflexive. The original hierarchy lookup now uses a directly dependent reader, avoiding unnecessary proof-dependent casts while retaining the same definition by label cases.

Lean: `Linear.occurrenceBijection_hole`

Lemma 19.3 (The context condition in fibre coordinates)**.** *A graph has unique holes exactly when any two fibre-coordinate occurrences observing the same hole label are equal.*

Proof. For the forward direction, apply unique holes to the inverse images of the two occurrences. Observation preservation supplies their lookup hypotheses. Map the resulting equality forwards and cancel the inverse pairs. Conversely, map the two original addresses forwards, use fibre-coordinate uniqueness and observation preservation, and conclude by injectivity of the bijection. □

Lean: `Linear.uniqueHoles_iff_occurrences`

19.2 Transport along equality and edge maps

If labels l, l' are equal, transport of $x \in F_d(l)$ along that equality gives an element of $F_d(l')$ with the same hole observation. Such transport is injective. For each assertion, eliminate the label equality; transport becomes the identity, so the observation equation is reflexive and injectivity is the given equality of transported elements.

Lean: `Linear.fiberHole_transport`

Lean: `Linear.fiber_transport_injective`

A shallow edge map $f : G_E \to H_E$ with equations $\lambda_G(e) = \lambda_H(f(e))$ maps all occurrences by

$$(e, x) \longmapsto (f(e), \mathrm{transport}(x)).$$

The descendant address is preserved through the label equality; it is not searched for again by comparing box contents.

Lean: `Linear.mapOccurrences`

This map preserves hole observation by the equality-transport observation lemma. If f is injective, the occurrence map is injective: equality of mapped pairs first identifies their edge indices by injectivity of f. After substituting that equality, equality of their transported second components implies equality of the original components by transport injectivity. The dependent pairs are therefore equal.

Lean: `Linear.mapOccurrences_hole`

Lean: `Linear.mapOccurrences_injective`

20 Hole inventories and sharp shallow context closure

All hole-label comparisons in this section compare complete typed labels: the name and both interface lists. The occurrence quantifiers include every box depth.

20.1 Hole inventories, absence, and isomorphism

Definition 20.1 (Presence, absence, and disjointness)**.** Write $\mathsf{HasHole}(G, h)$ when some occurrence of G looks up to h. Write $\mathsf{HoleFree}(G)$ when no label has such an occurrence. Two graphs have disjoint holes when no complete hole label occurs in both. The graphs may have different external interfaces and depth bounds.

Lean: `Linear.HasHole`

Lean: `Linear.HoleFree`

Lean: `Linear.HolesDisjoint`

Hole presence is equivalently witnessed by a fibre-coordinate occurrence with the same observation. Transport a witness by the occurrence bijection for one direction and by its inverse for the other, using observation preservation and the inverse equation.

Lean: `Linear.hasHole_iff_occurrences`

A hole-free graph has unique holes vacuously: the first purported hole occurrence already contradicts absence. Disjointness is symmetric by exchanging its two presence premises. Any graph is disjoint from a hole-free graph because a hole in the second graph would contradict its absence property.

Lean: `Linear.holeFree_unique`

Lean: `Linear.HolesDisjoint.symm`

Lean: `Linear.holeFree_disjoint`

Boxing neither creates nor removes a hole label from the inventory of the body. A shallow box occurrence cannot be a hole. A deep occurrence is exactly an address in its body with the same lookup, so it supplies a body witness. Conversely, tag a body witness with the box's only outer edge. Consequently a hole-free body gives a hole-free box: transfer any proposed hole witness to the body and contradict its absence property.

Lean: `Linear.box_hasHole_iff`

Lean: `Linear.box_holeFree`

A label-preserving edge map preserves hole presence: map the fibre-coordinate witness using the occurrence map and its observation equation. An injective such edge map reflects unique holes: map two proposed equal-label occurrences into the target, apply target uniqueness, and use occurrence-map injectivity.

Lean: `Linear.hasHole_map`

Lean: `Linear.uniqueHoles_of_embedding`

It follows that graph isomorphisms preserve and reflect hole inventories, and preserve and reflect the context condition. Apply presence preservation to the forward and inverse edge

bijections for the first equivalence. Apply reflection of uniqueness to the inverse bijection for forward preservation and to the forward bijection for reflection. Label preservation is supplied by the isomorphism equations with the appropriate equality direction.

Lean: `Linear.iso_hasHole_iff`

Lean: `Linear.iso_uniqueHoles_iff`

These results use the literal-label isomorphism notion. Section 24 extends inventory and context invariance to independently renamed box contents.

20.2 Provenance and the abstract collision argument

Definition 20.2 (Provenance certificate)**.** For result R, host C, and replacement H, a provenance certificate is an injective function

$$p : \mathrm{Occurrences}(R) \longrightarrow \mathrm{Occurrences}(C) + \mathrm{Occurrences}(H)$$

that preserves hole observation in the chosen summand. Its left values identify retained host occurrences and its right values identify inserted occurrences. Different result occurrences must have different tagged origins.

Lean: `Linear.Provenance`

This is a reusable intermediate interface. Unlike the earlier specimen laws, the provenance certificates required for actual plugging are constructed below from the concrete operations, rather than left as unproved assumptions.

Every result hole label belongs to the host or replacement inventory. Choose its fibre-coordinate witness, inspect the origin tag, and use preservation to obtain a witness in the indicated graph.

Lean: `Linear.Provenance.holes_subset`

A provenance certificate is *compatible* when no inserted hole label is observed at any retained host origin. Formally, if $p(a) = \mathsf{left}(c)$ and c observes h, then H does not contain h. Host occurrences outside the image of the left branch are irrelevant to this condition.

Lean: `Linear.Provenance.Compatible`

Theorem 20.3 (Unique holes from compatible provenance)**.** *If C, H have unique holes and a provenance certificate for R, C, H is compatible, then R has unique holes.*

Proof. Take two result occurrences observing the same h and inspect their origins. If both origins are retained, host uniqueness identifies their host addresses. If both are inserted, replacement uniqueness identifies their replacement addresses. In either case the tagged origins agree and provenance injectivity identifies the result occurrences. If one origin is retained and one inserted, observation preservation gives a retained host observation of h and a hole witness for h in H, contradicting compatibility. This covers all four tag combinations and proves fibre-coordinate uniqueness, hence the original condition. □

Lean: `Linear.Provenance.unique_of_compatible`

Full disjointness of the host and replacement inventories implies compatibility: any retained host occurrence is in particular a host occurrence. Substituting this fact into the preceding theorem gives unique holes under full disjointness.

Lean: `Linear.Provenance.unique`

20.3 Concrete shallow provenance and a first context operation

The provenance map for $C[e := H]$ reads its outer edge tag. A surviving host edge with fibre address x maps to the corresponding old host edge with x; an inserted edge maps to its replacement edge with x. In either case the edge label, and hence the fibre, is definitionally unchanged.

Injectivity and observation preservation. Mixed edge tags give different origin tags and cannot agree. Two retained origins first identify the old edge values. Their surviving-edge subtype values then agree because their inequality certificates are propositions; equality of the dependent second components identifies their fibre addresses. Two inserted origins are already the same replacement pair. In every case fibre observation is unchanged by definition. Thus these data construct a provenance certificate for any shallow replacement, including when the removed edge is a box. □

Lean: `Linear.shallowProvenance`

The general provenance theorem now proves context closure for shallow plugging under full hole disjointness. A context-valued operation packages the filled graph with this proof, taking certified host and replacement contexts and a proof of disjointness. All graph incidence invariants still come from the earlier certified net plugging construction.

Lean: `Linear.plug_uniqueHoles`

Lean: `Linear.Context.plug`

20.4 Exact inventory and the sharp freshness condition

Definition 20.4 (Retained hole label)**.** A retained hole label for removal of edge e in C is a hole observation in the fibre of an outer edge $f \neq e$. This includes deep occurrences below f. It excludes both the removed edge and every occurrence inside its label.

Lean: `Linear.RetainedHole`

Theorem 20.5 (Exact shallow hole inventory)**.** *For each label h,*

$$\mathsf{HasHole}(C[e := H], h) \iff \mathsf{RetainedHole}(C, e, h) \lor \mathsf{HasHole}(H, h).$$

Proof. Express presence using fibre coordinates. A result occurrence has an outer edge tag that is either a surviving host edge or an inserted edge. The first case supplies exactly the retained witness; the second supplies a replacement witness. Conversely, a retained witness gives a result occurrence at the left edge tag, using its inequality proof to construct the surviving-edge subtype. A replacement witness gives a result occurrence at the right tag. Their hole observations are unchanged by definition. □

Lean: `Linear.plug_hasHole_iff`

Define sharp plugging freshness to mean that no retained host hole label occurs in the replacement. This is weaker than requiring the replacement to be disjoint from the entire original host: labels confined to the removed edge's region may be reused.

Lean: `Linear.PlugFresh`

Sharp freshness implies uniqueness of the filled graph when both inputs have unique holes. Apply the compatible-provenance theorem to shallow provenance. Any result occurrence

with a retained origin must have a surviving host edge tag; its fibre is the retained witness required by sharp freshness. An inserted edge cannot have a retained origin, so the other case is impossible.

Lean: `Linear.plug_uniqueHoles_of_fresh`

Theorem 20.6 (Sharp freshness is necessary)**.** *If $C[e := H]$ has unique holes, sharp plugging freshness holds.*

Proof. Suppose a retained host fibre observes h and H contains h. Form the two result occurrences from their witnesses as in the inventory theorem. Result uniqueness identifies them. Equality of their first components would identify a surviving host edge tag with an inserted edge tag, which is impossible. □

Lean: `Linear.plug_fresh_necessary`

Consequently, for two contexts the filled graph has unique holes if and only if sharp plugging freshness holds. Necessity is the preceding theorem, which did not require uniqueness of either input; sufficiency uses the two input certificates.

Lean: `Linear.plug_uniqueHoles_iff_fresh`

20.5 Consumption of a selected hole and certified composition

In a context, if the removed edge's shallow lookup is h, that label has no retained occurrence elsewhere. Otherwise the retained occurrence and the selected edge's shallow occurrence would be two occurrences of h. Context uniqueness identifies their pairs, so their outer edges agree, contradicting the retained edge's inequality certificate.

Lean: `Linear.removed_hole_not_retained`

It follows that the consumed label h occurs in $C[e := H]$ exactly when it occurs in H. Use the exact inventory theorem and exclude its retained alternative by the preceding result. Thus consumption removes the old occurrence; it does not forbid the replacement from deliberately reintroducing that label.

Lean: `Linear.plug_consumed_hole_iff`

The operation Context.plugFresh packages shallow composition with this sharp freshness premise. Its result graph is ordinary net plugging, and its context certificate is the sharp sufficiency proof. It supports arbitrary finite depth and arbitrary selected outer edges, rather than requiring that the selected edge's label itself be a hole.

Lean: `Linear.Context.plugFresh`

21 Provenance and context closure at arbitrary box depth

The shallow origin map alone is insufficient for deep replacement: an enclosing box edge stays in place while occurrences in its body may change. We construct provenance through each selected enclosing box and then induct on the typed site.

21.1 Local provenance for a changed label

Definition 21.1 (Fibre provenance)**.** For old and new labels l, l' at the same outer depth and a replacement H, a fibre provenance certificate is an injective map

$$F_d(l') \longrightarrow F_d(l) + \mathrm{Occurrences}(H)$$

preserving the hole observation in the indicated summand. It accounts for changing one label while retaining or inserting the occurrences within that label.

Lean: `Linear.FiberProvenance`

An unchanged label has the left inclusion as its origin map, which is injective and preserves observation by reflexivity. If the new label merely has a proof of equality to the old label, first transport its fibre address along that equality and then use the left inclusion. Injectivity follows from transport injectivity; the hole equation is the inverse direction of the transport observation equation.

Lean: `Linear.FiberProvenance.unchanged`

Lean: `Linear.FiberProvenance.retained`

Lemma 21.2 (Assembly of local provenance)**.** *Suppose a signature-preserving per-edge relabelling of G has a fibre provenance certificate at every edge. If only a single designated edge can produce inserted origins, these certificates assemble into provenance for the relabelled graph relative to G, H.*

Proof. At a result pair (e, x) inspect its fibre origin. A retained origin y gives the old graph occurrence (e, y); an inserted origin is already an occurrence of H. Observation preservation is precisely the local certificate's equation. For injectivity, consider the two output tags. Two retained origins identify their outer edges and then their local retained addresses; local injectivity identifies the original fibre addresses. Mixed output tags cannot agree. If both origins are inserted, each outer edge must be the designated one: otherwise the hypothesis that all its origins are retained contradicts its inserted tag. With the outer edges identified, equality of inserted addresses and local injectivity again identify the fibre addresses. Thus the global origin map is injective in every case. □

Lean: `Linear.relabelProvenance`

The single-designated-edge restriction prevents two different outer edges from claiming the same inserted occurrence. The lemma proves this separation; it does not assume global injectivity as an extra graph law.

21.2 Lifting a body's provenance through one box

Suppose the old and new bodies B, B' have the same bound/free interface and a provenance certificate p for B', B, H. Their outer box labels differ only in body content. Construct fibre provenance as follows. The shallow occurrence of the box is retained as the old box's shallow occurrence. A new child address is converted to the fibre-coordinate address of B' and sent through p. Retained body origins are converted back to body addresses and tagged as old box children; inserted origins remain occurrences of H.

Injectivity and observation preservation. A retained shallow box occurrence is distinct from every retained child and every inserted origin by its tags. For two children, inspect the two body-origin tags. Mixed tags cannot give equal lifted origins. If both are retained, equality of the lifted children, followed by the old body's occurrence bijection, identifies the retained body origins. If both are inserted, their equality is already equality of body origins. In both cases body-provenance injectivity and the new body's occurrence-bijection injectivity identify the original children. For observations, the shallow box lookup is empty on both sides. A child's lookup agrees with its body-coordinate observation by the bijection lemma; body provenance gives the intermediate equality. In the retained case the inverse bijection of the old body supplies the remaining observation equation. In the inserted case the body equation already gives the replacement observation. □

Lean: `Linear.boxFiberProvenance`

To lift through a specified edge of a larger graph, use this box certificate at that edge and an equality-transported retained certificate at every other edge. Classical testing of edge equality selects the two cases. At the selected edge, the given label equality identifies its old box, and the new label contains the modified body with unchanged metadata. Elsewhere the label is unchanged, so all local origins are retained. The assembly lemma therefore constructs provenance for the existing concrete box-replacement operation.

Lean: `Linear.replaceBoxProvenance`

21.3 The deep context closure theorem

Theorem 21.3 (Provenance at every typed site)**.** *For every typed site s in G and matching replacement H, the defined result* plugAt(s, H) *has injective, hole-observation-preserving provenance relative to G, H.*

Proof. Induct on the site. A shallow site uses the concrete shallow provenance map. For an inside site, the induction hypothesis gives provenance for the replacement in the chosen body's inner site. Lift this certificate through the selected outer box using the preceding construction. That is exactly the inside branch of the definition of deep plugging. Induction follows the finite site address, so the proof works at every finite nesting depth. □

Lean: `Linear.siteProvenance`

Theorem 21.4 (Deep replacement preserves contexts under compatibility)**.** *If G, H have unique holes and the concrete site provenance is compatible, then* plugAt(s, H) *has unique holes.*

Proof. Use the constructed site provenance in the compatible-provenance uniqueness theorem. No locality or occurrence-injectivity hypothesis remains to be supplied: those properties have been proved for the actual replacement operation. The remaining compatibility premise is explicitly a restriction on colliding hole labels at retained origins. □

Lean: `Linear.plugAt_uniqueHoles_of_compatible`

Full disjointness of G, H gives a simpler sufficient condition: instantiate the full-disjointness provenance theorem with the same site certificate. This condition is stronger than compatibility because it also inspects discarded host occurrences; it is not asserted to be necessary.

Lean: `Linear.plugAt_uniqueHoles`

Every hole label of the deep result belongs to the inventory of the host or replacement, by the provenance inventory-subset theorem. This is an upper bound on the deep inventory, not an exact description of which host labels survive.

Lean: `Linear.plugAt_hasHole_subset`

If H is hole-free, it has unique holes and is disjoint from G. Substituting these two facts gives context closure for replacing any addressed edge by hole-free material, with no separate freshness witness required.

Lean: `Linear.plugAt_holeFree_unique`

Finally, Context.plugAt accepts a certified host context, a typed site, a certified replacement context, and full disjointness. It returns the concrete deeply filled graph paired with its proved context certificate. The original ordered outer interface and enclosing incidence laws are those already supplied by deep replacement.

Lean: `Linear.Context.plugAt`

This provenance construction supplies globally unique-hole context closure and concrete origin tracking through boxes. Section 23 strengthens the inventory bound to an exact syntactic characterization and gives a necessary-and-sufficient freshness condition at every finite box depth. Section 24 supplies recursive isomorphism of box contents and deep replacement congruence. Associativity of plugging remains a separate obligation. None of these results yet defines the concrete UAM.

22 Context composition: absence, reuse, and a collision

22.1 Hole-free filling inside a box

The value corolla is hole-free: any hole witness would claim that the lookup of a primitive operation is a hole label, whereas that lookup is empty. Applying deep context closure for a hole-free replacement to the old boxed-hole site proves that its filled result has unique holes. The context-valued operation also builds a certified filled context from the boxed context and the value corolla, using hole-free uniqueness and disjointness for the replacement.

Lean: `Linear.Examples.value_holeFree`

Lean: `Linear.Examples.filledBox_unique`

Lean: `Linear.Examples.filledContext`

The single-hole corolla has no retained occurrence when its only outer edge is removed. A proposed retained witness includes an edge different from zero, but every element of its edge carrier Fin(1) equals zero. This contradicts the witness's inequality certificate.

Lean: `Linear.Examples.singleHole_retained_absent`

Consequently replacing its only hole by the very same single-hole corolla still gives a context. Use sharp context closure: both input uniqueness proofs are available, and the retained part of the freshness premise is impossible by the preceding result. However, these two identical input graphs do not have fully disjoint inventories: their sole edge is a witness for the same label in each. Applying a proposed full-disjointness proof to those two witnesses gives a contradiction. Together these examples show that sharp freshness admits valid label reuse which the stronger sufficient condition would exclude.

Lean: `Linear.Examples.reuse_removed_label`

Lean: `Linear.Examples.reuse_not_fully_disjoint`

Filling the single-hole corolla with the value is hole-free, not just a context. The exact inventory theorem leaves a retained alternative or a value alternative; the first is impossible by absence of surviving old edges and the second by hole-freeness of the value.

Lean: `Linear.Examples.filled_single_holeFree`

The boxed filled result is hole-free at every depth as well. Express any proposed hole in fibre coordinates. Its outer edge has singleton carrier, so it is zero. The already proved label equation identifies that edge as a box whose body is the filled single-hole graph. Transport the fibre address along that equation, which preserves its hole observation. Its shallow alternative cannot observe a hole because it is a box; its child alternative contradicts the filled body's hole-freeness. Thus no shallow or deep witness exists in the result.

Lean: `Linear.Examples.filledBox_holeFree`

22.2 Two distinct shallow holes in series

For a natural-number name n, define a hole label χ_n with one term input and one term output. The two-hole graph has external type $* \to *$, one internal term vertex v, and edges e_0, e_1 with labels χ_0, χ_1 and incidence

$$\mathsf{input}_0 \xrightarrow{e_0} v \xrightarrow{e_1} \mathsf{output}_0.$$

This notation records the two source and target singleton lists, not an extra path assumption.

Lean: `Linear.Examples.chainHole`

Lean: `Linear.Examples.twoHoles`

Graph invariants. The carriers are Fin(1) and Fin(2), with finite range enumerations. Every incidence list is singleton and hence distinct. For source uniqueness, two edges of different indices have different source constructors, namely input and internal, so a shared source is impossible; equal-index cases give equal edges. The target argument uses internal and output constructors instead. Coverage chooses e_0 for the input source and internal target, and e_1 for the internal source and output target. Outer outputs have no source occurrence and outer inputs no target occurrence by constructor disjointness. There is only one internal index and one index at each boundary, so these cases exhaust all vertices. Every displayed vertex label is term, matching the singleton input and output lists of both typed hole labels. □

This graph has unique holes. Two equal observed labels give equal numeric hole names. If one edge index is zero and the other is not, their names are respectively zero and one, a contradiction. If both are zero they agree directly. If neither is zero, both are one because their ordinal carrier has size two, so they agree again. At depth zero these are all occurrence addresses.

Lean: `Linear.Examples.twoHoles_unique`

22.3 A checked failure without freshness

Let the replacement be the corolla of χ_1, with the same external interface as e_0. It is itself a context: any two occurrences belong to its singleton edge carrier and hence are equal, independently of their lookup hypotheses.

Lean: `Linear.Examples.repeatedHole`

Lean: `Linear.Examples.repeatedHole_unique`

Nevertheless, replacing e_0 in the two-hole graph by this corolla violates unique holes. In the result, the retained host edge e_1 and the inserted replacement edge both observe χ_1. Context uniqueness would identify their occurrences, but their edge tags are different constructors of the disjoint sum. This is a contradiction. Thus well-formed graph plugging and uniqueness of each input graph alone do not guarantee the context condition in the result.

Lean: `Linear.Examples.collision_breaks_unique`

The same witnesses explicitly refute sharp freshness: e_1 and its shallow fibre address witness retained presence of χ_1, and the replacement's only edge witnesses inserted presence. Feeding both to a proposed freshness proof gives false. This regression exercises the actual collision excluded by the necessary-and-sufficient theorem, alongside the valid label-reuse example above.

Lean: `Linear.Examples.collision_not_fresh`

23 Exact inventories and sharp context closure at every depth

The provenance argument establishes a useful upper bound on hole inventories. We now strengthen it to an equality described entirely by the original host and the selected address. This also gives a necessary-and-sufficient freshness condition. Throughout this section s is a site in G, H has the interface and local depth required by s, and $G[s := H]$ denotes $\mathsf{plugAt}(s, H)$. Write $\mathcal{H}(G)$ for the set of full typed hole labels present in G. These statements concern the concrete typed hierarchy; they do not presuppose any specimen laws or a concrete UAM semantics.

23.1 Separating one outer edge from the rest of a graph

Definition 23.1 (Inventory of an edge)**.** For an outer edge e of G, let $E_G(e, h)$ mean that some position in the fibre of e observes h. This includes the edge itself and all descendants within its label. Thus $E_G(e, h)$ is the predicate $\mathsf{EdgeHasHole}(G, e, h)$.

Lean: `Linear.EdgeHasHole`

Definition 23.2 (Uniqueness away from an edge)**.** The predicate $\mathsf{AwayUnique}(G, e)$ says that any two occurrences with outer edges different from e and with the same hole observation are equal. It places no condition on occurrences within the fibre of e.

Lean: `Linear.AwayUnique`

Lemma 23.3 (Partition of uniqueness)**.** *The graph G has unique holes if and only if all three conditions hold: occurrences away from e are unique; hole observations within the fibre of e are unique; and no label is observed both away from e and within that fibre.*

Proof. Global uniqueness restricts to each part. A collision between the two parts would force equality of their outer edges, contradicting the requirement that the first outer edge differ from e. Conversely, take two occurrences of the same hole label. If both lie away from e, use away-uniqueness. If both have outer edge e, use fibre uniqueness. The two mixed cases contradict the cross-part disjointness condition. These cases exhaust all occurrences. □

Lean: `Linear.uniqueHoles_partition`

Lemma 23.4 (Inventories of a box and of a whole graph)**.** *If the label of e is a box with body B, then $E_G(e, h)$ holds exactly when $h \in \mathcal{H}(B)$. For an arbitrary edge e, a label h occurs in G exactly when it occurs away from e or satisfies $E_G(e, h)$.*

Proof. The shallow position of a box is not a hole, while each of its child positions is precisely an occurrence of its body, with the same observation. For the whole-graph statement, write an occurrence as an outer edge and a fibre position, and split according to whether that edge equals e. In the reverse direction either witness is already an occurrence of G. □

Lean: `Linear.edgeHasHole_box`

Lean: `Linear.hasHole_partition`

Lemma 23.5 (Exact inventory after replacing a box body)**.** *Changing the body of box edge e to B' preserves the inventory away from e exactly. The entire new inventory is the union of the old inventory away from e and $\mathcal{H}(B')$.*

Proof. Every other edge has exactly the same label before and after the change. Transport its fibre positions along this label equality; fibre transport preserves observations in both directions. This proves the first claim. Partition the new graph at e and use the preceding box-inventory lemma to identify its selected fibre with the new body's inventory. □

Lean: `Linear.replaceBox_retained_iff`

Lean: `Linear.replaceBox_hasHole_iff`

Lemma 23.6 (Box uniqueness)**.** *The hole observations in a box fibre are unique exactly when its body has unique holes. Consequently, the body of every box in a context has unique holes.*

Proof. A box's shallow position cannot witness a hole. Two child positions observing the same hole are equal precisely when their body occurrences are equal. Injectivity of the child constructor proves the reverse implication. For the consequence, restrict the graph's uniqueness to the selected fibre using the partition lemma, then apply this equivalence. □

Lean: `Linear.boxFiber_unique_iff`

Lean: `Linear.uniqueHoles_box_body`

Lemma 23.7 (Uniqueness after changing one box)**.** *Suppose G is a context. Replacing the body of box edge e by B' yields a context if and only if B' has unique holes and its hole inventory is disjoint from the inventory of G away from e.*

Proof. First the unchanged part remains unique: transport two of its occurrences to G along their unchanged label equalities, apply uniqueness in G, and use injectivity of fibre transport to recover equality in the new graph. The uniqueness partition for the changed graph now has a proved first condition. Its second condition is equivalent to uniqueness of B', by box uniqueness. Its third condition is exactly the stated disjointness, by the preservation of the outside inventory and the exact inventory of a box. □

Lean: `Linear.replaceBox_awayUnique`

Lean: `Linear.replaceBox_uniqueHoles_iff`

23.2 A syntactic inventory of retained holes

Definition 23.8 (Retention along a site)**.** Let $R_s(h)$ be defined recursively on the site address. At a shallow site selecting e in G, it is $\mathsf{RetainedHole}(G, e, h)$: there is an occurrence of h under another outer edge. If the site descends through box edge e to an inner site t, set

$$R_s(h) \iff \mathsf{RetainedHole}(G, e, h) \lor R_t(h).$$

This is SiteRetainedHole. It depends only on the host and address, not on the replacement graph or a provenance certificate. It excludes all occurrences below the edge finally removed, while retaining the siblings of every enclosing box. Enclosing box edges are not themselves holes.

Lean: `Linear.SiteRetainedHole`

Theorem 23.9 (Exact deep hole inventory)**.** *For every site and every type-correct replacement, without uniqueness or freshness assumptions,*

$$h \in \mathcal{H}(G[s := H]) \iff R_s(h) \lor h \in \mathcal{H}(H).$$

Lean: `Linear.plugAt_hasHole_iff`

Proof. Induct on the site address. The shallow case is the exact shallow inventory theorem. For a site inside a box, the exact box-replacement inventory is the union of the unaffected outer inventory and the changed body's inventory. The induction hypothesis identifies the latter as the inner retained inventory union the replacement inventory. Reassociate these three alternatives; the first two are exactly R_s. Both directions are preserved at each step, so no retained or inserted label is lost from the characterization. □

Definition 23.10 (Exact deep freshness)**.** The predicate $\mathsf{SiteFresh}(s, H)$ says

$$\forall h, \quad R_s(h) \implies h \notin \mathcal{H}(H).$$

Only retained host labels constrain the replacement; a label that is wholly consumed is available for reuse.

Lean: `Linear.SiteFresh`

Lemma 23.11 (Retention implies original presence)**.** *If $R_s(h)$, then $h \in \mathcal{H}(G)$.*

Proof. Induct on the site. An outside witness is already a host occurrence. For an inner retained witness, the induction hypothesis supplies an occurrence in the original box body; the box-inventory lemma and whole-graph partition lift it back into the host. □

Lean: `Linear.siteRetainedHole_hasHole`

Lemma 23.12 (Replacement uniqueness is necessary for shallow plugging)**.** *If a shallow plugged graph has unique holes, its replacement has unique holes.*

Proof. The right-tagged injection embeds every replacement edge into the result, preserving labels and all fibre observations. Uniqueness reflects along this injective edge map by the established embedding theorem. □

Lean: `Linear.plug_uniqueHoles_replacement`

23.3 Necessary and sufficient context closure

Theorem 23.13 (Exact context criterion at arbitrary depth)**.** *Suppose G has unique holes. Then*

$$\mathsf{UniqueHoles}(G[s := H]) \quad \iff \quad \mathsf{UniqueHoles}(H) \ \wedge \ \mathsf{SiteFresh}(s, H).$$

Thus uniqueness of the replacement is a necessary conclusion in the forward direction, not an additional assumption of the theorem.

Proof. Induct on the address. At a shallow site, uniqueness of the result forces both replacement uniqueness and sharp shallow freshness. The reverse implication is the proved shallow closure theorem.

For an inside site, let B be the old body and t the inner address. Since G is a context, B is a context. The box-replacement criterion says that the result is a context exactly when $B[t := H]$ is a context and its inventory is disjoint from the outer inventory away from the selected box. The induction hypothesis replaces its first condition by uniqueness of H and inner freshness.

For necessity, inner freshness excludes collisions with inner retained labels. A label inserted from H is present in the changed body by the exact inventory theorem; outer disjointness therefore excludes collisions with every retained outer label. These are precisely the two alternatives defining R_s.

For sufficiency, site freshness supplies inner freshness, so induction proves uniqueness of the changed body. Consider a label shared by that body and an unaffected outer edge. The exact inventory theorem gives two cases. If the label comes from H, site freshness is contradicted. If

it is retained inside the body, retention implies its presence in the old body. It would then have occurred both inside and outside that box already in G, contradicting the uniqueness partition of G. This proves the required outer disjointness and completes the induction. □

Lean: `Linear.plugAt_uniqueHoles_iff`

If H is already a context, its uniqueness conjunct is known, leaving the necessary-and-sufficient condition $\mathsf{SiteFresh}(s, H)$ alone.

Lean: `Linear.plugAt_uniqueHoles_iff_fresh`

The constructor Context.plugAtFresh pairs the deep replacement graph with the proof supplied by this equivalence. Unlike full-label disjointness, its required certificate permits reuse of consumed labels at any box depth.

Lean: `Linear.Context.plugAtFresh`

Theorem 23.14 (Exact hole-freeness criterion)**.** *The result $G[s := H]$ is hole-free if and only if no label satisfies R_s and H is hole-free. No uniqueness assumption is needed.*

Proof. The exact inventory is empty precisely when both of its constituent inventories are empty. In either direction, an alleged witness in one part gives, or comes from, a witness in the union. □

Lean: `Linear.plugAt_holeFree_iff`

23.4 Consumption of the selected label

Definition 23.15 (Selected hole label)**.** The partial observation $\mathsf{Site.selectedHole}(s)$ follows the address to its selected edge and reads that edge's own shallow hole label. It does not inspect descendants of the selected edge. Thus selecting a box yields no hole label, even if its body contains holes.

Lean: `Linear.Site.selectedHole`

Lemma 23.16 (Presence and non-retention of a selected hole)**.** *If the selected label is h, then h occurs in the original host. If the host is a context, $R_s(h)$ is false.*

Proof. For presence, the shallow case uses the selected edge's shallow fibre position. The inside case lifts the induction hypothesis from the body through the box. For non-retention, the shallow statement is the existing consumed-label lemma. Inside a box, a retained witness outside the box contradicts the old graph's uniqueness partition, because the selected label occurs inside its body. An inner retained witness contradicts induction applied to that body's uniqueness. □

Lean: `Linear.Site.selectedHole_present`

Lean: `Linear.site_selected_not_retained`

Theorem 23.17 (Exact consumption at arbitrary depth)**.** *If G is a context and s selects a hole labelled h, then*

$$h \in \mathcal{H}(G[s := H]) \quad \Longleftrightarrow \quad h \in \mathcal{H}(H).$$

Proof. Use the exact deep inventory theorem and eliminate its retained-host alternative by non-retention of the selected label. This permits both genuine removal and intentional reintroduction, depending on the replacement. □

Lean: `Linear.plugAt_consumed_hole_iff`

23.5 Concrete regressions inside boxes

In the existing one-hole boxed graph, no label is retained when the inner hole is selected. At the outer level there is no other edge (the carrier has one element), and inside the box there is no other edge either. This proves $\neg R_s(h)$ for every h.

Lean: `Linear.Examples.boxedHole_retained_absent`

Replacing that hole by the same one-hole context therefore preserves uniqueness, by sharp deep freshness. Full disjointness fails because both original graphs contain the selected label. The consumed-label theorem also proves that this label is genuinely present after replacement, witnessed by its occurrence in the inserted graph.

Lean: `Linear.Examples.deep_reuse_unique`

Lean: `Linear.Examples.deep_reuse_not_disjoint`

Lean: `Linear.Examples.deep_reuse_present`

Replacing instead with the hole-free value graph yields a hole-free result: combine absence of retained labels and hole-freeness of the value using the exact hole-freeness criterion. This is a shorter independent derivation of the previous global filling result.

Lean: `Linear.Examples.deep_fill_holeFree`

For a negative example, box the earlier two-hole chain with zero bound variables and one free output. Its external interface is one zero-binding thunk input and one term output. Define a site descending into this box and selecting the first hole. The boxed graph is a context because the two-hole body is a context and boxing preserves uniqueness.

Lean: `Linear.Examples.boxedTwoHoles`

Lean: `Linear.Examples.boxedFirstSite`

Lean: `Linear.Examples.boxedTwoHoles_unique`

The one-hole replacement carrying label 1 is also a context, but it collides with the retained second hole. These two witnesses refute deep freshness. The necessary direction of the deep context theorem then proves that the result fails uniqueness.

Lean: `Linear.Examples.deep_collision_not_fresh`

Lean: `Linear.Examples.deep_collision_breaks_unique`

Finally, the exact inventory distinguishes the labels: label 0 is absent and label 1 is present. The first claim uses consumption and the fact that the replacement's label has name 1, not 0;

the second uses its inserted witness. An inventory records presence rather than multiplicity, so this is consistent with the independently proved duplicate occurrence of label 1.

Lean: `Linear.Examples.deep_collision_inventory`

24 Recursive isomorphism and substitution modulo renaming

Literal equality of edge labels is too restrictive when a label contains a whole graph. Renaming the body of a box changes its graph record, although its mathematical incidence structure is unchanged. This section defines recursive isomorphism, proves its invariants, and proves congruence of replacement at any addressed depth. All comparisons are within a fixed depth bound and fixed ordered outer interface. The quotient construction below does not identify different depth bounds. Section 45 supplies the actual-content representation and a further quotient identifying depth promotion.

24.1 Isomorphisms with related labels

Definition 24.1 (Isomorphism relative to a label relation)**.** Fix a signature and a relation R on its labels. An R-isomorphism from G to H consists of bijections $v : G_V \to H_V$ of internal vertices and $e : G_E \to H_E$ of edges. Internal vertex labels agree, corresponding edge labels are related by R, and mapping every source or target list through the induced port map gives the corresponding list in H. The port map fixes each ordered input and output position. We write $G \cong_R H$ for the existence of such data.

Lean: `Net.RelIso`

Lemma 24.2 (Identity, inverse, and composition)**.** *If R is reflexive, every graph has an identity R-isomorphism. If R is symmetric, every R-isomorphism has an inverse. If R is transitive, R-isomorphisms compose.*

Proof. For identity use identity carrier maps and reflexivity of R. For inverse use the inverse carrier bijections. Evaluate the original label equation at the inverse image, cancel the edge bijection, and use symmetry of R. For incidence, map the original incidence equation through the inverse port map; cancellation recovers the reversed equation. Composition uses composed carrier bijections, transitivity of the label relation, and the fact that a list mapped twice is mapped by the composite. Port maps compose because they act on internal names and fix both kinds of boundary position. □

Lean: `Net.RelIso.refl`

Lean: `Net.RelIso.symm`

Lean: `Net.RelIso.trans`

Lemma 24.3 (Incidence paths and ordinary isomorphisms)**.** *Every R-isomorphism preserves and reflects adjacency, hence preserves and reflects paths under its edge bijection. Every ordinary label-preserving isomorphism gives an R-isomorphism when R is reflexive.*

Proof. Map a shared vertex to obtain forward adjacency. For reflection, source and target preservation express the shared target vertex as images of two original ports. Injectivity of the port bijection makes those ports equal, giving the original shared vertex. Apply the existing list-path mapping lemma to this adjacency equivalence. For an ordinary isomorphism, retain all carrier and incidence data; equality of its labels and reflexivity give the required R-relation at each edge. □

Lean: `Net.RelIso.adjacent_iff`

Lean: `Net.RelIso.path_iff`

Lean: `Net.RelIso.ofIso`

24.2 Recursive comparison of box contents

Definition 24.4 (Related boxes)**.** For a relation on bodies of each fixed interface, two boxes are related when they have the same number of bound variables, the same number of free outputs, and related bodies at the interface specified by those two numbers. The constructor BoxRelated.bodies enforces both metadata equalities; it does not merely require equality of their sum.

Lean: `Linear.BoxRelated`

Definition 24.5 (Recursive label equivalence)**.** At depth zero, label equivalence is literal equality. At positive depth, primitive labels are equivalent exactly when they are equal; a primitive and a box are never equivalent. Two box labels are equivalent when their metadata agree and there exists an isomorphism of their bodies whose corresponding labels are recursively equivalent at the smaller depth. This is a well-founded definition because the depth decreases whenever a box body is compared.

Lean: `Linear.LabelEquiv`

Definition 24.6 (Recursive hypernet isomorphism)**.** A recursive isomorphism $G \cong_d H$, represented by DeepIso, is an isomorphism relative to recursive label equivalence at depth d. Its outer carrier bijections are explicit data. The label proof at a box supplies the existence of a further body isomorphism, independently of the names chosen at any other box. Every level preserves its own ordered boundary positions.

Lean: `Linear.DeepIso`

Lemma 24.7 (Equivalence laws for recursive labels)**.** *Recursive label equivalence is reflexive, symmetric, and transitive.*

Proof. Induct on depth. At depth zero the claims are equality laws. At positive depth, primitive cases again reduce to equality and mixed primitive/box cases are impossible. For box reflexivity, the induction hypothesis supplies reflexivity of the body-label relation, so the identity relative isomorphism witnesses body equivalence. For box symmetry, take a witnessing body isomorphism and invert it using symmetry at the smaller depth. For transitivity, matching the two box-relation constructors identifies all metadata, so the two body isomorphisms have a common middle graph and interface. Compose them using transitivity at the smaller depth. Package each resulting body isomorphism in the box relation. □

Lean: `Linear.labelEquiv_refl`

Lean: `Linear.labelEquiv_symm`

Lean: `Linear.labelEquiv_trans`

Identity, inverse, and composition of recursive graph isomorphisms follow by instantiating the generic constructions with these three laws. Ordinary isomorphisms embed into recursive ones by reflexivity of the label relation.

Lean: `Linear.DeepIso.refl`

Lean: `Linear.DeepIso.symm`

Lean: `Linear.DeepIso.trans`

Lean: `Linear.DeepIso.ofIso`

Lemma 24.8 (Signatures and operation labels are invariant)**.** *Equivalent labels have equal ordered input signatures, equal ordered output signatures, and the same operation-label predicate.*

Proof. At depth zero or for two primitives, substitute the label equality. Mixed labels are impossible. For two boxes, the relation constructor identifies the bound/free metadata, and both signatures depend only on those numbers. Both box labels fail the operation predicate. No information about body carrier names is used in these conclusions. □

Lean: `Linear.labelEquiv_input`

Lean: `Linear.labelEquiv_output`

Lean: `Linear.labelEquiv_isOperation`

24.3 Operation traces and paths

Theorem 24.9 (Recursive renaming preserves and reflects operation paths)**.** *Let $f : G \cong_d H$, with port map v and edge bijection e. An operation trace from x to y along the edge list ℓ in G exists exactly when the trace from $v(x)$ to $v(y)$ along* $\mathsf{map}(e, \ell)$ *exists in H. The analogous statement holds for forward operation paths.*

Proof. First prove forward trace transport by induction on the trace. An empty trace maps to an empty trace. For a nonempty trace, operation-label invariance supplies the required label predicate. Source and target preservation map its two incidence witnesses, and induction supplies the remaining trace.

For reflection transport the target trace through the inverse recursive isomorphism. The inverse port and edge maps cancel the original maps, recovering both original endpoints and exactly the original edge list. A forward operation path is the reverse-oriented trace with its edge list reversed. List mapping commutes with reversal, so the trace equivalence gives the path equivalence. In particular, input and output endpoints retain their boundary indices. □

Lean: `Linear.DeepIso.operationTrace`

Lean: `Linear.DeepIso.operationTrace_iff`

Lean: `Linear.DeepIso.operationPath_iff`

24.4 Transporting occurrences and holes through all boxes

Definition 24.10 (Fibre and hole embeddings)**.** A fibre embedding maps positions in one label to positions in another, injectively, and preserves their complete hole observations, including the absence of a hole. A hole embedding similarly maps graph occurrences injectively and preserves their observations. These maps concern occurrence addresses; their definition does not itself assert any incidence property.

Lean: `Linear.FiberEmbedding`

Lean: `Linear.HoleEmbedding`

Lemma 24.11 (Assembling fibre embeddings)**.** *An injective map of outer edges and a fibre embedding over each mapped edge together give a hole embedding of graphs.*

Proof. Map (a, x) to the mapped edge paired with the image of x under its fibre embedding. If two results coincide, projecting their edges and using edge-map injectivity identifies the original edges. Their remaining fibre positions now have the same type, and fibre-map injectivity identifies them. Observation preservation is exactly the preservation property of the corresponding fibre embedding. □

Lean: `Linear.HoleEmbedding.ofFibers`

A hole embedding also transports the original deep addresses: first use the occurrence-coordinate bijection of the source, apply the hole embedding, then use the inverse coordinate bijection of the target. This defines HoleEmbedding.allEdge.

Lean: `Linear.HoleEmbedding.allEdge`

The resulting address map is injective. Apply the target coordinate bijection to an equality of images, cancel its inverse, use embedding injectivity, and finally use source-coordinate

injectivity. It preserves hole lookup because both coordinate bijections preserve observation and the middle embedding does so as well.

Lean: `Linear.HoleEmbedding.allEdge_injective`

Lean: `Linear.HoleEmbedding.allEdge_hole`

Lemma 24.12 (Recursive labels admit fibre embeddings)**.** *If two labels are recursively equivalent, an injective, observation-preserving map of their fibres exists.*

Proof. Induct on depth. Equal primitives use the identity map. Mixed label cases are impossible. For equivalent boxes, choose a witnessing isomorphism of their bodies. For each body edge, the induction hypothesis supplies a fibre embedding from its recursive label-equivalence proof; choose one such map per edge. Assemble these maps over the body isomorphism's injective edge map to obtain a body hole embedding, and express it in original deep-address coordinates.

Map the box's shallow position to the shallow position, and each child address through this body map. The two alternatives remain disjoint. Within the shallow alternative equality is immediate; within the child alternative injectivity is the proved injectivity of body-address transport. Shallow observations are empty on both sides, while child observations agree by the body's transport theorem. This constructs the desired embedding. Classical choice selects witnesses of the existential body-isomorphism and embedding statements; it does not supply any graph invariant as an additional assumption. □

Lean: `Linear.labelEquiv_hasEmbedding`

The noncomputable function labelEquiv_embedding chooses the embedding whose existence was just proved. Applying it at every outer edge of a recursive isomorphism, and assembling over its edge bijection, defines DeepIso.holeEmbedding.

Lean: `Linear.labelEquiv_embedding`

Lean: `Linear.DeepIso.holeEmbedding`

Lemma 24.13 (Presence transports and uniqueness reflects)**.** *A hole embedding transports every present hole label from source to target. If the target has unique hole observations, so does the source.*

Proof. Map the occurrence witnessing a present label and use preservation of lookup. For uniqueness, map two source occurrences with equal hole observations. Target uniqueness makes their images equal, and embedding injectivity makes the original occurrences equal. □

Lean: `Linear.HoleEmbedding.hasHole`

Lean: `Linear.HoleEmbedding.unique`

Theorem 24.14 (Hole and context invariance under recursive isomorphism)**.** *Recursively isomorphic graphs have identical hole inventories. One has unique holes if and only if the other does; one is hole-free if and only if the other is.*

Proof. Construct hole embeddings from the isomorphism and its inverse. Presence transport gives the two implications for inventory equality. Uniqueness reflection, in the two directions, gives the context equivalence. Negating hole presence for every label gives the hole-freeness equivalence. The two chosen embeddings are not asserted to be mutually inverse; the two implications and injectivity are sufficient for these results. □

Lean: `Linear.DeepIso.hasHole_iff`

Lean: `Linear.DeepIso.uniqueHoles_iff`

Lean: `Linear.DeepIso.holeFree_iff`

24.5 Congruence of shallow and deep replacement

For a fixed host C and shallow selected edge h, an R-isomorphism $f : H \cong_R K$ gives the candidate carrier bijections for the plugged graphs: use identity on the host summand and f on the replacement summand. For edges, the host summand is the subtype of edges different from h.

Lean: `Net.RelIsoPlug.vertices`

Lean: `Net.RelIsoPlug.edges`

The induced port map commutes with host embedding, by a case split on internal, input, and output ports. It commutes with insertion after the replacement's own port map: an internal port uses the right summand, and a boundary port reduces to the host-embedding equation at the selected source or target port.

Lean: `Net.RelIsoPlug.map_host`

Lean: `Net.RelIsoPlug.map_insert`

Theorem 24.15 (Shallow congruence for related labels)**.** *For reflexive R, an R-isomorphism $H \cong_R K$ induces $C[h := H] \cong_R C[h := K]$. No non-self-loop condition is required.*

Proof. Use the preceding sum bijections. Host internal labels are unchanged and replacement internal labels are preserved by f. Host edge labels satisfy reflexivity of R, and inserted edge labels use the relation supplied by f. For a host incidence list, commute the port map with host embedding pointwise. For an inserted incidence list, commute it with insertion and then use the source or target preservation equation of f. This establishes every field of the resulting relative isomorphism without requiring insertion to be globally injective. □

Lean: `Net.RelIso.plug`

Lemma 24.16 (Box congruence)**.** *Boxing recursively isomorphic bodies with the same metadata gives recursively isomorphic boxes. Replacing one fixed box edge in a surrounding graph by either of those bodies also gives recursively isomorphic graphs.*

Proof. For a single boxed graph, use identity maps on its empty internal carrier and one-edge carrier. Its labels are related by the body-isomorphism witness, and its boundary lists depend only on the common metadata. In a surrounding graph, use identity maps on both outer carriers. At the selected edge use the related box-body witness; at every other edge use reflexivity of recursive label equivalence. The two outer incidence structures are identical, so their preservation equations reduce to mapping a list by identity. □

Lean: `Linear.DeepIso.box`

Lean: `Linear.replaceBox_deepIso`

Theorem 24.17 (Congruence at every addressed depth)**.** *For a fixed host G and typed site s, if $H \cong_r K$, then*

$$G[s := H] \cong_d G[s := K].$$

No context-uniqueness, freshness, or non-self-loop assumption is needed.

Proof. Induct on the site. A shallow site uses the generic shallow congruence with the reflexive recursive-label relation. For a site inside a box, induction gives an isomorphism between the two changed bodies. Lift it through the selected box using box congruence. Repeating this step crosses every enclosing box in the address. □

Lean: `Linear.plugAt_deepIso`

Two consequences connect this congruence with the preceding milestone. Exact site freshness is invariant when the replacement is changed by recursive isomorphism: the retained host predicate stays fixed, and the replacement hole inventories coincide. Context validity of the two plugged results is also equivalent, by context invariance applied to their constructed recursive isomorphism. The latter statement does not assume the host is a context.

Lean: `Linear.siteFresh_deepIso_iff`

Lean: `Linear.plugAt_deepIso_unique_iff`

24.6 Graphs and replacement modulo recursive renaming

Definition 24.18 (Recursive isomorphism classes)**.** At fixed depth d and interface (I, O), declare $G \sim H$ when a recursive isomorphism exists. Identity, inverse, and composition prove that this is an equivalence relation, packaged as deepSetoid. Its quotient is HypernetModuloIso, the type of graphs modulo independent carrier renaming at all levels of their hierarchy. Write $[G]$ for the class of G.

Lean: `Linear.deepSetoid`

Lean: `Linear.HypernetModuloIso`

Definition 24.19 (Plugging a replacement class)**.** For a fixed concrete host and site s, define

$$\mathsf{plugAtModuloIso}(s, [H]) = [G[s := H]].$$

Well-definedness. If H and K represent the same source class, their recursive isomorphism induces one between the plugged graphs by deep congruence. The results therefore belong to the same target class. Quotient elimination defines the displayed operation from this representative-independent function. Its value on a concrete representative is the displayed equation by the computation rule of quotient elimination. □

Lean: `Linear.plugAtModuloIso`

Lean: `Linear.plugAtModuloIso_mk`

Context validity descends to classes as well: define $\mathsf{moduloIsoUniqueHoles}([G])$ to mean that G has unique holes. The isomorphism-invariance theorem makes these propositions equivalent for any two representatives; propositional extensionality supplies the equality required by quotient elimination.

Lean: `Linear.moduloIsoUniqueHoles`

These constructions quotient the replacement argument for a fixed host and address. Section 25 extends invariance to corresponding sites in isomorphic hosts. Section 26 proves associativity and units. Section 36 constructs the quotient of host/site pairs.

24.7 A strict extension of literal-label isomorphism

Use the earlier two-hole chain. The permutation $i \mapsto 1 - i$ on its two-edge carrier exchanges edge names. Applying it twice is identity because $0 \le i < 2$. Reindex the chain's edges by this bijection and keep its internal vertex name unchanged; call the result H'. The existing reindexing isomorphism, embedded into recursive isomorphism, witnesses $H \cong_0 H'$.

Lean: `Linear.Examples.swapTwo`

Lean: `Linear.Examples.renamedTwoHoles`

Lean: `Linear.Examples.twoHoles_deepIso`

The records H and H' are unequal. If they were equal, their edge-label functions would agree at the edge named zero. Reading the hole name there gives 0 in H and 1 in H', a contradiction.

Lean: `Linear.Examples.renamedTwoHoles_ne`

Box H' with zero bound variables and one free output. Box congruence relates this box to the earlier box of H.

Lean: `Linear.Examples.renamedBoxedTwoHoles`

Lean: `Linear.Examples.boxedTwoHoles_deepIso`

No literal label-preserving isomorphism relates these two boxed graphs. Such an isomorphism's label equation at the sole source edge would make their box labels equal. Equality of these labels forces equality of the body records, contrary to the preceding result. This proves that recursive isomorphism identifies strictly more graphs than the earlier literal-label notion.

Lean: `Linear.Examples.boxedTwoHoles_no_literalIso`

The renamed box remains a context by recursive invariance of unique holes. The two different

box records represent the same quotient class by quotient soundness applied to their recursive isomorphism.

Lean: `Linear.Examples.renamedBoxedTwoHoles_unique`

Lean: `Linear.Examples.renamedBoxes_same_class`

For a nontrivial addressed replacement, use the earlier site selecting the first hole inside a box. Substituting H or H' there gives recursively isomorphic results by deep congruence. Finally, the primitive operations named value and otherValue are not equivalent: at depth zero recursive label equivalence is equality, and these are distinct constructors. Thus carrier renaming does not erase semantic operation labels.

Lean: `Linear.Examples.deep_plug_renaming`

Lean: `Linear.Examples.distinct_operations_not_equivalent`

25 Transporting replacement sites through isomorphisms

An address denotes an occurrence, not a permanent numerical edge name. If an isomorphism exchanges two edges, the selected occurrence must move with it. This section proves existence of that transport at arbitrary box depth, congruence when both host and replacement change, and invariance of the exact freshness condition. Depth bounds and ordered outer interfaces remain fixed.

25.1 Transport of boundary indices and shallow hosts

Definition 25.1 (Boundary transport)**.** Given equalities $I = I'$ and $O = O'$ of ordered interfaces, transport a port by fixing its internal name, or by retaining its numerical input or output index and using the equality to prove the new bound. Transport a graph to the new interface by keeping its internal vertices, edges, enumerations, and labels, and mapping every incidence list through this port transport. This graph is written $H^{I',O'}$ below.

All graph invariants follow by eliminating the two interface equalities: the port map is then the identity, and every invariant is exactly the original one. Reflexive port transport is pointwise the identity, hence is the identity function by function extensionality. Reflexive graph transport is equal to the original graph by record equality and the identity law for list mapping. Keeping carriers definitionally unchanged makes the construction useful even before the interface equality proofs have been eliminated.

Lean: `Port.cast`

Lean: `Port.cast_refl`

Lean: `Port.cast_refl_fun`

Lean: `Net.reboundary`

Lean: `Net.reboundary_refl`

Definition 25.2 (Complement restriction)**.** For a bijection $f : A \to B$ and $a \in A$, restrict f to a bijection $A \setminus \{a\} \to B \setminus \{f(a)\}$. Its inverse is the restriction of f^{-1}.

Injectivity shows that a surviving source cannot map to $f(a)$. Conversely, if $f^{-1}(b) = a$, the right inverse equation gives $b = f(a)$, contradicting survival. Both inverse laws follow from those of f, since subtype elements with equal underlying values are equal.

Lean: `Bijection.except`

Lemma 25.3 (Transport of attachment positions)**.** *Let $f : C \cong_R D$ and let e be a selected edge. If its corresponding input interfaces are equal, mapping the ith source port of e by f gives the ith source port of $f(e)$. The analogous assertion holds for targets when the output interfaces are equal.*

Proof. Apply optional list lookup at index i to the source-list equation of f. The typing invariant identifies the source-list length with the input-interface length, so both lookups are in bounds. Lookup commutes with mapping; cancelling the optional-result constructor gives the port equation. The interface equality supplies the bound for the target index. Repeat for the target-list equation. No disjointness between source and target lists is used. □

Lean: `Net.RelIso.sourcePort`

Lean: `Net.RelIso.targetPort`

Theorem 25.4 (Shallow host congruence)**.** *Suppose $f : C \cong_R D$, R is reflexive, and corresponding selected edges have equal input and output interfaces. For every replacement A of the source interface,*

$$C[e := A] \cong_R D[f(e) := A^{I',O'}].$$

Proof. On internal vertices use the disjoint sum of the host vertex bijection and the identity on replacement vertices. On edges use the complement restriction of the host edge bijection and the identity on replacement edges. These are the auxiliary maps `HostPlug.vertices` and `HostPlug.edges`. The whole-graph port map commutes with host inclusion by case analysis on a port. It also commutes with replacement insertion followed by boundary transport: internal vertices are immediate, and input and output ports use the attachment-position lemmas just proved.

For surviving edges, label relatedness is supplied by f; for inserted edges, use reflexivity of R. Internal vertex labels are preserved by the host isomorphism or by identity. For incidence of a surviving edge, rewrite with f's incidence equation, combine successive list maps, and apply commutation with host inclusion. For an inserted edge, combine the insertion and whole-port maps and use commutation with replacement insertion. The same argument proves the target equation. Self-loops are allowed throughout. □

Lean: `Net.HostPlug.vertices`

Lean: `Net.HostPlug.edges`

Lean: `Net.HostPlug.map_host`

Lean: `Net.HostPlug.map_insert`

Lean: `Net.RelIso.plugHost`

25.2 Structural correspondence of addresses

Definition 25.5 (Address interface transport)**.** Given $s : \mathsf{Site}(G, r, J, K)$ and equalities $J = J'$, $K = K'$, transport its interface indices to obtain an address of type $\mathsf{Site}(G, r, J', K')$. The selected occurrence is unchanged. Plugging A at this transported address equals plugging the boundary-transported A at the original address.

The definition is equality elimination. For the plugging equation eliminate the two equalities and use reflexive graph boundary transport; both sides then become the same substitution.

Lean: `Linear.Site.reinterface`

Lean: `Linear.plugAt_reinterface`

Lemma 25.6 (Corresponding boxes and their replacement)**.** *If $f : G \cong_{d+1} H$ and edge e of G carries box (n, m, B), then $f(e)$ carries a box (n, m, B') with $B \cong_d B'$. Moreover, replacing these two box bodies by any $A \cong_d A'$ gives recursively isomorphic whole graphs.*

Proof. For existence, inspect the target label in the label-relatedness proof for e. A primitive target contradicts the definition of recursive label equivalence. For a box target, the constructor of the box relation forces matching n, m and supplies a body isomorphism. This reasoning constructs an existential statement, so elimination of the propositional box relation is legitimate.

For replacement retain the vertex and edge bijections of f. At e the new labels are related by the given isomorphism $A \cong_d A'$. At every other edge injectivity ensures that its target is different from $f(e)$, so both labels remain unchanged and their original relation applies. Neither replacement changes incidence or internal vertex labels; the remaining equations are exactly those of f. □

Lean: `Linear.DeepIso.boxAt`

Lean: `Linear.DeepIso.replaceBox`

Definition 25.7 (Corresponding sites)**.** The proposition $s \simeq t$, implemented by `SiteIso`, is generated by two structural rules. A shallow site selecting e corresponds to the site selecting $f(e)$ for any recursive host isomorphism f; transport the latter's interface indices back along the signature equalities. For sites inside boxes, use a recursive outer-host isomorphism, corresponding selected box edges with equal binding/free metadata, and structurally corresponding inner sites. Both sites have the same local depth and replacement interface.

This is a relation on addresses themselves, defined before proving anything about the results of plugging. An inner correspondence may choose any suitable isomorphism between the box bodies; the outer relation stores existence of a body isomorphism, not a distinguished naming map for every inner vertex.

Lean: `Linear.SiteIso`

Theorem 25.8 (Existence of transported sites)**.** *For every recursive isomorphism* $f : G \cong_d H$ *and every site* $s : \mathsf{Site}(G, r, J, K)$*, there exists* $t : \mathsf{Site}(H, r, J, K)$ *with* $s \simeq t$*.*

Proof. Induct on s. For a shallow site use the shallow correspondence constructor and the proved equality of corresponding signatures. For an inside site, extract the corresponding target box and a body isomorphism using the preceding lemma. Apply the induction hypothesis to the inner site and that body isomorphism, then assemble the outer correspondence with the inside constructor. This follows a finite address down the hierarchy and therefore terminates. □

Lean: `Linear.Site.transport_exists`

Definition 25.9 (Chosen site transport)**.** Write s^f for a site chosen from this existence theorem by classical choice. It has the original local depth and interface in the target host, and satisfies $s \simeq s^f$ by the specification theorem for choice.

The construction is noncomputable. No uniqueness, definitional round-trip law, or canonical choice of inner box isomorphisms is asserted.

Lean: `Linear.Site.transport`

Lean: `Linear.Site.transport_spec`

25.3 Substitution and freshness under transport

Theorem 25.10 (Simultaneous host and replacement congruence)**.** *If* $s \simeq t$ *and* $A \cong_r B$*, then*

$$\mathsf{plugAt}(s, A) \cong_d \mathsf{plugAt}(t, B).$$

Proof. First take the same replacement on both sides and induct on the structural correspondence. In the shallow case rewrite plugging at a transported interface using its equation, then apply shallow host congruence. Recursive label equivalence supplies the signature equalities and reflexivity required there. In the inside case the induction hypothesis supplies an isomorphism of the replaced inner bodies. Lift it through the corresponding outer boxes using box replacement congruence. This proves `SiteIso.plug`. For distinct isomorphic replacements, compose that isomorphism with the previously proved congruence in the replacement at the fixed target site t. □

Lean: `Linear.SiteIso.plug`

Lean: `Linear.SiteIso.plug_congr`

Lemma 25.11 (Quotient substitution agrees after host transport)**.** *For every replacement isomorphism class* $[A]$*,* $\mathsf{plugAtModuloIso}(s, [A]) = \mathsf{plugAtModuloIso}(s^f, [A])$*.*

Proof. Use quotient induction to choose a representative A. The specification of s^f gives a structural correspondence; substitution congruence gives a recursive isomorphism between the two results. Quotient soundness identifies their classes. □

This is invariance under transporting the host/site pair. Section 36 constructs a quotient of these pairs. Section 26 proves the nested composition and unit laws.

Lean: `Linear.plugAtModuloIso_transport`

Lemma 25.12 (Invariance of retained holes)**.** *A recursive host isomorphism preserves and reflects the holes outside a selected outer edge. Structurally corresponding sites retain exactly the same hole labels at every depth.*

Proof. An outer retained occurrence consists of an edge $x \neq e$ and a hole-observing position in its label. Map x by the host edge bijection and the position by the previously constructed label-fibre embedding. Injectivity preserves $x \neq e$, and the embedding preserves its hole observation. For reflection, use the inverse edge bijection and the inverse isomorphism's fibre embedding; the right inverse equation ensures that the preimage survives.

Interface transport does not change retention: eliminate the interface equalities. Now induct on structural site correspondence. The shallow case uses this equality elimination and the outer retained-hole equivalence. The inside case uses the definition of deep retention as a disjunction: either the occurrence lies under a surviving outer edge, or it is retained by the inner site. Apply the outer equivalence and the induction hypothesis to the two disjuncts. □

Lean: `Linear.DeepIso.retainedHole_iff`

Lean: `Linear.SiteRetainedHole_reinterface`

Lean: `Linear.SiteIso.retainedHole_iff`

Lemma 25.13 (Invariance of freshness and result validity)**.** *For $s \simeq t$ and $A \cong_r B$, exact freshness holds for (s, A) if and only if it holds for (t, B). The two substituted results satisfy global unique-hole validity together, whether or not either original host is valid.*

Proof. Freshness forbids a label occurring both in the retained host and in the replacement. Transport the first occurrence with retained-hole equivalence and the second with the replacement isomorphism's hole-inventory equivalence. This proves each direction. For result validity, simultaneous substitution congruence supplies a recursive isomorphism; apply its preservation and reflection of unique holes. No original-host validity hypothesis is needed for this invariance statement. □

Lean: `Linear.SiteIso.fresh_iff`

Lean: `Linear.SiteIso.plug_unique_iff`

25.4 A moved address and a wrong-address counterexample

Use the two-hole chain and its edge swap from the preceding section. The first hole, formerly edge zero, is edge one after renaming. Define the target deep site to enter the renamed box and select edge one. The outer boxed isomorphism and the shallow correspondence for the edge swap prove structural correspondence with the original deep site.

Lean: `Linear.Examples.renamedFirstSite`

Lean: `Linear.Examples.firstSites_correspond`

Simultaneous congruence now allows both the host body and the inserted chain to be renamed independently. The replacement consisting of hole one still collides with the retained hole one:

freshness invariance transports the previously proved failure of freshness to the new site.

Lean: `Linear.Examples.simultaneous_deep_renaming`

Lean: `Linear.Examples.renamed_collision_not_fresh`

For contrast within this example, define the wrong target site by keeping the old numerical name zero. It selects hole one instead of hole zero. After inserting the one-hole replacement, the wrong result retains hole zero, witnessed by the surviving edge one inside the renamed box. The exact deep inventory theorem therefore proves that hole zero is present. In the original substitution, hole zero is consumed and absent, as already proved. A recursive isomorphism would preserve hole presence, contradicting these two facts. Thus the wrong-address substitution is not recursively isomorphic to the original substitution; moving the address is essential.

Lean: `Linear.Examples.wrongRenamedSite`

Lean: `Linear.Examples.wrong_address_changes_result`

26 Associativity, units, and exact context composition

We now compose addressed substitutions, rather than merely compare renamed versions of one substitution. If s selects an occurrence in G, inserting H there creates a copy of every occurrence of H. A site t in H thus induces a site in $\mathsf{plugAt}(s, H)$. The main theorem identifies substitution at that induced site with first substituting within H and then inserting the result at s. The identification is a concrete recursive isomorphism; on isomorphism classes it is equality. We also prove the corolla unit laws and an exact context-validity criterion for the composite, including cases where an intermediate collision is removed by the final substitution.

26.1 Shallow associativity with explicit carrier bijections

Let C be a graph, e a selected edge, H a replacement of its interface, f an edge of H, and A a replacement of the interface of f. Compare

$$L = (C[e := H])[\mathsf{inr}(f) := A], \qquad R = C[e := H[f := A]].$$

These have the same outer interface, but different carrier records.

Definition 26.1 (Reassociation bijections)**.** The internal-vertex bijection from L to R is

$$(C_V + H_V) + A_V \longrightarrow C_V + (H_V + A_V),$$

which keeps each original name and changes only its sum tags. Its three cases send a host vertex to the host summand, a middle vertex to the middle summand, and a final vertex to the final summand. The edge bijection does the same for surviving host edges, surviving middle edges, and final inserted edges.

More explicitly, the left surviving-edge carrier consists of edges of $(C_E \setminus \{e\}) + H_E$ other than $\mathsf{inr}(f)$. A surviving host edge maps to the host summand on the right. An edge $\mathsf{inr}(y)$ maps to the surviving-middle summand, with $y \neq f$ obtained by injectivity of the sum constructor. Final edges map to final edges. The inverse maps reverse these three cases; inverse equations are proved by cases, with proof irrelevance identifying the subtype certificates.

Lean: `Net.PlugAssoc.vertices`

Lean: `Net.PlugAssoc.edges`

Lemma 26.2 (Attachment and port commutation)**.** *The source and target positions of the inserted edge* $\mathsf{inr}(f)$ *are the images, under insertion into* C*, of the corresponding positions of* f *in* H*. The reassociation port map commutes with inclusion of original host ports, inclusion of middle ports, and insertion of final replacement ports.*

Proof. The attachment assertions unfold positional lookup in the inserted incidence lists and use lookup through a list map. For host ports, case analysis on internal, input, and output ports proves the equation directly. For middle ports, internal vertices are immediate; an input or output is attached to a host port, so use the host equation. For final ports, internal vertices are again immediate. A boundary port attaches at f; rewrite that attachment by the positional lookup assertion and apply the middle-port equation. No argument assumes that an input attachment differs from an output attachment. □

Lean: `Net.PlugAssoc.sourcePort_inserted`

Lean: `Net.PlugAssoc.targetPort_inserted`

Lean: `Net.PlugAssoc.map_host`

Lean: `Net.PlugAssoc.map_middle`

Lean: `Net.PlugAssoc.map_insert`

Theorem 26.3 (Shallow associativity)**.** *There is a literal-label graph isomorphism* $L \cong R$*.*

Proof. Use the two reassociation bijections. Vertex and edge labels agree by the three carrier cases. For sources, split an edge into surviving host, surviving middle, or final replacement. In the host case the left source list maps the original host list through two inclusions and reassociation; combine list maps and apply host commutation. The middle case uses middle commutation, and the final case uses final insertion commutation. These give exactly the three right-hand source lists. The target proof is identical with target lists. The isomorphism preserves literal labels and is valid for self-loops and cycles. □

Lean: `Net.plug_assoc`

26.2 Composition at arbitrary box depth

Lemma 26.4 (Overwrite and insertion of box bodies)**.** *Replacing the same box body twice has exactly the effect of the last replacement. Changing the body of an inserted box commutes, up to literal-label isomorphism, with changing that body before shallow insertion.*

Proof. For overwrite, the two graphs have the same carriers, incidence, and internal labels. At the selected edge both final labels contain the last body. At any other edge both conditional updates keep the original label. Extensionality of the label function and proof irrelevance give equality of the graph records.

For commutation with insertion use identity carrier bijections. Surviving host labels are unchanged. On inserted edges the two tests for being the selected edge agree, because $\mathsf{inr}(x) = \mathsf{inr}(f)$ exactly when $x = f$. Incidence is unchanged by box replacement. The insertion functions also agree: internal ports have the same tags, and boundary ports attach to the same outer host positions. List mapping then proves both incidence equations. □

Lean: `Linear.replaceBox_overwrite`

Lean: `Linear.plug_replaceBox_iso`

Definition 26.5 (Composed site)**.** Let $s : \mathsf{Site}(G, r, J, K)$ and $t : \mathsf{Site}(H, q, U, V)$, where H has the replacement interface of s. Define

$$s \star t : \mathsf{Site}(\mathsf{plugAt}(s, H), q, U, V).$$

If s is shallow and t is shallow, select the inserted edge $\mathsf{inr}(f)$. If s is shallow and t enters a box, enter the corresponding inserted box and retain t's inner address. If s enters a box, rebuild that box with the first replacement already performed, then descend to the recursively composed inner site.

This is recursion on the outer address. The new enclosing box carries exactly the body produced by the first substitution, so the box-label equation is the selected branch of the update. Local depth and interface indices are preserved by construction. Although the surrounding graph operations use classical finite-subset constructions, no isomorphism or target address is selected by an existence theorem here.

Lean: `Linear.Site.comp`

Theorem 26.6 (Deep associativity)**.** *For every compatible s, t, A,*

$$\mathsf{plugAt}(s \star t, A) \cong_d \mathsf{plugAt}(s, \mathsf{plugAt}(t, A)).$$

No uniqueness, freshness, or non-self-loop premise is required.

Proof. Induct on s. When s and t are shallow use the explicit shallow associator and embed its literal-label isomorphism into recursive isomorphism. When s is shallow and t enters a box, use the box/insertion commutation lemma. When s enters a box, the left expression first updates that box to contain $\mathsf{plugAt}(s_{\mathrm{inner}}, H)$ and later updates it again after following the composed inner address. Overwrite reduces these two updates to one. The induction hypothesis compares its final body to the body obtained by performing the inner substitution before the outer one. Box replacement congruence lifts that body isomorphism through the enclosing box. □

Lean: `Linear.plugAt_assoc`

Lemma 26.7 (Quotient composition and three-stage coherence)**.** *On recursive isomorphism classes,*

$$P_{s\star t}([A]) = P_s(P_t([A])), \qquad P_s = \mathsf{plugAtModuloIso}(s, -).$$

For a third compatible site u, the two functions associated with $(s \star t) \star u$ and $s \star (t \star u)$ also agree on every replacement class.

Proof. For the binary equation use quotient induction and quotient soundness applied to deep associativity on representatives. For three stages, apply the binary equation twice to each side; both reduce to $P_s(P_t(P_u([A])))$. The ambient concrete hosts of the two composite addresses can differ, so this is a statement about the induced quotient results, not literal equality of address records in different hosts. □

Lean: `Linear.plugAtModuloIso_comp`

Lean: `Linear.plugAtModuloIso_comp_three`

The selected-hole observation of $s \star t$ equals that of t. Induct on s; for shallow s, either form of t retains the same selected label, and for inside s the definition discards the enclosing box and the induction hypothesis applies.

Lean: `Linear.Site.selectedHole_comp`

26.3 Exact inventories and context validity of composites

Write $R_s(h)$ for $\mathsf{SiteRetainedHole}(s, h)$ and $H_A(h)$ for $\mathsf{HasHole}(A, h)$.

Lemma 26.8 (Retained inventories compose by union)**.** *Removing an inserted shallow edge* $\mathsf{inr}(f)$ *retains exactly the holes under surviving original-host edges and surviving middle-graph edges. More generally,*

$$R_{s\star t}(h) \quad \Longleftrightarrow \quad R_s(h) \vee R_t(h).$$

Consequently the final hole inventory is exactly $R_s(h) \vee R_t(h) \vee H_A(h)$.

Proof. For shallow inserted-edge removal, decompose a retained occurrence by its outer edge tag. A surviving host tag already supplies its inequality with the original selected edge. An inserted tag survives precisely when its middle edge differs from f. Its fibre position and hole observation are unchanged. Conversely, construct the appropriate tagged edge from either of these witnesses.

For the general statement induct on s. A shallow s and shallow t give the preceding result directly. If t enters a box, append its retained inner inventory and reassociate disjunction. For inside s, replacement of its selected box preserves all retained outer-edge fibres; apply that fact and the induction hypothesis to the inner composite, then reassociate disjunction. Finally apply the exact inventory theorem for replacement at $s \star t$. □

Lean: `Linear.retainedHole_inserted`

Lean: `Linear.siteRetainedHole_comp`

Lean: `Linear.plugAt_comp_hasHole_iff`

Lemma 26.9 (Composite freshness)**.** *Exact freshness for* $(s \star t, A)$ *is equivalent to both*

$$\forall h,\ \neg(R_s(h) \wedge H_A(h)) \quad \textit{and} \quad \mathsf{SiteFresh}(t, A).$$

Proof. Use the retained-union theorem. In the forward direction include either retained inventory into the union and apply composite freshness. In the reverse direction, split a retained witness according to the union and apply the corresponding freshness assumption. □

Lean: `Linear.siteFresh_comp_iff`

Theorem 26.10 (Sharp context closure for composition)**.** *Suppose G and H have unique holes. Then* $\mathsf{plugAt}(s \star t, A)$ *has unique holes if and only if all three conditions hold:*

1. *A has unique holes;*
2. *A is fresh for the composite site $s \star t$;*
3. $\forall h,\ \neg(R_s(h) \wedge R_t(h))$.

Thus the three inventories R_s, R_t, H_A must be pairwise disjoint. The intermediate graph $\mathsf{plugAt}(s, H)$ *need not have unique holes.*

Proof. By deep associativity it suffices to check uniqueness of $\mathsf{plugAt}(s, \mathsf{plugAt}(t, A))$. Apply sharp context closure first with host G, then with host H. This gives uniqueness of A, freshness for (t, A), and disjointness of R_s from the hole inventory of $\mathsf{plugAt}(t, A)$. The latter inventory is $R_t \cup H_A$ by the exact inventory theorem. Splitting that union gives disjointness of R_s from R_t and from H_A. Combine the last condition with freshness for (t, A) using the composite-freshness lemma. Conversely, reverse each implication: the pairwise exclusions make the inner result fresh for s, and both applications of sharp closure establish the final result's uniqueness. □

Lean: `Linear.plugAt_comp_unique_iff`

Definition 26.11 (Certified composed plugging)**.** Given contexts G, H, A, compatible sites s, t, composite freshness, and disjoint retained inventories R_s, R_t, package $\mathsf{plugAt}(s \star t, A)$ with the uniqueness proof just obtained. This defines `Context.plugAtComp`; it requires no certificate that the intermediate plugged graph is a context.

Lean: `Linear.Context.plugAtComp`

26.4 Corolla units

Definition 26.12 (Substitution unit)**.** For a signature label o, let $\mathbf{c}_o$ be its one-edge corolla, with its sole edge selected. This is a substitution unit of interface $(\sigma_{\text{in}}(o), \sigma_{\text{out}}(o))$. It is an addressed graph, with no uniqueness premise on its label; no free wire identifying an input vertex with an output vertex is introduced.

We prove $\mathbf{c}_o[0 := H] \cong H$ and $C[e := \mathbf{c}_{\ell(e)}] \cong C$ using literal-label isomorphisms.

A corolla has no surviving edge after removing zero: an element of $\{i : \mathsf{Fin}(1) \mid i \neq 0\}$ contradicts the fact that every index below one is zero. Its ith source and target attachment positions are its input and output ports at i, respectively. These equations follow by unfolding lookup in the mapped finite-range lists.

Lean: `Net.PlugUnit.noEdge`

Lean: `Net.PlugUnit.corolla_sourcePort`

Lean: `Net.PlugUnit.corolla_targetPort`

Lemma 26.13 (Left unit)**.** *Replacing the sole edge of* $\mathbf{c}_o$ *by* H *yields a graph isomorphic to* H*.*

Proof. The vertex bijection discards the empty left summand in $\varnothing + H_V$. The edge bijection discards the impossible surviving-corolla summand and retains H_E. Their inverses include into the right summands. Both inverse laws follow by cases, eliminating impossible elements.

Mapping an inserted port through the vertex projection gives the original port: internal ports are immediate; input and output ports use the corolla attachment equations. For each inserted edge, combine the incidence list maps and apply this port equation, leaving the original source or target list. Surviving corolla edges are impossible. All labels agree directly on the remaining summands. □

Lean: `Net.PlugUnit.leftVertices`

Lean: `Net.PlugUnit.leftEdges`

Lean: `Net.PlugUnit.left_insert`

Lean: `Net.corolla_plug_iso`

Lemma 26.14 (Right unit)**.** *Replacing* e *in* C *by its own labelled corolla yields a graph isomorphic to* C*.*

Proof. The vertex bijection discards the empty right summand. The edge bijection maps each surviving host edge to itself and the inserted corolla edge to e. Its inverse tests whether an edge equals e: if so use the sole corolla edge; otherwise use the surviving-host subtype. The inverse laws follow from this test and uniqueness of an element of $\mathsf{Fin}(1)$.

The vertex projection cancels inclusion of host ports. Enumerating all source positions of e reconstructs C's source list: the lengths agree by typing, and each indexed element agrees by lookup in the finite-range map. The target list is reconstructed in the same way. Thus surviving-edge incidence agrees by cancellation of host inclusion; the inserted corolla's full boundary list maps to exactly the source or target list of e. Labels agree on both edge cases. This proof also covers an edge whose source and target share vertices. □

Lean: `Net.PlugUnit.rightVertices`

Lean: `Net.PlugUnit.rightEdges`

Lean: `Net.PlugUnit.right_host`

Lean: `Net.PlugUnit.sourcePort_list`

Lean: `Net.PlugUnit.targetPort_list`

Lean: `Net.plug_corolla_iso`

26.5 Restoration and unit equations at deep sites

Definition 26.15 (Original selected fragment)**.** The original fragment at a site s is the corolla carrying the label of its selected edge. For an inside site descend to its inner site. If the selected edge is itself a box, its original fragment includes that complete box label and its original body. The fragment has exactly the local depth and interface required by s.

Lean: `Linear.Site.original`

Lemma 26.16 (Deep restoration)**.** *Substituting the original fragment at any site restores the entire host up to recursive isomorphism.*

Proof. First note that reinstalling a box's original body gives a literal-label isomorphism with identity carriers: at the selected edge use its original box-label equality, and elsewhere no label changes. Incidence and internal labels are untouched.

Now induct on the site. For a shallow site apply the right unit. For an inside site, the induction hypothesis restores its inner host up to recursive isomorphism. Box congruence transports that isomorphism to the surrounding updated box. Compose it with the just-proved isomorphism for reinstalling the original body. □

Lean: `Linear.replaceBox_original_iso`

Lean: `Linear.plugAt_original`

Consequently, plugging the original fragment class at s returns the original host class, by quotient soundness. A selected one-edge corolla acts as identity on every replacement class of its interface: quotient induction reduces this to the left-unit isomorphism. Together with the composition equations above, these are unit and associativity laws for the induced substitution operations on recursive isomorphism classes. The quotient of pointed hosts and the independent-site interchange law are developed in Section 36.

Lean: `Linear.plugAtModuloIso_original`

Lean: `Linear.plugAtModuloIso_corolla`

26.6 Regression proofs: self-loops, mixed depth, and repaired collisions

For the concrete one-edge self-loop, instantiate shallow associativity with two unary corollas. Instantiate the right unit with the loop's own corolla as well. Both retain the loop's shared source/target vertex, demonstrating that neither theorem relies on a non-self-loop hypothesis.

Lean: `Graph.Examples.loop_associativity`

Lean: `Graph.Examples.loop_unit`

Define a unary variable fragment V as a corolla of the variable label. It has interface $* \to *$ and no holes: any alleged hole observation would equate `none` with `some`. Let t select the sole edge of the earlier replacement containing hole one. Its retained inventory is empty, because there is no other edge in its one-element carrier.

Lean: `Linear.Examples.variableFragment`

Lean: `Linear.Examples.variableFragment_holeFree`

Lean: `Linear.Examples.repeatedSite`

Lean: `Linear.Examples.repeatedSite_retained_absent`

Take the earlier boxed two-hole host and let s select hole zero inside it. Inserting the one-hole replacement carrying label one gives the already proved collision with the surviving hole one. This intermediate graph is invalid. Now fill the inserted occurrence at $s \star t$ with V. The composite criterion proves uniqueness: V is hole-free, hence unique and fresh; and the empty retained inventory of t is disjoint from the retained host inventory.

Lean: `Linear.Examples.intermediate_collision_repaired`

The final graph still contains the host's hole one, witnessed by the surviving inner edge. Hole zero is absent: it cannot be retained by s because it was the selected unique hole; it cannot be retained by t because that inventory is empty; and it cannot occur in V because V is hole-free. The exact composite inventory theorem exhausts these three possibilities.

Lean: `Linear.Examples.repaired_inventory`

For a mixed-depth example, form a one-edge shell whose label is the box label of the boxed two-hole host, and select that shell's sole edge. Insert the boxed host, then follow its deep first-hole site. Deep associativity compares this with filling the boxed host first and then inserting it into the shell.

Lean: `Linear.Examples.boxShell`

Lean: `Linear.Examples.shellSite`

Lean: `Linear.Examples.shallow_then_deep_associativity`

A three-stage instance adds the sole site of the repeated-hole replacement and then the variable fragment; the two site-composition bracketings agree on the resulting quotient class by three-stage coherence. Finally, restoring the original fragment at the boxed first-hole site recovers the original boxed host by deep restoration.

Lean: `Linear.Examples.three_stage_composition`

Lean: `Linear.Examples.deep_restoration`

27 Concrete focuses and the boundary argument of Lemma 39(1)

This section returns to the proof order of Appendix F. We formalize the boundary argument in Proof F.4: replacing focus-free fragments preserves the context's focus mode and whether its source is an input. The result holds for arbitrarily deep sites and finite sequences of fillings. It does not use output-closure, rootedness, determinism, or transitions. It supplies the initial/final agreement needed in Lemma 39(1), for the present substitution representation. Connecting it to the paper's full specimen definition is still an obligation; no concrete `SpecimenSystem` is constructed here.

27.1 Global focus freedom and a unique shallow focus

Definition 27.1 (Focus labels and focus freedom)**.** For a mode m, the symbol $\mathsf{focusLabel}(d, m)$ is the primitive focus symbol in the alphabet of depth d. At depth zero it uses the universe-lift wrapper; at a successor depth it uses the primitive summand. A label is *focus-free* if it is a primitive other than every focus symbol, or a box whose body has only focus-free labels. A graph is focus-free when all its edge labels satisfy this recursively defined condition. Thus the condition excludes focuses inside boxes at every depth, not merely at the outer layer.

Lean: `Linear.focusLabel`

Lean: `Linear.LabelFocusFree`

Lean: `Linear.FocusFree`

Definition 27.2 (Focussed hypernet)**.** A focussed hypernet consists of a hypernet G, a distinguished shallow edge f, its mode m, and the following certificates. The label of f is $\mathsf{focusLabel}(d, m)$; every other edge has a focus-free label; the source and target lists of f are respectively $[s]$ and $[t]$; $s \neq t$; and it is not the case that s is an input and t an output. The label of f is primitive, so has no box children. Consequently these certificates enforce exactly one focus globally, with the shallowness, self-acyclicity, and non-exposure required in §6.2. The explicit singleton incidence certificates record the endpoints without choosing positional casts.

Lean: `Linear.Focussed`

Lemma 27.3 (Focus-free addressed substitution)**.** *If G and H are focus-free, so is $\mathsf{plugAt}(s, H)$ for every site s in G with the interface of H.*

Proof. Induct on the site. At a shallow site, every surviving edge retains its old label, and every inserted edge has a label of H, so both kinds are focus-free. Inside a box, focus freedom of the host implies focus freedom of that box's body. Apply the induction hypothesis to the body. Replacing its label preserves focus freedom of that edge; every other outer label remains unchanged. □

Lean: `Linear.focusFree_plugAt`

Definition 27.4 (Boundary initiality and finality)**.** For a focussed hypernet, define Initial to mean that the mode is search and the focus source is an input. Define Final by replacing search with backtrack. These predicates are defined on any interface for use in the preservation lemmas. On states of type $* \to \epsilon$, there is exactly one input, giving the boundary characterizations used in §6.4 and Proof F.4. Finality is a syntactic predicate here; absence of outgoing transitions from final states has not yet been proved, because transitions on this explicit focus-edge state representation are not yet given by all UAM rules. Sections 28 and 30 prove search-only boundary laws in coordinates and on explicit states, respectively.

Lean: `Linear.Focussed.Initial`

Lean: `Linear.Focussed.Final`

27.2 Preservation through one filling

Lemma 27.5 (Shallow filling preserves the focussed structure)**.** *Let G be focussed at f and let $e \neq f$. Replacing e by a focus-free H produces a focussed hypernet with the same mode.*

Proof. The new focus is the retained edge $\mathsf{inl}(f)$ with unchanged label. Every other retained edge inherits focus freedom from G; every inserted edge inherits it from H. Its endpoints are the host images of s, t, and its incidence lists are their singleton lists by functoriality of list mapping. The host map is injective, so these endpoints remain distinct. It maps internal ports to internal ports and fixes input and output constructors. Therefore exposure cannot be introduced. □

Lean: `Linear.Focussed.plug`

Lemma 27.6 (Shallow boundary preservation and agreement)**.** *A shallow filling as above is initial exactly when its host is initial, and final exactly when its host is final. Any two focus-free fillings of the same selected edge therefore agree on initiality and finality.*

Proof. The focus mode is unchanged. The host image of its source is an input if and only if the original source is an input, by case analysis on the three port constructors. Substitute these facts into the two boundary predicates. For two fillings, compose each equivalence with the inverse equivalence for the other filling. This is exactly the argument of Proof F.4. □

Lean: `Linear.Focussed.plug_initial_iff`

Lean: `Linear.Focussed.plug_final_iff`

Lean: `Linear.Focussed.specimen_boundary_agreement`

Lemma 27.7 (Replacing a box body preserves an outer focus)**.** *Replacing a box body by a focus-free body of the same interface preserves a surrounding focussed hypernet.*

Proof. The box edge cannot be the focus edge: their labels belong to distinct summands of the alphabet. Keep the focus, its mode, and its two endpoints. All incidence lists are unchanged by box-body replacement, so the self-acyclicity and non-exposure certificates remain valid. The changed label is focus-free by the hypothesis on the new body, and all other non-focus labels are unchanged. □

Lean: `Linear.Focussed.replaceBox`

Definition 27.8 (A site avoiding the focus)**.** A shallow site avoids the focus when its selected edge differs from the focus edge. A site strictly inside a box automatically avoids the focus, since the focus is shallow and primitive. This condition does not assert that the selected edge is a hole: the preservation results intentionally apply to a larger class of replacements than paper specimens require.

Lean: `Linear.Focussed.Avoids`

Lemma 27.9 (Deep focussed filling and boundary agreement)**.** *A focus-free filling of any site avoiding the focus preserves the focussed structure and both boundary predicates. Consequently two such fillings of one site agree on initiality and finality.*

Proof. For a shallow site use the shallow filling construction. For a site inside a box, the box is distinct from the focus, hence its old body is focus-free. Focus-free addressed substitution proves the modified body focus-free. Apply the box-body preservation construction. In this second case the outer mode and endpoints remain literally the same, so both boundary equivalences are reflexive. Finally compose the equivalences for two fillings through their common host. □

Lean: `Linear.Focussed.plugAt`

Lean: `Linear.Focussed.plugAt_boundary_iff`

Lean: `Linear.Focussed.deep_specimen_boundary_agreement`

27.3 Finite fillings and the state space

Definition 27.10 (Finite filling sequence)**.** A filling sequence starts with a focussed graph and consists of finitely many addressed substitutions of focus-free graphs at sites avoiding the current focus. The constructors are the empty sequence and extension by one filling. Sites may occur at any box depth. Each intermediate focus is the certified focus transported by the preceding construction. In particular, successive fillings of a context's holes are included whenever their sites are supplied. This sequence representation does not by itself identify all schedules modulo permutation. Finite original-hole tuple application is defined separately in Section 43.

Lean: `Linear.Focussed.Fillings`

Theorem 27.11 (Boundary agreement for finite fillings)**.** *Every finite filling sequence preserves initiality and finality. If two sequences start from the same focussed context, their endpoints agree on both predicates.*

Proof. Induct on the sequence. The empty case is reflexivity. In the extension case, compose the one-step boundary equivalence with the induction hypothesis. For two sequences, compose the resulting equivalences through their common initial context. No template relation between replacements is necessary. Thus once concrete specimens supply these filling witnesses, the initial/final conclusion of Lemma 39(1) follows without an extra assumed locality law. □

Lean: `Linear.Focussed.fillings_boundary_iff`

Lean: `Linear.Focussed.fillings_boundary_agreement`

Definition 27.12 (Concrete state)**.** A state packages a hypernet of type $* \to \epsilon$, a focussed structure on that hypernet, and a proof that it contains no holes at any depth. This is the state space of §6.4 in our graph representation. It is not yet equipped with a `Machine`: Section 30 now supplies an erasure-based search relation on this state space, but graph-rewrite adequacy, substitution, and compute transitions remain open.

Lean: `Linear.State`

27.4 Nonvacuous checks

The regression graph has an input, an internal vertex, and an output, with a focus edge from the input to the internal vertex followed by a unary hole. It is obtained by relabelling the existing two-edge chain, preserving all signatures. Its focus is certified directly: the only other edge is the hole, the endpoints differ, and its target is internal, so the focus is not exposed.

Lean: `Linear.Examples.focusChain`

Lean: `Linear.Examples.focusChain_focussed`

The unary variable corolla is focus-free because a variable label cannot equal a focus label. Filling the second edge of the search chain by this corolla remains initial by shallow boundary preservation. The backtracking chain is final, whereas the search chain is not, since the two mode constructors differ.

Lean: `Linear.Examples.variableFragment_focusFree`

Lean: `Linear.Examples.search_filling_initial`

Lean: `Linear.Examples.boundary_modes_distinct`

Finally, boxing the search chain gives a graph whose outer edge is a box but whose body contains a focus. If it were focus-free, applying that predicate to the outer box edge and then the first body edge would say that the search focus label differs from itself, a contradiction. This checks that the definition really inspects nested boxes.

Lean: `Linear.Examples.boxed_focus_not_free`

28 The five search rules on underlying graph coordinates

The next operational dependency is Fig. 9(a–e). We now define these five rules on actual ordered incidences of a fixed hypernet and prove determinism and preservation under shallow hole filling. The token is represented by a mode and a vertex of the underlying graph. This follows the paper's observation that search leaves the underlying graph unchanged. The relationship to the explicit focus-edge representation is still an adequacy obligation: explicit insertion and incidence recovery are now proved in Section 29, and Section 30 supplies general erasure and search on explicit states. Equivalence with the paper's contextual graph rewrites remains open. The relation is defined even on graphs with holes; its intended state interpretation restricts the underlying graph to be focus-free, hole-free, and of type $* \to \epsilon$, with a token on a term vertex. The stronger structural lemmas below do not need these restrictions.

28.1 Operation kinds and certified incidence views

Definition 28.1 (Operation kind)**.** The two operation kinds are passive and active. A function from operations to these kinds specifies the partition from §5.1. Finishing a passive operation produces backtrack mode; finishing an active operation produces rewrite mode. Neither finishing mode is search, by distinction of constructors.

Lean: `Linear.OperationKind`

Lean: `Linear.OperationKind.finish`

Lean: `Linear.OperationKind.finish_ne_search`

The embedding of primitive labels into each depth alphabet is the universe lift at zero and the left summand at successor depth. It is injective: project the lift at depth zero, or use injectivity of the left injection at successor depth.

Lean: `Linear.primitiveLabel`

Lean: `Linear.primitiveLabel_injective`

Definition 28.2 (Search operation view)**.** A search operation view records an actual edge e with operation label o, its source s, and lists A, T of arguments and thunks. Its certificates assert

$$\mathsf{source}(e) = [s], \qquad \mathsf{target}(e) = A +\!\!+ T, \qquad |A| = \mathsf{arity}(o).$$

Thus A is exactly the ordered term-argument prefix. The ambient graph's signature typing already specifies the types of the remaining thunk targets. This view exposes certified data; it adds no machine-law assumption.

Lean: `Linear.SearchOperation`

Lemma 28.3 (Existence and uniqueness of operation data)**.** *Every shallow operation edge has such a view. Two views of the same edge have the same operation, source, and argument list. Every argument is a target of the edge, and the argument list has no repeated vertices.*

Proof. Signature typing gives a singleton source list and a target list at least as long as the arity. Choose its unique source; take the first arity-many targets for A and drop that prefix for T. The take/drop identity gives the target certificate, and the length bound gives the arity certificate. For uniqueness, the edge label determines the operation by injectivity of

primitive labels. Equality of singleton source lists determines the source. The two argument lists have the same length; taking that many entries from the equal target lists identifies them. Argument membership follows by inclusion of the left append summand. No repetitions follows by restriction of the whole target list's no-repetition certificate. □

Lean: `Linear.SearchOperation.exists_view`

Lean: `Linear.SearchOperation.data_unique`

Lean: `Linear.SearchOperation.argument_mem`

Lean: `Linear.SearchOperation.arguments_nodup`

28.2 Local outcomes and the search relation

Definition 28.4 (Entry and return outcomes)**.** Entering an operation with argument list A yields search at its first argument if A is nonempty. If A is empty, it yields the operation's finishing mode at its source. Returning from argument index i yields search at argument $i + 1$ if that entry exists; otherwise it yields the finishing mode at the source. Indices are zero-based. Return transitions additionally require that argument i exists, so the last case cannot be reached by returning from an out-of-range index.

Lean: `Linear.SearchOperation.enter`

Lean: `Linear.SearchOperation.resume`

These outcomes are independent of the chosen view of an edge. Substitute the operation, source, and argument equalities from data uniqueness into their definitions; the resulting expressions are identical.

Lean: `Linear.SearchOperation.enter_unique`

Lean: `Linear.SearchOperation.resume_unique`

Definition 28.5 (Search position and step)**.** A search position is a pair (m, v) of mode and outer graph vertex. The search relation has three constructors covering the five rules:

1. At a variable edge's singleton source v, search becomes rewrite at v (Fig. 9a).
2. At an operation's source, search yields its entry outcome (Fig. 9b for nonempty arguments and Fig. 9e for empty arguments).
3. Backtracking at argument i yields its return outcome (Fig. 9c when another argument exists and Fig. 9d when it does not).

Every edge mentioned is shallow. Box contents and thunk targets are not searched by these rules. No rule changes the underlying graph, and this relation contains neither substitution nor extrinsic computation steps.

Lean: `Linear.SearchPosition`

Lean: `Linear.SearchStep`

The four named operation rules specialize the two grouped constructors by rewriting the

argument list or the lookup of the next argument. Together with the variable constructor, they expose precisely the five cases of Fig. 9(a–e).

Lean: `Linear.SearchStep.first_argument`

Lean: `Linear.SearchStep.nullary`

Lean: `Linear.SearchStep.next_argument`

Lean: `Linear.SearchStep.last_argument`

28.3 Determinism and initial/final boundaries

Theorem 28.6 (Search determinism)**.** *Two search steps with the same input coordinates have the same output coordinates.*

Proof. Case-split on their rule constructors. Search and backtrack inputs cannot agree. For two variable steps the result is rewrite at the same vertex. For a variable step and an operation-entry step, uniqueness of the edge having the given source forces their edges equal, contradicting their distinct primitive labels. For two operation-entry steps, source incidence uniqueness again identifies the edges; entry-outcome uniqueness identifies the results. For two return steps, both starting vertices occur in their operation target lists. Target incidence uniqueness identifies the edges and hence their argument lists. Since these lists have no repetitions, equal arguments force equal indices. Return-outcome uniqueness then identifies the results. The more general helper permits two separately named input coordinates with an explicit equality; the deterministic theorem specializes that equality to reflexivity. No determinism hypothesis is assumed. □

Lean: `Linear.SearchStep.result_unique`

Lean: `Linear.SearchStep.deterministic`

Lemma 28.7 (Search boundary laws)**.** *A rewrite-mode coordinate has no search step. A backtracking coordinate at an input has no search step. No search step yields search mode at an input.*

Proof. No constructor starts in rewrite mode. A step from backtrack must return from an argument, which is a target of an edge. The graph's incidence conditions forbid an input vertex from being a target. For the last assertion, a variable step ends in rewrite, and finishing an operation never produces search. Therefore a search-mode result must be an argument selected on entry or on return. It is again an edge target and cannot be an input. Apply this stronger non-input assertion to rule out any proposed step into an initial coordinate. □

Lean: `Linear.SearchStep.no_rewrite`

Lean: `Linear.SearchStep.final_no_step`

Lean: `Linear.SearchStep.search_target_not_input`

Lean: `Linear.SearchStep.no_initial_target`

The finality assertion here concerns search steps only. It is not yet the paper's assertion that final states have no transition of any kind; that also requires the substitution and compute rules and the focus-edge bridge.

28.4 Preservation under context filling

Lemma 28.8 (Transport of retained operation data)**.** *Replacing an edge other than an operation edge retains a view of that operation. Entry and return outcomes commute with the host port map.*

Proof. Keep the operation label and use the surviving-edge injection. Map the source, argument list, and thunk list through the host map. Mapping a singleton and an append gives the two incidence certificates, and mapping preserves argument length. Entry commutation follows by the empty/nonempty cases of the argument list. Return commutation follows because lookup in a mapped list is the mapped optional lookup; split according to whether the next argument exists. Modes are unchanged by the port map. □

Lean: `Linear.SearchOperation.plug`

Lean: `Linear.SearchOperation.plug_enter`

Lean: `Linear.SearchOperation.plug_resume`

Theorem 28.9 (Existing context search steps survive filling)**.** *Suppose G has a shallow hole edge h and a search step $p \to q$. For every interface-compatible filling H, the graph $G[h := H]$ has a search step from the host image of p to the host image of q.*

Proof. In every rule, the interacting edge is a variable or operation and therefore cannot equal the hole edge. In the variable case, retain that edge, its label, and its mapped singleton source to apply the same rule. In the entry case, use the retained operation view and entry commutation. In the return case, lookup commutes with mapping, so the old argument witness supplies the mapped argument witness; apply return commutation to identify the result. □

Lean: `Linear.SearchStep.plug`

This theorem starts with a step of the context itself. Lemma 39(3) instead starts with a step of a filled specimen and must establish that it comes from the context. Extracting that step requires the entering/exiting analysis, rootedness, output-closure, and the focus-edge bridge. The preservation theorem proves one needed direction of locality, not the whole lemma.

28.5 Concrete checks of rule selection

The fixture has one operation with two term arguments and one thunk. Its graph is a corolla with targets $0, 1, 2$ in that order; the operation view certifies that the argument list is $[0, 1]$ and the thunk list is $[2]$. The incidence certificates are checked by computation.

Lean: `Linear.SearchExamples.ops`

Lean: `Linear.SearchExamples.graph`

Lean: `Linear.SearchExamples.operation`

Entry selects target 0 and return from target 0 selects target 1. Return from target 1 finishes at the source in the declared mode, for either operation kind. These are direct applications of the corresponding rule theorems.

Lean: `Linear.SearchExamples.enter_first`

Lean: `Linear.SearchExamples.return_next`

Lean: `Linear.SearchExamples.finish_skips_thunk`

A hypothetical return step to search at thunk target 2 would, by determinism, have the same result as the finishing step. It would identify the finishing mode with search, which is impossible. This checks that the full target list has not accidentally been used as the argument list.

Lean: `Linear.SearchExamples.cannot_search_thunk`

The existing nullary value corolla admits the empty argument view and finishes by backtracking when declared passive. The unary variable corolla uses the variable constructor and triggers rewrite without moving the coordinate.

Lean: `Linear.SearchExamples.value_finishes`

Lean: `Linear.SearchExamples.variable_triggers`

29 Explicit focus insertion and realization of typed search positions

This section advances the operational obligation in the remaining-work list. A term-vertex coordinate now determines an actual focussed hypernet, and on a hole-free graph of type $* \to \epsilon$ it determines an actual state. The construction proves all incidence and typing laws, rather than assuming a realization interface. Contracting the inserted focus recovers the original ordered incidence lists. Search preserves the term-vertex condition, so every typed coordinate search step has two such state realizations.

This section proves the insertion direction of the bridge. Section 30 extends it to arbitrary closed focussed states by general erasure and an erasure-then-insertion isomorphism. Neither section yet identifies the coordinate relation with the paper's contextual interaction rules; the endpoint theorem below must not be read as that adequacy statement. Substitution, compute transitions, and the concrete specimen interpretation also remain open.

29.1 Splitting a vertex

Definition 29.1 (Focus insertion carriers and maps)**.** Let G be a hypernet and let v be a term vertex that is not an output. Extend the internal-vertex carrier from V to $V + 1$ and the edge carrier from E to $E + 1$. Write z for the fresh internal vertex, ι for the inclusion of old ports, and f for the fresh edge. Boundaries are unchanged. Define

$$S_v(w) = \begin{cases} z & w = v, \\ \iota(w) & w \neq v, \end{cases} \qquad C_v(\iota(w)) = w, \quad C_v(z) = v.$$

Here S_v redirects old source occurrences of v, while C_v contracts the new focus. Old internal vertices retain their wire labels and z has label term. The use of a singleton carrier gives explicit finite enumerations.

Lean: `Linear.FocusInsertion.Vertex`

Lean: `Linear.FocusInsertion.host`

Lean: `Linear.FocusInsertion.fresh`

Lean: `Linear.FocusInsertion.collapse`

Lean: `Linear.FocusInsertion.split`

Lean: `Linear.FocusInsertion.internalLabel`

Lemma 29.2 (Map properties)**.** *The inclusion ι is injective and never reaches z. The contraction satisfies $C_v\iota = \mathrm{id}$ and $C_vS_v = \mathrm{id}$; consequently S_v is injective. No image of S_v equals $\iota(v)$. Every new port is either an old port's image or z. Both inclusion and redirection preserve wire labels.*

Proof. Case analysis on internal, input, and output constructors proves the inclusion claims; on internals use injectivity and disjointness of sum tags. For C_vS_v, split on $w = v$: the fresh port contracts to v in the equality case, and otherwise contraction undoes inclusion. Applying C_v to equal redirection images proves injectivity. An image cannot equal $\iota(v)$: in the equality case it is fresh; otherwise inclusion injectivity would force $w = v$. The port classification uses the two internal sum cases and the boundary constructors; every inhabitant of the fresh singleton

carrier equals its sole member. Inclusion preserves labels by cases. Redirection preserves labels because only v changes image, and both v and z have term label. □

Lean: `Linear.FocusInsertion.collapse_host`

Lean: `Linear.FocusInsertion.host_ne_fresh`

Lean: `Linear.FocusInsertion.host_injective`

Lean: `Linear.FocusInsertion.collapse_split`

Lean: `Linear.FocusInsertion.split_injective`

Lean: `Linear.FocusInsertion.split_ne_host`

Lean: `Linear.FocusInsertion.vertex_cases`

Lean: `Linear.FocusInsertion.host_label`

Lean: `Linear.FocusInsertion.split_label`

Definition 29.3 (Inserted incidence lists)**.** For every old edge e, use $S_v(\mathsf{source}_G(e))$ for its source list and $\iota(\mathsf{target}_G(e))$ for its target list, mapping entry by entry. The fresh edge has source $[\iota(v)]$, target $[z]$, and the chosen focus mode as label. Old edge labels, including whole boxed bodies, are unchanged.

Lean: `Linear.FocusInsertion.source`

Lean: `Linear.FocusInsertion.target`

Lemma 29.4 (Incidence uniqueness and coverage)**.** *The inserted incidence lists satisfy source and target uniqueness. A port occurs in a source list exactly when it is not an output, and in a target list exactly when it is not an input.*

Proof. For two old source edges, pull equality of mapped vertices back along the injective S_v and apply source uniqueness in G. An old source cannot meet the fresh edge's source because S_v avoids $\iota(v)$. There is only one fresh edge. The target proof is identical with inclusion in place of redirection and disjointness from z in place of avoidance of $\iota(v)$.

For source coverage, all mapped old sources are non-outputs: a redirected one is internal, and any other one retains its old boundary kind. The fresh edge's source is a non-output by the hypothesis on v. Conversely, an old image $\iota(w)$ with $w = v$ is sourced by the fresh edge. If $w \neq v$, use coverage in G and its retained source edge. The fresh port z is sourced by the old edge that sourced v, which exists because v was not an output. For target coverage, retained targets remain non-inputs and the fresh target is internal. Conversely an old non-input image inherits a target edge from G, and z is targeted by the fresh focus edge. □

Lean: `Linear.FocusInsertion.source_unique`

Lean: `Linear.FocusInsertion.target_unique`

Lean: `Linear.FocusInsertion.source_coverage`

Lean: `Linear.FocusInsertion.target_coverage`

Theorem 29.5 (Well-formed explicit focus insertion)**.** *The construction gives a finite typed hypernet with the same external interface as G.*

Proof. Enumerate each extended carrier by the sum of the old enumeration and the singleton enumeration. Old incidence lists retain their no-repetition property because both maps are injective; the new lists are singletons. Use the proved uniqueness and coverage lemmas. For typing, mapping a port list and then reading labels is the same as reading the old labels, by the label-preservation laws. The new source and target both have term label, exactly the focus signature. These checks populate every field of the concrete net record. □

Lean: `Linear.FocusInsertion.insert`

29.2 Focus certification, contraction, and holes

Theorem 29.6 (Unique focus and contraction)**.** *If G is globally focus-free, the inserted graph is focussed in the sense of §6.2. Contracting the new focus restores each original source and target list.*

Proof. Choose the fresh edge as focus. Every old label remains globally focus-free; the singleton fresh-edge carrier has no second member. The focus endpoints are $\iota(v)$ and z, which are distinct. Its target is internal, so the focus is not exposed. Its incidence lists and mode are those specified by construction. To recover source lists, compose their map by S_v with the map by C_v and use $C_v S_v = \mathrm{id}$. For target lists use $C_v \iota = \mathrm{id}$. These statements concern the retained edges; the focus itself is deleted when interpreting contraction. They establish recovery for the constructed family, not a general erasure operation on arbitrary focussed graphs. □

Lean: `Linear.FocusInsertion.focussed`

Lean: `Linear.FocusInsertion.collapse_sources`

Lean: `Linear.FocusInsertion.collapse_targets`

Lemma 29.7 (Hole freedom and boundary observations)**.** *If G is hole-free at every depth, so is its focus insertion. The explicit focus is initial exactly when its mode is search and v is an input; it is final exactly when its mode is backtrack and v is an input.*

Proof. Represent a hypothetical hole occurrence by an outer edge and a label-local position. If the edge is retained, its unchanged label gives a hole occurrence in G, a contradiction. If it is the fresh edge, its focus label is not a hole and has no children. This covers zero and successor depth alphabets. For boundary observations, the mode is unchanged and the source is $\iota(v)$. Inclusion preserves input status, by cases on the port constructor. □

Lean: `Linear.FocusInsertion.holeFree`

Lean: `Linear.FocusInsertion.boundary_iff`

29.3 Search preserves term typing

Lemma 29.8 (Typed operation data)**.** *An operation view's source has term label. Mapping its argument list through vertex labels gives exactly the list of arity-many term labels. Consequently every argument vertex has term label.*

Proof. Rewrite the operation's source-typing certificate by its singleton source and operation label; the singleton signature forces the source label to be term. For arguments, rewrite the target-typing certificate using the argument/thunk append decomposition. Take an argument-length prefix on both sides. The arity certificate ensures that this is exactly the entire argument-label list on one side and the term prefix of the signature on the other. Finally map membership of an argument into this repeated-term list: every member equals term. □

Lean: `Linear.SearchOperation.source_term`

Lean: `Linear.SearchOperation.arguments_typed`

Lean: `Linear.SearchOperation.argument_term`

Theorem 29.9 (Term typing preservation)**.** *If a search step starts at a term vertex, its result is at a term vertex.*

Proof. A variable trigger does not move, so inherits the premise. Operation entry selects either the first argument or the source when there are no arguments. Operation return selects either the next argument or the source on completion. Apply the operation data lemmas in each case. Neither operation case needs the premise on the old vertex; its result's type follows from graph typing alone. □

Lean: `Linear.SearchStep.preserves_term`

29.4 Actual states from typed coordinates

Definition 29.10 (Typed position and realization)**.** For a graph $G : * \to \epsilon$, a typed position is a search coordinate whose vertex is certified to have term label. Such a vertex cannot be an output, since the output index type is empty. If G is globally focus-free and hole-free, realize a typed position by inserting its mode at its vertex and packaging the resulting graph, unique-focus certificate, and hole-freedom certificate into the concrete state record.

Lean: `Linear.TypedPosition`

Lean: `Linear.TypedPosition.not_output`

Lean: `Linear.TypedPosition.realize`

Definition 29.11 (Initial attachment)**.** The initial typed position is search at the sole input, whose wire label is term by the interface. Realizing it is an initial state: the constructed focus has search mode and its source is that input. More generally, the realized state's initial and final predicates coincide with their coordinate boundary predicates, by the insertion boundary lemma.

Lean: `Linear.TypedPosition.initial`

Lean: `Linear.TypedPosition.initial_realize`

Lean: `Linear.TypedPosition.realize_boundary`

This discharges the graph-construction part of initial attachment. Proposition 38 still needs the concrete observation interface and specimen witnesses required by `initial_lift`.

Definition 29.12 (Advancing a typed position)**.** Given a search step from a typed position, equip its result coordinate with the term-typing proof supplied by preservation. This gives the next typed position on the same underlying graph.

Lean: `Linear.TypedPosition.advance`

Theorem 29.13 (Realization of search endpoints)**.** *For every search step from a typed position on a focus-free, hole-free graph $G : * \to \epsilon$, realize the initial position and its typed successor as states s, t. Their explicit focus modes are the corresponding coordinate modes. At each endpoint, collapsing the inserted focus recovers every original edge's source and target list in G.*

Proof. Use the realization constructor for the input and for the typed successor. Their focus modes are correct by the definition of insertion. Apply the two incidence-recovery theorems at each coordinate. The common underlying graph is unchanged; only the position and mode of the inserted focus may differ. The statement supplies actual state endpoints and recovery certificates, not a new definition of a contextual graph-rewrite step between arbitrary states. □

Lean: `Linear.TypedPosition.step_realizes`

29.5 Boundary and interior regression checks

The existing value corolla is focus-free because its operation label cannot be a focus label. Realizing its initial typed position gives the explicit state `initialValue`. Advancing by its passive nullary search step and realizing the result gives `finalValue`. Their initial and final certificates follow from their modes and common input source. The initial state's distinct focus endpoints are guaranteed by its focussed certificate.

Lean: `Linear.Examples.value_focusFree`

Lean: `Linear.Examples.initialValue`

Lean: `Linear.Examples.finalValue`

Lean: `Linear.Examples.value_boundary_states`

Lean: `Linear.Examples.initial_value_has_distinct_endpoints`

The two-hole chain is also globally focus-free, because neither hole label can be a focus. Insert search at its internal vertex: the first edge still targets the old vertex, the second edge now starts at the fresh vertex, and the new focus connects the old vertex to the fresh one. Unfolding the incidence maps proves all four equalities. This focus is not initial because its source is internal. Unlike the value example, this test deliberately retains holes and therefore tests focussed hypernets rather than closed states.

Lean: `Linear.Examples.twoHoles_focusFree`

Lean: `Linear.Examples.interior_insertion`

Lean: `Linear.Examples.interior_not_initial`

30 Focus insertion, erasure, and recovery

We now erase the focus of an arbitrary focussed graph with no outputs and prove that reinsertion recovers the original graph and focus up to isomorphism. Unlike the previous section's recovery result, the input need not have been constructed by our insertion function. The converse round trip, insertion followed by erasure, recovers the underlying graph. These constructions support the state-search semantics developed in Section 31.

30.1 Contraction maps and the retained carriers

Fix a focussed graph $G : I \to \epsilon$, focus edge e_f, source s, and an internal vertex t equal to the focus target. We first work with this explicit witness, then prove that every such graph supplies it.

Definition 30.1 (General focus erasure maps)**.** Remove t from the internal carrier and e_f from the edge carrier, using subtypes of names unequal to the deleted names. Let j include the remaining ports into the old ports. If $w \neq t$, let $k(w)$ keep w with its non-deletion certificate. Self-acyclicity ensures $s \neq t$, so $p = k(s)$ is defined. Set

$$c(w) = \begin{cases} p & w = t, \\ k(w) & w \neq t, \end{cases} \qquad a(x) = \begin{cases} t & j(x) = s, \\ j(x) & j(x) \neq s. \end{cases}$$

The map c contracts the focus. The map a recovers the appropriate old *source* representative, choosing t at the joined port; j recovers the old *target* representative. Boundary ports and their order are unchanged.

Lean: `Linear.FocusErasure.Vertex`

Lean: `Linear.FocusErasure.lift`

Lean: `Linear.FocusErasure.keep`

Lean: `Linear.FocusErasure.position`

Lean: `Linear.FocusErasure.contract`

Lean: `Linear.FocusErasure.expand`

Lemma 30.2 (Port-map identities)**.** *The map j is injective, avoids t, and satisfies $j(k(w)) = w$. Moreover $j(p) = s$, $c(t) = c(s) = p$, $j(c(w)) = w$ for $w \neq t$, and $c(j(x)) = x$. The expansion $a(x)$ never equals s; it satisfies $a(c(w)) = w$ for $w \neq s$ and $c(a(x)) = x$.*

Proof. The inclusion and keeping claims follow by port-constructor cases; subtype extensionality gives injectivity on internal ports, and the subtype certificate excludes t. The inequality $s \neq t$ is exactly focus self-acyclicity rewritten by the chosen target witness. The identities for p, c follow by unfolding and splitting on whether the argument is t. Apply inclusion injectivity to derive $c(j(x)) = x$ from $j(c(j(x))) = j(x)$. For expansion, when $j(x) = s$ its result is $t \neq s$; otherwise it is already different from s. To prove $a(c(w)) = w$, split on $w = t$. At t, contraction lands at p and expansion returns t. Otherwise inclusion after contraction returns $w \neq s$, so expansion leaves it unchanged. For $c(a(x)) = x$, split on $j(x) = s$. In that case $x = p$ by inclusion injectivity; otherwise use $c(j(x)) = x$. □

Lean: Linear.FocusErasure.lift_keep

Lean: Linear.FocusErasure.lift_injective

Lean: Linear.FocusErasure.lift_ne_target

Lean: Linear.FocusErasure.source_ne_target

Lean: Linear.FocusErasure.lift_position

Lean: Linear.FocusErasure.contract_target

Lean: Linear.FocusErasure.lift_contract

Lean: Linear.FocusErasure.contract_lift

Lean: Linear.FocusErasure.contract_source

Lean: Linear.FocusErasure.expand_ne_source

Lean: Linear.FocusErasure.expand_contract

Lean: Linear.FocusErasure.contract_expand

Lemma 30.3 (Injectivity on retained incidence lists)**.** *A source of a retained edge cannot equal s, and a target of a retained edge cannot equal t. Contraction is injective on vertices other than s, and also on vertices other than t.*

Proof. If a retained edge sourced s, source uniqueness with the focus edge would identify the two edges, contrary to retention. Target uniqueness proves the other exclusion. For injectivity off s, apply a to equality of contracted ports and use $a(c(w)) = w$. For injectivity off t, apply j and use $j(c(w)) = w$. These are restricted injectivity assertions: c itself identifies the two distinct focus endpoints. □

Lean: Linear.FocusErasure.source_avoids

Lean: Linear.FocusErasure.target_avoids

Lean: Linear.FocusErasure.source_injective

Lean: Linear.FocusErasure.target_injective

Lemma 30.4 (Label preservation)**.** *Both focus endpoints have term label. Keeping an undeleted port preserves its label, and contraction preserves every port's label.*

Proof. Rewrite the focus source and target typing certificates using their singleton incidences and focus label; its signature is term-to-term. Keeping preserves labels by cases on internal and boundary ports. For contraction, all ports other than t use keeping. At t, its image is the kept source and both endpoints have the same term label. □

Lean: Linear.FocusErasure.source_term

Lean: Linear.FocusErasure.target_term

Lean: Linear.FocusErasure.keep_label

Lean: Linear.FocusErasure.contract_label

30.2 A valid focus-free underlying graph

Theorem 30.5 (General erasure construction)**.** *Delete e_f and t, retain every other edge label, and map all retained source and target lists through c. The result is a finite typed hypernet of the same interface $I \to \epsilon$.*

Proof. Finite subtype enumerations supply the carriers. Restricted contraction injectivity on retained sources and targets preserves no-repetition. Given a shared contracted source of two retained edges, restricted injectivity recovers a shared old source, so old source uniqueness identifies the edge names. The target proof is the same with the target restriction.

For source coverage, there are no output ports. Given an erased port x, use the old source coverage at $a(x)$. Since $a(x) \neq s$, its source edge cannot be the focus; contraction of that source gives $c(a(x)) = x$. For target coverage, a retained target w differs from t, so $j(c(w)) = w$ and its input status is unchanged. Conversely, if x is not an input, old target coverage at $j(x)$ gives an edge. It is not the focus because $j(x) \neq t$. Its contracted target is $c(j(x)) = x$. Finally, mapping contracted lists through labels equals reading the old labels, by label preservation, so both signature-typing certificates follow from G. □

Lean: `Linear.FocusErasure.erase`

Lemma 30.6 (Erased invariants)**.** *The erased graph is globally focus-free, its distinguished port p has term label, and erasure preserves global hole-freedom.*

Proof. Every retained edge was different from the unique focus, so its entire label, including any box body, is focus-free. The port p keeps the term-labelled source, hence has term label. A hypothetical hole in the erased graph lies at a retained edge with unchanged label and label-local position; the same occurrence would be a hole in the original graph. Thus a hole-free original has a hole-free erasure. □

Lean: `Linear.FocusErasure.erased_focusFree`

Lean: `Linear.FocusErasure.position_term`

Lean: `Linear.FocusErasure.erased_holeFree`

30.3 Reinsertion recovers an arbitrary focussed graph

Definition 30.7 (Restoring a deleted name)**.** For any type A and distinguished $b \in A$, the restoration bijection $(A \setminus \{b\}) + 1 \to A$ sends retained names to themselves and the fresh singleton name to b. Its inverse tests whether a name equals b. Applying this to t and e_f gives the vertex and edge bijections needed for reinsertion. Reinsert the original focus mode at p in the erased graph. The port p is a non-output because the output interface is empty, and its term-typing certificate was proved above.

Lean: `Linear.FocusErasure.restore`

Lean: `Linear.FocusErasure.position_not_output`

Lean: `Linear.FocusErasure.reinsert`

The bijection's inverse laws follow by retained/fresh cases; the fresh carrier has exactly one member. At ports, restoring host inclusion agrees with j, and restoring the fresh insertion

vertex gives t. Restoring the insertion's source-redirection map agrees with a: the test $x = p$ is equivalent to $j(x) = s$, by inclusion injectivity and $j(p) = s$.

Lean: `Linear.FocusErasure.restore_host`

Lean: `Linear.FocusErasure.restore_fresh`

Lean: `Linear.FocusErasure.restore_split`

Theorem 30.8 (Focus-preserving round trip)**.** *Reinserting after erasure is isomorphic to the original graph. The isomorphism restores the focus edge, source, and target, preserves boundary order, and preserves literal edge labels, including complete boxed bodies.*

Proof. Use the restoration bijections. Retained internal labels are unchanged; the fresh insertion vertex has term label, equal to the original label of t. Retained edge labels are unchanged, and the new focus label equals the original focus label by its certificate. For a retained source list, restoration after insertion-redirection after contraction acts as $a \circ c$, which is the identity on these sources because they avoid s. For a retained target list, restoration acts as $j \circ c$, which is the identity because these targets avoid t. Map these pointwise identities over the lists. The new focus source restores to $j(p) = s$, and its target restores to t, giving the original singleton incidence lists. The fresh edge restores to e_f by definition, and the preceding two port identities explicitly identify its endpoints. □

Lean: `Linear.FocusErasure.reinsert_iso`

Lean: `Linear.FocusErasure.reinsert_focus_edge`

Lean: `Linear.FocusErasure.reinsert_focus_endpoints`

Lemma 30.9 (Every closed focussed graph admits erasure)**.** *The focus target of any graph $G : I \to \epsilon$ is internal. Therefore every such focussed graph has a globally focus-free underlying graph, preserving hole-freedom, and reinserting its mode recovers it up to the proved isomorphism.*

Proof. The focus target occurs in a target list, so cannot be an input. There are no outputs. The remaining port constructor is internal. Choose its internal name with its equality certificate, and instantiate the preceding erasure, invariant, and round-trip constructions. This use of choice supplies a witness already proved to exist; it does not assume an erasure law. □

Lean: `Linear.Focussed.target_internal`

Lean: `Linear.Focussed.targetVertex`

Lean: `Linear.Focussed.targetVertex_spec`

Lean: `Linear.Focussed.underlying`

Lean: `Linear.Focussed.underlying_focusFree`

Lean: `Linear.Focussed.underlying_holeFree`

Lean: `Linear.Focussed.underlying_reinsert_iso`

The theorem proves the erasure-then-insertion direction for arbitrary closed focussed graphs. The opposite insertion-then-erasure isomorphism is proved in Section 30.5, and naturality under source isomorphisms in Section 31.6.

30.4 Focus-preserving isomorphisms and state recovery

Definition 30.10 (Focussed isomorphism)**.** A focussed isomorphism is a literal-label graph isomorphism together with equality of the two focus modes and an equation mapping the distinguished focus edge to the distinguished focus edge. Identity, inverse, and composition are obtained from the corresponding graph isomorphisms, using the edge bijection's inverse laws and composition for the focus equations.

Lean: `Linear.Focussed.Iso`

Lean: `Linear.Focussed.Iso.refl`

Lean: `Linear.Focussed.Iso.symm`

Lean: `Linear.Focussed.Iso.trans`

Lemma 30.11 (Endpoints and observations are invariant)**.** *A focussed isomorphism maps both focus endpoints correctly and preserves initiality and finality.*

Proof. Specialize the isomorphism's source-list equation to the focus edge and rewrite both lists as singletons. Singleton injectivity identifies the endpoints. Repeat for target lists. Port maps preserve input status; together with the mode equality, this yields both boundary-predicate equivalences. □

Lean: `Linear.Focussed.Iso.source`

Lean: `Linear.Focussed.Iso.target`

Lean: `Linear.Focussed.Iso.initial_iff`

Lean: `Linear.Focussed.Iso.final_iff`

Definition 30.12 (Extracted state coordinate)**.** For an arbitrary state s, erase its focus and take the typed position whose mode is the old mode and whose vertex is the joined port p. Its typing is the erased-position typing lemma. Re-realizing this coordinate in the globally focus-free, hole-free underlying graph yields a state focus-preservingly isomorphic to s, using the general round trip. Thus the input need not be a canonical insertion or carry an assumed representation certificate.

Lean: `Linear.State.coordinate`

Lean: `Linear.State.recover`

The extracted coordinate is initial or final exactly when s is. Compose the previous realization boundary equivalence with the focussed isomorphism's boundary equivalence.

Lean: `Linear.State.coordinate_boundary`

30.5 Insertion followed by erasure

Fix a focus-free graph $G : I \to []$, a term vertex v, and a mode m. Write $S = \mathrm{insert}(G, v, m)$ for the focussed graph constructed in Section 29. The preceding round trip started with an arbitrary focussed graph. We now establish the opposite direction, including the position at which the focus was inserted.

Definition 30.13 (Deleting a fresh singleton)**.** For any carrier A, deleting the fresh member from $A + \mathbf{1}$ leaves a carrier canonically bijective with A. The forward map sends a retained $\mathsf{inl}(a)$ to a; the inverse sends a to that retained element. An element of the right summand cannot survive, since its singleton index must be zero. Apply general erasure to the fresh focus and fresh target of S to define $E(S)$.

The inverse laws hold by cases on the summand: the retained case is reflexive, and the fresh case contradicts its non-deletion certificate.

Lean: `Linear.FocusInsertion.deleteFresh`

Lean: `Linear.FocusInsertion.erasedInsertion`

Lemma 30.14 (Cancellation on ports)**.** *Let D be the port map induced by deletion of the fresh singleton, j the inclusion of retained ports, and κ the collapse map of insertion. Then $D(x) = \kappa(j(x))$. The joined vertex maps to v, and for every port w of S, $D(c(w)) = \kappa(w)$, where c is general focus contraction.*

Proof. The first identity follows by cases on the retained port. An internal right-summand case is impossible; input ports are fixed. For the joined vertex, inclusion gives the focus source, namely the hosted copy of v; collapse sends it to v. Finally, if w is the fresh target, contraction gives the joined vertex and both sides equal v. Otherwise inclusion after contraction is w, and the first identity applies. □

Lean: `Linear.FocusInsertion.deleteFresh_port`

Lean: `Linear.FocusInsertion.erase_position`

Lean: `Linear.FocusInsertion.erase_contract`

Theorem 30.15 (Insertion–erasure cancellation)**.** *There is an isomorphism $E(S) \cong G$ preserving full labels and ordered incidences. The choice-based operation $S.\mathsf{underlying}$ has such an isomorphism too, and one can choose it to send the joined vertex to v.*

Proof. Use deletion of the singleton on both internal vertices and edges. All surviving labels are inherited. Sources were first split by insertion and then contracted by erasure; the port cancellation identity reduces their composite to collapse after split, which is the identity. Targets were hosted by insertion, so the analogous composite is collapse after hosting, also the identity. These pointwise identities give equality of ordered lists.

The target selected by general erasure is necessarily the fresh internal vertex: both it and the fresh vertex equal the focus target, and the internal port constructor is injective. Substituting this equality identifies the general underlying graph with $E(S)$. For the position-sensitive assertion, carry an arbitrary target together with its equality certificate, identify it with the fresh target, and use the explicit isomorphism and joined-port identity together. This keeps graph and coordinate recovery synchronized. □

Lean: `Linear.FocusInsertion.erase_iso`

Lean: `Linear.FocusInsertion.targetVertex_eq`

Lean: `Linear.FocusInsertion.underlying_iso`

Lean: `Linear.FocusInsertion.underlying_position_exists`

31 Search and state isomorphism classes

The focus recovery constructions define search on actual states. Search is deterministic up to focus-preserving isomorphism, final states have no search step, and no search step ends in an initial state. A concrete initial value state makes a search step to an actual final state.

We then prove source and target renaming invariance and descent to literal-label and recursive state isomorphism classes, where successor classes are unique. Section 41 now proves equivalence of erasure-based search with the paper's contextual local rewrites. Substitution and compute transitions for the full machine remain to be constructed.

31.1 Search between actual states

Definition 31.1 (Erasure-based state search)**.** For a fixed operation-kind function, write $s \to_{\mathrm{er}} t$ if there is a typed coordinate q on the underlying graph of s such that the extracted coordinate of s makes one of the five search steps to q, and the realization of q is focus-preservingly isomorphic to t. Thus the target may use arbitrary vertex and edge names. This is a relation on actual state records, rather than merely a construction of two endpoints from a separately supplied graph.

Lean: `Linear.State.Search`

Lemma 31.2 (Existence and target renaming)**.** *Every typed coordinate successor of an extracted state coordinate gives an actual state successor. Replacing a successor by a focus-preservingly isomorphic state preserves the step.*

Proof. For existence, realize the successor coordinate and use the identity focussed isomorphism as witness. For target renaming, retain the same coordinate step and compose its target isomorphism with the supplied isomorphism. □

Lean: `Linear.State.search_realize`

Lean: `Linear.State.search_target_iso`

Theorem 31.3 (Search determinism up to focussed isomorphism)**.** *Two erasure-based search successors of the same state are focus-preservingly isomorphic.*

Proof. Both coordinate steps use the same erased graph and the same extracted input. Coordinate search determinism identifies their output pairs; subtype extensionality identifies the typed positions. Their canonical realizations therefore coincide. Compose the inverse of the first target isomorphism with the second target isomorphism to identify the actual successors. □

Lean: `Linear.State.search_deterministic`

Lemma 31.4 (Explicit-state search boundary laws)**.** *A final state has no erasure-based search step, and no such step ends in an initial state.*

Proof. If the source is final, its extracted coordinate has backtrack mode at an input. Coordinate search cannot start there. If the target is initial, pull initiality back through the target's focussed isomorphism and the realization boundary equivalence. The successor coordinate would be search at an input, which the coordinate search boundary theorem excludes. □

Lean: `Linear.State.final_no_search`

Lean: `Linear.State.no_search_initial`

These are search-only laws. Neither all-transition finality nor the full robustness theorem follows until the remaining transition classes and specimen obligations are supplied.

31.2 Concrete execution and interior recovery

Erase the focus of the previously constructed initial value state. Its value edge survives and has an empty argument/thunk view. Its source contracts from the old focus target to the joined port, which is exactly the extracted state coordinate's vertex. This proves the view's source certificate; the other certificates follow from the nullary value signature.

Lean: `Linear.Examples.erasedValueOperation`

Apply the passive nullary rule, advance the typed coordinate, and realize it. The existence lemma gives an actual state search step. The result is final because its mode is backtrack and its vertex retains the input status of the initial coordinate. This proves that the new state relation has a concrete execution, rather than only universal implications about hypothetical steps.

Lean: `Linear.Examples.value_state_search`

For an interior test, relabel the old focus of the initial value graph as a variable, retaining the value edge and all incidences. Both labels exclude focuses and holes, so this is a focus-free, hole-free closed two-edge chain. Insert search at its internal vertex to obtain a state whose focus source is internal. It is not initial, directly from that source constructor. General state recovery erases and reconstructs it with a focussed isomorphism, including its distinguished focus and mode.

Lean: `Linear.Examples.closedChain`

Lean: `Linear.Examples.closedChain_focusFree`

Lean: `Linear.Examples.closedChain_holeFree`

Lean: `Linear.Examples.interiorState`

Lean: `Linear.Examples.interior_state_not_initial`

Lean: `Linear.Examples.interior_state_recovery`

31.3 Search coordinates respect renaming

Let $a : G \cong H$ be a literal-label isomorphism and write a_* for its port map. Internal wire labels are preserved by the isomorphism's label field; boundary labels are preserved because boundary positions are fixed.

Lean: `Linear.iso_vertexLabel`

Definition 31.5 (Transport of operation views and coordinates)**.** Transport an operation view by applying the edge bijection to its edge and a_* to its source and both ordered lists. Keep its operation symbol. Source and target certificates follow from the isomorphism's incidence laws; arity follows because mapping a list preserves length. Transport a typed coordinate (m, v) to $(m, a_* v)$. Wire-label preservation supplies its typing certificate.

Lean: `Linear.SearchOperation.rename`

Lean: `Linear.TypedPosition.rename`

Lemma 31.6 (Transport of the local rules)**.** *Operation entry and argument return commute with transport. Every coordinate search step transports forwards, and a step between transported coordinates exists if and only if the original step exists.*

Proof. Entry depends on whether the argument list is empty or has a first element; mapping preserves these alternatives. Return depends on the optional element at index $i + 1$; lookup in a mapped list is the mapped lookup. These facts prove the two local calculation identities.

Induct on the search rule. The variable rule uses the transported label and singleton source certificate. The operation-entry case uses the entry identity. For argument return, map the certified lookup at index i and use the return identity. This proves forward transport for all five rules. Apply the same result to a^{-1} for reflection: the composite port map is the identity, and neither map changes the mode. The same inverse law proves that transporting a typed coordinate by a and back by a^{-1} returns that coordinate; subtype extensionality discards the irrelevant typing certificates. □

Lean: `Linear.SearchOperation.rename_enter`

Lean: `Linear.SearchOperation.rename_resume`

Lean: `Linear.SearchStep.rename`

Lean: `Linear.SearchStep.rename_iff`

Lean: `Linear.TypedPosition.rename_symm`

31.4 Insertion respects renaming

Extend the vertex and edge bijections of a by the identity on each fresh singleton. The extended port map sends hosted vertices to hosted images and fixes the fresh vertex. It also commutes with splitting: injectivity gives $a_* w = a_* v$ exactly when $w = v$, so both splits select the fresh branch in exactly the same case.

Lean: `Linear.FocusInsertion.rename_host`

Lean: `Linear.FocusInsertion.rename_fresh`

Lean: `Linear.FocusInsertion.rename_split`

Theorem 31.7 (Naturality of insertion)**.** *Insertion at (m, v) in G is focus-preservingly isomorphic to insertion at $(m, a_* v)$ in H. In particular, realizing a typed coordinate commutes with its transport through a.*

Proof. Use the two extended bijections. Retained labels are preserved by a; fresh vertex and edge labels agree by construction. On retained edges, commutation with splitting proves the source equation and commutation with hosting proves the target equation, after applying the corresponding incidence equation of a. The fresh edge has exactly the hosted insertion position as source and the fresh vertex as target, so the two port identities prove its equations. The mode is unchanged, and the fresh focus edge maps to itself. Packaging these fields gives a focussed isomorphism. Applying this construction to typed coordinates gives the realization result; focus-free and hole-free certificates ensure both endpoints are states. □

Lean: `Linear.FocusInsertion.rename_iso`

Lean: `Linear.FocusInsertion.rename_focussed`

Lean: `Linear.TypedPosition.realize_rename`

31.5 Exact characterization of search from realized states

Lemma 31.8 (Recovery of a realized coordinate)**.** *If p is a typed coordinate of a focus-free, hole-free graph G, there is an isomorphism a from the underlying graph of its realization to G such that $a_* \,\mathrm{coordinate}(\mathrm{realize}(p)) = p$.*

Proof. Use the position-sensitive insertion–erasure isomorphism. It restores the vertex, while the extracted mode is definitionally the original mode. Equality of these two components gives equality of coordinates and then of the typed subtypes. □

Lean: `Linear.TypedPosition.coordinate_recovery`

Theorem 31.9 (Coordinate characterization)**.** *For every actual target state t,*

$$\text{realize}(p) \longrightarrow t \quad \Longleftrightarrow \quad \exists q, \quad p \longrightarrow_G q \ \wedge \ \text{realize}(q) \cong_f t.$$

Here the left relation is erasure-based state search, the middle relation is coordinate search on the original graph G, and $\cong_f$ preserves the focus and mode. Both realizations use the same original graph G.

Proof. Choose the recovery isomorphism a from the previous lemma. For the forward direction, a state-search witness consists of a successor coordinate on the erased graph and an isomorphism from its realization to t. Transport this coordinate and its step through a. Coordinate recovery identifies the source with p. Naturality of realization, inverted and composed with the supplied endpoint isomorphism, identifies the new realization with t.

Conversely, transport q and its step through a^{-1}. Inverse transport and coordinate recovery identify the source with the extracted coordinate of realize(p). Invert the realization isomorphism and compose it with the given isomorphism to t. The transported coordinate, transported step, and composite isomorphism are exactly the witnesses required by the state-search definition. □

Lean: `Linear.TypedPosition.search_realize_iff`

Taking $t = \text{realize}(q)$ and the identity endpoint isomorphism shows that every coordinate step realizes an actual state-search step between its two prescribed realizations. This strengthens the earlier endpoint-only realization theorem. It does not identify search with contextual graph rewriting.

Lean: `Linear.TypedPosition.search_realize`

31.6 Erasure and state search respect renaming

Fix a focussed isomorphism $a : f \cong_f g$. For any bijection carrying x to y, restrict it to the complements of x and y. Its forward and inverse maps are the original maps with non-deletion certificates: injectivity prevents a retained point from mapping to the deleted point, and the inverse law proves the analogous claim backwards. Subtype extensionality lifts both inverse laws.

Lean: `Linear.exceptOfEq`

The graph vertex bijection carries the selected internal target of f to that of g. Indeed, the focussed isomorphism identifies the targets, their selection certificates make them internal, and the internal constructor is injective. Restrict this vertex bijection at the selected targets and the edge bijection at the focus edges. These give bijections of the erased carriers.

Lean: `Linear.Focussed.Iso.targetVertex`

Lean: `Linear.Focussed.Iso.erasedVertices`

Lean: `Linear.Focussed.Iso.erasedEdges`

Lemma 31.10 (Naturality of contraction)**.** *Inclusion of retained ports commutes with the restricted vertex bijection. The joined positions correspond, and contraction commutes with renaming.*

Proof. Commutation with inclusion is a case analysis on ports. To compare joined positions, apply the injective inclusion on the target side. The resulting equation is precisely preservation of the focus source. For contraction, separate the deleted target from other vertices. In the first case both sides are the corresponding joined positions. In the second, injectivity ensures that the image is not the target either. Apply inclusion on the target side: both contraction–inclusion composites reduce to their original vertices, and the inclusion commutation identity proves equality. □

Lean: `Linear.Focussed.Iso.erased_lift`

Lean: `Linear.Focussed.Iso.erased_position`

Lean: `Linear.Focussed.Iso.erased_contract`

Theorem 31.11 (Naturality of erasure and state search)**.** *The restricted bijections form an isomorphism of the erased graphs, carrying the extracted coordinate of the first state to that of the second. Search is invariant under focus-preserving isomorphisms in both arguments.*

Proof. Retained labels inherit the original isomorphism's label equations. For each incidence list, use its original equation and commute contraction past the port map pointwise. This proves the erased graph isomorphism. Its coordinate equation follows from preservation of the mode and the joined position.

Given a search step from the first state, transport its successor coordinate through the erased isomorphism. The coordinate equation identifies the new source, and search transport supplies the step. Invert the realization isomorphism and compose with the old endpoint isomorphism to retain the same actual target. This proves source invariance. Combine it with the earlier target invariance to obtain invariance in both arguments; use the inverse isomorphisms for the reverse implication. □

Lean: `Linear.Focussed.Iso.erased`

Lean: `Linear.State.coordinate_iso`

Lean: `Linear.State.search_source_iso`

Lean: `Linear.State.search_iso_iff`

31.7 Search on isomorphism classes of states

Definition 31.12 (State isomorphism class)**.** Two states are equivalent when there exists a focus-preserving literal-label isomorphism between them. Reflexivity, symmetry, and transitivity follow from the corresponding isomorphism constructions. A state class is the quotient by this equivalence; write $[s]$ for the class of s.

This quotient preserves full edge labels literally, including box bodies. It does not identify independently renamed contents of nested boxes. Section 31.9 constructs the recursive state quotient and proves projection and exact-length search lifting between the two quotients.

Lean: `Linear.stateSetoid`

Lean: `Linear.StateClass`

Lean: `Linear.StateClass.ofState`

Definition 31.13 (Quotient search and observations)**.** Define $[s] \longrightarrow [t]$ by state search $s \longrightarrow t$. The preceding two-argument invariance proves independence of representatives, so quotient elimination defines a proposition on classes. Initiality and finality descend in the same way, using their focussed-isomorphism invariance.

On supplied representatives, all three descended propositions agree with the original propositions by definition. The representative equations are therefore reflexive equivalences, not extra assumptions.

Lean: `Linear.StateClass.Search`

Lean: `Linear.StateClass.search_iff`

Lean: `Linear.StateClass.Initial`

Lean: `Linear.StateClass.Final`

Lean: `Linear.StateClass.initial_iff`

Lean: `Linear.StateClass.final_iff`

Theorem 31.14 (Deterministic quotient search and its boundaries)**.** *If $S \longrightarrow T$ and $S \longrightarrow U$, then $T = U$. A final class has no search successor, and no search step reaches an initial class.*

Proof. Eliminate the three quotient arguments to representatives s, t, u. The two representative steps have a common source, so state-level determinism gives a focussed isomorphism $t \cong_f u$. Quotient soundness identifies their classes. For each boundary theorem eliminate its two class arguments; the goal becomes the already proved state-level boundary law. □

Lean: `Linear.StateClass.search_deterministic`

Lean: `Linear.StateClass.final_no_search`

Lean: `Linear.StateClass.no_search_initial`

These are results for the search relation alone. The full-UAM structural theorem, including substitution rootedness and uniqueness, is proved in Section 46.

31.8 Concrete quotient-search regression

The passive nullary value step now runs from the previously defined `initialValue` to the specific `finalValue`, using the general coordinate-to-state theorem. Erase and reconstruct `initialValue` to define `recoveredValue`. Its carriers are rebuilt by deletion and insertion, but state recovery supplies a focussed isomorphism to the original. Source invariance therefore gives a step from this reconstructed state to `finalValue`. Quotient soundness identifies its class with the original initial class. Finally, quotient determinism compares any successor of that initial class with the concrete value step: it must equal the class of `finalValue`.

Lean: `Linear.Examples.value_search_final`

Lean: `Linear.Examples.recoveredValue`

Lean: `Linear.Examples.recovered_value_search`

Lean: `Linear.Examples.recovered_value_class`

Lean: `Linear.Examples.value_class_successor`

31.9 Recursive renaming of focussed states

The preceding quotient fixes box-body labels literally. We now extend search to the recursive graph isomorphisms of Section 23, allowing independent changes of carrier names inside every nested box. All constructions stay at one fixed finite depth bound. Compatibility across depth bounds and the paper's typed graph-valued box-label presentation is established in Section 45.

Lemma 31.15 (Primitive labels and focus freedom)**.** *Recursive label equivalence preserves each specified primitive symbol exactly. It also preserves and reflects absence of focuses throughout a label, and recursive graph isomorphisms preserve and reflect global focus freedom.*

Proof. At depth zero label equivalence is equality. At positive depth a primitive can only relate to the same primitive: the primitive–box alternatives are false. This proves the first assertion, including the variable and operation symbols needed by search.

For focus freedom, induct on depth. Primitive cases again reduce to equality; mixed cases are impossible. For related boxes, take the supplied recursive isomorphism of their bodies. In the forward direction, choose the inverse image of each target edge under its edge bijection, apply the induction hypothesis to the related labels, and cancel the bijection with its inverse. In the reverse direction, apply the induction hypothesis to the image of each source edge. Applying this label result edgewise, with the same inverse-image argument, gives the graph-level equivalence. □

Lean: `Linear.labelEquiv_primitive`

Lean: `Linear.labelEquiv_focusFree`

Lean: `Linear.DeepIso.focusFree_iff`

Definition 31.16 (Focussed recursive isomorphism)**.** A focussed recursive isomorphism consists of a recursive graph isomorphism, equality of the modes, and an equation carrying the distinguished focus edge to the distinguished focus edge. Literal-label focussed isomorphisms embed by treating equality of labels as recursive label equivalence.

Identity uses recursive identity; inversion uses the inverse graph bijections and the reversed mode equation; composition composes the graph maps and both focus equations. Thus these constructions retain the distinguished data, not only the unpointed graph.

Lean: `Linear.Focussed.DeepIso`

Lean: `Linear.Focussed.DeepIso.refl`

Lean: `Linear.Focussed.DeepIso.symm`

Lean: `Linear.Focussed.DeepIso.trans`

Lean: `Linear.Focussed.DeepIso.ofIso`

The focus source and target correspond: apply the ordered incidence equations to the distinguished edge and use its singleton source and target lists. The port maps preserve input status, so these endpoint equations and mode equality preserve and reflect both initiality and finality.

Lean: `Linear.Focussed.DeepIso.source`

Lean: `Linear.Focussed.DeepIso.target`

Lean: `Linear.Focussed.DeepIso.initial_iff`

Lean: `Linear.Focussed.DeepIso.final_iff`

31.10 Search and insertion under recursive renaming

Let $a : G \cong_d H$ be a recursive graph isomorphism and a_* its outer port map. Wire labels still agree: internal labels agree by the graph-isomorphism field, and boundary labels agree because boundary positions are fixed.

Lean: `Linear.deepIso_vertexLabel`

Transport an operation view by the outer edge bijection and port map, retaining its operation symbol. The primitive-label lemma proves the operation-label certificate even when other edges carry independently renamed boxes. Ordered incidence certificates follow from the graph equations, and arity follows from preservation of list length. Transport a typed coordinate by keeping its mode and applying a_* to its vertex; wire-label preservation proves the new typing certificate.

Lean: `Linear.SearchOperation.deepRename`

Lean: `Linear.TypedPosition.deepRename`

Lemma 31.17 (Recursive transport of coordinate search)**.** *Operation entry and return commute with recursive transport. A search step between transported coordinates exists exactly when the original step does. Transporting a typed coordinate forwards and then backwards returns it.*

Proof. Entry examines the empty/nonempty argument-list alternatives, which mapping preserves. Return uses optional lookup at index $i+1$, which commutes with mapping. For search, induct on the rule: the variable case uses exact preservation of the variable label and singleton source; entry uses its commutation equation; return also maps the certified lookup at index i. This proves forward transport. Apply it to the inverse recursive isomorphism and cancel its port bijection to reflect steps. The same cancellation proves the typed-coordinate inverse law, by equality of the mode and vertex and then subtype extensionality. □

Lean: `Linear.SearchOperation.deepRename_enter`

Lean: `Linear.SearchOperation.deepRename_resume`

Lean: `Linear.SearchStep.deepRename`

Lean: `Linear.SearchStep.deepRename_iff`

Lean: `Linear.TypedPosition.deepRename_symm`

Theorem 31.18 (Recursive naturality of insertion)**.** *Inserting a focus at (m, v) in G is recursively focus-preservingly isomorphic to inserting it at $(m, a_* v)$ in H. Realization of typed coordinates respects this isomorphism.*

Proof. Extend the outer vertex and edge bijections by fixing the inserted singleton names. Hosted vertices map to hosted images, and the fresh vertex is fixed. Splitting commutes with this map because injectivity says $a_* w = a_* v$ exactly when $w = v$; hence both sides take the fresh branch in the same case.

Retained edge labels inherit their recursive equivalence from a. The fresh focus labels agree literally, so reflexivity supplies their recursive equivalence. Retained source lists use commutation with splitting, retained target lists use commutation with hosting, and the fresh focus equations use the hosted and fresh port identities directly. The mode and fresh focus edge are fixed. These fields give the focussed recursive isomorphism, and the state-realization specialization follows by supplying its focus-free and hole-free certificates. □

Lean: `Linear.FocusInsertion.deepRename_host`

Lean: `Linear.FocusInsertion.deepRename_fresh`

Lean: `Linear.FocusInsertion.deepRename_split`

Lean: `Linear.FocusInsertion.deepRename_iso`

Lean: `Linear.FocusInsertion.deepRename_focussed`

Lean: `Linear.TypedPosition.realize_deepRename`

31.11 Erasure under recursive renaming

Let $a : f \cong_{f,d} g$ be a focussed recursive isomorphism. Its outer vertex bijection carries the selected internal target to the selected internal target: use target preservation, rewrite both selection certificates, and cancel the internal-port constructor. Restrict this bijection to the undeleted vertices and restrict the edge bijection to the non-focus edges, using the existing complement-bijection construction.

Lean: `Linear.Focussed.DeepIso.targetVertex`

Lean: `Linear.Focussed.DeepIso.erasedVertices`

Lean: `Linear.Focussed.DeepIso.erasedEdges`

Lemma 31.19 (Retained ports and contraction)**.** *Inclusion of retained ports commutes with these restricted maps. The joined vertices correspond, and contraction commutes with recursive renaming.*

Proof. The inclusion equation holds by cases on ports. Apply the injective inclusion on the target side to compare joined vertices: their images are the focus sources, already identified by a. For contraction, if the port is the deleted target, both sides are corresponding joined vertices. Otherwise injectivity ensures that its image is also not deleted. Apply inclusion and use the contraction–inclusion identities on both sides to reduce to the inclusion commutation equation. □

Lean: `Linear.Focussed.DeepIso.erased_lift`

Lean: `Linear.Focussed.DeepIso.erased_position`

Lean: `Linear.Focussed.DeepIso.erased_contract`

The restricted maps form a recursive isomorphism of the erased graphs. Internal wire labels and retained recursive edge-label relations are inherited from a. For sources and targets, rewrite the original incidence equation and commute contraction with the port map pointwise along its ordered list. Nothing requires literal equality of boxed bodies.

Lean: `Linear.Focussed.DeepIso.erased`

31.12 State search modulo recursive endpoint renaming

Definition 31.20 (Recursively renamed state search)**.** Write $s \longrightarrow_d t$ when a typed coordinate q on the erased underlying graph of s satisfies the coordinate search rule from the extracted coordinate of s, and the realization of q is recursively focus-preservingly isomorphic to t.

Taking the actual realization as target gives a step by the identity isomorphism. Composing the supplied endpoint isomorphism with another proves closure under recursive target renaming.

Lean: `Linear.State.DeepSearch`

Lean: `Linear.State.deepSearch_realize`

Lean: `Linear.State.deepSearch_target_iso`

Theorem 31.21 (Recursive search invariance, determinism, and boundaries)**.** *Two recursive-search successors of the same state are focussed recursively isomorphic. Recursive search respects recursive isomorphisms in both arguments. Final states have no such search step, and no such step reaches an initial state.*

Proof. For determinism, compare the two witnessing coordinate steps on the identical erased source graph. Coordinate determinism identifies their results; subtype extensionality identifies their typed coordinates. Compose the inverse of one endpoint isomorphism with the other.

For source invariance, erasure naturality identifies the extracted coordinates: the mode equation gives the first component and the joined-position equation the second. Transport the successor coordinate and step through the erased recursive isomorphism. Naturality of realization identifies the transported realization with the old one, so compose its inverse with the old endpoint isomorphism. Combine this with target closure; apply inverse isomorphisms to obtain the converse and hence invariance in both arguments.

For a final source, its extracted coordinate is backtrack at an input, which has no coordinate search successor. For an initial target, transport its initiality backwards through the endpoint isomorphism and then through the realization boundary law. The successor coordinate would be search at an input, excluded by the coordinate no-initial-target theorem. □

Lean: `Linear.State.deepSearch_deterministic`

Lean: `Linear.State.coordinate_deepIso`

Lean: `Linear.State.deepSearch_source_iso`

Lean: `Linear.State.deepSearch_iso_iff`

Lean: `Linear.State.final_no_deepSearch`

Lean: `Linear.State.no_deepSearch_initial`

Theorem 31.22 (Exact relation to literal-label state search)**.** *Every literal-label state-search step is a recursive step. More precisely,*

$$s \longrightarrow_d t \quad \Longleftrightarrow \quad \exists u,\ s \longrightarrow u \ \wedge\ u \cong_{f,d} t.$$

Proof. A literal endpoint isomorphism embeds into a recursive one, giving the first claim. For the forward implication of the equivalence, choose u to be the actual realization of the witnessing successor coordinate. This is a literal state-search successor by identity, and the supplied recursive endpoint isomorphism identifies it with t. Conversely, embed the literal step and compose its endpoint witness with the supplied recursive isomorphism. □

Lean: `Linear.State.search_deepSearch`

Lean: `Linear.State.deepSearch_iff`

Thus the new relation changes the permitted endpoint representatives; it does not introduce another local search rule. It is not a claim that literal-label search between two fixed representatives is itself invariant under arbitrary changes of box-body labels.

31.13 Recursive state classes and their search relation

Definition 31.23 (Recursive state class)**.** Quotient states by existence of a focussed recursive isomorphism. The identity, inverse, and composition constructions prove this is an equivalence relation. Write $[s]_d$ for the class of s. Descend recursive search to pairs of classes using its proved two-argument invariance. Descend initiality and finality using their recursive-isomorphism invariance.

Lean: `Linear.deepStateSetoid`

Lean: `Linear.DeepStateClass`

Lean: `Linear.DeepStateClass.ofState`

Lean: `Linear.DeepStateClass.Search`

Lean: `Linear.DeepStateClass.Initial`

Lean: `Linear.DeepStateClass.Final`

Each descended proposition reduces to its representative proposition by definition, giving the three representative equivalences.

Lean: `Linear.DeepStateClass.search_iff`

Lean: `Linear.DeepStateClass.initial_iff`

Lean: `Linear.DeepStateClass.final_iff`

Theorem 31.24 (Deterministic search on recursive classes)**.** *Recursive quotient search has at most one successor class. Final classes have no search successor, and no search step reaches an initial class.*

Proof. Eliminate the quotient arguments to representatives. Two steps from a common representative have recursively isomorphic targets by state-level determinism; quotient soundness equates their classes. Each boundary assertion reduces to the corresponding representative-level boundary theorem. □

Lean: `Linear.DeepStateClass.search_deterministic`

Lean: `Linear.DeepStateClass.final_no_search`

Lean: `Linear.DeepStateClass.no_search_initial`

31.14 Projection and lifting between the two state quotients

Definition 31.25 (Forgetting literal body representatives)**.** Define $\pi([s]) = [s]_d$, mapping literal-label state classes to recursive state classes. This is well-defined because every literal focussed isomorphism embeds into a recursive one. On a specified representative the equation is reflexive. The map is surjective: choose the literal class of any representative of a recursive class.

Lean: `Linear.StateClass.toDeep`

Lean: `Linear.StateClass.toDeep_ofState`

Lean: `Linear.StateClass.toDeep_surjective`

Theorem 31.26 (Search projection and lifting)**.** *A literal-class step $S \longrightarrow U$ gives $\pi(S) \longrightarrow_d \pi(U)$. Conversely, for every specified literal class S, a step $\pi(S) \longrightarrow_d T$ lifts to some step $S \longrightarrow U$ with $\pi(U) = T$. Consequently,*

$$\pi(S) \longrightarrow_d T \quad \Longleftrightarrow \quad \exists U,\ S \longrightarrow U \ \wedge\ \pi(U) = T.$$

Proof. For projection, eliminate both literal classes to representatives and embed the literal state-search step. For lifting, eliminate the source literal class and target recursive class. The exact endpoint characterization supplies a literal successor u of the fixed source representative together with a recursive isomorphism to the chosen target representative. Its literal class is the desired U, and quotient soundness proves the projection equation. The displayed equivalence combines projection and lifting. □

Lean: `Linear.StateClass.search_toDeep`

Lean: `Linear.StateClass.search_lift`

Lean: `Linear.StateClass.search_toDeep_iff`

Initiality and finality agree before and after projection, by quotient elimination and their representative definitions. Absence of search successors also agrees: a literal successor projects

to a recursive successor, and a recursive successor lifts to a literal successor. Taking the contrapositive in each direction proves the normal-form equivalence.

Lean: `Linear.StateClass.initial_toDeep`

Lean: `Linear.StateClass.final_toDeep`

Lean: `Linear.StateClass.no_search_toDeep`

31.15 Exact-length executions across the quotients

Definition 31.27 (Search-only machines)**.** On each state quotient, form a `Machine` whose step relation is its search relation and whose final predicate is its boundary finality predicate. The search boundary theorem proves the required final-normal law. Its search-determinism theorem proves determinism of this machine.

These machines deliberately contain only search. They are not the full UAM: substitution and computation are absent. Their finite executions use the already defined `Machine.Steps` relation, so the length counts search steps exactly, without introducing a second execution datatype.

Lean: `Linear.StateClass.searchMachine`

Lean: `Linear.DeepStateClass.searchMachine`

Lean: `Linear.StateClass.searchMachine_deterministic`

Lean: `Linear.DeepStateClass.searchMachine_deterministic`

Theorem 31.28 (Exact-length projection and lifting)**.** *For every $n \in \mathbb{N}$, literal executions project with length unchanged, and every recursive execution of length n from $\pi(S)$ lifts from the specified literal class S with length unchanged. Thus*

$$\pi(S) \longrightarrow_d^n T \quad \Longleftrightarrow \quad \exists U,\ S \longrightarrow^n U \ \wedge\ \pi(U) = T.$$

Proof. Projection is induction on the execution: the zero-step execution remains zero, and a step followed by a shorter run maps to its projected step followed by the projected shorter run.

For lifting, induct on n, allowing arbitrary source and target classes. At zero, the execution identifies the target with $\pi(S)$, so take $U = S$. At $n + 1$, lift the first recursive step from the specified source to a literal successor V. Rewrite the recursive tail's source using $\pi(V)$, apply the induction hypothesis from V, and prepend the lifted first step. The endpoint equality and the exact length are retained. The equivalence combines both constructions. □

Lean: `Linear.StateClass.steps_toDeep`

Lean: `Linear.StateClass.steps_lift`

Lean: `Linear.StateClass.steps_toDeep_iff`

Theorem 31.29 (Exact-length search termination)**.** *For each prescribed n, a successful search run of length n exists from S if and only if one exists from $\pi(S)$.*

Proof. Project or lift the run using the previous theorem. In the lifting direction, rewrite finality along the endpoint projection equality and use reflection of finality. In the projection direction, use preservation of finality. □

Lean: `Linear.StateClass.final_run_toDeep_iff`

31.16 Recursive-renaming and execution regression proofs

The concrete two-hole box is globally focus-free because every edge in its body is a hole rather than a focus. The established recursive isomorphism to the box with its body's edge names exchanged transports this focus freedom. These two boxed graphs were already proved to admit no literal-label isomorphism, so the test exercises genuinely recursive label comparison.

Lean: `Linear.Examples.boxedTwoHoles_focusFree`

Lean: `Linear.Examples.renamedBoxedTwoHoles_focusFree`

The earlier concrete value step embeds into recursive search. Prepending that step to the zero-step execution gives an execution of exactly one step from its initial recursive class to its final recursive class. To test lifting from a reconstructed source, identify its literal class with the original initial class, use this recursive execution, and apply the exact-length termination equivalence backwards. The result is a successful literal search run of exactly one step from the reconstructed source. Finally, the final value's boundary finality and the recursive no-search-from-final theorem exclude every recursive search successor.

Lean: `Linear.Examples.value_deep_run`

Lean: `Linear.Examples.recovered_value_lift`

Lean: `Linear.Examples.final_value_deep_normal`

32 Rootedness and operation paths

32.1 Canonical roots of concrete states

This milestone connects the existing search semantics to Appendix C. We construct the paper's rooted predicate, prove that search preserves it, and establish concrete versions of Lemmas 23 and 26(1). These results use our erasure-based search semantics on recursive state classes; identifying that semantics with the paper's contextual local rewrites is established in Section 41. The proofs below do not assume the abstract specimen locality laws.

Definition 32.1 (Canonical initial attachment)**.** For a concrete state s, erase its focus, attach a searching focus at the sole input of the resulting graph, and call the resulting state $r(s)$. Focus freedom and hole freedom of erasure supply the certificates required for realization. Its focus is initial by the initial-realization theorem.

Lean: `Linear.State.root`

Lean: `Linear.State.root_initial`

Lemma 32.2 (Roots respect renaming and initiality)**.** *Recursively isomorphic underlying graphs give recursively focus-preservingly isomorphic roots. In particular, a focussed recursive isomorphism between states gives one between their roots. If s is already initial, $r(s)$ is recursively focus-preservingly isomorphic to s.*

Proof. The coordinate "search at input zero" is fixed by every boundary-preserving port map. Apply recursive naturality of realization to the given underlying isomorphism. For focussed state isomorphisms, first use erasure naturality.

For an initial state, the extracted mode is search and its extracted vertex is an input. The state interface has exactly one input, so that vertex is input zero. Thus the extracted typed coordinate equals the initial coordinate, by component equality and subtype extensionality. Rewrite the general recovery isomorphism using this coordinate equality, and embed the literal-label isomorphism into a recursive one. □

Lean: `Linear.State.root_of_underlying`

Lean: `Linear.State.root_iso`

Lean: `Linear.State.coordinate_initial`

Lean: `Linear.State.root_initial_iso`

Lemma 32.3 (Search preserves the underlying graph)**.** *If $s \longrightarrow_d t$, their erased underlying graphs are recursively isomorphic.*

Proof. Let q be the witnessing successor coordinate on the erased graph of s. Insertion–erasure cancellation identifies the underlying graph of realize(q) with the underlying graph of s. Invert that isomorphism and compose it with erasure of the supplied recursive endpoint isomorphism from the realization to t. □

Lean: `Linear.State.deepSearch_underlying`

Descend r to recursive state classes using root isomorphism invariance. Its value on a representative is its representative root class, by definition. Every root class is initial. An initial class is fixed by r, using quotient soundness and the initial-state recovery lemma. In

particular $r(r(S)) = r(S)$. Search preserves $r(S)$ because it preserves the underlying graph; induction on a search execution then proves preservation of the root along the whole run.

Lean: `Linear.DeepStateClass.root`

Lean: `Linear.DeepStateClass.root_ofState`

Lean: `Linear.DeepStateClass.root_initial`

Lean: `Linear.DeepStateClass.root_eq_of_initial`

Lean: `Linear.DeepStateClass.root_idempotent`

Lean: `Linear.DeepStateClass.search_root`

Lean: `Linear.DeepStateClass.steps_root`

32.2 Concrete rootedness and its search invariant

Definition 32.4 (Concrete rooted state)**.** A recursive state class S is rooted when there is a finite search-only execution from $r(S)$ to S:

$$\mathsf{Rooted}(S) \quad \Longleftrightarrow \quad \exists n \in \mathbb{N},\ r(S) \longrightarrow_d^n S.$$

A named state is rooted when its recursive isomorphism class is rooted. This definition depends on the operation classification used by search.

Lean: `Linear.DeepStateClass.Rooted`

Lean: `Linear.State.Rooted`

Theorem 32.5 (Initial states are rooted; search preserves rootedness)**.** *Every initial class is rooted, every canonical root is rooted, and search steps and finite search executions preserve rootedness.*

Proof. An initial class equals its root, so the zero-step run witnesses rootedness. Every root is initial, giving the second claim. For a step $S \longrightarrow_d T$, append that step to the witnessing run from $r(S)$ to S. Search preservation of roots identifies $r(S)$ with $r(T)$, so this is the required run to T. Induction on a finite execution gives the final claim. □

Lean: `Linear.DeepStateClass.initial_rooted`

Lean: `Linear.DeepStateClass.root_rooted`

Lean: `Linear.DeepStateClass.rooted_search`

Lean: `Linear.DeepStateClass.rooted_steps`

Theorem 32.6 (Equivalent initial-run characterization)**.** *A class S is rooted exactly when some initial class has a finite search run to S. The initial class in any such run must equal $r(S)$.*

Proof. The forward direction chooses $r(S)$, which is initial. Conversely, suppose I is initial and reaches S. Initiality gives $I = r(I)$, and preservation of roots along the run gives $r(I) = r(S)$. Rewrite the run's source by these equalities to obtain the rootedness witness. □

Lean: `Linear.DeepStateClass.rooted_iff_initial_run`

32.3 Operation reachability under coordinate search

Definition 32.7 (Operation-reachable vertex)**.** A vertex v is operation-reachable if there are an input index i and an edge list E with an operation trace from v backwards to input i. Equivalently, reversing E gives an operation path from that input to v. An input is reachable using the empty trace. Each certified operation view has an actual operation label, by its label certificate and the definition of primitive operation labels at each depth.

Lean: `Linear.OperationReachable`

Lean: `Linear.OperationReachable.input`

Lean: `Linear.SearchOperation.isOperation`

Lemma 32.8 (Argument and source reachability)**.** *If the source of an operation is reachable, every term argument is reachable. Conversely, if one of its arguments is reachable, its source is reachable.*

Proof. Forwards, prepend the operation edge to the reverse trace from its source; its certified argument membership supplies the target incidence. Backwards, inspect a witnessing trace from the argument to an input. It cannot be empty, since an edge cannot target an input. Its first edge equals the given operation edge by uniqueness of incoming incidence. The operation's singleton source then identifies the next vertex in the trace with its source. The remaining trace proves source reachability. □

Lean: `Linear.OperationReachable.argument`

Lean: `Linear.OperationReachable.source`

Theorem 32.9 (Search preserves operation reachability)**.** *Every coordinate search rule preserves operation reachability of its vertex. Recursive renaming preserves and reflects this property.*

Proof. A variable trigger keeps the vertex. Operation entry either keeps its source, for no term arguments, or moves to the first argument, using forward argument reachability. Return from an argument first gives reachability of the operation source by the converse lemma. It then either stays there on completion or moves to the next argument by the forward lemma.

For renaming, transport the reverse operation trace and its edge list through the recursive graph isomorphism. The boundary input is fixed. Apply the same construction to the inverse isomorphism and cancel the port maps for reflection. □

Lean: `Linear.SearchStep.operationReachable`

Lean: `Linear.OperationReachable.rename`

Lean: `Linear.OperationReachable.rename_iff`

32.4 Incoming paths and insertion of the focus

Definition 32.10 (Incoming incidence trace)**.** An incoming trace is a reverse path with explicit vertex incidences but no label restriction. It is either empty at a vertex or consists of an edge whose target contains the current vertex, a chosen source vertex of that edge, and a remaining incoming trace from that source.

Lean: `Linear.IncomingTrace`

Lemma 32.11 (Incoming paths to reachable vertices)**.** *An edge targeting an operation-reachable vertex is an operation, and each of its sources is operation-reachable. Consequently, every incoming trace ending at a reachable vertex is an operation trace with the same vertices and edge list.*

Proof. Compare the incoming edge with the first edge of a witnessing reverse operation trace to an input. The trace cannot be empty because inputs have no incoming edge. Target-incidence uniqueness identifies the two edges, proving its operation label. An operation has exactly one source, so every chosen source is the next vertex in the witnessing trace and is reachable by its tail. Induct on the arbitrary incoming trace and apply this argument at each step. □

Lean: `Linear.OperationReachable.predecessor`

Lean: `Linear.IncomingTrace.operations`

Lean: `Linear.OperationReachable.incoming_operation`

Lemma 32.12 (A path never departs from its final vertex earlier)**.** *Suppose an operation trace from v to an input has length k. In any operation trace to an input with length at most k, no edge can have v as a source. In particular the original trace never has v as a source of one of its edges.*

Proof. Induct on the shorter trace's edge list. If its first edge has v as source, its singleton-source property identifies the next vertex with v. The tail and the original length-k trace therefore both run from v to inputs. Uniqueness of operation traces to inputs equates their edge lists and hence their lengths. But the tail is shorter than the trace of length at most k, a contradiction. For a later edge, apply the induction hypothesis to the tail, whose length is still at most k. □

Lean: `Linear.operationTrace_sources_avoid`

Theorem 32.13 (Lifting the operation path through focus insertion)**.** *An operation path from an input to v survives insertion of a focus at v: it becomes a path from the same input to the hosted copy of v, using the retained copies of its original edges.*

Proof. The previous lemma says no source occurrence on this path equals v. Insertion therefore maps all its source occurrences by hosting rather than to the fresh focus target. Targets are always hosted. Induct on the reverse trace, retaining each operation label and mapping each source and target membership into its incidence list. The empty trace and the boundary input are fixed. This proves a path in the actual inserted graph, not just in its focus erasure. □

Lean: `Linear.operationTrace_insert`

32.5 Rooted states have actual operation paths: Lemma 23

Define the intermediate invariant $U(s)$ to mean that the extracted coordinate of s is operation-reachable in its erased graph. It is invariant under focussed recursive isomorphisms: erasure naturality transports the graph, coordinate naturality identifies the vertex, and reachability is invariant under this transport.

Lean: `Linear.State.UnderlyingReachable`

Lean: `Linear.State.underlyingReachable_iso`

Lemma 32.14 (Realization and search preserve the intermediate invariant)**.** *Realizing a reachable typed coordinate satisfies U. Every canonical root satisfies U, and recursive state search preserves U.*

Proof. Coordinate recovery provides one isomorphism that restores both the original graph and the chosen coordinate after realizing and erasing. Reflect operation reachability along this isomorphism to prove the realization assertion. The initial coordinate is an input, hence reachable, which proves the root assertion. For a search step, coordinate search preserves reachability; apply the realization assertion to its successor coordinate and transport the result through its supplied recursive endpoint isomorphism. □

Lean: `Linear.State.underlyingReachable_realize`

Lean: `Linear.State.root_underlyingReachable`

Lean: `Linear.State.deepSearch_underlyingReachable`

The invariant descends to recursive classes using its isomorphism invariance. Quotient elimination proves the root and one-step preservation laws there. Induction on the witnessing root run proves that every rooted class satisfies U. This is the search-sequence induction used for the path argument in Appendix C, expressed through an invariant of erased coordinates.

Lean: `Linear.DeepStateClass.UnderlyingReachable`

Lean: `Linear.DeepStateClass.root_underlyingReachable`

Lean: `Linear.DeepStateClass.search_underlyingReachable`

Lean: `Linear.DeepStateClass.rooted_underlyingReachable`

Theorem 32.15 (Concrete rooted-focus operation path)**.** *If a concrete state is rooted, its actual focus source is reachable from its unique input by an operation path. If the focus source is not an input, this path is nonempty.*

Proof. First consider any state satisfying U. Lift its witnessing erased operation trace through insertion at its extracted coordinate, by the insertion theorem. The general recovery isomorphism carries this inserted graph to the actual state and carries the inserted focus source to its actual focus source. Transport the lifted operation trace through that isomorphism. Boundary preservation fixes the input. This gives operation reachability in the actual state graph.

A rooted state satisfies U by the invariant proof, so the construction applies. The interface has one input, identifying its index with zero. Reverse the trace list to obtain the existing forward operation-path representation. If this list were empty, the zero-step reverse trace would identify the focus source with the input; hence the path is nonempty whenever the source is not an input. □

Lean: `Linear.State.focus_operationReachable`

Lean: `Linear.State.rooted_focus_operationReachable`

Lean: `Linear.State.rooted_focus_path`

This establishes the graph-path conclusion of Lemma 23 for the present concrete search model. Unlike the paper's presentation, the proof does not first require a separate uniqueness theorem for whole search runs: it uses the operation-path uniqueness already proved and a direct search invariant.

32.6 The non-operation-path obstruction: Lemma 26(1)

Theorem 32.16 (Incoming-path obstruction)**.** *Every incoming incidence trace to a rooted focus source is an operation trace. Thus an incoming trace that is not an operation trace proves that the state is not rooted.*

Proof. The preceding rooted-focus theorem gives reachability of the actual source. Apply the incoming-trace theorem at that vertex. The obstruction is its contrapositive. □

Lean: `Linear.State.rooted_incoming_operations`

Lean: `Linear.State.not_rooted_of_nonoperation_path`

Lemma 32.17 (The existing edge-path representation)**.** *For any nonempty edge path whose last edge targets a reachable vertex, every edge of the path is an operation, and every source of each edge is reachable. Repeated edges are allowed in the supplied path datatype.*

Proof. Induct on the path. For a single edge, use the incoming-operation and source reachability lemmas at its target. For a path beginning with adjacent edges e, f, apply the induction hypothesis to the tail beginning at f. The adjacency witness is a source of f, hence reachable, and a target of e. The incoming-edge lemmas then prove the required claims for e, while the induction hypothesis proves them for all remaining edges. □

Lean: `Linear.path_operations_of_reachable`

Theorem 32.18 (Concrete Lemma 26(1))**.** *If a nonempty graph path ends at the focus source of a rooted state, all of its edges are operations. Therefore a path ending there and containing any non-operation edge is a certificate that the state is not rooted.*

Proof. Apply the edge-path lemma to the actual focus-source reachability supplied by rootedness. For the obstruction, select the non-operation edge from the path and contradict the resulting operation-label assertion. □

Lean: `Linear.State.rooted_path_operations`

Lean: `Linear.State.not_rooted_of_path`

This is the first part of the paper's obstruction lemma in our concrete path and search representations. The one-way replacement argument in Lemma 26(2) is proved separately in Section 33.4. The general output-closed specimen argument in Lemma 26(3) is proved for shallow specimens in Section 34.6; its deep extension remains open.

32.7 Concrete refocusing and reduction to rewriting

Definition 32.19 (Concrete refocusing)**.** A transition relation on recursive state classes is refocusing when each transition from a concretely rooted state leads to a concretely rooted state. The rooted predicate is the one constructed from actual search above; it is not the abstract specimen interface's supplied rooted predicate.

Lean: `Linear.DeepStateClass.Refocusing`

Theorem 32.20 (Search refocusing and reduction to rewriting)**.** *Search is refocusing. For any supplied rewrite relation R, the relation* Search $\cup R$ *is refocusing if and only if R preserves rootedness. For any machine on these concrete classes whose step relation is refocusing, every execution from an initial class ends in a rooted class.*

Proof. Search refocusing is the proved one-step rootedness invariant. If the union is refocusing, apply its law to transitions from its R branch. Conversely, case-split a union transition: use search preservation in the search branch and the supplied preservation law in the rewrite branch. Finally induct on a machine execution, starting with the theorem that initial classes are rooted and applying its refocusing hypothesis at each step. □

Lean: `Linear.DeepStateClass.search_refocusing`

Lean: `Linear.DeepStateClass.refocusing_with_search`

Lean: `Linear.DeepStateClass.refocusing_execution`

The last two statements are conditional on the indicated rewrite or machine preservation hypothesis. They do not construct substitution or compute transitions, and do not assert that arbitrary rewrite relations preserve roots. Section 46 subsequently proves substitution refocusing and reduces full-UAM refocusing exactly to its extrinsic behaviour.

32.8 Rooted and non-rooted concrete examples

The initial value state is rooted for any operation classification, by initiality. Its successful passive search execution gives a rooted final value state by preservation along search runs.

Lean: `Linear.Examples.initial_value_rooted`

Lean: `Linear.Examples.final_value_rooted`

The previously constructed interior-focus state is valid and hole-free but is not rooted, for any operation classification. Its underlying closed chain has a variable before the focus position and a value after it. In the actual inserted graph, the retained variable edge targets the focus source. The single-edge path consisting of that variable therefore ends at the source, and its label is not an operation. The concrete Lemma 26(1) obstruction rules out rootedness. This separates graph validity from the operational invariant and exercises the obstruction on an actual named state.

Lean: `Linear.Examples.interior_state_not_rooted`

33 One-way hypernets and the single-hole obstruction

33.1 One-way hypernets and the term-boundary restriction

This milestone proves the single-hole case of Appendix C, Lemma 26(2), for actual shallow substitution and the concrete rooted-state predicate. The proof follows the paper's reason for introducing one-way hypernets: an operation path from the global input cannot pass through such a filling and emerge at a term output. We make the path-extraction argument explicit and allow the replaced edge to have a self-loop.

Lemma 33.1 (Sources of operation paths have term type)**.** *Every source vertex of an actual operation edge has type $\star$. If a nonempty reverse operation trace ends at an input, that input has type $\star$.*

Proof. The signature of every primitive operation has exactly one source, of type $\star$. Apply the source-typing equation to any member of its source list. For the second assertion, induct on the trace's edge list. If its tail is empty, its endpoint is the source of the first operation and the first assertion applies. Otherwise apply the induction hypothesis to the tail. Boxes, variables, focuses, and holes are excluded by the operation predicate. □

Lean: `Linear.operation_source_term`

Lean: `Linear.operationTrace_input_term`

Definition 33.2 (One-way hypernet)**.** A hypernet $H : I \to O$ is *one-way* if, for every term input i and term output j, there is no operation path from i to j. Paths to thunk outputs are deliberately not excluded. This is the restriction in the paper's Section 6.1, expressed using our existing operation-path datatype.

Lean: `Linear.OneWay`

Theorem 33.3 (Reachability characterization)**.** *A hypernet is one-way if and only if none of its term outputs is operation-reachable from an input.*

Proof. A reverse trace from an output to an input cannot be empty: the boundary constructors are disjoint. Its input therefore has term type by the preceding lemma. Reverse its recorded edge list to obtain the forbidden forward operation path. Conversely any forbidden forward path supplies precisely a reverse trace witnessing operation reachability of its term output. □

Lean: `Linear.oneWay_iff`

One-wayness is invariant under recursive graph isomorphism. Such an isomorphism fixes the ordered boundary and preserves and reflects operation reachability, so the characterization applies in both directions. A graph with no outputs is one-way vacuously, since its output index type is empty.

Lean: `Linear.DeepIso.oneWay_iff`

Lean: `Linear.oneWay_no_outputs`

33.2 Extracting an operation path from a filling

Fix a host C, a selected edge h, and a filling H with the interface of h. Write $P = C[H]$, ι_C for the retained-host vertex map, and ι_H for the filling map. These are the existing maps of shallow replacement; ι_H need not be injective when h has a self-loop.

The elementary decomposition used in the induction says that a nonempty reverse operation trace with edge list $e :: E$, from v to z, consists exactly of an intermediate vertex w, an operation label on e, incidences $v \in \mathrm{target}(e)$ and $w \in \mathrm{source}(e)$, and a trace with edge list E from w to z. Both directions follow directly from the trace constructor.

Lean: `Linear.operationTrace_cons_iff`

Lemma 33.4 (An inserted non-input stays a non-input)**.** *If a vertex v of H is not an input, its image $\iota_H(v)$ is not an input of P.*

Proof. Target coverage in H supplies an edge targeting v. Its inserted copy targets $\iota_H(v)$ in P. Inputs of a well-formed graph have no incoming edge, which proves the claim, including when v is a filling output. □

Lean: `Linear.OneWayReplacement.insert_not_input`

Theorem 33.5 (Extraction of a filling-local input path)**.** *If $\iota_H(v)$ is operation-reachable in P, then v is operation-reachable inside H.*

Proof. Choose a reverse operation trace from $\iota_H(v)$ to a global input and induct on its length, generalizing v. If v is already an input of H, the empty local trace is the required witness. Otherwise, in the zero-length case, $\iota_H(v)$ would be a global input, contradicting the previous lemma.

In the nonempty case, target coverage supplies an edge f of H targeting v. Its inserted copy and the first edge of the global trace target the same vertex. Unique incoming incidence identifies these two edges. The first edge is consequently a filling edge and retains its operation label. Its chosen source is the image of some source w of f. Apply the induction hypothesis to the shorter global trace from that image; this yields a local operation trace from w to an input of H. Prepending f yields the desired local trace from v.

The induction stops on first reaching a filling input. It neither reflects an arbitrary whole edge path through ι_H nor assumes injectivity of that map. This is why the proof remains valid for self-looping holes, unlike unrestricted path reflection discussed earlier. □

Lean: `Linear.OneWayReplacement.extract`

33.3 Reflecting host reachability through a one-way filling

Theorem 33.6 (Host term reachability reflects)**.** *Suppose H is one-way and v is a term vertex of C. If $\iota_C(v)$ is operation-reachable in $C[H]$, then v is operation-reachable in C.*

Proof. Induct on the edge list of a reverse operation trace from $\iota_C(v)$ to a global input. In the empty case injectivity of the retained-host map identifies v with that same input.

For a nonempty trace, inspect its first edge. If it is a surviving host edge, reflect its target incidence by injectivity of ι_C and express its source as $\iota_C(w)$. The corresponding host edge is an operation, so w has term type. Apply the induction hypothesis to the shorter trace and prepend the host edge.

If the first edge is inserted from H, its target is some $\iota_H(x) = \iota_C(v)$. A filling target cannot be an input. The boundary classification of replacement therefore forces x to be an output of H. Both replacement maps preserve vertex labels, so this output has term type. Extraction applied to the original trace now makes it operation-reachable inside H, contradicting one-wayness. Thus only the retained-host case can occur. □

Lean: `Linear.OneWayReplacement.reflect`

Theorem 33.7 (Incoming host paths consist of operations)**.** *Under the same hypotheses, every edge of any host path ending at v is an operation. The given host path is allowed to contain the replaced edge.*

Proof. Reflect operation reachability to C, then apply the earlier theorem that all edges of any path ending at an operation-reachable vertex are operations. That theorem uses the existing nonempty edge-path representation and allows repeated edges. No assertion that the supplied host path itself survives filling is needed. □

Lean: `Linear.OneWayReplacement.path_operations`

Theorem 33.8 (One-way filling closure)**.** *If both C and H are one-way, then $C[H]$ is one-way.*

Proof. Suppose a term output of $C[H]$ were operation-reachable. It is the retained copy of the corresponding term output of C. Reflection would make that output operation-reachable in C, contradicting the host's one-wayness. □

Lean: `Linear.OneWayReplacement.oneWay_plug`

33.4 The concrete one-way obstruction: Lemma 26(2)

For a focussed host $C : \star \to \epsilon$, replace an edge other than its focus by a globally focus-free filling H. Once the resulting graph is proved globally hole-free, package it with its retained focus as an actual state. The constructor requires these focus-freedom and hole-freedom proofs; it does not infer them merely from the interfaces. Its focus source is $\iota_C(s)$, where s is the host's focus source.

Lean: `Linear.Focussed.filledState`

Theorem 33.9 (Concrete Lemma 26(2))**.** *Suppose the focussed host has a path ending at its focus source and that path contains a non-operation edge. If the filling H is one-way, the resulting actual state is not rooted.*

Proof. Assume the filled state is rooted. The concrete Lemma 23 gives operation reachability of its focus source $\iota_C(s)$ in the actual filled graph. The source s of the host focus has term type. Apply the preceding host-path theorem: every edge of the supplied host path must be an operation. This contradicts its specified non-operation edge. □

Lean: `Linear.Focussed.oneWay_not_rooted`

This proves the single-hole one-way obstruction for the current concrete search semantics. It follows the uniqueness-and-extraction argument of Proof C.8, packaged as a reflection lemma rather than a split on whether the supplied path contains the hole. In fact the Lean result permits replacement at any non-focus edge with the right interface, so it includes the paper's hole-labelled context case. The example below uses a genuine hole label.

Theorem 33.10 (Paired single-hole obstruction)**.** *If H and K are two admissible fillings of the same focussed host and at least one is one-way, then at least one of the two filled states is not rooted, provided the host has the same non-operation path to its focus source.*

Proof. Split the hypothesis that H or K is one-way. Apply the preceding theorem to that filling and introduce the corresponding side of the conclusion. □

Lean: `Linear.Focussed.oneWay_pair_not_rooted`

This is a single-hole instance of the output-closure argument. Section 34.6 defines concrete pre-templates, output-closure, and shallow multi-hole specimens and proves their Lemma 26(3) obstruction. Section 37 extends this representation through box bodies; Section 43 establishes finite original-hole tuple compatibility for that recursive representation.

33.5 One-way examples and a filled-state obstruction

The variable corolla is one-way. Its term output is targeted by a variable edge. If that output were operation-reachable, the incoming-edge theorem would force this variable edge to be an operation, a contradiction. By contrast, the existing operation corolla with two term arguments is not one-way: its sole edge is a one-step operation path from the input to the first term output.

Lean: `Linear.Examples.variable_oneWay`

Lean: `Linear.Examples.operation_not_oneWay`

Define an operation signature with no term arguments and one thunk argument, and take its one-edge corolla. This graph has a term input and only a thunk output. It is one-way because no output has term type, yet its sole operation edge explicitly witnesses reachability of its thunk output. Together these examples verify both the nontriviality of the predicate and the importance of retaining the paper's term-output restriction.

Lean: `Linear.Examples.thunkOnlyOperations`

Lean: `Linear.Examples.thunkOnlyGraph`

Lean: `Linear.Examples.thunk_only_oneWay`

Lean: `Linear.Examples.thunk_output_reachable`

For a nonvacuous state-level application, start with the earlier closed chain whose interior focus follows a variable and precedes a value. Relabel that variable by a hole with one term input and one term output, preserving all incidences. The other labels remain the focus and the value operation. Their signatures are unchanged, so the relabelled graph is well formed. Retain the same distinguished focus, mode, and endpoints. The only other edges are a hole and an operation, both focus-free; the focus therefore remains globally unique and satisfies its original endpoint conditions.

Lean: `Linear.Examples.oneWayContext`

Lean: `Linear.Examples.oneWayContext_focus`

Fill the genuine hole by the variable corolla. The exact hole inventory of shallow substitution shows the resulting graph is hole-free: the filling is hole-free and no hole survives in the host. Indeed the only host edge that could carry a hole is exactly the removed edge. Global focus freedom of the variable filling was already checked.

Lean: `Linear.Examples.oneWayContext_filled_closed`

Finally, take the host path consisting only of its hole edge. Its target is the focus source, and its label is not an operation. The variable filling is one-way, so the concrete Lemma 26(2) theorem proves that this actual filled state is not rooted, for every operation classification. The offending host edge is removed by substitution: this exercises the new extraction argument, not merely preservation of a path of retained host edges.

Lean: `Linear.Examples.oneWay_filled_not_rooted`

34 Simultaneous substitution and shallow specimens

34.1 Simultaneous shallow substitution: carriers and incidence

The next milestone proves the multi-hole obstruction for a concrete *shallow* specimen representation. This needs a simultaneous filling construction: the one-way side of a related pair may differ at different holes, so one cannot simply assume that all left fillings, or all right fillings, are one-way. The construction below works for arbitrary finite typed graphs; only the later path arguments specialize to hypernets.

Definition 34.1 (Simultaneous expansion)**.** Let $C : I \to O$ be a finite typed graph and, for each edge e of C, choose a graph H_e with exactly the source and target interface of e. The expansion $C[H]$ replaces every edge at once. Its internal vertices and edges have carriers

$$V_C + \sum_{e \in E_C} V_{H_e}, \qquad \sum_{e \in E_C} E_{H_e}.$$

An unchanged edge will be represented by its corolla. The vertex carrier adds the same ordered external input and output constructors as before. The host map ι_C retains each original vertex. For each e, the map ι_e sends a filling internal vertex to its tagged fresh copy, its ith input to $\iota_C(\mathrm{sourcePort}(e, i))$, and its jth output to $\iota_C(\mathrm{targetPort}(e, j))$.

Lean: `Graph.Net.Expansion.Internal`

Lean: `Graph.Net.Expansion.Vertex`

Lean: `Graph.Net.Expansion.Edge`

Lean: `Graph.Net.Expansion.host`

Lean: `Graph.Net.Expansion.insert`

The host map is injective and its image is disjoint from every fresh internal copy. Both facts follow by cases on the boundary/internal constructors and on the disjoint-sum tag.

Lean: `Graph.Net.Expansion.host_injective`

Lean: `Graph.Net.Expansion.host_ne_internal`

Lemma 34.2 (Uniqueness of a filling's owner)**.** *If two non-output filling vertices have equal images, their owning host edges are equal. The same holds for two non-input filling vertices. Within one filling, the insertion map is injective on non-output vertices, and separately on non-input vertices.*

Proof. For the first assertion, two fresh internal images identify their owner tags. An internal image cannot equal a boundary attachment in the host image. The only remaining case compares two filling inputs. Their images are source ports of their owning host edges; injectivity of the host map and unique outgoing incidence identify those edges. For non-input vertices the same argument uses output ports and unique incoming incidence.

Once the owner is fixed, two internal images identify their internal names; two input images identify their input indices by distinctness of the host source ports. The output argument uses target-port distinctness. No injectivity is asserted between a filling input and output: a self-looping host edge can identify them. □

Lean: Graph.Net.Expansion.source_owner

Lean: Graph.Net.Expansion.target_owner

Lean: Graph.Net.Expansion.insert_source_injective

Lean: Graph.Net.Expansion.insert_target_injective

34.2 Well-formedness of simultaneous expansion

The source and target lists of a new edge (e, x) are those of x in H_e, mapped by ι_e. Each list remains duplicate-free: source vertices are never outputs, target vertices are never inputs, and the respective insertion injectivity lemmas therefore apply to every member of the list.

Lean: Graph.Net.Expansion.source

Lean: Graph.Net.Expansion.target

Lean: Graph.Net.Expansion.source_nodup

Lean: Graph.Net.Expansion.target_nodup

Lemma 34.3 (Unique incidence in the expanded graph)**.** *If two expanded edges share a source vertex, they are equal; the analogous statement holds for a shared target vertex.*

Proof. Pull the common incidence back along each mapped list. The owner lemma first identifies the two original host edges. Insertion injectivity then identifies the two filling vertices. Unique incidence within that one filling identifies the two filling edges, hence the tagged expanded edges. □

Lean: Graph.Net.Expansion.source_unique

Lean: Graph.Net.Expansion.target_unique

Lemma 34.4 (Boundary exclusion and coverage)**.** *No expanded source is an output and no expanded target is an input. Every vertex that is not an output has a source incidence, and every vertex that is not an input has a target incidence.*

Proof. For exclusion, inspect the local vertex witnessing a mapped incidence. A fresh internal image has neither boundary status. A local input used as a source maps to a host source port, which is not an output; a local output cannot be a source. The target argument is dual.

For a retained host vertex requiring a source incidence, use host coverage to find an original edge e and its corresponding source-port index. Input coverage in H_e supplies an edge sourced at that filling input, whose image is the original host vertex. Target coverage uses a filling output instead. For a fresh internal vertex in a copy of H_e, apply coverage directly inside H_e and insert the resulting incidence. These cases exhaust the expanded vertex datatype. □

Lean: Graph.Net.Expansion.source_not_output

Lean: Graph.Net.Expansion.target_not_input

Lean: Graph.Net.Expansion.source_host_covered

Lean: Graph.Net.Expansion.target_host_covered

Lean: Graph.Net.Expansion.source_coverage

Lean: Graph.Net.Expansion.target_coverage

Internal labels come from the indicated host or filling summand, and an edge (e, x) has the label of x in H_e. The host map preserves vertex labels by inspection. Each insertion map preserves them because its boundary cases are exactly the host source-port and target-port typing equations.

Lean: Graph.Net.Expansion.internalLabel

Lean: Graph.Net.Expansion.edgeLabel

Lean: Graph.Net.Expansion.host_label

Lean: Graph.Net.Expansion.insert_label

Applying these label equations to each mapped incidence list reduces its typing equation to the filling's original typing equation. Finite enumeration is obtained by a dependent sum over the finite host-edge enumeration, and a disjoint sum for the internal carriers. The constructor therefore proves all fields of `Net`; it does not postulate the incidence invariants of the simultaneous substitution.

Lean: Graph.Net.Expansion.source_typed

Lean: Graph.Net.Expansion.target_typed

Lean: Graph.Net.expand

34.3 Retained corollas, identity expansion, and hole freedom

Definition 34.5 (Corolla copy)**.** A corolla copy of an original edge e has one distinguished edge with the same label as e, source list equal to the complete ordered input boundary, and target list equal to the complete ordered output boundary. Every edge of the copy equals the distinguished edge. The existing corolla constructor supplies such a copy directly; its sole edge index is 0.

Lean: Linear.ExpansionPaths.CorollaCopy

Lean: Linear.ExpansionPaths.corollaCopy

Every source or target incidence of e becomes the corresponding incidence of its expanded corolla copy. Choose its port index in the host list; that boundary vertex occurs in the corolla's full boundary list and maps to the retained host vertex. More precisely, the copy's expanded incidence lists are the original host incidence lists mapped by ι_C. Mapping the full port-index enumeration reconstructs the original ordered list, proving these equalities. If the original edge label is globally focus-free, so is its corolla copy: every edge of the copy is the distinguished one and has that same label.

Lean: Linear.ExpansionPaths.copy_source

Lean: Linear.ExpansionPaths.copy_target

Lean: Linear.ExpansionPaths.copy_source_eq

Lean: Linear.ExpansionPaths.copy_target_eq

Lean: Linear.ExpansionPaths.copy_focusFree

Theorem 34.6 (Identity expansion)**.** *Expanding every edge by its original corolla gives a graph isomorphic to C.*

Proof. Forget the empty internal carrier of every corolla and the unique edge index inside each corolla. These give explicit vertex and edge bijections; their inverses retain a host vertex or attach edge index 0. Internal and edge labels are unchanged. For incidence, the exact copy-list equations reduce the result to a host inclusion followed by its inverse projection, which is the identity on every internal or boundary vertex. □

Lean: `Linear.ExpansionPaths.identityVertices`

Lean: `Linear.ExpansionPaths.identityEdges`

Lean: `Linear.ExpansionPaths.identityIso`

Lemma 34.7 (Hole-free expansion)**.** *If every H_e is globally hole-free, then $C[H]$ is globally hole-free.*

Proof. A hole occurrence in the expansion lies in the label of some tagged edge (e, x), possibly within its nested box contents. That label is exactly the label of x in H_e. The same occurrence in that label would be a hole occurrence in H_e, contradicting its global hole freedom. □

Lean: `Linear.ExpansionPaths.holeFree`

34.4 Paired path reflection for many fillings

A non-input vertex of H_e cannot be inserted as a global input: target coverage in H_e supplies an incoming edge whose expanded copy still targets its image. Global inputs have no incoming edge.

Lean: `Linear.ExpansionPaths.insert_not_input`

Theorem 34.8 (Local extraction under simultaneous filling)**.** *If $\iota_e(v)$ is operation-reachable in $C[H]$, then v is operation-reachable in H_e, independently of all other fillings.*

Proof. Induct on a reverse operation trace to the global input, generalizing v. If v is a filling input, take the empty local trace. Otherwise an empty global trace is impossible by the previous lemma. In the nonempty case, target coverage in H_e provides an edge at v. Unique incoming incidence in the expanded graph forces the first trace edge to be this inserted edge. Pull its chosen source back into H_e, extract a local trace from the shorter global trace by induction, and prepend that edge. This is the single-filling extraction proof with the simultaneous incidence theorem providing uniqueness. □

Lean: `Linear.ExpansionPaths.extract`

Consequently, if H_e is one-way, no term-typed host target of e can be operation-reachable in $C[H]$. Write that target as a target port of e. Its corresponding filling output has the same term type and the same image. Extraction would make this output operation-reachable in H_e, contradicting one-wayness.

Lean: `Linear.ExpansionPaths.oneWay_target`

Definition 34.9 (Paired output-closure for expansion)**.** Two filling families H, K satisfy the paired edge condition if, for every original edge e, either both fillings are corolla copies of e, or H_e is one-way, or K_e is one-way. The disjunction is chosen separately for each e; there need not be one side that is one-way at all replaced edges.

Lean: `Linear.ExpansionPaths.OutputClosedPair`

Lemma 34.10 (Paired predecessor argument)**.** *Suppose a term vertex v of the host is the target of e and its retained copies are operation-reachable in both expansions. Under the paired edge condition, e is an operation, and every source of e is a term vertex whose copies are operation-reachable on both sides.*

Proof. Either one-way branch contradicts reachability on that side by the previous result. In the remaining branch both sides retain corolla copies. Their expanded edges target the corresponding copies of v. The incoming-edge theorem forces their label, hence the original label of e, to be an operation. Every source of e is therefore term-typed. Its mapped incidences in the two copies and the predecessor reachability theorem give reachability on each side. □

Lean: `Linear.ExpansionPaths.predecessor`

Theorem 34.11 (Paired host-path reflection)**.** *If a host path ends at a term vertex reachable on both sides, all its edges are operations. In addition, every source of every edge on the path is term-typed and reachable on both sides.*

Proof. Induct on the nonempty host path. For a singleton path use the predecessor lemma. For a path $e :: f :: E$, apply the induction hypothesis to $f :: E$. An adjacency witness is both a target of e and a source of f; the strengthened induction conclusion supplies its type and reachability on both sides. Apply the predecessor lemma to e, then combine its conclusion with that for the tail. Repeated edges and any number of replaced holes are allowed. □

Lean: `Linear.ExpansionPaths.path_operations`

34.5 Actual focussed expansions and their obstruction

Retain the host focus by a corolla copy and require every other filling to be globally focus-free. The expanded focus is the tagged distinguished edge of that copy, with the same mode and the retained copies of the original endpoints. The copy-list equalities give its singleton incidences. Host-map injectivity preserves distinct endpoints, and the unchanged boundary constructors preserve the prohibition on an exposed input-to-output focus. For uniqueness, an edge from the focus copy equals its distinguished edge; any edge from another filling is globally focus-free.

Lean: `Linear.ExpansionPaths.focussed`

Initiality and finality are unchanged because both the mode and the input status of the focus source are unchanged. Once global hole freedom of the expanded graph is supplied, the construction packages the graph and its proved focus as an actual state.

Lean: `Linear.ExpansionPaths.focussed_boundary`

Lean: `Linear.ExpansionPaths.state`

Theorem 34.12 (Actual-state paired obstruction)**.** *If the host has a path containing a non-operation edge and ending at its focus source, and the two filling families satisfy the paired edge condition, at least one of the two expanded states is not rooted.*

Proof. If both states were rooted, Lemma 23 would make both actual focus sources operation-reachable. They are the retained copies of the host focus source, which has term type. Paired host-path reflection would therefore make every edge of the given path an operation, contradicting its specified non-operation edge. Classical case distinction turns this incompatibility into the stated disjunction of non-rootedness. □

Lean: `Linear.ExpansionPaths.pair_not_rooted`

34.6 Concrete pre-templates and shallow specimens

Definition 34.13 (Concrete pre-template)**.** At a fixed depth bound, a concrete pre-template is an interface-indexed binary relation on hypernets, with a proof that both members of each related pair are globally focus-free. The relation and its domain condition are defining data, not a law asserting operational behavior.

Lean: `Linear.ConcretePreTemplate`

Definition 34.14 (Concrete output-closure)**.** A concrete pre-template is output-closed when every related pair has a one-way member on at least one side. This is the paper's output-closure predicate, using the concrete one-way definition.

Lean: `Linear.ConcretePreTemplate.OutputClosed`

For each original edge, the shallow specimen records one of two kinds of data: retained corolla copies on both sides, or an actual hole label on that edge together with a related pair of fillings. Thus arbitrary changes to non-hole edges are not permitted by this specimen definition.

Lean: `Linear.SpecimenEdge`

Definition 34.15 (Concrete shallow specimen)**.** A shallow specimen consists of a globally unique-hole context of interface $\star \to \epsilon$, a distinguished focus, and two simultaneous filling families with the preceding edge-wise data. The erasure of the focussed context must belong to the supplied allowed-context predicate. Both expanded graphs must be globally hole-free. These fields express actual typing, context, relation-membership, and closedness restrictions; they do not include rootedness, path reflection, or a context-traversal law as assumptions.

Lean: `Linear.ShallowSpecimen`

This is the shallow restriction of the paper's specimen construction. It permits arbitrarily many outer holes and arbitrarily deep boxed contents in the fillings. It does not replace holes inside the original host's boxes. Compatibility with arbitrary addressed substitution and extension to deep hole occurrences remain required before identifying it with the paper's full specimen notion.

A focus label cannot equal a hole label, by disjointness of primitive labels at either zero or successor depth. Therefore the focus edge must be in the retained branch of the specimen data.

This proves existence of its two corolla copies, which are chosen to construct the focussed expansions.

Lean: `Linear.focusLabel_ne_hole`

Lean: `Linear.ShallowSpecimen.focus_copies`

Lean: `Linear.ShallowSpecimen.focusCopies`

Every other filling is globally focus-free. For a retained copy, use the original focus's uniqueness and the copy-label theorem. For a filled hole, use the pre-template's focus-free domain. These derived facts and the specimen's proved graph closedness construct its two actual states.

Lean: `Linear.ShallowSpecimen.other_free`

Lean: `Linear.ShallowSpecimen.leftState`

Lean: `Linear.ShallowSpecimen.rightState`

34.7 Lemma 26(3) and the boundary bridge for shallow specimens

Theorem 34.16 (Output-closure supplies paired reflection)**.** *An output-closed concrete pre-template supplies the paired edge condition for every one of its shallow specimens.*

Proof. At a retained edge, take the two recorded copies. At a filled hole, apply output-closure to the recorded related pair to obtain a one-way side. The side may change at the next hole. □

Lean: `Linear.ShallowSpecimen.outputClosedPair`

Theorem 34.17 (Concrete shallow Lemma 26(3))**.** *For a shallow specimen of an output-closed pre-template, any context path to its focus source containing a non-operation edge implies that at least one of its two actual states is not rooted.*

Proof. The preceding theorem supplies the paired edge condition. The focus-copy and focus-freedom lemmas supply actual expanded states. Apply the actual-state paired obstruction to those states and the given context path. □

Lean: `Linear.ShallowSpecimen.not_both_rooted`

Equivalently, when both specimen states are rooted, every edge of every context path to the focus source is an operation. The formal proof applies paired host-path reflection directly to the two operation-reachability witnesses supplied by Lemma 23. This is the path consequence used in the paper's Lemma 39(2), here proved for shallow specimens; it is not yet the full entering/exiting classification or context-traversal theorem.

Lean: `Linear.ShallowSpecimen.rooted_context_path`

The proof uses the same incoming-incidence uniqueness and one-way extraction as Proof C.9. Rather than selecting a last hole and changing substitution order, it reasons backwards simultaneously on both sides along the supplied path. The paired predecessor lemma isolates exactly the contradiction that the paper obtains at its last hole. This proof organization avoids assuming an unproved interchange theorem.

Theorem 34.18 (Concrete shallow Lemma 39(1))**.** *The two actual states of a shallow specimen agree on initiality and finality.*

Proof. Each expanded focus has the same mode and source-input status as the original context focus. Compose the left and right equivalences supplied by the focussed-expansion boundary theorem. Output-closure and rootedness are not needed for this boundary assertion. □

Lean: `Linear.ShallowSpecimen.boundary_agreement`

34.8 A constructed two-hole focussed specimen

The earlier one-hole context has exactly one hole occurrence: inspection of its labels rules out the focus and value edges, leaving its selected hole edge. This proves global hole-label uniqueness and the absence of any retained hole when that edge is removed.

Lean: `Linear.Examples.oneWayContext_hole_only`

Lean: `Linear.Examples.oneWayContext_unique`

Lean: `Linear.Examples.oneWayContext_no_retained`

Replace that hole by the existing chain of two distinct holes. The exact freshness theorem gives a globally unique-hole context, since the host retains no hole and the inserted chain has distinct hole labels. Its focus is retained by the previously checked shallow focus-free plugging operation.

Lean: `Linear.Examples.twoHoleFocusedContext`

Lean: `Linear.Examples.twoHoleFocusedContext_unique`

Lean: `Linear.Examples.twoHoleFocus`

The simultaneous filling family replaces both new holes by variable corollas and retains the focus and value by their original corollas. Define a pre-template relating a graph to itself when it is focus-free and one-way. This relation has the required focus-free domain, and is output-closed by its one-way premise. At each hole the pair of variable corollas belongs to this relation; at every other edge the corolla-copy fields hold by construction.

Lean: `Linear.Examples.twoHoleFilling`

Lean: `Linear.Examples.oneWayEquality`

Lean: `Linear.Examples.oneWayEquality_closed`

Lean: `Linear.Examples.twoHoleFilling_edges`

Every filling is globally hole-free. A variable corolla has no hole, while a hole in a retained corolla would supply a retained hole occurrence in the earlier one-hole context, already excluded. The expansion hole-freedom lemma therefore proves closedness. These data assemble an actual two-hole specimen, with the unrestricted allowed-context predicate for this example.

Lean: `Linear.Examples.twoHoleFilling_closed`

Lean: `Linear.Examples.twoHoleSpecimen`

The second hole by itself is a path to the focus source and is not an operation. The concrete shallow Lemma 26(3) proves the required disjunction of non-rootedness for the two constructed

states, for every operation classification. Their initial/final agreement follows from the concrete specimen boundary theorem.

Lean: `Linear.Examples.twoHoleSpecimen_not_rooted`

Lean: `Linear.Examples.twoHoleSpecimen_boundary`

34.9 Alternating one-way choices: a necessary regression

Use an operation signature with one unary operation, and form both its operation corolla and a variable corolla. Relabel the existing two-hole chain to this signature, leaving its hole interfaces and incidences unchanged. On the left fill the first hole with a variable and the second with an operation; on the right reverse these choices.

Lean: `Linear.AlternatingFillings.ops`

Lean: `Linear.AlternatingFillings.variableWire`

Lean: `Linear.AlternatingFillings.operation`

Lean: `Linear.AlternatingFillings.context`

Lean: `Linear.AlternatingFillings.left`

Lean: `Linear.AlternatingFillings.right`

The variable corolla is one-way: reachability of its output would force its incoming variable edge to be an operation. The operation corolla is not one-way, since its unique edge explicitly supplies an operation path between its term input and output.

Lean: `Linear.AlternatingFillings.variable_oneWay`

Lean: `Linear.AlternatingFillings.operation_not_oneWay`

The paired edge condition holds: use the left one-way filling at the first hole and the right one-way filling at the second. Neither entire family is one-way edge by edge, as evaluation at the second left hole or the first right hole yields the non-one-way operation corolla. Finally, the two-edge host path cannot end at an output reachable on both sides. Otherwise paired path reflection would make its hole labels operations, a contradiction. This regression rules out an accidental strengthening of pointwise output-closure to a single global one-way-side choice.

Lean: `Linear.AlternatingFillings.output_closed`

Lean: `Linear.AlternatingFillings.no_global_oneWay_side`

Lean: `Linear.AlternatingFillings.not_both_reachable`

35 Sequential realization of simultaneous substitution

The simultaneous construction is now connected to the existing addressed replacement operation. Every finite filling family can be performed by ordinary shallow substitutions, and every completed schedule described below has the same result up to explicit graph isomorphism. This supplies the shallow compatibility obligation in Appendix B. It does not identify these structural substitutions with UAM transitions, nor by itself prove interchange for arbitrary deep sites; the latter is established in Section 36.

35.1 Peeling one filling

Definition 35.1 (Residual filling family)**.** Fix a finite typed graph C and a family $H = (H_f)_{f \in E_C}$ of interface-compatible fillings. Select $e \in E_C$ and put $C' = C[e := H_e]$, using ordinary replacement. A retained edge of C' is an original edge $f \neq e$, and an inserted edge is an edge x of H_e. The residual family R assigns H_f to the former and the identity corolla with label $\ell(x)$ to the latter. Thus subsequent expansion completes the original assignments without replacing the newly inserted edges nontrivially.

Lean: `Graph.Net.ExpansionPeel.rest`

Lemma 35.2 (Peeling isomorphism)**.** *There is an explicit interface-preserving isomorphism*

$$C'[R] \;\cong\; C[H].$$

No acyclicity, absence of self-loops, or restriction on edge labels is needed.

Proof. First construct the bijection of internal vertices. An original host vertex becomes its host copy. An internal vertex already inserted from H_e becomes the copy tagged by e in $C[H]$. An internal vertex subsequently inserted from H_f, with $f \neq e$, becomes the copy tagged by f. A corolla has no internal vertices, so there is no fourth case. The inverse retains host vertices and splits a filling tag f according to whether $f = e$. In the equality case, transporting along that equality places the vertex in the already inserted H_e carrier; otherwise it is placed in the still-pending H_f carrier. Case analysis proves both inverse identities.

Lean: `Graph.Net.ExpansionPeel.vertexTo`

Lean: `Graph.Net.ExpansionPeel.vertexFrom`

Lean: `Graph.Net.ExpansionPeel.vertices`

The edge bijection sends an edge of a pending H_f to the pair (f, x) and the sole edge of the corolla for an inserted $x \in E_{H_e}$ to (e, x). Its inverse again splits on equality with e. The only extra inverse equation says that every element of Fin(1) is zero.

Lean: `Graph.Net.ExpansionPeel.edgeTo`

Lean: `Graph.Net.ExpansionPeel.edgeFrom`

Lean: `Graph.Net.ExpansionPeel.edges`

It remains to verify incidence, rather than merely count vertices and edges. The composite inclusion of an original host vertex agrees with its simultaneous inclusion. The corresponding

assertion for a vertex of H_e holds directly for internal vertices; for boundary vertices it reduces to the host assertion at the appropriate attachment port.

Lean: `Graph.Net.ExpansionPeel.map_host`

Lean: `Graph.Net.ExpansionPeel.map_filled`

For a retained edge f, each source or target port after the first replacement is the host inclusion of its original port. This follows by taking the indexed element of the mapped incidence list. Consequently inclusion of any vertex of a pending H_f commutes with the vertex bijection: the internal case is definitional, and each boundary case uses the appropriate retained-port identity followed by the host identity.

Lean: `Graph.Net.ExpansionPeel.sourcePort_retained`

Lean: `Graph.Net.ExpansionPeel.targetPort_retained`

Lean: `Graph.Net.ExpansionPeel.map_other`

For an inserted edge x of H_e, the input of its residual corolla maps to the simultaneous inclusion of the corresponding source port of x; the output maps to the corresponding target port. These identities use the already proved port formulas for an inserted edge of an ordinary plug, followed by the filled-vertex identity above.

Lean: `Graph.Net.ExpansionPeel.map_corolla_input`

Lean: `Graph.Net.ExpansionPeel.map_corolla_output`

These bijections and incidence identities form the promised isomorphism. Vertex and edge labels are preserved by inspection of the tags. For a pending filling, preservation of its source and target lists follows by composing list maps and applying the vertex identity pointwise. For a corolla, its ordered list of inputs or outputs enumerates all port indices. Mapping this list through the corresponding port function recovers exactly the incidence list of x in H_e. This completes the proof of the lemma, including the ordering of incidence. □

Lean: `Graph.Net.ExpansionPeel.iso`

35.2 Stopping at arbitrary identity copies

The termination condition must permit arbitrary graphs certified as corolla copies, rather than require the literal corolla constructor. The existing copy record specifies a sole edge with the original label and the full ordered input and output lists.

Lemma 35.3 (A corolla copy has no internal vertices)**.** *If H_e is a corolla copy, its internal carrier is empty.*

Proof. If v were internal, source coverage would place it in the source list of some edge. That edge must be the sole edge of the copy, whose source list contains only input vertices. The disjointness of input and internal constructors gives a contradiction. □

Lean: `Linear.ExpansionPaths.copy_no_internal`

Lemma 35.4 (Restoring a host from copies)**.** *If every H_e is a corolla copy, then $C[H] \cong C$.*

Proof. The internal-vertex bijection drops the host tag; its putative filling cases are impossible by the preceding lemma. Its inverse inserts the host tag. The edge bijection forgets the owning tag's unique filling edge, sending (e, x) to e; the inverse chooses the sole edge of the copy. Uniqueness proves the inverse equations. Labels are the original labels by the copy certificate. For each incidence list, expansion of the copy gives the original host list mapped through host inclusion. The internal-vertex bijection cancels that inclusion, including at boundary vertices, so mapping back restores the original list. □

Lean: `Linear.ExpansionPaths.copiesVertices`

Lean: `Linear.ExpansionPaths.copiesEdges`

Lean: `Linear.ExpansionPaths.copiesIso`

35.3 Certified executions and order independence

Fix a hypernet alphabet, a depth bound, and an outer interface.

Definition 35.5 (Shallow substitution run)**.** A shallow substitution run from C to D of length n is either the empty run ($n = 0$, $D = C$), or an ordinary addressed replacement at a shallow edge of C, followed by a run of length $n - 1$ from the resulting graph. Each step uses the existing operation plugAt. These are structural graph replacements, not machine evaluation steps.

Lean: `Linear.ShallowRun`

Definition 35.6 (Certified filling execution)**.** An execution is indexed by the current host C, its prescribed family H, a length n, and a final graph D. It may stop with $n = 0$ and $D = C$ when every assignment is certified as a corolla copy. Otherwise it selects an edge e, performs $C[e := H_e]$, and continues with the residual family defined above. The length increases by one. The definition records the actual steps and a legitimate stopping condition; it does not assume any isomorphism of the final result.

Lean: `Linear.ExpansionSequential.Execution`

Every certified execution yields a shallow substitution run. Induction sends the stopping constructor to the empty run, and each peeling constructor to the corresponding addressed replacement followed by the inductively obtained run.

Lean: `Linear.ExpansionSequential.Execution.ordinary`

Theorem 35.7 (Correctness of a completed execution)**.** *An execution from (C, H) to D gives $D \cong C[H]$.*

Proof. Induct on the execution. At termination, choose the certified corolla copies and invert the host-restoration isomorphism. At a step selecting e, the induction hypothesis identifies D with $C'[R]$. Compose this isomorphism with the peeling isomorphism $C'[R] \cong C[H]$. □

Lean: `Linear.ExpansionSequential.Execution.iso`

Theorem 35.8 (Order independence)**.** *Any two completed executions of the same prescribed family have isomorphic final graphs, even if their lengths differ. They also represent the same recursive graph class.*

Proof. If their results are D and E, correctness gives isomorphisms $D \cong C[H]$ and $E \cong C[H]$. Compose the first with the inverse of the second. An ordinary graph isomorphism lifts to a recursive graph isomorphism, which implies equality in the recursive quotient. This does not assert confluence for runs that change the prescribed fillings, or for arbitrary UAM transitions. □

Lean: `Linear.ExpansionSequential.Execution.order_independent`

Lean: `Linear.ExpansionSequential.Execution.order_independent_modulo`

35.4 Existence and a finite length bound

Definition 35.9 (Pending list)**.** A list L of host edges is sufficient if every edge absent from it has a corolla-copy assignment. Repetitions are allowed. After selecting e, filter the tail of L: remove every occurrence of e, and send each other edge f to its retained copy in $C[e := H_e]$. Call the resulting list L'.

Lean: `Linear.ExpansionSequential.remaining`

Filtering cannot increase length. Furthermore, every retained edge whose original lies in the tail appears in L'. The first claim is the standard length bound for filtering with a partial map; for the second, the original edge is a witness in the membership characterization of that map.

Lean: `Linear.ExpansionSequential.remaining_length`

Lean: `Linear.ExpansionSequential.remaining_mem`

Every edge outside L' has a corolla-copy assignment in the residual family. For a retained f, absence from L' and the membership lemma imply absence from the original pending list: $f \neq e$ excludes its head. Use the original copy certificate. An inserted edge has a literal corolla assignment, whose certificate uses its unique element of Fin(1) and its defining incidence lists.

Lean: `Linear.ExpansionSequential.remaining_copies`

Theorem 35.10 (Finite sequential realization)**.** *For every sufficient list L, there are $n \leq |L|$ and a final graph D with a certified execution of (C, H) of length n ending at D. Consequently every finite family has an ordinary shallow substitution run with final graph isomorphic to $C[H]$. Its length is at most the length of the host's finite edge enumeration.*

Proof. Use strong induction on the pending-list length, allowing both the host and family to vary. An empty list certifies every assignment as an identity copy, so stop. For $e :: L$, first replace e with H_e. The residual list has length at most $|L|$, strictly less than the original length, and the preceding lemma supplies the required certificates outside it. Apply the induction hypothesis and prepend the selected step. Its total length is at most $|L| + 1$. For an arbitrary finite family, use the host's complete edge enumeration; there are no edges outside it. Forget the resulting execution to obtain an ordinary run, and apply correctness. The enumeration may contain repetitions, so the stated bound is its list length, not an asserted minimal number of substitutions. □

Lean: `Linear.ExpansionSequential.realize_list`

Lean: `Linear.ExpansionSequential.realize`

The same witness supplies equality of the final result and simultaneous expansion in the recursive graph quotient: lift the constructed graph isomorphism and use quotient soundness.

Lean: `Linear.ExpansionSequential.realize_modulo`

35.5 Two-hole examples and scope

Both filling families in the alternating one-way example admit ordinary runs of length at most two. Apply finite realization to each family; the explicit context enumeration has length two.

Lean: `Linear.AlternatingFillings.sequential_realizations`

For any filling family on this context, there are completed executions starting by replacing hole 0 and by replacing hole 1. Each requires at most one further step, and their endpoints are isomorphic. To prove this, verify that the two-element list contains every context edge. After either initial choice, the residual-list certificate permits finite realization on the remaining list of length at most one. Prepend the chosen step to each execution and apply order independence. The statement retains the residual execution witnesses after each distinct first choice, so it records which hole was filled first.

Lean: `Linear.AlternatingFillings.reverse_schedule`

The dependency chain is now: explicit peeling and identity-copy isomorphisms, then correctness of executions, then finite realization and schedule independence. These results remove a representation gap between simultaneous shallow specimens and ordinary graph replacement. Section 36 adds a quotient of host/site pairs and independent deep-site interchange. Section 37 constructs recursive deep specimens. Adequacy for paper tuple substitution and transport of the full UAM structure along deep replacement schedules remain separate obligations.

36 Independent deep substitution and pointed host classes

This section completes the host/site quotient and independent-site interchange obligation formerly listed in Appendix B. The theorem constructs surviving addresses after each substitution, at arbitrary depth, and compares the final graphs by recursive isomorphism. It complements nested associativity: independence excludes replacing a box and one of its descendants as the two independent steps. No hole-freshness or intermediate context-validity assumption is needed for the graph isomorphism.

36.1 Interchange at distinct shallow edges

Let $e \neq f$ be edges of a finite typed graph C, and let H and K have their respective interfaces. After replacing e, write f/e for the retained occurrence of f, and define e/f symmetrically.

Theorem 36.1 (Shallow interchange)**.** *There is an explicit ordered-interface-preserving isomorphism*

$$(C[e := H])[f/e := K] \;\cong\; (C[f := K])[e/f := H].$$

The host may contain cycles or self-loops.

Proof. The internal carriers are $(V_C + V_H) + V_K$ and $(V_C + V_K) + V_H$. The bijection fixes the original host component, sends the first filling component to its new position, and sends the second filling component to its new position. The inverse performs the reverse rearrangement; both identities follow by the three cases. The edge bijection similarly fixes the surviving original edges, each different from both selected edges, and exchanges the positions of the two filling-edge carriers. Its subtype proofs follow from $e \neq f$ and the fact that the original surviving edge differs from each selected edge. Again the three cases give the inverse identities.

Lean: `Graph.Net.PlugInterchange.vertices`

Lean: `Graph.Net.PlugInterchange.edges`

For any retained edge, its source and target attachment ports after one replacement are the original ports mapped through host inclusion. Taking an indexed element of the mapped incidence list proves these two statements.

Lean: `Graph.Net.PlugInterchange.sourcePort_retained`

Lean: `Graph.Net.PlugInterchange.targetPort_retained`

The composite inclusion of a host vertex commutes with the carrier bijection, by cases on internal, input, and output vertices. For a vertex of H, its inclusion after inserting H first agrees with its inclusion when H is inserted second. The internal case is definitional. The input and output cases use the retained-port formula for e after inserting K, followed by the host-vertex identity. For K the argument is reversed: first use its retained-port formula after inserting H, then the host identity. These three commuting maps cover every vertex used by an incidence list.

Lean: `Graph.Net.PlugInterchange.map_host`

Lean: `Graph.Net.PlugInterchange.map_first`

Lean: `Graph.Net.PlugInterchange.map_second`

Labels are unchanged under the carrier rearrangement. For source and target lists, split an edge into surviving host, H, or K origin; compose the list maps and apply the corresponding commuting identity pointwise. This verifies ordered incidence and completes the isomorphism, not merely a bijection of carriers. □

Lean: `Graph.Net.PlugInterchange.iso`

36.2 Changing a box beside another substitution

A change to the body of box edge f leaves the label of every $e \neq f$ unchanged. Unfolding the conditional label update proves this directly. The source and target interfaces at e are therefore equal to their original interfaces. The surviving shallow site is obtained by transporting its interface indices along these equalities.

Lean: `Linear.replaceBox_other_label`

Lean: `Linear.Site.afterBox`

Lemma 36.2 (Mixed shallow and box interchange)**.** *Replacing a shallow edge e and changing the body of a distinct box edge f commute up to ordinary graph isomorphism, with the shallow site transported to its unchanged interface.*

Proof. A body update preserves the host's entire outer incidence structure. To reuse the general incidence-transport construction, first regard its edge labels as related by the always-true relation. Identity carrier bijections then give an incidence isomorphism from the original host to the updated host. This auxiliary relation is used only for transporting incidence; it is not the final equality condition on labels.

Lean: `Linear.replaceBox_incidence`

Lift that incidence isomorphism through substitution at e, transporting the replacement's boundary indices along the equal interfaces. Use its vertex and edge bijections, internal labels, and source and target proofs for the desired final isomorphism. Check actual edge-label equality separately: a retained original edge has its updated box label precisely when it is f, while every inserted edge has its original replacement label. Equality with f is equivalent to equality of its retained copy with the retained copy of f. This proves literal label equality and hence the stated ordinary graph isomorphism. □

Lean: `Linear.plug_retained_box_iso`

Lemma 36.3 (Distinct box updates commute)**.** *Changing the bodies at two distinct box edges, in either order, yields equal graph records.*

Proof. All carriers and incidence fields are unchanged. For the label function, split an edge into the cases that it is the first selected edge, the second selected edge, or neither. Distinctness excludes the two updates interfering, so the same final label is obtained in every case. □

Lean: `Linear.replaceBox_comm`

36.3 Structural independence and residual addresses

Definition 36.4 (Independent sites)**.** A certificate of independence is generated structurally as follows. Two distinct shallow edges are independent. A shallow edge and a site inside a different box are independent. Sites inside two different boxes are independent, regardless of their inner depths. Two sites inside the same box are independent when their inner sites are independent. Finally independence can be reversed. These constructors allow any finite common chain of enclosing boxes before the addresses diverge; they give no constructor relating a selected edge to its own descendant. The certificate contains structural occurrence information, not an assumed interchange theorem.

Lean: `Linear.IndependentSites`

Definition 36.5 (Interchange result)**.** For independent s, t in C and replacements A, B, an interchange result contains a residual site t/s in $C[s := A]$, a residual site s/t in $C[t := B]$, and a recursive isomorphism

$$(C[s := A])[t/s := B] \simeq (C[t := B])[s/t := A].$$

Here the residual sites are constructed by following retained occurrences, including their enclosing boxes. They are not chosen by searching for graphs with equal results.

Lean: `Linear.InterchangeResult`

Theorem 36.6 (Independent deep interchange)**.** *Every structural independence certificate and pair of interface-compatible replacements produces an interchange result.*

Proof. Induct on the independence certificate, with replacements arbitrary. For distinct shallow edges, use their retained copies and the shallow interchange isomorphism. For a shallow edge and a site inside another box, retain that box address after the shallow plug; in the other order, use the shallow site transported across the body update. The mixed interchange lemma supplies the isomorphism.

For sites in different boxes, retain each enclosing edge and inner address after updating the other box. The unchanged-label lemma supplies the necessary box-label certificates. Distinct box updates commute as records, so recursive reflexivity gives the comparison.

For two sites in a shared box, apply the induction hypothesis in its body. Lift each residual address through the updated enclosing box. Performing the second substitution updates that same box a second time; the overwrite lemma reduces each side to a single update with the final inner body. Lift the inner recursive isomorphism through this common outer update. This case is what permits arbitrary shared nesting depth.

For a reversed independence certificate, apply the induction hypothesis with replacements reversed, exchange its two residual sites, and invert its isomorphism. All cases construct both addresses and their comparison. □

Lean: `Linear.IndependentSites.interchange`

Transporting a site's interface along equalities does not change its selected hole label: eliminate the equalities and the claim is reflexive. Induction on independence then proves that both residual addresses select the original hole labels. The shallow and separate-box cases are definitional; the mixed case uses interface transport and the unchanged-label lemma; the shared-box case uses the induction hypothesis; reversing independence exchanges the two assertions. This also covers non-hole sites, whose selected label is absent.

Lean: `Linear.Site.selectedHole_reinterface`

Lean: `Linear.IndependentSites.residual_selectedHole`

The two final recursive graph classes are equal by quotient soundness applied to the constructed isomorphism. They have unique holes simultaneously, by invariance of that property under recursive isomorphism. The latter equivalence concerns the final graphs; neither intermediate graph is required to be a valid context.

Lean: `Linear.IndependentSites.interchange_modulo`

Lean: `Linear.IndependentSites.unique_iff`

36.4 A quotient of hosts with selected occurrences

Definition 36.7 (Pointed hypernet)**.** A pointed hypernet consists of a host $C : I \to O$ at depth bound d and a selected site s with local depth r and interface $J \to K$. All these type indices are fixed for a particular space of pointed hypernets, while the host and its carrier names may vary.

Lean: `Linear.PointedHypernet`

Definition 36.8 (Pointed renaming)**.** Two pointed hypernets are equivalent under the equivalence relation generated by structural site correspondences. Its constructors are a single existing SiteIso correspondence, reflexivity, reversal, and composition. Thus the relation consists of finite renaming zigzags. Equality of substituted results is not a generator: the selected occurrence remains part of the object being quotiented. This construction does not assert that the previously defined SiteIso itself has already been proved to be an equivalence.

Lean: `Linear.PointedRenaming`

A single site correspondence includes an outer recursive host isomorphism: inspect either its shallow or inside constructor. A finite pointed renaming therefore relates recursively isomorphic hosts, by induction; identity, inverse, and composition of graph isomorphisms handle the three equivalence constructors.

Lean: `Linear.SiteIso.host`

Lean: `Linear.PointedRenaming.host`

Pointed renaming preserves plugging of a fixed replacement. For one structural correspondence use the existing site-substitution theorem. For reflexivity, reversal, and composition use the corresponding recursive isomorphism operations on results. If the replacements are also recursively isomorphic, compose this result with replacement congruence at the second selected site.

Lean: `Linear.PointedRenaming.plug`

Lean: `Linear.PointedRenaming.plug_congr`

Retained-hole inventories also agree along any pointed renaming. The single-step case is the existing site correspondence invariant; the other cases use reflexivity, symmetry, and transitivity of logical equivalence.

Lean: `Linear.PointedRenaming.retainedHole_iff`

Definition 36.9 (Pointed isomorphism class)**.** Use pointed renaming as a setoid relation, with its three equivalence constructors supplying the equivalence laws. A pointed isomorphism class is its quotient. The quotient identifies hosts and their selected occurrences together, with outer and local interfaces fixed.

Lean: `Linear.pointedSetoid`

Lean: `Linear.PointedModuloIso`

Definition 36.10 (Fully quotiented substitution)**.** Given a pointed class $[C, s]$ and a replacement class $[A]$, define

$$\mathsf{pointedPlug}([C, s], [A]) = [C[s := A]].$$

For a fixed pointed representative, the previously constructed replacement-quotient operation already makes the right-hand side independent of the representative of $[A]$. To eliminate the pointed quotient, use function extensionality and quotient induction on $[A]$. Pointed-renaming substitution invariance then proves equality of the result classes. On concrete representatives the defining equation is reflexive.

Lean: `Linear.pointedPlug`

Lean: `Linear.pointedPlug_mk`

Forgetting the selected site gives a well-defined host class, because pointed renaming preserves recursive host isomorphism. The retained-hole predicate likewise descends to pointed classes, using the retained-inventory equivalence and propositional extensionality.

Lean: `Linear.pointedHost`

Lean: `Linear.pointedRetainedHole`

Definition 36.11 (Quotient freshness)**.** The predicate $\mathsf{pointedFresh}([C, s], [A])$ means that no hole retained outside s occurs in A. Replacement-renaming invariance permits eliminating the replacement quotient. Retained-inventory invariance permits eliminating the pointed quotient: in either direction, transfer the retained-hole witness and apply the original freshness condition. Function and propositional extensionality turn these equivalences into the equalities required by quotient elimination.

Lean: `Linear.pointedFresh`

Theorem 36.12 (Sharp context validity on classes)**.** *If the host class of P has unique holes, then*

$$\mathsf{UniqueHoles}(\mathsf{pointedPlug}(P, [A])) \iff \mathsf{UniqueHoles}([A]) \wedge \mathsf{pointedFresh}(P, [A]).$$

Proof. Apply quotient induction to both arguments. The assertion becomes the existing sharp context criterion at the concrete selected site, with the supplied host-uniqueness hypothesis. No stronger disjointness condition on the whole host is introduced. □

Lean: `Linear.pointedPlug_unique_iff`

Transporting a site along a recursive host isomorphism leaves its pointed class unchanged: the transport specification is a single structural site correspondence, hence a generator of pointed renaming. Finally, applying fully quotiented substitution at either of the constructed residual pointed hosts gives equal result classes by deep interchange and quotient soundness. This is the independent substitution law expressed through the new operation.

Lean: `Linear.pointed_transport`

Lean: `Linear.IndependentSites.pointed_interchange`

36.5 Examples that distinguish occurrences

The second site in the earlier boxed two-hole chain selects its second inner hole. The two sites are independent by the common-box constructor applied to distinct shallow holes. Filling one by the repeated-hole fragment and the other by a variable fragment gives equal final classes in either order. The earlier collision theorem proves that one intermediate graph has a duplicated hole label; interchange still applies because it is a theorem about well-formed graphs, not only valid intermediate contexts.

Lean: `Linear.Examples.boxedSecondSite`

Lean: `Linear.Examples.boxedSitesIndependent`

Lean: `Linear.Examples.boxed_interchange`

A second host has two outer thunk inputs, no outputs or internal vertices, and two distinct box edges. Both edges have exactly the same single-hole body. Each source list contains its own input; both target lists are empty. Injectivity of the input constructor proves source uniqueness, while target uniqueness is vacuous. Inputs witness source coverage; there are no other vertices requiring coverage. Splitting the two-element edge type proves the source typings, and the empty target typings are reflexive. This constructs an actual finite graph. It need not be a unique-hole context, since the bodies deliberately repeat the same hole label.

Lean: `Linear.Examples.twinBoxes`

Select the sole inner hole of either box. The separate-box constructor certifies independence despite equality of the box labels. Fill the first with the value operation and the second with the other-value operation. Deep interchange proves agreement of the two orders; each residual address follows its own enclosing edge.

Lean: `Linear.Examples.twinSite`

Lean: `Linear.Examples.twinIndependent`

Lean: `Linear.Examples.twin_interchange`

The correctly transported first site in the renamed boxed chain gives the same pointed class as the original site, by the previously checked structural correspondence. In contrast, keeping its former numerical index selects the wrong hole and gives a different pointed class. If those classes were equal, quotient exactness would give a pointed renaming. Substitution invariance with the repeated-hole replacement would then give recursively isomorphic results, contradicting the existing wrong-address counterexample.

Lean: `Linear.Examples.pointed_renamed_first`

Lean: `Linear.Examples.pointed_wrong_address`

The proof dependency is explicit: shallow carrier interchange and local box-update laws yield arbitrary-depth residual construction; site correspondence invariants yield the pointed quotient and its substitution operation; recursive isomorphism connects interchange to equality of result classes. Section 37 constructs recursive deep specimens. Their adequacy for paper tuple substitution and the concrete UAM rules remain obligations in Appendix B.

37 Recursive deep specimens and their boundary and rooted-path laws

The shallow specimen construction now extends recursively through original box bodies. It supplies actual filled states and proves the conclusions of Lemma 26(3), Lemma 39(1), and the jointly rooted context-path consequence for this deeper representation. These are structural results toward the paper's specimen argument in §8.1 and Appendix F [1]. Finite original-hole tuple compatibility and the full concrete safety definitions are established later in Section 43. The complete specimen-interface interpretation remains a separate obligation.

37.1 Copies with an updated box label

Definition 37.1 (Specified-label copy)**.** A specified-label copy is a graph with a distinguished sole edge, a specified label on that edge, and source and target lists enumerating the full ordered input and output boundaries. Unlike an unchanged corolla copy, its label need not be the original host label: it may contain a new box body. The graph itself still satisfies all finite typed graph invariants. The literal corolla is such a copy, with its sole edge 0; every element of Fin(1) equals 0.

Lean: `Linear.LabelCopy`

Lean: `Linear.LabelCopy.corolla`

If the specified label is globally focus-free, so is the copy. Every edge equals its distinguished edge, whose label is the specified label, reducing the claim to the supplied label predicate.

Lean: `Linear.LabelCopy.focusFree`

Lemma 37.2 (A non-operation copy is one-way)**.** *If a copy's specified label is not an operation, its graph is one-way.*

Proof. Use the characterization by operation reachability of term outputs. Every output belongs to the sole edge's target list, because that list enumerates the whole output boundary. If such an output were operation-reachable, incoming-edge uniqueness would force that edge to be an operation. Its label equality contradicts the hypothesis. □

Lean: `Linear.LabelCopy.oneWay`

37.2 Recursive paired filling certificates

Definition 37.3 (Layered pre-template)**.** A layered pre-template chooses a concrete, interface-indexed pre-template T_d for every depth bound d. Each related pair is globally focus-free. This definition does not identify graphs or relations at different depth bounds. Section 45 constructs coherent families from a single global pre-template; arbitrary families need not be coherent.

Lean: `Linear.LayeredPreTemplate`

Definition 37.4 (Recursive paired filling)**.** The predicate $\mathsf{DeepFilling}_T(C, L, R)$ certifies two actual filled graphs L, R of the same interface and depth as C. A node chooses two outer filling families H, K, certifies each original edge as below, and sets $L = C[H]$ and $R = C[K]$ using the concrete expansion construction. It is mutually defined with the edge certificates, so every recursive body filling is itself such a finite construction.

Lean: `Linear.DeepFilling`

Definition 37.5 (Recursive edge certificate)**.** An original edge has one of three certificates. It can be retained by unchanged corolla copies on both sides. It can have a hole label and be filled by a pair related by T_d. Or it can be a box with bound/free metadata (b, f) and original body B: recursively fill B to bodies L, R, and use specified-label copies of the boxes (b, f, L) and (b, f, R) as the two outer fillings. The body interfaces remain one term input and $b + f$ term outputs. Recursion follows original boxes, not the interiors of newly supplied hole replacements. Retained holes are permitted by this graph relation; the specimen's final hole-freedom requirement excludes unresolved holes.

Lean: `Linear.DeepFillingEdge`

Every shallow specimen-edge certificate is a recursive edge certificate: use the retained or filled constructor without descending into any box. Thus this representation extends the earlier one directly.

Lean: `Linear.SpecimenEdge.deep`

Theorem 37.6 (Global focus freedom under recursive filling)**.** *If* $\mathsf{DeepFilling}_T(C, L, R)$ *and* C *is globally focus-free, then both* L *and* R *are globally focus-free.*

Proof. Induct on the depth bound. Each result edge is an edge in one outer filling, so it suffices to prove focus freedom of every filling. At depth zero an edge certificate is retained or filled. A retained copy inherits the original edge's focus freedom; a filled pair obtains it from the pre-template domain.

At successor depth the same cases apply, together with a box case. The original box's label is focus-free, so its original body is globally focus-free. Apply the induction hypothesis to the certified body filling. The two new box labels are therefore focus-free, and the specified-label copy lemma gives focus freedom of their outer copies. This accounts for all descendant edges, not just outer labels. □

Lean: `Linear.DeepFilling.focusFree`

For one edge, focus freedom of its original label already suffices for focus freedom of both fillings. Retained and hole cases are as above; the box case applies the global recursive theorem

to its body and then the copy lemma.

Lean: Linear.DeepFillingEdge.focusFree

A shallow focus must take the retained branch of a recursive edge certificate. Its label cannot equal a hole label, and cannot equal a box label. Excluding those two cases leaves the pair of unchanged focus-copy certificates.

Lean: Linear.DeepFillingEdge.focus_copies

Lemma 37.7 (Outer paired path hypothesis)**.** *If T_d is output-closed, the outer families of a recursive filling satisfy the paired expansion path hypothesis.*

Proof. A retained edge supplies its two unchanged copies. At a hole, output closure gives a one-way filling on at least one side, with the choice allowed to vary by edge. At a rebuilt box, its specified label is still a box and hence is not an operation. The left copy is therefore one-way by the non-operation copy lemma (as is the right copy, although one side suffices). This argument needs output closure only for the outer layer's related hole fillings. Inner focus freedom was proved recursively; inner output closure is unnecessary for this outer path obstruction. □

Lean: Linear.DeepFillingEdge.outputClosed

37.3 Actual deep specimen states

Definition 37.8 (Concrete recursive specimen)**.** A recursive specimen consists of a unique-hole context C with outer interface $[\mathsf{term}] \to []$, a distinguished shallow focus, an allowed erased context, two outer filling families, and a recursive edge certificate at every original outer edge. Both resulting expansions must be globally hole-free. These last fields certify that the constructed results are states; the record contains no assumed boundary agreement, rootedness obstruction, or path-reflection law.

Lean: Linear.DeepSpecimen

Its edge certificates assemble into a recursive paired filling. At the focus edge, the preceding retained-focus theorem supplies a pair of copy certificates; classical choice selects one such pair for the subsequent state constructors.

Lean: Linear.DeepSpecimen.filling

Lean: Linear.DeepSpecimen.focus_copies

Lean: Linear.DeepSpecimen.focusCopies

Every other original edge is globally focus-free by uniqueness of the original focus. Its recursive edge certificate therefore gives globally focus-free fillings on both sides. The expansion focus construction can now retain the original focus with its mode and mapped source and target. Together with the specimen's two closedness certificates this constructs actual left and right states.

Lean: Linear.DeepSpecimen.other_free

Lean: Linear.DeepSpecimen.leftState

Lean: Linear.DeepSpecimen.rightState

Output closure supplies the outer paired path hypothesis edge by edge, using the recursive edge lemma just proved.

Lean: `Linear.DeepSpecimen.outputClosedPair`

Theorem 37.9 (Deep specimen obstruction: Lemma 26(3))**.** *Suppose the outer pre-template is output-closed. If an original context path ends at the focus source and contains a non-operation edge, at least one of the two constructed states is not rooted.*

Proof. Apply the checked paired expansion obstruction to the two families. The retained focus copies and all other focus-free fillings have just been derived, the states are closed by their certificates, and the paired path hypothesis follows from output closure and the treatment of rebuilt boxes. If both states were rooted, their focus sources would be operation-reachable. Paired path reflection would make every edge of the original path an operation, contradicting the chosen edge. The original path is an outer context path; the fillings may occur arbitrarily deeply inside its box labels. □

Lean: `Linear.DeepSpecimen.not_both_rooted`

Equivalently, under joint rootedness every edge of any context path to the focus source is an operation. The direct proof applies paired path reflection to the two operation-reachability witnesses obtained from rootedness, using that the focus source has term type. This is the rooted-context path consequence needed in the later locality argument, not the entire entering/exiting classification of Lemma 39.

Lean: `Linear.DeepSpecimen.rooted_context_path`

Theorem 37.10 (Deep specimen boundary agreement: Lemma 39(1))**.** *The two actual states are initial simultaneously and final simultaneously.*

Proof. Expansion retains the focus mode and preserves whether its source is an input. Thus each expanded focus is initial exactly when the original focus is initial, and final exactly when the original focus is final. Compose the left equivalences with the inverses of the right equivalences. Output closure and rootedness are not required. □

Lean: `Linear.DeepSpecimen.boundary_agreement`

Each result graph also admits a finite ordinary shallow-substitution realization of its outer filling family, up to graph isomorphism. Apply finite sequential realization to the left family and then the right. These runs may rebuild a whole outer box edge from its already filled body. Consequently this result alone does not identify the recursive representation with the paper's original-hole-only tuple substitution. That separate adequacy theorem is proved in Section 43.

Lean: `Linear.DeepSpecimen.outer_realizations`

To embed an earlier shallow specimen, retain its context, focus, allowedness certificate, two families, and closedness proofs, and convert each shallow edge certificate to a recursive one.

Lean: `Linear.ShallowSpecimen.toDeep`

37.4 A genuinely deep closed-state example

Use two Boolean-named operations. Both have no term arguments; the true operation exposes one thunk of binder arity zero, while the false operation exposes no outputs. Let the unique hole have name 7 and one term input and output. Its corolla is the original box body. The replacement is a unary variable corolla.

Lean: Linear.DeepSpecimenExamples.ops

Lean: Linear.DeepSpecimenExamples.hole

Lean: Linear.DeepSpecimenExamples.body

Lean: Linear.DeepSpecimenExamples.wire

At every depth, the example pre-template relates equal graphs that are focus-free and one-way. Equality transfers focus freedom to the second member; output closure follows from the supplied one-wayness of the first. The variable wire is focus-free and hole-free by its label, and is one-way by the non-operation copy lemma.

Lean: Linear.DeepSpecimenExamples.template

Lean: Linear.DeepSpecimenExamples.template_closed

Lean: Linear.DeepSpecimenExamples.wire_free

Lean: Linear.DeepSpecimenExamples.wire_closed

Lean: Linear.DeepSpecimenExamples.wire_oneWay

The sole body edge is assigned the variable wire. Expand this family to obtain the filled body. The hole-label equation and related pair give a recursive filling certificate. The resulting body is hole-free because all its inserted graphs are the hole-free variable wire.

Lean: Linear.DeepSpecimenExamples.bodyFill

Lean: Linear.DeepSpecimenExamples.filledBody

Lean: Linear.DeepSpecimenExamples.body_filling

Lean: Linear.DeepSpecimenExamples.filledBody_closed

The outer graph has three edge names: maker, box, and stop. Its two internal vertices have thunk and term type, respectively. The maker runs from the unique term input to the thunk vertex; the box runs from that vertex to the term vertex; the stop operation consumes the term vertex and has no outputs. The box has no bound variables, one free term port, and the original hole corolla as its body. Finite enumerations list the two vertices and three edges. All nonempty incidence lists are singletons. Case analysis on the three edges proves source and target uniqueness. Each internal vertex has exactly the required incoming and outgoing occurrence; the input belongs to the maker's source, and no outputs exist. These observations prove coverage. The typing equations hold by the definitions of the two operations, the box interface, and the vertex labels.

Lean: Linear.DeepSpecimenExamples.Part

Lean: Linear.DeepSpecimenExamples.outer

The outer graph is globally focus-free: its operation labels contain no focus, and the box's body is a hole corolla. Insert a search focus at the term vertex immediately after the box,

using the existing verified focus-insertion construction. The vertex has term type and is not an output, so the construction gives a well-formed focussed context.

Lean: Linear.DeepSpecimenExamples.outer_free

Lean: Linear.DeepSpecimenExamples.context

Lean: Linear.DeepSpecimenExamples.focus

Its only hole occurrence is the sole edge inside the original box. To prove this, split the outer edge into the three original edges or the newly inserted focus. Operation and focus labels have neither shallow holes nor children. At the box, the shallow occurrence is not a hole; its only inner edge has index 0. Thus any two hole occurrences coincide, proving the unique-hole context property. There is an explicit witness for that deep occurrence, while no outer edge has a hole label.

Lean: Linear.DeepSpecimenExamples.context_hole_only

Lean: Linear.DeepSpecimenExamples.context_unique

Lean: Linear.DeepSpecimenExamples.context_has_deep_hole

Lean: Linear.DeepSpecimenExamples.context_no_shallow_hole

Lemma 37.11 (This context cannot be filled shallowly to a closed graph)**.** *No pair of families certified by the old shallow specimen-edge rules on this same context has a hole-free left expansion.*

Proof. Every shallow filled-hole branch is impossible, since there is no shallow hole label. Thus every left filling is an unchanged corolla copy. The identity-copy expansion theorem gives an isomorphism from that expansion to the original context. Recursive isomorphism preserves the explicit deep hole occurrence, contradicting claimed hole freedom of the expansion. □

Lean: Linear.DeepSpecimenExamples.no_shallow_closed_filling

For the new recursive family, retain both operation edges and the focus by their literal corollas, but replace the outer box by a box whose body is the already filled body. The recursive box certificate uses the body-filling proof and the literal specified-label copy certificates. Retained edges use their corolla identities. Each filling is hole-free: the rebuilt box inherits its body's hole freedom, and the operation and focus corollas contain no shallow or deep holes. The expansion is therefore globally hole-free.

Lean: Linear.DeepSpecimenExamples.filling

Lean: Linear.DeepSpecimenExamples.filling_edges

Lean: Linear.DeepSpecimenExamples.filling_closed

Package these data as a recursive specimen with unrestricted allowed contexts. Both sides use the displayed family. The original box edge alone forms a context path to the focus source and is not an operation. The deep obstruction theorem yields the disjunction of non-rootedness for every classification of operations. The general boundary theorem also gives initial/final agreement for the actual constructed states.

Lean: Linear.DeepSpecimenExamples.specimen

Lean: Linear.DeepSpecimenExamples.specimen_not_rooted

Lean: `Linear.DeepSpecimenExamples.specimen_boundary`

This example distinguishes the new representation from the old one by a proved impossibility result, not merely by its choice of notation. Finite tuple compatibility and concrete quasi-specimen restrictions are proved in Section 43. The remaining specimen work includes the locality, safety, and robustness bridges listed in Appendix B.

38 Exact search transport and reflection through recursive fillings

The preceding section constructs recursive fillings and actual specimen states. We now prove a complementary operational result: context search survives those fillings with its exact length, and a filled-graph search step at an original vertex either comes from the context or touches an original shallow hole. This is a concrete traversal theorem for the coordinate search relation. It also yields executions and rootedness of actual states obtained by inserting a focus after filling. No output-closure hypothesis is needed for these results.

The scope of this milestone is precise. The state construction here first fills a focus-free context and then inserts a focus at a transported coordinate. The earlier deep specimen construction expands an already focussed context. Section 39 proves compatibility of these constructions and identifies the boundary predicate below with entering/exiting in the original focussed context. Repackaging successor coordinates as allowed specimens and connecting search to the full UAM remain necessary to discharge the abstract `context_step` field.

38.1 Transporting individual search rules

Fix a graph $C : I \to O$ at depth d and a family H of replacements for its outer edges. Write $E = C[H]$ for simultaneous expansion and $j_H : V_C \to V_E$ for its injective host-vertex map.

Definition 38.1 (Search-preserving family)**.** A family preserves search when every original operation edge and every original variable edge is replaced by a literal corolla copy. No restriction is imposed here on replacements of other labels. In particular, this predicate alone does not assert that newly available steps can be reflected back to the context.

Lean: `Linear.ExpansionSearch.PreservesSearch`

For a search coordinate $p = (m, v)$, define $j_H p = (m, j_H v)$: the mode is unchanged.

Lean: `Linear.ExpansionSearch.position`

An operation view consists of its edge, operation symbol, singleton source, ordered argument list, and ordered thunk list. Given a copy of that edge, its expanded view uses the copy edge and maps its source, arguments, and thunks through j_H. The copy-incidence equalities prove the required source and target equations; mapping preserves the argument-list length.

Lean: `Linear.ExpansionSearch.operation`

Lemma 38.2 (Entry and return commute with expansion)**.** *The expanded operation's entry coordinate is j_H of the original entry coordinate. Its return coordinate after argument i is likewise j_H of the original return coordinate.*

Proof. For entry, split the argument list into the empty and nonempty cases. The empty case keeps the source and uses the operation's finish mode; the nonempty case selects its first mapped argument. For return, lookup at $i + 1$ commutes with list mapping. A successful lookup selects the mapped argument; a failed lookup keeps the mapped source with the same finish mode. □

Lean: `Linear.ExpansionSearch.operation_enter`

Lean: `Linear.ExpansionSearch.operation_resume`

Theorem 38.3 (Forward search transport)**.** *If H preserves search and $p \longrightarrow_C q$, then $j_H p \longrightarrow_E j_H q$.*

Proof. Consider the three constructors grouping the five search rules. A variable trigger uses its retained copy and the copied singleton source. Operation entry uses the expanded operation view and the entry lemma. Operation return maps the witnessed argument lookup and applies the return lemma. The argument order and the exclusion of thunk targets are preserved in each case. □

Lean: `Linear.ExpansionSearch.step`

If a context step exists, determinism on E forces every successor of $j_H p$ to equal its transported result. This conclusion does not require reflection of arbitrary steps at a hole boundary.

Lean: `Linear.ExpansionSearch.successor_unique`

For a recursive paired edge certificate, an original operation or variable cannot be a filled-hole edge and cannot be a box. Its certificate must therefore be retained, supplying copies on both sides. Applying this argument at each outer edge proves that both filling families preserve search. Changes inside box bodies are allowed throughout this argument.

Lean: `Linear.DeepFillingEdge.search_copies`

Lean: `Linear.deepFilling_preservesSearch`

38.2 Exact executions and successful search costs

Definition 38.4 (Coordinate search machine)**.** The states are all mode–vertex pairs of C, and the transition relation is coordinate search. A successful final coordinate has backtrack mode at an input vertex. This machine may be defined on a context containing holes; it is not asserted to be a closed UAM state. No edge targets an input, so a final coordinate has no successor. Search determinism gives determinism of this machine.

Lean: `Linear.coordinateMachine`

Lean: `Linear.coordinateMachine_deterministic`

Theorem 38.5 (Exact execution transport)**.** *Every n-step context execution $p \longrightarrow_C^n q$ yields $j_H p \longrightarrow_E^n j_H q$ for a search-preserving family.*

Proof. Induct on the execution. The empty execution maps to the empty execution. For a nonempty execution, transport its first step and prepend it to the inductively transported tail. No extra step is inserted and no step is deleted. □

Lean: `Linear.ExpansionSearch.steps`

The host map retains the mode and whether a vertex is an input; hence finality of $j_H p$ is equivalent to finality of p. This is checked for each of the internal, input, and output vertex constructors.

Lean: `Linear.ExpansionSearch.final_iff`

Theorem 38.6 (Uniqueness of the successful cost)**.** *Suppose a context execution from p finishes successfully after n steps. Every successful expanded execution from $j_H p$ has length n and ends at the transported context result.*

Proof. Transport the context run and its finality. Compare the supplied successful run with this run in the deterministic coordinate machine. The previously proved uniqueness theorem for successful executions gives equality both of lengths and of endpoints. A separate exclusion of hole boundaries is unnecessary because a complete successful context run has already been supplied. □

Lean: `Linear.ExpansionSearch.final_run_unique`

38.3 Realizing executions as actual states

A typed coordinate of a closed-interface context is sent through j_H with its term-typing proof: host expansion preserves vertex labels. The initial search coordinate remains the initial coordinate at the unique input.

Lean: `Linear.TypedPosition.expand`

Lean: `Linear.TypedPosition.expand_initial`

Theorem 38.7 (Execution realization)**.** *On a focus-free, hole-free graph, an n-step execution between typed coordinates induces an n-step search execution between the recursive isomorphism classes of their actual focussed realizations.*

Proof. Induct on the coordinate execution. In the empty case the two typed coordinates are equal by equality of their underlying pairs. In the successor case, search preserves term typing, so the intermediate coordinate can be realized as an actual state. The established coordinate-to-state search theorem supplies the first state step, and the induction hypothesis supplies the tail. Literal state search embeds into recursive state search, which is the relation on the classes used here. □

Lean: `Linear.TypedPosition.realize_steps`

An execution beginning at the initial coordinate consequently proves rootedness of the realized endpoint. Its first realized state is initial; the concrete characterization of rootedness by a run

from an initial class applies.

Lean: `Linear.TypedPosition.realize_rooted`

38.4 Reading a step back from the expanded graph

Definition 38.8 (Hole boundary)**.** A context coordinate (m, v) is at a hole boundary if some original shallow edge has a hole label and either m is search and v is one of that edge's sources, or m is backtrack and v is one of its targets. This is a predicate on the outer context graph. A hole strictly inside a box does not by itself satisfy this predicate.

Lean: `Linear.ExpansionSearch.HoleBoundary`

Lemma 38.9 (Incident host vertices determine incident owners)**.** *If an expanded edge owned by e has source $j_H v$, then v is a source of e. The analogous statement holds for targets.*

Proof. Unpack membership in the mapped incidence list of the inserted edge. An internal vertex of the replacement cannot equal a host vertex because their carriers are disjoint. An output cannot occur in a source list, and an input cannot occur in a target list. In the remaining boundary case, injectivity of j_H identifies v with the corresponding original source or target port. □

Lean: `Linear.ExpansionSearch.source_owner_mem`

Lean: `Linear.ExpansionSearch.target_owner_mem`

Lemma 38.10 (Reading back an operation copy)**.** *An operation view whose edge belongs to a retained corolla has an original operation view with matching source and ordered arguments under j_H.*

Proof. The copy has only one edge, so its label identifies the original label as the same operation. The existence-of-view theorem constructs an original view. Transport that view through the copy. It and the supplied expanded view name the same edge; uniqueness of operation data therefore identifies their sources and argument lists. The proof normalizes the edge equality explicitly, so the dependent copy types are not silently identified. □

Lean: `Linear.ExpansionSearch.operation_readback`

Definition 38.11 (Search origins)**.** A family has search origins if every operation or variable edge in a replacement either belongs to a retained corolla copy or is owned by an original hole edge. This supplements forward search preservation with the origin information needed for reflection.

Lean: `Linear.ExpansionSearch.SearchOrigins`

Every recursive edge certificate has this origin property. A retained certificate supplies a copy; a filled certificate supplies the original hole label. In the boxed case, the single replacement edge still has a box label, so it cannot be an operation or variable. The same argument on the right side proves the origin property for both families of every recursive paired filling.

Lean: `Linear.DeepFillingEdge.search_origin`

Lean: `Linear.deepFilling_searchOrigins`

Theorem 38.12 (Reflection or a hole boundary)**.** *Assume search preservation and search origins. If $j_H p \longrightarrow_E q$, then either there is a context step $p \longrightarrow_C r$ with $q = j_H r$, or p is at a hole boundary.*

Proof. Inspect the interacting variable or operation edge. If its owner is a hole, source-incidence reflection in the search cases, or target-incidence reflection in the return case, gives the required boundary witness with its mode. Otherwise the owner is retained. For a variable trigger, the original variable has a singleton source, and the reflected source membership identifies it with the original coordinate. For operation entry, read back the operation view and use injectivity of the host map to recover the original source. For return, read back the argument list; injectivity of the mapped optional lookup recovers the same argument index in the original list. These cases produce an original search step. Forward transport and determinism identify its image with the supplied successor q. □

Lean: `Linear.ExpansionSearch.reflect_or_boundary`

As an immediate corollary, away from hole boundaries, search on expanded graphs is exactly the transported context relation. The forward implication rules out the boundary alternative; the reverse implication is forward search transport.

Lean: `Linear.ExpansionSearch.step_iff`

38.5 Reflection of entire executions

Let R be a predicate on context coordinates such that every coordinate in R avoids hole boundaries and every context step from R stays in R.

Theorem 38.13 (Execution reflection in an invariant region)**.** *Every n-step expanded execution starting at $j_H p$ with $R(p)$ is the image of an n-step context execution ending at a coordinate still in R.*

Proof. Induct on n. For zero steps use equality of the endpoints. Otherwise reflect the first step using the boundary exclusion, obtain its original successor, and use closure of R to apply the induction hypothesis to the tail. The result retains the exact length and equality of the expanded endpoint with the transported context endpoint. □

Lean: `Linear.ExpansionSearch.steps_reflect`

Theorem 38.14 (Exact successful costs when no hole is shallow)**.** *If C has no outer hole-labelled edge, then an n-step successful expanded execution from $j_H p$ exists if and only if an n-step successful context execution from p exists. Holes inside boxes are permitted.*

Proof. Use the entire coordinate space for R: a boundary witness would exhibit a forbidden shallow hole, and closure is automatic. Reflect the run and then reflect finality. Conversely transport the given context run and its finality. This is an equivalence for search-only executions; it says nothing about compute or substitution transitions that might subsequently open or copy boxes. □

Lean: `Linear.ExpansionSearch.final_runs_iff`

38.6 Paired actual executions and rooted endpoints

Fix a focus-free context $C : [\mathsf{term}] \to []$, recursive paired edge certificates for H, K, and hole-free expanded results. For a typed context coordinate p, define its left filled state by realizing $j_H p$ on $C[H]$. Recursive filling supplies global focus freedom, and the stated hole-freedom certificate supplies closedness. The right construction uses K in the same way. Uniqueness of hole names and allowed-context membership are not needed for these search theorems, so they are not required by this construction.

Lean: `Linear.ExpansionSearch.filledState`

Theorem 38.15 (Paired exact traversal)**.** *An n-step context execution between typed coordinates yields n-step executions between the corresponding actual state classes on both sides. If the context execution starts at the initial coordinate, both endpoint states are rooted.*

Proof. The recursive certificates supply search preservation separately for H and K. Transport the same context execution through each family, then apply execution realization. For rootedness apply the initial-run characterization on each expanded graph; the host map retains the initial input. This does not assert rootedness preservation for arbitrary UAM rewrite transitions. □

Lean: `Linear.ExpansionSearch.paired_steps`

Lean: `Linear.ExpansionSearch.paired_rooted`

Two executions of the same length from the same recursive state class have the same endpoint: induction on the length uses search determinism to equate the first successors, then compares the tails. Therefore an independently supplied actual-state execution, of the length of a given context trace and starting at its left filled state, must finish at the constructed left endpoint. The result is about equality of state classes, not equality of chosen carriers.

Lean: `Linear.ExpansionSearch.state_run_unique`

Lean: `Linear.ExpansionSearch.actual_run_endpoint`

38.7 A deep example and the necessary boundary restriction

Reuse the three-edge graph of Section 37: a nullary operation produces a thunk, a box contains the only hole, and a final operation consumes the box's term output. This time start from its focus-free outer graph. The filling retains both operations and replaces the box by the box containing the previously constructed filled body. The edge certificates are retained at the two operations and recursive at the box. Hole freedom follows from the closed filled body and the absence of holes in the operation corollas.

Lean: `Linear.ExpansionSearchExamples.fillings`

Lean: `Linear.ExpansionSearchExamples.edges`

Lean: `Linear.ExpansionSearchExamples.closed`

The maker's operation view has no term arguments and one thunk target. Let the returned coordinate be backtrack at the input. With passive operation kinds, the nullary rule gives a

one-step execution from the initial coordinate to this returned coordinate; the thunk is not searched.

Lean: `Linear.ExpansionSearchExamples.maker`

Lean: `Linear.ExpansionSearchExamples.returned`

Lean: `Linear.ExpansionSearchExamples.context_run`

The paired-rootedness theorem gives a rooted actual state with the deep-filled box still present. Its mode and input position prove finality directly. The successful-cost uniqueness theorem shows that every successful coordinate run from the expanded initial position has exactly one step and this endpoint.

Lean: `Linear.ExpansionSearchExamples.deep_return_rooted`

Lean: `Linear.ExpansionSearchExamples.deep_return_final`

Lean: `Linear.ExpansionSearchExamples.deep_return_cost`

For the negative example, take the one-hole corolla used as that box body. Before filling, no search constructor applies: the sole label is neither a variable nor an operation. After filling with the variable corolla, the inserted variable triggers rewrite mode at the same host input. There is a hole-boundary witness there, namely the original hole and its input source. There cannot be a context step whose transport is this trigger, because the context has no search step at all. Thus the boundary exception in reflection is essential, even for a single linear hole. The example concerns triggering substitution; it does not implement the substitution transition itself.

Lean: `Linear.ExpansionSearchExamples.hole_no_search`

Lean: `Linear.ExpansionSearchExamples.filled_hole_triggers`

Lean: `Linear.ExpansionSearchExamples.entering_hole`

Lean: `Linear.ExpansionSearchExamples.reflection_needs_boundary`

39 Erasure commutation and traversal of actual deep specimens

This section connects the coordinate traversal results to the actual states of recursive deep specimens. The main geometric construction commutes general focus erasure with simultaneous expansion. Its coordinate equation then proves that expanding an already focussed context agrees, up to a focus-preserving isomorphism, with filling the erased context and inserting a focus afterwards. The resulting traversal theorems start at the original specimen states.

For this representation we also prove the non-exiting conclusion of Lemma 39(2), and synchronized search under the settled-mode hypotheses of Lemma 39(3). The proof uses the paper's adjacency interpretation of entering and exiting focuses and its output-closure obstruction [1]. A returned coordinate determines actual rooted endpoints and is remote. Packaging it as a new allowed specimen, and extending the argument from the checked search relation to the full UAM, remain separate obligations. No arbitrary rewrite relation is treated as search in these results.

39.1 Erasing a copied focus

Fix a focussed context $C : I \to []$, focus f, and a filling family H. Assume that the focus is replaced by a literal corolla copy and that all other fillings are globally focus-free. Let t be an internal vertex with $f.\mathsf{target} = t$. These data give an expanded focus on $C[H]$ whose target is the host copy of t.

The abbreviations `erased`, `focus`, and `erasedExpansion` denote, respectively, erasure of f from C, the retained focus on $C[H]$, and erasure of that retained focus using the explicit host copy of t. The restricted family $\overline{H}$ assigns $H(e)$ to each non-focus edge e of the erased context. Erasure changes incidences, but not the labels of these edges, so the replacement interfaces are unchanged.

Lean: `Linear.ExpansionErasure.erased`

Lean: `Linear.ExpansionErasure.focus`

Lean: `Linear.ExpansionErasure.erasedExpansion`

Lean: `Linear.ExpansionErasure.family`

Definition 39.1 (Erasure–expansion carrier bijections)**.** The vertex bijection from erasure after expansion to expansion after erasure sends a retained host vertex to its retained host copy and an inserted internal vertex to the same vertex in its non-focus owner's filling. The edge bijection sends a surviving expanded edge to that same edge paired with its non-focus owner.

Verification of the bijections. The copied focus has no internal vertices. Thus the only potential obstruction to mapping an inserted internal vertex to a non-focus owner is impossible. The inverse vertex map reinstates the original owner and the proof that the host target was not selected. Both composites reduce to the original element; proof fields agree by proof irrelevance.

For edges, a surviving edge cannot belong to the focus fibre: the copy has exactly one edge, which is precisely the deleted focus. Hence its owner differs from the focus edge. Conversely every edge of a non-focus filling survives. The maps retain its owner and local edge, so both inverse equations are immediate. These arguments apply to arbitrary corolla names, not just the literal corolla constructor's chosen singleton name. □

Lean: `Linear.ExpansionErasure.vertices`

Lean: `Linear.ExpansionErasure.edges`

39.2 Commutation of incidences and the join coordinate

For a host vertex different from t, retaining it during erasure commutes with the vertex bijection. This is proved by the internal, input, and output vertex cases: the resulting ports agree directly.

Lean: `Linear.ExpansionErasure.keep_host`

Lemma 39.2 (Contraction at a host vertex)**.** *Contracting a host vertex in $C[H]$ and applying the carrier bijection gives the host copy of its contraction in C.*

Proof. If the original vertex is t, both contractions identify it with the focus source. The retained-vertex calculation applies to that source, which differs from t because a focus is not a self-loop. Otherwise injectivity of the host map proves that its image is not the expanded target; both contractions retain the vertex and the same retained-vertex calculation applies. □

Lean: `Linear.ExpansionErasure.contract_host`

The source and target port at index i of an erased non-focus edge are the contractions of its original port at index i. Indeed erasure maps contraction over the ordered incidence lists, and indexed lookup commutes with this map.

Lean: `Linear.ExpansionErasure.sourcePort`

Lean: `Linear.ExpansionErasure.targetPort`

Lemma 39.3 (Contraction at an inserted vertex)**.** *For every vertex of a non-focus filling, contraction after its insertion, followed by the carrier bijection, equals insertion into the erased context.*

Proof. An inserted internal vertex is disjoint from the deleted host target and keeps its original non-focus owner. A filling input is glued to an original source port; apply contraction at host vertices and the source-port calculation. A filling output is treated by the target-port calculation. These three cases exhaust the filling's vertex type, including fillings with internal structure. □

Lean: `Linear.ExpansionErasure.contract_insert`

Theorem 39.4 (Erasure commutes with expansion)**.** *There is a literal-label graph isomorphism*

$$|C[H]| \;\cong\; |C|[\overline{H}],$$

where the left erasure removes the retained copied focus and the right erasure removes the original focus. Both use the specified internal target.

Proof. Use the carrier bijections above. Host and inserted vertex labels are retained; the impossible focus-fibre internal case is excluded as before. Edge labels are identical. For each source list, compose its insertion map, contraction, and the vertex bijection. The inserted-vertex lemma identifies this composition pointwise with insertion in the erased context. Equality of mapped lists proves the source equation. The target equation is the same argument on the ordered target list. □

Lean: `Linear.ExpansionErasure.iso`

The isomorphism also maps the joined focus coordinate to the host copy of the original joined coordinate. A join is the retained source of the deleted focus, so this is exactly the retained-host-vertex calculation.

Lean: `Linear.ExpansionErasure.position`

The canonical underlying graph chooses an internal target rather than taking one as an argument. Its chosen target is necessarily the host copy of the original chosen target, since both are equal to the expanded focus target. Eliminating this equality transfers the isomorphism and its coordinate equation to the canonical underlying graphs. The theorem returns the isomorphism and the position equation together, ensuring that the same map establishes both.

Lean: `Linear.ExpansionErasure.underlying_exists`

39.3 Recovering expansion of an already focussed context

Definition 39.5 (Focussed-context coordinate)**.** For a focussed context f, its coordinate in the erased graph consists of its original mode and the joined term vertex. The term-typing proof comes from the focus's typed singleton source. Unlike the earlier state-coordinate operation, this definition does not require the context to be hole-free.

Lean: `Linear.Focussed.coordinate`

The abbreviation `erasedFamily` uses the canonical chosen target to restrict the original family. The expanded erased graph is focus-free, since it is isomorphic to the underlying graph of the retained expanded focus. If the original expansion is hole-free, so is this expanded erased graph: erasure preserves hole freedom and the isomorphism transports it. These proofs provide the certificates required to realize coordinates as actual states.

Lean: `Linear.ExpansionErasure.erasedFamily`

Lean: `Linear.ExpansionErasure.expanded_free`

Lean: `Linear.ExpansionErasure.expanded_closed`

Theorem 39.6 (The focussed expansion bridge)**.** *Fill the erased context, transport its original coordinate through that filling, and realize the coordinate by focus insertion. The resulting state is focus-preservingly isomorphic to expansion of the original focussed context.*

Proof. Choose the erasure-commutation isomorphism together with its coordinate equation. The extracted coordinate of the expanded state maps to exactly the transported original coordinate; the modes agree as well. Realization respects renaming, so their inserted states are focus-preservingly isomorphic. Compose the inverse of this realization isomorphism with the general recovery isomorphism from insertion after erasure to the original expanded state. This composition identifies the distinguished edge and its mode, not merely the two underlying graphs. □

Lean: `Linear.ExpansionErasure.realization_iso`

39.4 Actual-state search and exact executions

Removing the focus restricts every recursive edge certificate to the surviving edges. A retained certificate keeps the same copy data; a filled certificate keeps its hole label and related pair; a boxed certificate keeps its recursive inner filling. None of these certificates depends on the host's changed source or target lists, except through unchanged edge interfaces.

Lean: `Linear.DeepFillingEdge.erased`

For any typed coordinate q of the erased context, `stateAt` realizes its transported coordinate using the focus-freedom and hole-freedom certificates just proved. At the original coordinate, the focussed expansion bridge applies.

Lean: `Linear.ExpansionErasure.stateAt`

Theorem 39.7 (Search successors of the original expanded state)**.** *Suppose the restricted family preserves search and has search origins. If the original coordinate avoids hole boundaries, an actual-state search successor is precisely a context-coordinate successor realized in the filling, followed by a focus-preserving endpoint isomorphism.*

Proof. Use source-isomorphism invariance of state search to replace the original expanded source by its bridged realization. The coordinate characterization of search from a realized state produces a step in the expanded erased graph. Reflect it through the filling, using the boundary exclusion. Search preserves term typing, so the reflected coordinate can itself be realized. The coordinate equality identifies this realization with the supplied target up to the supplied isomorphism. Conversely, transport the context step, realize it, and apply the source bridge and endpoint isomorphism. □

Lean: `Linear.ExpansionErasure.search_iff`

An n-step context execution from the original coordinate gives an n-step execution from the original expanded state class to `stateAt` of its endpoint. Transport and realize the run, then replace its initial class using the bridge. Similarly, a context run from the initial input to the original coordinate proves rootedness of the original expanded state: the earlier rootedness theorem applies to its realization, and the bridge equates its class with the original state's class.

Lean: `Linear.ExpansionErasure.steps`

Lean: `Linear.ExpansionErasure.rooted`

39.5 Entering, exiting, and remote coordinates

Definition 39.8 (Concrete entering and exiting focuses)**.** A focus is entering when its target is a source of an original shallow hole-labelled edge. It is exiting when its source is a target of such an edge. The predicates do not themselves restrict the mode, and are defined on the original focussed context rather than on its filling.

Lean: `Linear.Focussed.Entering`

Lean: `Linear.Focussed.Exiting`

Lemma 39.9 (Adjacency after erasure)**.** *The joined coordinate is a source of an erased non-focus edge exactly when the original focus target is a source of that edge. It is a target of the erased edge exactly when the original focus source is a target of that edge.*

Proof. For sources, unpack membership in the contraction-mapped source list. A source of a non-focus edge cannot equal the focus source, by source-incidence uniqueness. The inverse expansion of contraction therefore recovers that source; expanding the joined coordinate gives the original focus target. Conversely, contraction maps the target to the joined coordinate.

For targets, the corresponding target of a non-focus edge avoids the deleted focus target, by target-incidence uniqueness. Lifting its contraction recovers it, while lifting the joined coordinate gives the focus source. Conversely the focus source contracts to the joined coordinate. These arguments retain the incidence direction; they do not identify entering and exiting with each other. □

Lean: `Linear.Focussed.erased_source_iff`

Lean: `Linear.Focussed.erased_target_iff`

Consequently the erased coordinate is at a hole boundary if and only if the original focus is searching and entering, or backtracking and exiting. A hole edge cannot be the focus edge, since their primitive labels differ; thus every original hole witness supplies a surviving erased edge. The two adjacency lemmas then prove both directions of the equivalence.

Lean: `Linear.Focussed.boundary_iff`

Definition 39.10 (Remote coordinate)**.** A coordinate is remote if its mode is not rewrite, or its vertex is not a source of any hole-labelled edge. This is the coordinate counterpart of excluding only an entering rewrite focus.

Lean: `Linear.RemotePosition`

Lemma 39.11 (Search has remote targets)**.** *Every coordinate search successor is remote.*

Proof. Search can produce rewrite mode only at a variable source or an operation source. Such a source cannot also belong to a hole: source uniqueness would identify the variable or operation edge with that hole edge, contradicting their labels. An entry with arguments or a return with a next argument instead produces search mode and is immediately remote. Nullary entry and final-argument return use the operation-source argument, regardless of the operation's finish mode. □

Lean: `Linear.SearchStep.remote_target`

39.6 Specialization to recursive deep specimens

Let S be a recursive deep specimen. Its edge certificates restrict to the erased context by the erasure theorem. Define $S.\mathsf{leftAt}(q)$ and $S.\mathsf{rightAt}(q)$ by the corresponding `stateAt` constructions. At the original coordinate these are focus-preservingly isomorphic to the existing $S.\mathsf{leftState}$ and $S.\mathsf{rightState}$; the original specimen representation has not been replaced by an assumed locality law.

Lean: `Linear.DeepSpecimen.erased_edges`

Lean: `Linear.DeepSpecimen.leftAt`

Lean: `Linear.DeepSpecimen.rightAt`

Lean: `Linear.DeepSpecimen.at_original`

The recursive certificates give search preservation and search origins on both restricted families. Applying the actual-state search characterization yields left and right successor equivalences whenever the original coordinate avoids hole boundaries.

Lean: `Linear.DeepSpecimen.left_search_iff`

Lean: `Linear.DeepSpecimen.right_search_iff`

The equivalences also hold for recursive state search, with recursive focus-preserving endpoint isomorphisms. In the forward direction, factor a recursive search step into a literal search step and a recursive endpoint renaming. Extract the context coordinate from the literal step and compose the isomorphisms. In the reverse direction, take the canonical literal realization as the intermediate state and apply the supplied recursive renaming.

Lean: `Linear.DeepSpecimen.left_deepSearch_iff`

Lean: `Linear.DeepSpecimen.right_deepSearch_iff`

In particular, a left literal search step determines a context coordinate, a literal isomorphism from its left realization to the supplied target, and a matching right search step to its right realization. The right step uses the same context-coordinate witness and the identity endpoint isomorphism.

Lean: `Linear.DeepSpecimen.match_left_step`

Exact context runs give paired executions starting from the original specimen states, by applying the general execution bridge separately on both sides. A context run from the initial input to the original coordinate similarly proves rootedness of both original states, by the general rootedness bridge.

Lean: `Linear.DeepSpecimen.context_steps`

Lean: `Linear.DeepSpecimen.rooted_of_context_run`

39.7 Output closure and settled traversal

Theorem 39.12 (Non-exiting rooted specimens: concrete Lemma 39(2))**.** *If the outer pre-template is output-closed and both states of S are rooted, then its original focus is not exiting.*

Proof. An exiting witness is a hole edge with the focus source among its targets. The singleton path consisting of this edge ends at the focus source and contains a non-operation edge, because a hole is not an operation. The proved recursive-specimen form of Lemma 26(3) says that one of the actual states is not rooted, contradicting joint rootedness. The argument concerns the current recursive representation; its finite tuple compatibility is now proved in Section 43. The separate comparison with set-based hypernets is now proved in Section 45. □

Lean: `Linear.DeepSpecimen.not_exiting`

If the focus is backtracking, or is searching without entering, then joint rootedness and output closure exclude the coordinate hole boundary. Use the boundary equivalence: its entering/search alternative contradicts the mode or the non-entering premise; its exiting/backtrack alternative contradicts the non-exiting theorem. The one-sided matching theorem therefore applies under these settled hypotheses.

Lean: `Linear.DeepSpecimen.settled_away`

Lean: `Linear.DeepSpecimen.settled_match_left`

Theorem 39.13 (Synchronized settled traversal)**.** *Assume output closure, joint rootedness, and a backtracking or non-entering search focus. If recursive search is enabled on either actual specimen side, there is a typed successor coordinate q such that both actual sides search to their respective realizations at q. Both endpoints are rooted and q is remote.*

Proof. The settled hypotheses give boundary exclusion. Extract the context step from whichever side is enabled, using its recursive-search characterization. Apply both successor characterizations to that same step, choosing identity endpoint isomorphisms. The search-target lemma proves remoteness of q. Concrete search preserves rootedness, so each of the two constructed endpoints is rooted. □

Lean: `Linear.DeepSpecimen.settled_traversal`

This proves synchronized search and the required endpoint facts. The result here supplies a coordinate rather than a new specimen. Section 47 now completes that packaging, preserving unique holes and allowed membership, and proves the full-UAM context-step law.

39.8 Traversal and rootedness regressions

Return to the original deep specimen with its focus immediately before the consumer and its sole hole inside a box. It has no shallow hole-labelled edge, so neither entering nor exiting is possible. The boundary equivalence therefore excludes the erased-coordinate hole boundary.

Lean: `Linear.SpecimenTraversalExamples.no_boundaries`

Lean: `Linear.SpecimenTraversalExamples.away`

The consumer is an actual operation of the erased context. Its singleton source is the joined coordinate: in the focussed context its source is the focus target, and erasure contracts that target to the join. Its argument and thunk lists are empty. Define the returned coordinate by changing only the mode to backtrack. With passive operation kinds, nullary entry supplies the corresponding context step.

Lean: `Linear.SpecimenTraversalExamples.consumer`

Lean: `Linear.SpecimenTraversalExamples.returned`

Lean: `Linear.SpecimenTraversalExamples.context_step`

The left actual-state search characterization realizes this as a step from the original expanded specimen state. If any recursively renamed left successor is supplied instead, extract its context coordinate. Coordinate determinism equates it with the returned coordinate, giving its recursive endpoint isomorphism and the matching right transition.

Lean: `Linear.SpecimenTraversalExamples.actual_step`

Lean: `Linear.SpecimenTraversalExamples.matching`

Finally no context run from the initial input can reach this specimen's original coordinate. Such a run would make both original states rooted by the new bridge, contradicting the earlier deep-box obstruction. Thus the new reconstruction and traversal results preserve the previous distinction between well-formed states and rooted states; a valid local search step does not imply rootedness.

Lean: `Linear.SpecimenTraversalExamples.no_initial_context_run`

40 Independent local incidence semantics for search

This section proves that erasure-based state search coincides with a presentation that inspects adjacency in the original graph and specifies the target by explicit incidence equations. Neither the local interaction predicate nor the rewiring structure mentions erasure or coordinate search. The result applies to arbitrary states, including states with cycles, and preserves ordered arguments and complete non-focus labels.

The incidence correspondence alone does not establish contextual-rewrite adequacy. Section 41 now completes the additional construction of finite interaction patterns and proves their factorization and plugging correspondence through explicit focus-free simple hole contexts.

40.1 Interactions in the original graph

Definition 40.1 (Local interaction)**.** Let f be a focus of a graph C with no output boundary. A local interaction selects a pair (m, w) consisting of the next mode and an original graph port. The variable rule requires search mode, a variable-labelled edge, and that edge's source list to be exactly $[f.\mathsf{target}]$; it selects rewrite mode at $f.\mathsf{source}$. Entry requires search mode and an operation whose source equals $f.\mathsf{target}$, and uses its ordered entry selection. Return requires backtracking mode and an argument index i whose lookup is $f.\mathsf{source}$, and uses the operation's ordered return selection. Entry and return include both the next-argument and finished-operation cases.

The selected port is not required to differ from the old focus target. For example a nullary operation immediately adjacent to the focus selects its own source, which is that target. The joining operation below handles this case.

Lean: `Linear.LocalInteraction`

Lemma 40.2 (Erasure and operation views)**.** *An operation edge differs from the focus edge. Its erasure retains its label, contracts its source and all argument and thunk ports, and is again an operation view. Erasure commutes with entry and return selection. Conversely every erased operation has an original operation view with the same edge and operation label, and with precisely those contracted source and argument data.*

Proof. Operation and focus labels are disjoint at every depth, proving the edge inequality. Mapping contraction over the singleton source and concatenated target lists proves the erased view's incidence equations; list length is unchanged. For entry, split on whether the argument list is empty. For return, commute lookup with list mapping and split on the next-argument lookup. Finally, recover an original operation view from the surviving edge's label. Uniqueness of the source and ordered argument data for a given edge identifies its erasure with the supplied view. □

Lean: `Linear.SearchOperation.ne_focus`

Lean: `Linear.SearchOperation.erase`

Lean: `Linear.SearchOperation.erase_enter`

Lean: `Linear.SearchOperation.erase_resume`

Lean: `Linear.SearchOperation.unerase`

Theorem 40.3 (Adjacency correspondence)**.** *A coordinate step from the erased focus coordinate to q exists exactly when an original-graph local interaction selects (m, w) and $q = (m, \mathsf{contract}(w))$.*

Proof. For the forward construction from local interactions, erase the interacting edge. A variable or operation source adjacent to the focus target contracts to the erased focus position. A returning argument equal to the focus source contracts to that same position. Apply the corresponding coordinate rule and the operation-selection commutation equations.

For the converse, analyze the coordinate rule. In the variable case, erased-source membership reflects to adjacency at the original focus target; the variable signature makes the original source list a singleton. In the entry case, recover the original operation view and use the same source-membership reflection. In the return case, invert lookup in the mapped argument list to recover the original argument at the same index. A non-focus target incidence cannot equal the deleted focus target. Thus lifting its contraction recovers the original argument, and the coordinate equality forces this argument to be the focus source. Uniqueness of operation selections gives the required target coordinate in both operation cases. □

Lean: `Linear.LocalInteraction.sound`

Lean: `Linear.LocalInteraction.complete`

Lean: `Linear.LocalInteraction.iff_search`

40.2 Independent equations for the target graph

Definition 40.4 (Joining and moving ports)**.** Write a and b for the old focus source and target. Define functions on the original graph's ports by

$$J(v) = \begin{cases} a & v = b, \\ v & v \neq b, \end{cases} \qquad M_w(v) = \begin{cases} b & J(v) = J(w), \\ J(v) & J(v) \neq J(w). \end{cases}$$

The first joins the old focus endpoints. The second redirects source incidences at the selected joined port to the reusable focus target. These equations use only original ports and equality, without constructing an erased graph.

Lean: `Linear.Focussed.joinPort`

Lean: `Linear.Focussed.movePort`

Joining a port equals contracting it and then lifting the retained port back to original names. If it is the focus target, both sides yield the focus source; otherwise contraction followed by lifting fixes it. Moreover $M_w(v) = v$ when $v \neq b$ and $v \neq J(w)$: both tests in the displayed definitions are false.

Lean: `Linear.Focussed.joinPort_eq_lift_contract`

Lean: `Linear.Focussed.movePort_unchanged`

Definition 40.5 (Local rewiring)**.** For states s, t and a selected pair (m, w), a local rewiring comprises bijections V and E of their internal vertex and edge carriers. Extend V to ports by fixing the ordered boundary. The bijections preserve internal vertex labels and identify the distinguished focus edges. The target focus mode is m, and every non-focus edge retains its entire label, including box bodies. The target incidence lists obey

$$\begin{aligned} t.\mathsf{source}(E(f)) &= [V(J(w))], \\ t.\mathsf{source}(E(e)) &= \mathsf{map}(V \circ M_w, s.\mathsf{source}(e)) && (e \neq f), \\ t.\mathsf{target}(E(e)) &= \mathsf{map}(V, s.\mathsf{target}(e)) && \text{for every } e. \end{aligned}$$

Here f is the source state's focus edge. The focus label follows from the target state's focus certificate and its mode equation. Well-formedness of both states is part of their types; these equations do not postulate that an arbitrary raw graph patch is well formed.

Lean: `Linear.LocalRewiring`

The private port-composition lemma says that extending a composite vertex bijection to ports equals composing its two extensions. Check the internal, input, and output constructors separately; each equality is immediate.

Lean: `Linear.LocalRewiring.port_trans`

40.3 Canonical construction and uniqueness

Fix a typed erased coordinate $q = (m, \mathsf{contract}(w))$. Reinsertion at q introduces a singleton vertex and edge. The restoration bijections send those fresh elements to the old focus target and old focus edge, and retain all other names.

Lemma 40.6 (Restoration implements port movement)**.** *For every original port v, restoring the names of the split contracted port at q yields $M_w(v)$.*

Proof. Injectivity of lifting shows that the contracted ports of v and w agree exactly when $J(v) = J(w)$. In that case insertion splits to the fresh vertex, which restoration sends to the old focus target. Otherwise insertion retains the contracted port; restoring that host port yields its lift, namely $J(v)$. These are precisely the two defining cases of M_w. □

Lean: `Linear.LocalRewiring.restore_move`

Theorem 40.7 (Characterization of a rewired target)**.** *Every target satisfying the local equations is focus-preservingly isomorphic to the canonical state realized from q. Conversely the canonical state satisfies the local equations. Replacing a target by a focus-preservingly isomorphic state preserves local rewiring.*

Proof. For the first direction compose restoration with the given carrier bijections. On a retained vertex its label equation is the supplied one; on the fresh vertex use that the old focus target has term type. Retained edge labels use the non-focus label equation; the fresh edge uses the target focus's label and the selected mode. For retained sources apply the restoration lemma pointwise. For the focus source use joining equals lifted contraction. For retained targets, contraction followed by lifting is the identity since no non-focus edge targets the deleted focus target. The fresh target restores to that target. These equations provide every field of the focussed isomorphism.

For the converse use the inverse restoration bijections. The existing reinsertion isomorphism supplies internal-label and target equations, which are independent of the new coordinate and mode. Non-focus labels are unchanged by construction. Apply injectivity of the forward restoration port map to the source equations: the retained-source equation becomes the restoration lemma, and the focus-source equation becomes joining equals lifted contraction. The focus edge and mode equations follow from the definitions.

Finally compose carrier bijections with a target isomorphism. Its incidence preservation, list-map composition, and the port-composition lemma prove all new source and target equations. Labels, mode, and focus identity compose in the same order. □

Lean: `Linear.LocalRewiring.iso`

Lean: `Linear.LocalRewiring.canonical`

Lean: `Linear.LocalRewiring.transport`

40.4 State-level equivalence and regression examples

Definition 40.8 (Incidence search)**.** An incidence-search step $s \to t$ consists of a selected pair (m, w), an original-graph local interaction selecting it, and a local rewiring from s to t at that pair. This definition contains no coordinate-search premise.

Lean: `Linear.State.IncidenceSearch`

Theorem 40.9 (Exact correspondence on actual states)**.** *Erasure-based state search is equivalent to incidence search. Recursive state search is equivalent to an incidence step followed by recursive focussed isomorphism. Incidence search is deterministic up to literal-label focussed isomorphism.*

Proof. From erasure-based search, recover an original local interaction, construct the canonical rewiring, and transport it through the supplied target isomorphism. Conversely a local interaction gives a coordinate step. Type preservation of coordinate search certifies its target as a typed position; the rewiring characterization supplies the required target isomorphism. The recursive statement follows by substituting this equivalence into the existing characterization of recursive search as literal search plus recursive endpoint renaming. Determinism follows by converting two incidence steps to state search and applying state-search determinism. □

Lean: `Linear.State.search_iff_incidence`

Lean: `Linear.State.deepSearch_iff_incidence`

Lean: `Linear.State.incidenceSearch_deterministic`

The examples recover the actual nullary value edge in the initial value state. Its source is the focus target and its argument and thunk lists are empty; the source incidence follows by splitting the unique input port. Direct entry therefore selects backtracking at the old target. This exercises the joining case $J(b) = a$, not merely an unaffected port. The known terminating value step converts to an incidence step whose target is final.

Lean: `Linear.LocalSearchExamples.valueOperation`

Lean: `Linear.LocalSearchExamples.value_interaction`

Lean: Linear.LocalSearchExamples.value_terminates

A second example inserts an initial search focus before the variable in the closed variable/value chain. Its variable source list is checked directly to be the singleton focus target, so the local variable rule selects rewrite mode at the old source. Type preservation and canonical rewiring construct an actual incidence successor in rewrite mode. This proves only the trigger, not a substitution transition.

Lean: Linear.LocalSearchExamples.initialChain

Lean: Linear.LocalSearchExamples.variable_interaction

Lean: Linear.LocalSearchExamples.variable_successor

41 Finite interaction patterns and simple-context search

This section discharges the search adequacy obligation formerly listed in Appendix B. The principal theorem identifies the checked erasure-based search relation with actual plugging of finite interaction fragments into one focus-free simple hole context, in both directions. It also identifies the independent incidence presentation with that same relation. The result concerns search: it does not add substitution or compute transitions.

We use the five coordinate clauses already specified for Fig. 9 to index pairs of actual two-edge graphs. Each graph contains one variable or operation edge and one focus edge. A single shallow slot supplies the surrounding graph; opening it creates an explicitly labelled hole. General slot-label transport and a closing theorem prove that arbitrary simple contexts are covered, rather than restricting context application to a chosen family of examples.

41.1 Subdividing at an output port

Definition 41.1 (Incoming-side focus subdivision)**.** For a graph $G : I \to O$ and a term port v that is not an input, retain the old carriers and add one internal term vertex z and one focus edge f. Original source lists are mapped through the old-vertex inclusion. Original target lists are mapped through the split function that replaces v by z and includes all other ports. Set the focus source to z and its target to the included copy of v. Thus a focus can be placed at an output boundary, which the earlier outgoing-side subdivision cannot do.

Lean: Linear.OutputFocusInsertion.target

Lean: Linear.OutputFocusInsertion.source

Lemma 41.2 (Incidence uniqueness and coverage)**.** *The incoming subdivision satisfies source and target uniqueness and the two coverage laws of a finite graph.*

Proof. For two original target lists, injectivity of splitting reflects shared membership to the original graph's target uniqueness. An original target cannot equal the focus target: a split port never equals the unsplit inclusion of the selected port. Two focus-edge names coincide because their carrier is a singleton. For sources, old-vertex inclusion is injective and its image is disjoint from the fresh focus source. Original source uniqueness and the singleton focus carrier then give source uniqueness.

For target coverage, a retained port unequal to v has its original target edge, a retained copy of v is targeted by the focus, and the fresh vertex is targeted by the original incoming edge of v.

That edge exists because v is not an input. Conversely each target is not an input, using the original coverage law, the hypothesis on v, or the internal status of z. Source coverage retains every old source incidence and supplies the fresh vertex with the focus edge; no input-side exception is needed for this direction. □

Lean: `Linear.OutputFocusInsertion.target_unique`

Lean: `Linear.OutputFocusInsertion.source_unique`

Lean: `Linear.OutputFocusInsertion.target_coverage`

Lean: `Linear.OutputFocusInsertion.source_coverage`

The constructor packages these incidence functions and laws as a typed finite hypernet. Finite enumerations are disjoint sums with a singleton. Nodup follows from injectivity of the host or split map. The existing host-label and split-label lemmas prove typing of original incidences; the new focus has a singleton term source and target. Given global focus freedom of G, the focus certificate selects the fresh edge, retains freedom of every old label, and proves distinct endpoints by freshness. Its source is internal, so it cannot be exposed from an input directly to an output.

Lean: `Linear.OutputFocusInsertion.insert`

Lean: `Linear.OutputFocusInsertion.focussed`

41.2 Finite fragments and their plugging equations

Definition 41.3 (Boundary focus fragment)**.** For a selected edge e of C, its corolla has the same label and ordered interface, no internal vertices, and one edge. The input fragment at a term input index inserts a focus before that corolla input. The output fragment at a term output index uses incoming subdivision after that corolla output. Both fragments have one internal vertex and two edges. The corresponding whole-graph subdivisions insert at the selected edge's source or target port.

Lean: `Linear.PatternPlugging.corolla`

Lean: `Linear.PatternPlugging.inputPattern`

Lean: `Linear.PatternPlugging.inputWhole`

Lean: `Linear.PatternPlugging.outputPattern`

Lean: `Linear.PatternPlugging.outputWhole`

The vertex bijection from a plugged fragment to the corresponding whole subdivision removes the empty corolla-vertex summand and retains the host and fresh vertices. The edge bijection retains every unselected host edge, restores the selected edge name from the corolla edge, and sends the fragment focus to the fresh whole-graph focus. Its inverse tests whether an old edge is the selected edge; the singleton carriers prove the inverse equations.

Lean: `Linear.PatternPlugging.vertices`

Lean: `Linear.PatternPlugging.edges`

Lemma 41.4 (Input-fragment incidence equations)**.** *Under those bijections, host ports retain their host inclusions; included corolla ports become inclusions of their attachment ports; and a split corolla input becomes the split attached source port.*

Proof. The host equation is a case analysis on internal, input, and output ports. For an included corolla port, the internal case is empty; the input and output cases reduce to that host equation at the corresponding attachment. For a split input, test equality of its index with the selected index. If equal, both sides are the fresh vertex. Otherwise injectivity of ordered source-port lookup proves that the attachment differs from the selected attachment, so both sides are the retained host port. □

Lean: `Linear.PatternPlugging.input_host`

Lean: `Linear.PatternPlugging.input_insert_host`

Lean: `Linear.PatternPlugging.input_insert_split`

Theorem 41.5 (Input-fragment factorization)**.** *Plugging the input fragment at e is isomorphic to subdividing the whole graph at that edge's selected source port.*

Proof. Use the carrier bijections above. Labels are unchanged. A source port of an unselected host edge cannot equal the selected source port: otherwise source uniqueness would identify its edge with e. Its source list therefore agrees with the whole-graph split list. For the selected edge, enumerate the corolla's input indices and apply the split-input equation pointwise; enumerating the host's source ports reconstructs its source list. The focus source is the included attachment. All old targets use the host equation, and enumerating corolla outputs reconstructs the selected target list. Both focus targets are fresh. These cases establish every isomorphism field. □

Lean: `Linear.PatternPlugging.input_iso`

The output-fragment equations are proved separately with targets in place of sources. Host and included-port equations again follow by port cases. For split outputs, equality of output indices selects the fresh vertex; inequality is reflected by injectivity of ordered target-port lookup.

Lean: `Linear.PatternPlugging.output_host`

Lean: `Linear.PatternPlugging.output_insert_host`

Lean: `Linear.PatternPlugging.output_insert_split`

Theorem 41.6 (Output-fragment factorization)**.** *Plugging the output fragment at e is isomorphic to incoming subdivision of the whole graph at its selected target port.*

Proof. Use the same carrier bijections. Target uniqueness ensures that an unselected host edge never targets the selected target port, so its target list is unchanged by splitting. Enumerate output indices and apply the split-output equation for the selected edge. Its focus target is the included attachment. Sources use host inclusion, with the selected source list reconstructed by its ordered input enumeration; the focus source is fresh. Labels and finite carriers are preserved exactly as for input fragments. □

Lean: `Linear.PatternPlugging.output_iso`

Neither factorization assumes that an input attachment differs from an output attachment. Each uses injectivity only within the relevant ordered side. This distinction is necessary for self-looping slots.

41.3 Reconciling the two subdivision orientations

Definition 41.7 (Subdivision vertex exchange)**.** At an internal port v, exchange its old copy with the fresh focus vertex and fix every other old vertex. This self-inverse function is a bijection on the extended internal carrier; singleton uniqueness handles the fresh carrier's only index.

Lean: `Linear.FocusSubdivision.swap`

Its port extension sends an included old port to its split version and the fresh port to the included copy of v. The first claim follows by cases on the port and equality with v; the second follows directly from the definition. Applying the exchange again shows that a split port returns to its inclusion.

Lean: `Linear.FocusSubdivision.swap_host`

Lean: `Linear.FocusSubdivision.swap_fresh`

Lean: `Linear.FocusSubdivision.swap_split`

Theorem 41.8 (Agreement of subdivision orientations)**.** *Incoming and outgoing focus subdivision at an internal term port are isomorphic, with the focus edge and its mode retained.*

Proof. Use the exchange on vertices and the identity on edges. The two exchanged vertices have term type; all other labels are fixed. On original sources, the exchange turns inclusion into splitting. On original targets it turns splitting into inclusion. Its fresh-port and selected-port equations identify the focus source and target lists. Edge labels are unchanged, including the focus mode. □

Lean: `Linear.FocusSubdivision.iso`

41.4 Factoring the search rules through one corolla

Definition 41.9 (Corolla attachment)**.** Attach each corolla input index to the corresponding source port of e, and each output index to the corresponding target port. There is no internal case. Typing of ordered port lookup proves preservation of vertex labels. Mapping attachment over either corolla incidence list reconstructs the corresponding host list by ordered port enumeration.

Lean: `Linear.PatternPlugging.attach`

Lean: `Linear.PatternPlugging.attach_label`

Lean: `Linear.PatternPlugging.attach_source`

Lean: `Linear.PatternPlugging.attach_target`

An operation view on the corolla becomes a view on e by attaching its source, argument list, and thunk list. The reconstructed incidence equations and preservation of list lengths prove

the view's fields. Attachment commutes with entry by inspecting whether the argument list is empty, and with return by commuting lookup with mapping and inspecting the next argument.

Lean: `Linear.PatternPlugging.operation`

Lean: `Linear.PatternPlugging.operation_enter`

Lean: `Linear.PatternPlugging.operation_resume`

Lemma 41.10 (Rule attachment and operation recovery)**.** *Every coordinate rule on the corolla attaches to a coordinate rule of C. Every operation view of C is recovered from a view on its edge's corolla, with equal source, ordered arguments, and operation label after attachment.*

Proof. For a variable rule, reconstruct the singleton source list by attachment and apply the host variable rule. For entry, use the attached operation view and entry commutation. For return, map the argument lookup and use return commutation. For operation recovery, the corolla has the original operation label and hence admits a view. Its attached view has the original edge name; view uniqueness identifies the required data. □

Lean: `Linear.PatternPlugging.sound`

Lean: `Linear.PatternPlugging.operation_preimage`

Theorem 41.11 (Finite-rule factorization)**.** *Every coordinate search step on C is the attachment of a search step on the corolla of one interacting edge, with both endpoint coordinates attached by the same map.*

Proof. Analyze the search rule. For a variable, the signature forces its corolla source list to have length one. Extract that source port; reconstruction of the host source list identifies its attachment with the original source. The corolla variable rule has the required two endpoints. For entry, recover the corolla operation view. Equality of source data and uniqueness of entry selection give the endpoint equations. For return, recover the view and invert lookup in its mapped argument list. A failed lookup contradicts the given host lookup. A successful lookup supplies the same argument index and a port attaching to the host argument. The corolla return rule and uniqueness of return selection give the target equation. □

Lean: `Linear.PatternPlugging.complete`

Inspecting any corolla rule shows that its label is either the variable label or an operation label: the variable constructor supplies the first, and the operation views in entry and return supply the second. Such a primitive label is focus-free and has no hole in its occurrence fibre. At either depth form, its shallow occurrence is not a hole and its child-occurrence type is empty. These facts are derived from the rule, not assumed of context application.

Lean: `Linear.PatternPlugging.interaction_label`

Lean: `Linear.PatternPlugging.interaction_closed`

41.5 Actual state endpoints

A literal graph isomorphism between globally uniquely focussed graphs must preserve their distinguished focuses. Otherwise the image of the source focus would be a non-focus edge of the target and therefore have a focus-free label, contradicting label preservation. Once

the edge is identified, injectivity of focus labels identifies its mode. This supplies a focussed isomorphism from an ordinary graph isomorphism.

Lean: `Linear.Focussed.isoOfGraph`

Definition 41.12 (Typed fragment coordinate and matching)**.** A fragment coordinate is a mode and term port of the selected corolla. Unlike a state coordinate it may lie at an output. Its fragment is the input or output graph defined above, according to the port constructor. Attaching it gives a typed state coordinate, by label preservation. A fragment matches an actual state when its plugging is literally graph-isomorphic to that state.

Lean: `Linear.PatternPlugging.Position`

Lean: `Linear.PatternPlugging.fragment`

Lean: `Linear.PatternPlugging.attached`

Lean: `Linear.PatternPlugging.Matches`

Every output attachment in a host of interface $\star \to []$ is internal: its target-list membership excludes inputs, and there are no output indices.

Lean: `Linear.PatternPlugging.output_internal`

Theorem 41.13 (Fragment realization)**.** *In a focus-free, hole-free state-interface host, plugging any typed fragment is isomorphic to realization of its attached coordinate. Consequently matching an actual state is equivalent to a focussed isomorphism from that realization.*

Proof. For input coordinates apply input-fragment factorization. For output coordinates apply output-fragment factorization, then exchange the old internal attachment and fresh vertex to obtain outgoing subdivision. The previous internality lemma justifies that exchange. The corolla has no internal coordinate case. Compose this graph isomorphism with a matching isomorphism and use uniqueness of the focus to obtain the focussed endpoint condition. Conversely discard the focus fields and compose graph isomorphisms in the opposite direction. □

Lean: `Linear.PatternPlugging.realization`

Lean: `Linear.PatternPlugging.matches_iff`

Definition 41.14 (Slot-context search)**.** Choose a focus-free, hole-free host, one selected edge, and two typed corolla coordinates related by a finite search rule. Require their two plugged fragments to match the source and target actual states. The same selected edge is removed by both pluggings.

Lean: `Linear.State.ContextualSearch`

Theorem 41.15 (Slot-context correspondence)**.** *Slot-context search and erasure-based state search coincide in both directions. They therefore also coincide with incidence search. Recursive state search adds precisely recursive renaming of a slot-context successor.*

Proof. For soundness, attach the finite corolla rule to the host. Realize the resulting coordinate step and transport its source and target through the matching focussed isomorphisms. For completeness, erase the source state's focus and factor its coordinate search through a corolla. The source coordinate is typed by attachment-label preservation; search type preservation types the target coordinate. The source endpoint matches by general focus recovery, and the target matches by the endpoint isomorphism in the state search witness. The incidence equivalence follows from the previous section's theorem. Substitution into the characterization of recursive state search proves the recursive endpoint statement. □

Lean: `Linear.State.contextualSearch_sound`

Lean: `Linear.State.contextualSearch_complete`

Lean: `Linear.State.search_iff_contextual`

Lean: `Linear.State.incidenceSearch_iff_contextual`

Lean: `Linear.State.deepSearch_iff_contextual`

41.6 Opening and closing an explicit simple context

Definition 41.16 (Signature-preserving slot relabelling)**.** Replace the selected edge label by a label with equal ordered input and output signatures, keeping all other data. The selected-label and other-label equations follow by the equality test defining relabelling. Identity carrier maps give an incidence isomorphism with the unrestricted label relation. The supplied signature equalities identify the old and new selected interfaces.

Lean: `Linear.SearchContexts.relabel`

Lean: `Linear.SearchContexts.selected_label`

Lean: `Linear.SearchContexts.other_label`

Lean: `Linear.SearchContexts.geometry`

Lean: `Linear.SearchContexts.input_eq`

Lean: `Linear.SearchContexts.output_eq`

Theorem 41.17 (The discarded slot label is immaterial)**.** *Plugging any replacement before or after slot relabelling gives isomorphic graphs, after transport along the equal interfaces. This works in both directions.*

Proof. Apply the existing host-plugging transport to the identity incidence isomorphism. For an unselected host edge the labels actually agree, and an inserted edge retains its label, so the transported incidence isomorphism is a literal graph isomorphism. For the reverse direction transport a replacement back to the old interface, apply the forward result, and cancel consecutive opposite interface transports. Cancellation follows by eliminating the two interface equalities and simplifying identity transport. Invert the result. □

Lean: Linear.SearchContexts.plug_iso

Lean: Linear.SearchContexts.plug_iso_reverse

Relabelling by a focus-free label preserves global focus freedom: inspect the selected edge separately and retain the other labels. If every existing hole occurrence belongs to the selected edge, and the new label's entire fibre has no hole, relabelling removes all holes. A hypothetical hole in the new selected fibre contradicts its emptiness; one at another edge transports to an old hole occurrence whose owner would have to be the selected edge.

Lean: Linear.SearchContexts.relabel_free

Lean: Linear.SearchContexts.close_holeFree

Definition 41.18 (Opened slot)**.** Give a hole any natural-number name and the selected edge's complete input and output signatures. Opening the slot relabels that edge by this hole. The selected label is exactly the specified hole label. If the host is focus-free, so is the opened graph: a hole label contains no focus.

Lean: Linear.SearchContexts.hole

Lean: Linear.SearchContexts.openSlot

Lean: Linear.SearchContexts.open_label

Lean: Linear.SearchContexts.focusFree

Lemma 41.19 (Exactly one shallow hole)**.** *Opening a slot in a hole-free host gives a context with exactly one shallow hole occurrence, of the specified label, and no nested holes.*

Proof. Any occurrence at another edge transports along its unchanged label to a hole in the original host, contradicting hole freedom. The selected edge's fibre is a shallow singleton plus an empty child type, and is therefore a subsingleton. Two hole occurrences thus have the same owner and equal fibre positions, proving global hole uniqueness. The shallow occurrence explicitly witnesses presence. Finally any hole occurrence has that owner and that shallow position, so its label is exactly the chosen hole. □

Lean: Linear.SearchContexts.occurrence_owner

Lean: Linear.SearchContexts.selected_fiber

Lean: Linear.SearchContexts.unique

Lean: Linear.SearchContexts.present

Lean: Linear.SearchContexts.only_hole

Definition 41.20 (Focus-free simple search context)**.** A selected edge carries a specified hole label, the graph is globally focus-free, and every hole occurrence has that edge as its outer owner. Because a primitive hole has no child occurrences, this permits exactly one shallow hole and no others. Opening a slot in a focus-free, hole-free host supplies these fields. Conversely, closing any such simple context with a compatible focus-free, hole-free label gives a focus-free, hole-free host.

The constructor for opening uses the selected-label, focus-freedom, and occurrence-owner proofs above. The closing theorem pairs relabelling's focus-freedom theorem with the hole-elimination theorem. Its hypotheses on the new label are automatically satisfied by finite interaction rules.

Lean: `Linear.SearchContexts.Simple`

Lean: `Linear.SearchContexts.simple`

Lean: `Linear.SearchContexts.close_simple`

41.7 The explicit-hole correspondence

Definition 41.21 (Explicit-hole pattern search)**.** Transport a finite fragment's interface to that of the explicitly opened hole. An explicit-hole match is a graph isomorphism from plugging that transported fragment into the opened context to the supplied state. Explicit-hole search chooses one host, slot, and hole name, and one finite interaction rule whose two fragments match the two states using that same opened context.

Lean: `Linear.PatternPlugging.holeFragment`

Lean: `Linear.PatternPlugging.HoleMatches`

Lean: `Linear.State.HoleContextSearch`

The discarded-slot-label isomorphism shows that explicit-hole matching and slot matching are equivalent: compose it with an endpoint match in one direction and compose its inverse in the other. Hence slot-context search and explicit-hole search are equivalent. To open a slot witness one may choose name zero; in the reverse direction any supplied name is retained by the matching equivalence.

Lean: `Linear.PatternPlugging.holeMatches_iff`

Lean: `Linear.State.contextual_iff_holeContext`

Theorem 41.22 (Search adequacy for explicit simple contexts)**.** *Erasure-based state search holds exactly when a finite search interaction is plugged into one explicit focus-free simple hole context, up to graph names. The same equivalence holds for the independent incidence equations. Recursive search adds precisely recursive renaming of the resulting state.*

Proof. Compose slot-context correspondence with explicit-hole matching equivalence. Compose once more with the incidence correspondence for the incidence claim. For recursive search, substitute this equivalence into its existing literal-step-plus-recursive-renaming characterization. □

Lean: `Linear.State.search_iff_holeContext`

Lean: `Linear.State.incidenceSearch_iff_holeContext`

Lean: `Linear.State.deepSearch_iff_holeContext`

Theorem 41.23 (Arbitrary simple contexts are covered)**.** *Take any focus-free simple context, any compatible interaction label, and a finite corolla rule at that label. If its two fragments, with interface transport, plug into that original context to match two states, those states are related by search.*

Proof. The rule forces its label to be a variable or operation, so focus freedom and absence of holes in its label fibre are derived, not added as hypotheses. Close the original hole with this label. The closing theorem gives a focus-free, hole-free host. Reverse slot-label transport identifies each plugging into this host with the supplied plugging into the original context. Compose with the supplied endpoint isomorphisms to obtain a slot-context search witness and apply soundness. Thus representing a context by a closed host and a discarded edge does not exclude an arbitrary simple context. □

Lean: `Linear.State.arbitrary_simple_context`

41.8 Regression examples and scope

The existing terminating value incidence step converts to an explicit-hole context step with a final target. The variable trigger similarly converts to a context step ending in rewrite mode. Opening the value edge with name seven constructs an actual simple context, proves global hole uniqueness, and exhibits the specified shallow hole.

Lean: `Linear.ContextualSearchExamples.value_contextual`

Lean: `Linear.ContextualSearchExamples.variable_contextual`

Lean: `Linear.ContextualSearchExamples.value_simple_context`

An output-fragment regression checks the actual incidence lists for an operation with two term arguments and a thunk. The focus source is the fresh internal vertex and its target is the first output. The operation targets that fresh vertex, then its unchanged second argument and thunk outputs. The proof unfolds the finite constructors and evaluates the split tests.

Lean: `Linear.ContextualSearchExamples.output_fragment_incidence`

The cyclic regression defines a unary operation signature and a well-typed one-edge self-loop with no outer boundary. It reuses the finite self-loop's incidence laws and proves the new term typing directly. Its input and output corolla attachments coincide. Both input and output factorization theorems still apply, providing explicit isomorphisms; no global attachment injectivity or acyclicity is silently assumed.

Lean: `Linear.ContextualSearchExamples.unaryOps`

Lean: `Linear.ContextualSearchExamples.loop`

Lean: `Linear.ContextualSearchExamples.loop_attachment`

Lean: `Linear.ContextualSearchExamples.loop_input`

Lean: `Linear.ContextualSearchExamples.loop_output`

The former search-adequacy item is removed from Appendix B. The following section defines the other transition families and assembles the full UAM. Search adequacy alone does not supply its structural laws or the concrete specimen and robustness interpretation.

42 Concrete substitution, compute behaviour, and the full UAM

This section discharges the transition-definition item in Appendix B. All three transition families of the linear UAM are now defined on actual typed states. Search has the contextual adequacy proved in the preceding section. Substitution is defined by the two actual graphs in Fig. 9(f), plugged into one focus-free simple context. Extrinsic behaviour is a parameter constrained by the compute shape in paper §6.4. The assembled relation has successful final states as normal forms and gives an instance of the abstract machine interface used earlier in this document.

Section 46 proves determinism and refocusing of the full UAM under the corresponding behaviour conditions; the concrete specimen laws remain separate. In this unary linear substitution rule, the variable prefix itself proves the required one-wayness, so no stability assumption on G_S is needed for substitution refocusing. The definitions here use the fixed-depth typed representation; its graph-valued presentation and depth compatibility are now proved in Section 45.

42.1 Constructing the substitution fragments

Definition 42.1 (Variable prefix)**.** Let $G : \star \to O$ be a hypernet. Construct $V; G$ by splitting its sole input: add an internal term vertex z and a unary variable edge from the original input to z, and redirect all original source incidences at the input to z. All other edges, labels, target incidences, and box bodies are retained.

The implementation reuses the certified incidence construction for focus insertion and replaces the new label by V. A variable and a focus have the same singleton term source and target signatures, so the two new typing obligations are immediate; all incidence invariants are inherited.

Lean: `Linear.Substitution.prefixVariable`

Lemma 42.2 (Freedom from focuses and holes)**.** *If G is focus-free, then $V; G$ is focus-free. If G is hole-free, then $V; G$ is hole-free.*

Proof. Every edge is either retained from G or is the new variable. In the first case use the hypothesis, including its recursive assertions inside boxes. The variable label is neither a focus nor a hole and has no children. For hole freedom, extract an occurrence from a proposed hole and apply this case distinction to its owning edge and occurrence fibre. □

Lean: `Linear.Substitution.variable_free`

Lean: `Linear.Substitution.variable_closed`

Definition 42.3 (Substitution fragments)**.** For the same $G : \star \to O$, define

$$L(G) = \mathsf{rewrite}; (V; G), \qquad R(G) = \mathsf{search}; G.$$

Both prefixes are the certified input focus insertion. These are precisely the two shapes in Fig. 9(f), with G playing the role of G_S and O the ordered output list denoted by $\vec{\ell}$ there. No copy of G is modified.

Lean: `Linear.Substitution.before`

Lean: `Linear.Substitution.after`

Lemma 42.4 (Adjacency and unique focus)**.** *In $L(G)$ the variable source list equals the focus target list. If G is focus-free, each fragment has a unique shallow focus, with its prescribed mode and no focus hidden in a box.*

Proof. The second splitting replaces the variable's input source by the new focus target. Both lists therefore consist of that one fresh vertex. The certified focus-insertion theorem applies to $V; G$ using variable-prefix focus freedom, and directly to G on the right. □

Lean: `Linear.Substitution.before_adjacent`

Lean: `Linear.Substitution.before_focussed`

Lean: `Linear.Substitution.after_focussed`

When O is empty and G is also hole-free, both fragments are actual states. Their state constructors use the preceding focus certificates and the theorem that focus insertion preserves hole freedom. On the left this theorem is applied after variable-prefix hole freedom. On the right the search focus starts at the sole input, so the resulting state is initial.

Lean: `Linear.Substitution.beforeState`

Lean: `Linear.Substitution.afterState`

42.2 Context application and its consequences

Lemma 42.5 (Recognizing the unique focus)**.** *If a labelled edge of a focussed graph has focus label m, it is the distinguished focus and its mode is m.*

Proof. A different edge would be focus-free by the focussed certificate, contradicting its label. Having identified the edge, injectivity of the primitive focus constructor identifies the modes. This argument works at every depth bound. □

Lean: `Linear.Focussed.edge_of_label`

Lean: `Linear.Focussed.mode_of_label`

Definition 42.6 (Concrete substitution transition)**.** States s, t satisfy $s \to_V t$ if there exist a closed-interface context $C : \star \to \epsilon$, a selected shallow edge e certified as its unique hole, and a focus-free, hole-free graph $G : \star \to O$, such that the hole's ordered signature is $\star \to O$ and

$$C[L(G)] \cong s, \qquad C[R(G)] \cong t.$$

Here C is focus-free, and the two substitutions use the same C, e, and G. Equalities of the ordered boundary lists provide the explicit interface transports. The isomorphisms preserve graph labels and incidences.

This is application outside boxes at one place, as required in paper §6.4. The witnesses are concrete graphs and plugging operations, not an abstract transition axiom. The endpoint state certificates enforce all global focussed and hole-free restrictions.

Lean: `Linear.State.Substitution`

Lemma 42.7 (Modes and adjacent variable)**.** *Every substitution starts in rewrite mode and ends in search mode. Its source has a variable edge whose unique source is the focus target.*

Proof. Transport each fragment's explicit focus label through plugging and its endpoint isomorphism. Recognition of the unique focus proves the mode claim. For adjacency, transport the left fragment's equality of the variable source and focus target lists, first through boundary casts, then through the common context attachment, and finally through the isomorphism. Recognition identifies the transported focus with the source state's distinguished focus. Its target list is the singleton containing the distinguished target vertex. □

Lean: `Linear.State.substitution_modes`

Lean: `Linear.State.substitution_variable`

Lemma 42.8 (Endpoint renaming)**.** *Substitution is preserved when either endpoint is replaced by a graph-isomorphic state.*

Proof. Keep the context, slot, bound computation, and boundary equalities; compose the two witness isomorphisms with the supplied endpoint isomorphisms. □

Lean: `Linear.State.substitution_iso`

42.3 Extrinsic behaviour and assembly

Definition 42.9 (Compute shape)**.** A pair s, t has compute shape if s is in rewrite mode, t is in search mode, and an active operation edge of s has source list consisting of the focus target. The operation partition is a parameter. The target graph is otherwise unrestricted beyond being a state.

This follows the paper's decision to allow arbitrary effects: compute rules need not be local graph replacements.

Lean: `Linear.State.Compute`

Lemma 42.10 (Substitution excludes compute)**.** *A substitution and a compute transition cannot have the same source, even if their targets differ.*

Proof. The substitution variable and the compute operation would have the same source vertex. Source uniqueness forces their edges to coincide. Their labels would then be equal, contradicting the distinct variable and operation constructors. □

Lean: `Linear.State.substitution_not_compute`

Lemma 42.11 (Search excludes rewrite mode)**.** *The source of a search transition is not in rewrite mode.*

Proof. Extract its underlying coordinate-search witness. If the source mode were rewrite, its starting coordinate would have rewrite mode, contradicting the already proved absence of coordinate-search rules from that mode. □

Lean: `Linear.State.search_not_rewrite`

Definition 42.12 (Admissible behaviour)**.** A behaviour B is a relation on states together with a proof that every pair it relates has compute shape. This is the paper's extrinsic parameter $B_{\mathbb{O}}$. Its shape law is a requirement on the supplied parameter; it is not an assertion that arbitrary relations satisfy that law. Neither determinism nor locality is part of this definition.

Lean: `Linear.Behaviour`

Definition 42.13 (Intrinsic, rewrite, and full transitions)**.** Write $\to_\bullet$ for the checked search relation and $\to_V$ for concrete substitution. Define

$$\to_{\mathrm{int}} = (\to_\bullet) \cup (\to_V), \qquad \to_{\mathrm{rw}} = (\to_V) \cup B, \qquad \to = (\to_{\mathrm{int}}) \cup B.$$

Lean: `Linear.UAM.Intrinsic`

Lean: `Linear.UAM.Step`

Lean: `Linear.UAM.Rewrite`

Proposition 42.14 (Mode discipline and disjointness)**.** *Every rewrite changes rewrite mode to search mode. Every full step is a search or a rewrite. Search and rewrite cannot start at the same state. Intrinsic and extrinsic transitions cannot start at the same state either.*

Proof. For the rewrite-mode claim, use substitution's proved modes or the behaviour's compute-shape certificate. Associativity of disjunction gives the search/rewrite partition. Search excludes rewrite mode, proving the first disjointness claim. For the second, split an intrinsic step into search or substitution: the former is excluded by mode, and the latter by the distinct adjacent edge labels. □

Lean: `Linear.UAM.rewrite_modes`

Lean: `Linear.UAM.step_iff`

Lean: `Linear.UAM.search_rewrite_exclusive`

Lean: `Linear.UAM.intrinsic_extrinsic_exclusive`

Theorem 42.15 (Final normality and the concrete machine)**.** *For every admissible behaviour, successful final states have no full-UAM successor. Hence the full transition relation and the original successful finality predicate form a machine in the abstract sense of Section 2.*

Proof. Search from a final state is impossible by the previously established search boundary theorem. A rewrite would require rewrite mode, whereas finality requires backtrack mode. These cases exhaust full transitions. The concrete machine record contains exactly this relation, this finality predicate, and this proved normality law; it carries no unproved determinism field. □

Lean: `Linear.UAM.final_normal`

Lean: `Linear.UAM.machine`

The empty relation is an admissible behaviour because it has no transitions whose shape needs checking. With this parameter, a full step is equivalent to an intrinsic step, by eliminating the false extrinsic disjunct.

Lean: `Linear.UAM.emptyBehaviour`

Lean: `Linear.UAM.empty_step`

42.4 Actual executions and non-vacuity

The substitution example prefixes a closed passive value by a variable and a rewrite focus, obtaining an actual source state. A one-edge hole corolla is its context witness. It is focus-free and has exactly its sole hole. The corolla plugging unit isomorphisms identify the two plugged fragments with that source and the existing initial value state. This proves an actual substitution, not merely a conditional statement about one. Here and elsewhere, *corolla* is terminology introduced by this formalization for a one-edge graph with its ordered boundary; it is not a term used in the paper.

Lean: `Linear.UAMExamples.source`

Lean: `Linear.UAMExamples.substitute`

With empty behaviour and the passive partition, this substitution followed by the already checked value-search transition gives exactly two full-machine steps to a final state. The

counted execution is built by two applications of the execution constructor and its zero-step base case. The source cannot have compute shape for any operation partition, by the general exclusion theorem.

Lean: `Linear.UAMExamples.terminates`

Lean: `Linear.UAMExamples.source_not_compute`

A separate example changes the operation partition to active and places a rewrite focus before the value symbol. Directly unfolding the incidence lists proves compute shape for the pair consisting of this state and the initial value state. The singleton relation containing that pair is therefore an admissible behaviour; equality substitution reduces its shape certificate to the checked example. Its pair is an actual full-UAM step by the extrinsic injection. This tests the behaviour parameter independently of substitution.

Lean: `Linear.UAMExamples.activeSource`

Lean: `Linear.UAMExamples.compute_shape`

Lean: `Linear.UAMExamples.behaviour`

Lean: `Linear.UAMExamples.compute_step`

The following section discharges the tuple and concrete-definition obligation. Section 46 now supplies substitution rootedness and uniqueness and the full-UAM structural theorem. The concrete specimen interpretation and final robustness instantiation remain unproved.

43 Original-hole tuples and concrete safety predicates

This section discharges the former third item in Appendix B: compatibility of recursive specimens with finite tuple substitution, and definitions of the concrete quasi-specimen, input-safety, and robustness predicates in paper §§8.1–8.3. The compatibility theorem concerns the existing finite-depth representation. The separate comparison with set-based hypernets and changes of depth bound is now proved in Section 45.

The tuple semantics uses simultaneous graph expansion at each original graph layer. Its recursive calls visit original box bodies; a hole replacement is a leaf, never a fresh context to be traversed. This is a direct semantics for parallel application at named holes, with retained edges represented by one-edge copies. It is not a new claim about arbitrary sequential schedules. The earlier ordinary-substitution and interchange theorems concern those schedules. Here the substantive checks are that the tuple is finite and aligned, that its names are distinct original holes, that no original hole is omitted when the result is closed, and that both representations produce the same filled graphs.

The concrete safety predicates below are definitions, not proofs that arbitrary pre-templates satisfy them. They retain the clauses that the abstract interface specialized away. Two lemmas then derive the rooted consequences needed by the abstract proof. Constructing that interface, its full-UAM context-step law, and the final sufficiency instantiation remains unfinished.

43.1 Finite tuples and their application

Definition 43.1 (Aligned substitution entry)**.** An entry records a depth r, ordered interfaces I, O, a hole name h, equalities $I = h.\mathsf{input}$ and $O = h.\mathsf{output}$, and two graphs $H, K : I \to O$ at depth r. A finite list of entries therefore represents the three equally long vectors of names, left replacements, and right replacements. The varying depth and interface of an entry are stored with its two graphs.

The constructor at an actual primitive hole reads its interface from the selected edge. Rewriting its label to the hole label proves the two interface equalities. An entry is related when its two graphs satisfy the pre-template at its recorded depth and interface; this is exactly the pointwise condition on the two vectors.

Lean: `Linear.TupleEntry`

Lean: `Linear.TupleEntry.at`

Lean: `Linear.TupleEntry.Related`

Definition 43.2 (Distinct traversal enumeration)**.** Remove repetitions from a finite enumeration by recursively processing the tail and retaining the head only when it is absent from the processed tail. Apply this operation to the graph's finite edge enumeration.

Lemma 43.3 (Enumeration laws)**.** *Removing repetitions preserves membership and gives a list with no duplicates. Consequently every original shallow edge occurs exactly once in the traversal.*

Proof. Induct on the input list. If the head is already in the processed tail, removing it does not change membership; otherwise prepend it. The same cases preserve no-duplicates, using the tested absence in the second case. Completeness of the graph's supplied enumeration now gives membership of each edge in the distinct traversal list. This matters because the underlying finite-carrier structure permits repeated enumeration entries. □

Lean: `Linear.tupleDistinct`

Lean: `Linear.mem_tupleDistinct`

Lean: `Linear.tupleDistinct_nodup`

Lean: `Linear.tupleEdges`

Lean: `Linear.mem_tupleEdges`

Definition 43.4 (Tuple application in traversal order)**.** A tuple-filling certificate simultaneously describes two filled graphs and an aligned list of entries. At a graph C, choose a left and right replacement for each shallow edge, expand both families, and concatenate the local entry lists in the distinct edge traversal order. Each original edge has one of three local certificates:

1. Retain its label and ordered boundary as one-edge copies on both sides; its entry list is empty.
2. At an actual hole, insert a pointwise related pair and record the one corresponding entry. Do not recurse into either inserted graph.
3. At a box, recursively apply tuples to the original body, retain its binder metadata and external interface, and install one-edge copies of the boxes with those two resulting bodies. Its entry list is the body's list.

The clauses allow partial filling of an open context. The completeness theorem below proves that a closed result forces all original holes to have been filled. Thus the general certificate is also useful before the state restriction is imposed.

Lean: `Linear.TupleFilling`

Lean: `Linear.TupleFillingEdge`

Theorem 43.5 (Compatibility with recursive filling)**.** *There exists a finite tuple-filling certificate from C to L, R if and only if there is a recursive deep-filling certificate with those same three graphs.*

Proof. For the forward direction, forget the entry lists and induct on the depth. The retained and filled clauses give their original certificates directly; apply the induction hypothesis to a boxed body. For the reverse direction, induct on depth again. At each edge record the empty list, the singleton replacement entry, or the recursively obtained body's list. Choose those local witnesses and concatenate them over the finite distinct traversal. The two expansions have exactly the original carriers, incidences, and labels, so neither endpoint graph is changed by extracting the tuple. □

Lean: `Linear.TupleFilling.forget`

Lean: `Linear.DeepFilling.tuple`

Lean: `Linear.tupleFilling_iff`

Lemma 43.6 (Pointwise relatedness)**.** *Every recorded entry satisfies the pre-template relation at its own depth.*

Proof. Membership in a concatenated list selects one original edge and one entry in its local list. A retained edge contributes no entries. A filled edge carries the required relation. For a box use induction on its original body's depth. □

Lean: `Linear.TupleFilling.related`

43.2 Exact original-hole inventory

Lemma 43.7 (Provenance)**.** *Every recorded name is a hole of the original context. More precisely, an entry in the local list for edge e comes from a hole in the occurrence fibre owned by e.*

Proof. A filled edge is itself the named hole. A box's entries come, by induction, from holes in its original body; these are exactly its child occurrences in the outer graph. A retained edge contributes nothing. The graph-level result selects the owning edge using membership in the concatenation. No step of this argument looks for a name inside a replacement graph. □

Lean: `Linear.TupleFilling.provenance`

Lean: `Linear.TupleFillingEdge.provenance`

Theorem 43.8 (Distinct tuple names)**.** *If the original context has unique hole labels, the list of tuple names has no duplicates.*

Proof. Within one local list, the retained and filled cases are empty and singleton. For a box, uniqueness of holes in the original outer context implies uniqueness in its body; apply induction there. Between different original edges, two entries with the same name would give occurrences with that name in both fibres, by local provenance. Original hole uniqueness identifies the occurrences and hence their owning edges, a contradiction. Finally the edge traversal itself has no duplicates. These within-list and between-list facts prove pairwise distinctness of the concatenation and therefore of its names. □

Lean: `Linear.TupleFilling.unique`

Lemma 43.9 (Reflecting closedness)**.** *If a simultaneous expansion is hole-free, every graph inserted by that expansion is hole-free. A hole-free one-edge copy of a box has a hole-free body.*

Proof. An occurrence of a hole in an inserted graph lifts to the expansion by tagging its outer edge with the owning original edge; the label-local occurrence is unchanged. This contradicts closedness. For a copied box, a body hole lifts to a child occurrence of its unique edge, using the equality of the box label. □

Lean: `Linear.tuple_component_closed`

Lean: `Linear.LabelCopy.body_closed`

Theorem 43.10 (Completeness of the tuple)**.** *If the left filled graph is hole-free, every original hole name occurs in the tuple. If the context also has unique holes, its holes are enumerated exactly once by the tuple names.*

Proof. Choose an occurrence of the original hole and its owning edge. If that edge were retained, its one-edge copy would retain the same hole occurrence, contradicting the reflected closedness of its inserted graph. If the edge was filled, the primitive hole label has only its shallow occurrence, so the recorded name is exactly the given name. If it is a box, reflect closedness to the resulting body and apply induction to the original body occurrence. Membership of the owning edge in the traversal puts the entry in the full list. Combining completeness, provenance, and distinctness gives the exact inventory statement. □

Lean: Linear.TupleFilling.complete

Lean: Linear.TupleFilling.inventory

For an actual recursive specimen, its context already has unique holes and both expanded graphs are closed. Tuple extraction therefore supplies the same left and right state graphs, pointwise related entries, distinct names, and the exact original-hole inventory.

Lean: Linear.DeepSpecimen.tuple

Lemma 43.11 (Closed replacement entries)**.** *When both filled results are hole-free, both graphs in every recorded entry are hole-free.*

Proof. Reflect closedness through expansion to each selected local graph. In the filled case these are exactly the entry's two replacements. In the box case reflect through the two copied box labels to their bodies, then apply induction. There are no entries in a retained case. □

This also rules out holes hidden inside replacement graphs in a successful state-producing tuple; they cannot have disappeared through later recursive visits, because such visits are absent from the application clauses.

Lean: Linear.TupleFilling.entries_closed

Definition 43.12 (Named tuple application)**.** An arbitrarily ordered aligned tuple applies when it is a permutation of a traversal-order tuple-filling certificate with the required endpoints. All components of an entry move together. Naming by distinct original holes makes the order of the displayed vectors immaterial.

Existence of such an application is equivalent to recursive filling: erase the permutation witness and forget the tuple in one direction; use the identity permutation on an extracted tuple in the other. Permuting an application again preserves it by composition of list permutations. This is order invariance of the parallel named-tuple presentation, not a separate sequential-scheduling theorem.

Lean: Linear.AppliesTuple

Lean: Linear.appliesTuple_iff

Lean: Linear.AppliesTuple.permute

Definition 43.13 (Complete tuple of a specimen)**.** A specimen tuple packages its entry list, named application to its actual filled state graphs, pointwise relatedness, closed replacements, distinct names, and exact coverage of the context's original holes.

Every recursive specimen supplies this package: choose its extracted list, use identity permutation for application, and apply the relatedness, closedness, and inventory theorems. No compatibility assertion is postulated as a field without constructing its proof.

Lean: Linear.SpecimenTuple

Lean: Linear.DeepSpecimen.substitutionTuple

Theorem 43.14 (Reconstructing a specimen)**.** *Conversely, tuple application in an allowed focussed unique-hole context, with closed left and right results, constructs a recursive specimen with that context, the same focus mode, and exactly those two result graphs.*

Proof. Forget each local tuple certificate, using the already proved graph-level forgetting theorem for boxed bodies. The outer tuple constructor supplies its two replacement families. Put those families and the forgotten local certificates in the specimen record, together with the given context, focus, allowedness, and closedness proofs. Its left and right state graph projections are definitionally the supplied expansions. For an arbitrarily ordered named tuple, first take its traversal-order application witness. □

Lean: `Linear.TupleFillingEdge.forget`

Lean: `Linear.TupleFilling.specimen`

43.3 The full quasi-specimen predicate

In the remaining definitions, fix a layered pre-template T, an allowed class $\mathbb{C}$ of erased focus-free contexts, an operation partition, and an admissible behaviour B. Let M be the concrete full UAM of Section 42. All executions below are executions of M, not search-only executions. A concrete specimen is the existing recursive specimen, whose tuple interpretation was just established.

Definition 43.15 (Settled and remote concrete specimens)**.** A specimen is settled when its context focus is backtracking, or searching without entering a hole. It is remote when its focus is not a rewrite focus or is not entering a hole. Settled implies remote, by distinguishing the two settled modes from rewrite mode.

Lean: `Linear.ConcreteSafety.Settled`

Lean: `Linear.ConcreteSafety.Remote`

Lean: `Linear.ConcreteSafety.settled_remote`

Definition 43.16 (Concrete quasi-specimen)**.** For relations A, D on actual states, a pair s, t is a quasi-specimen up to (A, D) when there exists an allowed specimen S satisfying:

1. The focus modes of S, s, and t agree.
2. If s and t are both rooted, then both filled states S_L, S_R are rooted, $A(s, S_L)$, and $D(S_R, t)$.

The existence of S and the mode equalities are unconditional. Only the second clause is conditional on rootedness. A non-rewrite quasi-specimen additionally requires that the common mode is not rewrite.

These are the clauses of paper §8.1. In particular, an unrooted pair does not satisfy the definition merely by making a rootedness implication vacuous: a mode-compatible specimen must still exist.

Lean: `Linear.ConcreteSafety.Quasi`

Lean: `Linear.ConcreteSafety.NonRewriteQuasi`

Lemma 43.17 (Elementary quasi-specimen laws)**.** *Every actual specimen yields a quasi-specimen of its two filled states up to equality. Every quasi-specimen has equal endpoint modes. Weakening either up-to relation preserves quasi-specimens.*

Proof. For the first claim use the specimen itself: focus preservation gives the mode equalities, and the conditional conclusions are the supplied rootedness hypotheses and reflexive equalities. For the second, compose the two mode equalities through the witness's mode. For weakening, retain that same witness and compose its two conditional comparison facts with the supplied implications of relations. □

Lean: `Linear.ConcreteSafety.specimen_quasi`

Lean: `Linear.ConcreteSafety.Quasi.modes`

Lean: `Linear.ConcreteSafety.Quasi.mono`

Lemma 43.18 (Rooted non-rewrite consequence)**.** *Given a concrete non-rewrite quasi-specimen and rooted endpoints, there is a specimen whose mode is not rewrite, whose two filled states are rooted, and which provides the two up-to comparisons.*

Proof. Take the unconditional witness. Its mode equality transfers the endpoint's non-rewrite property to it. Apply its conditional clause to the two rootedness hypotheses. This derives the consequence used by the abstract proof; it does not identify the stronger concrete definition with the weaker abstract one. □

Lean: `Linear.ConcreteSafety.NonRewriteQuasi.rooted_witness`

43.4 Input-safety and robustness with the paper's clauses

Definition 43.19 (Concrete input-safety)**.** For every allowed specimen S with an entering search focus, at least one of the following holds. There is no rootedness restriction on S.

1. Both filled states reach stuck states in finitely many full-UAM steps.
2. For some settled specimen D and $k, l \in \mathbb{N}$, $(1+k)\,Q\,l$, $S_L \longrightarrow^{1+k} D_L$, and $S_R \longrightarrow^{l} D_R$.
3. For some $k, l \in \mathbb{N}$ and non-rewrite quasi-specimen s, t up to $(\simeq_P, \simeq_P)$, $(1+k)\,Q\,(1+l)$, $S_L \longrightarrow^{1+k} s$, and $S_R \longrightarrow^{1+l} t$.

Here stuckness and the state equivalences use M. In the third alternative both up-to relations are equivalences, as in the paper, not only refinements.

Lean: `Linear.ConcreteSafety.InputSafe`

Definition 43.20 (Concrete robustness relative to a rewrite)**.** For a rewrite pair $x \to x'$, consider every allowed specimen S with $S_L = x$ and a non-entering rewrite focus. At least one of these holds:

1. S_L or S_R is not rooted.
2. x' reaches a stuck state in finitely many full-UAM steps.
3. For some $k, l \in \mathbb{N}$ and non-rewrite quasi-specimen s, t up to $(\preceq_P, \preceq_R)$, $(1+k)\,Q\,l$, $x' \longrightarrow^k s$, and $S_R \longrightarrow^l t$.

Robustness for the UAM requires this property for every substitution or extrinsic rewrite transition of the supplied behaviour.

The relative predicate is stated on a pair; the global predicate restricts its quantified pairs to actual rewrites. This preserves the paper's unrooted alternative and its asymmetric step counts, including the one already taken on the left.

Lean: `Linear.ConcreteSafety.RobustRelative`

Lean: `Linear.ConcreteSafety.Robust`

Definition 43.21 (Concrete template)**.** A template is input-safe and output-closed. In the depth-indexed presentation, output closure is required at every available layer of the pre-template.

Lean: `Linear.ConcreteSafety.Template`

Lemma 43.22 (Restricting robustness to rooted specimens)**.** *For a robust pre-template and a non-entering rewrite specimen with two rooted filled states, every rewrite of its left state has either the stuck-tail alternative or the matching non-rewrite quasi-specimen alternative above.*

Proof. Apply concrete robustness to that actual rewrite and specimen. Its first alternative contradicts one of the two supplied rootedness hypotheses; the remaining disjunction is exactly the claimed result. This is the legitimate reason the abstract rooted interface can omit the unrooted alternative. □

Lean: `Linear.ConcreteSafety.Robust.rooted`

43.5 Regression examples and scope

The existing example has one hole strictly inside a box and no shallow hole. Its complete substitution tuple contains that deep hole. In fact its list of names is exactly the singleton containing that name: completeness proves membership, provenance and the example's occurrence classification exclude any other name, and no-duplicates excludes a second entry. Every recorded replacement is closed by the general reflected-closedness theorem. These checks exercise genuine recursion through a box, not just shallow filling.

Lean: `Linear.TupleSafetyExamples.nested_hole_recorded`

Lean: `Linear.TupleSafetyExamples.nested_tuple_singleton`

Lean: `Linear.TupleSafetyExamples.nested_entries_closed`

The same example gives a quasi-specimen up to equality although at least one of its filled states is not rooted, by the previously proved path obstruction. Thus the definition has not

accidentally required unconditional rootedness. Conversely, an initial search-mode value and a final backtrack-mode value cannot form a quasi-specimen, even if both up-to relations are universally true: the unconditional endpoint-mode equality would equate two distinct focus constructors.

Lean: `Linear.TupleSafetyExamples.unrooted_quasi`

Lean: `Linear.TupleSafetyExamples.different_modes_not_quasi`

The tuple and concrete-definition obligation is removed from Appendix B. This does not assert that a particular nontrivial pre-template is input-safe or robust. Section 47 now constructs the named-state specimen interface and proves the safety and robustness bridges. General initial observation lifting and state-class integration still separate these results from the final theorem.

44 Set presentations, depth bounds, and a box-label obstruction

The representation question has two different parts. Changing arbitrary finite vertex sets into tagged internal and boundary vertices is now proved harmless. So is recursively making that change inside binder-annotated boxes, and normalizing a finite family of different depth allowances to one common bound. However, these results do not yet justify forgetting the binder annotations from box labels. The final subsection gives a checked counterexample to that particular proposed forgetful translation. Section 45 supplies the freshening translation that resolves this obstruction and completes the representation obligation.

44.1 Arbitrary vertex sets and ordered boundaries

Definition 44.1 (Set presentation)**.** For a signature σ and ordered boundary types I, O, a set presentation has finite vertex and edge carriers V, E, vertex and edge labels, source and target lists, and injective boundary maps $i : \{0, \dots, |I| - 1\} \to V$ and $o : \{0, \dots, |O| - 1\} \to V$. The boundary images are disjoint and have the prescribed vertex labels. Source and target lists are separately duplicate-free, and no vertex occurs in the source lists of two different edges, or in the target lists of two different edges. A vertex has a source incidence exactly when it is not an output, and a target incidence exactly when it is not an input. Mapping vertex labels over an incidence list gives the signature of its edge label.

This is the fixed-signature form of the paper's outer graph definition. The two boundary functions encode ordered lists by their indices. Disjointness and coverage exclude isolated vertices, but allow cycles and a vertex shared by the source and target of the same edge. The signature formulation implies the paper's equal-label/equal-signature condition. This implication concerns the chosen label alphabet; changing that alphabet requires a separate check.

Lean: `Graph.SetNet`

Definition 44.2 (Boundary normalization)**.** The interior V° consists of vertices outside both boundary images. The normalized vertex carrier is the disjoint union of V°, input indices, and output indices. The map decode sends an interior to its old vertex and a boundary index to the corresponding boundary vertex. The inverse candidate encode first tests input membership, then output membership, and otherwise records the vertex as interior. Classical choice supplies the index when membership is known.

Lean: `Graph.SetNet.Interior`

Lean: `Graph.SetNet.decode`

Lean: `Graph.SetNet.encode`

Lemma 44.3 (Inverse maps)**.** *Decoding an encoded vertex returns the vertex. Decoding is injective, and encoding a decoded normalized vertex returns that normalized vertex. Consequently the two maps form a bijection.*

Proof. The first identity follows by the three membership cases and the specification of the chosen index. For injectivity, compare the constructors of both normalized vertices. Two interiors are equal by subtype extensionality; two inputs or two outputs are equal by boundary injectivity. Mixed cases contradict either an interior's defining exclusions or boundary disjointness. Apply this injectivity to the first identity to obtain the other inverse law. □

Lean: `Graph.SetNet.decode_encode`

Lean: `Graph.SetNet.decode_injective`

Lean: `Graph.SetNet.encode_decode`

Lean: `Graph.SetNet.vertices`

Lemma 44.4 (Labels and boundary classification)**.** *Encoding preserves vertex labels. An encoded vertex is tagged as output exactly when its old vertex is in the output image; the analogous statement holds for inputs.*

Proof. For label preservation first prove the claim for decoding, by the three constructors and the two boundary typing laws, then use the inverse identity. For boundary classification, a tag supplies its index; conversely, an old boundary vertex is the decoding of that tag, so the second inverse identity forces the correct tag. No assumption of decidable carrier equality is needed. □

Lean: `Graph.SetNet.encode_label`

Lean: `Graph.SetNet.output_iff`

Lean: `Graph.SetNet.input_iff`

Definition 44.5 (Normalized graph)**.** Keep the edge carrier and edge labels, use V° as the internal carrier, and map encode over every source and target list. Interior finiteness is inherited by taking a subtype of the old finite vertex enumeration.

Verification of the graph laws. Injectivity of encoding preserves duplicate-free incidence lists. Two source occurrences at a normalized vertex decode to the same old vertex, so source uniqueness identifies their edges; the target argument is identical. Inverse maps transfer list membership, and the boundary-classification lemma transfers the coverage equivalences. Finally, mapping labels after encoding is the same as mapping the old labels, so the old signature equations prove typing. □

Lean: `Graph.SetNet.toNet`

44.2 Outer round trips and transport of isomorphisms

Definition 44.6 (Set-graph isomorphism and exposure)**.** A set-graph isomorphism comprises bijections of vertices and edges preserving labels, boundary indices, and the ordered incidence lists. Exposing an existing `Net` uses its entire tagged vertex carrier as one ordinary finite carrier; the input and output maps are its boundary constructors.

The exposure enumeration concatenates the internal enumeration and the two finite index enumerations. The constructor cases prove coverage, disjointness, and boundary typing; all remaining incidence laws are inherited.

Lean: `Graph.SetNet.Iso`

Lean: `Graph.Net.toSet`

Theorem 44.7 (Outer round trips)**.** *Every set presentation is isomorphic to its normalization followed by exposure. Every tagged graph is isomorphic to its exposure followed by normalization.*

Proof. For the first direction use the encoding bijection on vertices and the identity on edges. Its inverse laws, label preservation, and boundary identities already supply all fields. For the second direction, the interior of an exposed tagged graph contains precisely its original internal vertices. The bijection sends an old internal vertex to its internal tag with the boundary exclusions. Its inverse eliminates impossible boundary cases. Encoding an arbitrary old vertex agrees with mapping this interior bijection over its port constructor; apply that equality to every source and target list to finish the isomorphism. □

Lean: `Graph.Net.set_roundtrip`

Lean: `Graph.Net.interiorBijection`

Lean: `Graph.Net.encode_toSet`

Lean: `Graph.Net.roundtrip`

Definition 44.8 (Related-label isomorphism and relabelling)**.** A related-label set isomorphism replaces literal edge-label equality by a supplied relation R; all other conditions remain unchanged. Relabelling along a signature-preserving function changes only edge labels. The new typing proof composes the old signature equation with the signature-preservation identity; all vertex and incidence fields are unchanged.

Lean: `Graph.SetNet.RelIso`

Lean: `Graph.SetNet.relabel`

Lemma 44.9 (Normalization of related-label isomorphisms)**.** *A related-label set isomorphism induces a related-label isomorphism between the normalized graphs, with the same edge bijection.*

Proof. Boundary membership is preserved by the boundary equations and reflected by vertex injectivity. Hence the vertex bijection restricts to the interiors; its inverse uses the inverse vertex map and the same boundary equations. Encoding commutes with that restricted bijection: decode both sides, inspect the port constructor, and use the inverse identities. Mapping this commuting square over each incidence list proves the two required list equations. Vertex labels and related edge labels come directly from the original isomorphism. These are the declarations `input_iff`, `output_iff`, `interiors`, `encode_map`, and `RelIso.toNet`, respectively. □

Lean: `Graph.SetNet.RelIso.input_iff`

Lean: `Graph.SetNet.RelIso.output_iff`

Lean: `Graph.SetNet.RelIso.interiors`

Lean: `Graph.SetNet.RelIso.encode_map`

Lean: `Graph.SetNet.RelIso.toNet`

44.3 Recursive annotated boxes

Definition 44.10 (Annotated set hierarchy)**.** An annotated set box records a bound count b, a free count f, and a set graph body with interface $\star \Rightarrow \star^{b+f}$. Its external signature is $T^b(\star) \Rightarrow \star^f$. The depth-zero alphabet contains primitives; the next alphabet adds boxes over the preceding alphabet. A hypernet is a set presentation over this alphabet. This intentionally retains the annotations of the existing Lean hierarchy.

Lean: `Linear.SetHierarchy.SetBox`

Lean: `Linear.SetHierarchy.nextAlphabet`

Lean: `Linear.SetHierarchy.alphabet`

Lean: `Linear.SetHierarchy.Hypernet`

Definition 44.11 (Recursive conversion)**.** Normalization at depth zero uses outer normalization. At a successor depth it also relabels every box by the same counts and the recursively normalized body. Exposure performs the inverse-direction construction: expose the outer carrier and recursively expose every box body. Primitives are unchanged, and the external signatures agree because neither operation changes the counts.

Lean: `Linear.SetHierarchy.normalize`

Lean: `Linear.SetHierarchy.expose`

Theorem 44.12 (Recursive typed round trip)**.** *Every typed hypernet is recursively isomorphic to its exposure followed by normalization.*

Proof. Induct on the depth. The base is the outer round trip. At a successor, use its interior bijection, identity edge bijection, and incidence equations. A primitive label remains literally equal. A box retains both counts, and its bodies are recursively isomorphic by induction. This supplies exactly the recursive label relation required by `Linear.DeepIso`. □

Lean: `Linear.SetHierarchy.typed_roundtrip`

Definition 44.13 (Recursive set isomorphism)**.** Related set boxes have equal binder counts and related bodies at their common interface. Set-label equivalence is equality for primitives and existence of a recursive set-graph isomorphism for boxes. A recursive set isomorphism is an outer related-label isomorphism using that relation.

Lean: `Linear.SetHierarchy.BoxRelated`

Lean: `Linear.SetHierarchy.LabelEquiv`

Lean: `Linear.SetHierarchy.DeepIso`

Theorem 44.14 (Recursive set round trip)**.** *Every annotated set hypernet is recursively isomorphic to its normalization followed by exposure.*

Proof. Again induct on depth. The outer encoding bijection preserves labels and ordered boundaries. At a box label the counts are unchanged and the induction hypothesis supplies the body isomorphism. Thus independent vertex renaming at every nesting level is accounted for, rather than just renaming the outer graph. □

Lean: `Linear.SetHierarchy.set_roundtrip`

Lemma 44.15 (Metadata determined by external type)**.** *If two annotated boxes have equal external input and output types, their bound and free counts agree.*

Proof. Equality of the singleton input lists gives equality of the thunk labels, and injectivity of the thunk constructor gives the bound count. Taking lengths of the replicated output lists gives the free count. Notice the premise: the *external type* is fixed. Equality of contents alone does not supply it. □

Lean: `Linear.SetHierarchy.metadata_unique`

Theorem 44.16 (Transport and reflection of recursive isomorphism)**.** *Normalization transports recursive set isomorphisms; exposure transports recursive typed isomorphisms. Two typed hypernets are recursively isomorphic if and only if their exposures are recursively set-isomorphic.*

Proof. For normalization, restrict the outer vertex bijection to interiors as above. Induct on depth for edge labels: primitive pairs retain their equality, primitive/box pairs are impossible, and a related box pair supplies the body isomorphism to which induction applies. Exposure uses the port bijection of the original internal-vertex bijection and the same recursive argument for labels. For reflection, normalize the proposed set isomorphism and compose with the typed round trip on the left and its inverse on the right. The forward direction is exposure transport itself. □

Lean: `Linear.SetHierarchy.normalize_iso`

Lean: `Linear.SetHierarchy.expose_iso`

Lean: `Linear.SetHierarchy.expose_iso_iff`

44.4 Coherence of depth allowances

Definition 44.17 (Depth promotion)**.** Promotion increases the permitted depth by one while preserving the outer carrier, incidence and primitive labels; each box body is recursively promoted. Repeated promotion, written $\mathsf{raiseBy}(n, k)$, increases a bound n to $n + k$. There is a corresponding operation on typed hypernets.

Lean: `Linear.SetHierarchy.raise`

Lean: `Linear.SetHierarchy.raiseBy`

Lean: `Linear.SetHierarchy.typedRaiseBy`

Theorem 44.18 (Normalization commutes with promotion)**.** *Normalizing after one promotion equals promoting after normalization. The same equality holds for any finite number of promotions.*

Proof. Induct on the original depth for one promotion. At depth zero the two graph records are definitionally equal. At a successor all outer data still agree; primitive labels agree immediately and the box-body fields agree by induction. Proof fields are irrelevant to equality. Induction on the number of promotions then proves the iterated equation using the one-step equation at the current bound. These are equalities of the resulting typed graphs, not merely existence of some isomorphism. □

Lean: `Linear.SetHierarchy.normalize_raise`

Lean: `Linear.SetHierarchy.normalize_raiseBy`

Definition 44.19 (Uniformization of a finite box family)**.** A box family indexed by a finite carrier E assigns each member its own depth, bound count, free count, and body. Take the maximum of the depths in an exhaustive finite enumeration of E, with maximum zero for an empty list. Pad each member from its original depth to that common maximum.

Lean: `Linear.SetHierarchy.BoxFamily`

Lean: `Linear.SetHierarchy.maximum`

Lean: `Linear.SetHierarchy.BoxFamily.common`

Lean: `Linear.SetHierarchy.BoxFamily.uniform`

Lemma 44.20 (Common bound and conversion)**.** *Each member's depth is at most the common maximum. Normalizing its uniformized body equals promoting its normalized original body to that same maximum.*

Proof. Membership in a list implies that an entry is bounded by its recursively computed maximum: for the head use the left maximum inequality, and for a tail member combine induction with the right maximum inequality. Exhaustiveness puts every family member in the mapped enumeration, even if the enumeration contains duplicates. Hence $n + (N - n) = N$, which supplies the interface-preserving depth transport used in uniformization. Normalization commutes with this transport by equality elimination. Apply the iterated promotion equation to finish the claimed equality. □

Lean: `Linear.SetHierarchy.le_maximum`

Lean: `Linear.SetHierarchy.BoxFamily.le_common`

Lean: `Linear.SetHierarchy.BoxFamily.uniform_normalize`

This proves that a finite family of separately bounded annotated contents can be represented at a common bound. It does not yet formalize a translation of an undecorated graph-label alphabet from the paper; the distinction below is independent of finite-depth padding.

44.5 The obstruction to naive binder erasure

Definition 44.21 (Body consistency for binder erasure)**.** A successor-depth typed graph is body-consistent when any two box edges whose contents are literally equal also have equal bound and free counts. The Lean premise uses heterogeneous equality so contents at definitionally different interface expressions can be compared. This is the condition needed for the naive operation that forgets binder annotations but leaves contents unchanged to respect the paper's equal-label/equal-signature requirement.

Lean: `Linear.SetHierarchyExamples.BodyConsistent`

Proposition 44.22 (Checked obstruction to naive binder erasure)**.** *There is a well-formed typed graph which is not body-consistent.*

Proof. Let $V : \star \Rightarrow \star$ be the unary variable corolla. Use this exact body in two boxes: one with $(b, f) = (0, 1)$, and one with $(b, f) = (1, 0)$. Their body records agree, but their external signatures are respectively $T^0(\star) \Rightarrow \star$ and $T^1(\star) \Rightarrow \epsilon$. Construct a graph with two input vertices of these thunk types and one term output. The first edge connects the first input to the output; the second edge has only the second input and no targets. The graph has no internal vertices. Direct finite case analysis verifies all incidence, coverage and typing conditions. Applying body consistency to its two edges would assert $0 = 1$, a contradiction. □

Lean: `Linear.SetHierarchyExamples.exposed`

Lean: `Linear.SetHierarchyExamples.bound`

Lean: `Linear.SetHierarchyExamples.identical_contents`

Lean: `Linear.SetHierarchyExamples.different_signatures`

Lean: `Linear.SetHierarchyExamples.twoBoxes`

Lean: `Linear.SetHierarchyExamples.forgetting_binders_fails`

The annotated normalization and round-trip theorems still apply to this counterexample, as checked by `twoBoxes_roundtrip`. A second example checks a box with two holes, exercising recursive contents rather than just outer incidence. Finally, `promoted_box` checks normalization of that box after two promotions. These examples test the exact scope of the theorems; none silently assumes body consistency.

Lean: `Linear.SetHierarchyExamples.twoBoxes_roundtrip`

Lean: `Linear.SetHierarchyExamples.nested_roundtrip`

Lean: `Linear.SetHierarchyExamples.promoted_box`

Relation to the completed representation theorem. The results above prove arbitrary-vertex boundary normalization, recursive round trips for the annotated hierarchy, isomorphism transport, and finite mixed-depth uniformization. The counterexample refutes only naive binder erasure. Section 45 now gives the required freshening translation: it changes actual vertex names inside contents, proves both recursive round trips and quotient inverse laws, and satisfies the literal graph-label signature condition. No body-consistency restriction is imposed on the typed graphs.

45 Actual graph labels by fresh vertex naming

This section resolves the representation item that was left open by Section 44. The repair changes the names of vertices inside box contents, rather than discarding their binder counts while leaving the contents literally unchanged. The translated box label is an actual graph. Its sole input has a name from which its external signature can be read. Equal graph labels therefore have equal signatures, as required by the paper.

There are no added vertices or edges. All old incidence, wire types, primitive labels, and ordered boundaries are preserved by explicit bijections, recursively inside every box. Both representation round trips are proved. Encoding preserves and reflects recursive isomorphism; encoding and decoding are mutually inverse on graph classes. The construction accepts every typed hypernet, including the previous counterexample, and commutes with increasing the depth allowance.

45.1 Finite sets of actual vertex names

Definition 45.1 (Named graph)**.** Take $\mathbb{N}^3$ as a common set of vertex names. A named graph stores a finite list of such names, with vertex carrier the subtype of names occurring in the list. Thus a name occurring twice in the enumeration is still only one vertex. The edge carrier is an arbitrary finite carrier. Vertex labels, edge labels, ordered incidence, and ordered boundary maps satisfy the same conditions as `SetNet`. All boundary vertices have ordinary names in this common space. The operation `toSet` exposes this subtype as the graph's vertex carrier.

Its finite enumeration is the list of names equipped with their membership proofs. Every vertex is in that enumeration by its defining membership proof. All other fields of the set presentation are inherited unchanged.

Lean: `Graph.VertexName`

Lean: `Graph.NamedNet`

Lean: `Graph.NamedNet.toSet`

Definition 45.2 (Fresh naming)**.** Given an exhaustive enumeration of a finite carrier, assign each vertex the position of its first occurrence. For chosen natural numbers a, b, give it the name $(a, b, \mathsf{index}(v))$. The finite name list is the image of the old enumeration under this function. The map `naming` sends a vertex to this name with its membership proof; its inverse chooses a preimage.

Injectivity and inverse laws. If two first-occurrence indices coincide, looking up the enumeration at that index returns both vertices, so the vertices coincide. Equality of names implies equality of their third coordinates, hence equality of vertices. Every member of the finite name list has a preimage by membership in a mapped list. Choosing one gives a right inverse; injectivity makes it a left inverse as well. These facts prove `index_injective` and every field of the naming bijection. Exhaustiveness is essential; duplicate enumeration entries are harmless. □

Lean: `Graph.NamedNet.index`

Lean: `Graph.NamedNet.index_injective`

Lean: `Graph.NamedNet.name`

Lean: `Graph.NamedNet.freshNames`

Lean: `Graph.NamedNet.naming`

Definition 45.3 (Freshened set graph)**.** The operation `ofSet` changes a set graph's vertex carrier to its fresh name set. It keeps the edge carrier and edge labels, maps the naming bijection over every incidence list and boundary vertex, and reads the new vertex labels through the inverse bijection.

Verification of the graph laws. Injectivity transfers boundary injectivity, disjointness, and duplicate-free incidence. Equal named occurrences decode to equal old occurrences, so old incidence uniqueness identifies their edges. A named vertex belongs to a mapped incidence list exactly when its inverse belongs to the old list. Combined with the inverse laws for the boundary maps, this transfers both coverage equivalences. For typing, composing vertex labels with the naming map cancels the inverse bijection; the old signature equations then apply. □

Lean: `Graph.NamedNet.ofSet`

Lemma 45.4 (Freshening changes only names)**.** *The original set graph and its freshening are label-preservingly isomorphic. Every freshened input has the chosen first two name coordinates. Starting with a tagged graph, fresh naming followed by boundary normalization also returns an ordinary isomorphic graph.*

Proof. Use the naming bijection and the identity edge map for the first isomorphism; all incidence equations hold by construction and labels by the inverse law. The input-prefix statement is definitional. For the typed round trip, compose the old tagged-to-set normalization isomorphism with the normalization of the freshening isomorphism. Literal label equality is reversed where the ordinary `Net.Iso` convention requires it. □

Lean: `Graph.NamedNet.ofSet_iso`

Lean: `Graph.NamedNet.input_prefix`

Lean: `Graph.NamedNet.typed_roundtrip`

45.2 Isomorphism infrastructure for transported boundaries

Definition 45.5 (Boundary-list isomorphism)**.** `HRelIso` relates set presentations whose interface indices may have different presentations. It has bijections of full vertex and edge carriers, preserves vertex labels and ordered incidence, and relates edge labels by a supplied relation R. It compares the lists enumerated by the boundary maps: mapping the vertex bijection over each original boundary list gives the new boundary list. This avoids a premature cast between finite index types.

Lean: `Graph.SetNet.HRelIso`

Lemma 45.6 (Indexed and list formulations)**.** *An indexed related-label isomorphism gives a boundary-list isomorphism. A literal-label isomorphism is a special case. Conversely, for identical interface types, a boundary-list isomorphism gives an indexed one.*

Proof. For the forward map, mapping a function over a list generated by finite indices is the list generated by the composite function; use the indexed boundary equations pointwise. Literal equality supplies the special-case label relation. For the converse, look up each index in the preserved boundary list. The two lookups agree, giving exactly the indexed boundary equation. Carrier maps and incidence fields are unchanged in all three constructions. □

Lean: `Graph.SetNet.HRelIso.ofRelIso`

Lean: `Graph.SetNet.HRelIso.ofIso`

Lean: `Graph.SetNet.HRelIso.same`

Lemma 45.7 (Identity, composition, inversion, and relabelling)**.** *Boundary-list isomorphisms have identities for reflexive label relations, compose for transitive relations, and invert for symmetric relations. More generally, label relations R, S can compose to T whenever $R(x, y) \wedge S(y, z) \Rightarrow T(x, z)$. A signature-preserving translation of labels transports an isomorphism whenever it transports the label relation.*

Proof. Identity uses identity carrier maps. Composition composes carrier bijections; the list equations follow by mapping twice and composing the vertex functions. The edge-label proof is precisely the specified relational implication. For inversion, map the inverse vertex function over each original list equation and cancel the forward map; inverse edge indices cancel by the edge bijection law. Apply symmetry to the edge-label comparison. Relabelling retains all carrier and incidence fields and applies its supplied implication to each edge-label proof. The two composition declarations are the same argument, first for one relation and then for three potentially different relations. □

Lean: `Graph.SetNet.HRelIso.trans`

Lean: `Graph.SetNet.HRelIso.comp`

Lean: `Graph.SetNet.HRelIso.symm`

Lean: `Graph.SetNet.HRelIso.relabel`

Lean: `Graph.SetNet.HRelIso.refl`

Lemma 45.8 (Round trips and independent fresh names)**.** *A named graph is isomorphic to its normalization followed by fresh naming. Exposing a typed related-label isomorphism gives a full-vertex boundary-list isomorphism. Independently freshening both sides preserves such an isomorphism.*

Proof. The first statement composes the outer set round trip with freshening. For exposure, map internal vertices by the original bijection and fix every boundary index; the three port constructors verify labels and boundaries. For independent freshening, compose the inverse freshening on the left, the given isomorphism, and the forward freshening on the right. The mixed-relation composition law uses equality to discharge the two new label comparisons. □

Lean: `Graph.NamedNet.roundtrip`

Lean: `Graph.Net.RelIso.toSet`

Lean: `Graph.NamedNet.map_iso`

Lemma 45.9 (Relabelling coherence)**.** *Signature-preserving relabelling of a named graph retains all actual vertex names. Fresh naming commutes with relabelling. Pointwise equal label translations induce equal relabelled set graphs.*

Proof. Relabelling copies every field except the edge labels and their typing proofs; the new typing equation composes the old equation with signature preservation. The fresh name list and naming bijection depend only on the old vertex carrier and its enumeration, so the commuting equation is definitional. Finally, function extensionality identifies pointwise equal label maps, after which the graph records agree by proof irrelevance. □

Lean: `Graph.NamedNet.relabel`

Lean: `Graph.NamedNet.ofSet_relabel`

Lean: `Graph.SetNet.relabel_congr`

45.3 Boxes labelled solely by actual contents

Definition 45.10 (Body package)**.** A body package is a named graph with input type $\star$ and output type $\star^r$, together with the actual output count r. This count describes the graph's own interface. It is not the bound count or free count of an enclosing box. Indeed, `body_output_length` proves that it equals the length of the body's ordered output list, simply by the length of a finite-index list.

Lean: `Linear.PaperEncoding.Body`

Lean: `Linear.PaperEncoding.body_output_length`

Definition 45.11 (Actual-content label)**.** For a body package C, read numbers $b(C), f(C)$ from the first two coordinates of the name of its sole input vertex. A content label consists of C and a proof that $b(C) + f(C) = r$. There are no separately stored binder fields. Its external signature is $T^{b(C)}(\star) \Rightarrow \star^{f(C)}$. The base alphabet is the primitive alphabet; a successor alphabet adds these actual-content labels. A hypernet is a named graph over that alphabet.

The displayed name format chooses convenient representatives. It does not restrict the old typed graph classes: arbitrary finite carriers admit the proved naming bijection, and the encoder below chooses the needed prefix separately for each box. Wire labels and incidence are untouched by that choice.

Lean: `Linear.PaperEncoding.bound`

Lean: `Linear.PaperEncoding.free`

Lean: `Linear.PaperEncoding.Content`

Lean: `Linear.PaperEncoding.nextAlphabet`

Lean: `Linear.PaperEncoding.alphabet`

Lean: `Linear.PaperEncoding.Hypernet`

Lemma 45.12 (Equal contents have equal signatures)**.** *If two content labels have literally equal body packages, their complete external signatures agree. Forgetting their admissibility proofs adds no label information. For any graph in the presentation, equal literal labels give equal mapped vertex labels on both source lists and both target lists.*

Proof. Subtype extensionality identifies content labels with equal underlying packages; substitution proves both signature equalities. Define `bareLabel` to keep a primitive or the actual body package and omit the proof. It is injective: equal primitives are handled by constructor injectivity, mixed constructors cannot agree, and equal body packages identify their content subtypes. For two edges, injectivity therefore identifies their original labels. Rewrite the two incidence-typing equations with that equality. This is the paper's literal equal-label/equal-signature condition, now checked directly for graph-valued box labels. It is not a claim that arbitrary isomorphic contents must give the same enclosing type. □

Lean: `Linear.PaperEncoding.same_content_signature`

Lean: `Linear.PaperEncoding.bareLabel`

Lean: `Linear.PaperEncoding.bareLabel_injective`

Lean: `Linear.PaperEncoding.equal_labels_same_types`

45.4 Encoding, decoding, and recursive round trips

Definition 45.13 (Signature-preserving alphabet map)**.** An alphabet map consists of a function on symbols and proofs that it preserves the complete ordered input and output signatures. These proofs make relabelling of well-typed graphs possible without changing their incidence.

Lean: `Linear.PaperEncoding.AlphabetMap`

Definition 45.14 (Encoding)**.** The base alphabet map is the identity. At a successor level a primitive remains unchanged. An annotated box with counts b, f becomes the actual recursively translated body, freshly named with prefix (b, f), and with output count $b + f$. The prefix equation supplies its content-label admissibility proof. Encoding a whole graph translates its labels and then freshly names its outer vertices with any chosen prefix (a, b). The prefix of an outer graph is arbitrary; when the graph is used as box contents, the enclosing box chooses its prefix.

The input-prefix theorem is definitional. The alphabet-map signature proofs are also by the primitive/box cases: in the box case, reading the fresh input name returns exactly the old bound and free counts.

Lean: `Linear.PaperEncoding.encodeMap`

Lean: `Linear.PaperEncoding.encode`

Lean: `Linear.PaperEncoding.encode_prefix`

Definition 45.15 (Decoding)**.** Decoding at depth zero normalizes the named vertex carrier. At a successor it also translates each content label to an annotated box whose counts are read from the body's input name. Recursively decode the body, and transport its output interface along the content proof $b + f = r$. The primitive case remains unchanged. The resulting alphabet map preserves signatures by construction.

Lean: `Linear.PaperEncoding.decodeMap`

Lean: `Linear.PaperEncoding.decode`

Theorem 45.16 (Typed round trip)**.** *For every typed hypernet G, every depth, and every chosen outer prefix, G is recursively isomorphic to* $\mathsf{decode}(\mathsf{encode}(G))$*. Every typed hypernet therefore has an actual-content representative.*

Proof. Induct on depth. The base is fresh naming followed by outer normalization. At a successor use that same outer isomorphism for carriers, incidence, and vertex labels. A primitive edge label is unchanged. For a box, decoding reads back exactly the prefix used by encoding, hence recovers both counts. The body-interface transport is along reflexivity, and induction supplies the recursive body isomorphism. For representability choose the encoder with outer prefix $(0, 0)$ and this round-trip witness. No body-consistency hypothesis occurs. □

Lean: `Linear.PaperEncoding.roundtrip`

Lean: `Linear.PaperEncoding.representable`

Lemma 45.17 (Transport of an invariant property)**.** *Any typed-graph predicate invariant under recursive isomorphism has the same truth value before and after the decoded encoding.*

Proof. Apply the stated invariance hypothesis to the proved typed round-trip isomorphism. This lemma transports an invariance theorem; it does not assert invariance for an arbitrary predicate on raw representatives. □

Lean: `Linear.PaperEncoding.roundtrip_property`

Definition 45.18 (Recursive actual-content isomorphism)**.** Primitive labels are compared literally. Two content labels are related when their derived external counts agree and their actual bodies are related by a boundary-list isomorphism with recursively related labels. A recursive graph isomorphism uses this relation for its outer edge labels. Boundary-list isomorphisms allow the body interface expressions to differ before the admissibility equations have been used.

The comparison of counts preserves the enclosing external types. It does not identify boxes of different external types merely because their contents are isomorphic as standalone graphs.

Lean: `Linear.PaperEncoding.LabelEquiv`

Lean: `Linear.PaperEncoding.DeepIso`

Theorem 45.19 (Named round trip)**.** *Every graph in the actual-content presentation is recursively isomorphic to its decoding followed by encoding, for any chosen outer prefix.*

Proof. At the outer level use normalization followed by fresh naming. Induct on depth for labels. A primitive returns unchanged. For a content label C, the new body is named with the original derived counts $b(C), f(C)$, so both external counts agree. Its output interface is transported from r to $b(C) + f(C)$. A local equality-elimination lemma first handles an arbitrary equation between these output counts, then invokes induction at the common interface. Thus neither the cast nor the recursively renamed body is assumed harmless. □

Lean: `Linear.PaperEncoding.named_roundtrip`

45.5 Faithfulness and equivalence of graph classes

Theorem 45.20 (Isomorphism transport and reflection)**.** *Encoding transports recursive isomorphisms, even with independent outer prefixes on the two sides. Decoding also transports recursive isomorphisms. Consequently two encoded typed graphs are recursively isomorphic exactly when the original typed graphs are recursively isomorphic.*

Proof. For encoding, expose the original isomorphism, translate its labels, then apply independent fresh naming on the two sides. Induct on depth to prove the label translation respects related labels. Primitive cases are literal equality; related boxes have equal counts and an induction hypothesis for their bodies. For decoding, normalize the boundary-list isomorphism at equal outer interfaces. In its box case, the relation supplies equal bound and free counts. A helper lemma eliminates these count equalities and both body-interface equations, leaving the recursive body isomorphism at a common typed interface. Apply induction there. Reflection now follows by decoding an encoded isomorphism and composing with the typed round trip on the left and its inverse on the right; the converse is encoding transport. □

Lean: `Linear.PaperEncoding.encode_iso`

Lean: `Linear.PaperEncoding.decode_iso`

Lean: `Linear.PaperEncoding.encode_iso_iff`

Definition 45.21 (Representation class)**.** A representation class is a named actual-content hypernet modulo recursive, type-preserving graph isomorphism. The equivalence relation uses existence of such an isomorphism. Encoding and decoding descend to maps between these classes and the existing typed recursive-isomorphism classes.

Equivalence laws and well-definedness. Induct on depth for reflexivity, symmetry, and transitivity of label equivalence. Primitives use the corresponding equality laws. Boxes retain or compose their derived-count equalities, and their body isomorphisms use identity, inverse, or composition with the induction hypothesis on labels. These laws supply the setoid. The quotient maps use the proved encoding and decoding transport lemmas, so changing a representative cannot change the resulting class. □

Lean: `Linear.PaperEncoding.label_refl`

Lean: `Linear.PaperEncoding.label_symm`

Lean: `Linear.PaperEncoding.label_trans`

Lean: `Linear.PaperEncoding.setoid`

Lean: `Linear.PaperEncoding.Class`

Lean: `Linear.PaperEncoding.encodeClass`

Lean: `Linear.PaperEncoding.decodeClass`

Theorem 45.22 (Mutually inverse class maps)**.** *Decoding after encoding is the identity on typed graph classes, and encoding after decoding is the identity on actual-content graph classes.*

Proof. Use quotient induction to choose a representative. The corresponding recursive round trip supplies the isomorphism witnessing equality of the two quotient classes. Reverse that equality to obtain the stated inverse law. □

Lean: `Linear.PaperEncoding.decode_encodeClass`

Lean: `Linear.PaperEncoding.encode_decodeClass`

Lemma 45.23 (Transport of class predicates and relations)**.** *For every predicate P on typed graph classes, and every binary relation R on those classes, evaluating on decoded encodings has exactly the original truth value.*

Proof. Rewrite each decoded encoding by the first inverse law. For a binary relation do this at both arguments. No special property of P or R is needed because they are already defined on classes. A raw context-membership or transition predicate must first be proved to descend to classes; this result does not supply an unproved invariance or operational locality law. □

Lean: `Linear.PaperEncoding.class_property`

Lean: `Linear.PaperEncoding.class_relation`

45.6 Compatibility with larger depth allowances

Definition 45.24 (Promotion of actual contents)**.** At depth zero promotion moves each primitive to the primitive summand. At a successor it recursively promotes each actual body while retaining every vertex name. The input prefix and output count are consequently unchanged, so the old admissibility proof gives a valid new content label. Relabelling by this alphabet map promotes a whole graph. Repeated promotion iterates it.

Lean: `Linear.PaperEncoding.raiseMap`

Lean: `Linear.PaperEncoding.raise`

Lean: `Linear.PaperEncoding.raiseBy`

Theorem 45.25 (Exact depth coherence)**.** *Encoding after promotion of a typed graph equals direct promotion of its encoding. The same equality holds for every finite number of promotions.*

Proof. Induct on depth for the one-step assertion. The base is definitional equality. At a successor isolate the two composed label maps. Fresh naming commutes with relabelling, so it suffices to prove those maps pointwise equal. Primitive labels agree immediately. For a box, subtype extensionality removes the proof field, equality of the output-count package reduces to equality of the actual bodies, and the induction hypothesis supplies that equality. This proof avoids expanding the entire naming bijection in each graph field. Induct on the number of promotions for the iterated statement, applying the one-step result at the current depth and then the induction hypothesis. □

Lean: `Linear.PaperEncoding.encode_raise`

Lean: `Linear.PaperEncoding.encode_raiseBy`

Together with the common-bound theorem for finite families in Section 44, this accounts for box contents initially presented at different finite depth allowances. The equality is for the encoded graphs themselves, not just their isomorphism classes.

45.7 Relations on the finite union of depth bounds

Definition 45.26 (Depth-independent graph class)**.** A bounded graph is a pair (n, G) where G has depth allowance n and a fixed ordered outer interface. Define global renaming as the equivalence relation generated by recursive isomorphism at a fixed bound and the pairs $(n, G) \sim (n+1, \mathsf{raise}(G))$. Its constructors give these two generators, reflexivity, symmetry, and transitivity. The global setoid uses exactly these constructors for its equivalence laws. A depth-independent graph class is a quotient by this setoid; $\mathsf{global}(G)$ denotes its canonical class. Operational transitions are not generators of this equivalence.

Lean: `Linear.PaperEncoding.Bounded`

Lean: `Linear.PaperEncoding.GlobalRenaming`

Lean: `Linear.PaperEncoding.globalSetoid`

Lean: `Linear.PaperEncoding.GlobalClass`

Lean: `Linear.PaperEncoding.global`

Lemma 45.27 (Class invariance)**.** *Recursively isomorphic graphs have equal global classes, and increasing a graph's depth allowance preserves its class.*

Proof. Apply quotient soundness to the isomorphism and promotion generators, respectively. □

Lean: `Linear.PaperEncoding.global_iso`

Lean: `Linear.PaperEncoding.global_raise`

Definition 45.28 (Interpretation)**.** Interpret a typed graph by encoding it with outer prefix $(0, 0)$ and taking its global class.

Lean: `Linear.PaperEncoding.interpret`

Proposition 45.29 (Interpretation is invariant and surjective)**.** *Interpretation preserves recursive isomorphism and depth promotion. Every global graph class is the interpretation of a typed graph at some finite bound.*

Proof. For isomorphism invariance, encode the given isomorphism and apply class invariance. For promotion invariance, use the exact encoding–promotion commuting square, followed by global promotion invariance. For surjectivity, choose a representative (n, H) by quotient induction and decode H. The named round trip gives an isomorphism from H to its decoded re-encoding; class invariance and symmetry give the required equality. □

Lean: `Linear.PaperEncoding.interpret_iso`

Lean: `Linear.PaperEncoding.interpret_raise`

Lean: `Linear.PaperEncoding.interpret_surjective`

Definition 45.30 (Relation pullback)**.** For any relation R on depth-independent classes, define its relation at bound n by $R_n(G, H) \iff R(\mathsf{interpret}(G), \mathsf{interpret}(H))$.

Lean: `Linear.PaperEncoding.relation`

Proposition 45.31 (Relation coherence)**.** *The pulled-back relation is unchanged by independent recursive renaming of its arguments. It is also unchanged by promoting both arguments once, or by any common finite number of promotions.*

Proof. Unfold the pullback and substitute the equal interpretations supplied by isomorphism invariance or promotion invariance. For an arbitrary number of promotions, induct on that number. The zero case is reflexivity; the successor case composes the induction hypothesis with the one-step equivalence at the current bound. □

Lean: `Linear.PaperEncoding.relation_iso`

Lean: `Linear.PaperEncoding.relation_raise`

Lean: `Linear.PaperEncoding.relation_raiseBy`

Definition 45.32 (Global pre-template)**.** A global pre-template consists of a relation on global classes at each outer interface, with the requirement that any related typed representatives are focus-free. This is the defining focus-freedom condition on a pre-template; it does not supply an operational safety or robustness hypothesis. Its layered interpretation takes the relation pullback at every depth and uses the given focus-freedom proof for the existing pre-template interface.

Lean: `Linear.PaperEncoding.GlobalPreTemplate`

Lean: `Linear.PaperEncoding.GlobalPreTemplate.toLayered`

Proposition 45.33 (Coherent layered interpretation)**.** *The layered interpretation of a global pre-template respects simultaneous depth promotion and independent recursive renaming.*

Proof. Apply relation coherence to its class relation. The layered interpretation unfolds to precisely that pullback. □

An arbitrary family of unrelated depth-indexed relations is not claimed to satisfy these properties. This construction supplies a coherent family from one relation on graph classes; the remaining operational bridges must still establish their own invariance and transition hypotheses.

Lean: `Linear.PaperEncoding.GlobalPreTemplate.coherent`

Lean: `Linear.PaperEncoding.GlobalPreTemplate.rename`

45.8 The repaired counterexample and the remaining theorem

Proposition 45.34 (The two-box obstruction is repaired)**.** *The previous two-box graph admits an actual-content representation without any restriction on the original graph. The two encoded box labels have different actual contents, while decoding recovers the old graph up to recursive isomorphism.*

Proof. Encode the unary variable body once with prefix $(0, 1)$ and once with prefix $(1, 0)$. If the actual body packages were equal, applying the input-name bound-coordinate function would give $0 = 1$. Thus they are distinct graphs. The first and second edges of the encoded two-box graph carry precisely these labels by construction. Pair the old negative body-consistency theorem with the new typed round trip to obtain the repaired-counterexample statement. The old counterexample remains correct for naive erasure; it is not a counterexample to freshening. □

Lean: `Linear.PaperEncodingExamples.exposedContent`

Lean: `Linear.PaperEncodingExamples.boundContent`

Lean: `Linear.PaperEncodingExamples.distinct_contents`

Lean: `Linear.PaperEncodingExamples.first_label`

Lean: `Linear.PaperEncodingExamples.second_label`

Lean: `Linear.PaperEncodingExamples.repaired_counterexample`

Further regressions check that a boxed two-hole graph encoded with prefixes $(2, 3)$ and $(7, 11)$ gives the same recursive graph class, by applying encoding transport to the identity

isomorphism. The quotient inverse law is instantiated on that graph. Promoting the repaired two-box graph by two layers commutes with encoding by the iterated depth theorem. Finally, focus freedom, hole freedom, and unique-hole contexts are all preserved and reflected by decoded encoding, using their existing recursive-isomorphism invariance theorems and the new round-trip isomorphism.

Lean: `Linear.PaperEncodingExamples.prefix_independent`

Lean: `Linear.PaperEncodingExamples.quotient_roundtrip`

Lean: `Linear.PaperEncodingExamples.promoted_repair`

Lean: `Linear.PaperEncodingExamples.context_properties`

Scope of the completed obligation. The construction supplies freshly named representatives for the typed UAM signature of the paper, using its literal graph-label equality in §3.1(3). It does not claim a representation theorem for unrelated arbitrary signatures outside the UAM discipline. Naming is a choice of representative; no vertices, edges, or operations are added. Box counts are no longer separate label fields, but can be recovered from names in the actual contents. All typed graph classes are covered, including graphs rejected by naive binder erasure.

The representation item is removed from Appendix B. Section 46 also discharges the full-UAM structural item, leaving the concrete specimen interface with its safety, robustness, and initial-lifting bridges, and the final application of Theorem 6. The class-transport lemmas preserve established hypotheses; they do not establish those remaining operational bridges. The current mathematical-effort estimate is given in Section 46.

46 Search history, stationary rewriting, and the full UAM

This section completes the first structural obligation previously listed in Appendix B. The argument follows the paper's Lemma 24, the stationary replay proof of Lemmas 27–28, and Lemma 3. The proof of Lemma 24 records an operation stack rather than repeating the paper's case analysis informally: each frame stores an actual earlier search visit with a chronological execution split. The stack is a proof device and adds no state or transition to the machine.

The stationary theorem is proved first for simultaneous expansion and then for ordinary shallow plugging. The latter applies directly to the concrete substitution relation. Independently, the determinism proof identifies a canonical reduct by erasing the old focus and relabelling the selected variable as a search focus. It proves that different decompositions of one source have isomorphic reducts. Consequently the full UAM is deterministic and refocusing exactly when its extrinsic behaviour has the corresponding property. These remain conditions on the behaviour; arbitrary compute rules do not acquire them automatically.

46.1 Chronological search visits and the operation stack

Definition 46.1 (Search visit)**.** For a run of total length n ending at coordinate p, a visit to vertex v is a pair of lengths a, b with $n = a + b$, an initial run of length a to $(?, v)$, and a suffix of length b from $(?, v)$ to p. Thus this predicate records an occurrence on the actual run, not merely reachability in another run.

Lean: `Linear.SearchHistory.Visit`

Definition 46.2 (Operation stack)**.** For a predicate P on vertices, a stack ends either at the unique input or at an argument of an operation. An argument frame carries the parent stack at the operation's source, evidence of P at that source, and membership of the current vertex in the ordered argument list. In the history invariant, P is the search-visit predicate for the current execution.

Lemma 46.3 (Stack transport and parent recovery)**.** *A pointwise implication $P \Rightarrow Q$ transports a stack. If the current vertex is an argument of operation a, the stack supplies both the parent stack and the stored predicate at a's source.*

Proof. Transport follows by induction on the stack. For parent recovery, the input case is impossible because an input cannot be an edge target. In an argument frame, both the stored operation and a target the current vertex. Target uniqueness identifies their edges; uniqueness of operation views identifies their sources. Transport the stored parent and predicate along that equality. □

Lean: `Linear.SearchHistory.Stack`

Lean: `Linear.SearchHistory.Stack.mono`

Lean: `Linear.SearchHistory.Stack.parent`

Lemma 46.4 (Extending visits)**.** *Appending one search step preserves every recorded visit, increasing its suffix length by one. A current search coordinate is a visit with empty suffix.*

Proof. Keep the initial prefix and append the new step to the old suffix. The length equation follows by addition. For a current visit, use the entire run as the prefix and the zero-step run as suffix. □

Definition 46.5 (History invariant)**.** At coordinate p after n steps, require a stack whose frame predicates are visits on that run, and a visit to the current vertex itself.

Lean: `Linear.SearchHistory.Visit.step`

Lean: `Linear.SearchHistory.Visit.current`

Lean: `Linear.SearchHistory.Invariant`

Lemma 46.6 (History preservation)**.** *Every search step preserves the history invariant. Hence every finite initial search execution has that invariant.*

Proof. First extend all stored visits by the new step. A variable trigger keeps the same stack and vertex. Entering an operation with no arguments also keeps the same vertex. Entering its first argument pushes a frame with the already recorded source visit; the new search position records itself. On return from an argument, recover the unique parent frame. Advancing to the next argument replaces the current frame and records the new search position. Finishing pops the frame and uses its stored source visit. Induct along an arbitrary suffix to propagate the invariant, adding execution lengths. At the unique input the stack is empty and the zero-step visit initializes the induction. □

Lean: `Linear.SearchHistory.invariant_step`

Lean: `Linear.SearchHistory.invariant_steps`

Lean: `Linear.SearchHistory.history`

Theorem 46.7 (Lemma 24 in coordinates)**.** *If an initial search run of length n ends at p in a mode other than search, then there exist a, b with $n = a + b$ and $b > 0$, such that the run splits at $(?, p_2)$ into a prefix of length a and suffix of length b.*

Proof. Take the current-vertex visit from the history invariant. If its suffix length were zero, its endpoints would coincide, forcing the final mode to be search. This contradicts the hypothesis. □

Lean: `Linear.SearchHistory.revisit`

46.2 Actual states, restart, and coordinate reflection

Definition 46.8 (Search restart)**.** Restarting a typed coordinate keeps its vertex and changes only its mode to search. Restarting an actual state erases its focus, restarts its extracted coordinate, and realizes that coordinate again. This is the paper's change of focus mode, up to the already proved insertion–erasure isomorphisms.

Lean: `Linear.TypedPosition.restart`

Lean: `Linear.State.restart`

Lemma 46.9 (Lifting class executions)**.** *A recursively renamed successor of a realized coordinate has a coordinate on the same original graph. The same holds for every finite execution, preserving its exact length.*

Proof. For one step, factor recursive search into literal search followed by recursive endpoint isomorphism. Apply the existing coordinate characterization of literal search and compose the endpoint isomorphisms. Quotient soundness and exactness turn that isomorphism into equality of state classes. For a finite run, induct on its length. Lift its first step, rewrite the intermediate class using the resulting equality, and lift the suffix. Prepend the lifted coordinate step. □

Lean: `Linear.TypedPosition.deepSearch_realize_iff`

Lean: `Linear.TypedPosition.class_search_iff`

Lean: `Linear.TypedPosition.class_steps_lift`

Lemma 46.10 (Restart respects representation)**.** *Restart commutes with recursive state isomorphism and therefore descends to state classes. Restarting a realized coordinate yields the class of its coordinate restart.*

Proof. The erased isomorphism carries the extracted position to the other extracted position. Resetting both modes preserves this equality. Apply the theorem that realization commutes with recursive renaming. This proves well-definedness of the quotient operation. For an inserted focus, use coordinate recovery and literal realization transport to identify the restarted class. □

Lean: `Linear.State.restart_iso`

Lean: `Linear.DeepStateClass.restart`

Lean: `Linear.TypedPosition.restart_class`

Theorem 46.11 (Lemma 24 for actual rooted states)**.** *A rooted state in non-search mode admits an initial search run through its restart class, followed by a strictly positive search suffix to its own class.*

Proof. Lift its rooted execution to coordinates on its erased graph. The endpoint isomorphism preserves mode, so apply the coordinate revisit theorem. Realize both parts of the resulting split. Restart well-definedness identifies the intermediate class with the restart of the original state. □

Lean: `Linear.State.rooted_revisit`

Lemma 46.12 (Recovering specified coordinates)**.** *Coordinate renaming composes with graph isomorphisms. An isomorphism between two realized states induces an underlying-graph isomorphism carrying their specified coordinates to each other. The root class of a realized state is the class of initial insertion on its original graph. Coordinate executions commute with recursive renaming.*

Proof. For composition, compare the vertex maps on internal, input, and output ports. For recovery, compose the inverse insertion-recovery isomorphism, the erased state isomorphism, and the second insertion-recovery isomorphism. The extracted coordinate equations and inverse laws give the required coordinate equality. The root claim follows by transporting the initial input, which ordered boundary preservation fixes. Induction on an execution transports every step using search renaming. □

Lean: `Linear.TypedPosition.deepRename_trans`

Lean: `Linear.TypedPosition.realization_coordinate_iso`

Lean: `Linear.TypedPosition.root_class`

Lean: `Linear.TypedPosition.steps_deepRename`

Theorem 46.13 (Exact coordinate characterization of rootedness)**.** *A realized state is rooted if and only if its specified coordinate is reachable by search from the unique input on its original graph.*

Proof. The reverse direction is realization of an initial coordinate run. For the forward direction, identify the root with initial insertion and lift the class run. Its endpoint may initially be another coordinate with an isomorphic realization. Recover the underlying isomorphism carrying that coordinate to the specified one, and transport the entire run. Its initial input is fixed. □

Lean: `Linear.TypedPosition.rooted_iff`

46.3 Stationary replay for simultaneous expansion

Lemma 46.14 (Uniqueness of normal runs)**.** *In a deterministic machine, two executions from the same source to normal forms have equal lengths and endpoints.*

Proof. Induct on the first execution. If it has length zero, normality forces the other run to be empty. Otherwise the other run cannot be empty; determinism identifies the first successors, and induction identifies the remaining lengths and endpoints. This uses normality, not successful finality. □

Lemma 46.15 (Rooted coordinate properties)**.** *Initial coordinate runs preserve term typing and operation reachability.*

Proof. Both hold at the input. Propagate them together along the run using the already proved one-step typing and operation-reachability invariants. □

Lean: `Linear.StationarySearch.run_properties`

Lean: `Machine.Steps.normal_unique`

Definition 46.16 (Stationary replay conditions)**.** Fix a context, an old filling family, and a designated entry vertex. Every hole filling is one-way. At each other term-typed hole input, a search focus has an immediate search step to rewrite mode. The old family preserves and reflects context search away from boundaries; the new family preserves context search. These last conditions concern corolla copies and label origins, as defined earlier, rather than an assumed rootedness result.

A rewrite coordinate is search-normal: replace its mode by its known rewrite value and apply the no-rewrite-source search theorem.

Lean: `Linear.StationarySearch.OneWayHoles`

Lean: `Linear.StationarySearch.OtherInputsTrigger`

Lean: `Linear.StationarySearch.rewrite_normal`

Lemma 46.17 (No boundary before the designated visit)**.** *Consider a rooted prefix ending at a host coordinate, a positive-length suffix to the designated search entry, and a further suffix to rewrite mode. The host coordinate is not a search-entry or backtrack-exit boundary of a hole.*

Proof. For entry at the designated vertex, the positive suffix would return to the same search coordinate. Appending the rewrite suffix then gives two runs from that coordinate to a normal form with different lengths, contradicting normal run uniqueness. At another input, the immediate-rewrite condition and search determinism force the next state to be rewrite-normal, so the run cannot continue to the designated search coordinate. For a backtrack exit, the rooted prefix supplies term typing and operation reachability. The one-way filling obstruction rules out that reachable hole output. □

Lean: `Linear.StationarySearch.prefix_away`

Theorem 46.18 (Extracting and replaying the context prefix)**.** *The prefix preceding the designated search visit is a run in the unchanged context. It can therefore be replayed after replacing the old filling family.*

Proof. Induct on the length of the suffix to the designated visit, retaining the initial prefix and the eventual rewrite suffix as induction data. At length zero, injectivity of the host map identifies the context coordinates. At a successor, the preceding lemma excludes a boundary. Search reflection extracts one context step and identifies its image with the actual successor. Extend the initial prefix by that step and apply induction to the remaining suffix. For replay, apply Lemma 24 to the initial run ending at the designated rewrite coordinate. Extract its prefix in the context, then transport that prefix through the new filling family. □

Lean: `Linear.StationarySearch.extract_prefix`

Lean: `Linear.StationarySearch.replay`

Theorem 46.19 (Rooted stationary endpoints)**.** *The actual search-focused replacement is rooted whenever the old rewrite-focused state is rooted and the stationary replay conditions hold. Independent recursive renaming of both endpoints preserves this conclusion.*

Proof. Reflect rootedness to the specified old coordinate, replay its prefix, and realize the resulting initial coordinate run on the replacement. For renamed endpoints, quotient soundness identifies each endpoint class with its realized representative; transport the rooted predicate along these equalities. □

Lean: `Linear.StationarySearch.rooted`

Lean: `Linear.StationarySearch.rooted_iso`

46.4 Moving an inserted focus through ordinary plugging

Definition 46.20 (Reassociation and focused fragments)**.** The reassociation bijection sends $A + (B + D)$ to $(A + B) + D$ by preserving each summand; its inverse reverses this operation, with both inverse laws by cases. The fragment inserts a focus at a chosen input of a replacement graph. The whole construction inserts it at the corresponding port after plugging.

Lean: `Linear.PlugFocusInsertion.reassoc`

Lean: `Linear.PlugFocusInsertion.fragment`

Lean: `Linear.PlugFocusInsertion.whole`

Lemma 46.21 (Port identities)**.** *Reassociation carries retained host ports to their focused whole-graph ports, and inserted old ports to the same whole-graph ports. For a non-output port, it also commutes with splitting at the selected input.*

Proof. The first two claims are cases on the port constructors. For splitting, separate the selected input from all other ports. The selected input goes to the fresh singleton on both sides. For another non-output port, injectivity of insertion on source-side ports excludes equality with the selected whole-graph port; both splits then reduce to the old-port identity. This restriction avoids assuming that the complete insertion map is injective for self-looping holes. □

Lean: `Linear.PlugFocusInsertion.host`

Lean: `Linear.PlugFocusInsertion.insert_host`

Lean: `Linear.PlugFocusInsertion.insert_split`

Theorem 46.22 (Focus insertion commutes with plugging)**.** *Plugging the input-focused replacement is isomorphic to inserting the focus at its attached port in the filled whole graph.*

Proof. Use reassociation for both the internal-vertex and edge carriers. Labels are unchanged. A retained context source cannot meet the selected input port: source uniqueness would identify its edge with the removed hole. Its source list therefore uses the host identity. Sources of inserted edges use the split identity; their ports are non-outputs by well-formedness. Targets use the old-port identities. The new focus's singleton source and target satisfy the same equations directly. □

Lean: `Linear.PlugFocusInsertion.iso`

46.5 Reflecting search through one shallow filling

Definition 46.23 (Host coordinates and boundary)**.** A host coordinate is mapped through the retained-vertex injection. Its boundary predicate means either search mode at a source of the selected hole, or backtrack mode at a target of that hole.

Lemma 46.24 (Reading back retained operations)**.** *An operation in the filled graph whose edge is retained has exactly the source and ordered arguments of an original context operation, mapped by the host injection.*

Proof. Its label supplies an operation view on the original edge. Plug that view back into the whole graph; it has the same edge as the given view. Uniqueness of operation data gives the source and argument equations. □

Lean: `Linear.PlugSearchReflection.position`

Lean: `Linear.PlugSearchReflection.Boundary`

Lean: `Linear.PlugSearchReflection.operation_readback`

Lemma 46.25 (Inserted incidence meets the boundary)**.** *An inserted source meeting a host vertex meets an input of the replacement; an inserted target meeting a host vertex meets an output.*

Proof. Unpack membership in the mapped incidence list. Source ports cannot be outputs, and target ports cannot be inputs. The existing insertion/host intersection lemmas identify the corresponding boundary index, whose attachment belongs to the hole's original source or target list. □

Lean: `Linear.PlugSearchReflection.source_boundary`

Lean: `Linear.PlugSearchReflection.target_boundary`

Theorem 46.26 (Ordinary search reflection)**.** *Away from the selected hole's boundary, every search step of the filled graph comes from a context step, with exactly the corresponding successor coordinate.*

Proof. Inspect the variable-trigger, operation-entry, and operation-return rules. For a retained edge, recover its label and singleton source, or read back its operation view. In the return case, injectivity of the host map recovers the indexed argument lookup. These data construct a context step. Its plugged step and the original step have the same successor by search determinism. An inserted interacting edge instead forces the appropriate boundary case by the preceding incidence lemmas, contrary to the hypothesis. □

Lean: `Linear.PlugSearchReflection.reflect`

46.6 The stationary criterion for ordinary contexts

Lemma 46.27 (Shallow transport and one-way exclusion)**.** *Every context search run survives filling its hole. A one-way filling rules out an operation-reachable term vertex at a hole output.*

Proof. Transport the run by induction, using preservation of a context search step under plugging. For exclusion, write the hole target as an indexed output port, extract its operation path inside the replacement, transfer the term type through the boundary signature, and apply one-wayness. □

The other-input condition for one hole requires immediate rewrite at every term-typed source distinct from the designated source.

Lean: `Linear.StationaryPlug.steps`

Lean: `Linear.StationaryPlug.oneWay_target`

Lean: `Linear.StationaryPlug.OtherInputsTrigger`

Theorem 46.28 (Ordinary stationary prefix replay)**.** *For a one-way filling satisfying the other-input condition, the prefix leading to the designated search position before a rewrite is an unchanged-context run. It replays with any new filling of the same interface.*

Proof. Use the three boundary exclusions already described for simultaneous expansion: normal-run uniqueness at the same input, immediate rewrite at another input, and one-way exclusion at an output. The single-hole reflection theorem then extracts each context step by induction on the remaining prefix. Lemma 24 supplies the chosen search visit, and ordinary plugging transports the extracted run to the new filling. No proof of rootedness of the reduct is included among these assumptions. □

Lean: `Linear.StationaryPlug.prefix_away`

Lean: `Linear.StationaryPlug.extract_prefix`

Lean: `Linear.StationaryPlug.replay`

Lemma 46.29 (Closing a simple context)**.** *A focus-free, hole-free replacement plugged into a focus-free simple context produces a focus-free, hole-free graph.*

Proof. Focus freedom is checked separately on retained and inserted edges. A hole occurrence either comes from the replacement, contradicting its hole freedom, or from a retained context edge. Simplicity forces that edge to be the selected hole, contradicting retention. □

Theorem 46.30 (Stationary rewriting preserves rootedness)**.** *Let C be a focus-free simple context and H a focus-free, hole-free one-way filling. Fix an input i of term type and impose the immediate-rewrite condition at other term inputs. Replacing $C[\mathsf{rewrite};_i H]$ by $C[?;_i K]$, for any focus-free, hole-free K of the same interface, preserves rootedness, allowing arbitrary literal endpoint names.*

Proof. The preceding lemma certifies both erased whole graphs. Commutation of focus insertion with plugging identifies the actual endpoints with their specified whole-graph coordinates. Reflect the source's rootedness to a coordinate run, apply ordinary stationary replay, and realize the resulting target run. Transport both endpoint predicates through their focus-preserving isomorphisms. □

Lean: `Linear.StationaryPlug.filled_conditions`

Lean: `Linear.StationaryPlug.rooted`

46.7 Rootedness of the concrete substitution rule

Lemma 46.31 (Blocked inputs imply one-wayness)**.** *If no operation edge can leave a term input, the graph is one-way.*

Proof. Induct on the length of a reverse operation trace ending at that input. An empty trace starts at the input. For a nonempty trace, induction identifies the source of its last relevant edge with the input, contradicting the blocked operation-source condition. Thus an operation path from that input to a boundary output would identify an input with an output, which is impossible. □

Corollary 46.32 (The variable prefix is one-way)**.** *Prefixing any one-input graph by the unary variable edge produces a one-way graph.*

Proof. There is only one input index. The new variable edge owns that input's source incidence. Any operation leaving it would be the same edge by source uniqueness, contradicting its variable label. □

Lean: `Linear.oneWay_of_blocked_inputs`

Lean: `Linear.Substitution.prefix_oneWay`

Lemma 46.33 (Interface and single-input facts)**.** *A one-input hole has no non-designated input, so its other-input condition is vacuous. Reindexing equal interfaces commutes with initial focus insertion and preserves one-wayness and hole freedom.*

Proof. For the first statement, recover the boundary index of a source occurrence; the input signature has length one, so that index is the designated one. For interface transport, eliminate the two interface equalities. The claims reduce to reflexive reboundary transport; focus insertion uses congruence of the graph-valued constructor. The same equality elimination proves preservation of one-wayness and closure. □

Lean: `Linear.Substitution.single_input`

Lean: `Linear.Substitution.transport_insert`

Lean: `Linear.Substitution.oneWay_transport`

Lean: `Linear.Substitution.holeFree_transport`

Theorem 46.34 (Concrete substitution preserves rootedness)**.** *Every transition of the concrete substitution relation preserves rootedness.*

Proof. Unpack its simple context, bound graph G_S, and endpoint isomorphisms. Take the old filling to be the variable-prefixed G_S, and the new filling to be G_S, both transported to the hole's interface. The old filling is one-way by the variable-prefix corollary, and the single-input lemma supplies the other-input clause. Focus freedom and closure follow from the existing prefix theorems and interface transport. Transported focus insertion identifies the two original fragments with the stationary fragments. Apply the ordinary stationary rootedness theorem to the given endpoints. □

Relation to the paper's stability assumption. Lemma 3 of the paper assumes that G_S is stable for its refocusing conclusion. For the unary linear substitution rule formalized here, the argument above is stronger: the variable prefix itself is one-way, independently of G_S, and there are no other inputs to check. Stability is neither added to the transition definition nor replaced by an assumed refocusing field. This observation concerns this rule; it is not a statement about the copying UAM.

Lean: `Linear.State.substitution_rooted`

46.8 Determinism despite different context decompositions

Lemma 46.35 (Focus preservation from recursive graph isomorphism)**.** *A recursive graph isomorphism between uniquely focussed graphs preserves their distinguished focuses and modes.*

Proof. Primitive-label preservation carries the first focus label to the image edge. Uniqueness of the second focus identifies that image edge and its mode. □

Definition 46.36 (Resuming a variable)**.** On a graph with a selected variable edge, change only that edge's label to a search focus. The old and new signatures are both one term input and one term output, so the graph remains well-typed.

Lean: `Linear.Focussed.deepIsoOfGraph`

Lean: `Linear.Substitution.resumeVariable`

Lemma 46.37 (Resumption respects isomorphism)**.** *An isomorphism carrying one selected variable to another induces an isomorphism between their resumed graphs.*

Proof. Keep the vertex and edge bijections and all incidence equations. At the selected edge both new labels are the same search focus. At another edge, injectivity prevents its image being the selected target edge, so the original recursive label relation still applies. □

Definition 46.38 (Substitution presentation)**.** A presentation consists of the erased base graph, focus freedom and closure, a rewrite-mode coordinate, its adjacent variable edge and singleton source equation, an isomorphism from realizing that coordinate to the actual source, and an isomorphism from the variable-resumed base to the actual target. These are proved data extracted from a substitution transition.

Lean: `Linear.Substitution.resumeVariable_iso`

Lean: `Linear.Substitution.Presentation`

Theorem 46.39 (Uniqueness from two presentations)**.** *Two presentations of the same source yield recursively isomorphic focussed targets.*

Proof. Compose their source isomorphisms and recover the underlying-graph isomorphism carrying the two selected coordinates to each other. The first variable's mapped singleton source is therefore the source of the second variable. Source uniqueness identifies the two edges. Resumption respects this isomorphism; compose it with the two target isomorphisms. The resulting graph isomorphism preserves focus by uniqueness. □

Lean: `Linear.Substitution.Presentation.deterministic`

Lemma 46.40 (Presentations are constructed from substitution)**.** *Relabelling the selected variable in a plugged variable prefix gives a graph isomorphic to the ordinary reduct. Every substitution transition consequently supplies a presentation.*

Proof. For the first claim use identity bijections: the two graphs have exactly the same carriers, vertex labels, and incidence lists. Only the selected variable label changes to a search focus. Inspect retained context edges, old filling edges, and the added singleton separately. Reboundary transport merely maps the same ports on both sides. For extraction, use the plugged variable prefix as base graph, certify its focus freedom and closure, and choose its new variable edge. Focus-insertion commutation and interface transport give the source isomorphism. The relabelling lemma gives the target isomorphism. □

Theorem 46.41 (Substitution determinism)**.** *Two concrete substitutions from the same state have targets isomorphic at all box depths, preserving their focuses. Their contexts and fragment interfaces need not coincide.*

Proof. Extract both presentations and apply their uniqueness theorem. □

Lean: `Linear.Substitution.resume_prefix_iso`

Lean: `Linear.Substitution.presentation`

Lean: `Linear.State.substitution_deterministic`

46.9 The full UAM structural theorem

Definition 46.42 (Determinism modulo isomorphism)**.** A relation on actual named states is deterministic when two successors of the same source are related by a recursive focus-preserving isomorphism. Raw carrier names are not required to be equal.

Definition 46.43 (Refocusing of actual-state transitions)**.** A relation is refocusing when it sends every rooted source to a rooted target, using the rooted predicate already defined on recursive state classes.

Lean: `Linear.DeterministicModulo`

Lean: `Linear.RefocusingStates`

Theorem 46.44 (Full-UAM determinism)**.** *The full UAM is deterministic modulo recursive isomorphism if and only if its extrinsic behaviour is deterministic in that sense.*

Proof. For sufficiency, inspect the two transitions. Two search steps agree by search determinism. Two substitutions agree by the new substitution theorem. Two extrinsic steps agree by the supplied behaviour condition. Search and rewrite cannot coexist at a source because their focus modes are exclusive. A substitution and computation cannot coexist because source uniqueness would identify a variable with an active operation. These exclusions discharge all mixed cases. Necessity follows because every behaviour step is a UAM step. □

Lean: `Linear.UAM.deterministic`

Lean: `Linear.UAM.deterministic_iff`

Theorem 46.45 (Full-UAM refocusing)**.** *Intrinsic transitions preserve rootedness. The full UAM is refocusing if and only if its extrinsic behaviour preserves rootedness.*

Proof. Search preserves rootedness by appending its step to the rooted run. Substitution preserves it by the stationary argument. These are the two intrinsic cases. The full machine adds only extrinsic transitions, handled by the behaviour hypothesis. Conversely, restrict a full refocusing law to those extrinsic transitions. □

Lean: `Linear.UAM.intrinsic_refocusing`

Lean: `Linear.UAM.refocusing`

Lean: `Linear.UAM.refocusing_iff`

Corollary 46.46 (Rooted execution invariant)**.** *Under the behaviour refocusing condition, every finite full-UAM execution preserves rootedness. Every execution from an initial state therefore ends in a rooted state. The empty-behaviour machine is both deterministic and refocusing without additional assumptions.*

Proof. Induct on the full execution and apply one-step refocusing at each successor. Initial states are rooted by the zero-step initial run. For empty behaviour, both required behaviour properties follow by elimination of its false step relation; apply the two structural theorems. □

Lean: Linear.UAM.rooted_execution

Lean: Linear.UAM.initial_execution_rooted

Lean: Linear.UAM.empty_structural

46.10 Regression executions and the remaining work

The variable-prefix example has an actual one-step initial search run to its rewrite-focused substitution source, obtained by the variable trigger rule. Realization proves that source rooted. The concrete substitution theorem then proves its reduct rooted, and the determinism theorem identifies the reduct class of every alternative substitution decomposition. Applying Lemma 24 to the rooted source produces a strictly positive restart suffix; this exercises the theorem on an actual execution rather than an empty hypothesis.

Lean: Linear.UAMStructureExamples.source_rooted

Lean: Linear.UAMStructureExamples.substitution_rooted

Lean: Linear.UAMStructureExamples.substitution_unique

Lean: Linear.UAMStructureExamples.source_revisit

The earlier singleton extrinsic behaviour has a genuine compute transition. Its two successor witnesses identify the same target, proving determinism. That target is initial, so it is rooted independently of the source, proving refocusing. Applying the full-UAM theorems supplies both structural properties for this nonempty behaviour.

Lean: Linear.UAMStructureExamples.behaviour_deterministic

Lean: Linear.UAMStructureExamples.behaviour_refocusing

Lean: Linear.UAMStructureExamples.full_structural

Completed obligation and honest scope. These results prove actual transition properties. They do not assert that raw named-state successors are literally equal, or that arbitrary behaviour is deterministic or refocusing. Section 47 subsequently constructs the named-state specimen laws and safety/robustness bridges and updates the effort estimate. Appendix B lists only the remaining observation, state-class, and final integration obligations.

47 Relocating specimens and constructing the concrete interface

The context-step field of the abstract specimen interface is now constructed from the concrete graph semantics. The missing step after synchronized search was to rebuild an allowed specimen at the successor coordinate. This section supplies that construction, with recursive fillings and actual endpoint transitions, and proves the safety and robustness bridges.

Allowed-context membership must respect literal-label graph isomorphism. Output closure and refocusing of the extrinsic behaviour remain the paper's semantic hypotheses. No context-step law, successor-specimen existence, or safety bridge is supplied as a new hypothesis.

The interface in this section is on named states. The abstract simulation theorem requires literal determinism, whereas the concrete structural theorem proves determinism modulo recursive isomorphism. Section 48 subsequently resolves this distinction with a deterministic class machine and its own concrete interface. General initial observation lifting remains open.

47.1 Exact hole inventories under focus movement

Lemma 47.1 (A focus has no hole occurrence)**.** *Every occurrence address in a focus label observes no hole.*

Proof. At every depth the focus label is primitive. Its shallow occurrence is not a hole and its child-occurrence type is empty. Case analysis on the depth and address proves the assertion. □

Lean: `focus_fiber_no_hole`

Lemma 47.2 (Insertion preserves hole occurrences and uniqueness)**.** *Every hole occurrence after focus insertion comes from a retained old edge with exactly the same label-local address. Consequently insertion preserves and reflects the existence of every hole and the unique-hole property.*

Proof. An occurrence has an outer edge and a label-local address. Split the edge into an old edge or the fresh singleton focus. The first case retains the address; the second contradicts the preceding lemma. The converse inclusion embeds old edges into the left summand. To preserve uniqueness, pull two equal-labelled hole occurrences back to the old graph, use its uniqueness, and map the equal addresses forward. Reflection follows from the injectivity of the old-edge map. □

Lean: `hole_occurrence`

Lean: `hasHole_iff`

Lean: `uniqueHoles_iff`

Lemma 47.3 (Erasure preserves holes and uniqueness)**.** *Erasing a focussed context preserves every hole in its full recursive inventory and preserves uniqueness of hole occurrences.*

Proof. Retained edges are a subtype of old edges with identical labels. This injective embedding reflects uniqueness and maps every surviving occurrence back. In the other direction an old hole cannot lie on the erased focus: transporting its address along the focus-label equality would contradict the no-hole lemma. Thus its edge belongs to the retained subtype, with its entire address unchanged. □

Lean: `underlying_uniqueHoles`

Lean: `underlying_hasHole_iff`

Definition 47.4 (Isomorphism-invariant allowed contexts)**.** A predicate of allowed erased contexts is isomorphism-invariant if a literal-label graph isomorphism transports membership from its source to its target. Applying the law to the inverse also reflects membership. This is invariance under names, not closure under rewriting.

Lean: `IsoClosedContexts`

Definition 47.5 (Context at a new coordinate)**.** For specimen S and typed coordinate q on its erased context, erase the old focus and insert the mode of q at its term vertex. The resulting graph is a unique-hole context, with the newly inserted edge as its distinguished focus.

Verification. Erasure preserves uniqueness and insertion reflects it back to a focussed context. The old erasure is focus-free, so insertion supplies the required unique focus. The insertion–erasure round trip identifies the new erased context with the old one. Applying allowed-membership invariance to the inverse of that isomorphism proves the new context allowed. Composing the two inventory laws proves that exactly the original holes remain, including those in boxes. □

Lean: `contextAt`

Lean: `focusAt`

Lean: `contextAt_underlying_iso`

Lean: `contextAt_allowed`

Lean: `contextAt_hasHole_iff`

Lemma 47.6 (Entering and remoteness after relocation)**.** *The new focus enters a shallow hole precisely when the new coordinate belongs to that hole's old source list. A remote coordinate therefore gives a remote focussed context.*

Proof. A hole edge cannot be the newly inserted focus. On a retained edge, membership of the fresh target in the split source list supplies an old source port. Collapse the equality to identify it with the chosen coordinate. Conversely, a source occurrence of that coordinate is split to the fresh target. For remoteness, either the mode is not rewrite, or the coordinate avoids every hole source; the equivalence just proved excludes entering in the second case. □

Lean: `focusAt_entering_iff`

Lean: `focusAt_remote`

47.2 Insertion commutes with simultaneous filling

Definition 47.7 (Extending the filling across a focus)**.** Insert the focus into a focus-free context C. Extend a filling family H by keeping $H(e)$ at every old edge and assigning the new focus its one-edge corolla. This new corolla has no internal vertices.

Lean: `inserted`

Lean: `family`

Definition 47.8 (Carrier reassociation)**.** The vertex bijection sends retained context vertices, filling vertices, and the fresh focus target to the corresponding three components of the graph obtained by filling first and inserting afterwards. An apparent fourth component, internal vertices of the focus corolla, is empty. The edge bijection sends old filling edges to their original expansion pairs and the new corolla's sole edge to the inserted focus edge.

Bijection laws. The inverse maps return each component to its original summand. Each inverse law is a case split. The empty component is eliminated; all indices in the singleton focus carrier equal zero. □

Lean: `vertices`

Lean: `edges`

Lemma 47.9 (Port compatibility)**.** *Retained host ports commute with reassociation, as do split host ports. A filling source port maps to the split expanded port; a filling target port maps to the retained expanded port.*

Proof. The host identity is a case split on internal, input, and output ports. For a split host port, test equality with the chosen coordinate. Host injectivity preserves this test through expansion. For a filling source, an internal port cannot equal a retained host port, an input is attached to the corresponding split host source, and an output is excluded by source well-formedness. The target proof is dual: an internal port is retained, an output is attached to the old target, and an input is impossible. The proof uses only incidence-side injectivity and therefore does not exclude self-looping attachments by assuming a globally injective attachment map. □

Lean: `host`

Lean: `split`

Lean: `source_port`

Lean: `target_port`

Theorem 47.10 (Commutation of focus insertion and filling)**.** *Expanding the inserted context by the extended family is isomorphic to inserting the same focus at the corresponding host coordinate of $C[H]$.*

Proof. Use the two carrier bijections. Internal and edge labels agree on each summand. For an old filling edge, map its source and target lists twice and apply the corresponding port identities pointwise. For the fresh corolla, its source and target lists are singletons; they map to the retained chosen port and the fresh target respectively. These are precisely the inserted focus's incidence lists. Thus all fields of a graph isomorphism are proved. □

Lean: `iso`

47.3 The successor specimen and full-machine traversal

Lemma 47.11 (Extending recursive certificates)**.** *Every old retained, filled-hole, or boxed filling certificate extends to the inserted context. The new focus is a retained corolla on both sides. If the old expansion is hole-free, so is the new expansion.*

Proof. Inspect an old edge certificate. Retained copies keep the same edge, label, and incidence proofs; filled holes keep their related replacements; boxed edges keep their recursive body-filling certificate and rebuilt box copies. The new edge has the explicit singleton-corolla certificate. Closure follows by transporting insertion's hole-free result through the commuting isomorphism. □

Lean: `edge_lift`

Lean: `edges_lift`

Lean: `closed`

Definition 47.12 (Relocated specimen)**.** The specimen at q consists of the newly focussed context, its proved allowed membership, and the two extended families obtained from the old erased families. The extended certificates and closure proofs supply all remaining specimen fields. No new relation pair is introduced.

Lemma 47.13 (Endpoint identification and remoteness)**.** *Its two actual endpoints are respectively isomorphic, preserving focus, to the previously constructed left and right coordinate realizations. If q is remote, so is the relocated specimen.*

Proof. Use insertion–expansion commutation on each side. A graph isomorphism between uniquely focussed graphs preserves the focus by its label and uniqueness, yielding the focussed isomorphisms. The remote assertion is the earlier coordinate-to-context result applied to the constructed focus. □

Lean: `relocate`

Lean: `relocate_left_iso`

Lean: `relocate_right_iso`

Lean: `relocate_remote`

Theorem 47.14 (Allowed successor specimen for search)**.** *For an output-closed template, a settled specimen with jointly rooted endpoints and a search successor on either side admits a remote allowed specimen reached by search on both sides. Recursive renaming of the originally supplied target is permitted.*

Proof. The checked traversal theorem supplies a synchronized typed coordinate and proves it remote. Relocate the specimen there. Compose each search endpoint with the inverse of its relocation isomorphism to obtain the two transitions into the new specimen. □

Lean: `settled_successor`

Lemma 47.15 (Settled full-machine steps are search)**.** *A full-UAM step from a settled specimen's endpoint must be search.*

Proof. Split the transition into search or rewrite. The latter requires a rewrite source, contradicting either backtrack mode or search mode in the settled hypothesis. No locality or determinism assumption on the behaviour is used. □

Theorem 47.16 (Full-UAM context-step law)**.** *Under allowed-membership invariance and output closure, a jointly rooted settled specimen with a full-UAM successor on either side has a remote allowed successor specimen reached by full-UAM steps on both sides.*

Proof. Joint rootedness and output closure exclude the exiting boundary; settledness excludes the entering search boundary. Reduce whichever full-machine step is available to search, reflect it to the erased context, and obtain a coordinate q. Its search-step theorem proves it remote. Build the relocated specimen. The literal search-reflection equivalences, composed with the two literal relocation isomorphisms, give actual search steps into its endpoints. Inject these into the full-UAM relation. The conclusion uses actual named states, not a substituted assumption about reachability or a quotient-only step. □

Lean: `settled_step_search`

Lean: `full_context_step`

Theorem 47.17 (Constructed initial specimens)**.** *Relocating any allowed specimen to the unique input produces a specimen with two initial, hence rooted, endpoints and search mode. The old specimen need not have rooted endpoints.*

Proof. The inserted focus has search mode and an input source. The expansion boundary law transfers initiality to the left endpoint, and the specimen boundary agreement transfers it to the right. Zero-step initial runs establish both rootedness claims. Search mode establishes remoteness. This constructs an initial witness from an existing specimen; it does not yet construct the observation/context witnesses for arbitrary related fragments. □

Lean: `relocate_initial`

Lean: `initial_witness`

47.4 The concrete abstract-interface instance and its bridges

Definition 47.18 (Constructed specimen system)**.** For behaviour B, use actual recursive specimens as the abstract specimen type and their actual filled states as endpoints. Use the context's focus mode and entering predicate. Rootedness is the existing concrete predicate. Supply rooted-step preservation from the full-UAM structural theorem, final agreement from the boundary theorem, and context traversal from the newly proved full-machine context-step theorem.

The parameters are an isomorphism-invariant allowed-context class, output closure of the template at the chosen depth, and refocusing of the extrinsic behaviour. All operational interface fields are now derived.

Lean: `system`

Lemma 47.19 (Quasi-specimen bridge)**.** *A concrete non-rewrite quasi-specimen supplies the abstract rooted quasi-specimen predicate for the constructed system.*

Proof. After receiving both endpoint-rootedness hypotheses, apply the existing rooted-witness theorem. Its witness has non-rewrite mode, rooted filled endpoints, and exactly the two required comparison relations. The concrete unconditional witness and mode agreement have not been discarded from the original definition; this theorem derives the consequence used by the interface. □

Lean: `quasi_bridge`

Theorem 47.20 (Input-safety bridge)**.** *The unrestricted concrete input-safety predicate implies the abstract input-safety law of the constructed specimen system.*

Proof. Apply concrete safety at the entering search specimen. Its stuck alternative is already the first abstract alternative. Its direct alternative has the same specimen, settled predicate, exact run lengths, and cost comparison. For the third alternative retain both runs and their cost comparison and apply the quasi-specimen bridge. In particular the equivalence relations in this alternative are preserved, not weakened prematurely to refinement. □

Lean: `inputSafe_bridge`

Theorem 47.21 (Robustness bridge)**.** *Concrete robustness implies the abstract robustness law of the constructed system.*

Proof. The abstract source has rewrite mode, so its full-machine transition cannot be search and must be a rewrite. Apply the concrete rooted robustness theorem: the unrooted alternative contradicts the two given rootedness hypotheses. The stuck-tail case is already the abstract case. In the matching case keep the tail lengths, right run, and cost comparison and apply the quasi-specimen bridge with the supplied refinement relations. □

Lemma 47.22 (Exact concrete lifting)**.** *The abstract lifting relation for this system is precisely existence of a remote concrete specimen with rooted endpoints equal to the given states.*

Proof. Unfold the constructed system and the two definitions. The predicates coincide definitionally. □

Lean: `robust_bridge`

Lean: `lifting_iff`

47.5 Nested-hole regression and remaining obligations

Use the earlier specimen whose sole hole lies strictly inside a box and whose outer focus searches at the consumer. All contexts are allowed in this regression, making membership invariance immediate. Relocate it to the consumer's actual backtrack successor. The inventory theorem retains the nested hole, the context carries a uniqueness proof, and both filled endpoints are closed. Applying the two literal search equivalences to the known consumer step gives actual transitions into the rebuilt specimen on both sides. Its coordinate is remote, so the rebuilt specimen is remote.

Lean: `allowed`

Lean: `moved`

Lean: `nested_hole`

Lean: `unique`

Lean: `closed`

Lean: `paired_steps`

Lean: `remote`

The same example was previously proved unrooted. Moving its focus to the input now supplies a genuine initial, rooted specimen, exercising construction without assuming the old source rooted. Finally, instantiate the concrete system with the empty behaviour: its refocusing condition is vacuous, output closure is the earlier template theorem, and the nested specimen witnesses that the constructed abstract specimen type is nonempty.

Lean: `initial_rooted`

Lean: `concreteSystem`

Lean: `interface_nonempty`

What this milestone closes. The synchronized-coordinate packaging, unique-hole preservation, isomorphism-invariant membership transport, full-UAM context-step field, and concrete safety and robustness bridges are complete. Appendix B retains only the unfinished portions. Initial observation lifting still needs its general context and endpoint witnesses. Section 48 supplies the deterministic class machine, class safety/robustness bridges, and forward counting simulation, and updates the effort estimate. No raw-state determinism assumption is substituted for the paper's determinism modulo isomorphism.

48 The full class machine and concrete forward simulation

The determinism required by the abstract counting-simulation theorem is literal equality of successors. Named hypernets instead have unique successors up to recursive focus-preserving isomorphism. This section constructs the full UAM on those isomorphism classes, proves its literal determinism, constructs its concrete specimen interface, and obtains the concrete forward counting simulation and quantitative refinement of remote rooted specimens.

The behaviour is required to respect recursive endpoint isomorphism when using its named-state determinism hypothesis. A closure construction below shows that this invariance concerns representative names: closing compute rules under renaming leaves the entire class transition relation unchanged. No deterministic transition on raw carrier names is assumed.

Safety and robustness here are evaluated directly on the full class machine, with all the concrete clauses retained. We do not infer class safety merely from the previously defined named-state safety: projection of finite runs alone would not justify preservation of stuckness or refinement. General initial observation lifting and the converse integration remain separate.

48.1 Comparing steps from isomorphic sources

Definition 48.1 (Renaming a presentation's source)**.** A substitution presentation can replace its source by any recursively isomorphic state while leaving its base graph, selected variable, and target unchanged. Compose the presentation's source isomorphism with the given one.

Theorem 48.2 (Substitution uniqueness across source classes)**.** *Two substitution steps whose sources are recursively isomorphic have recursively isomorphic targets.*

Proof. Extract a presentation of each substitution. Rename the first presentation's source to the second source, then apply the already proved uniqueness theorem for presentations of a common source. This comparison does not assert a new literal substitution transition at the renamed source; the presentation is sufficient to identify the two actual reducts. □

Lean: `renameSource`

Lean: `substitution_iso_deterministic`

Lemma 48.3 (Substitution and compute remain exclusive under renaming)**.** *A substitution source cannot be recursively isomorphic to a compute source.*

Proof. The substitution exposes a variable with singleton source at the focus target. The source isomorphism transports its primitive variable label, ordered source list, and focus target to the other graph. Computation exposes an active operation with that same singleton source. Source uniqueness identifies the two edges, contradicting their different primitive labels. □

Lean: `substitution_compute_iso_exclusive`

Definition 48.4 (Behaviour invariance under names)**.** An extrinsic behaviour respects isomorphism if independently renaming either endpoint recursively preserves every behaviour transition. Inverse isomorphisms also give reflection. This condition is distinct from determinism, refocusing, and locality of a compute rule.

Lean: `RespectsIso`

Theorem 48.5 (Full-UAM uniqueness across isomorphic sources)**.** *If the behaviour respects isomorphism and is deterministic modulo recursive isomorphism at each common named source, full-UAM steps from recursively isomorphic sources have recursively isomorphic targets.*

Proof. Inspect the two transition families. For two search steps, transport the first source recursively and use search determinism. For two substitutions, use uniqueness across source classes. For two extrinsic steps, transport the first source while fixing its target and apply behaviour determinism at the now common source. Search versus rewrite is impossible because source modes are preserved by isomorphism. Substitution versus compute is excluded by the preceding incidence lemma, in either direction. These cases exhaust the UAM. □

Lean: `iso_deterministic`

48.2 The full UAM on recursive state classes

Definition 48.6 (Full class transition)**.** A transition from class s to class t exists when there are named states a, b representing those respective classes and an actual full-UAM step $a \to b$. No global representative is selected. The original named transition projects by taking its own endpoints as witnesses.

Lean: `Step`

Lean: `project`

Theorem 48.7 (Literal class determinism)**.** *Under the preceding behaviour hypotheses, two full transitions from the same class have equal target classes.*

Proof. Extract the two actual source and target pairs. Equality of their source classes yields a recursive focus-preserving source isomorphism by quotient exactness. The full-UAM comparison theorem supplies a target isomorphism. Quotient soundness identifies the two target classes, and their original endpoint equalities finish the proof. □

Lemma 48.8 (Finality and the class machine)**.** *A final class has no full transition. The full class machine uses this transition relation and the existing class-invariant boundary finality.*

Proof. For a purported successor, extract its representative step. Transfer finality to that source representative and apply the concrete full-UAM final-normality theorem. This proves the machine's final-normality field. □

Lean: `deterministic`

Lean: `final_normal`

Lean: `machine`

Lemma 48.9 (Rootedness and exact run projection)**.** *Refocusing of the extrinsic behaviour makes every full class step preserve rootedness. Every named finite run projects to a class run of exactly the same length. Every class run preserves rootedness.*

Proof. For a single class step, use its representative equalities to transfer the rooted premise, apply concrete full-UAM refocusing, and transfer the result back. For run projection, induct on the named run, mapping each transition and retaining its successor count. For the class-run invariant, induct on the run and apply the single-step rootedness theorem. The zero-step cases are identities. □

Lean: `rooted_step`

Lean: `project_steps`

Lean: `rooted_steps`

Lemma 48.10 (Recursive search inclusion and fixed-source reflection)**.** *Recursive search gives a full class transition. Conversely, a class transition from the class of a specified non-rewrite state admits a recursively renamed search step from that very state to a representative of the target class.*

Proof. For inclusion, factor recursive search into literal search and a recursive target isomorphism. Project the literal transition and identify its target class with the required one. For reflection, extract the class transition's actual representatives and obtain a source isomorphism by quotient exactness. A rewrite representative would have rewrite mode and hence contradict the specified source mode. The step is therefore search. Transport its source along the inverse isomorphism, retaining its actual target and target-class equality. This reflection theorem concerns non-rewrite sources; it does not claim that arbitrary rewrite runs lift to every raw representative. □

Lean: `search`

Lean: `search_of_nonrewrite`

48.3 The class-valued specimen interface

Definition 48.11 (Class-valued specimen system)**.** Retain the actual allowed recursive specimens, their mode, and their entering predicate, but take the classes of their filled states as endpoints. Use the existing rooted predicate on classes. Supply rooted-step preservation from the class refocusing theorem and final agreement from the concrete boundary law.

Construction of context traversal. For a settled specimen, either mode alternative excludes rewrite. Reflect an available class transition from either endpoint to recursive search at its specified actual representative. The concrete successor-specimen theorem constructs a remote allowed specimen with synchronized search transitions. Include these in the full class relation. This supplies the final interface field for arbitrary source representatives, without postulating a class locality law. □

Lean: `system`

Lemma 48.12 (Concrete class lifting)**.** *Class lifting is exactly existence of a remote concrete specimen with rooted endpoints and the specified endpoint classes. Named lifting projects into class lifting. The previously constructed initial relocation belongs to class lifting.*

Proof. The first statement is definitional. For projection, unpack a named lifting witness, keep its specimen and predicates, and apply the class constructor to its endpoint equalities. For initial relocation, both endpoints are initial by the constructed initiality theorem, hence rooted by zero-step runs. Its search mode makes it remote, and its endpoint-class equalities are reflexive. This last theorem starts with an existing specimen and is not yet the general observation-lifting theorem for arbitrary context and fragment observations. □

Lean: `lifting_iff`

Lean: `lifting_project`

Lean: `initial_lifting`

48.4 Class safety, robustness, and forward sufficiency

Definition 48.13 (Mode of a state class)**.** The focus mode of a class is the focus mode of any representative. Recursive focus-preserving isomorphisms prove the required equality of modes, so quotient lifting is well-defined.

Lean: `mode`

Definition 48.14 (Class quasi-specimen)**.** A quasi-specimen between two classes has an unconditional concrete specimen witness whose mode agrees with both endpoint modes. Conditional on rootedness of both classes, its filled endpoints are rooted and the two specified comparison relations hold. The non-rewrite variant additionally excludes rewrite mode at the first endpoint.

This retains the full witness and mode clauses of the earlier concrete predicate, with comparisons evaluated on the class machine.

Lean: `Quasi`

Lean: `NonRewriteQuasi`

Definition 48.15 (Input safety on the class machine)**.** Every entering search specimen satisfies one of the same three alternatives: both endpoint classes run to stuck states; or the two runs reach a settled specimen with lengths $1 + k, l$ satisfying the cost relation; or lengths $1 + k, 1 + l$ satisfy the cost relation and reach a non-rewrite quasi-specimen up to equivalence. No source-rootedness restriction is imposed.

Lean: `InputSafe`

Definition 48.16 (Robustness on the class machine)**.** For every non-entering rewrite specimen and every successor class of its left endpoint, either a source side is unrooted, the successor runs to a stuck class, or the two sides have appropriately cost-related runs to a non-rewrite quasi-specimen up to the specified refinements. Every successor class is quantified, independently of the names of the transition witnesses.

Since the source mode is rewrite and mode is invariant under isomorphism, search cannot witness such a successor. Thus the full-step quantifier here ranges precisely over the applicable rewrite transitions.

Lean: `Robust`

Lemma 48.17 (The three class bridges)**.** *Class quasi-specimens yield the abstract rooted quasi predicate. Class input safety and robustness yield the corresponding laws of the constructed class-valued specimen system.*

Proof. For quasi-specimens, receive the two rootedness hypotheses, use the witness's conditional comparisons, and transport the endpoint's non-rewrite mode along its equality with the witness mode. For input safety, retain the stuck and direct alternatives unchanged; in the third alternative retain costs and runs and apply the quasi bridge, keeping equivalence on both sides. For robustness, the given rootedness contradicts exactly the unrooted disjunct. The stuck-tail case is unchanged. In the matching case retain both runs and the cost comparison and use the quasi bridge with the given refinement relations. □

Lean: `quasi_bridge`

Lean: `inputSafe_bridge`

Lean: `robust_bridge`

Theorem 48.18 (Concrete forward counting simulation)**.** *Assume behaviour invariance, behaviour determinism modulo isomorphism, behaviour refocusing, allowed-context invariance, output closure, the reasonable cost-order hypotheses, and the class input-safety and robustness predicates. Then the concrete class lifting is a counting simulation for the full class UAM.*

Proof. Instantiate the abstract forward counting-simulation theorem with the constructed class specimen system. Class determinism supplies its literal determinism premise; the two concrete bridges supply safety and robustness. All locality fields have already been proved from the concrete hypernet semantics. □

Corollary 48.19 (Quantitative refinement of concrete specimens)**.** *Under the same hypotheses, the left endpoint class of every remote jointly rooted concrete specimen refines its right endpoint class with the given cost order.*

Proof. The specimen, remoteness, rootedness, and reflexive class equalities form a lifting witness. Apply counting-simulation soundness using literal class determinism and the reasonable cost orders. The remaining contextual step is to show that arbitrary allowed initial observations give such witnesses. □

Lean: `countingSimulation`

Lean: `sufficiency`

48.5 Behaviour closure and non-vacuous regressions

Lemma 48.20 (Compute shape respects recursive renaming)**.** *Independent recursive renamings of the two endpoints preserve compute shape.*

Proof. Transport both focus modes. Carry the active-operation edge through the source edge bijection; primitive-label equivalence preserves its operation symbol and hence activity. Mapping its singleton source list gives the renamed focus target, because the isomorphism preserves that target. □

Lean: `State.Compute.rename`

Definition 48.21 (Behaviour closure under names)**.** The isomorphism closure of a behaviour contains a transition when an original behaviour transition has recursively isomorphic endpoints. The preceding compute-shape lemma verifies that this is again an admissible behaviour.

Lemma 48.22 (Inclusion and invariance)**.** *The closure contains each original transition and respects independent recursive endpoint renaming.*

Proof. Use identity isomorphisms for inclusion. For invariance, compose each original witness isomorphism with the corresponding new endpoint isomorphism. □

Lean: `isoClosure`

Lean: `into_isoClosure`

Lean: `isoClosure_invariant`

Theorem 48.23 (Behaviour closure preserves the full class relation)**.** *Replacing the behaviour by its isomorphism closure leaves every full-UAM class transition unchanged.*

Proof. Extract a representative step. Intrinsic steps are unchanged. For an extrinsic closure step, take its original compute witnesses and compose the endpoint-class equalities with quotient soundness for their isomorphisms. This gives an original class step. Conversely, retain the representatives and include any original extrinsic step in the closure. Thus closing under names adds no new class computation. The theorem does not assert that an arbitrary behaviour becomes deterministic after closure. □

Lean: `isoClosure_class_step`

The substitution regression projects its actual two-step terminating execution to an exactly two-step class execution. The compute regression closes the earlier nonempty singleton behaviour under recursive renaming. Every result is isomorphic to its one fixed initial result, so two results are isomorphic to each other; initiality also makes each result rooted. The general theorem therefore proves literal determinism of this nonempty full class machine. Projection supplies its genuine extrinsic transition, and the closure theorem checks equality with the original singleton's class relation.

Lean: `two_step_execution`

Lean: `behaviour`

Lean: `behaviour_deterministic`

Lean: behaviour_refocusing

Lean: deterministic

Lean: compute_step

Lean: same_class_steps

Finally the nested-hole example instantiates the class-valued specimen system with empty behaviour and the previously proved output-closed template. Its constructed initial relocation is a concrete class-lifting witness. This checks that the interface is exercised on a real recursive specimen, not only an empty specimen type.

Lean: specimenSystem

Lean: initial_lifting

Status. These results are subsequently used in Section 50, which completes independent observations and both contextual clauses for the fixed-depth machine. The remaining transfer to the paper's unbounded operational semantics is recorded in Appendix B.

49 Concrete converse simulation and two-sided refinement

This section completes the converse integration left open in Section 48. It follows the reverse Proposition 40 argument in Appendix F.9 of the paper, already proved abstractly in this development. The new work supplies its concrete class-machine predicates and bridges. Safety stays on the original template with the converse cost relation; robustness exchanges the specimen endpoints. General observation lifting remains separate.

Definition 49.1 (Converse quasi-specimen)**.** Write $[S_L]$ and $[S_R]$ for the endpoint classes of an original concrete specimen S. A converse quasi-specimen from s to t, up to relations A, R, requires that s is not in rewrite mode and that there exists an original specimen S whose mode agrees with both endpoint modes. If s, t are rooted, require rootedness of S_R, S_L and comparisons

$$A(s, [S_R]) \quad \text{and} \quad R([S_L], t).$$

The context, its allowed-membership proof, hole inventory, and related filling vectors are those of the original specimen. Exchanging the roles of the vectors represents a converse specimen without assuming symmetry of the template. The witness and mode agreement are unconditional; only the rooted comparisons are conditional.

Lean: ConverseQuasi

Lemma 49.2 (Characterisation by reversed comparisons)**.** *The converse predicate on (s,t) is equivalent to the original non-rewrite quasi predicate on (t,s) with comparisons $(x,y) \mapsto R(y,x)$ and $(x,y) \mapsto A(y,x)$.*

Proof. Use the same specimen in each direction. Exchange its mode equalities, its rootedness premises, its two rootedness conclusions, and its comparisons. Both endpoint modes equal the specimen mode, so non-rewrite status transfers between them. Repeating these exchanges gives the inverse implication. □

Lean: converseQuasi_iff

Lemma 49.3 (Preserving equivalence in safety III)**.** *An original non-rewrite quasi-specimen from s to t with equivalence comparisons at cost order P gives a converse quasi-specimen from t to s with the same equivalence comparisons and the same order P.*

Proof. Apply the preceding characterisation. Each equivalence consists of two refinements; exchange these two components on both sides of the witness. No symmetry of the numerical cost relation is used, and neither refinement direction is discarded. □

Lean: converseQuasi_equiv

Definition 49.4 (Concrete converse robustness)**.** For every original specimen S with remote rewrite focus and every full-class successor $[S_R] \to u$, require one of the following alternatives. A source endpoint is unrooted; or u runs to a stuck class; or there are runs $u \to^k s$ and $[S_L] \to^l t$ with $Q(1+k,l)$ and a converse quasi-specimen from s to t up to refinements at orders P, R. All successors are quantified, regardless of the names of representatives witnessing the class step. This is the concrete converse robustness obligation; it is not inferred from forward robustness.

Lean: ConverseRobust

Lemma 49.5 (Bridge for converse quasi-specimens)**.** *A concrete converse quasi-specimen implies the quasi law of the opposite class-valued specimen interface.*

Proof. Given rootedness of the endpoint classes, apply the conditional part of the concrete witness. Its comparison endpoints are exactly the opposite interface's left and right endpoints. The equality between specimen mode and source mode transfers the non-rewrite condition. Thus every field of the abstract witness is supplied. □

Lean: converseQuasi_bridge

Lemma 49.6 (Bridge for converse robustness)**.** *Concrete converse robustness implies robustness of the opposite constructed class-valued interface.*

Proof. The interface supplies rootedness of the right and left source endpoints. These contradict either unrooted alternative. Keep the stuck alternative unchanged. In the matching alternative retain the run lengths, costs, and executions and apply the converse quasi bridge. There is no additional locality assumption. □

Lean: converseRobust_bridge

Theorem 49.7 (Concrete converse counting simulation)**.** *Assume allowed-context invariance, output closure, behaviour refocusing and invariance under recursive isomorphism, behaviour determinism modulo isomorphism, and reasonable cost orders Q, P, R. Input safety of the original template at Q^{op}, P, together with concrete converse robustness at Q, P, R, makes the opposite concrete lifting a counting simulation at Q, P, R.*

Proof. Instantiate the previously proved reverse Proposition 40 with the constructed class specimen system. Literal class determinism supplies determinism. The original input-safety bridge preserves all three alternatives and both equivalence directions; the converse robustness bridge supplies the other hypothesis. The existing reverse argument handles safety II with a zero-length right run by an additional context traversal, whose locality is now a proved concrete theorem. In safety III it uses the reverse refinement from each equivalence. Thus this instantiation retains precisely the cost and direction choices that a naive exchange of forward safety would lose. □

Lean: `countingSimulation_converse`

Theorem 49.8 (Reverse quantitative refinement)**.** *Under the preceding assumptions, every remote jointly rooted original specimen satisfies $[S_R] \preceq_Q [S_L]$.*

Proof. The specimen, with its rootedness proofs exchanged, witnesses opposite lifting. Apply counting-simulation soundness using literal class determinism and the reasonable cost orders. □

Lean: `sufficiency_converse`

Theorem 49.9 (Two-sided concrete refinement)**.** *Assume the common behaviour, context, and cost hypotheses; input safety at both Q, P and Q^{op}, P; forward robustness and converse robustness at Q, P, R. Then every remote jointly rooted specimen satisfies $[S_L] \simeq_Q [S_R]$.*

Proof. The previously proved forward sufficiency theorem supplies the first refinement. The new converse sufficiency theorem supplies the second. Their conjunction is the definition of quantitative equivalence. This theorem is about actual specimen endpoints; it does not yet quantify over arbitrary fragment observations. □

Lean: `equivalence`

Corollary 49.10 (Equivalence of constructed initial specimens)**.** *Under the same global hypotheses, let D be the initial relocation of any concrete specimen S. Then $[D_L] \simeq_Q [D_R]$, without separate remoteness or rootedness assumptions.*

Proof. The relocation theorem makes both endpoints initial. Initial states are rooted. The relocated focus is in search mode and therefore cannot be in rewrite mode, which proves remoteness. Apply two-sided concrete refinement. The input remains an existing concrete specimen; construction of such a witness from arbitrary related fragments and allowed observation contexts is still outstanding. □

Lean: `initial_equivalence`

Status. These results are subsequently used in Section 50, which completes independent observations and both contextual clauses for the fixed-depth machine. The remaining transfer to the paper's unbounded operational semantics is recorded in Appendix B.

50 Concrete observations and contextual robustness

The two contextual sufficiency clauses are now proved for the concrete fixed-depth class machine. Observation is defined independently of a template: fill the declared holes of a context, then initialise the resulting closed, focus-free graph. Related fragments are subsequently used to construct the paired specimen needed by Proposition 38. Thus the theorem no longer assumes initial lifting or graph locality.

The scope qualification matters. The operational carrier here is `DeepStateClass ops d`, with a fixed depth allowance d throughout an execution. The paper states Theorem 6 on the union of all finite depths. The graph-level finite-union encoding alone does not establish operational transfer. Sections 51 and 52 supply that transfer, culminating in both clauses on the unbounded machine.

50.1 Independent recursive substitution

Definition 50.1 (Typed slots and independent fragment vectors)**.** A slot interface J is a predicate on pairs (n, h) of depth allowances and typed hole labels. A fragment vector assigns to every declared pair a graph of depth n with exactly the input and output lists of h. The assignment does not depend on a template, another vector, or a filling certificate. Interfaces may declare unused slots; only the finitely many holes in the context contribute to an observation.

Lean: `Slots`

Lean: `Fragments`

Definition 50.2 (Coverage of an original context)**.** A graph is covered by J when each of its edge labels is covered. A primitive hole h at depth n requires $J(n, h)$; every other primitive label is covered automatically. A box requires coverage of its original body at the smaller depth. The mutual recursion thus visits every original nested hole.

Lean: `Covers`

Lean: `CoversLabel`

Lemma 50.3 (Coverage at an edge)**.** *Coverage of a graph supplies coverage of the label of every edge.*

Proof. Unfold the graph-level coverage equation and apply the resulting universally quantified assertion to the selected edge. □

Lean: `Covers.edge`

Definition 50.4 (Independent filling)**.** Fill each edge of the context by its label's replacement and expand the resulting family simultaneously. At a primitive hole use the supplied fragment. At another primitive label use its corolla. At a box first fill its original body, then use the corolla labelled by the rebuilt box. Inserted fragments are leaves: substitution never recurses into them. These mutually recursive definitions decrease the depth on entry into a box.

Lean: `fill`

Lean: `fillLabel`

Definition 50.5 (Pointwise relatedness)**.** Vectors a, b are related by a layered pre-template T when $(T_n)(a(n,h), b(n,h))$ holds at every declared slot. This is a relation on independent fragment data; it is not defined by existence of an observation or specimen.

Lean: `Related`

Definition 50.6 (Retained label copies)**.** A label-copy certificate for the graph assigned to an edge yields its retained-corolla certificate. Copy the distinguished edge, its label equality, its ordered source and target equations, and its uniqueness field.

Lean: `retainedCopy`

Definition 50.7 (Local paired labels)**.** A pair of label replacements is certified in one of three ways: both are copies of the original label; the original label is a hole and the replacements are template-related; or the original label is a box, its two new bodies form a recursive paired filling, and the replacements copy the two rebuilt box labels. This certificate separates the local label proof from the dependent types of a surrounding edge family.

Lean: `PairedLabel`

Lemma 50.8 (From local labels to edge certificates)**.** *A paired-label certificate at an actual context edge gives the existing recursive filling-edge certificate.*

Proof. Eliminate the certificate. Retained labels use the preceding copy conversion; filled holes preserve the label equality and template membership; rebuilt boxes preserve the recursive body certificate and both label copies. □

Lean: `PairedLabel.edge`

Lemma 50.9 (Related vectors pair at every label)**.** *If a, b are related, their independent replacements of any covered label form a paired-label certificate.*

Proof. Induct on the depth allowance. At a hole apply pointwise relatedness. At each non-hole primitive use two identical corolla-copy certificates. At a box apply the induction hypothesis to every original body edge, convert the local certificates to filling-edge certificates, and form a paired body expansion. Copies of the rebuilt box labels complete the box case. □

Lean: `paired_label`

Lemma 50.10 (Paired edges and whole fillings)**.** *Independent substitution of related vectors yields paired certificates at all original edges and a paired recursive filling of the whole context.*

Proof. Apply the preceding local theorem to each edge and convert its certificate. Unfold the two independent filling equations; both are expansions of the same original context. The expansion constructor with those edge certificates proves the whole-graph assertion. □

Lean: `paired_edges`

Lean: `paired`

Theorem 50.11 (Finite exact substitution tuple)**.** *For a unique-hole context and related vectors whose left filled graph is hole-free, there is a finite tuple realising the same two filled graphs. Its hole names have no duplicates and are exactly the original context's holes.*

Proof. Convert the proved paired filling into a tuple using the existing recursive tuple theorem. Apply the tuple inventory theorem with context uniqueness and left-result closure. Declared but unused slots add no tuple entries; the finite tuple is extracted from the context rather than from an enumeration of the slot interface. □

Lean: `finite_tuple`

50.2 Initial witnesses without a pre-existing specimen

Definition 50.12 (Initial specimen from a paired filling)**.** Given an unfocussed, focus-free unique-hole context C, allowed membership, two paired edge families, and hole-free results, insert a search focus at the unique input of C. Keep the old filling families and add a retained focus corolla on each side. Insertion preserves unique holes; the insertion–erasure isomorphism transports allowed membership. The insertion–expansion isomorphism supplies closure of both filled endpoints. All specimen fields are constructed from these data.

Lean: `specimen`

Definition 50.13 (Initial graph observation)**.** A class s observes a graph G when G is focus-free and hole-free and s is the class obtained by realising the initial position of G. The witnesses are graph-validity proofs. This definition has no template parameter.

Lean: `Observe`

Lemma 50.14 (Independent endpoint representation)**.** *The left endpoint of the constructed specimen observes the independently filled left graph; likewise on the right.*

Proof. Paired-filling focus freedom supplies focus freedom of each result. Use the corresponding given hole-freedom proof. The insertion–expansion commuting isomorphism identifies the specimen endpoint with independent initialisation; promote it to a recursive focussed isomorphism and apply quotient soundness. □

Lean: `left_observation`

Lean: `right_observation`

Lemma 50.15 (Uniqueness of initial observation)**.** *Any two observations of the same filled graph give equal state classes.*

Proof. Unpack both observations. The initial realisations differ only in proofs of the same propositions. Proof irrelevance identifies those arguments, leaving reflexive equality. □

Lean: `observe_unique`

Theorem 50.16 (Initial lifting for paired fillings)**.** *Independent observations of the two closed paired fillings belong to the constructed class specimen system's lifting relation.*

Proof. Construct the initial specimen. Its focus is search at the unique input, so it is remote. Boundary preservation makes both filled endpoints initial, and initiality gives rootedness. The two endpoint representation lemmas and uniqueness of observation identify the given classes with the witness endpoints. These facts supply every field of lifting. □

Lean: `lifting`

50.3 The concrete contextual theorem

Definition 50.17 (Observation contexts and observations)**.** An observation context consists of an actual unique-hole graph with outer interface [term] → [], a proof of focus freedom, and coverage by its declared slot interface. Allowedness is the supplied context predicate on that graph. Observing vector a means independently filling the graph with a and taking an initial graph observation. Neither context data nor observation mentions T. Observation remains partial when the filled graph is not a valid closed state.

Lean: `ObservationContext`

Lean: `observations`

Theorem 50.18 (Concrete Proposition 38)**.** *Two independent observations of related vectors in the same allowed observation context belong to concrete class lifting.*

Proof. Extract hole freedom from each observation. The independent-substitution theorem supplies paired edge certificates on the original context. Apply initial lifting for paired fillings, retaining the original allowed-membership proof and the two observed endpoint classes. Thus the previously abstract initial-lifting premise is now proved from graph data. □

Lean: `initial_lifting`

Theorem 50.19 (Concrete forward contextual sufficiency)**.** *Fix d. Assume an isomorphism-invariant class of allowed contexts, output closure of T_d, behaviour refocusing, behaviour invariance under recursive isomorphism, and behaviour determinism modulo isomorphism. Let Q, P, R be reasonable cost orders. If class input safety and forward robustness hold at Q, P, R, then any pointwise related vectors satisfy contextual refinement at Q for the concrete observations and the full class machine at depth d.*

Proof. Instantiate abstract contextual sufficiency with the constructed class specimen system and the independent observation interface. Supply literal class determinism, the already proved safety and robustness bridges, and the preceding concrete initial-lifting theorem. Every graph-locality and observation obligation is discharged; the remaining assumptions are the stated semantic and cost hypotheses. □

Lean: `sufficiency`

Theorem 50.20 (Concrete converse contextual sufficiency)**.** *Under the same structural and reasonable-cost hypotheses, input safety on the original template at Q^{op}, P and converse robustness at Q, P, R imply contextual refinement from the right vector to the left at Q.*

Proof. Use the abstract converse contextual theorem with the same concrete initial lifting. Supply the original input-safety bridge with converse costs and the converse robustness bridge. The previously checked converse counting simulation preserves the zero-right-step safety case and both directions of the safety equivalences. □

Lean: `sufficiency_converse`

Corollary 50.21 (Concrete contextual equivalence)**.** *When both sets of safety and robustness hypotheses hold, related vectors are contextually equivalent at Q.*

Proof. Pair the forward and converse contextual refinements. This is exactly the definition of observational equivalence. □

Lean: `equivalence`

50.4 Single fragments and a nested regression

Definition 50.22 (Single-slot specialisation)**.** The singleton interface for (n, h) declares precisely pairs equal to (n, h). A graph G of that depth and signature gives a fragment vector by transporting G along those equalities. Therefore the vector theorem includes ordinary replacement of one arbitrary graph.

Lean: `singleSlot`

Lean: `singleFragment`

Lemma 50.23 (Exact original relation in the singleton case)**.** *The two singleton vectors are related exactly when their underlying graphs are related by T_n.*

Proof. In the forward direction evaluate relatedness at the distinguished slot with reflexive equalities. In the reverse direction eliminate the two equalities carried by any declared slot and use the original relation proof. □

Lean: `single_related_iff`

Corollary 50.24 (Both single-fragment contextual clauses)**.** *The forward and converse contextual sufficiency theorems hold for arbitrary original related graphs, represented as singleton vectors, including contexts whose selected hole is nested.*

Proof. Convert the original relation proof by the preceding equivalence and apply the corresponding vector theorem. No relation on arbitrary filler functions is demanded from a single-graph user. □

Lean: `single_sufficiency`

Lean: `single_sufficiency_converse`

Definition 50.25 (A genuinely nested observation context)**.** Use the earlier outer graph with a maker, a box, and a consumer, with its sole hole inside the original box body at depth zero. Remove the earlier focus by using the unfocussed outer graph itself. Unique holes follow from insertion's uniqueness equivalence; focus freedom was already proved. Coverage checks the two non-hole operations and the single nested hole. The independently supplied singleton fragment is the variable corolla.

Lean: `context`

Lean: `fragments`

Lemma 50.26 (Related and closed nested fillings)**.** *The regression's fragment is related to itself by the one-way equality template, and its independently filled outer graph is hole-free.*

Proof. Relatedness uses equality, focus freedom, and one-wayness of the variable corolla. For closure, use expansion's componentwise hole-freedom theorem. The outer operations have no holes. The box case reduces to its filled body, whose hole is replaced by the hole-free variable corolla. □

Lean: `related`

Lean: `closed`

Lemma 50.27 (Non-vacuous observation and lifting)**.** *The regression has an actual initial observation, and any two such independently supplied observations satisfy concrete initial lifting.*

Proof. Paired-filling focus freedom and the preceding closure theorem construct an observation. Apply the general concrete initial-lifting theorem with the genuine related singleton vectors, the allowed nested context, the empty behaviour, and the established output-closure and context-invariance proofs. This tests a hole below a box, not merely an empty specimen type. □

Lean: `observation_exists`

Lean: `initial_lifting`

Scope of this intermediate result. This section completes the fixed-depth contextual theorem. The operational transfer requires further work: a common bound for finite runs, exact costs, finality and the semantic hypotheses must all be justified. That work is now completed in Section 52. The copying UAM and the language-specific examples remain outside the general-theorem milestone.

51 Depth padding and finite executions in the unbounded carrier

This section constructs a union of the concrete state spaces at all finite depths and proves exact finite-run adequacy for search. The construction is faithful: two different recursive state classes at a fixed depth never become equal merely by passing to the union. Padding is a change of representation, not a transition and not a unit of execution cost.

The general finite-run theorem requires preservation of the transition relation under padding. That premise is proved below for the five search rules. This section supplies the search case; Section 52 proves full-UAM compatibility, transfers the semantic interfaces, and concludes both unbounded contextual sufficiency clauses.

51.1 Padding graphs, inventories and state classes

Lemma 51.1 (One additional depth allowance)**.** *Padding fixes every primitive label and sends a box body to its existing recursive raised representation. Its input and output lists are unchanged. Applying this label map to a graph leaves every outer carrier, port and incidence list unchanged; it equals the previously defined operation* raise*. Focus labels in particular are fixed.*

Proof. Inspect the depth and the label constructor. At depth zero the primitive is unwrapped; at positive depth inspect primitive versus box. The two graph constructions use the same relabelling function, proved pointwise equal. □

Lean: `label`

Lean: `label_input`

Lean: `label_output`

Lean: `graph`

Lean: `graph_eq_raise`

Lean: `label_primitive`

Lean: `label_focus`

Lemma 51.2 (Preservation and reflection of focus freedom)**.** *A label, and hence a graph, is focus-free exactly when its padding is focus-free.*

Proof. Induct on the depth. Primitive labels are unchanged. For a box, rewrite raising as graph padding and apply the induction hypothesis to each edge of its body. The graph statement applies the label equivalence edgewise. □

Lean: `label_free`

Lean: `focusFree_iff`

Lemma 51.3 (Padding a distinguished focus)**.** *Keep the distinguished edge, mode, source and target ports. Its label is still the required focus label, and all other labels remain focus-free. The initial and final predicates are unchanged.*

Proof. Transport the distinguished label equality by the padding map and use its focus-label equation. Transport other-edge freedom by the preceding equivalence. Incidence and boundary conditions are definitionally unchanged. □

Lean: `focussed`

Lean: `focussed_initial_iff`

Lean: `focussed_final_iff`

Definition 51.4 (Exact inventories across depths)**.** An inventory isomorphism is a bijection of all recursive edge addresses preserving the optional hole label. Its local version is a bijection of the occurrence fibre of one label. Local fibre isomorphisms assemble into a graph inventory isomorphism by keeping the outer edge and applying its fibre bijection; conjugate by the existing occurrence-address bijections.

Lean: `InventoryIso`

Lean: `FiberIso`

Lean: `assemble`

Lemma 51.5 (The padding inventory)**.** *Every graph has an exact inventory isomorphism to its padding.*

Proof. Induct on depth. Primitive fibres are identical. For a box, retain its shallow unit occurrence and use the inductively obtained body inventory on its nested occurrences. Assemble these label-local bijections at the surrounding graph. □

Lean: `inventory`

Lemma 51.6 (Hole predicates under inventory isomorphism)**.** *An inventory isomorphism preserves and reflects existence of each hole, absence of all holes, and uniqueness of occurrences of any given hole.*

Proof. Transport existence witnesses by the bijection or its inverse and the hole-label equation. Negate the existence equivalence for hole freedom. For uniqueness, transport the two occurrences to the other graph, use uniqueness there, and cancel the injective bijection. □

Lean: `hasHole_iff`

Lean: `holeFree_iff`

Lean: `unique_iff`

Lemma 51.7 (Padding a closed state)**.** *Pad the graph and its distinguished focus, and obtain hole freedom from the exact inventory. This produces an actual closed state at the next depth. Initiality and finality are unchanged.*

Proof. The state constructor uses the preceding inventory equivalence. The boundary predicates reduce to the same mode and ports. □

Lean: `state`

Lean: `state_initial_iff`

Lean: `state_final_iff`

Definition 51.8 (Transport of recursive graph isomorphisms)**.** Given preservation, or reflection, of the recursive label relation by padding, keep the vertex and edge bijections of a graph isomorphism and transport its edge-label certificates. All remaining fields retain their original proofs.

Lean: `mapIso`

Lean: `unmapIso`

Lemma 51.9 (Exact recursive label and graph equivalence)**.** *Two padded labels are recursively equivalent precisely when their original labels are. Consequently recursive graph isomorphisms can be padded and reflected.*

Proof. Induct on depth and split the label constructors. Primitive equality is unchanged and mixed constructors cannot be related. Two boxes are related by an isomorphism of their bodies; apply the induction hypothesis through the preceding isomorphism transports, rewriting graph padding as raising. □

Lean: `label_equiv_iff`

Lean: `deepIso`

Lean: `reflectIso`

Lemma 51.10 (Faithful embedding of state classes)**.** *Padding preserves and reflects focus-preserving recursive isomorphisms and therefore defines an injective function on recursive state classes.*

Proof. The graph component uses the preceding transports, while the distinguished edge and mode remain unchanged. Quotient elimination defines the map. Equality of two padded classes yields a focussed isomorphism; reflect it and use quotient soundness to recover equality of the original classes. □

Lean: `stateIso`

Lean: `reflectStateIso`

Lean: `stateClass`

Lean: `stateClass_injective`

Lemma 51.11 (Boundary predicates on padded classes)**.** *Initiality and finality are preserved and reflected by the class embedding.*

Proof. Choose representatives by quotient induction and apply the corresponding actual-state equivalences. □

Lean: `class_initial_iff`

Lean: `class_final_iff`

51.2 Exact compatibility of concrete search

Lemma 51.12 (Operation views and resumption)**.** *An operation view pads by retaining its edge and ordered argument data; the view also reflects from a padded graph. Padding equals a specified primitive label exactly when the original label does. The next resume position of either transported view is unchanged.*

Proof. The primitive-label equivalence follows by cases on depth and label. Transport the view label equation by that equivalence. For resumption, split on whether the next argument-list entry exists; the argument ports and mode in either result are identical. □

Lean: `operation`

Lean: `label_primitive_iff`

Lean: `reflectOperation`

Lean: `operation_resume`

Lean: `reflectOperation_resume`

Lemma 51.13 (Coordinate search is preserved and reflected)**.** *For any two search positions, a coordinate search step in the padded graph holds exactly when it holds in the original graph.*

Proof. Eliminate the search derivation in each direction. Variable triggering uses primitive-label preservation or reflection. Operation entry uses the transported view and the first argument, when present. Resumption uses the view-resume equations. These constructors cover the five coordinate rules. □

Lean: `searchStep_iff`

Lemma 51.14 (Erasing a padded focus)**.** *Padding commutes with retaining an erased port, contracting the focus port, and forming the focus-free underlying graph. There is an explicit graph isomorphism between the two orders of erasure and padding.*

Proof. For retention, split on the port constructor. For contraction, split on the test selecting the contracted port, then apply retention. Graph equality follows edgewise on source and target lists. The explicit isomorphism uses identity carrier bijections and the same list equations; it avoids transporting dependent position types along graph equality. □

Lean: keep_padding

Lean: contract_padding

Lean: underlying

Lean: underlyingIso

Lemma 51.15 (Inserting a focus commutes with padding)**.** *Padding a focus insertion is isomorphic to inserting the same focus at the same typed vertex in the padded graph.*

Proof. Use identity vertex and edge bijections. Retained labels map directly; the new edge uses the focus-label equation. Split on retained versus new edges and then on port constructors to verify the insertion source and target lists. □

Lean: insertion

Lemma 51.16 (Coordinates and realised endpoints)**.** *A padded state has the same coordinate. A typed position on the erased original state therefore defines a typed position on the erased padded state. Padding the realised original endpoint is recursively focus-isomorphic to realising this transported position.*

Proof. The coordinate equation is the retention equation. For realisation, compose the insertion-padding isomorphism with the existing realisation-renaming isomorphism induced by the explicit erasure isomorphism. The transported position equals the intended position by subtype extensionality. □

Lean: coordinate

Lean: position

Lean: realizeIso

Lemma 51.17 (Exact state-level search adequacy)**.** *Erased-graph coordinate search, actual recursive search, and recursive-class search all preserve and reflect padding.*

Proof. Rename coordinate search along the erasure isomorphism. For actual search, transport its witnessing position and coordinate transition, then compose the realisation isomorphism with the padded target isomorphism. For reflection, use the same port as an original position and reflect the composite endpoint isomorphism. Finally apply quotient induction to both endpoints. □

Lean: underlying_search_iff

Lean: deepSearch

Lean: deepSearch_reflect

Lean: class_search_iff

51.3 The union of all finite state spaces

Definition 51.18 (Finite presentations and repeated padding)**.** A stage is a pair (n, s) with s a recursive state class at depth n. The successor operation pads s once and increments n. Write $A_k(s)$ for k repetitions, with $A_0(s) = s$.

Lean: `Stage`

Lean: `next`

Lean: `advance`

Lemma 51.19 (Arithmetic and injectivity of padding)**.** $A_{i+j}(s) = A_j(A_i(s))$*, repeated paddings commute, and the depth of* $A_k(s)$ *is the original depth plus* k*. Successor and every* A_k *are injective.*

Proof. Induction on the iteration count proves composition and depth; commutation follows from commutativity of addition. Equality after successor first gives equality of depths by cancellation of successor, then equality of state classes by injectivity of class padding. Induction cancels any finite number of successors. □

Lean: `advance_add`

Lean: `advance_comm`

Lean: `advance_depth`

Lean: `next_injective`

Lean: `advance_injective`

Lemma 51.20 (Equality of presentations at a common depth)**.** *Declare* $s \sim t$ *when* $A_i(s) = A_j(t)$ *for some* i, j*. This is an equivalence relation.*

Proof. Use zero paddings for reflexivity and exchange witnesses for symmetry. For transitivity, if $A_i(s) = A_j(t)$ and $A_k(t) = A_l(u)$, apply k more paddings to the first equality and j to the second. Commutation identifies the two middle presentations. □

Lean: `Same`

Lean: `same_refl`

Lean: `same_symm`

Lean: `same_trans`

Lean: `stageSetoid`

Lemma 51.21 (The unbounded carrier)**.** *Take the quotient of stages by common-depth equality. A stage, or a class at a specified depth, maps to its quotient class. Every padded presentation has the same image as its original.*

Proof. The quotient identifies precisely the declared equivalence. For padding invariance, its witnesses are zero on the padded presentation and k on the original. □

Lean: `State`

Lean: `ofStage`

Lean: `ofClass`

Lean: `ofStage_advance`

Lemma 51.22 (Fixed-depth classes remain distinct)**.** *The map from any one fixed-depth class space into the unbounded carrier is injective.*

Proof. Quotient equality supplies $A_i(n, s) = A_j(n, t)$. Comparing depths gives $n + i = n + j$, hence $i = j$. Cancel the common injective iteration and then the common depth component to obtain $s = t$. □

Lean: `ofClass_injective`

Lemma 51.23 (Invariant predicates on the union)**.** *A stage predicate is invariant when successor preserves and reflects it. It then has the same truth value after any iteration, on any two equivalent stages, and hence defines a predicate on the quotient.*

Proof. Induct on the iteration count. For equivalent stages, compare both predicates at their common padded representative. Propositional extensionality supplies the equality required by quotient elimination. □

Lean: `Invariant`

Lean: `invariant_advance`

Lean: `invariant_same`

Lean: `predicate`

Lemma 51.24 (Unbounded initial and final states)**.** *Descend the existing class initiality and finality predicates using their proved padding invariance. On the image of a fixed-depth class these definitions recover exactly the original predicates.*

Proof. Apply the invariant-predicate construction; the comparison on a class image is definitional. □

Lean: `Initial`

Lean: `Final`

Lean: `initial_ofClass`

Lean: `final_ofClass`

51.4 Finite runs and their exact costs

Definition 51.25 (Families of transition relations)**.** A relation family supplies a transition relation R_n at each depth. A staged edge is an R_n edge with both endpoints explicitly at that depth. Preservation means every R_n edge pads to an R_{n+1} edge. This is an explicit proof obligation on a family; no global axiom asserts it.

Lean: `Relations`

Lean: `At`

Lean: `Preserved`

Lemma 51.26 (Padding edges)**.** *For a preserved family, one or any finite number of paddings preserves a staged edge.*

Proof. Eliminate the staged-edge certificate and apply the preservation premise for one step. Induct on the iteration count for repeated padding. □

Lean: `at_next`

Lean: `at_advance`

Definition 51.27 (Unbounded steps and exact runs)**.** An unbounded step has representatives joined by a staged edge. A run of length zero has identical endpoints; adjoining one transition to a run of length k gives length $k + 1$. Thus padding itself is never counted as a transition.

Lean: `Step`

Lean: `Run`

Lemma 51.28 (Mapping, padding and projecting runs)**.** *A relation-preserving function maps a run without changing its length. Consequently a staged run can be padded, or projected to the unbounded carrier, with the same number of transitions.*

Proof. Induct on the run, mapping the empty run to the empty run and its first edge together with the induction hypothesis in the successor case. Apply this to padding with the edge-preservation lemma, and to projection with the original staged edge as its witness. □

Lean: `run_map`

Lean: `run_advance`

Lean: `project_run`

Lemma 51.29 (A common finite presentation of an unbounded run)**.** *Every finite unbounded run for a preserved family lifts to a staged run of exactly the same length, whose endpoint images are the specified unbounded endpoints.*

Proof. Induct on the run. For an empty run choose a representative by quotient induction. For a nonempty run take the finite representatives of its first edge and the inductively lifted tail. Their two middle presentations have equal quotient images, hence coincide after possibly different numbers of paddings. Pad the first edge and the whole tail by those numbers. They now compose literally. Padding leaves endpoint images unchanged and adds no transitions. □

Lean: `lift_run`

Lemma 51.30 (Recovering a single original relation)**.** *A staged edge, and every staged run, has equal endpoint depths. A staged edge at depth n is an R_n edge, and a staged run at that depth is an actual R_n run with the same count.*

Proof. An edge certificate already has a common depth. Compose these depth equalities inductively along a run. To extract the original run, induct on the staged run; its first edge forces the next state to the same depth, so the induction hypothesis applies to its tail. In the empty case use equality of the paired presentations to identify their class components. □

Lean: `at_depth`

Lean: `run_depth`

Lean: `at_same_depth`

Lean: `run_same_depth`

Lemma 51.31 (Exact finite-run adequacy)**.** *For a preserved family, an unbounded run of length k from x to y exists exactly when, for some depth n, there are representatives a, b of x, y and an R_n run of length k from a to b. The depth may depend on the entire run.*

Proof. Lift the unbounded run to stages, use equality of endpoint depths, and extract the original fixed-depth run. Conversely embed an original run as a staged run and project it. □

Lean: `finite_run_iff`

Lemma 51.32 (Application to concrete search)**.** *The family of concrete recursive search relations satisfies padding preservation. Its unbounded finite runs therefore have exactly the common-depth representatives and counts described above.*

Proof. Use the already proved state-class search equivalence in its forward direction to discharge the preservation premise, then instantiate exact finite-run adequacy. □

Lean: `search_preserved`

Lean: `search_run_iff`

Lemma 51.33 (Final observations with exact cost)**.** *For a preserved family, an unbounded run of length k from x to a final state exists exactly when some fixed-depth representative of x runs in k steps to a final class at that depth.*

Proof. Apply finite-run adequacy and rewrite finality along the final endpoint image. The descended finality predicate is definitionally the original class finality. Conversely project the run and its final endpoint. □

Lean: `terminating_run_iff`

52 The full unbounded UAM and contextual sufficiency

The final depth-transfer obligation is discharged here. Both contextual sufficiency clauses hold on the union of all finite depths, with genuinely global extrinsic behaviour. An execution is not restricted by the depth of its initial presentation. Every finite execution has a common finite-depth presentation with exactly the same transition count.

The proof retains the mathematical hypotheses of Theorem 6: deterministic and refocusing behaviour, output closure of the pre-template, the appropriate input-safety and robustness alternatives, and a reasonable cost triple. These are not asserted for every language or every relation. In particular, safety and robustness are evaluated on the *unbounded* machine. We do not infer them from fixed-depth safety: a state stuck at one depth could have a successor at a larger depth. The bridges below start with the concrete paper predicates, not an assumed abstract specimen-system certificate.

The final contextual relation uses all finite representatives of a global fragment class and all finite original contexts, including nested holes. Common representatives exist at every sufficiently large bound. This makes representation depth an implementation device, not a restriction on the contexts or executions being quantified. The copying UAM and the paper's language-specific examples remain outside this general-theorem milestone.

52.1 Substitution and extrinsic computation

Lemma 52.1 (Renaming and interface transport)**.** *Padding an ordinary labelled isomorphism keeps its vertex and edge bijections and applies the label map to its label equation. An isomorphism also transports across equal ordered boundary lists on both sides.*

Proof. All incidence equations are unchanged by relabelling. For boundary transport, eliminate the two interface equalities, reducing the assertion to the original isomorphism. □

Lean: `iso`

Lean: `reboundaryIso`

Lemma 52.2 (Padding commutes with plugging)**.** *Pad a plugged graph, or first pad its context and replacement and then plug with the transported interface equalities. The resulting graphs are isomorphic.*

Proof. Use identity maps on the sum carriers. A retained edge has the original context incidence; an inserted edge has the replacement incidence composed with boundary identification. In both cases compose the port maps, then inspect internal, input and output ports. Boundary casts keep the same index values, so the corresponding context source and target ports agree. □

Lean: `plug`

Lemma 52.3 (The substitution fragments commute with padding)**.** *The variable prefix and the before and after fragments of the concrete substitution rule each commute with depth padding up to labelled graph isomorphism.*

Proof. For the prefix, reuse the focus-insertion padding isomorphism, replacing its new-edge label equation by the variable-label equation. For the before fragment compose insertion-padding with insertion-renaming along the prefix isomorphism. The after fragment is directly the focus-insertion padding isomorphism. □

Lean: `variablePrefix`

Lean: `before`

Lean: `after`

Lemma 52.4 (Simple contexts under padding)**.** *A local occurrence fibre has a bijection to its padded fibre. Hence a shallow simple context stays simple: its distinguished hole label is unchanged, it stays focus-free, and all holes still belong to its distinguished edge.*

Proof. Primitive fibres are identical. For a box, retain the shallow occurrence and apply the already proved nested-body inventory. To establish ownership, map any padded hole occurrence back through the fibre bijection, use the original ownership condition, and observe that the outer owner edge never changed. □

Lean: `fiber`

Lean: `simple`

Lemma 52.5 (Preservation of concrete substitution)**.** *Each concrete substitution step pads to one concrete substitution step.*

Proof. Unpack its simple context, shared fragment and endpoint isomorphisms. Pad the context and fragment, transporting focus freedom and hole freedom. Use the two fragment-padding isomorphisms under plugging, followed by the plugging-padding isomorphism and the padded original endpoint isomorphisms. These construct the full witnesses of the concrete substitution definition; no rule compatibility is assumed. □

Lean: `substitution`

Lemma 52.6 (Compute shape and full class transitions)**.** *Compute shape is preserved and reflected by padding. A full class transition pads whenever the chosen extrinsic behaviours preserve padding.*

Proof. Compute shape consists of rewrite and search modes, an active primitive operation and its adjacency to the focus. Modes and incidence are unchanged; the primitive-label equivalence gives both directions. For a full class step, split search, substitution and extrinsic computation. Use the proved search and substitution results, or the supplied extrinsic preservation map respectively. □

Lean: `compute_iff`

Lean: `uamClass`

Definition 52.7 (A genuinely global behaviour)**.** Compute shape on two finite classes is witnessed by actual representatives with the prescribed active operation and modes. A global behaviour is a relation on unbounded states whose every edge admits such a finite compute-shape witness. No common depth is fixed for the relation, its sources, or its successors. Determinism and refocusing are separate hypotheses.

Lean: `ComputeAt`

Lean: `Behaviour`

Lemma 52.8 (Finite restrictions of global behaviour)**.** *The image of a finite class is unchanged by padding. Restrict a global behaviour at depth n by requiring its global edge and actual compute shape of the chosen representatives. These finite restrictions preserve padding.*

Proof. The quotient image equation is one-step padding invariance. For an extrinsic edge, the two global endpoint images remain equal after padding and the concrete compute-shape equivalence supplies admissibility. Thus preservation is proved from the global definition, rather than required of an arbitrary family of behaviours. □

Lean: `ofClass_padding`

Lean: `Behaviour.atDepth`

Lean: `Behaviour.padding`

Lemma 52.9 (The compatible full-machine family)**.** *At each depth use the original full class machine with the restriction of the global behaviour. This whole family preserves depth padding.*

Proof. Apply full class transition preservation to the restrictions at successive depths and use the derived extrinsic padding theorem. □

Lean: `RelationsOf`

Lean: `full_preserved`

Definition 52.10 (The unbounded transition relation)**.** The intrinsic relation is the union of the original empty-behaviour machines. A full unbounded step is either intrinsic or belongs to the given global extrinsic relation.

Lean: `Intrinsic`

Lean: `UAMStep`

Lemma 52.11 (Exact one-step operational adequacy)**.** *A union step of the finite restricted full machines is exactly a full unbounded UAM step.*

Proof. In the forward direction unpack its finite representatives. An intrinsic edge supplies an empty-behaviour witness; an extrinsic edge supplies the global edge from its restriction. Conversely an intrinsic witness is also a full-machine witness. For a global extrinsic edge, its compute-shape certificate supplies actual representatives; combine the same global edge with their shape to obtain an edge of the finite restriction. □

Lean: `full_step_iff`

Lemma 52.12 (The unbounded machine)**.** *Final unbounded states have no outgoing full-machine transition. Consequently the global relation and descended finality form a machine in the existing transition-system interface.*

Proof. Reflect a proposed transition to finite representatives and rewrite finality at its source. The fixed-depth class final-normal theorem contradicts that edge. □

Lean: `final_normal`

Lean: `machine`

Lemma 52.13 (Exact full runs and termination)**.** *An unbounded full-machine run of length k is precisely a run of length k in some finite restricted class machine between representatives of its endpoints. Termination additionally agrees with finality of the finite target.*

Proof. Use the generic finite-run theorem with the proved full-family compatibility and convert edges using one-step adequacy. For termination rewrite the descended final predicate at the endpoint image. The common bound is chosen after the entire run, and can exceed the initial presentation depth. □

Lean: `full_run_iff`

Lean: `full_termination_iff`

52.2 Determinism, rootedness and finite-source locality

Lemma 52.14 (Renaming invariance of global behaviour restrictions)**.** *Compute shape survives recursive focus-preserving renaming. Each restricted behaviour therefore respects recursive isomorphisms of both endpoints.*

Proof. Transport the active operation label and its source list through the source graph isomorphism, using preservation of the focus target and both modes. The endpoint classes of renamed states are equal, so the global edge is unchanged; only its admissibility proof needs transport. □

Lean: `compute_iso`

Lean: `Behaviour.respectsIso`

Lemma 52.15 (Global extrinsic determinism)**.** *Determinism of the global extrinsic relation means equal-source edges have equal targets as unbounded states. It implies determinism modulo recursive isomorphism of each finite restriction.*

Proof. Apply global determinism to the two projected extrinsic edges. Injectivity of the fixed-depth class embedding gives equality of target classes; quotient exactness gives their focussed recursive isomorphism. □

Lean: `Behaviour.Deterministic`

Lean: `Behaviour.deterministicAt`

Lemma 52.16 (Determinism of the full unbounded UAM)**.** *If every relation in a padding-compatible family is deterministic, its union relation is deterministic. In particular, deterministic global extrinsic behaviour makes the whole unbounded UAM deterministic.*

Proof. Two staged edges at the same source reduce to determinism in that layer. For union edges, their source presentations agree after possibly different paddings. Pad both edges, apply staged determinism, and project the equal targets. For the full UAM, the existing finite class determinism theorem applies because both renaming invariance and finite extrinsic determinism have just been proved. □

Lean: `at_deterministic`

Lean: `step_deterministic`

Lean: `deterministic`

Lemma 52.17 (Transfer in the original execution language)**.** *The auxiliary exact-run relation agrees with the existing machine execution relation. Full-UAM finite-run adequacy consequently holds for the same execution type used throughout the counting-simulation development.*

Proof. Convert either run presentation by induction, preserving the empty and successor cases. Apply these conversions on both sides of full-run adequacy. □

Lean: `run_steps_iff`

Lean: `machine_steps_iff`

Lemma 52.18 (Roots and all search successors)**.** *Padding commutes with canonical initial attachment. Moreover every search successor of a padded class is itself the padding of an original search successor.*

Proof. For the root use the realisation-padding isomorphism at the initial position. For a successor, unpack its position on the padded erased graph, use the same coordinate as an original typed position, reflect its coordinate step, and realise it. The realisation isomorphism identifies the padded result with the supplied target. □

Lean: `root`

Lean: `search_successor`

Lemma 52.19 (Exact rootedness under padding)**.** *Every search execution starting in the padding image remains in that image, with exactly the same length. A padded class is rooted exactly when its original class is rooted.*

Proof. Induct on the search run, using successor reflection to recover each original endpoint. For reflected rootedness, commute padding with the root, reflect its entire witnessing run and cancel the injective padding map at the target. For forward rootedness, pad every edge of the witnessing run. □

Lean: `search_run_reflect`

Lean: `rooted_iff`

Lemma 52.20 (Unbounded rootedness)**.** *Descend finite rootedness using its preservation and reflection. Its value at a class image is exactly finite rootedness. Equivalently, the unbounded state is reachable by search from an initial unbounded state.*

Proof. For the forward implication choose a representative, use its finite initial-search characterisation, and project the run. For the reverse implication use exact search-run adequacy, reflect initiality at the source, apply the finite characterisation, and project the rooted endpoint. □

Lean: `Rooted`

Lean: `rooted_ofClass`

Lean: `rooted_iff_initial_run`

Lemma 52.21 (Global refocusing)**.** *Global extrinsic refocusing is preservation of unbounded rootedness by extrinsic steps. It restricts to finite refocusing, extends to every full unbounded step, and hence makes all executions from an initial state rooted.*

Proof. Use the exact rootedness predicate at finite images to restrict the hypothesis. Reflect a full step to a finite machine and apply its existing full-UAM refocusing theorem. Finally induct on an execution, starting from the rootedness of an initial state. □

Lean: `Behaviour.Refocusing`

Lean: `Behaviour.refocusingAt`

Lean: `rooted_step`

Lean: `initial_execution_rooted`

Lemma 52.22 (Non-rewrite global transitions are search)**.** *Mode is preserved by class padding, so non-rewrite mode descends to the union. Every full transition at such a source is a global search step.*

Proof. Mode preservation is quotient induction. Reflect the full step to finite actual representatives. Substitution and extrinsic computation both require rewrite mode, contradicting the source condition. The remaining search edge supplies the global search witness. □

Lean: `mode_padding`

Lean: `NonRewrite`

Lean: `nonrewrite_search`

Lemma 52.23 (Locality at a specified source depth)**.** *A global search successor can be represented at the depth of any specified finite source class. In particular, an enabled non-rewrite actual state admits an actual recursive search transition at its own depth.*

Proof. Lift the successor-image theorem to stages and then induct on the padding count. Given a global search witness, align its source presentation with the specified source, pad the edge to their common presentation, and reflect the iterations back. Projecting the resulting target gives the original global target. For a non-rewrite full edge first identify it as global search, then choose an actual representative of the recovered target. □

Lean: `search_next_reflect`

Lean: `search_advance_reflect`

Lean: `search_ofClass`

Lean: `full_of_nonrewrite`

52.3 Concrete specimen systems and semantic hypotheses

Definition 52.24 (Concrete unbounded specimens)**.** A specimen is a finite depth paired with an existing concrete deep specimen: an original context, its focus, paired recursive fillings and closure proofs. Its two endpoints are the unbounded images of its actual filled states. The witnesses have real graph data; no locality law is stored in a specimen.

Lean: `Specimen`

Lean: `left`

Lean: `right`

Lemma 52.25 (The proved unbounded specimen system)**.** *Finite class transitions project to full unbounded transitions. The concrete specimens form a specimen system on the unbounded machine, with every locality field proved.*

Proof. Rooted-step preservation is global refocusing and final agreement is the concrete boundary theorem. For an enabled settled specimen, its focus is non-rewrite. Reflect the enabled edge to an actual search edge at the specimen depth. The existing concrete settled-successor theorem then constructs a paired successor specimen at that depth; project its two search edges. The allowed-context, output-closure and refocusing hypotheses retain their original roles. □

Lean: `project_step`

Lean: `system`

Lemma 52.26 (Mode on unbounded states)**.** *Repeated padding preserves mode, so quotient elimination defines the mode of an unbounded state. Its inequality to rewrite mode is exactly the earlier descended non-rewrite predicate.*

Proof. Induct on the padding count. For two equivalent presentations, compare their modes at their common padded representative. The predicate comparison is quotient induction. □

Lean: `mode_advance`

Lean: `mode`

Lean: `nonrewrite_iff`

Definition 52.27 (Concrete global quasi-specimens and settledness)**.** A quasi-specimen retains an unconditional concrete specimen witness and equality of its mode to both endpoint modes. When both endpoints are rooted, require rootedness of the witness endpoints and the prescribed two comparisons. The non-rewrite version additionally excludes rewrite mode. Settledness is the original backtrack-or-remote-search condition on the context focus.

Lean: `Quasi`

Lean: `NonRewriteQuasi`

Lean: `Settled`

Definition 52.28 (Input safety on the unbounded machine)**.** For every concrete specimen entering a hole in search mode, require one of the three paper alternatives: both sides reach globally stuck states; the sides reach a common settled specimen with costs $(1+k, l)$ related by Q; or both take positive runs of costs $(1+k, 1+l)$ related by Q to a non-rewrite quasi-specimen, whose two comparison relations are P-equivalences. All executions, stuckness and comparisons are evaluated on the full unbounded machine.

Lean: `InputSafe`

Definition 52.29 (Robustness on the unbounded machine)**.** For every concrete remote rewrite specimen and every global successor u of its left endpoint, require: at least one original endpoint is unrooted; or u reaches a globally stuck state; or matching runs from u and the right endpoint lead to a non-rewrite quasi-specimen. In the third case Q relates costs $(1+k, l)$, and the two comparisons are refinements for P and R. Global successors can require greater depth than the specimen.

Lean: `Robust`

Lemma 52.30 (The concrete-to-abstract semantic bridges)**.** *The explicit global quasi, safety and robustness predicates supply the corresponding laws of the proved specimen system. Under deterministic behaviour and a reasonable triple they yield the unbounded forward counting simulation.*

Proof. For quasi witnesses, supply the conditional rooted conclusion and use mode agreement to exclude rewrite. Safety passes each alternative unchanged, applying the quasi bridge only to its third case. In robustness the system already supplies rooted endpoints, so only the unrooted alternative is impossible; pass the stuck alternative or bridge the matching quasi. Instantiate the existing counting-simulation theorem with these results. □

Lean: `quasi_bridge`

Lean: `inputSafe_bridge`

Lean: `robust_bridge`

Lean: `countingSimulation`

Definition 52.31 (The converse semantic hypotheses)**.** A converse quasi-specimen uses the same original witness, with its right endpoint first and its left endpoint second. Converse robustness quantifies over successors of the original right endpoint, and uses the same three alternatives with the two witness endpoints exchanged. No symmetry of the pre-template or cost order is assumed.

Lean: `ConverseQuasi`

Lean: `ConverseRobust`

Lemma 52.32 (Converse bridges and counting simulation)**.** *The converse concrete predicates give the opposite specimen system its quasi and robustness laws. Together with input safety for Q^{-1} they yield reverse counting simulation with the required original cost orientation.*

Proof. The witness is unchanged; exchange its endpoint roles in the rooted conclusion. Eliminate only the impossible unrooted robustness alternative. Apply the already proved converse counting-simulation theorem with the concrete safety and converse robustness bridges. □

Lean: `converseQuasi_bridge`

Lean: `converseRobust_bridge`

Lean: `countingSimulation_converse`

52.4 Independent observations and the final theorem

Definition 52.33 (Unbounded independent observations)**.** An observation context is an original independent context at any finite depth. To observe a fragment vector, independently fill and initialise that context using the previous construction, then take the image of the resulting finite state class in the union. Neither context membership nor observation mentions a pre-template or a specimen.

Lean: `ObservationContext`

Lean: `observations`

Lemma 52.34 (Concrete global initial lifting)**.** *Independent observations of related fragment vectors in an allowed context construct a remote rooted specimen whose unbounded endpoints are the observed states.*

Proof. Both observations provide actual finite class witnesses at the context depth. Apply the existing concrete initial-lifting theorem with the finite restriction of global behaviour and the derived finite refocusing property. Retain its specimen, remote and rooted proofs, and project the endpoint equalities into the union. □

Lean: `initial_lifting`

Lemma 52.35 (Both contextual sufficiency clauses on the union)**.** *Under output closure, allowed-context invariance, refocusing, determinism and a reasonable triple, concrete global safety and robustness imply contextual refinement of related vectors. Safety for converse costs and converse robustness imply the reverse refinement. Supplying both sets of hypotheses gives contextual equivalence.*

Proof. Instantiate the abstract contextual theorems with the proved global specimen system, independent observations and the constructed initial-lifting proof. Global determinism and the concrete semantic bridges discharge every machine and interface premise. Pair the forward and reverse results for equivalence. □

Lean: `sufficiency`

Lean: `sufficiency_converse`

Lean: `equivalence`

Definition 52.36 (Contextual refinement of depth-independent fragments)**.** For global graph classes G, H of the same ordered interface, quantify over every hole name, every finite depth and every pair of typed representatives interpreting to G, H. Require unbounded contextual refinement of their singleton fragment vectors in every original context of the independent observation interface. Quantifying over all representations permits as much unused depth as any context requires; no depth bound is imposed on execution.

Lean: `PaperRefines`

Lemma 52.37 (Existence of common representations)**.** *A typed representative can be moved to every larger finite bound without changing its global interpretation. Every pair of global fragment classes therefore has typed representatives at a common bound and at every larger bound.*

Proof. Write the bound difference as a natural number and induct, applying interpretation invariance under raising at each successor. For a pair of global classes use interpretation surjectivity to choose representatives at bounds n, k, then raise each to any bound at least $n + k$. This prevents the universal representative formulation from holding merely because no representatives exist. □

Lean: `representative_at`

Lean: `common_representatives`

Lemma 52.38 (The two clauses for global pre-templates)**.** *For a depth-independent pre-template, the forward and converse contextual sufficiency implications hold for its related global fragment classes. Both hypotheses together give the two directed contextual refinements.*

Proof. For arbitrary common-depth representatives, rewrite their interpretation equalities in the global pre-template relation. Its layered interpretation then relates the actual graphs. The singleton-vector relatedness equivalence supplies the premise of the unbounded contextual sufficiency theorem. Apply the converse theorem with endpoints reversed for the second clause, and pair the two results. □

Lean: `paper_sufficiency`

Lean: `paper_sufficiency_converse`

Lean: `paper_equivalence`

Lemma 52.39 (A global set of contexts)**.** *A set of depth-independent context classes induces finite context membership by interpretation. Membership is automatically invariant under ordinary graph isomorphism and under increasing depth allowance.*

Proof. Interpretation maps a recursive graph isomorphism to equality of global classes, so ordinary isomorphism also preserves membership. Interpretation invariance under raising supplies padding invariance. □

Lean: `globalAllowed`

Lean: `globalAllowed_iso`

Lean: `globalAllowed_padding`

Lemma 52.40 (Unbounded sufficiency of robustness)**.** *For one global context set, one global pre-template, deterministic and refocusing global behaviour, output closure, and a reasonable triple (Q, P, R), both implications hold: input safety for (Q, P) and robustness for (Q, P, R) imply forward contextual refinement; input safety for (Q^{-1}, P) and converse robustness imply reverse contextual refinement.*

Proof. Apply the two preceding global-pre-template theorems. The allowed-context invariance premise is now discharged by interpretation of the global context set. All operational compatibility, exact-cost transfer, rootedness, determinism extension, specimen locality, semantic bridges and initial lifting have been proved. Safety, output closure and robustness remain the actual mathematical hypotheses of sufficiency, not additional postulated locality certificates. □

Lean: `theorem6`

53 Copying and store: foundations for Proposition 12.2

53.1 Scope, dependency map, and semantic boundary

The new namespace `Hypernets.Copying` begins the extension needed for Proposition 12.2 of the PDF (Proposition 12 in the HTML). The requested endpoint is the observational equivalence of the private-cell program and the constant program in every distinct-list environment. That endpoint is *not yet proved*. In particular, the new syntax is not an operational semantics, and its substitution equations are not unrestricted call-by-value beta rules.

The following dependency map distinguishes checked prerequisites from pending semantic work. The paper references use the PDF numbering, with HTML aliases where different.

1. **Checked foundations:** Figure 13 syntax and substitution algebra; term/atom wire distinction and box interfaces from Sections 9–10; concrete canonical trees of Definition 10.1; recursive renaming, addressed replacement, and preservation of global label properties. Ordered distributors, parallel assembly and serial assembly are now also checked; see Section 54. Arbitrary contraction trees, root termination, and uniqueness of atom-store resolution are proved in Section 55. All Figure 16 clauses, the full recursive translation, and global alphabet properties are checked in Sections 56–57. Copyable and stable graphs (Definitions 10.2–10.3), their structural properties and recursive-isomorphism invariance are proved in Section 58.

2. **Graph correspondence:** context translation is defined and its constructor interpretation laws are checked. Actual filling is proved for every original annotated context, with cumulative padding, sole-hole and binding-free certificates (Section 66). Padding preserves and reflects stability and preserves copyability (Section 70). Distributor resource permutations are proved in Section 72; this is a component of the renaming argument, not whole-term renaming. Section 73 adds a separate typed boundary-isomorphism interface and the original distributor's explicit resource automorphism. Selected-resource and all atomic bijective-renaming cases are proved (Section 74). Full-syntax bijective renaming is now proved by explicit BPerm derivations (Section 78); its semantic interpretation, non-bijective graph renaming, graph-level substitution and the remaining operational invariants remain pending. Recursive label safety and stable-input inertness are now proved in Section 89.

3. **Checked search, pending full machine:** operation search and the extra search rules (Figure 18) now have concrete incidence rules, determinism, typing, value recognition, exact isomorphism/padding transport and shallow plugging preservation (Section 79). Reference creation (Figure 19(a)) now has its actual stable-argument pair, freshness and resolution laws, and a search/allocation execution subsystem (Section 80). Dereferencing (Figure 19(b)) now has a general local rule for arbitrary atom contraction trees, store-resolution preservation and an execution to copy request (Section 81). The copy rule (Figure 17) now has a literal finite-graph construction, local execution, safety, renaming, forward padding and shared-store resolution (Section 82). Stable assignment (Figure 19(c)) now has a concrete local rule, preserved store resolution and old contents, checked execution, safety, renaming and forward padding (Section 83). Both equality-testing cases (Figure 19(d)) now have concrete rules, exact local runs, safety, representation laws and context-stable store-identity outcomes (Section 84). Stable micro-beta application now has a concrete rule and quantitative local executions (Section 85). All six local machines now embed in one shallow context-closed subsystem, with exact run costs and term-token typing. Arithmetic rules now extend the same relation, with five-step unary and seven-step binary executions. Its concrete allowance-indexed execution has exact finite-run correspondence and proved recursive isomorphism invariance (Section 86). Every combined run and closed observation pads forward at exactly

the same cost (Section 87). Every padded match and run now also reflects at exactly the same cost; copyability reflection and preservation of representability are proved from the concrete graph rules (Section 88). Canonical rule/operand selection is proved for independently chosen matches in arbitrary hosts; so are copy-head and read/write-store identity, disjoint equality branches, and recursive runtime safety for complete finite runs (Section 89). Allocation now has complete successor-state coherence for independently chosen contextual matches, with exact-cost consequences and a unique successful cost for the literal translated reference expression (Section 90). Both reference-equality branches now have complete successor-state coherence, including arbitrary hosts, whole store attachments and token positions. Their full-machine successful cost is exactly seven with arbitrary contents and finite padding (Section 91). Assignment now has complete successor coherence too, retaining the store, updated payload, discarded old contents and token position. Typed target exchange commutes with arbitrary context formation; stable writes have exactly seven successful steps with arbitrary attached contents and translated-data padding regressions (Section 92). Dereference now also has complete contextual successor coherence, closing this obligation for all store-operation heads. Its exact cost law retains the five-step read prefix and the subsequent copying continuation, including unchanged translated data at every finite padding (Section 93). Arithmetic now has complete contextual successor coherence, exact five- and seven-step successful costs, and an exact seven-step handoff to an arbitrary host continuation (Section 94). Application has complete contextual successor coherence, including independently represented bodies and arbitrary contexts, with an exact six-step handoff (Section 95). Copying now completes the final successor comparison, including independent tree boundaries, complete recursive payloads, original distributors and retained hosts. The full finite machine has successor determinism up to recursive state isomorphism, and the unbounded machine up to its representation relation. Exact prefix costs and unique successful-run lengths are proved without extra machine-law premises (Section 96).

Section 97 now proves refocusing for search, every actual contextual copying rewrite, and every actual allocation. Copying replays the original search prefix with exactly its final trigger removed. Exact search history, including backtracking, also proves refocusing for replacement of a single operation head. Rootedness is preserved and reflected by recursive representation and finite padding. Full finite and unbounded refocusing is reduced equivalently to dereference, assignment, equality, application and arithmetic; those cases, focused-state correspondence and the specimen interface remain unfinished. The proved determinism up to representation and these refocusing results must be integrated with the concrete specimen laws for the robustness theorem. Copy transitions replace the linear machine's substitution transitions; they must not be hidden in its existing compute parameter.

4. **Contextual interface:** binding-free closure at sourced holes, binding-freedom of all translated term contexts, sole-hole certificates and addressed context composition are checked. Graph-context congruence has an explicit execution-isomorphism premise, now discharged for the combined subsystem, including arithmetic. Observation reflection and the cofinal allowance criterion from Section 67 now have both execution premises discharged. Section 88 instantiates full term-context congruence with the concrete execution, without an equal-minimum-depth restriction. These representation laws must be preserved through any semantic correction required by the remaining full-machine and paper-correspondence obligations of Section 1. Unrestricted zero-input-hole closure is refuted, as recorded in Appendix B.1.

5. **Pending templates:** Table 3 requires contraction associativity, commutativity and idempotence; copying and garbage collection; box permutation, pulling contractions and pulling

weakening; micro-beta; reference creation; and parametricity. Their safety and robustness proofs use the transfer results of Appendices G–H. Prove base structural templates before the dependent copying, box-pulling and parametricity arguments; do not assume their contextual consequences in a circular proof.

6. **Pending assembly:** the semantic weakening laws (Proposition 12.1; HTML Proposition 11), the empty-environment chain of Figure 22, and extension to arbitrary environments (Proposition 12.2; HTML Proposition 12). The copying and garbage-collection templates are indirect dependencies, as Table 4 records.

The final public theorem is reserved as `Copying.Examples.proposition12_2`; no placeholder theorem with that name is introduced. Definitions and theorems below are reusable prerequisites rather than a conditional assertion of the requested conclusion.

53.2 Scoped terms and the two programs

Constants are naturals, booleans and unit. Unary syntax contains numeric negation, allocation and dereference; binary syntax contains addition, subtraction, assignment and atom equality. A term $t \in \mathsf{Term}(k, h)$ has k available variables and h available atoms. Variables and atoms are finite indices in separate namespaces. A lambda body has $k + 1$ variable slots; the newly bound variable has index zero. Both children of application and binary operations use the same environments, so variable sharing is permitted. These are formation rules, without an evaluation relation.

Lean: `Syntax.Constant`

Lean: `Syntax.Unary`

Lean: `Syntax.Binary`

Lean: `Syntax.Term`

Lifting a renaming fixes index zero and maps each successor index by the old renaming followed by successor. The zero and successor equations hold by definition. Simultaneous renaming recursively acts on variables and atoms; only the variable map is lifted under lambda. Non-injective maps are allowed, as identifying variable names is a legitimate substitution operation here.

Lean: `Syntax.lift`

Lean: `Syntax.lift_zero`

Lean: `Syntax.lift_succ`

Lean: `Syntax.rename`

Lifting preserves identity and composition: split a finite index into zero or successor and both statements reduce to equality by definition. Structural induction then proves identity and composition for renaming all terms. The lambda cases use the corresponding lifting identities; every other constructor uses the induction hypotheses on its children.

Lean: `Syntax.lift_id`

Lean: `Syntax.lift_comp`

Lean: `Syntax.rename_id`

Lean: Syntax.rename_comp

Weakening inserts a fresh variable at the front, shifting every old variable index by one and leaving atoms unchanged. The let form is application of a lambda to its bound expression. Sequencing $u; t$ is this let form with weakened t, ensuring the fresh sequencing binder does not capture any variable in t.

Lean: Syntax.weaken

Lean: Syntax.letIn

Lean: Syntax.seq

A lifted substitution maps zero to the new bound variable and maps a successor to the weakened old replacement. Substitution recursively replaces variable leaves with terms, renames atom leaves, and lifts the replacement environment under lambda. Replacements are not substituted into again by this same operation. This is capture-avoiding simultaneous syntactic substitution.

Lean: Syntax.liftSub

Lean: Syntax.subst

Lifting a substitution consisting only of variable leaves equals lifting the underlying renaming. Splitting the index proves this equation. Induction on terms then identifies substitution by variables with renaming. Its identity instance follows from the previously proved renaming identity.

Lean: Syntax.liftSub_var

Lean: Syntax.subst_var

Lean: Syntax.subst_id

The definitions `privateCell` and `constantResult` encode the two programs in every scope (k, h). In the desugared left program, the dereferenced x has index two beneath the binders for f and the sequencing variable. The two desugaring theorems are definitional equalities. Neither program uses an ambient variable or atom: applying any ambient renaming produces exactly the same program in the target scope. These equalities also hold by reduction of the syntax and lifting definitions, not by an observational argument.

Lean: Syntax.privateCell

Lean: Syntax.constantResult

Lean: Syntax.privateCell_desugar

Lean: Syntax.constantResult_desugar

Lean: Syntax.privateCell_rename

Lean: Syntax.constantResult_rename

A named environment consists of ordered variable and atom lists with separate no-duplicates proofs. The two lists need not be disjoint: they designate separate namespaces. Their lengths determine the indices of the scoped term type. This retains the distinct-list presentation of the requested statement without requiring fresh printed binder names inside the implementation.

Lean: Syntax.Environment

53.3 Substitution algebra

Renaming a weakened term under a lifted renaming equals weakening its renamed term. Expand weakening, apply renaming composition, and compare the two maps pointwise. Splitting zero/successor lifts this identity to substitution environments. Structural induction now shows that renaming after substitution renames every replacement and composes the atom maps. The lambda case is exactly the lifted-environment identity.

Lean: `Substitution.rename_weaken`

Lean: `Substitution.rename_liftSub`

Lean: `Substitution.rename_subst`

Reindexing a lifted replacement family equals lifting its reindexed family; zero and successor prove the assertion directly. Consequently substitution after renaming simply reindexes the replacement family and composes atom maps. The proof again follows term induction, using the family equation under lambda.

Lean: `Substitution.liftSub_rename`

Lean: `Substitution.subst_rename`

Substitution under a fresh binder commutes with weakening. This follows by applying the two preceding interchange laws to the successor renaming. Splitting an index lifts the result to replacement families. Finally, two successive substitutions equal one substitution whose replacements are themselves substituted, and whose atom map is the composite. Induction on the original term proves this associativity law; the lambda case uses the lifted-family law. Thus the binder cases have been discharged, rather than left as freshness assumptions.

Lean: `Substitution.subst_weaken`

Lean: `Substitution.subst_liftSub`

Lean: `Substitution.subst_comp`

Instantiation replaces variable zero with a supplied term and keeps older variables, shifting them down by one. Instantiating a weakened term returns the original term, by substitution–renaming interchange and substitution identity. Instantiating variable zero returns the supplied term by definition. These syntactic identities do not license reducing a call-by-value application before its argument is a value.

Lean: `Substitution.instantiate`

Lean: `Substitution.instantiate_weaken`

Lean: `Substitution.instantiate_zero`

53.4 Copying signatures and finite boxed graphs

Wire types are term, atom, and a thunk annotated by its number of bound variables. Sharing sorts are only term and atom; the inclusion into wire types therefore cannot select a thunk. Operations comprise all constants, lambda, application, and all unary/binary syntax operations. Constants and lambda are passive; the remaining operations are active. Every operation has one term input. Outputs are empty for constants, a one-binder thunk for lambda, two terms for application/binary operations, and one term for unary operations.

Lean: `Hierarchy.Wire`

Lean: `Hierarchy.ShareSort`

Lean: `Hierarchy.ShareSort.wire`

Lean: `Hierarchy.Operation`

Lean: `Hierarchy.Operation.kind`

Lean: `Hierarchy.Operation.outputs`

Holes have names and ordered typed interfaces. Primitive edges are operations, contractions, weakenings, instances, stores, focuses and holes. Contraction has two inputs and one output of its sharing sort; weakening has no inputs and one output. An instance has type $\star \Rightarrow \diamond$, and store has the opposite type. Focus has type $\star \Rightarrow \star$. The distinction between term and atom contractions is present in the labels; no copy rule for atom contractions is supplied. There is no primitive linear variable edge.

Lean: `Hierarchy.Hole`

Lean: `Hierarchy.Primitive`

Lean: `Hierarchy.primitiveSignature`

The environment interface lists all term outputs before all atom outputs. An alphabet is a label type with its signature. A box records a bound-variable count, a free-variable count, a free-atom count, and a body whose outputs are bound variables, free variables, then free atoms. Its external input is the appropriate thunk, and its external outputs omit the bound variables. The zero-depth alphabet contains primitives; a successor alphabet adds boxes whose bodies use the previous alphabet. A hypernet is the existing finite interfaced graph over this alphabet, retaining every incidence and typing law.

Lean: `Hierarchy.environmentInterface`

Lean: `Hierarchy.Alphabet`

Lean: `Hierarchy.Box`

Lean: `Hierarchy.baseAlphabet`

Lean: `Hierarchy.nextAlphabet`

Lean: `Hierarchy.alphabet`

Lean: `Hierarchy.Hypernet`

Primitive labels embed into every finite alphabet. Both signature equations are proved by splitting the depth into zero or successor. The primitive graph is the existing one-edge corolla

at the embedded label; those equations give its stated interface without changing its incidence.

Lean: `Hierarchy.primitiveLabel`

Lean: `Hierarchy.primitive_input`

Lean: `Hierarchy.primitive_output`

Lean: `Hierarchy.primitiveNet`

The operation predicate recognises only primitive operation labels. Their sources have length one, by graph typing and case analysis on labels. Hence the existing generic operation-path uniqueness theorem applies to copying hypernets as well: two operation paths to the same vertex from inputs have the same edges and the same starting input. A contraction is explicitly not an operation, so its two sources do not violate this argument.

Lean: `Hierarchy.IsOperation`

Lean: `Hierarchy.unary_operations`

Lean: `Hierarchy.operation_path_unique`

Lean: `Hierarchy.contraction_not_operation`

Boxing constructs a one-edge graph with the given typed body. Raising increases the nesting allowance, relabelling primitives and recursively raising box bodies while retaining all three counts. It changes neither outer carriers nor incidence. Heterogeneous equalities of source and target functions follow by splitting the old depth; the heterogeneous formulation accommodates their dependent carrier types.

Lean: `Hierarchy.box`

Lean: `Hierarchy.raise`

Lean: `Hierarchy.raise_source`

Lean: `Hierarchy.raise_target`

Deep edge occurrences are either shallow edges or a pair consisting of an outer box edge and a child occurrence. Equal box labels at different edges therefore still give different occurrences. Sum and dependent-sum enumeration construct an explicit finite enumeration recursively. Every occurrence belongs to it by the enumeration's completeness field.

Lean: `Hierarchy.AllEdge`

Lean: `Hierarchy.allEdgesFinite`

Lean: `Hierarchy.every_edge_enumerated`

53.5 Recursive isomorphism and addressed replacement

Box relatedness requires identical bound, free-variable and free-atom counts, and related bodies at that exact interface. Label equivalence is literal on primitives and recursive graph isomorphism on box bodies. Mixed primitive/box labels are unrelated. A deep isomorphism uses the generic incidence-isomorphism structure with this label relation; it preserves ordered boundaries at every level while permitting independent internal renaming.

Lean: `RecursiveIso.BoxRelated`

Lean: `RecursiveIso.LabelEquiv`

Lean: `RecursiveIso.DeepIso`

Induction on depth proves reflexivity, symmetry and transitivity of label equivalence. Primitive cases are the corresponding equality laws. In a box case, use the induction hypothesis to build, reverse or compose the body's incidence isomorphism. Incompatible label cases are impossible.

Lean: `RecursiveIso.labelEquiv_refl`

Lean: `RecursiveIso.labelEquiv_symm`

Lean: `RecursiveIso.labelEquiv_trans`

The graph-level identity, inverse and composition apply the generic isomorphism constructors with the three label laws just proved. An ordinary literal-label isomorphism embeds into a deep isomorphism using reflexivity of label equivalence.

Lean: `RecursiveIso.refl`

Lean: `RecursiveIso.symm`

Lean: `RecursiveIso.trans`

Lean: `RecursiveIso.ofIso`

Related labels have equal input and output signatures. For primitives this follows by applying the signature function to label equality. For boxes the relatedness constructor identifies all interface counts. The result follows by cases, with mixed labels eliminated by contradiction.

Lean: `RecursiveIso.labelEquiv_input`

Lean: `RecursiveIso.labelEquiv_output`

Replacement of a selected box changes just its body, preserving its metadata and all outer incidence. A typed site is either a shallow selected edge or a site recursively inside a selected box. Its indices carry the replacement's local allowance and interface. Addressed plugging performs the existing finite graph substitution at a shallow site and reconstructs enclosing boxes at an inner site. It is defined for arbitrary selected edges, not only holes.

Lean: `Plugging.replaceBox`

Lean: `Plugging.Site`

Lean: `Plugging.plugAt`

Shallow addressed plugging equals ordinary graph plugging by definition. Inside a box, source and target functions of the outer graph remain unchanged, also by definition. Every other outer

label is unchanged: unfold replacement and use the inequality of the selected and retained edge. Equal contents of distinct boxes do not trigger simultaneous replacement.

Lean: `Plugging.plugAt_shallow`

Lean: `Plugging.plugAt_inside_source`

Lean: `Plugging.plugAt_inside_target`

Lean: `Plugging.plugAt_inside_other_label`

53.6 Global predicates and closure

A predicate on primitive labels extends recursively through every box body. The graph predicate requires it on all outer labels and therefore at every depth. Focus freedom excludes all focus modes, and hole freedom excludes all hole labels. These are global predicates, not shallow tests.

Lean: `Predicates.LabelAll`

Lean: `Predicates.AllLabels`

Lean: `Predicates.FocusFree`

Lean: `Predicates.HoleFree`

A graph class is closed under replacement when replacing any well-typed addressed site of one member by another member produces a member. This is stronger than closure only at holes. Global label predicates satisfy it: at a shallow site every resulting edge comes from either the host or the replacement; inside a box apply the induction hypothesis to the selected body and leave other labels unchanged. Focus-free and hole-free closure are the two immediate instances. No binding-free path condition is asserted here.

Lean: `Predicates.ClosedUnderReplacement`

Lean: `Predicates.allLabels_plugAt`

Lean: `Predicates.focusFree_closed`

Lean: `Predicates.holeFree_closed`

Raising preserves and reflects a global label property. Induction on depth reduces the primitive case to identity and the box case to the induction hypothesis. Recursive label equivalence likewise preserves and reflects the property: map edges through the body's bijection, using its inverse for the reverse coverage direction. Applying this label fact to the outer edge bijection proves invariance under deep graph isomorphism.

Lean: `Predicates.allLabels_raise_iff`

Lean: `Predicates.labelAll_equiv`

Lean: `Predicates.allLabels_iso_iff`

53.7 The ordered canonical contraction tree

For m inputs, internal vertices are $0, \ldots, m-1$ and edges are $0, \ldots, m$. Edge zero is weakening. If $m = 0$, it targets the output. Otherwise edge one is the root contraction, fed on the left by the weakening. The other contractions form the ordered right spine from Definition 10.1. For $e > 1$, its left source is input $e - 2$; its right source is the next internal vertex unless it is the last contraction, whose right source is input $m - 1$. Thus the one-input case still contains both edges. These ordered sources follow the PDF figure, rather than merely choosing an arbitrary binary tree with the same leaves.

Lean: `CanonicalTree.Vertex`

Lean: `CanonicalTree.node`

Lean: `CanonicalTree.sources`

Lean: `CanonicalTree.label`

Distinct edges have distinct target nodes: inspect the empty tree, weakening, root and remaining spine cases. No target is an input. The two sources of a contraction are distinct, and a vertex cannot be sourced by two different edges; the proofs inspect the ordered source cases and compare finite indices. Every non-output vertex occurs as a source: choose the root for the weakening vertex, the matching spine edge for other internal vertices, and the appropriate contraction for each input. Every non-input vertex is the target of its indexed edge. Finally all vertices have the sharing sort's label, including boundary vertices, by inspection of the repeated interface lists.

Lean: `CanonicalTree.node_injective`

Lean: `CanonicalTree.node_not_input`

Lean: `CanonicalTree.sources_nodup`

Lean: `CanonicalTree.source_unique`

Lean: `CanonicalTree.source_coverage`

Lean: `CanonicalTree.target_coverage`

Lean: `CanonicalTree.vertex_label`

The tree graph packages these finite carriers and incidence functions. The preceding results discharge uniqueness and coverage. Source and target typing follow because every vertex has the same sharing label and the primitive signature has the corresponding arity. The label characterisations prove that edge zero is the unique weakening and all remaining m edges are contractions.

Lean: `CanonicalTree.net`

Lean: `CanonicalTree.weakening_iff`

Lean: `CanonicalTree.contraction_iff`

Two tree edges are adjacent exactly when they are weakening followed by the root, or consecutive edges in the upward right spine. Unfold singleton target membership and the source cases

to prove this equivalence. Assign weakening rank zero and a nonzero edge e rank $m + 1 - e$. Each adjacency strictly increases rank, by the just-proved characterisation and arithmetic.

Lean: CanonicalTree.adjacent_iff

Lean: CanonicalTree.rank

Lean: CanonicalTree.adjacent_rank_lt

Induction on the path list proves that earlier positions have strictly smaller rank than later positions. At its head, combine the first adjacency with the induction hypothesis on the tail; two tail positions reduce directly to that hypothesis. Equal edges at different positions would then have strictly ordered equal ranks, a contradiction. Hence every path has distinct edges. This acyclicity is a theorem for these trees, not an axiom imposed on general store graphs. Finally signature-preserving relabelling embeds the tree into any finite copying alphabet.

Lean: CanonicalTree.path_rank_lt

Lean: CanonicalTree.path_nodup

Lean: CanonicalTree.atDepth

53.8 Regression examples and what they establish

The first regression substitutes an open variable beneath a lambda and checks that its index is shifted rather than captured. The second inserts an atom beneath a lambda and checks that the atom index is not shifted. Both equations are proved by evaluating the recursive syntax definitions.

Lean: Examples.substitution_avoids_capture

Lean: Examples.atom_not_captured

The one-input regression checks the ordered sources: weakening first, input second. The two-input regression checks the lower contraction's ordered input ports. The three-input regression checks weakening-to-root and both spine adjacencies, and excludes the reverse edge. The first two reduce directly; the last uses the adjacency characterisation and finite arithmetic.

Lean: Examples.unary_tree_sources

Lean: Examples.binary_tree_sources

Lean: Examples.ternary_tree_adjacency

The boxed-instance graph has no bound or free term variables and one free atom. Its interface is a zero-binder thunk to an atom output. Its body is an instance corolla. Inspection of the sole primitive inside the box proves both global focus freedom and global hole freedom.

Lean: Examples.boxedInstance

Lean: Examples.boxedInstance_focusFree

Lean: Examples.boxedInstance_holeFree

For every named distinct-list environment, both displayed programs are the renamings of their empty-environment syntax. Apply the two program renaming theorems to the unique maps from empty index sets. This does not prove the semantic weakening laws, which compare

translated graphs and remain on the dependency map. The new axiom audit checks these prerequisites separately; there is no assertion of Proposition 12.2 in this snapshot.

Lean: `Examples.programs_in_environment`

Binding-free closure audit. Section 59 completes positive-path correspondence, recursive-isomorphism invariance and plugging closure at sourced holes. All translated term holes satisfy the source condition. The checked zero-input counterexample qualifies unrestricted Lemma E.1; copying observations and lambda-context congruence are still pending.

The remaining translation dependency for context composition is discharged in Section 71: independently translated hosts compose by actual addressed plugging, with the required padding, and both the composite host and all its fillings agree with the original direct translation. Graph-level term renaming and substitution remain separate obligations.

The serial-renaming dependency is discharged in Section 75: typed boundary maps glue when their shared actions agree, with a converse and concrete distributor instances. Structural stability and nested binding-freedom transport are proved. The composite-term induction, including the box-body boundary correspondence, is still required; this milestone does not discharge concrete execution laws.

Unary translation renaming is now discharged by Section 76, including the mandatory single-use forests and all three unary operations. The remaining bijective cases are application, binary operations, and lambda boxes, not unary operations.

The remaining application and binary renaming dependencies are discharged in Section 77, including exact resource matching, the original shuffle, and independent padding to a common allowance. Every lambda-free term now has a public graph-renaming theorem. Lambda-box interface renaming and graph-level substitution remain separate obligations.

The lambda dependency now factors through the actual BPerm graph pair in Section 78. Its exact bound/free interface laws and abstraction-shell compatibility complete the full syntax induction. The remaining edge in the dependency map is semantic: interpret the explicit derivation using BPerm robustness and execution congruence. Non-bijective graph substitution and the copying-machine obligations remain open.

54 Ordered distributors and graph assembly

The next prerequisite is a concrete construction of the distributors of Definition 10.1. We assemble actual finite typed graphs, with the paper's ordered interfaces, and prove that assembly preserves incidence and paths. These results provide wiring for the future translation; they do not yet supply copying transitions, store evaluation, template robustness, or Proposition 12.2.

54.1 Parallel assembly with an ordered boundary

Let A be a finite index set and let $G_a : I_a \to O_a$ be typed graphs. A boundary specification consists of type-preserving bijections from the disjoint unions of their input and output ports to the chosen ordered interfaces J and K. Thus every component boundary port occurs exactly once. This supports interleaving, rather than merely concatenating interfaces.

The boundary structure records these bijections and their typing equations. The assembled vertex type consists of tagged internal vertices and the new external ports. The embedding tags an internal vertex by a and sends an external port through the appropriate boundary bijection. Its inverse recovers the tag and local port. The inverse equations follow by cases on internal, input, and output vertices and by the two boundary inverse laws.

Lean: `Parallel.Boundary`

Lean: `Parallel.Vertex`

Lean: `Parallel.embed`

Lean: `Parallel.vertices`

Embedding preserves whether a vertex is an input or output, as a direct case analysis shows. It is injective because the joint vertex map is a bijection. It preserves wire labels: internal labels are inherited, and external labels are preserved by the boundary specification.

Lean: `Parallel.embed_isInput`

Lean: `Parallel.embed_isOutput`

Lean: `Parallel.embed_injective`

Lean: `Parallel.embed_label`

For an edge (a, e), transport its incidence list through the component embedding. Injectivity preserves absence of repetitions. If two transported incidence lists contain the same vertex, the joint inverse first identifies the component tags and then the original vertex; local uniqueness therefore identifies the edges. For coverage, a local witness transports forward. Conversely, take the unique preimage of a global vertex and use local coverage there. This works for either source or target incidence by parameterising the vertex predicate.

Lean: `Parallel.incidence`

Lean: `Parallel.incidence_nodup`

Lean: `Parallel.incidence_unique`

Lean: `Parallel.incidence_coverage`

The assembled graph has edges and internal vertices given by dependent disjoint unions. Finite enumerations are formed from the enumeration of A and the component enumerations.

Source and target incidence use the preceding transport. Their uniqueness, coverage, and absence of duplicates are exactly the lemmas just proved. Typing follows by mapping the label-preservation equation over each incidence list. Thus this construction satisfies all finite-graph fields, including the exhaustive boundary laws.

Lean: `Parallel.net`

If two assembled edges are adjacent, a shared target/source vertex has preimages in both components. Injectivity forces the tags to agree. Within a component, adjacency is equivalent to the original adjacency: reflect the shared vertex using injectivity, or transport it forward. Induction on a nonempty path proves that all its edges have the same tag. A second induction proves preservation and reflection of paths under a component inclusion. Empty lists are excluded by the existing path definition; single-edge paths give the base case. Finally a label predicate holds of all assembled edges precisely when it holds of every component edge, since edges are tagged pairs and their labels are unchanged.

Lean: `Parallel.adjacent_tag`

Lean: `Parallel.adjacent_same`

Lean: `Parallel.path_tags`

Lean: `Parallel.path_same`

Lean: `Parallel.labels_iff`

54.2 The finite input permutation and distributor forest

For k resources each used m times, the distributor $D_{k,m}$ consists of k canonical trees. In zero-based indexing, resource i at use j has input position $i + kj$ and output position i. This is the ordered interface of Definition 10.1, not just an equality of interface sizes.

The grid index maps (i, j) to $i+kj$. Its bound follows from $i < k$ and $j < m$. The inverse sends n to $(n \bmod k, \lfloor n/k \rfloor)$. The existence of $n < km$ implies $k > 0$, so the division bounds are valid even though the construction allows an empty grid. The inverse equations use Euclidean division, $i \bmod k = i$, and $\lfloor i/k \rfloor = 0$. They give a bijection. The final remainder and quotient lemmas explicitly recover the resource and use coordinates.

Lean: `FinGrid.index`

Lean: `FinGrid.split`

Lean: `FinGrid.split_index`

Lean: `FinGrid.index_split`

Lean: `FinGrid.bijection`

Lean: `FinGrid.index_mod`

Lean: `FinGrid.index_div`

The distributor input functions are this grid bijection with the list-length casts needed by typed interfaces. Their inverse proofs reduce to the same remainder and quotient equations.

Each tree has one output, so the output bijection forgets its unique local output index. Both boundary label equations hold because all involved ports have the chosen sharing sort.

Lean: `Distributor.inputIndex`

Lean: `Distributor.inputSplit`

Lean: `Distributor.inputSplit_index`

Lean: `Distributor.inputIndex_split`

Lean: `Distributor.boundary`

Instantiate parallel assembly with k copies of the canonical m-input tree. The input- and output-position equations follow by unfolding the boundary maps. An edge (i, e) is a weakening exactly when $e = 0$, and is a contraction exactly when $e \neq 0$, by the canonical-tree label lemmas. Relabelling primitive labels embeds this graph into every finite box allowance; it does not create any boxes or operational transitions.

Lean: `Distributor.net`

Lean: `Distributor.input_position`

Lean: `Distributor.output_position`

Lean: `Distributor.weakening_iff`

Lean: `Distributor.contraction_iff`

Lean: `Distributor.atDepth`

A general rank argument supplies acyclicity. Suppose a natural-number rank strictly increases across adjacency. Induction on a path, using transitivity at its head, proves that the rank at any earlier position is less than the rank at any later position. If an edge occurred twice, compare its two positions; either strict ordering would make its rank less than itself. Hence every path has no repeated edge.

Lean: `PathRank.path_rank_strict`

Lean: `PathRank.path_nodup_of_rank`

The generic parallel lemmas show that adjacent distributor edges belong to the same tree and that their local edges are adjacent there. Therefore the canonical-tree rank remains strictly increasing on distributor adjacency. Apply the general rank theorem to obtain absence of repeated edges on every path. Interleaving boundary ports has introduced no path between resources.

Lean: `Distributor.adjacent_tag`

Lean: `Distributor.adjacent_same`

Lean: `Distributor.adjacent_rank_lt`

Lean: `Distributor.path_nodup`

When $k = 0$, there is no edge or internal vertex: either would require an element of the empty resource index set. When $m = 0$, each component has only its edge numbered zero, so every edge is a weakening. Thus zero uses does not silently discard resources. More generally, any

primitive-label property true of weakening and contraction holds throughout the depth-indexed distributor. Unfold its labels and split on whether the local edge number is zero. Since neither constructor is a focus or hole, this also proves global focus freedom and hole freedom for both term and atom distributors.

Lean: `Distributor.zero_edges`

Lean: `Distributor.zero_internal`

Lean: `Distributor.zero_uses`

Lean: `Distributor.allLabels_atDepth`

Lean: `Distributor.focusFree`

Lean: `Distributor.holeFree`

54.3 Serial composition through a shared interface

Given $G : I \to J$ and $H : J \to K$, serial composition uses a two-hole pattern whose internal vertices are exactly the ordered ports of J. The left hole has interface $I \to J$; the right hole has interface $J \to K$. Their hole identities differ even when their interfaces coincide.

The pattern has two edges. Its left edge runs from outer inputs to shared internal vertices; its right edge runs from those internal vertices to outer outputs. Enumerating the corresponding finite index lists establishes all incidence and typing laws. Distinct port constructors exclude false matches between outer and internal vertices. For every j in J, the left target port and right source port are the same internal vertex; the connection lemma proves this by evaluating the indexed incidence lists. The construction also handles an empty shared interface.

Lean: `Serial.hole`

Lean: `Serial.pattern`

Lean: `Serial.interface_connection`

Fill the left slot with G and the right with H, using the signature equalities to check their interfaces. Simultaneous expansion replaces both pattern edges and glues their boundary vertices at the pattern ports. This defines the composed graph; the pattern holes are no longer edges. For a primitive-label predicate, each expanded edge comes from one filling, so its label property is inherited from that filling. Specialise this fact to the two slots, then to focus freedom and hole freedom. These are structural closure laws, not a theorem that an operational reduction can cross an arbitrary surrounding context.

Lean: `Serial.filling`

Lean: `Serial.net`

Lean: `Serial.allLabels_expand`

Lean: `Serial.allLabels_net`

Lean: `Serial.focusFree_net`

Lean: `Serial.holeFree_net`

54.4 Concrete checks and remaining obligations

The input-order regression computes all six positions for three resources used twice: resource zero occupies positions $0, 3$, resource one $1, 4$, and resource two $2, 5$. The separation check rules out adjacency between the two distinct resources in a two-resource distributor by tag equality. A single-use tree still contains its weakening and contraction; it is not replaced by an identity. The zero-use check retains two distinct weakenings for two resources. These label calculations hold for both sharing sorts. Finally, serially assemble an instance edge from term to atom with a store edge from atom to term. The resulting term-to-term graph is focus-free and hole-free by the serial closure lemmas and inspection of the two labels. This last example checks atom-interface wiring, not allocation, reading, or writing in a running machine.

Lean: `AssemblyExamples.distributor_input_order`

Lean: `AssemblyExamples.distributor_separation`

Lean: `AssemblyExamples.distributor_single_use`

Lean: `AssemblyExamples.distributor_no_uses`

Lean: `AssemblyExamples.instanceStore`

Lean: `AssemblyExamples.instanceStore_focusFree`

Lean: `AssemblyExamples.instanceStore_holeFree`

The full build and all three axiom audits pass. The new audited declarations use only Lean's standard propositional extensionality, classical choice, and quotient soundness. No machine determinism, template robustness, or translation-correctness assumption has been introduced. Arbitrary contraction trees are now treated in Section 55. Copyability and stability are treated in Section 58. Context translation and the translation laws, and the operational and template arguments remain to be proved.

55 Arbitrary contraction trees and deterministic store tracing

The store rules of Figure 19 locate a store by following atom contraction trees. This section proves two prerequisites for that matching operation. First, every edge of an arbitrary finite acyclic contraction tree reaches its unique root. Second, two successful atom traces in the same host agree on the store they find. The second theorem does not assume that the host is acyclic: stored references may form cycles.

Neither result is a definition of reference creation, reading, assignment, or equality reduction. Nor do they prove that every atom in an arbitrary host reaches a store. They establish structural termination for contraction trees and uniqueness of successful store lookup, to be used by the future machine rules. They also do not prove the copying rule's independence of its chosen contraction-tree match.

55.1 Exact traces and finite termination

For a relation R, an exact trace $a\ R^n\ b$ consists of n successive steps. Its zero-step constructor ends where it starts; its step constructor prepends one step. A terminal element has no outgoing R-step. Here n counts graph adjacencies or atom hops, not machine transitions or the cost relation used in contextual refinement.

Mapping every element of a trace along a relation-preserving function preserves its exact length, by induction on the trace constructors. A bijection which preserves and reflects steps also reflects traces: map back through its inverse, use step reflection, and apply the two inverse equations to the endpoints. These lemmas will transfer store lookup across graph renaming and depth padding.

Lean: `FiniteTrace.Trace`

Lean: `FiniteTrace.Terminal`

Lean: `FiniteTrace.map`

Lean: `FiniteTrace.map_iff`

If R is functional, two traces starting at the same element and ending at terminal elements have equal endpoints and equal lengths. Induct on the first trace. A zero-step trace cannot compete with a positive trace, because its endpoint is terminal. If both traces take a step, functionality identifies the next elements; induction identifies the remaining endpoints and lengths.

For an arbitrary relation, bounded exploration either reaches a terminal or produces the requested number of steps. At bound zero take the reflexive trace. At a positive bound, stop if the current element is terminal; otherwise classical reasoning supplies a successor. Apply the induction hypothesis there and prepend the chosen step. This is a proof of existence, not an executable decision procedure for arbitrary relations.

Lean: `FiniteTrace.terminal_unique`

Lean: `FiniteTrace.terminal_or_long`

An adjacency trace of length n in a graph gives a path of $n + 1$ edges: induction prepends the starting edge at each step. If every path has no repeated edge, its length is at most the length of the graph's finite edge enumeration, since every path edge occurs in that enumeration. Repeated entries in the enumeration are harmless. Consequently the trace length is strictly less than the enumeration length.

Apply bounded exploration with the enumeration length as its bound. The alternative producing that many adjacency steps contradicts this strict inequality. Thus every edge in a finite edge-acyclic graph reaches a terminal edge. This proof does not assume a rank, a balanced tree, or a chosen canonical presentation.

Lean: `FiniteTrace.trace_path`

Lean: `FiniteTrace.trace_bound`

Lean: `FiniteTrace.exists_terminal_trace`

If an edge has exactly one target vertex, two adjacent successors must share that same vertex in their source lists. Source-incidence uniqueness then makes the two successors equal. This local fact supplies functionality both for contraction trees and for atom tracing in larger hosts.

Lean: `FiniteTrace.adjacent_functional_at`

55.2 Arbitrary contraction trees on concrete graphs

For sharing sort s and m exposed leaves, a contraction tree is a concrete finite graph $s^m \to s$ whose edges are all s-contractions or s-weakenings and whose paths have no repeated edge. The structure does not assume connectedness to the output; that is proved below. It does not require a weakening edge: the single binary contraction is a valid tree. This is broader than the canonical trees, which have exactly one weakening.

The structure records the finite graph, its label restriction, and the acyclicity proof. Both permitted labels have a single target by their signatures, so every tree edge has exactly one target vertex. The generic single-target lemma therefore proves that forward adjacency is functional.

Lean: `ContractionTree.ContractionTree`

Lean: `ContractionTree.target_single`

Lean: `ContractionTree.adjacent_functional`

An edge is terminal exactly when its target is the sole external output. For the forward implication, inspect its unique target vertex. If that vertex were not an output, source coverage would supply a successor edge, contradicting terminality. For the reverse implication, the unique target is an output, and outputs never occur in source lists, so no successor exists.

Target coverage supplies an edge producing the external output. Define the root to be that edge, using classical choice. Its output incidence follows from the choice specification, and the preceding equivalence makes it terminal. Any other terminal edge produces the same output, so target uniqueness identifies it with the root. Thus the chosen name has no effect on root uniqueness.

Lean: `ContractionTree.terminal_iff_output`

Lean: `ContractionTree.root`

Lean: `ContractionTree.root_output`

Lean: `ContractionTree.root_terminal`

Lean: `ContractionTree.terminal_eq_root`

Finite acyclic termination supplies a terminal trace from every edge. Its endpoint must be the root. Functionality additionally shows that any two terminal traces from that edge have the

same endpoint and length. For an exposed input, source coverage first supplies its consuming edge; then use that edge's root trace. The same argument applies to every vertex other than the external output. This derives the required connection of the entire tree to its root, rather than placing connectivity in the structure's assumptions.

Lean: `ContractionTree.reaches_root`

Lean: `ContractionTree.root_trace_unique`

Lean: `ContractionTree.input_reaches_root`

Lean: `ContractionTree.vertex_reaches_root`

A graph isomorphism transports the tree property. Labels are preserved, and a path in the renamed graph maps back to a path in the original tree. The latter has no repeated edge; equality of positions in the former would map to equality of positions in the latter, so the former is also without repetitions. Finally, each canonical tree is an instance: its earlier label classification supplies the sharing-label restriction, and its rank proof supplies acyclicity.

Lean: `ContractionTree.transport`

Lean: `ContractionTree.canonical`

55.3 Atom tracing in arbitrary hosts

An atom hop may leave an instance edge, an atom contraction, or an atom weakening and follows adjacency to the next edge. These three labels have one atom output. A successful store resolution is an exact sequence of such hops whose endpoint is labelled as a store. Unlike unrestricted graph adjacency, this relation stops at a store and does not follow its term output.

The forward-label predicate lists the three allowed labels at any box allowance. A hop is that predicate together with adjacency. The output signature of every forward label is the singleton atom interface, by cases on the predicate. Hence the target incidence list has length one, and single-target adjacency functionality proves uniqueness of the next hop. No hypothesis about cycles elsewhere in the host is used.

Lean: `StoreTrace.ForwardLabel`

Lean: `StoreTrace.Hop`

Lean: `StoreTrace.forward_output`

Lean: `StoreTrace.hop_functional`

The common vertex of a hop lies in the forward edge's target list. Target typing makes its label an atom; source typing therefore puts an atom in the successor edge's input signature. In a hole-free host this forces the next edge to be an atom contraction or a store. Inspect the alphabet: operations, instances, and focuses have term inputs; boxes have thunk inputs; weakenings have no inputs; term contractions have term inputs; holes are excluded by the global hole-free predicate. Thus the permitted successors are proved from typing and the alphabet, rather than assumed as a matching convention.

Lean: `StoreTrace.hop_atom_input`

Lean: `StoreTrace.hop_successor_kind`

A store is terminal for atom hopping because its output is a term, whereas all forward labels output an atom. A successful resolution records the trace and the final store label. Apply

the functional-relation terminal uniqueness theorem to two successful resolutions: both store endpoints and hop counts agree. Apply this result separately to two starting atoms to show that testing whether their resolved store edges are equal is independent of the chosen successful traces. This establishes the lookup premise needed by atom equality; it does not yet prove the equality compute rule.

Lean: `StoreTrace.store_terminal`

Lean: `StoreTrace.Resolves`

Lean: `StoreTrace.resolves_unique`

Lean: `StoreTrace.same_store_iff`

A graph isomorphism preserves labels and preserves and reflects adjacency, so it preserves and reflects atom hops. The exact-trace bijection lemma and preservation of the final store label then give equivalence of successful resolutions under renaming, including equality of the hop counts.

Lean: `StoreTrace.hop_iso_iff`

Lean: `StoreTrace.resolves_iso_iff`

Depth padding leaves the edge carrier unchanged; an explicit bijection exposes this fact despite the depth-indexed type. At depth zero, lifted primitive labels become primitive labels at depth one. At positive depth, primitive labels remain primitives and box labels remain boxes. Case analysis therefore preserves and reflects the forward-label predicate and the store-label predicate. Incidence lists are unchanged. These facts give hop equivalence, and exact-trace transport gives resolution equivalence at the larger allowance. Padding creates no new store lookup behaviour.

Lean: `StoreTrace.paddingEdges`

Lean: `StoreTrace.hop_raise_iff`

Lean: `StoreTrace.resolves_raise_iff`

55.4 Regressions: no weakening and a cyclic store

The weakening-free binary tree is the corolla consisting of one contraction. Its label condition is immediate. It has no adjacency because its targets are output ports and its sources are input ports. A constant rank therefore vacuously increases on adjacency and proves path acyclicity. The generic tree theorem applies to every edge of this tree and proves root reachability. This checks that the arbitrary-tree definition has not silently inherited the canonical tree's extra weakening.

Lean: `TracingExamples.binaryTree`

Lean: `TracingExamples.binaryTree_reaches`

The cyclic-store graph has two internal vertices, one term and one atom, and two edges. An instance runs from the term vertex to the atom vertex; a store runs from the atom vertex back to the term vertex. It has empty external interfaces. Boolean enumeration supplies finiteness, and direct case analysis verifies all typing, incidence uniqueness, and coverage fields. The

path consisting of instance, store, instance witnesses a genuine cycle, so global acyclicity is unavailable in this example.

Lean: `TracingExamples.cyclicStore`

Lean: `TracingExamples.cyclicStore_cycle`

Nevertheless, its instance resolves to its store in exactly one atom hop: the shared atom vertex supplies adjacency, and the endpoint has the store label. Any successful resolution of that instance must have the same store and length by the general uniqueness theorem. Both labels are different from every hole label, giving global hole freedom at every depth allowance. This checks that the tracing relation stops before following a stored value back into the cycle. It is a graph-level regression, not a machine execution or an allocation/read/write test.

Lean: `TracingExamples.cyclicStore_resolves`

Lean: `TracingExamples.cyclicStore_unique`

Lean: `TracingExamples.cyclicStore_holeFree`

All new declarations pass the full build and axiom audit with only Lean's standard axioms. Arbitrary contraction-tree construction and root tracing are completed prerequisites. Copyable and stable hypernets are now defined in Section 58. Context translation and the translation laws, actual copying and store transitions, their full matching and locality proofs, the templates, and Proposition 12.2 remain open.

56 Literal clauses of the paper's term translation

56.1 Fixed target and correspondence requirement

The terms of Proposition 12.2 remain exactly

$$\texttt{let } x = \texttt{ref } 1 \texttt{ in } \lambda f.(f();!x) \qquad \text{and} \qquad \lambda f.(f();1).$$

Their existing scoped syntax and desugaring definitions are unchanged. The formalisation must not replace either term by a different program, even an intended equivalent one, to make the theorem easier. The translation must follow Figure 16, including its canonical sharing trees, unused-resource weakening, environment order, and box interfaces. In particular, $D_{k,1}$ is not simplified to an identity: it contains a weakening and a contraction for each resource.

This section implements the variable, atom, constant, unary, and abstraction graph constructors literally as drawn in Figure 16. Section 57 now completes the application and binary clauses and the full recursive term translation. Translation laws for substitution and contexts and the observational equivalence proof remain open.

56.2 Ordered binary juxtaposition

For $G : I \to O$ and $H : J \to K$, ordered binary juxtaposition is a graph $G \otimes H : (I +\!\!+ J) \to (O +\!\!+ K)$, placing all left ports before all right ports. It is constructed by the previously proved finite parallel assembly, rather than by adding identity wires.

A Boolean chooses the left or right interface. The port map sends a left index i to i, and a right index j to $|I| + j$. Its inverse tests whether the combined index is below $|I|$ and, in the right case, subtracts $|I|$. The inverse equations follow by that case distinction and the index bounds; they define a boundary bijection. Looking up a wire label in the concatenated list gives the original component label, by the same left/right distinction.

Lean: `Tensor.interfaces`

Lean: `Tensor.join`

Lean: `Tensor.split`

Lean: `Tensor.split_join`

Lean: `Tensor.join_split`

Lean: `Tensor.ports`

Lean: `Tensor.join_label`

The family selects G or H. Apply the port bijections to both their input and output interfaces to obtain the parallel boundary specification. The tensor is the parallel construction for this two-element family, with an explicit Boolean enumeration. Its graph laws are inherited from that construction. Every tensor edge is a component tag and an original edge, with unchanged label, so a label predicate holds on the tensor exactly when it holds on both factors.

Lean: `Tensor.family`

Lean: `Tensor.boundary`

Lean: `Tensor.net`

Lean: `Tensor.labels_iff`

56.3 Interface equalities and environment distributors

An interface cast changes only provably equal type indices, leaving the graph unchanged. Its input and output equalities are propositional equalities of ordered wire lists, not a permission to permute ports. Eliminating these equalities proves that every global primitive-label property is preserved and reflected. Such casts reconcile, for example, $k \cdot 1$ with k and $1 + k$ with $k + 1$; they perform no term rewrite or graph simplification.

Lean: `TranslationClauses.castInterfaces`

Lean: `TranslationClauses.allLabels_cast`

Juxtapose $D^{\star}_{k,m}$ with $D^{\diamond}_{h,m}$. This gives input interface $\star^{km} \otimes \diamond^{hm}$ and output interface $\star^{k} \otimes \diamond^{h}$, preserving the variables-before-atoms order. The single-use constructor sets $m = 1$ and uses only an interface-equality cast. It retains every weakening and contraction in these distributors.

Lean: `TranslationClauses.environmentDistributor`

Lean: `TranslationClauses.singleUse`

56.4 The five implemented Figure 16 clauses

For resource i among n resources of sort s, construct the ordered juxtaposition

$$D^{s}_{i,0} \otimes D^{s}_{1,1} \otimes D^{s}_{n-i-1,0}.$$

It has one input of sort s and n outputs. The index bound $i < n$ proves $i + 1 + (n - i - 1) = n$, so the output cast preserves the exact order. Thus the selected resource is in position i, while all other resources retain weakening edges. The variable clause uses this construction at term sort, then juxtaposes the weakening distributor for all ambient atoms. This is the first diagram of Figure 16, with the two variable-environment portions represented by the prefix and suffix of the selected finite index.

Lean: `TranslationClauses.selectedResource`

Lean: `TranslationClauses.variableTerm`

The atom clause keeps the instance edge. Serially connect that edge to the selected atom-resource graph, and juxtapose weakening for all ambient term variables on the left. The instance produces the atom input of its $D^{\diamond}_{1,1}$; that tree is not omitted. Outputs remain variables followed by atoms. For a constant, juxtapose variable weakening, the constant corolla, and atom weakening in exactly the order shown in Figure 16. Interface arithmetic checks the sole term input and the full ambient output interface, including environments whose resources are unused.

Lean: `TranslationClauses.atom`

Lean: `TranslationClauses.constant`

For a unary operation, serially assemble the unary-operation corolla, the supplied translated subterm, and the term/atom single-use distributors. This follows the bottom-left diagram of Figure 16 for every unary operation, including reference creation and dereference.

For abstraction, the supplied body has one extra term output. Place that body in a box with one bound variable, k free variables, and h free atoms. The bound variable is the first body output, as in the existing scoped syntax. The equality of $k + 1$ and $1 + k$ reconciles the

numeric type indices; it does not reorder that body interface. The box has a $T^1(\star)$ input and exposes only the free environment. Serially assemble the lambda corolla, this box, and the two single-use distributors. Only this clause increases the box-depth allowance. These are graph constructors; they do not replace lambda or sequencing in the terms being formalised.

Lean: `TranslationClauses.unary`

Lean: `TranslationClauses.lambdaBox`

Lean: `TranslationClauses.abstraction`

56.5 Global alphabet and absence of focus and holes

The translation language permits operation, contraction, weakening, and instance primitives. It excludes store, focus, and hole primitives. A language graph satisfies this restriction recursively inside every box. Store edges arise later from execution of reference creation, not from translating the syntax of a reference-creation expression.

The primitive predicate explicitly enumerates the allowed constructors; the graph predicate lifts it through all boxes. For any primitive-label predicate, tensor closure follows from the tensor label equivalence. A primitive corolla satisfies the lifted predicate whenever its sole primitive label does, by inspecting the alphabet at zero or positive depth.

Lean: `TranslationClauses.LanguagePrimitive`

Lean: `TranslationClauses.LanguageGraph`

Lean: `TranslationClauses.allLabels_tensor`

Lean: `TranslationClauses.allLabels_primitive`

Every environment distributor is a language graph because each component contains only contraction and weakening edges. Interface casting gives the single-use case. Repeated tensor closure proves the selected-resource case. The variable case adds only atom weakening; the atom case adds term weakening and the permitted instance primitive; the constant case adds only its permitted operation primitive. These arguments establish the global alphabet property for every ambient scope, not just for closed terms.

Lean: `TranslationClauses.environmentDistributor_language`

Lean: `TranslationClauses.singleUse_language`

Lean: `TranslationClauses.selectedResource_language`

Lean: `TranslationClauses.variable_language`

Lean: `TranslationClauses.atom_language`

Lean: `TranslationClauses.constant_language`

The unary clause preserves the language property by serial closure, combining its operation, the supplied subterm property, and the distributor property. For the lambda box, the recursive label predicate is precisely the body's label predicate, after the harmless interface cast. Serial closure then proves the abstraction clause from the lambda primitive, that box, and the

single-use distributors. These are conditional preservation laws for constructor arguments, not unproved assumptions about the final programs.

Lean: `TranslationClauses.unary_language`

Lean: `TranslationClauses.lambdaBox_language`

Lean: `TranslationClauses.abstraction_language`

A language graph is globally focus-free and hole-free. To prove each claim, first lift the corresponding implication on primitive labels through the recursive alphabet by induction on depth. At depth zero and for a primitive at positive depth, equality with a forbidden label contradicts the language predicate. For a box, apply the induction hypothesis to each body edge. Apply this implication to every edge of the outer graph.

Lean: `TranslationClauses.language_focusFree`

Lean: `TranslationClauses.language_holeFree`

56.6 Checks that preserve the paper's graph structure

The reference-one graph assembles the literal constant and unary clauses for the unchanged subterm `ref 1`. It is a language graph in every ambient environment by the two preservation lemmas. This is not the whole left side of Proposition 12.2: the enclosing let and applications are assembled by the complete translation in Section 57.

Lean: `TranslationExamples.referenceOne`

Lean: `TranslationExamples.referenceOne_language`

Kernel-checked finite-enumeration calculations guard the unsimplified shapes. In an environment of two variables and one atom, a constant has four outer edges: one constant and three weakenings. Selecting the second of three variables with two ambient atoms gives six edges, including the selected weakening/contraction pair. Selecting the second of two atoms with three ambient variables gives seven, including the instance edge. The graph for `ref 1` with two variables and one atom has eleven edges: four for the constant clause, one for reference creation, and six for the single-use environment distributors. These are structural checks, not allocation or termination tests.

Lean: `TranslationExamples.constant_environment_edges`

Lean: `TranslationExamples.variable_environment_edges`

Lean: `TranslationExamples.atom_environment_edges`

Lean: `TranslationExamples.referenceOne_environment_edges`

The identity-abstraction regression supplies a variable graph at bound index zero as the boxed body. Its language property follows from variable and abstraction preservation. Its outer graph has exactly two edges, lambda and box, when the free environment is empty; the variable's sharing graph remains inside the box. Finally, the single-use distributors for two variables and one atom have six edges. This directly checks the feature that would be lost by replacing them with identity wiring.

Lean: `TranslationExamples.identityAbstraction`

Lean: `TranslationExamples.identityAbstraction_language`

Lean: `TranslationExamples.identityAbstraction_outer_edges`

Lean: `TranslationExamples.singleUse_retains_edges`

The full build and all three axiom audits pass with only standard Lean axioms. The original proposition terms and their desugaring files have not been modified. The outstanding translation and machine obligations remain explicit in Appendix B; Proposition 12.2 is not yet proved.

57 Complete recursive translation and the original proposition graphs

The Figure 16 graph construction is now implemented for the entire scoped term datatype, including application and every binary example operation. Both unchanged programs of Proposition 12.2 are translated in arbitrary variable and atom environments. The construction retains all canonical trees, weakenings, instances, boxes, and interface permutations prescribed by the paper.

This completes construction of the term translation, not the observational equivalence theorem. The translation's renaming and substitution laws beyond its depth calculation, translation of contexts and their composition, the copying/store machine, and the template robustness arguments remain open. The well-formedness result for the two proposition graphs is not a replacement for Proposition 12.2.

57.1 The middle-block permutation

The parallel subterms expose variables and atoms separately on each side: $(\Gamma, \Delta, \Gamma, \Delta)$. Figure 16 exchanges the middle blocks to obtain $(\Gamma, \Gamma, \Delta, \Delta)$ before the two-use distributors. We implement the more general typed permutation $(A, B, C, D) \mapsto (A, C, B, D)$, preserving each local port index.

First express concatenated ports by a disjoint sum: left ports have their original index, and right ports are offset by the left interface length. The existing ordered-tensor join and split maps prove the inverse laws. The two label lemmas state that looking up a left or right joined port returns its original wire label. Empty interfaces are included.

Lean: `InterfaceShuffle.ports`

Lean: `InterfaceShuffle.left_label`

Lean: `InterfaceShuffle.right_label`

On four disjoint summands, interchange sends an A index to A, a B index to the third block, a C index to the second, and a D index to D. Its inverse performs the reverse case distinction. Both inverse equations are immediate by the four cases. The original-port bijection groups the summands as $(A + B) + (C + D)$. The shuffled-port bijection first interchanges the summands and then groups them as $(A + C) + (B + D)$. Composing the inverse of the original bijection with the shuffled bijection gives the required permutation of finite interface positions.

Lean: `InterfaceShuffle.interchange`

Lean: `InterfaceShuffle.original`

Lean: `InterfaceShuffle.shuffled`

Lean: `InterfaceShuffle.bijection`

For each of the four component cases, repeated application of the left and right label lemmas shows that the original and shuffled positions have the same wire label. To prove this for an arbitrary original interface position, take its inverse image under the original-port bijection and use its right inverse equation. Thus the permutation is type preserving, not merely a bijection between interfaces of the same size.

Lean: `InterfaceShuffle.component_label`

Lean: `InterfaceShuffle.label`

57.2 Application and binary-operation clauses

The pair boundary leaves the two term inputs in left-to-right order. For outputs it first concatenates the two subterm interfaces, then applies the middle-block permutation with $A = C = \star^k$ and $B = D = \diamond^h$. Type preservation is the composition of the tensor and permutation label lemmas. Parallel assembly therefore constructs the actual two-subterm graph with input $\star \otimes \star$ and grouped outputs $\star^{2k} \otimes \diamond^{2h}$. The final cast uses only equalities of replicate-list lengths and concatenations; it does not perform another permutation or simplify an edge.

Lean: `BinaryTranslation.pairBoundary`

Lean: `BinaryTranslation.pair`

For application, serially assemble the application corolla, the paired subterms, and $D^{\star}_{k,2} \otimes D^{\diamond}_{h,2}$. For a binary operation use the corresponding operation corolla with the same wiring. This covers addition, subtraction, assignment, and atom equality. These are precisely the two right-hand constructions in Figure 16. In particular, both full ambient environments occur in the subterm graphs before sharing; the construction does not prune unused occurrences or weakenings.

Lean: `BinaryTranslation.application`

Lean: `BinaryTranslation.binary`

The pair's edge labels come from exactly one of its two subterms, so their global language predicates imply the pair's predicate. Interface casting preserves that predicate. Serial closure, the permitted operation label, and the proved distributor predicate establish the language property for both application and binary operations. These proofs inspect all nested box contents through the existing recursive label predicate.

Lean: `BinaryTranslation.pair_language`

Lean: `BinaryTranslation.application_language`

Lean: `BinaryTranslation.binary_language`

57.3 Common depth allowance without graph simplification

A finite representation records a maximum box allowance. Two subterms of an application need not have the same allowance, although they must be assembled in one alphabet. Padding raises the smaller allowance while preserving the graph incidence and recursively represented box contents. It adds no term constructor, sharing edge, or box.

A depth cast changes only provably equal numeric indices, and equality elimination proves preservation and reflection of all global label predicates. Repeated padding is defined by iterating the earlier one-level raising operation. Induction on the number of iterations proves label invariance, using the previously proved raising equivalence at each step. To pad a graph from allowance d to $n \geq d$, iterate $n - d$ times and cast along $d + (n - d) = n$. This gives padding to any chosen common allowance with the same global label properties.

Lean: `Translation.castDepth`

Lean: Translation.allLabels_castDepth

Lean: Translation.pad

Lean: Translation.allLabels_pad

Lean: Translation.padTo

Lean: Translation.allLabels_padTo

One-level raising leaves the outer edge enumeration unchanged, by cases on the original depth. Induction proves the same count equality for any number of padding steps. A depth equality cast also preserves that count, by equality elimination. Combining these results proves that padding to a common allowance allocates no additional outer edge. These counts are structural checks, not execution costs.

Lean: Translation.raise_edgeCount

Lean: Translation.pad_edgeCount

Lean: Translation.castDepth_edgeCount

Lean: Translation.padTo_edgeCount

The required allowance is zero for variables, atoms, and constants; one more than the body's allowance for abstraction; unchanged for unary operations; and the maximum of the two subterm allowances for application and binary operations. Induction on terms shows that variable and atom renaming preserves this number, including renaming under a binder. This is the depth part of a renaming law only; graph-level compatibility with general environment renaming remains a separate obligation.

Lean: Translation.depth

Lean: Translation.depth_rename

57.4 The full term translation

Translate recursively by the seven scoped term constructors. The leaf cases use the literal variable, atom, and constant clauses. Abstraction boxes the recursively translated body; unary operations use the unary clause. Application and binary operations first pad both recursive graphs to the maximum of their allowances, then use their Figure 16 constructors. The result type states the exact interface $\star \to \star^k \otimes \diamond^h$ for every term in scope (k, h).

Induction on the same syntax proves that every result is a language graph. Each leaf uses its clause theorem, abstraction and unary operations use their preservation laws, and the two-argument cases also use padding invariance before their preservation laws. The generic implications from the language predicate give global focus freedom and hole freedom. Thus these properties hold recursively inside every translated box, with no well-formedness assumptions on particular translated subterms left open.

Lean: Translation.term

Lean: Translation.term_language

Lean: Translation.term_focusFree

Lean: Translation.term_holeFree

The named-environment entry point accepts the original ordered distinct lists and a term scoped

by their lengths. It invokes exactly the same translation; it does not restrict the environments to be empty or identify variables with atoms. The two proposition graph definitions apply this translation directly to the existing, unchanged `Term.privateCell` and `Term.constantResult`. Their global language properties are instances of the general theorem. Neither graph definition contains an assumed observational equivalence or an alternate program.

Lean: `Translation.inEnvironment`

Lean: `Translation.privateCell`

Lean: `Translation.constantResult`

Lean: `Translation.privateCell_language`

Lean: `Translation.constantResult_language`

57.5 Permutation, sharing, and full-program regressions

For two variables and one atom in each subterm, the permutation sends positions $(0, 1, 2, 3, 4, 5)$ to $(0, 1, 4, 2, 3, 5)$. This checks the required movement of the left atom past the right variables. An application of two constant graphs in this environment has eighteen outer edges: one operation, four in each constant graph, and nine in the three two-use canonical trees. The same count is checked for every binary operation, including both store operations. These kernel-checked calculations would detect omission of the two-use distributor edges.

Lean: `FullTranslationExamples.binary_environment_permutation`

Lean: `FullTranslationExamples.application_environment_edges`

Lean: `FullTranslationExamples.binary_environment_edges`

The unchanged left program requires three nested abstraction levels, including the paper's let and sequencing sugar; the right requires two. Their closed translated graphs have respectively five and two outer edges. On the left these are the application, lambda, box, reference-creation, and numeral edges. On the right they are lambda and box. The remaining body graphs are inside boxes, not erased. For every named distinct-list environment, a single theorem gives focus freedom and hole freedom of both actual translated programs. It asserts structural well-formedness only; no statement of termination equivalence follows from it alone.

Lean: `FullTranslationExamples.privateCell_depth`

Lean: `FullTranslationExamples.constantResult_depth`

Lean: `FullTranslationExamples.privateCell_outer_edges`

Lean: `FullTranslationExamples.constantResult_outer_edges`

Lean: `FullTranslationExamples.proposition12_2_graphs_wellFormed`

A self-application term provides a structural regression for syntax that can express divergence. The generic language theorem checks its translation; no evaluation or divergence theorem is claimed. A separate application of an identity abstraction to unit has subterms of unequal box depth. Its translated outer graph has four edges, checking that common-depth padding

has added no outer structure. Operational regressions for divergent callbacks and repeated closure use still require the unfinished machine.

Lean: `FullTranslationExamples.omegaTerm`

Lean: `FullTranslationExamples.omega_translation_language`

Lean: `FullTranslationExamples.unequal_depth_application_edges`

The full build and all three axiom audits pass with only standard Lean axioms. Construction of all Figure 16 clauses and the recursive term translation is complete. The proposition terms and their desugaring remain unchanged. The remaining correspondence, context, machine, and robustness obligations are listed in Appendix B.

58 Stable graphs and copyable payloads

Definitions 10.2 and 10.3 serve different purposes. Copyability describes what Figure 17 may duplicate; stability restricts the graph used by the value-sensitive rules. We now formalise both for the paper's full example operation alphabet. Neither the proposition terms nor their Figure 16 translation has changed. In particular, extracting a copyable payload below is an exact factorisation of the original translated graph, not a replacement of that graph by an equivalent program.

These are structural predicates and proofs. Their use in copying, store transitions and the operational stability/refocusing invariants remains unfinished. The existing correspondence audit of the earlier linear substitution rule remains open.

58.1 The complete stability predicate

A stable graph has a single term input; every edge is an instance or a passive operation; every external output is an atom or a thunk; and every vertex is reachable from that input. Boxes are not stable edges: a lambda head may expose a thunk output to which a box is attached outside the stable fragment. The definition retains all three conditions of Definition 10.3, rather than replacing reachability with a language-label approximation.

The primitive predicate accepts an instance and accepts an operation exactly when its declared kind is passive. Its lifted label predicate requires an actual primitive label, excluding boxes even if their contents happen to contain passive operations. The output predicate accepts atoms and thunks and rejects terms. A vertex step follows one edge from any source to any target. Reachability is existence of an exact finite trace of such steps, including the zero-step trace. The stability structure then records the label, output, and all-vertex reachability requirements explicitly.

Lean: `Stability.StablePrimitive`

Lean: `Stability.StableLabel`

Lean: `Stability.StableOutput`

Lean: `Stability.VertexStep`

Lean: `Stability.VertexReach`

Lean: `Stability.Stable`

Every permitted primitive has one term input. In the example alphabet, its outputs are non-term: a constant has no outputs, lambda has a thunk output, and an instance has an atom output. Case analysis on the primitive and operation constructors proves these signature facts. Mapping labels over source and target incidence lists transfers them to actual vertices: a source vertex of a permitted edge has term type, whereas a target vertex has a permitted non-term type.

Lean: `Stability.primitive_input`

Lean: `Stability.primitive_outputs`

Lean: `Stability.source_type`

Lean: `Stability.target_type`

An internal vertex must occur in some source and some target, by the graph's two coverage

laws. The preceding typing facts would therefore make it both term and non-term. Thus no internal vertex exists. Each edge has a nonempty singleton source list. Its source cannot be an output by boundary discipline or an internal vertex by the preceding contradiction, so it is the unique external input. Source-incidence uniqueness now makes any two edges equal. Coverage at the input also supplies an edge, proving existence as well as uniqueness. These conclusions use the concrete example signatures; they are not asserted for an arbitrary operation alphabet.

Lean: `Stability.no_internal`

Lean: `Stability.input_source`

Lean: `Stability.edge_unique`

Lean: `Stability.exists_unique_edge`

For this alphabet, the output and reachability conditions follow from the permitted labels. Each output has a producing edge by target coverage, so its type is permitted. For reachability, the input reaches itself in zero steps, internal vertices are impossible, and each output is reached in one step through its producer, which consumes the input. These proofs construct the full stability record rather than assuming the extra fields. Finally, an adjacency would share a vertex between a target and a source, again forcing a non-term vertex to be term. Hence there is no outer edge adjacency in a stable graph of this alphabet.

Lean: `Stability.of_labels`

Lean: `Stability.no_adjacency`

Constant, lambda-head, and instance corollas satisfy the label restriction; the construction just proved supplies their output and reachability fields at every box allowance. Stability also implies global focus freedom and hole freedom. Every outer label is a primitive instance or passive operation, so it cannot equal either forbidden constructor. No box case is hidden: the stability label predicate excludes boxes outright.

Lean: `Stability.constant`

Lean: `Stability.lambda`

Lean: `Stability.instanceEdge`

Lean: `Stability.focusFree`

Lean: `Stability.holeFree`

Recursive label equivalence preserves the stable-label predicate. At depth zero it is literal equality. At positive depth, related primitive labels are equal; primitive/box pairs cannot be related; and neither box satisfies the stable-label predicate. To transport stability through a recursive graph isomorphism, pull each target edge back through the edge bijection, transfer its stable label, and invoke the proved construction from labels. Applying the same argument to the inverse gives equivalence of stability.

Lean: `Stability.label_equiv`

Lean: `Stability.iso`

Lean: `Stability.iso_iff`

58.2 Copyability specialised to the full example alphabet

Definition 10.2 permits an instance, or an operation followed by the parallel box forest attached to its thunk outputs. In $\mathbb{O}^{\mathrm{ex}}$, only lambda has a thunk output, and it has exactly one. Thus lambda has exactly one attached box; constants, application, unary operations and binary operations have no attached boxes. Specialising the diagram gives the constructors below without removing any case of the example language.

A lambda payload is serial assembly of the lambda corolla with the box of its body, before adding the translation's outer distributors. The box has one bound variable and the original ordered free variable/atom interface. Copyability has constructors for instance, constant, application, each unary operation, each binary operation, and this lambda payload. A final constructor closes these shapes under recursive graph isomorphism, including independent renaming inside boxes. The predicate retains the interface counts of each shape. Box bodies are not assumed stable: they may contain computations. Copyability is a shape predicate, not an assertion of termination.

Lean: `Copyability.lambdaPayload`

Lean: `Copyability.Copyable`

The outer-label predicate allows only instance, operation, and box labels. It is invariant under recursive label equivalence by the same primitive/box case analysis. Induction on a copyability derivation proves that every outer edge has an allowed label. Primitive cases inspect the corolla; the lambda case splits between its lambda and box edges; the renaming case pulls back through the edge bijection and uses label invariance. Consequently no outer sharing, store, focus or hole edge is part of a copyable payload. This is an outer-level statement; arbitrary box bodies require their own invariants. Copyability is preserved and reflected by recursive isomorphism, using its renaming constructor in both directions.

Lean: `Copyability.OuterLabel`

Lean: `Copyability.outer_equiv`

Lean: `Copyability.outer_labels`

Lean: `Copyability.iso_iff`

The literal abstraction translation factors as the lambda payload followed by $D^{\star}_{k,1} \otimes D^{\diamond}_{h,1}$. The equation is definitional: it only names an existing part of the assembled graph. In particular, none of the distributors is deleted. If the body is a language graph, the payload is a language graph too, by the box-language lemma, the permitted lambda primitive, and serial closure. This supplies the stronger global invariants when the body comes from the term translation.

Lean: `Copyability.abstraction_factor`

Lean: `Copyability.lambdaPayload_language`

58.3 Distinguishing the predicates and using the original program

Every unary operation corolla is copyable, including reference creation and dereference, but it is not stable: its term output contradicts the stable-output field. A lambda payload is also not stable, even though its head is passive. Serial assembly introduces an internal thunk vertex between lambda and box, contradicting the proved absence of internal vertices in a stable graph. These negative results prevent treating "copyable" and "stable" as interchangeable approximations to values.

For every scoped term supplied as a lambda body, its translated payload is copyable by construction. The full translation's language theorem and payload language preservation prove global focus and hole freedom. Thus arbitrary computations may remain inside the box while the fragment used by the copy rule has the required structural form.

Lean: `ValueExamples.unary_copyable_not_stable`

Lean: `ValueExamples.lambdaPayload_not_stable`

Lean: `ValueExamples.translated_lambda_payload`

For every ambient (k, h), the translation of the unchanged right-hand program $\lambda f.(f(); 1)$ factors into a copyable payload and its two single-use distributors. The witness is the lambda payload of the translated body $f(); 1$, using exactly the existing sequencing sugar. The factorisation equality is reflexivity after unfolding the construction; no contextual or semantic equivalence is assumed. The graph has allowance two because sequencing introduces the second abstraction. Finally, the unit corolla is stable and has a unique edge by the general stability theorems.

Lean: `ValueExamples.constantResult_copyable_factor`

Lean: `ValueExamples.unit_stable_unique`

All three axiom audits and the full build pass with only standard Lean axioms. The definitions and structural properties of stable and copyable graphs are completed. Required depth and operational preservation laws, copying/store transitions, translation/context laws, and template robustness remain in Appendix B. Proposition 12.2 is not yet proved.

59 Binding-free paths and the boundary of plugging closure

This section treats the plugging-closure prerequisite of Appendix B.1, item 3. It does not complete that item: the copying observations and the translation/congruence theorem for lambda-contexts remain unfinished. We retain Definition 11.2 literally, prove the positive-path correspondence, and prove Lemma E.1 with a sourced-hole hypothesis. That hypothesis holds for every translated term hole. A finite counterexample shows why it cannot be dropped for arbitrary hole interfaces allowed by the current graph model. Neither program of Proposition 12.2 nor its Figure 16 translation changes.

59.1 The literal path restriction

A binding-free graph is focus-free and has no positive path, at any depth, from a source of a contraction, instance, box, or hole edge to a source of a hole. A store edge is not an instance edge. Paths remain at one depth; the condition is checked separately in each box body.

The binding-label predicate classifies the four prohibited starting kinds; the hole-label predicate recognises an actual primitive hole. Reflexive edge ancestry follows zero or more incidence adjacencies. Strict ancestry requires a first adjacency and hence at least one traversed edge before the terminal hole. The shallow restriction additionally requires a source of the starting edge. This is substantive for holes with an empty input list. The recursive path predicate conjoins the shallow restriction with the same predicate in every box body. Finally, binding-freedom adds global focus-freedom. These predicates do not themselves enforce unique hole names; that independent context condition must also be supplied where required.

Lean: `BindingFree.BindingLabel`

Lean: `BindingFree.HoleLabel`

Lean: `BindingFree.Ancestor`

Lean: `BindingFree.StrictAncestor`

Lean: `BindingFree.ShallowBindingFree`

Lean: `BindingFree.BindingPaths`

Lean: `BindingFree.BindingFree`

Ancestry is transitive by induction on its first derivation. An ancestry between distinct edges must have a first step, since its reflexive case would identify the endpoints. Any adjacency-preserving edge map transports ancestry by induction. These facts concern incidence alone, without an operational or termination hypothesis.

Lean: `BindingFree.trans`

Lean: `BindingFree.positive_of_ne`

Lean: `BindingFree.map`

A vertex step traverses a single edge from one of its sources to one of its targets. Starting at any source of an ancestor edge, induction on ancestry yields a finite vertex trace to a source of the terminal edge. Conversely, a vertex trace beginning at a target of an edge yields strict ancestry to any edge having the final vertex as a source. The induction prepends each traversed edge.

Consequently, strict ancestry together with a starting-source witness is *equivalent* to a vertex trace of length $n + 1$ between the two source lists, for some n. In the reverse implication, source uniqueness identifies the first traversed edge with the proposed starting edge. The positive length is essential: reflexive vertex reachability would wrongly exclude an isolated hole from binding-free contexts.

Lean: `BindingPathCorrespondence.BindingPathStep`

Lean: `BindingPathCorrespondence.ancestor_vertex_trace`

Lean: `BindingPathCorrespondence.target_trace_strict`

Lean: `BindingPathCorrespondence.strict_iff_vertex_path`

Every hole is a prohibited starting label. A self-loop at a hole would supply both a source witness and a positive path back to that hole, so a binding-free hole cannot self-loop. Globally hole-free graphs satisfy the path restriction vacuously, first at the outer level and then by induction inside boxes. Combining this with focus-freedom proves binding-freedom. A general pullback lemma transports the shallow restriction along an edge map preserving adjacency, starting-label classification, source existence, and terminal-hole classification: map any proposed counterexample and contradict the target graph's restriction.

Lean: `BindingFree.hole_binding`

Lean: `BindingFree.hole_nonSelfLoop`

Lean: `BindingFree.shallow_of_holeFree`

Lean: `BindingFree.paths_of_holeFree`

Lean: `BindingFree.of_holeFree`

Lean: `BindingFree.shallow_of_map`

59.2 Plugging closure at sourced holes

A sourced hole is an addressed hole occurrence whose input list has at least one port. The condition concerns only the replaced hole, including when that hole lies inside boxes. It does not exclude unrelated zero-input holes from the surrounding observation context.

For an adjacency from a surviving host edge into an inserted edge, source incidence of the inserted graph forces the shared vertex to be an input port. Thus the old host edge was adjacent to the replaced hole. Dually, an inserted-to-host adjacency comes from an output port and makes the old hole adjacent to the surviving host edge.

Induction backwards along an ancestry ending at a hole now identifies its origin. A surviving starting edge has an ancestry to a host hole, possibly the replaced hole. An inserted starting edge has an ancestry to an inserted hole. The only problematic crossing is from insertion back to host: it would create a strict ancestry from the replaced hole to a host hole. Binding-freedom excludes this because the replaced hole has a source. An inserted-to-inserted adjacency reflects to the original insertion using the previously proved absence of a self-loop at the replaced hole.

For shallow plugging closure, split the first edge and the first successor of an alleged forbidden path by host/inserted origin. Host-to-host and inserted-to-inserted paths contradict the respective original restriction. A host-to-inserted crossing produces a forbidden host path to

the replaced hole. An inserted-to-host crossing contradicts the host restriction at that sourced hole. Source witnesses survive the incidence-list maps.

Lean: BindingClosure.adjacent_host_insert

Lean: BindingClosure.adjacent_insert_host

Lean: BindingClosure.ancestor_origin

Lean: BindingClosure.shallow_plug

The addressed sourced-hole predicate descends through boxes to the selected shallow occurrence. Shallow closure of the recursive path predicate uses the path theorem at the current depth and retains the body predicates of all surviving or inserted boxes. Replacing only a box body changes no outer incidence and no outer binding/hole classification; the pullback lemma therefore preserves the outer restriction.

Induction on the addressed occurrence combines these two cases and proves closure at arbitrary nesting depth. The final binding-free plugging theorem adds the existing global focus-freedom closure. No copying transition is assumed or encoded by this argument.

Lean: BindingClosure.SourcedHole

Lean: BindingClosure.paths_shallow

Lean: BindingClosure.shallow_replaceBox

Lean: BindingClosure.paths_plugAt

Lean: BindingClosure.plugAt

Recursive label equivalence preserves both binding-label and hole-label classification: primitive labels agree, boxes match boxes, and mixed cases are impossible. A recursive graph isomorphism preserves adjacency and maps source witnesses by its vertex bijection. The pullback lemma gives shallow invariance. Induction on depth then applies the same argument to corresponding box bodies. Applying it to an isomorphism and its inverse, together with focus-freedom invariance, proves preservation and reflection of binding-freedom.

Lean: BindingInvariance.bindingLabel_equiv

Lean: BindingInvariance.holeLabel_equiv

Lean: BindingInvariance.shallow_iso

Lean: BindingInvariance.paths_iso

Lean: BindingInvariance.iso_iff

59.3 Term-hole checks and a zero-input counterexample

Every already constructed term translation is binding-free because it is focus-free and hole-free. This is a theorem about terms, not yet about lambda-context translations. A stand-alone hole corolla is binding-free: its targets are outputs and its sources are inputs, so no positive return adjacency is possible. Finally, the hole assigned to any term judgement has interface

$\star \Rightarrow \star^k \otimes \diamond^h$ and hence has the source witness required by the plugging theorem, for every k and h.

Lean: `BindingExamples.translated_bindingFree`

Lean: `BindingExamples.hole_bindingFree`

Lean: `BindingExamples.term_hole_sourced`

Consider a context with two holes in series,

$$\chi_0 : \epsilon \Rightarrow \diamond, \qquad \chi_1 : \diamond \Rightarrow \epsilon.$$

Their names are distinct. The context is binding-free under the literal source-based definition: χ_0 has no source, while χ_1 has no target. Now replace χ_0 with a term weakening followed by an instance:

$$\epsilon \xrightarrow{\text{weakening}_\star} \star \xrightarrow{\mathsf{I}} \diamond.$$

This insertion is focus-free and hole-free, hence binding-free. After plugging, there is a one-edge path from the source of I to the source of χ_1. The result is not binding-free.

Lean checks both finite constructions, both original binding-freedom proofs, the uniqueness of the host's hole labels, the resulting forbidden path, and the negation of unrestricted hole-replacement closure. The source witness and the final adjacency are explicit vertices of the constructed graph; no external decision procedure or new axiom is used.

This exposes a boundary in the literal general formulation of Lemma E.1. Its proof prefixes the old hole to a path, but that prefix has no starting source if the hole has no inputs. The qualified theorem proved here suffices for plugging at translated term holes. It does not silently strengthen Definition 11.2, nor remove zero-input holes from the observation class. The broader correspondence audit must record this qualification.

Lean: `BindingExamples.zeroInputContext`

Lean: `BindingExamples.zeroInputReplacement`

Lean: `BindingExamples.zeroInputContext_bindingFree`

Lean: `BindingExamples.zeroInputReplacement_bindingFree`

Lean: `BindingExamples.zeroInputContext_unique_holes`

Lean: `BindingExamples.zeroInput_plug_not_bindingFree`

Lean: `BindingExamples.unrestricted_hole_closure_false`

The full build and all three axiom audits pass. Remaining work in Appendix B.1, item 3 includes constructing copying observations, translating lambda-contexts and proving their binding-freedom and plugging correspondence, and deriving term-context congruence. That item is not marked complete.

60 Annotated term contexts and concrete closing observations

This section constructs the remaining syntax and observation interfaces of Appendix B.1, item 3, without claiming its congruence theorem. The Figure 16 translation of both proposition programs remains unchanged. Two different composition operations must be distinguished: composing graph-building constructors is proved here, whereas correspondence with addressed graph plugging is still required. Execution remains an explicit parameter until the copying UAM of item 2 is constructed.

60.1 Exact single-hole closing contexts

A sole-hole certificate identifies the selected occurrence as the only hole at every depth. At a shallow occurrence it requires a hole label there and global hole-freedom below every other edge. Inside a box it requires the corresponding certificate for the body and hole-freedom elsewhere outside. Induction on the occurrence proves that filling it with a hole-free graph leaves no holes. In the shallow case, every remaining edge is either a retained non-hole or an inserted hole-free edge. In the deep case, only the selected box body changes.

If the local input list is nonempty, the selected hole has a source: its source-list length is its input-interface length. The same argument descends through boxes. Thus the sourced-hole premise of the previous section follows from the interface and sole-hole certificate; it is not a new assumption on translated term contexts.

Lean: `Contexts.SoleHole`

Lean: `Contexts.fill_holeFree`

Lean: `Contexts.sourced`

A closing context records an arbitrary outer depth, a graph of interface $\star \Rightarrow \epsilon$, an addressed term hole of the requested depth and environment interface, its sole-hole certificate, and binding-freedom. The surrounding context can contain arbitrary example-language graph structure permitted by these conditions; it is not restricted to syntax. Filling is the existing addressed graph replacement. Hole-freedom follows from the sole-hole lemma, focus-freedom from global replacement closure, and binding-freedom from the sourced-hole theorem. The latter's source condition is derived from the hole's single term input.

Lean: `Contexts.ClosingContext`

Lean: `Contexts.fill`

Lean: `Contexts.fill_holeFree`

Lean: `Contexts.fill_focusFree`

Lean: `Contexts.fill_bindingFree`

60.2 The annotated grammar and literal graph interpretation

A term context has a fixed hole scope and a separately indexed outer scope. Its constructors are the hole, abstraction, left and right application, unary operation, and left and right binary operation, exactly the grammar of Section 11. An abstraction removes one variable from the outer scope; the hole annotation retains any variables bound by enclosing abstractions. Filling is structural recursion using the original term constructors. Context composition substitutes an

annotated context for the hole, with matching scopes enforced by the type indices. Induction on the outer context proves the filling law, the right identity law, and associativity. The left identity is definitional. No variable names or program terms are changed.

Lean: TermContexts.TermContext

Lean: TermContexts.fill

Lean: TermContexts.compose

Lean: TermContexts.fill_compose

Lean: TermContexts.compose_hole

Lean: TermContexts.compose_assoc

A scoped graph packages its finite depth together with a graph of the corresponding term and environment interface. This is a dependent pair, not a quotient or a new translation. The packaged abstraction, unary, application, and binary constructors call the existing Figure 16 constructors. Binary branches use the original maximum-depth padding. Thus all distributor edges, port ordering and boxes remain present.

Interpret a term context by replacing its hole with a supplied scoped graph and applying these graph constructors recursively. Induction proves that supplying the original translation of a term returns exactly the original translation of the filled term, including its depth. Each inductive case rewrites the recursive result and closes by definitional equality. A second induction proves that interpretation respects context composition.

The actual holed translation is obtained by supplying the prescribed single hole edge, at any chosen allowance and hole name. Its constructor composition law follows from the preceding interpretation theorem. This law does not yet identify interpretation with the operation that replaces the selected edge in the holed graph: that addressed-plugging correspondence, including padding and recursive isomorphism, remains to be proved.

Lean: TermContexts.ScopedGraph

Lean: TermContexts.ofTerm

Lean: TermContexts.abstraction

Lean: TermContexts.unary

Lean: TermContexts.application

Lean: TermContexts.binary

Lean: TermContexts.apply

Lean: TermContexts.apply_ofTerm

Lean: TermContexts.apply_compose

Lean: TermContexts.hole

Lean: TermContexts.graph

Lean: TermContexts.graph_compose

60.3 Termination observations with an explicit execution interface

An open program contains an actual graph of a fixed term/environment interface and proofs of global focus-freedom and hole-freedom. A closed program contains a graph of interface $\star \Rightarrow \epsilon$ with the same two proofs and its finite depth. Closing an open program in a certified context constructs a closed program using the filling lemmas. An execution interface supplies a machine and an initializer for these closed programs. The interface is a parameter: no copying machine, machine law, or translation-correctness hypothesis is asserted by its definition.

Lean: Observations.OpenProgram

Lean: Observations.ClosedProgram

Lean: Observations.ClosingContext.close

Lean: Observations.Execution

An observation interface fixes the fragment's depth and the two environment sizes. Its context type is the concrete closing-context structure above, whose outer depth is unrestricted. Observation initializes the filled closed graph. Since validity is built into that structure, every inhabitant is allowed. Successful termination means a finite machine run to a final state; a stuck nonfinal state is not counted.

Quantitative refinement universally quantifies over these concrete contexts and compares their initialized states by the existing machine refinement. Equivalence is mutual refinement. Unfolding the observation relation proves that this direct definition equals the generic observation interface used by the robustness infrastructure. Reflexivity and transitivity hold pointwise from the corresponding machine results; equivalence is consequently reflexive, symmetric and transitive. No context closure is used to prove these order laws. Closure is needed for congruence, a distinct theorem still outstanding.

For the universal cost relation, refinement is exactly implication of successful termination in each context. One direction extracts the target run promised by refinement; the other supplies that run and the trivial cost witness. Applying both directions proves that equivalence is precisely agreement on termination, without determinism or a runtime bound.

Lean: Observations.Interface

Lean: Observations.observations

Lean: Observations.Terminates

Lean: Observations.Refines

Lean: Observations.Equiv

Lean: Observations.refines_iff_observations

Lean: Observations.refines_refl

Lean: Observations.refines_trans

Lean: Observations.equiv_refl

Lean: Observations.equiv_symm

Lean: Observations.equiv_trans

Lean: Observations.refines_any_iff

Lean: Observations.equiv_any_iff

The original translation of a term is packaged at any adequate allowance using the existing padding operation. Its global label theorems prove focus-freedom and hole-freedom after padding. Term observational equivalence then compares both translations at every common adequate allowance, against all certified closing contexts, using the universal cost relation. The execution parameter remains visible: the concrete copying UAM must instantiate it later. Padding/initialization invariance and the full paper-correspondence argument are not consequences of this definition alone.

Lean: Observations.translatedProgram

Lean: Observations.ObservationalEquiv

The closing-context type is nonempty for closed terms at every allowance: the identity observation is the single $\star \Rightarrow \epsilon$ hole. Its sole-hole proof uses the singleton edge carrier and its binding-freedom is the isolated-hole theorem. This checks that the observation quantifier is not vacuous in the closed case.

Three exact translation checks exercise the annotation and constructor laws. An enclosing lambda binds the variable at index zero in the hole's annotated scope. The left proposition program is reconstructed by filling the argument of its original lambda application with `ref` 1. The right program is reconstructed by filling an abstraction context with its original sequence $f(); 1$. Each proof invokes the exact interpretation theorem. These are structural equalities, not semantic equivalence claims.

Lean: ObservationExamples.identityClosingContext

Lean: ObservationExamples.bound_hole_translation

Lean: ObservationExamples.privateCell_context_factor

Lean: ObservationExamples.constantResult_context_factor

The full build and three axiom audits pass. Binding-freedom is now proved in Section 61. The sole-hole certificate is proved in Section 62. Item 3 remains open pending addressed plugging correspondence and term-context congruence with matching observation and padding quantification. The execution interface must also receive its copying-UAM implementation from item 2. No conditional wrapper is presented as Proposition 12.2 or Lemma 11.3.

61 Binding-freedom of every translated lambda-context

We now prove the binding-free graph obligation in the paper's proof of Lemma 11.3: every annotated lambda-context translates to a binding-free graph, for every hole name and every finite hole allowance. The proof uses the existing Figure 16 construction, including the prescribed distributors, middle-block exchange and padding. No term or translation clause changes. Section 62 supplies the sole-hole certificate and graph-context congruence with an explicit execution hypothesis. Addressed-plugging correspondence and term-context congruence remain separate obligations in Appendix B.1.

The central observation is directional. A serial graph has no path that crosses from its right component back into its left component. An operation prefix has no forbidden starting label. A sharing suffix has no hole that could be the endpoint of a forbidden path. These facts justify the actual translation graphs without deleting or identifying distributor edges.

61.1 Adjacency through finite expansion

For an expanded edge with disjoint source and target lists, the insertion map is injective on all vertices. Internal vertices carry the component tag; input and output coincidences are ruled out by the old edge's absence of a self-loop; equal ports of the same kind are identified using port injectivity. Consequently adjacency between two edges in that component is preserved and reflected: pull their shared vertex back through the injective insertion map.

For two distinct components, any adjacency must cross their boundaries. A shared internal vertex would identify the component tags. The remaining possibility is an output port of the first component identified with an input port of the second. Host injectivity then gives an adjacency between the corresponding old pattern edges. These lemmas are generic finite-graph results, independent of the copying alphabet or machine.

Lean: `ExpansionAdjacency.insert_injective`

Lean: `ExpansionAdjacency.adjacent_same_iff`

Lean: `ExpansionAdjacency.adjacent_cross`

61.2 Depth and ordered parallel assembly

Padding leaves adjacency unchanged under the existing edge bijection. It also preserves and reflects both the forbidden-starting-label predicate and the hole-label predicate: primitives retain their constructor, and boxes remain boxes. Source witnesses are unchanged. The general pullback lemma therefore proves shallow binding-freedom in both directions. Induction on depth extends this equivalence to every box body. Together with focus-freedom invariance it proves binding-freedom invariance under one-step raising. Equality transport handles depth casts, induction handles any finite amount of padding, and combining these proves the result for padding to a requested common allowance. Unsafe paths cannot be concealed by changing a representation bound.

Lean: `BindingPadding.padding_adjacent`

Lean: `BindingPadding.padding_bindingLabel`

Lean: `BindingPadding.padding_holeLabel`

Lean: `BindingPadding.shallow_raise_iff`

Lean: BindingPadding.paths_raise_iff

Lean: BindingPadding.raise_iff

Lean: BindingPadding.castDepth_iff

Lean: BindingPadding.pad_iff

Lean: BindingPadding.padTo_iff

In parallel assembly, adjacent edges have the same component tag. Induction on ancestry therefore keeps every edge in its starting component and reflects the ancestry to that component. A proposed forbidden path in the parallel graph yields its source witness and terminal hole in that same component, contradicting the component's shallow binding-freedom. At deeper allowances, each inserted label retains its original box body, so the recursive condition and focus-freedom are inherited as well.

Interface casts preserve these predicates by equality elimination. Hence the translated pair of subterms is binding-free when both subterms are: the middle-block exchange only changes the boundary ordering, and the preceding parallel theorem allows any typed boundary bijection. This proves the property for the exact variable-before-atom interface of Figure 16.

Lean: BindingParallel.ancestor_parallel

Lean: BindingParallel.shallow_parallel

Lean: BindingParallel.paths_parallel

Lean: BindingParallel.parallel

Lean: BindingParallel.castInterfaces_iff

Lean: BindingParallel.pair

61.3 Directed serial assembly

Neither edge of the two-slot serial pattern self-loops: its external and internal port constructors distinguish its source and target vertices. A backward adjacency in the expansion would, by the crossing lemma, give an adjacency from the right pattern edge to the left one. That would identify an external output with an external input, which is impossible.

Induction on ancestry proves that a path starting in the right component stays there. If the two endpoints have the same tag, every intermediate tag must agree too: a left-to-right crossing could not return to a left endpoint. The within-component adjacency reflection theorem then reconstructs the whole ancestry in the original component.

Lean: BindingSerial.serial_nonSelfLoop

Lean: BindingSerial.serial_no_backward

Lean: BindingSerial.serial_ancestor_monotone

Lean: BindingSerial.serial_ancestor_same

Serial shallow closure requires a condition on paths crossing from left to right. It suffices that either the left graph has no prohibited starting labels or the right graph has no holes. Under either alternative, a forbidden path must have both endpoints on the same side. Directionality

puts its first successor on that side as well, and the ancestry theorem reflects the rest of the path. Pulling back the source witness contradicts that component's shallow condition.

The recursive version retains the original body condition below every expanded edge. Combining it with the existing focus-free serial theorem gives full binding-freedom. In particular, a focus-free, hole-free suffix can be appended to any binding-free graph. This is the case needed for the translation's contraction and weakening distributors. Their labels need not be operations, and their edges are not removed.

Lean: `BindingSerial.shallow_serial`

Lean: `BindingSerial.paths_serial`

Lean: `BindingSerial.serial`

Lean: `BindingSerial.serial_suffix`

61.4 The complete context-translation certificate

A graph in the translated language is focus-free and hole-free, so it is binding-free. An operation corolla is such a graph and has no prohibited starting label; serial closure therefore permits it as a prefix of any binding-free argument graph. A box has no outer hole, while its body inherits the binding-free condition independently at the lower depth.

For abstraction, apply the interface cast to the certified body, box it, prefix the lambda operation and append the original single-use distributors. For a unary operation, prefix its operation edge and append those distributors. For application and every binary operation, use the certified ordered pair, prefix the operation edge, and append the original two-use distributors. Each suffix is focus-free and hole-free by its existing language theorem.

Lean: `BindingTranslation.language`

Lean: `BindingTranslation.operation_prefix`

Lean: `BindingTranslation.box`

Lean: `BindingTranslation.abstraction`

Lean: `BindingTranslation.unary`

Lean: `BindingTranslation.application`

Lean: `BindingTranslation.binary`

Induction on an annotated lambda-context now proves the stronger statement: its constructor interpretation preserves binding-freedom for *any* binding-free graph supplied at the hole. The hole case is the hypothesis. Abstraction and unary cases use the constructor lemmas. Application and binary cases combine the recursive certificate with the hole-free translation of the fixed subterm. Both branches are padded to the original common maximum allowance, using the proved padding equivalence, before applying the pair and operation lemmas.

Finally, supply the isolated hole corolla, whose binding-freedom was already proved. The result is the actual holed context translation, with no well-behavedness assumption on the context beyond its typed grammar. This is the first of the two graph properties used in the

paper's proof of Lemma 11.3. The other is correspondence between translated term filling and addressed graph plugging; it is not asserted by this certificate.

Lean: `BindingTranslation.context_apply`

Lean: `BindingTranslation.context_graph`

A nested example combines abstraction, application and reference creation, with the enclosing lambda binding a variable in the hole annotation. Its translation is certified at every allowance and every hole name by the general theorem.

A negative example places a term contraction immediately before a term hole. An explicit input vertex supplies a source of the contraction; the shared internal vertex is both its target and the hole's source. This is a positive forbidden path, so the graph is not binding-free. The padding equivalence proves that the same rejection holds at every larger allowance. Thus the certificate does not accidentally admit the copying configuration that the binding-free restriction is intended to exclude.

Lean: `BindingTranslationExamples.nestedReferenceContext`

Lean: `BindingTranslationExamples.nestedReferenceContext_bindingFree`

Lean: `BindingTranslationExamples.contractionBeforeHole`

Lean: `BindingTranslationExamples.contractionBeforeHole_not_bindingFree`

Lean: `BindingTranslationExamples.padded_contractionBeforeHole_not_bindingFree`

The full build and all three axiom audits pass. The binding-free certificate is removed from the unfinished obligations of Appendix B.1, item 3. The exact sole-hole certificate is subsequently proved in Section 62. That item still requires addressed-plugging correspondence and the term observation/congruence proof. The copying machine and its execution instance remain part of item 2.

62 Certified context occurrences and graph-context congruence

Every annotated lambda-context now has an actual sole-hole address in its unchanged Figure 16 translation. Its binding-free certificate was proved in Section 61. We also prove composition of addressed graph contexts, associativity of filling up to recursive graph isomorphism, and graph-context congruence for an execution that respects that isomorphism. The latter is an explicit semantic hypothesis to be discharged by the copying machine, not an axiom of this development.

These results do not yet prove the paper's Lemma 11.3. That requires a further structural bridge (whose unary case is subsequently proved in Section 63): filling this certified graph with a suitably padded term translation must agree, up to recursive isomorphism, with translating the filled term. The constructor-interpreter equality from Section 60 is not this bridge: it concerns re-evaluation of graph constructors, rather than addressed replacement. Appendix B.1, item 3 therefore remains open.

62.1 Exact occurrence inventory

Fix a named hole h, including its ordered input and output interfaces. A primitive label has the single-hole property precisely when it is that hole. A box label has the property precisely when exactly one edge of its body has the property recursively and every other body edge is globally hole-free. A graph has the property when exactly one outer edge has the label property and all its other edges are globally hole-free. This counts occurrences, including occurrences below boxes; repeated copies of the same name do not count as one occurrence.

Lean: `SingleHole.SingleHoleLabel`

Lean: `SingleHole.SingleHole`

Interface equality transports an existing addressed site, and equality elimination preserves its sole-hole certificate. Induction on depth then turns the occurrence inventory into an actual site with exactly h's interfaces. At a primitive occurrence, choose its shallow edge and transport the interface equations obtained from its label. At a box, use the induction hypothesis in its body and extend the address through the box. The other-edge conditions are precisely those required by the recursive sole-hole predicate.

The property is unchanged by an outer interface cast. A hole corolla has it at every allowance: its only edge is the hole, and any proposed different edge contradicts the singleton edge enumeration.

Lean: `SingleHole.Site.castInterfaces`

Lean: `SingleHole.SoleHole.castInterfaces`

Lean: `SingleHole.site`

Lean: `SingleHole.castInterfaces_iff`

Lean: `SingleHole.corolla`

62.2 Preservation by the original assembly operations

An edge bijection transports the inventory provided it preserves and reflects both marked labels and hole-free labels. For the forward direction, map the marked edge and pull each

other edge back through the inverse; injectivity shows that the inverse is different from the original marked edge. The reverse argument is symmetric.

For an ordered parallel family, choose the marked edge in its unique marked component. Another edge either belongs to that component, where the existing certificate applies, or to another component, which is hole-free. Finite expansion has the same tagged edge inventory and uses the same argument. Serial composition is expansion of a two-edge pattern, so the left and right versions follow by choosing the appropriate Boolean tag. Enclosing the graph in a box makes that box the unique marked outer edge; the recursive label condition is exactly the original graph inventory.

Lean: `SingleHoleAssembly.transport`

Lean: `SingleHoleAssembly.parallel`

Lean: `SingleHoleAssembly.expand`

Lean: `SingleHoleAssembly.serial_left`

Lean: `SingleHoleAssembly.serial_right`

Lean: `SingleHoleAssembly.box`

Raising preserves and reflects hole-freedom of an individual edge label. For a primitive, the label constructor is unchanged. For a box, the existing recursive all-labels padding theorem supplies the equivalence for its body. Induction on allowance proves the marked-label equivalence as well; the edge bijection then transports the entire single-hole inventory. Depth equality casts preserve it by equality elimination. Iterating the raising theorem handles finite padding, and combining padding with the depth cast handles padding to any specified larger allowance.

Lean: `SingleHoleAssembly.freeLabel_raise`

Lean: `SingleHoleAssembly.raise_iff`

Lean: `SingleHoleAssembly.castDepth_iff`

Lean: `SingleHoleAssembly.pad_iff`

Lean: `SingleHoleAssembly.padTo_iff`

A translated ordered pair retains exactly the marked occurrence in either component when the other is hole-free. The proof uses the parallel theorem and the existing boundary cast; it leaves the middle-block exchange intact. Operation corollas are hole-free. Consequently abstraction preserves the inventory by casting and boxing the body, prefixing the lambda operation, and appending the original hole-free single-use distributor. Unary operations use the same prefix and suffix argument without boxing. Application and binary operations prefix their operation edge to the certified pair and append the existing two-use environment distributor. No distributor edge is eliminated or identified in any of these proofs.

Lean: `SingleHoleTranslation.pair_left`

Lean: `SingleHoleTranslation.pair_right`

Lean: `SingleHoleTranslation.operation_holeFree`

Lean: `SingleHoleTranslation.abstraction`

Lean: `SingleHoleTranslation.unary`

Lean: SingleHoleTranslation.application

Lean: SingleHoleTranslation.binary

Structural induction on the annotated term context proves preservation for any supplied single-hole graph. The hole case is immediate; abstraction and unary cases use their constructor lemmas. In application and binary cases, the induction hypothesis supplies the marked component and the fixed term translation is hole-free. Padding both to the prescribed maximum allowance preserves these facts. Thus the constructor lemma applies to the literal graph used by the translator.

Supplying a hole corolla proves the exact named-hole inventory of the context translation. Extracting the site and combining its sole-hole certificate with the previously proved binding-free theorem gives an existential certificate at the annotated term and atom interface. Its local allowance is existential because enclosing fixed subterms can force additional padding. No occurrence-existence assumption remains.

Lean: SingleHoleTranslation.context_apply

Lean: SingleHoleTranslation.context_graph

Lean: SingleHoleTranslation.context_certificate

62.3 Composition of actual addressed contexts

Replacing a fixed box by recursively isomorphic bodies gives recursively isomorphic outer graphs. Keep the outer vertex and edge maps equal to the identity; at the selected edge use the given body isomorphism, and at every other edge use reflexivity. Induction on an addressed site then proves that plugging recursively isomorphic graphs preserves recursive isomorphism: use generic shallow replacement at the outer level and the box result below an enclosing box.

Replacing the same box twice retains only the last body. This follows pointwise from the conditional label replacement and does not change ports. Changing an inserted box body commutes with shallow insertion up to a graph isomorphism whose carrier maps are identities. Retained edges have the old labels, inserted edges have the same changed labels, and both port maps follow the same host or inserted-vertex cases.

Lean: DeepComposition.replaceBox_deepIso

Lean: DeepComposition.plugAt_deepIso

Lean: DeepComposition.replaceBox_overwrite

Lean: DeepComposition.plug_replaceBox_iso

Let s be an address in G and t an address in a graph H that fits s. Their composite is an address in $G[s := H]$. If s is shallow, an outer edge of H receives the inserted-edge tag; a deeper t retains its body address under that tag. If s descends through a box, recursively compose inside its body and then extend through the updated box.

Filling the composite address with A is recursively isomorphic to $G[s := H[t := A]]$. When both addresses are shallow this is the generic finite graph associativity isomorphism. A shallow outer address and an inner box use the preceding box-insertion interchange. For an outer box, remove the overwritten intermediate body and lift the induction hypothesis through that box. This proves associativity for arbitrary finite nested addresses.

If both original sites are sole holes, so is the composite: retained edges are hole-free by the outer certificate and inserted non-selected edges by the inner certificate. Induction handles enclosing boxes. Finally, when the outer hole has a nonempty input interface, the sourced-hole closure theorem proves that the composite host is binding-free. The zero-input qualification from Section 59 is retained.

Lean: DeepComposition.Site.comp

Lean: DeepComposition.plugAt_assoc

Lean: DeepComposition.SoleHole.comp

Lean: DeepComposition.context_comp_certified

62.4 The semantic congruence theorem and its hypothesis

An open graph context packages a graph, an addressed term hole, its sole-hole proof and its binding-free proof. Its outer environment may remain open. Filling it with an open program uses actual addressed replacement; focus-freedom follows from replacement closure and hole-freedom from the sole-hole theorem.

Compose a closing context C with an open graph context D by inserting D's graph and using the composite site. The preceding proofs supply the sole-hole condition and binding-freedom; the hole's singleton term input supplies the required nonempty interface. Associativity gives a recursive isomorphism between closing G with this composite context and closing $D[G]$ with C.

Lean: ContextCongruence.GraphContext

Lean: ContextCongruence.fill

Lean: ContextCongruence.ClosingContext.compose

Lean: ContextCongruence.ClosingContext.compose_fill_iso

The execution interface currently accepts an arbitrary initial-state map on closed graph presentations. Such a map need not respect renaming of internal carriers. We therefore state its representation obligation explicitly: recursively isomorphic closed, focus-free, hole-free graphs must have universally cost-related executions. By the earlier observation theorem, this means that they agree on successful termination. The copying machine of Appendix B.1, item 2 must eventually prove this property. It is a parameter of the present congruence theorem, not a newly postulated axiom and not a proved property of a concrete copying execution.

Suppose G refines H under all closing graph contexts and execution has this invariance property. To prove refinement of $D[G]$ by $D[H]$, choose an arbitrary closing context C. Apply the hypothesis to the proved closing context $C \circ D$. Execution invariance transports the source and target along the two filling-associativity isomorphisms. Transitivity then yields the required comparison under C. Applying this argument to both refinements proves graph-context equivalence congruence. The theorem uses the universal cost relation; no stronger quantitative congruence is claimed here.

Lean: ContextCongruence.RespectsIso

Lean: ContextCongruence.refines_context

Lean: ContextCongruence.equiv_context

Choose the allowance and address supplied by the translated-context certificate and package

them as an open graph context. Its graph is *definitionally equal* to the existing translation. Classical choice selects a certified address, not a replacement graph or a new translation. The graph-context congruence theorem now applies directly to this package, under the explicit execution invariance hypothesis. Its conclusion compares actual fillings of that graph. It does not yet identify those fillings with translations of filled terms.

Lean: `CertifiedContexts.certified`

Lean: `CertifiedContexts.certified_graph`

Lean: `CertifiedContexts.certified_graph_congruence`

A negative regression places two occurrences of the same hole in a parallel graph. Whichever occurrence is selected, the other violates the requirement that all remaining labels be hole-free. Thus the certificate does not confuse unique names with unique occurrences.

The nested reference context from the preceding section receives an actual address, sole-hole proof and binding-free proof at every hole allowance and name. A reflexivity proof checks that its certified graph is exactly the old Figure 16 graph. These structural regressions do not assert any copying machine execution behaviour.

Lean: `CertifiedContextExamples.duplicate_holes_rejected`

Lean: `CertifiedContextExamples.nestedReferenceContext_certified`

Lean: `CertifiedContextExamples.nestedReferenceContext_graph_unchanged`

The full Lean build and all three axiom audits pass. The completed sole-hole certification and graph-context composition obligations are removed from item 3's to-do wording. Addressed filling correspondence, its depth-padding compatibility, and the resulting term-context congruence remain to be proved; the concrete execution must also discharge its representation laws. Neither Proposition 12.2 nor item 3 is marked complete.

63 Actual filling through expansion and unary term contexts

This section proves a structural part of the missing graph-filling bridge. Replacing an edge within a component of a finite expansion agrees, up to explicit graph isomorphism, with replacing that edge in the component before expansion. The result applies to the original serial constructor and hence to the full unary-operation clause of Figure 16, including its distributor suffix. For a unary term context we obtain an actual addressed-filling law whose target is the unchanged original term translation.

The result holds for arbitrary finite variable and atom environments and arbitrary argument terms, including terms containing boxes. It proves a unary context with its hole at the outer level; it does not yet prove filling below an enclosing abstraction or inside a parallel argument pair. Section 64 subsequently proves the enclosing abstraction and nested lambda/unary cases. Parallel assembly and common-allowance padding remain in Appendix B.1, item 3. No execution hypothesis is needed for the structural isomorphisms proved here.

63.1 The replacement–expansion isomorphism

Let C be a finite pattern and F_b its assigned component at edge b. Choose an edge a of C, an edge e of F_a, and a graph H with the interface of e. Write $F'_a = F_a[e := H]$ and $F'_b = F_b$ for $b \neq a$. The theorem constructs

$$(C[F])[(a, e) := H] \;\cong\; C[F'].$$

It assumes no non-self-loop condition and no hole-free or binding-free condition: it is a theorem about finite typed graph construction itself.

The changed family is defined by testing whether its component index is a. At a it is ordinary shallow replacement; elsewhere it is the original component. Equality transport preserves the dependent interface type when the index is identified with a. The two evaluation lemmas reduce the selected and non-selected cases, respectively.

Lean: `ExpansionPlug.family`

Lean: `ExpansionPlug.family_same`

Lean: `ExpansionPlug.family_other`

The source graph's internal carrier has the form $(V_C + \sum_b V_{F_b}) + V_H$. The target carrier is $V_C + \sum_b V_{F'_b}$. Map each old pattern vertex to itself. A vertex in a component different from a keeps its tag. A vertex in F_a becomes an old vertex of $F_a[e := H]$, and a vertex of H becomes a newly inserted vertex of that same component. The inverse splits these cases in reverse. The two inverse laws follow by case analysis, using equality transport to identify the selected component's internal carrier with $V_{F_a} + V_H$.

Lean: `ExpansionPlug.vertexTo`

Lean: `ExpansionPlug.vertexFrom`

Lean: `ExpansionPlug.vertices`

The source edge carrier consists of all tagged component edges except (a, e), together with H's edges. For a surviving edge in component a, its inequality with (a, e) proves that its local edge differs from e; place it in the retained part of $F_a[e := H]$. Other component edges are unchanged, and new edges receive component tag a and the inserted tag. The inverse removes those tags. The inverse laws preserve both the dependent component index and the

retained-edge inequality; proof irrelevance removes any difference between witnesses of the same inequality.

Lean: ExpansionPlug.edgeTo

Lean: ExpansionPlug.edgeFrom

Lean: ExpansionPlug.edges

Transporting a graph vertex across an equality of graphs fixes all external ports and transports only its internal carrier. Equality elimination proves this fact. The vertex regrouping fixes every original pattern vertex, including pattern boundary vertices. An unchanged component's vertex has exactly the same attachment to the pattern before and after regrouping. An old vertex of the selected component becomes a retained vertex in its replacement graph, again with the same attachment.

For a newly inserted vertex, the internal case is the defining carrier map. An input port attaches to the selected edge's corresponding source vertex; an output attaches to its target vertex. The selected-component lemma, together with expansion's ordered source and target list definitions, shows that these attachments agree on both sides. Thus the maps preserve actual incidence, not just counts or labels.

Lean: ExpansionPlug.cast_vertex

Lean: ExpansionPlug.map_host

Lean: ExpansionPlug.map_other

Lean: ExpansionPlug.map_component

Lean: ExpansionPlug.map_inserted

Equality transport preserves internal labels and edge labels. It transports source and target lists by mapping the corresponding vertex transport over the original list. All four statements follow by eliminating the graph equality; the incidence statements then reduce to mapping the identity over a list. These lemmas expose the dependent casts required by the changed family without changing graph structure.

Lean: ExpansionPlug.cast_internalLabel

Lean: ExpansionPlug.cast_edgeLabel

Lean: ExpansionPlug.cast_source

Lean: ExpansionPlug.cast_target

Three equations describe the edge map: one for retained edges of the selected component, one for edges of other components, and one for newly inserted edges. They reduce directly from the edge-map definition and the component-index equality test.

The final graph isomorphism uses the proved vertex and edge bijections. Internal labels agree in each carrier case, and the three edge equations reduce edge-label preservation to the transport lemmas. For source incidence, a retained selected-component edge uses the selected-component vertex lemma; an unchanged edge uses the other-component lemma; a new edge uses the inserted-vertex lemma. Mapping these pointwise equalities over the entire source list proves preservation of its order as well as its entries. The target proof is identical with target lists.

This completes the replacement–expansion isomorphism with every interface and incidence law checked by Lean.

Lean: ExpansionPlug.edgeTo_same

Lean: ExpansionPlug.edgeTo_other

Lean: ExpansionPlug.edgeTo_inserted

Lean: ExpansionPlug.iso

63.2 Component isomorphisms and serial assembly

A second generic construction assembles component-wise isomorphisms under a common pattern. It permits a supplied relation on edge labels, so it also applies to recursive graph isomorphism, where corresponding box bodies may be isomorphic rather than literally equal.

Keep each pattern vertex fixed and apply the relevant component's vertex bijection inside each tagged component. Apply its edge bijection inside that component as well. The inverse laws are the component inverse laws. Pattern boundary attachment is fixed, and component insertion commutes with these maps by the internal/input/output cases. The resulting label-related isomorphism inherits the internal-label and edge-relation proofs pointwise. Its ordered incidence equations follow by mapping the insertion equality over the component incidence lists and using the component isomorphism's source or target equation.

Lean: ExpansionIso.vertices

Lean: ExpansionIso.edges

Lean: ExpansionIso.map_host

Lean: ExpansionIso.map_insert

Lean: ExpansionIso.iso

Serial composition is the original expansion of a two-slot pattern. A left-component occurrence is tagged by the left Boolean slot, and a right-component occurrence by the right slot. The edge maps are defined at each allowance, respecting the copying alphabet's base and successor representations. Both mapped occurrences retain exactly their original edge labels, and hence their ordered interfaces.

Lean: SerialPlug.leftEdge

Lean: SerialPlug.rightEdge

Lean: SerialPlug.leftEdge_label

Lean: SerialPlug.rightEdge_label

To replace an edge on the left, apply the generic replacement–expansion isomorphism with the selected slot equal to the left slot. The changed family is pointwise equal to the serial filling family formed from the replaced left graph and the untouched right graph. The two Boolean cases prove that equality. Interface transport aligns the inserted graph with the occurrence's retained label. The right-component theorem uses the same argument with the other slot selected.

Finally, the generic component-isomorphism construction applied to these two slots proves recursive isomorphism congruence for serial assembly. These are laws for the original expanded graph, with no replacement serial representation or additional semantic hypothesis.

Lean: SerialPlug.plug_left

Lean: SerialPlug.plug_right

Lean: SerialPlug.congr

63.3 The actual unary term-context filling law

The unary translation first serially connects its operation corolla to the argument graph, then appends the prescribed single-use environment distributor. Follow an argument occurrence through the right slot of the first composition and then the left slot of the second. Its label remains the argument edge's label by the two serial label lemmas.

For replacement at that occurrence, use the left serial replacement theorem to move replacement past the distributor suffix. Use the right serial replacement theorem to move it past the operation prefix. Lift the latter isomorphism through the unchanged suffix by serial congruence and compose the two isomorphisms. This gives the literal unary replacement law for any argument edge and any fitting replacement graph. The corresponding unary congruence theorem follows by applying serial congruence twice, fixing the operation and distributor graphs.

Lean: UnaryPlug.unaryEdge

Lean: UnaryPlug.unaryEdge_label

Lean: UnaryPlug.unary_plug

Lean: UnaryPlug.unary_congr

Specialise the argument graph to the annotated hole corolla. Following its single edge through the unary clause gives an explicit shallow site at the hole's term-and-environment interface.

To certify that particular site, first prove a general occurrence fact. If a graph has exactly one specified hole and an outer edge is labelled by that hole, the outer edge is the selected occurrence: otherwise the inventory's other-edge condition would declare the hole label hole-free, a contradiction. Its shallow site is therefore sole. Applying this fact to the existing unary single-hole inventory certifies the explicit address used in the filling theorem. Thus the theorem does not rely on a different existentially chosen hole address.

Lean: UnaryPlug.unaryHoleSite

Lean: UnaryPlug.sole_of_single_hole

Lean: UnaryPlug.unaryHoleSite_sole

Fill this actual site with an arbitrary graph G at the annotated interface. The unary replacement law identifies the result with the unary translation of the hole corolla filled by G. The generic corolla left-unit isomorphism identifies that filled corolla with G itself. Unary congruence lifts this isomorphism through the unchanged operation and distributor construction. Writing $\mathcal{T}$ for the original translation, composing proves

$$\mathcal{T}(o([-]))[\mathcal{T}(t)] \;\cong\; \mathcal{T}(o(t))$$

when G is the original translation of t, with the hole allowance equal to t's translation depth. The final declaration states precisely this specialisation using actual addressed replacement and the original `Translation.term` on the right. All unary operations are covered, including reference creation and dereference, for arbitrary scopes and argument terms. This is a structural translation theorem, not a theorem about executing either store operation.

Lean: `UnaryPlug.unary_hole_filling`

Lean: `UnaryPlug.unary_term_filling`

The full build and all three axiom audits pass. The dependency chain now contains a proved unary filling case: replacement–expansion interchange, serial replacement and congruence, unary replacement, then corolla elimination. Section 64 extends the result below boxes. The general context induction still needs parallel argument assembly and its padding compatibility. Term-context congruence and the copying execution's representation laws remain separate obligations. The displayed programs in Proposition 12.2 and every existing translation clause are unchanged.

64 Certified filling under arbitrarily nested binders

The abstraction case of addressed graph filling is now proved for the literal Figure 16 translation. Combining it with the unary clause gives a certified filling theorem for every original term context built from a hole, lambdas and unary operations, with arbitrarily many enclosing binders. The argument term is unrestricted: it may itself contain applications, binary operations and boxes. Its hole annotation includes every bound and ambient variable available there, and arbitrary atom environments are retained.

The theorem constructs a single address before inspecting the argument graph. It proves that the address is sole, that its translated host is binding-free, and that filling it with an original term translation reconstructs the original translation of the filled term, up to recursive isomorphism and equality of the depth indices. This last equality is not an equivalence between different padding allowances. Parallel argument assembly and common-allowance padding remain the structural cases needed for the unrestricted context theorem in Appendix B.1, item 3.

64.1 Deep replacement through serial assembly

Changing the body of a box in either component of a serial graph commutes with serial assembly up to ordinary graph isomorphism. The box's bound, free-variable and atom interfaces are fixed. Consequently the outer pattern vertices and the two components' tagged vertex and edge carriers remain in bijection: each tag and each local element is retained. The explicit maps split on the component tag because the two dependent filling families are not definitionally the same family of types.

For a left box, a left edge is either the selected occurrence, whose label changes to the new body on both sides, or another occurrence, whose label is unchanged. Every right label is unchanged. The right-box proof reverses those roles. Both source and target lists retain their original component entries. Mapping a component vertex through assembly gives the same result for an internal vertex, an input port or an output port; the serial pattern identifies the corresponding shared boundary in both graphs. Mapping these equalities over the ordered lists proves incidence preservation. Equal-labelled boxes at other occurrences are not changed.

Lean: `SerialDeepPlug.replaceBox_left`

Lean: `SerialDeepPlug.replaceBox_right`

An arbitrary addressed site can now be carried into either serial component. For a shallow site, use the component edge's original slot tag and transport the local interfaces along the already proved label equality. For a site inside a box, tag the outer box edge and keep its body address unchanged. This keeps the local depth and both local interfaces exactly as before.

To commute filling with the lifted site, split on the original site's first constructor. A shallow site is handled by the preceding section's serial replacement theorem; equality transport supplies the matching interfaces. A site inside a box is handled by the new box-body interchange theorem, using the fully filled inner body. No induction over the remaining address is necessary at this step: serial assembly never changes that box's body before filling. The resulting laws cover addresses at every finite nesting depth on either side of serial composition.

Lean: `SerialDeepPlug.leftSite`

Lean: `SerialDeepPlug.rightSite`

Lean: `SerialDeepPlug.plugAt_left`

Lean: SerialDeepPlug.plugAt_right

Hole-free labels outside a chosen component edge remain hole-free in the assembled graph, provided the other component is entirely hole-free. An assembled edge either has the selected component tag, where inequality of tagged edges implies inequality of local edges, or the other tag, where the other component's hypothesis applies. This proves both left and right laws.

A sole-hole certificate then transports with the actual site. For a shallow site, its marked label remains the same hole and the other-edge law certifies all remaining edges. For a deep site, retain the inner sole-hole certificate and use the other-edge law outside its enclosing box. These results certify the addresses used by the filling laws themselves.

Lean: SerialDeepPlug.left_other_free

Lean: SerialDeepPlug.right_other_free

Lean: SerialDeepPlug.leftSite_sole

Lean: SerialDeepPlug.rightSite_sole

64.2 Boxes and the full abstraction clause

An equality cast on the host's outer interfaces transports its address without changing the local hole interface or depth. Filling the transported site equals casting the original filling's outer interfaces. Both definitions and the equality proof follow by eliminating the two interface equalities. This is the precise transport needed for the existing lambda-box convention, which writes the bound variable first.

Lean: BoxContextPlug.Site.castHost

Lean: BoxContextPlug.plugAt_castHost

To enclose an address in a fresh box, select that box's singleton outer edge and retain the original body address below it. Filling this new address is isomorphic to boxing the filled body. Keep the empty outer internal carrier and singleton edge carrier fixed. Every outer edge is the selected edge, so its replacement label is exactly the box with the filled body. Sources and targets agree because the bound and free interfaces are unchanged.

Recursive body isomorphisms also lift through a box. The same outer carrier maps preserve incidence, while the recursive box-label relation is witnessed by the given body isomorphism. This separates the exact box-construction interchange from the later use of an isomorphism between body results.

Lean: BoxContextPlug.boxSite

Lean: BoxContextPlug.box_filling

Lean: BoxContextPlug.box_congr

The sole-hole predicate is invariant under outer interface equality casts, by equality elimination. Enclosing a sole site in a box preserves its certificate: the inner certificate is unchanged and there is no other outer edge, since the box's edge carrier is a singleton. Thus neither transport nor boxing introduces an extra hole.

Lean: BoxContextPlug.SoleHole.castHost

Lean: BoxContextPlug.boxSite_sole

Outer interface casts preserve recursive isomorphism, again by equality elimination. For the

actual lambda box, first apply the existing cast from $k+1$ variables to $1+k$ variables, then enclose the transported address in the box. The box-filling isomorphism together with the host-cast filling equality proves the law for this exact lambda-box constructor. The ordered bound-variable convention is preserved; no alternative box interface is introduced.

Lean: BoxContextPlug.castInterfaces_congr

Lean: BoxContextPlug.lambdaBoxSite

Lean: BoxContextPlug.lambdaBox_filling

Follow the body address into its lambda box, through the right side of the lambda-operation prefix, and through the left side of the prescribed single-use distributor suffix. This defines the full abstraction address.

The filling law is a composition of three proved isomorphisms: commute filling past the distributor suffix, commute it past the lambda prefix, and apply the lambda-box filling law. Serial congruence lifts the inner isomorphisms through the unchanged surrounding components. Abstraction congruence follows from the same serial and box congruence laws, with the original interface cast applied to the given body isomorphism. In particular, these proofs retain both the lambda operation and all distributor edges.

Lean: BoxContextPlug.abstractionSite

Lean: BoxContextPlug.abstraction_filling

Lean: BoxContextPlug.abstraction_congr

A unary constructor lifts any body address through its operation prefix and distributor suffix, even when that address is below boxes. The two deep serial filling laws prove the unary filling law directly. This strictly extends the preceding section's shallow unary edge result.

For abstraction, sole-hole preservation follows in order from host-cast preservation, singleton boxing, the hole-free lambda prefix, and the hole-free distributor suffix. For a unary operation, use the corresponding hole-free operation and suffix. Each constructor therefore supplies both the filling isomorphism and the sole-hole proof for the same explicitly lifted address.

Lean: BoxContextPlug.unarySite

Lean: BoxContextPlug.unary_filling

Lean: BoxContextPlug.abstractionSite_sole

Lean: BoxContextPlug.unarySite_sole

64.3 Induction over the original nested contexts

A scoped graph packages its finite allowance with its graph. To compare such packages, define scoped isomorphism to mean an equality of their allowances and a recursive graph isomorphism after transporting along that equality. This merely handles dependent indices; it does not quotient different depths, assert padding invariance, or alter any graph.

Reflexivity uses the identity recursive isomorphism. An isomorphism between graphs at the same depth gives a scoped isomorphism immediately. For transitivity, unpack the three depths, eliminate the two equalities, and compose the recursive isomorphisms. At an already common depth, proof irrelevance reduces any equality witness to reflexivity, so scoped isomorphism

yields an actual recursive isomorphism. Abstraction and unary congruence follow by eliminating the depth equality and applying their proved graph-constructor congruence laws.

Lean: NestedContextFilling.ScopedIso

Lean: NestedContextFilling.refl

Lean: NestedContextFilling.ofIso

Lean: NestedContextFilling.trans

Lean: NestedContextFilling.sameDepth

Lean: NestedContextFilling.abstraction

Lean: NestedContextFilling.unary

The allowed-context predicate is an inductive predicate on the original annotated context grammar: a hole is allowed, an enclosing lambda preserves the predicate, and any enclosing unary operation preserves it. It introduces no new term syntax and imposes no restriction on the eventual argument term. It intentionally does not yet cover application or binary-operation context constructors.

The literal hole corolla has a shallow address at its annotated term and environment interface at every finite allowance. Its sole-hole proof uses the singleton edge carrier. Filling that address is recursively isomorphic to the supplied graph by the generic corolla left-unit isomorphism.

Lean: NestedContextFilling.UnaryLambda

Lean: NestedContextFilling.holeSite

Lean: NestedContextFilling.holeSite_sole

Lean: NestedContextFilling.hole_filling

Induct on the allowed-context derivation. The induction statement constructs one actual address in the literal context graph, at the original hole's allowance, proves that it is sole, and proves the filling correspondence for *every* graph at that interface. The address therefore does not depend on the eventual argument.

The hole case is the corolla law. For a lambda, lift the induction address through the abstraction constructor. Its sole-hole certificate follows from the abstraction certificate. Compose the abstraction filling isomorphism with abstraction congruence applied to the induction hypothesis. For a unary operation, use the analogous unary address, certificate, filling law and congruence. The resulting target is exactly the existing constructor interpretation at the supplied argument graph.

Finally choose the hole allowance to be the original argument term's translation depth and supply its original translated graph. The existing constructor-interpretation theorem identifies the target with translation of the syntactically filled term. The independent binding-free theorem certifies the literal host context graph. This proves the final conjunction: an actual address, its sole-hole proof, host binding-freedom, and scoped isomorphism between actual filling and the unchanged filled-term translation. The proof does not appeal to execution or observational equivalence.

Lean: NestedContextFilling.unaryLambda_filling

Lean: NestedContextFilling.unaryLambda_term_filling

The regression context is $\lambda.\texttt{ref}(\lambda.![-])$. Its hole has two more variable slots than its outer

environment and the same atom slots. The allowed-context derivation follows the two lambdas and two unary operations in order. A reflexivity theorem checks the exact filled term, with neither binder nor operation moved or erased.

For every ambient environment size, hole name and argument term at that annotated scope, the final regression obtains a sole-hole, binding-free host and the actual graph-filling correspondence with the direct translation of $\lambda.\,\mathtt{ref}(\lambda.\,!t)$. This exercises address transport below two boxes with operations between and below them. It is a structural regression; it makes no claim about executing reference creation or dereference on the resulting term.

Lean: `NestedFillingExamples.twoBinderContext`

Lean: `NestedFillingExamples.twoBinderContext_allowed`

Lean: `NestedFillingExamples.twoBinderContext_fill`

Lean: `NestedFillingExamples.twoBinderContext_graph_filling`

All new declarations pass the full build, and the three axiom audits report only standard Lean axioms. This closes the standalone abstraction and nested lambda/unary filling cases. Item 3 still needs parallel argument assembly, compatible padding, the unrestricted context induction, and term-context congruence. Execution's isomorphism and padding laws must be discharged by the copying machine. Proposition 12.2 and its translation remain unchanged.

65 Parallel argument filling and depth padding

This section proves the remaining individual constructor laws for actual addressed filling: ordered parallel argument assembly, both arguments of application and binary operations, and any finite amount of representation padding. These are laws about the existing finite graphs and their actual replacement operation. They do not change the term language, the proposition's programs, the Figure 16 translation, or its variable/atom ordering.

The dependency chain is generic parallel replacement, recursive parallel congruence, arbitrary-depth component addresses, the full pair and operation clauses, and padding of addresses and fillings. The previous section supplies abstraction and unary filling. The unrestricted context induction must still combine these ingredients with the maximum-depth choices of the constructor interpreter. In particular, a padded host requires padding the replacement at the hole's local allowance, not merely at the host's allowance. Execution invariance under these representation changes remains a separate obligation of the copying machine. No observational equivalence or robustness claim is inferred solely from a graph isomorphism.

65.1 Replacement in an ordered parallel family

Let F_b be a finite family of graphs, let a be a selected component, and let e be an edge of F_a. A graph H has exactly the input and output interface of e. The updated family replaces e by H in component a and retains every other component. Its two evaluation lemmas split on whether the component index equals a, eliminating the dependent interface transport in the selected case. The parallel boundary is arbitrary but fixed; it may permute component boundary blocks.

Lean: `ParallelPlug.family`

Lean: `ParallelPlug.family_same`

Lean: `ParallelPlug.family_other`

Before regrouping, internal vertices are either a tagged vertex of an old component or a vertex of the inserted graph. After regrouping, they are tagged vertices of the updated family. The forward map keeps every unchanged component tag, marks old vertices in component a as retained, and places inserted vertices in component a with the inserted tag. The two evaluation lemmas identify its action on old vertices in the selected and unselected components. The inverse inspects the component tag and, in component a, the retained or inserted tag. Splitting on those same alternatives proves both inverse laws. Thus the regrouping is a bijection without any assumptions about vertex names.

Lean: `ParallelPlug.vertexTo`

Lean: `ParallelPlug.vertexTo_same`

Lean: `ParallelPlug.vertexTo_other`

Lean: `ParallelPlug.vertexFrom`

Lean: `ParallelPlug.vertices`

The edge bijection uses the analogous regrouping, but the old carrier omits the selected edge. A retained edge in component a carries a proof that it is not e; inequality of tagged edges supplies this proof. Edges of other components keep their names and tags. New edges are placed in the selected component. The inverse either returns the unchanged tagged edge or

strips the inserted tag. Case analysis proves the inverse identities, with equality transport of carriers cancelling in each case.

Lean: ParallelPlug.edgeTo

Lean: ParallelPlug.edgeFrom

Lean: ParallelPlug.edges

Three attachment identities prove that regrouping commutes with the parallel boundary: for a vertex of another component, a vertex of the selected component, and a vertex inserted from H. Internal vertices follow the explicit maps. Input and output vertices use the same boundary bijection on both sides. For an inserted boundary vertex, attachment first reaches the selected edge's source or target port, then the selected component's external embedding; this is exactly its attachment after regrouping. The three edge evaluation lemmas record the retained-selected, unselected, and newly inserted cases, allowing the later dependent label and incidence expressions to be rewritten.

Lean: ParallelPlug.map_other

Lean: ParallelPlug.map_component

Lean: ParallelPlug.map_inserted

Lean: ParallelPlug.edgeTo_same

Lean: ParallelPlug.edgeTo_other

Lean: ParallelPlug.edgeTo_inserted

Replacing the tagged edge after parallel assembly is isomorphic to assembling the updated family. Use the vertex and edge bijections above. Labels agree by the selected/unselected family equations. For sources and targets, split each edge into the same three cases. Mapping an ordered incidence list twice equals mapping the composite; apply the appropriate attachment identity to every entry. This proves equality of entire ordered lists, not merely membership or unordered incidence. The theorem introduces no acyclicity or self-loop premise.

Lean: ParallelPlug.iso

Parallel assembly also transports component-wise isomorphisms whose edge labels are related by a supplied relation. Both carrier maps retain the component index and apply that component's bijection. Their inverses work component-wise. Boundary embedding commutes with the vertex map: internal vertices use the component bijection, while external vertices use the fixed parallel boundary. Component label and incidence laws then prove the assembled related-label isomorphism by mapping this identity over each incidence list. Instantiating the label relation with recursive box isomorphism gives the congruence used below.

Lean: ParallelIso.vertices

Lean: ParallelIso.edges

Lean: ParallelIso.map_embed

Lean: ParallelIso.iso

65.2 Deep sites in either parallel component

Specialise the family to two graphs using the existing Boolean component index and an arbitrary ordered boundary. The net abbreviation denotes that exact parallel constructor. A left or right edge retains its local name and receives the corresponding tag; its label is unchanged. Specialising the generic replacement isomorphism proves the two shallow filling laws. The updated family agrees component-wise with the pair containing the replaced graph, and equality transport yields the stated interfaces. Specialising parallel related-label congruence gives recursive congruence for both graphs.

Lean: `ParallelContextPlug.net`

Lean: `ParallelContextPlug.leftEdge`

Lean: `ParallelContextPlug.rightEdge`

Lean: `ParallelContextPlug.leftEdge_label`

Lean: `ParallelContextPlug.rightEdge_label`

Lean: `ParallelContextPlug.plug_left`

Lean: `ParallelContextPlug.plug_right`

Lean: `ParallelContextPlug.congr`

Changing a box body in either component commutes with parallel assembly. The outer carrier bijections retain the component tags and local elements. In the selected component, split an edge into the chosen box edge and every other edge: only the chosen label changes. In the opposite component every label is unchanged. Box metadata fixes the external interface, so neither source nor target incidence changes. The boundary embeddings agree on internal, input and output vertices, proving both ordered incidence laws.

Lean: `ParallelContextPlug.replaceBox_left`

Lean: `ParallelContextPlug.replaceBox_right`

Lift an arbitrary-depth occurrence into either component. A shallow address uses the tagged edge and the label equality to transport the local interfaces. An address inside a box uses the tagged enclosing box and retains its body address. Thus parallel assembly changes neither the local allowance nor the hole interface. Filling a lifted shallow site uses the shallow replacement law; filling a deep site uses the box-body interchange law with the already filled inner body. This covers arbitrary finite nesting without flattening boxes or identifying distinct equally labelled occurrences.

Lean: `ParallelContextPlug.leftSite`

Lean: `ParallelContextPlug.rightSite`

Lean: `ParallelContextPlug.plugAt_left`

Lean: `ParallelContextPlug.plugAt_right`

To preserve a sole-hole certificate, assume the fixed opposite component is hole-free. Any other edge is either in the selected component, where inequality of tags reduces to inequality of local edges, or in the fixed component, where hole-freedom applies directly. A shallow site's marked hole label is preserved. A deep site's inner certificate is retained and the same other-edge

argument applies outside its enclosing box. This certifies precisely the addresses used in the filling laws.

Lean: ParallelContextPlug.left_other_free

Lean: ParallelContextPlug.right_other_free

Lean: ParallelContextPlug.leftSite_sole

Lean: ParallelContextPlug.rightSite_sole

65.3 The unchanged pair, application and binary clauses

The Figure 16 pair uses the prescribed middle-block exchange of the two variable/atom interfaces. Instantiate the parallel laws with that exact boundary and apply the existing outer-interface equality cast. This defines both pair addresses. The host-cast filling law followed by parallel filling proves both pair filling laws. Parallel recursive congruence lifts through the same cast. The component sole-hole certificates also survive this cast, giving both certified pair addresses.

Lean: BinaryContextPlug.pairLeftSite

Lean: BinaryContextPlug.pairRightSite

Lean: BinaryContextPlug.pairLeft_filling

Lean: BinaryContextPlug.pairRight_filling

Lean: BinaryContextPlug.pair_congr

Lean: BinaryContextPlug.pairLeftSite_sole

Lean: BinaryContextPlug.pairRightSite_sole

An application is the serial composite of the application operation, the ordered pair of arguments, and the original environment distributor. To lift an argument address, first lift into the pair, then into the right side of the operation prefix, and finally into the left side of the distributor suffix. Compose the two serial filling isomorphisms with pair filling and serial congruence. This proves the left and right application laws separately. The operation and distributor contain no holes, so the serial sole-hole laws and pair certificates prove uniqueness of the selected occurrence. Finally, recursive congruence of both arguments follows by pair congruence and two applications of serial congruence. No distributor is removed or replaced.

Lean: BinaryContextPlug.applicationLeftSite

Lean: BinaryContextPlug.applicationLeft_filling

Lean: BinaryContextPlug.applicationLeftSite_sole

Lean: BinaryContextPlug.applicationRightSite

Lean: BinaryContextPlug.applicationRight_filling

Lean: BinaryContextPlug.applicationRightSite_sole

Lean: BinaryContextPlug.application_congr

For each binary operation, use its original primitive operation edge in the same serial construction. The left and right site definitions follow the corresponding argument tag through

the pair, prefix and suffix. Their filling proofs compose serial and pair filling as above; their sole-hole proofs use hole-freedom of this binary operation and the environment distributor. Recursive congruence follows from pair and serial congruence. The operation parameter is arbitrary, so these results do not restrict the surrounding operation alphabet.

Lean: BinaryContextPlug.binaryLeftSite

Lean: BinaryContextPlug.binaryLeft_filling

Lean: BinaryContextPlug.binaryLeftSite_sole

Lean: BinaryContextPlug.binaryRightSite

Lean: BinaryContextPlug.binaryRight_filling

Lean: BinaryContextPlug.binaryRightSite_sole

Lean: BinaryContextPlug.binary_congr

65.4 Padding the address and the replacement together

Two equality transports are needed. Changing the local interface of a site is equivalent, upon filling, to reindexing the replacement by the inverse interface equalities. Transporting a site's host along graph equality leaves the resulting filling equal to the old filling. Define the host transport by equality elimination. Both filling equalities follow by eliminating the relevant equalities, after which the reflexive reindexings disappear.

Lean: PaddingPlug.plugAt_castInterfaces

Lean: PaddingPlug.Site.castGraph

Lean: PaddingPlug.plugAt_castGraph

The uniform padding label map preserves primitive labels, and maps a box to the same bound/free-variable/free-atom metadata with its body raised once. At allowance zero it removes the primitive label's type-level wrapper. Splitting on the depth and label shows that both ordered signatures are unchanged. Relabelling by this map gives a graph with definitionally unchanged outer carriers and incidence. It equals the existing recursive raise operation: at depth zero the definitions coincide; at positive depth equality of the label functions follows by primitive/box case analysis. This is a proof about the existing padding, not a replacement definition of translation.

Lean: PaddingPlug.label

Lean: PaddingPlug.label_input

Lean: PaddingPlug.label_output

Lean: PaddingPlug.graph

Lean: PaddingPlug.graph_eq_raise

One-step padding commutes with shallow replacement up to ordinary isomorphism. Both graphs retain the same old and inserted carrier tags, so identity carrier bijections suffice. The labels coincide under the uniform padding map. For retained edges the source and target lists coincide. For inserted edges, the local signature casts cancel and each internal/input/output attachment agrees; map composition proves equality of the ordered incidence lists.

Padding also commutes with changing a particular box body. Keep all outer carrier names fixed. At the selected edge both labels are the same padded box with the new padded body; all other labels agree. The outer incidence never changes. This second law handles a site's enclosing box.

Lean: `PaddingPlug.plug`

Lean: `PaddingPlug.replaceBox_iso`

Raise an address by structural induction. A shallow site in the uniform relabelling has the old local interface by the two signature equalities; transport its host to the original raise operation. A deep site retains its outer edge, pads the enclosing box body, and recursively raises its inner address. Both the host and the local hole allowance increase by one.

The filling theorem states that padding the filled graph is recursively isomorphic to filling the raised address with the raised replacement. In the shallow case, the equality-transport laws reduce the statement to shallow padding/replacement interchange. In the deep case, compose box-body interchange with recursive congruence of the changed box body, using the induction hypothesis for the inner address. The replacement is raised at its own local depth, which may differ from the host depth.

Lean: `PaddingPlug.site`

Lean: `PaddingPlug.filling`

Host equality transport preserves a sole-hole certificate by equality elimination. For every other outer edge, the existing per-label padding law preserves hole-freedom; the uniform presentation has the same property by primitive/box case analysis. Induction on the site now proves that its raised address is still sole. A shallow marked hole stays the same primitive hole, and every other edge stays hole-free. A deep address uses the inner induction hypothesis and the same other-edge argument for its enclosing graph.

Lean: `PaddingPlug.site_castGraph_sole`

Lean: `PaddingPlug.graph_other_free`

Lean: `PaddingPlug.site_sole`

Padding preserves recursive isomorphism. Retain the original outer carrier bijections and incidence equations. At depth zero equal primitive labels remain equal. At positive depth, related primitives remain equal, mismatched primitive/box cases are impossible, and related box bodies remain related by the depth induction hypothesis. Bound and free metadata stay identical.

Iterate the raised-address construction n times. A site with host allowance d and local allowance r then has allowances $d + n$ and $r + n$. Iterating the preceding congruence gives isomorphism preservation under the original padding function. Induction on n proves the filling law: raise the preceding filling isomorphism, then compose it with the one-step filling theorem for the current padded site and replacement. A separate induction iterates sole-hole preservation. These statements retain the exact same number of padding steps on the host and replacement.

Lean: PaddingPlug.congr

Lean: PaddingPlug.padSite

Lean: PaddingPlug.pad_congr

Lean: PaddingPlug.pad_filling

Lean: PaddingPlug.padSite_sole

65.5 Combined regressions and completion boundary

The regressions combine two padding steps, an arbitrary-depth occurrence and the full application clause, on each side separately. The fixed opposite argument is any hole-free graph at the resulting allowance and may contain boxes. The same explicitly constructed address is proved sole, and its filling is recursively isomorphic to the application of the padded original filling. The proof composes application filling with the inverse iterated-padding filling law under application congruence. Thus both component tags, the ordered boundary exchange, the serial distributors, and distinct host/local allowances are checked together. These are structural regressions, not executions of a copying machine; callback divergence and store behaviour still require item 2.

The individual parallel and padding filling laws are now discharged. Remaining item 3 work is the unrestricted context induction, including cumulative padding at maximum-depth joins, and the passage from structural correspondence to observational congruence with proved machine representation laws. The other Appendix B.1 obligations are unchanged.

Lean: ParallelFillingExamples.paddedApplicationLeft_filling

Lean: ParallelFillingExamples.paddedApplicationRight_filling

66 Unrestricted term-context filling

The structural graph-filling theorem is now proved for every context in the original annotated grammar, with arbitrary finite variable and atom scopes. It includes holes below binders, unary operations, and either argument of application and binary operations. The terms of Proposition 12.2, the Figure 16 translation and its depth choices are unchanged.

The theorem supplies one actual sole-hole address before the argument graph is given. The translated host is binding-free. Filling this address with the appropriately padded translation of any term recovers the original translation of the filled term up to recursive graph isomorphism and equality of the outer depth indices. Neither machine determinism nor execution invariance is a premise of this structural result. Conversely, no termination equivalence follows without the remaining semantic connection: the copying machine must respect isomorphism and padding, and observational congruence must be established.

66.1 Exact composition of depth padding

Write $P_d^n(G)$ for the original operation that raises the allowance of a graph G from d to $d+n$. Equality transport changes only a type index; it does not insert another graph constructor. Padding and one-step raising commute with transport along $d = e$, because eliminating that equality leaves the same expressions on both sides.

Lean: `PaddingComposition.pad_castDepth`

Lean: `PaddingComposition.raise_castDepth`

Successive padding satisfies

$$P_{d+n}^m(P_d^n(G)) = \mathrm{cast}_{d+(n+m)=(d+n)+m}(P_d^{n+m}(G)).$$

Induct on m. With no additional step the expressions coincide. At a successor, unfold the last raising operation, apply the induction hypothesis, and move the equality transport through raising using the preceding lemma. The remaining transports agree because they express the same associativity of addition. This is graph equality, not an observational claim.

Recursive graph isomorphism also survives depth equality transport by equality elimination. Raising to a specified common allowance consists of iterated padding followed by such a transport. Composing the previous section's padding congruence with transport congruence therefore proves common-allowance congruence.

Lean: `PaddingComposition.pad_add`

Lean: `PaddingComposition.castDepth_congr`

Lean: `PaddingComposition.padTo_congr`

An address has two independently indexed allowances: its host depth and its local hole depth. Transporting the host depth leaves the local depth fixed. Filling the transported address equals transporting the filled graph, and the sole-hole certificate is preserved. Each statement follows by eliminating the host-depth equality.

Transporting the local depth instead leaves the host graph unchanged. Filling this transported address with a graph at the new local index equals filling the old address with that graph transported by the inverse equality. Its sole-hole certificate is again preserved. These definitions

and proofs separate the two indices rather than silently identifying them.

Lean: PaddingComposition.Site.castDepth

Lean: PaddingComposition.plugAt_castDepth

Lean: PaddingComposition.SoleHole.castDepth

Lean: PaddingComposition.Site.castLocal

Lean: PaddingComposition.plugAt_castLocal

Lean: PaddingComposition.SoleHole.castLocal

Suppose a site lies in a host of allowance h, while its local allowance is $d + n$. Padding the host by m steps yields a site with local allowance $(d + n) + m$. Transport this local index to $d + (n + m)$; this is the extended site. The previous padding certificate and local-transport certificate prove that it remains sole.

Filling the extended site with $P_d^{n+m}(A)$ is recursively isomorphic to P_h^m applied to the original site filled with $P_d^n(A)$. First use the local-transport filling equality. The additive padding equality rewrites its replacement as $P_{d+n}^m(P_d^n(A))$. The inverse of the previous section's iterated padding/filling isomorphism then gives the result. Thus accumulated padding is applied to the original argument at its own allowance.

Lean: PaddingComposition.extendSite

Lean: PaddingComposition.extendSite_sole

Lean: PaddingComposition.extend_filling

For a target host allowance $N \geq h$, take $m = N - h$ in the preceding construction, then transport the host index $h + (N - h)$ to N. This gives a site in the existing common-allowance padding of the host, with local depth $d + (n + (N - h))$. The certificate follows from extended-site and host-transport certificate preservation. The filling law follows by transporting the extended-site isomorphism along the same host equality. No additional padding is charged for surrounding boxes already counted in h.

Lean: PaddingComposition.toSite

Lean: PaddingComposition.toSite_sole

Lean: PaddingComposition.to_filling

66.2 Congruence of the constructor interpretation

Recall that scoped isomorphism includes equality of the two graph depths, followed by recursive isomorphism at that depth. For application, unpack both argument isomorphisms and eliminate their depth equalities. The two expressions then choose the same maximum allowance. Apply common-allowance congruence to each argument and the original application congruence to their padded graphs. The resulting scoped isomorphism has reflexive equality of outer depths. For each binary operation the proof is identical with the existing binary constructor congruence in place of application congruence.

Lean: ContextFilling.application

Lean: ContextFilling.binary

Every original context preserves scoped isomorphism of its argument graphs. Induct on the

context. The hole returns the given relation. Lambda and unary constructors apply their established congruence theorems. At either argument position of application or a binary operation, use the corresponding new scoped congruence with the induction hypothesis in the selected position and reflexive scoped isomorphism of the fixed sibling in the other position. This is a structural congruence theorem; it makes no claim about execution.

Lean: `ContextFilling.apply_iso`

66.3 The unrestricted addressed filling theorem

Fix a context C whose hole has scope (v, a) and whose outer scope is (k, h), an argument allowance d, and a hole name. Let T_C denote the original translated holed graph, and let D_C denote its outer allowance. There exist a natural number n and a sole site s of T_C, with local allowance $d + n$ and interface

$$[\mathsf{term}] \longrightarrow \mathsf{environmentInterface}(v, a),$$

such that, for every fitting graph A at allowance d,

$$(D_C, \mathrm{plugAt}(s, P_d^n(A))) \quad \mathsf{ScopedIso} \quad \mathrm{apply}(C, (d, A)).$$

The quantifier over A follows the choices of n and s: both are fixed by the context, allowance and hole name. In particular, the proof cannot choose a different occurrence after inspecting the argument graph.

Hole. Choose $n = 0$ and the hole corolla's unique edge. The existing corolla replacement isomorphism recovers A, and the singleton carrier proves the sole-hole certificate.

Lambda. Retain the induction hypothesis's n. Lift its address through the original abstraction clause. Its sole-hole law supplies the certificate. Compose the abstraction filling isomorphism with abstraction congruence applied to the induction hypothesis. The host allowance increases by one; the hole's local allowance does not.

Unary operation. Retain n, lift the address through the original unary clause, and use its sole-hole law. Compose unary filling with unary congruence of the induction hypothesis. The operation and distributor are unchanged.

Application, left argument. Let the selected subcontext host have allowance D and the fixed sibling term have depth q. Set $N = \max(D, q)$. Pad the fixed sibling to N; it is hole-free because term translation and padding preserve this property. Lift the induction hypothesis's site to allowance N using the new common-allowance site. Its local padding count is now $n + (N - D)$. Lift that address into the application's left argument. The padding and application certificates prove that it remains sole.

For a graph A, first use application filling, then application congruence with the common-allowance filling isomorphism in the left position and the identity in the fixed position. This yields application of the padded original subcontext filling and the fixed sibling. Finally use scoped application congruence with the induction hypothesis to reach the original constructor interpretation. The intermediate scoped isomorphism ensures that the maximum depth chosen on the target side is exactly the same N.

Application, right argument. Choose $N = \max(q, D)$ and the same deficit $N - D$ for the selected subcontext. Use the right-site and right-filling laws. Place the identity and induction isomorphisms in the opposite positions. The ordered pair boundary is still the original one.

Binary operation, either argument. Repeat the corresponding left or right application argument with that binary operation's filling and congruence laws. The operation parameter remains unrestricted. These seven cases exhaust the original context grammar; no restricted-context predicate is required.

Lean: `ContextFilling.context_filling`

For a term t, instantiate the theorem at $d = \mathsf{depth}(t)$ and take A to be its unchanged translation. The previously proved constructor interpretation identity identifies the target with the original translation of $C[t]$, including its exact outer depth. The existing theorem that every translated term context is binding-free supplies that certificate. Thus the same site is sole, its host is binding-free, and its actual filling has the required structural correspondence for arbitrary scopes and argument terms.

The graph-context interface can also be packaged using this very site, rather than selecting a potentially different site merely from hole existence. Combine the translated host, the constructed site, its sole-hole proof and the host's binding-free proof into a graph context. Its graph is literally equal to the original translated host, and the universal filling law is the one just proved. This packages a checked structural interface; it introduces no assumption about a copying execution.

Lean: `ContextFilling.context_term_filling`

Lean: `ContextFilling.context_graph_filling`

66.4 Mixed-context regression and semantic boundary

The fixed sibling term is $\lambda x.\lambda y.y$, represented with the original scoped variable indices, in arbitrary ambient environments. The mixed context has an outer lambda and dereference, and its path to the hole passes through left application, right binary addition, right application, and left binary assignment. Its fixed siblings include the two-box term and unit. Hence every context constructor occurs on one path, and the hole's scope includes the variable bound by the outer lambda.

For a hole allowance of zero the translated host has depth three: the binary/application portion meets a depth-two sibling, and the outer lambda adds one. This equality is definitional, so the regression genuinely includes a maximum-depth mismatch. A second definitional equality checks that term filling retains the exact displayed constructor expression. The unrestricted term-filling theorem then proves the actual sole-hole, binding-free graph correspondence for every argument term and every ambient variable/atom scope. This is a structural regression, not a claim that the mixed expression terminates or is well behaved under a store machine.

Appendix B.1 item 3 no longer contains unrestricted addressed correspondence as an unfinished task. It still contains observational term-context congruence and the observation/padding audit. In particular, arguments of different minimal depths must be compared through common representations, and the concrete execution must discharge its isomorphism and padding invariance obligations. The copying machine and the template proofs remain pending; Proposition 12.2 is not yet proved.

Lean: `ContextFillingExamples.twoBoxSibling`

Lean: `ContextFillingExamples.mixedArgumentContext`

Lean: `ContextFillingExamples.mixedArgumentContext_depth`

Lean: `ContextFillingExamples.mixedArgumentContext_fill`

Lean: `ContextFillingExamples.mixedArgumentContext_graph_filling`

67 Common allowances and observation transfer

The preceding section proves structural filling for all original term contexts. To use that correspondence semantically, comparisons made at different finite representation allowances must be related. This section proves that every closing context lifts through padding, that observational refinement at a larger allowance reflects to the original allowance, and that the existing all-allowances definition of term equivalence is equivalent to checking a cofinal collection of common allowances.

The structural statements are unconditional. The semantic transfer theorems explicitly require execution invariance under recursive graph isomorphism and one-step padding of closed programs. They do not assume determinism or template robustness. These representation laws remain obligations for the copying UAM; they are not project axioms. A checked counterexample below shows that even determinism and isomorphism invariance do not imply padding invariance.

67.1 Exact padding of programs and closing contexts

Padding a graph at allowance d to $d+n$ equals its original n-step padding. The proof transports the equality $(d + n) - d = n$ through the dependent graph index; the resulting equality of depth proofs is immaterial.

Choosing an intermediate sufficient allowance changes nothing:

$$\mathsf{padTo}_N(\mathsf{padTo}_e(G)) = \mathsf{padTo}_N(G) \qquad (d \le e \le N).$$

Write $e = d + n$ and $N = (d + n) + m$. Reduce the two successive paddings to their iterated forms, apply the previous section's additive padding equality, and transport the final allowance along associativity of addition. Equality elimination shows that transporting a padded graph is the same as choosing that transported final allowance directly. This is literal graph equality.

Lean: `ObservationPadding.padTo_add`

Lean: `ObservationPadding.padTo_trans`

An open program consists of its graph together with focus-free and hole-free proofs. Pad its graph either by a given number of steps or to a specified adequate allowance, and transport both certificates using the established label-property laws. Equality of the underlying graphs implies equality of these packaged programs, since the other fields are proofs. Consequently padding an open program to $d + n$ agrees with padding it by n steps.

Lean: `ObservationPadding.pad`

Lean: `ObservationPadding.padTo`

Lean: `ObservationPadding.ext_graph`

Lean: `ObservationPadding.padTo_add`

For a closed program, pad its stored graph and increase its stored depth by the same number of steps. Focus-freedom and hole-freedom again follow from padding preservation. Thus every padded program remains an admissible input to the existing execution initializer.

Lean: `ObservationPadding.pad`

A closing context with local allowance d and host allowance D lifts to one with allowances $d + n$ and $D + n$. Pad its actual host graph, take the previously constructed padded site,

and retain its sole-hole and binding-free certificates using their proved padding laws. This construction works for every original closing context, including holes below arbitrary boxes; no new restriction on observation contexts is imposed.

Closing a padded program in this padded context is recursively isomorphic to padding the original closed program. This is exactly the iterated deep-filling law, now with the closing-context and program certificates packaged around it. The isomorphism compares graphs at the same resulting host allowance.

Lean: `ObservationPadding.pad`

Lean: `ObservationPadding.pad_close_iso`

67.2 The execution obligation and observation reflection

Define execution padding invariance by requiring, for every admissible closed program P, universal-cost machine equivalence between its initial state and the initial state of P padded once. Universal cost forgets the number of steps but still observes successful termination, not stuckness. This property is explicitly passed to every theorem that uses it.

The one-step property implies invariance under any finite number of steps of padding. Induct on their number. Zero padding gives reflexive refinement in both directions. At a successor, apply the one-step obligation to the already padded program and compose each direction with the induction hypothesis. The original iterative definition of padding makes the final program coincide with the desired successor padding.

Lean: `ObservationPadding.RespectsPadding`

Lean: `ObservationPadding.initial_pad`

Let C be a closing context and G an open program. First apply iterated execution padding invariance to the closed program $C[G]$. Next apply the execution's separate isomorphism obligation to the proved isomorphism between its padded graph and the graph obtained by closing the padded G in the padded C. Compose the two machine equivalences, reversing their order in the converse direction. This relates the two actual initialized closed observations, including their admissibility proofs.

Lean: `ObservationPadding.close_pad`

Suppose the two padded open programs refine at allowance $d + n$. To prove refinement at d, take any original closing context C. Transfer the first closed observation to the padded context, apply the supplied refinement there, and transfer the second observation back. Transitivity gives the requested refinement in C. Since C was arbitrary, the theorem covers the original full observation class. Apply this argument in both directions to obtain reflection of equivalence.

For a specified adequate allowance $N \geq d$, write $N = d + n$ and use the exact open-program padding equality to reduce to this result. These are reflection laws from larger allowances. No converse, no automatic equivalence between all representations, and no copying execution is inferred from them.

Lean: `ObservationPadding.refines_of_pad`

Lean: `ObservationPadding.equiv_of_pad`

Lean: `ObservationPadding.equiv_of_padTo`

Recursive isomorphism of open program graphs also gives observational equivalence when execution respects isomorphism of closed graphs. In each closing context, deep replacement

congruence lifts the fragment isomorphism to an isomorphism of the closed fillings. Apply the closed execution obligation and select its forward and converse refinements for the two contextual directions. Padding invariance is unnecessary for this equal-allowance law.

Lean: `ObservationPadding.equiv_iso`

67.3 A cofinal criterion for the unchanged term equivalence

The translated-program wrapper uses the original term translation padded to the chosen allowance. Padding that wrapper through an intermediate adequate allowance gives exactly the wrapper at the final allowance. Apply graph-level padding transitivity and then the open-program equality principle. The source term, its translation and its interfaces are unchanged.

Lean: `ObservationPadding.translatedProgram_padTo`

Under the two execution representation laws, the original definition $\mathsf{ObservationalEquiv}(t, u)$ is equivalent to:

$$\forall b\ \exists N \geq b, \quad \mathsf{depth}(t) \leq N \ \wedge\ \mathsf{depth}(u) \leq N \ \wedge\ \mathsf{Equiv}_{\mathrm{any}}(T_N(t), T_N(u)),$$

where T_N is the unchanged translated-program wrapper. The witnessing allowances need not be every natural number; they must be arbitrarily large.

For the forward direction, given b, choose the maximum of b and the two term depths, then instantiate the original all-allowances hypothesis. For the converse, fix any adequate original allowance d. Obtain a witnessing $N \geq d$ from the cofinal hypothesis. The translated-program transitivity law identifies the translations at N with the translations at d padded to N. Equivalence reflection therefore supplies the required comparison at d. This argument handles different minimal term depths and preserves quantification over all original binding-free closing contexts.

This reduction supports later common-allowance proofs. It does not prove the cofinal hypothesis for Proposition 12.2, or remove the need to establish the two representation laws for its concrete machine.

Lean: `ObservationPadding.observationalEquiv_iff_cofinal`

67.4 A negative regression for the missing machine law

Consider an artificial execution with natural-number states, no transitions, and final predicate $d = 0$. Its initializer returns the closed graph's stored allowance. Final states are normal because no transitions exist. The execution is deterministic for the same reason: an alleged pair of outgoing transitions already includes a contradiction.

It respects recursive graph isomorphism. Such isomorphisms compare graphs at the same allowance, and both initializers therefore return exactly the same state. Reflexive machine refinement supplies both directions, regardless of the graph's labels or carriers. This execution is deliberately not the copying UAM; it tests which laws the abstract observation interface actually implies.

Lean: `ObservationPaddingExamples.depthSensitiveExecution`

Lean: `ObservationPaddingExamples.depthSensitive_deterministic`

Lean: `ObservationPaddingExamples.depthSensitive_respectsIso`

Take the unchanged closed unit translation at allowance zero, with its proved focus-free and hole-free certificates. Its initial state is zero, so the zero-step run is a successful observation. Pad the same closed program once. Its initial state is now one. Every run in the transition-free machine has zero length and unchanged state; it therefore cannot reach a final state. This proves failure of termination for the padded program.

If the execution satisfied padding invariance, its forward refinement for this unit program would turn the successful zero-step run into a successful run from the padded initializer, contradicting the preceding result. Thus padding invariance is independent of the supplied determinism and isomorphism proofs. An axiom audit of a conditional transfer theorem cannot substitute for establishing this missing machine property.

Appendix B.1 retains observational term-context congruence and the concrete execution obligations. The common-allowance reduction and explicit context lift are now proved; the copying UAM, the template proofs and the final proposition remain unfinished.

Lean: `ObservationPaddingExamples.zeroDepthUnit`

Lean: `ObservationPaddingExamples.depthSensitive_unit_terminates`

Lean: `ObservationPaddingExamples.depthSensitive_padded_unit_not_terminates`

Lean: `ObservationPaddingExamples.depthSensitive_not_respectsPadding`

68 Observational context congruence at equal input depths

Unrestricted addressed filling now yields an observational congruence theorem for every original annotated term context, provided the two input terms have equal minimal translation depths. The conclusion is the existing all-allowances observational equivalence of the original filled terms. It is not merely equivalence of two graph fillings. The execution must respect recursive graph isomorphism; no execution padding-invariance hypothesis is needed for this equal-depth result.

The restriction is material; Section 69 removes it. Lean checks that the unchanged private-cell program of Proposition 12.2 has minimal translation depth three, whereas the constant-result program has depth two. This section therefore does not settle congruence for that pair, and does not prove their equivalence. The precise remaining structural obligation is described below.

68.1 Congruence through padded open graph contexts

An open graph context has local allowance r and host allowance D. Padding its graph and selected address by m steps yields allowances $r + m$ and $D + m$. Sole-hole and binding-free preservation supply the corresponding certificates, so the result remains an admissible graph context. The iterated filling law gives a recursive isomorphism between padding an original filling and filling this padded context with the padded argument. Neither the outer interface nor the selected local interface changes.

Lean: `PaddedContextCongruence.pad`

Lean: `PaddedContextCongruence.pad_fill_iso`

For any open program, padding first by n and then by m equals padding it directly to the final allowance $(d + n) + m$. Rewrite both incremental paddings as padding to a specified allowance, apply the exact graph-level transitivity law, and use equality of the underlying graphs to identify the packaged programs. This law includes all admissibility certificates by proof irrelevance.

Lean: `PaddedContextCongruence.pad_padTo`

Suppose two fragments are observationally equivalent at every padded local allowance. Their fillings in any fixed certified graph context are then equivalent at every padded host allowance. Fix an increment m. The padded filling isomorphisms give observational equivalences between each padded original filling and its filling in the padded context, using the execution's isomorphism law. The hypothesis supplies equivalence of the padded input fragments. Apply graph-context congruence in the padded context, then compose with the two surrounding equivalences, reversing the second one.

This argument does not claim that padding preserves execution of an individual program. Instead, it uses an input comparison already supplied at the required padded allowance. Thus the execution's isomorphism obligation suffices here.

Lean: `PaddedContextCongruence.equiv_context_padded`

A scoped isomorphism also yields recursive graph isomorphism after padding both graphs to any common adequate allowance. Unpack the two graph depths and the scoped depth equality; eliminate that equality. The two graphs now have the same index, so the previously proved common-allowance isomorphism congruence applies. This transports structural filling

correspondence to the allowance at which an observation is requested.

Lean: `PaddedContextCongruence.padTo`

68.2 From actual filling to term-context congruence

Padding a graph to its existing allowance is the identity. This is the zero-increment instance of the exact padding-to-addition law. Consequently, interpreting a term context with the translated-program wrapper at the term's own minimal depth gives exactly the original translation of the filled term. Eliminate the stated equality of the chosen depth and the term's depth, remove the identity padding, and apply the original constructor-interpretation law. No term or distributor is simplified.

Lean: `TermContextCongruence.padTo_self`

Lean: `TermContextCongruence.apply_program_at_depth`

Let t and u have the same minimal translation depth d, and suppose t and u are observationally equivalent in the existing all-allowances sense. For every original context C, the theorem proves

$$\mathsf{ObservationalEquiv}(C[t], C[u]),$$

provided the execution respects recursive isomorphism. All finite variable and atom environments are retained; the hole may use variables bound by surrounding lambdas.

One shared addressed context. Apply unrestricted graph filling to C at allowance d. It supplies one certified graph context D, with host allowance q and local allowance $d + n$, whose filling law works for every argument graph at allowance d. Take G and H to be the original translated-program wrappers for t and u at d. Their equality of minimal depths permits both exact constructor-interpretation identities. The universal filling law therefore relates $D[G \text{ padded by } n]$ to the original translation of $C[t]$, and similarly for u. Each relation includes equality of the filled term's depth with q.

Input comparison at every needed allowance. For an additional increment m, the two input programs are G and H padded first by n and then by m. The exact program-padding law and the translated-program coherence law identify these with the original translations of t and u at allowance $(d + n) + m$. The assumed term equivalence supplies precisely this comparison. No preservation of individual executions through padding is used.

Every output observation. Fix any adequate output allowance N for the two filled terms. The scoped filling laws imply $q \leq N$, so write $N = q + m$. The padded graph-context congruence theorem gives equivalence of the two filled graphs padded by m. Pad each structural filling isomorphism to N. The resulting isomorphisms identify those two padded fillings with the original translated-program wrappers for $C[t]$ and $C[u]$ at N. Execution isomorphism invariance turns these into observational equivalences. Compose the inverse left equivalence, the contextual comparison, and the right equivalence. Since N was arbitrary, this proves the full existing all-allowances conclusion.

The input equivalence and the execution isomorphism law are explicit premises of this inference theorem. They are not asserted for the proposition's programs or its still-unconstructed copying execution.

Lean: `TermContextCongruence.observationalEquiv_context_of_depth_eq`

68.3 Exact context depths and the remaining unequal-depth case

Define $\delta_C(d)$ to be the context's outer allowance when its hole has allowance d. A hole returns d, a lambda adds one, and a unary operation retains the subcontext allowance. Application and binary contexts take the maximum of the selected subcontext allowance and the fixed sibling's minimal depth, in the original left/right order.

Induction on the context proves that this numerical function equals the actual graph interpreter's depth: each constructor selects exactly the stated successor, unchanged value or maximum. Applying that equality to a translated term and then the exact constructor-interpretation identity gives $\mathsf{depth}(C[t]) = \delta_C(\mathsf{depth}(t))$. These are statements about the unchanged interpreter, not a new translation.

Lean: `ContextDepth.depthAt`

Lean: `ContextDepth.depth_apply`

Lean: `ContextDepth.depth_fill`

Three inequalities follow by context induction and elementary natural-number arithmetic:

$$d \le \delta_C(d), \qquad d \le e \Longrightarrow \delta_C(d) \le \delta_C(e), \qquad \delta_C(d+n) \le \delta_C(d) + n.$$

The hole cases are immediate. Lambda adds the same one to the inductive comparison, and unary retains it. For a maximum-depth join the fixed sibling is unchanged, so it cannot reverse monotonicity or increase the difference beyond n. In particular, a fixed deep sibling may absorb an increase in the hole allowance. These facts control the indices of the remaining graph-level comparison, but do not themselves prove graph isomorphism.

Lean: `ContextDepth.le_depthAt`

Lean: `ContextDepth.depthAt_mono`

Lean: `ContextDepth.depthAt_add_le`

For every ambient variable and atom scope, the private-cell term has minimal depth three and the constant-result term has minimal depth two. Both equalities hold by reducing the original syntax, its let/sequencing sugar and the original translation depth function. Rewriting with these equalities proves that the two depths differ. Thus the equal-depth theorem above cannot be applied directly to this pair.

To remove the restriction, prove that interpreting C with a graph A at d, or with A padded to $e \ge d$, gives recursively isomorphic graphs after both results are padded to a common allowance. This must retain the original boxes, ordered argument exchange and distributors. The numeric bounds establish adequate indices. Section 69 proves the graph comparison and removes the restriction using unrestricted filling and the cofinal criterion. The concrete execution laws, copying UAM and template proofs remain in Appendix B.1.

Lean: `ContextDepth.privateCell_depth`

Lean: `ContextDepth.constantResult_depth`

Lean: `ContextDepth.proposition12_2_depths_differ`

69 Unrestricted observational context congruence

The equal-input-depth restriction of the preceding section is now removed. Every original annotated term context preserves the existing observational equivalence, for arbitrary input terms and finite variable/atom environments, provided execution respects recursive graph isomorphism and one-step padding. These are explicit obligations for the concrete copying UAM, not project axioms or consequences of determinism. The proposition's terms, the Figure 16 translation and the observation class are unchanged.

The key new structural result compares interpreting a context with an argument at its original allowance and interpreting it with the padded argument. The two resulting graphs are proved recursively isomorphic after padding to a sufficient common allowance. Combining this with the universal addressed filling law and the cofinal observation criterion yields unrestricted semantic congruence. The final equivalence of the private-cell and constant-result programs is still pending: congruence preserves an established equivalence; it does not supply that equivalence or the template proofs.

69.1 One-step padding through the literal graph constructors

The uniform relabelling presentation of padding commutes with literal serial assembly up to ordinary graph isomorphism. Keep each serial pattern vertex, component tag, component vertex and component edge. The inverse maps retain the same data, so both inverse identities follow by splitting on the left or right tag. Internal labels are unchanged, and each edge has the same raised component label. For sources and targets, map composition reduces the claim to an individual component port. Internal, input and output ports have the same attachment to the original two-hole serial pattern on both sides. Thus entire ordered incidence lists are preserved.

The proof splits depth zero from positive depth to respect the existing alphabet representation. Replacing the uniform relabelling presentation by its proved equality with the original raising operation gives recursive isomorphism between raising a serial composite and composing its raised components.

Lean: `PaddingAssembly.serial_graph`

Lean: `PaddingAssembly.serial`

Uniform relabelling also commutes with ordered parallel assembly, in this case as graph equality. The carriers, enumerations and labels coincide; equality of source and target functions follows by inspecting each internal, input or output port in the fixed boundary embedding. Replacing uniform relabelling by the original raising operation gives the corresponding exact parallel padding equality. Binary tensor composition is its concatenating boundary instance. The Boolean family of raised components equals the family of the two raised graphs by its two defining cases.

Lean: `PaddingAssembly.parallel_graph`

Lean: `PaddingAssembly.parallel`

Lean: `PaddingAssembly.tensor`

Padding commutes with outer-interface equality casts by eliminating those equalities. A primitive corolla padded once equals the same primitive corolla at the next allowance, by splitting on the zero or positive alphabet case. Distributor forests satisfy the analogous equality: their unchanged graph is relabelled by the primitive-label embedding at the selected allowance.

The environment distributor is the ordered tensor of its term and atom forests. Tensor padding and both forest equalities give its padding law. The single-use distributor adds only the existing interface cast; commute padding with that cast and apply the environment-distributor law. No contraction, weakening edge or single-use distributor is discarded.

Lean: `PaddingAssembly.castInterfaces`

Lean: `PaddingAssembly.primitive`

Lean: `PaddingAssembly.distributor`

Lean: `PaddingAssembly.environmentDistributor`

Lean: `PaddingAssembly.singleUse`

Raising a box preserves its bound-variable, free-variable and atom metadata, and raises its body once. This is the original recursive definition of raising. For a lambda box, commute raising with the existing body-interface cast as well. The variable bound by the lambda remains in the original first slot.

A translated pair uses the prescribed variable/atom middle-block exchange. Commute padding with its outer cast and with parallel assembly, then identify the two Boolean component families. The exchange itself is unchanged, so the resulting equality retains the exact Figure 16 pair wiring.

Lean: `PaddingAssembly.box`

Lean: `PaddingAssembly.lambdaBox`

Lean: `PaddingAssembly.pair`

For a unary clause, compose the serial padding isomorphism for the complete clause with serial congruence applied to the inner operation-prefix padding isomorphism. The primitive and single-use equalities identify the remaining components with the original clause at the next allowance.

For abstraction, use the same two serial steps and then the primitive, lambda-box and single-use equalities. Its host is raised at allowance $d + 1$ while its body is raised at d, preserving their one-level offset.

Application and each binary operation use two serial steps around their ordered pair. Apply the primitive, pair and environment-distributor equalities to obtain their respective recursive isomorphisms. Each binary operation parameter is arbitrary, retaining the full operation alphabet.

Lean: `PaddingAssembly.unary`

Lean: `PaddingAssembly.abstraction`

Lean: `PaddingAssembly.application`

Lean: `PaddingAssembly.binary`

69.2 Padding to any specified adequate allowance

Padding a graph of allowance d to $e + 1$, where $d \leq e$, equals raising its padding to e once. Write $e = d + n$ and use the exact padding-to-addition identity twice. The final equality is the

defining successor clause of iterated padding.

Lean: PaddingConstructors.padTo_succ

Each one-step constructor law extends to any adequate allowance by induction on the allowance bound. At equality of the original and requested allowance, identity padding gives reflexive isomorphism. At a successor, rewrite each padding with the preceding successor identity, raise the induction isomorphism, and compose with the corresponding one-step constructor law. This proves the unary, application and binary laws separately.

For abstraction, a body padded from d to e is boxed at $e+1$; its original abstraction is padded from $d+1$ to $e+1$. The same induction maintains this shift. Existence of each isomorphism is proved before the Lean definition selects it; no unproved structural assumption is introduced.

Lean: PaddingConstructors.unary_to

Lean: PaddingConstructors.abstraction_to

Lean: PaddingConstructors.application_to

Lean: PaddingConstructors.binary_to

69.3 Structural comparison at a sufficient common allowance

For two scoped graphs G, H, define $\mathsf{AgreeAt}(G, H, N)$ to consist of proofs that both depths are at most N, together with a recursive isomorphism between their original graphs padded to N. This is an auxiliary structural predicate, not a quotient of graphs or a replacement observation relation.

Agreement at N widens to any $M \geq N$: pad the isomorphism to M and collapse both intermediate paddings by exact padding transitivity. Agreement is reflexive at every adequate allowance, symmetric by reversing the isomorphism, and transitive at the same allowance by composing the two isomorphisms. Different proofs of the middle graph's adequate bound carry no additional graph data.

Lean: CommonAllowanceIso.AgreeAt

Lean: CommonAllowanceIso.widen

Lean: CommonAllowanceIso.refl

Lean: CommonAllowanceIso.symm

Lean: CommonAllowanceIso.trans

Agreement is preserved by unary interpretation at the same allowance. Commute padding with the left unary graph, apply unary congruence to the given padded argument isomorphism, and compose with the inverse right unary padding law. For abstraction, use allowance $N+1$ and the corresponding body-to-host padding law; both adequacy bounds increase by one. This preserves the full metadata and body interface of each box.

Lean: CommonAllowanceIso.unary

Lean: CommonAllowanceIso.abstraction

The scoped application interpreter first pads its two arguments to their maximum depth. Padding its result further to an adequate N agrees up to recursive isomorphism with applying

the literal constructor directly to both arguments padded to N. Use the application padding law, then collapse each argument's two successive paddings to N. The binary interpreter has the same proof with its selected binary operation. Thus the original intermediate maximum is retained in the construction and eliminated only by a proved equality of padded graphs.

Lean: CommonAllowanceIso.application_to

Lean: CommonAllowanceIso.binary_to

Suppose the two first arguments agree at N and the two second arguments also agree at N. Both scoped applications have maximum depth at most N. Use the preceding interpreter-padding isomorphism on the left, application congruence of the two argument isomorphisms, and the inverse interpreter law on the right. This proves agreement of applications at N. The analogous three-step composite proves agreement for every binary operation.

Lean: CommonAllowanceIso.application

Lean: CommonAllowanceIso.binary

Define common-allowance isomorphism by the existence of some N at which the two scoped graphs agree. Reflexivity uses the graph's own depth; symmetry retains the witness and reverses its agreement. For transitivity, widen the two witnessing agreements to the maximum of their witnesses, then compose there. A scoped isomorphism, which already includes equality of the depths, supplies common-allowance isomorphism by padding it to the maximum of those depths. All witnesses remain actual recursively isomorphic padded graphs.

Lean: CommonAllowanceIso.CommonIso

Lean: CommonAllowanceIso.refl

Lean: CommonAllowanceIso.symm

Lean: CommonAllowanceIso.trans

Lean: CommonAllowanceIso.ofScoped

Unary and abstraction interpretation preserve common-allowance isomorphism by using the respective fixed-allowance laws. For application and binary interpretation, first widen both argument comparisons to the maximum of their witnesses, then apply the fixed-allowance constructor theorem.

Induction on the complete original annotated context now proves that its constructor interpretation preserves common-allowance isomorphism. The hole uses the supplied relation; lambda and unary use their preservation laws. At either application or binary position, the fixed sibling is related to itself and the selected subcontext uses the induction hypothesis. Every original context constructor is covered.

Finally, a scoped graph agrees with its own padding to any adequate allowance: choose that allowance as witness, remove identity padding on the already padded side, and use reflexive isomorphism. Combined with context preservation, this is the missing unequal-depth structural comparison.

Lean: CommonAllowanceIso.unary

Lean: CommonAllowanceIso.abstraction

Lean: CommonAllowanceIso.application

Lean: CommonAllowanceIso.binary

Lean: CommonAllowanceIso.apply

Lean: CommonAllowanceIso.padTo

69.4 The unrestricted semantic congruence theorem

Let execution respect recursive isomorphism and one-step padding, and assume t and u are equivalent in the existing all-allowances observational sense. For every original annotated context C, the theorem proves

$$\mathsf{ObservationalEquiv}(C[t], C[u]).$$

There is no premise equating the minimal depths of t and u.

A common input allowance and one actual hole. Choose d to be the maximum of the two minimal term depths. Let G, H be the unchanged translated-program wrappers at d. Unrestricted filling supplies one certified graph context D, of some host allowance q, with local allowance $d + n$, whose actual filling with any graph padded by n agrees with its constructor interpretation. The same address works for both G and H.

Correspondence with the original filled terms. Each original term translation is common-allowance isomorphic to its padding to d. Context preservation transports these comparisons through C. The exact constructor-interpretation identity identifies their unpadded endpoints with the original translations of $C[t]$ and $C[u]$. Compose the inverse comparisons with the addressed filling laws. This gives finite witness allowances N_t, N_u comparing each actual filling of D with its original filled-term translation. The two witness allowances may differ.

The semantic comparison at arbitrarily large outputs. Given any requested lower threshold b, choose $N = \max(b, \max(N_t, N_u))$. Widen both structural comparisons to N. Their adequacy proofs ensure $q \le N$; write $N = q + m$. The input programs of D padded by this extra m are the original translations of t and u at allowance $(d + n) + m$, by exact program-padding coherence. The assumed observational equivalence supplies their comparison. Padded graph-context congruence yields equivalence of the two actual fillings at host allowance N.

The two widened graph isomorphisms relate those fillings to the original translated-program wrappers for $C[t]$ and $C[u]$ at N. Execution isomorphism invariance turns them into observational equivalences; compose the inverse left one, the contextual comparison, and the right one. This establishes the cofinal comparison for every b. The previously proved cofinal criterion, using the explicit execution padding law as well, yields the required all-allowances conclusion.

No template robustness or determinism premise is used. The theorem is an inference from input equivalence under stated representation laws. It does not assert those laws for a copying execution or assert the input equivalence of Proposition 12.2.

Lean: GeneralContextCongruence.observationalEquiv_context

The structural comparison can also be stated directly: interpreting any original context with a graph at its original allowance, or with that graph padded to a larger adequate allowance, gives common-allowance isomorphic results. Apply context preservation to the proved comparison

of a graph with its own padding. This theorem has no execution premise and changes no term, constructor, boundary permutation or observation context.

Lean: `GeneralContextCongruence.apply_padTo`

69.5 Original-program regression and completion boundary

For every original annotated context and every finite variable/atom scope, compare its direct filled-term translation with its constructor interpretation when the proposition's argument is supplied at allowance three. The private-cell program has depth three; the constant-result program has depth two and is padded once. Apply the preceding structural theorem to each unchanged term translation and rewrite its unpadded interpretation with the exact filled-term identity. This proves both correspondences independently. It does not compare the private-cell program with the constant-result program, so it neither assumes nor disguises the desired final equivalence.

The general context-filling and observational-congruence development of former Appendix B.1 item 3 is now complete. Its concrete execution hypotheses are consolidated into the copying-machine task, where they remain unproved. Appendix B.1 therefore has four outstanding work packages: remaining translation and invariant laws; the full copying machine and its representation laws; the eleven template families; and semantic weakening, Figure 22 and the final proposition. This bookkeeping does not declare the application complete.

Lean: `GeneralCongruenceExamples.proposition12_2_context_representations`

70 Padding stability and copyable payloads

The copying machine must retain the side conditions of Definitions 10.2 and 10.3 when a finite representation is embedded at a larger allowance. We prove this structural prerequisite for the unchanged graph definitions. Stability is preserved and reflected; copyability is preserved. These results neither define machine transitions nor prove execution padding invariance. They make no change to the terms or translation of Proposition 12.2.

70.1 Stability is independent of unused allowance

A label is stable exactly when it denotes an instance or a passive operation. The padding label map leaves the primitive unchanged. At allowance zero, the proof removes the primitive wrapper; at positive allowance it preserves the primitive branch of the sum. A box remains a box and cannot satisfy the stable-label predicate on either side. Thus the stable-label test is equivalent before and after one padding.

Lean: `Stable.padding_label`

For a graph with one term input, the existing theorem `Stable.of_labels` reconstructs the entire stability predicate from its label condition for this specific example alphabet. Its proof uses the graph's incidence coverage and typing laws to derive both the permitted outputs and input reachability. Consequently these conditions are not dropped in the new proof. Express padding as the already proved uniform relabelling of the original graph; the outer edge carrier is unchanged. Transport each edge's stable-label condition in either direction and apply `Stable.of_labels`. This proves

$$\mathrm{Stable}(\mathrm{raise}_d\, G) \quad \Longleftrightarrow \quad \mathrm{Stable}(G).$$

Lean: `Stable.raise_iff`

Induction on the number of padding steps gives the same equivalence for iterated padding: the zero case is identity and the successor case composes one-step equivalence with the induction hypothesis. Transport along an equality of depth indices preserves stability by equality elimination. Combining this transport with iterated padding proves equivalence at every specified adequate allowance. In particular, unused allowance cannot make an unstable graph stable.

Lean: `Stable.pad_iff`

Lean: `Stable.castDepth_iff`

Lean: `Stable.padTo_iff`

70.2 Copyable shapes survive padding

The lambda payload is the original serial composition of a lambda operation and its box, before the surrounding environment distributor. Padding this serial composition is recursively isomorphic to composing the padded components. The primitive-padding equality identifies the lambda operation, and the lambda-box equality identifies the original box with its body padded once. This gives a recursive isomorphism to the lambda payload of the padded body, with the same bound, free-variable and atom interfaces.

Lean: `Copyable.raise_lambdaPayload`

Induct on the derivation of copyability. For instances, constants, application, unary operations

and binary operations, the primitive-padding identity gives precisely the corresponding copyable constructor at the next allowance. For a lambda payload, the padded body is arbitrary: the definition of copyability places no stability condition on that body. Its payload is copyable by the lambda constructor, and the inverse of the preceding isomorphism transports copyability to the padded original payload. Finally, if copyability was obtained by recursive renaming, pad that isomorphism and transport the induction hypothesis along it. This proves preservation for every constructor of the full example operation alphabet.

Lean: `Copyable.raise`

Induction on the padding count gives iterated preservation. Equality elimination handles a change of depth index, and composing these two facts proves preservation at an arbitrary adequate allowance. No converse for copyability is claimed here; no assumption about the behaviour of an execution or about deterministic matching is used.

Lean: `Copyable.pad`

Lean: `Copyable.castDepth`

Lean: `Copyable.padTo`

70.3 Remaining operational boundary

These theorems discharge the forward padding obligations for both side conditions, and the reflection obligation for stability. They belong to the structural prerequisites in Appendix B.1. Preservation under actual machine steps, copying-rule matching, concrete execution isomorphism and padding laws, template robustness and the final equivalence remain to be proved. The four outstanding work packages are therefore retained.

71 Actual composition of translated contexts

The constructor interpretation already respects composition of annotated term contexts. We now connect this identity to actual replacement at certified graph holes. Two independently translated contexts, with the inner host padded to fit its outer occurrence, compose to a graph recursively isomorphic to the direct translation of the composed context. Filling its selected hole with any suitably padded graph also gives the direct constructor interpretation. The isomorphisms have equal outer depths; no additional common-allowance comparison or execution hypothesis is used.

71.1 Certified open graph contexts compose

Let A have a sole hole accepting the outer interface of B, and let B have a sole hole accepting the argument graph. Form $A \circ B$ by replacing the hole of A with the graph of B. Its selected occurrence is the address of the original hole of B, followed through that replacement. The existing sole-hole composition theorem proves that it is still the only hole. Both contexts have binding-free certificates. Since the hole input is the singleton term interface, it is sourced; the proved binding-free plugging closure therefore certifies the composite. This use respects the zero-input qualification of Section 59.

Lean: `GraphContext.compose`

For every graph G at the local allowance, addressed plugging associativity gives a recursive isomorphism

$$(A \circ B)[G] \;\cong\; A[B[G]].$$

It retains all nested boxes and ordered interfaces, allowing the carrier reassociation already handled by the generic graph proof. Specialising G to the graph of an open program yields the corresponding equality up to recursive isomorphism between composed filling and successive filling. The open-program certificates do not add operational premises.

Lean: `GraphContext.compose_plug_iso`

Lean: `GraphContext.compose_fill_iso`

The exact depth function also composes: $(C \circ D).\,\mathrm{depthAt}(d) = C.\,\mathrm{depthAt}(D.\,\mathrm{depthAt}(d))$. Induction on the outer context proves this identity: a lambda applies a successor, a unary operation retains the depth, and each binary or application branch takes the original maximum with its fixed sibling's depth.

Lean: `TermContext.depthAt_compose`

71.2 Composing the unchanged translated hosts

Choose annotated contexts D and C, an argument allowance d, and arbitrary inner and outer hole names. Translate D at d. The universal filling theorem provides a certified graph context B, an increment n, and a filling law for every graph of allowance d. Write q for its outer allowance. Translate C at q; the same theorem supplies a certified context A and increment m. The outer hole of A accepts allowance $q + m$, so the actual composite is $A \circ \mathrm{pad}_m(B)$. Its remaining hole accepts allowance $(d + n) + m$. Both A and B have literally the original translated host graphs; the construction changes neither their translation clauses nor their interfaces.

There are two simultaneous conclusions. First, apply the outer filling law to the graph of B itself. Constructor interpretation of C on the translated host of D is exactly translation of $C \circ D$, by the previously proved `graph_compose`. Hence the composite host is recursively isomorphic to the directly translated composed host, at the same allowance.

Second, fix any argument graph G. Associativity rewrites filling the composite as filling A with the filled padded B. The padding/filling isomorphism identifies that inner result with padding B filled by $\mathrm{pad}_n(G)$. The outer filling law now applies. Transport the inner filling law through the original constructor interpretation of C, using its isomorphism preservation theorem. Finally, the literal `apply_compose` identity identifies the endpoint with interpretation of $C \circ D$ on G.

The same increments, hosts and addresses work for every G: they are chosen before the universal quantifier. In particular, the proof is not restricted to language graphs or closed programs. No inequality between the two hole names is required. The sole-hole certificates account for removal of the outer hole before insertion of the inner context.

Lean: `ContextTranslation.composed_context_filling`

71.3 Successive filling of terms

Supply the preceding theorem with the original graph translation of a term t at its minimal allowance. The existing exact interpretation identity `apply_ofTerm` and syntactic identity `fill_compose` identify the final graph with the original translation of $C[D[t]]$, up to recursive isomorphism at the same outer depth. This corollary covers arbitrary finite variable and atom scopes, including holes beneath binders, and preserves all original maximum-depth padding choices.

Lean: `ContextTranslation.composed_term_filling`

This closes the pending context-translation composition law in Appendix B.1. It does not supply graph-level term renaming or substitution, machine-step invariants, the concrete copying execution or the eleven template proofs. Proposition 12.2 remains the endpoint of those outstanding tasks.

72 Resource permutation of literal distributors

Graph-level environment renaming must respect the actual distributors of Figure 16. We prove their resource-permutation law, without deleting their weakening or contraction edges. This is a prerequisite for bijective renaming of entire translated terms. It is not the whole-term renaming theorem, and it does not handle weakening or identification of distinct resources.

72.1 Permuting resources while retaining use order

Let π be a bijection of the k resource positions. For any component carrier X, define a bijection on tagged elements by $(i, x) \mapsto (\pi(i), x)$, with inverse using π^{-1}. The inverse identities follow directly from those of π; the component vertex or edge is unchanged.

Lean: `Distributor.resourceTags`

Recall that $D_{k,m}$ places leaf j of resource i at input position $i + kj$ and its output at position i. Define the permuted boundary by precomposing the original input and output bijections with resource retagging. The resulting input position is $\pi(i) + kj$ and the output position is $\pi(i)$. Thus each use retains its position within the use-major ordering, and the same permutation is applied at every use. The original boundary's typing proofs supply the new boundary's typing proofs. No resource is removed, identified or duplicated by this operation.

Lean: `Distributor.permutedBoundary`

Lean: `Distributor.permuted_input_position`

Lean: `Distributor.permuted_output_position`

Assemble the original family of canonical contraction trees with this boundary. Every component retains its internal vertices, labels and ordered incidences. The original finite enumeration of resource positions is used. This defines the boundary-permuted forest, including $k = 0$ and $m = 0$ without separate non-emptiness assumptions.

Lean: `Distributor.permutedNet`

72.2 An isomorphism of the actual finite graphs

Retagging an internal vertex after embedding it into the permuted forest has the same result as embedding the unchanged component vertex at resource $\pi(i)$ in the original forest. For internal vertices this is the definition of retagging. For inputs and outputs it is the defining boundary permutation. Importantly, the statement fixes the resulting external port itself; it is stronger than equality of its wire type or equality of boundary sizes.

Lean: `Distributor.permuted_embed`

Use resource retagging for both the internal-vertex and edge bijections. Labels agree because all resource components are copies of the same canonical tree. For each source list, compose its two maps and apply the preceding embedding identity to each entry, preserving list order. The same argument applies to the target list. This constructs an ordinary graph isomorphism from the permuted forest to the original $D_{k,m}$. The zero-use case still contains every canonical weakening edge; the single-use case is not simplified to a wire.

Lean: `Distributor.permuted_iso`

Relabel the permuted forest into the copying alphabet at any allowance d, using the same primitive embedding as the original distributor. Its carriers and incidence lists are unchanged.

The preceding bijections therefore give a recursive isomorphism: corresponding embedded primitive labels satisfy reflexivity of the recursive label relation, and the same source and target proofs apply. The result holds at every finite box allowance.

Lean: `Distributor.permutedAtDepth`

Lean: `Distributor.permuted_deepIso`

72.3 Separate variable and atom environments

Apply independent bijections ρ and α to the term and atom forests. Their ordered tensor is the permuted environment distributor. Term resources remain before atom resources, and no permutation mixes the two wire sorts. Tensor congruence assembles the two proved recursive isomorphisms into one for the original environment distributor.

Lean: `Translation.permutedEnvironmentDistributor`

Lean: `Translation.environmentDistributor_permutation`

Finally, take one use and transport along the same interface equalities used in the original single-use constructor. Isomorphism preservation under that transport gives the single-use permutation law. All original distributor edges remain in both graphs.

Lean: `Translation.singleUse_permutation`

The remaining translation work must propagate boundary renaming through selected-resource clauses, ordered pairing, binders and their boxes, and all term constructors. Non-bijective renamings and substitution require additional sharing arguments. No observational or template theorem follows merely from this distributor isomorphism; Proposition 12.2 remains unfinished.

73 Typed boundary isomorphisms for renaming

The existing recursive graph isomorphism fixes each external boundary position. Environment renaming may move those positions. We introduce a separate typed boundary-isomorphism interface and prove a direct resource-permuting automorphism of the original distributor. Neither the existing isomorphism nor the observation relation is replaced. These are structural prerequisites for whole-term renaming; the selected-resource clauses and propagation through term constructors remain to be proved.

73.1 The interface and its elementary laws

A boundary isomorphism between two copying hypernets at the same allowance consists of bijections on all vertices and on edges. The vertex bijection preserves wire labels and preserves and reflects the predicates of being an input and being an output. Hence it also preserves internal vertices. It may permute external positions, but cannot exchange input and output roles or mix term and atom wire types. Corresponding edge labels satisfy the existing recursive label relation. Mapping an entire source or target list gives the corresponding list in the target graph, in the same order. The definition permits different interface lists when these conditions can be met.

Lean: `BoundaryIso`

Identity uses the identity bijections, reflexivity of label equivalence and identity list maps. To reverse an isomorphism, use the two inverse bijections. Wire labels and boundary roles are transported by their right-inverse laws. Reverse the recursive edge-label relation. For an incidence list, apply the inverse vertex map to its proved forward equality; cancellation of the two vertex maps and the edge inverse identity give the required reversed equality.

Lean: `BoundaryIso.refl`

Lean: `BoundaryIso.symm`

Composition uses composition of the two vertex bijections and the two edge bijections. Wire-label and boundary-role preservation compose directly; recursive label equivalence is transitive. The list-map composition identity reduces each incidence condition to the two component incidence conditions.

Lean: `BoundaryIso.trans`

Directed adjacency is preserved and reflected. A common vertex in a source and target list maps to a common vertex in the image lists. Conversely, recover preimages from the two mapped-list memberships; injectivity of the vertex bijection identifies those preimages and supplies a common original vertex. The statement therefore concerns actual incidence adjacency, not just a relation between labels.

Lean: `BoundaryIso.adjacent_iff`

Every existing boundary-fixing recursive isomorphism embeds into the new interface. Extend its internal-vertex bijection by the identity on input and output constructors; reuse its edge bijection, label relation and incidence proofs. For boundary ports, wire-label and role preservation are reflexive; for internal ports use the original internal-label equation. This is an embedding of the stronger boundary-fixing data, not a converse identification.

Lean: `BoundaryIso.ofDeepIso`

73.2 An automorphism of the original distributor

The original ordered parallel assembly provides a bijection between its vertices and pairs consisting of a resource index and a component vertex. Invert this bijection, apply the resource-tag permutation from Section 72, and apply the assembly bijection again. This defines a bijection on all vertices of the original distributor itself.

Lean: `Distributor.resourcePorts`

On a vertex embedded from component i, the inverse assembly cancels the embedding. The result is exactly the same component vertex embedded at $\pi(i)$. This cancellation identity proves the concrete boundary action: the input for resource i at use j is sent to the input for $\pi(i)$ at the same use j, and output i is sent to output $\pi(i)$. Thus the input positions transform as $i + kj \mapsto \pi(i) + kj$, matching the preceding permuted-boundary construction.

Lean: `Distributor.resourcePorts_embed`

Lean: `Distributor.resourcePorts_input`

Lean: `Distributor.resourcePorts_output`

Use this all-vertex bijection and resource retagging on edges to construct a boundary isomorphism from the original distributor to itself, at every allowance. To prove wire-label preservation, express an arbitrary vertex through the inverse assembly bijection and use the typing equation of each component embedding. Input and output roles are preserved by a case split on the component vertex. Edge labels are unchanged. For source and target lists, compose the list maps and apply the embedding cancellation identity entry by entry. All canonical weakening and contraction edges remain, with their original ordered incidences; no positivity assumption on the resource or use count is needed.

Lean: `Distributor.boundaryIso`

This interface makes the external action explicit on the original graph, whereas Section 72 compared a separately boundary-permuted assembly with the original assembly using boundary-fixing isomorphism. Neither result proves observational equivalence under arbitrary boundary permutations. The original terms and translation of Proposition 12.2 remain unchanged, and whole-term renaming, substitution and the copying machine remain outstanding.

74 Renaming the literal atomic translations

This section closes the selected-resource and all three atomic cases of bijective environment renaming for the original Figure 16 translation. The construction works for arbitrary finite variable and atom environments, at every adequate allowance for the individual clauses. The proof retains all weakening and contraction edges and the atom clause's instance edge. It neither simplifies a distributor to a wire nor replaces the proposition's terms. Composite-term renaming and general substitution remain separate tasks.

74.1 Coordinates for the selected-resource graph

For a resource $i < n$, the raw assembly is precisely $(D_{i,0} \otimes D_{1,1}) \otimes D_{n-i-1,0}$, the expression inside the existing interface cast. We introduce coordinates for its carriers, not a new translation. There is one weakening for every output resource and one additional contraction belonging to the selected resource.

Lean: `SelectedResourceShape.raw`

The edge-coordinate map sends a weakening in the left forest to its original resource number, the selected weakening to i, and a weakening at offset j in the right forest to $i + 1 + j$. The selected contraction is the extra summand of $\mathrm{Fin}(n) + 1$. The inverse compares a resource number with i: less than i selects the left forest; equality selects the middle weakening; greater than i selects the right forest after subtracting $i+1$. The extra summand selects the contraction. Checking these three cases proves both inverse identities, including the empty left or right forest cases, and packages them as an edge bijection.

Lean: `SelectedResourceShape.edgeIndex`

Lean: `SelectedResourceShape.edgeAt`

Lean: `SelectedResourceShape.edgeIndex_at`

Lean: `SelectedResourceShape.edgeAt_index`

Lean: `SelectedResourceShape.edges`

The internal carrier is a singleton: only the middle one-use tree has an internal vertex. The two weakening forests have empty internal carriers. Case analysis on the tensor and forest tags proves the corresponding bijection with the unit type. The input length is one; the output length is $i + 1 + (n - i - 1) = n$, using $i < n$. These identities give a bijection from all raw vertices to $\mathrm{Port}(1, 1, n)$: preserve each boundary index and use the singleton coordinate internally. The inverse laws follow from the internal bijection and equality of finite indices with the same value.

Lean: `SelectedResourceShape.internal`

Lean: `SelectedResourceShape.input_length`

Lean: `SelectedResourceShape.output_length`

Lean: `SelectedResourceShape.ports`

In these coordinates every weakening has empty source. The contraction's source is the ordered pair consisting first of the internal vertex and then of the sole input. Weakening i targets the internal vertex; every other weakening j targets output j; the contraction targets output i.

The label-coordinate function assigns weakening to the resource summand and contraction to the extra summand.

Lean: SelectedResourceShape.source

Lean: SelectedResourceShape.target

Lean: SelectedResourceShape.label

The source, target and label correspondence lemmas prove these descriptions for the actual raw assembly, by splitting the resource comparison and the two middle edges. The target proof checks the right-forest offset arithmetic and the equality test for the selected resource. All vertices have the selected wire sort: the sole internal case is direct, and boundary cases use membership in the three concatenated replicated interface lists. Thus coordinates describe the entire ordered incidence data, not merely edge counts or an informal shape.

Lean: SelectedResourceShape.source_at

Lean: SelectedResourceShape.target_at

Lean: SelectedResourceShape.label_at

Lean: SelectedResourceShape.vertex_label

74.2 Renaming the original selected resource

For a resource bijection π, permute output coordinates by π and fix the internal vertex and input. On edge coordinates apply π to weakening resources and fix the extra contraction. The source lists commute with these maps directly. Target lists commute because $\pi(j) = \pi(i)$ if and only if $j = i$. This uses injectivity and is the point where the argument is specifically bijective renaming.

Lean: SelectedResourceRenaming.outputPermutation

Lean: SelectedResourceRenaming.source_permutation

Lean: SelectedResourceRenaming.target_permutation

Applying the inverse port-coordinate bijection to the source and target correspondence lemmas reconstructs the actual incidence lists. Compose the source coordinate bijection, the permutation, and the inverse target coordinate bijection, on vertices and edges alike. Wire labels agree because all these vertices have the same wire sort; input and output roles are preserved. The coordinate label lemma gives recursive equality of edge labels. Reconstruct the source and target lists and use their permutation identities to finish the boundary-isomorphism proof. The raw output-action lemma states explicitly that output j is sent to output $\pi(j)$.

Lean: SelectedResourceRenaming.source_eq

Lean: SelectedResourceRenaming.target_eq

Lean: SelectedResourceRenaming.rawIso

Lean: SelectedResourceRenaming.rawIso_output

Interface casts transport boundary isomorphisms by equality elimination. A companion lemma transports a specified output action along the induced finite-index casts. Applying these to the raw isomorphism gives the theorem for the existing public selected-resource constructor,

and proves that its output action is exactly $j \mapsto \pi(j)$. This checks the boundary action of the actual Figure 16 clause after its original casts.

Lean: SelectedResourceRenaming.castInterfaces

Lean: SelectedResourceRenaming.castInterfaces_output

Lean: SelectedResourceRenaming.selectedResource_rename

Lean: SelectedResourceRenaming.selectedResource_rename_output

74.3 Transport through ordered parallel assembly

For a family of component boundary isomorphisms, map tagged vertices and edges componentwise, retaining their component tags. Each inverse identity reduces to the component inverse identity. Compose the inverse original parallel vertex assembly with this tagged vertex map and the target assembly to get the whole-graph port bijection. The embedding lemma says that this bijection commutes with embedding each component vertex.

Lean: BoundaryParallel.componentPorts

Lean: BoundaryParallel.componentEdges

Lean: BoundaryParallel.ports

Lean: BoundaryParallel.embed

The resulting assembly isomorphism preserves wire labels using the two embedding-label equations and the component isomorphism. The input and output role proofs reduce through the embedding equations to the corresponding component role proofs. Edge labels are related componentwise. For sources and targets, map the component incidence equality through the embedding and use the embedding lemma entry by entry. Specialising to the original Boolean family and concatenated boundary gives ordered tensor congruence.

Lean: BoundaryParallel.iso

Lean: BoundaryParallel.tensor

The variable clause is now immediate from selected-resource renaming on the term side and the zero-use atom-distributor automorphism on the atom side, followed by the original interface casts. For a constant, tensor the zero-use term-distributor automorphism, the identity of the constant operation, and the zero-use atom-distributor automorphism, then use the original casts. Unused resources remain represented by their weakening edges.

Lean: BoundaryParallel.variableTerm_rename

Lean: BoundaryParallel.constant_rename

74.4 Retaining the atom's instance edge

The atom clause additionally has the literal serial prefix of an instance edge. Let H be its remaining graph, with one atom input. The prefixed graph has one new external term input; every old vertex survives, with the old atom input becoming internal. A bijection between its vertices and $1 + \mathrm{Vertex}(H)$ records this fact, checking both depth-zero and positive-depth

representations. Lift a boundary isomorphism of H by fixing the new input and applying its old port bijection.

Lean: BoundaryInstance.graph

Lean: BoundaryInstance.ports

Lean: BoundaryInstance.liftPorts

A boundary isomorphism between graphs with one input fixes that input: input-role preservation implies its image is an input, and the index set has only one element. Embedding an old vertex uses the inverse coordinate map on the right summand. The lifted map commutes with this embedding by cancellation. Embedded vertices retain their wire labels; none is a new external input; and being an output is preserved and reflected. These three facts follow by cases on the old vertex. The actual expansion insertion agrees with this embedding, including the shared internal atom vertex and the original ordered output positions.

Lean: BoundaryInstance.input_fixed

Lean: BoundaryInstance.embed

Lean: BoundaryInstance.lift_embed

Lean: BoundaryInstance.vertex_embed

Lean: BoundaryInstance.input_embed

Lean: BoundaryInstance.output_embed

Lean: BoundaryInstance.insert

Lean: BoundaryInstance.insert_eq_embed

Map the new instance edge to itself and the old edges by the given edge bijection. Its inverse identities hold separately in the two tags. For the prefix isomorphism, the vertex and role laws use the preceding coordinate facts. The new edge's source is fixed; its target is fixed by uniqueness of the old atom input. For every old edge, source and target preservation follow from the old incidence equations, insertion/embedding correspondence, and commutation of the lifted port map with embedding. Tensor this prefix result with the ambient term-weakening forest, and apply the original casts, to obtain renaming for the public atom translation.

Lean: BoundaryInstance.edges

Lean: BoundaryInstance.iso

Lean: BoundaryInstance.atom_rename

74.5 The atomic-case endpoint and remaining work

The predicate `Atomic` selects precisely variables, atoms and constants from the original scoped syntax. The atomic renaming theorem proves existence of a boundary isomorphism between the original translation and the translation of the syntactically renamed term. Its sole depth cast uses the already proved depth-renaming identity. Cases on the atomic predicate reduce to the three public clause results above. There are no operational or template hypotheses.

Lean: AtomicRenaming.Atomic

Lean: `AtomicRenaming.atomic_rename`

This completes the selected-resource and atomic cases of bijective translation renaming. It does not prove the inductive cases for application, unary and binary operations, or lambda boxes; nor does it prove non-bijective renaming, weakening or graph-level substitution. Boundary-permuting isomorphism is still distinct from the boundary-fixing equivalence used for observations. The original terms and translation of Proposition 12.2 are unchanged.

Completion estimate. As of 25 October 2026, the estimated completion of the requested Proposition 12.2 development is about **40% by mathematical effort**, with a plausible range of **30–50%**. This is a judgement about remaining proof difficulty, not a measured percentage of declarations or checklist items. The current milestone closes actual translation cases; it does not erase the large remaining concrete copying-machine and eleven-template obligations.

75 Compatible serial renaming and structural safety

This milestone closes the compatibility argument for renaming the original serial assembly. Suppose $G : I \to J$ and $H : J \to K$ are replaced by boundary isomorphic graphs $G' : I' \to J'$ and $H' : J' \to K'$. The two isomorphisms must send each shared position of J to the same position of J'. Under precisely this condition, the original two-hole expansion admits a boundary isomorphism. All ordered incidences and all recursive edge labels are preserved. No identity wire, contraction, weakening, or box is removed.

The result supplies a composition step needed by Figure 16's composite clauses; it does not by itself finish their renaming induction. We also prove that typed boundary renaming preserves stability, binding-freedom and exact structural paths. These are not machine-execution traces or a proof of Proposition 12.2.

75.1 Extracting the three carrier actions

A boundary isomorphism preserves the predicates "is an input" and "is an output". Consequently an internal port must map to an internal port; either other result would contradict role preservation. Pattern matching on the port image therefore extracts a map on internal vertices, and the equation saying that this map reconstructs the original port image follows by the same cases. The input and output maps are extracted similarly, using the positive role condition to exclude the other two constructors.

Lean: `BoundaryComponents.internalMap`

Lean: `BoundaryComponents.map_internal`

Lean: `BoundaryComponents.inputMap`

Lean: `BoundaryComponents.map_input`

Lean: `BoundaryComponents.outputMap`

Lean: `BoundaryComponents.map_output`

Apply those constructions to the inverse isomorphism to obtain inverse maps. Embed an alleged inverse identity into the corresponding port constructor, rewrite both extracted maps as the original port map, and use the original bijection's inverse identity. Injectivity of each constructor gives the desired identity on the carrier. This constructs bijections on internal vertices, input indices and output indices separately. Finally, specialising the original vertex-label law to each constructor proves type preservation for all three bijections. Thus compatibility below compares actual typed positions, not merely interface cardinalities.

Lean: `BoundaryComponents.internals`

Lean: `BoundaryComponents.inputs`

Lean: `BoundaryComponents.outputs`

Lean: `BoundaryComponents.internal_label`

Lean: `BoundaryComponents.input_label`

Lean: `BoundaryComponents.output_label`

75.2 The exact gluing condition

Let f and g be the two component isomorphisms. Write f_O for the output index bijection of f, and g_I for the input index bijection of g. Their compatibility is

$$\forall j \in \mathrm{Fin}\,|J|, \qquad f_O(j) = g_I(j).$$

It is inherited by inverses: apply the injective input map to the proposed identity of inverse images, then use compatibility and both right-inverse laws. It is inherited by composition because extracting an input or output map commutes with composing port bijections. Equality of the two composite actions then follows by the two given compatibility equations.

Lean: `BoundarySerial.Compatible`

Lean: `BoundarySerial.compatible_symm`

Lean: `BoundarySerial.compatible_trans`

The existing serial graph has internal vertices comprising shared-interface positions, tagged left internal vertices, and tagged right internal vertices. Its boundary consists of the left inputs and right outputs. Map these five classes by f_O, the internal maps of f and g, f_I, and g_O, respectively. The inverse uses the five inverse bijections. Checking each constructor proves both inverse laws. The edges retain their left/right tags and are mapped by the appropriate component edge bijection.

Lean: `BoundarySerial.ports`

Lean: `BoundarySerial.edges`

The two insertion maps are the actual maps used by the existing two-hole expansion. For the left insertion, the new port map commutes on internal, input, and output ports by its definition. For the right insertion, the internal and output cases are again direct; its input case is exactly the compatibility equation, since that input has become a shared internal vertex. Conversely, specialise commutation with the right insertion to each right input. Injectivity of the internal-port and sum constructors recovers compatibility. Thus agreement is necessary as well as sufficient for this explicit map to respect both insertions. The converse does not claim that every possible isomorphism between the assembled graphs must arise from these components.

Lean: `BoundarySerial.leftInsert`

Lean: `BoundarySerial.rightInsert`

Lean: `BoundarySerial.left_commutes`

Lean: `BoundarySerial.right_commutes`

Lean: `BoundarySerial.compatible_iff_right_commutes`

To prove the assembled isomorphism, type preservation is checked on the same five port classes. Boundary roles follow immediately from their constructors. An edge retains its component tag, so its recursive label relation is the component's label relation. Its ordered source list is the component source list mapped by its insertion. Use insertion commutation, the component's source law, and composition of list maps. The target proof is identical with target lists. Finally, evaluation on external inputs and outputs gives their exact actions: f_I on inputs, g_O

on outputs. All these proofs work at every allowance, including zero, without requiring any interface to be inhabited.

Lean: `BoundarySerial.iso`

Lean: `BoundarySerial.iso_input`

Lean: `BoundarySerial.iso_output`

75.3 Paths, nested binding restrictions, and stability

For a boundary isomorphism $f : G \to H$, membership of the image of a port in an image edge's source list is equivalent to original source membership. Rewrite the source law and use injectivity of the port map to reflect membership in the mapped list. Targets are identical. The existing adjacency equivalence then transports ordered edge paths and exact-length adjacency traces. A vertex step consists of a common edge witnessing source and target membership; transport that edge forward, or recover it with the inverse edge bijection. This proves vertex-step equivalence, hence exact vertex-trace and reachability equivalence. The length preserved here counts graph incidences, not UAM transition costs.

Lean: `BoundarySafety.source_mem_iff`

Lean: `BoundarySafety.target_mem_iff`

Lean: `BoundarySafety.path_iff`

Lean: `BoundarySafety.edge_trace_iff`

Lean: `BoundarySafety.vertex_step_iff`

Lean: `BoundarySafety.vertex_trace_iff`

Lean: `BoundarySafety.vertex_reach_iff`

Every recursive primitive-label property is invariant: move each edge back through the edge bijection and apply the existing recursive label-equivalence law. Focus-freedom and hole-freedom are special cases. For shallow binding-freedom, pull forbidden ancestry back using adjacency preservation, preserve binding and hole labels, and transport the explicit source witness by source membership. The proof retains the source requirement; it does not strengthen Lemma E.1 at zero-input holes. For a nested box, the existing recursive label relation provides a boundary-fixing isomorphism of its body, to which the already proved nested binding-path law applies. Combine this with the shallow result and focus-freedom, and apply the inverse isomorphism, to obtain full binding-freedom in both directions.

Lean: `BoundarySafety.allLabels_iff`

Lean: `BoundarySafety.focusFree_iff`

Lean: `BoundarySafety.holeFree_iff`

Lean: `BoundarySafety.shallow_bindingFree`

Lean: `BoundarySafety.bindingPaths`

Lean: `BoundarySafety.bindingFree_iff`

Stability is also invariant when atom or thunk outputs are permuted. Transport the permitted flat labels through the edge bijection and apply the previously established characterisation

of stability by these labels for the example alphabet. That characterisation supplies the actual output and all-vertex reachability conditions; neither condition is dropped from the definition. Inversion proves reflection. This does not assert that arbitrary execution steps preserve stability, nor does it alter the copyability predicate.

Lean: `BoundarySafety.stable_iff`

75.4 Recovering the boundary-fixing interface

If an isomorphism's extracted input and output maps fix every position, its internal bijection and edge bijection form an existing recursive isomorphism. For ordered incidences, compare the all-port map with the extension of the internal map: they agree internally by extraction and on the boundary by the two fixing hypotheses. Applying this to compatible serial assembly needs only the left external inputs and right external outputs to be fixed. Its internal shared-interface action may be nontrivial. The exact external action laws proved above supply the two fixing hypotheses. This is a proved bridge to the original observation infrastructure; the separate execution-invariance assumptions of that infrastructure remain to be discharged by the concrete machine.

Lean: `BoundaryIso.toDeepIso`

Lean: `BoundarySerial.toDeepIso`

75.5 Concrete distributor compositions and mismatch rejection

The input-action transport lemma eliminates the four interface equalities and then uses the original action equation. We use it to expose $D_{k,1}$ at interface $k \to k$, by the equality $k \cdot 1 = k$ only. The graph retains its full forest of weakening and contraction edges. Its isomorphism is the original distributor resource automorphism, transported through that equality.

Lean: `DistributorSerialRenaming.castInterfaces_input`

Lean: `DistributorSerialRenaming.single`

Lean: `DistributorSerialRenaming.singleIso`

For a resource bijection π, the one-use forest sends input i and output i to $\pi(i)$. The input position formula $i + k \cdot 0 = i$ and the previous distributor output law prove these claims. Injectivity of the port constructors then gives the same formula for the extracted input and output index maps.

Lean: `DistributorSerialRenaming.singleIso_input`

Lean: `DistributorSerialRenaming.singleIso_output`

Lean: `DistributorSerialRenaming.outputIndex_eq`

Lean: `DistributorSerialRenaming.singleInputIndex_eq`

Consequently the original $D_{k,m}$ followed by the original $D_{k,1}$ admits the assembled automorphism when both forests use π. This is an unconditional family for every finite k, m and every allowance; matching is proved, not supplied as an application premise. Conversely, if the first forest uses π and the second uses τ, compatibility holds exactly when the two resource functions agree. Substitute the two explicit index formulas and compare their finite-index

values. The reverse direction substitutes equality of the functions. This validates the gluing condition on actual sharing graphs rather than only on an abstract interface.

Lean: `DistributorSerialRenaming.single_compatible`

Lean: `DistributorSerialRenaming.serial_singleIso`

Lean: `DistributorSerialRenaming.single_compatible_iff`

The regression family uses the involution $i \mapsto 1 - i$ on two resources. Finite arithmetic proves its two inverse identities. Apply the distributor composition theorem to two-use atom forests at arbitrary depth. A swap on one side and identity on the other cannot satisfy compatibility: the characterisation would force their functions to agree, and evaluation at zero gives $1 = 0$. Separate instances cover no resources and no uses, so the theorem includes empty forests and pure weakening. Finally, combining the generic serial isomorphism with recursive binding-freedom invariance proves the safety equivalence of the two actual serial assemblies, with the compatibility condition explicit.

Lean: `BoundaryRenamingExamples.swapTwo`

Lean: `BoundaryRenamingExamples.twoAtomUses`

Lean: `BoundaryRenamingExamples.unmatched_swap`

Lean: `BoundaryRenamingExamples.emptyForest`

Lean: `BoundaryRenamingExamples.weakeningForest`

Lean: `BoundaryRenamingExamples.serial_bindingFree_iff`

Remaining obligations and effort estimate. The completed subtask is typed serial gluing, including its exact compatibility criterion, structural safety transport, and concrete distributor instances. Whole-term renaming still requires the composite clauses and lambda boxes; non-bijective renaming and graph substitution remain open. In particular, the current recursive label relation fixes a box body's ordered boundary: a permutation outside a box does not automatically justify permuting its body. The induction must address that interface explicitly while retaining Figure 16. As of 25 October 2026, estimated completion of the requested Proposition 12.2 formalisation is roughly **40–45% by mathematical effort**, with a broad plausible range of **30–50%**. This is a judgement about remaining proof work, not a count of declarations. The concrete copying machine and eleven template families still account for a large part of the remaining effort.

76 Renaming the complete unary fragment

The serial compatibility results now yield a full constructor theorem for the original unary translation, including its final single-use distributors. Every finite chain of negation, reference creation, and dereferencing over an atomic term respects bijective variable and atom renaming. In particular this covers the original `ref` 1 and $!x$ subterms of Proposition 12.2, as well as dereferencing a named atom. It is a theorem about the original translated graphs; it does not assert an allocation or read transition of the unfinished machine.

76.1 Decomposing a typed environment permutation

An environment consists of k term positions followed by h atom positions. Embed a variable index i at position i, and an atom index a at position $k + a$. Looking up their wire types gives term and atom, respectively. Conversely, a position has term type exactly when it is less than k, and atom type exactly when it is at least k. Split the appended boundary list at k to prove these two characterisations. They work with either block empty.

Lean: `EnvironmentPermutation.termIndex`

Lean: `EnvironmentPermutation.atomIndex`

Lean: `EnvironmentPermutation.term_label`

Lean: `EnvironmentPermutation.atom_label`

Lean: `EnvironmentPermutation.term_iff`

Lean: `EnvironmentPermutation.atom_iff`

Let π be any bijection of environment positions preserving wire types. Its variable component sends i to the numeric value of $\pi(i)$; type preservation proves that value is less than k. Its atom component sends a to the numeric value of $\pi(k + a)$ minus k. The result is less than h by the environment bound. Type preservation proves $\pi(k + a) \geq k$, so embedding the new atom index reconstructs $\pi(k + a)$ exactly. The variable reconstruction is definitional.

Lean: `EnvironmentPermutation.termMap`

Lean: `EnvironmentPermutation.atomMap`

Lean: `EnvironmentPermutation.term_action`

Lean: `EnvironmentPermutation.atom_action`

The inverse of π preserves types as well: apply type preservation at an inverse image and then the right-inverse equation. Extract the two maps from this inverse. To prove the extracted variable maps inverse, embed their indices in the environment and use the original inverse equation; equality of numeric values suffices for finite indices. For atoms, reconstruct both embedded images and cancel the common offset k. These arguments package independent resource bijections ρ and α from an arbitrary typed environment bijection. No premise says in advance that the two sorts are preserved separately.

Lean: `EnvironmentPermutation.inverse_typed`

Lean: `EnvironmentPermutation.terms`

Lean: `EnvironmentPermutation.atoms`

76.2 Exact actions through the original distributors

The tensor boundary action on a left input is the left component's input map followed by the left block injection. The right input, left output and right output laws are analogous. Each follows from the existing parallel-insertion commutation theorem, specialised to the relevant component and boundary port, then the extracted input or output equation. These laws explicitly retain the variable-before-atom ordering, including the offset of the right block.

Lean: `TensorBoundaryActions.tensor_input_left`

Lean: `TensorBoundaryActions.tensor_input_right`

Lean: `TensorBoundaryActions.tensor_output_left`

Lean: `TensorBoundaryActions.tensor_output_right`

For separate resource bijections ρ and α, form the tensor of the original one-use distributor automorphisms, and transport its input interface through $k \cdot 1 = k$ and $h \cdot 1 = h$. This gives an automorphism of the actual `singleUse` translation graph. Its term input and output actions are $i \mapsto \rho(i)$, while its atom actions are $k + a \mapsto k + \alpha(a)$. For the input laws, use the original use-major position $i + k \cdot 0 = i$, then the tensor input law and equality transport. The output laws use the original distributor output formula and tensor output law. These proofs do not simplify the single-use graph to an identity wire.

Lean: `SingleUseBoundaryRenaming.iso`

Lean: `SingleUseBoundaryRenaming.input_term`

Lean: `SingleUseBoundaryRenaming.input_atom`

Lean: `SingleUseBoundaryRenaming.output_term`

Lean: `SingleUseBoundaryRenaming.output_atom`

Apply this construction to the extracted pair (ρ, α) of any typed permutation π. Every environment position is in the variable or atom block; in each case use its reconstruction equation and the corresponding block action. The resulting original single-use graph therefore realises precisely π on both its inputs and its outputs. This is the critical extension theorem: no separate distributor-matching premise is required from its caller.

Lean: `SingleUseBoundaryRenaming.ofTyped`

Lean: `SingleUseBoundaryRenaming.ofTyped_input`

Lean: `SingleUseBoundaryRenaming.ofTyped_output`

76.3 The complete unary constructor theorem

Let $f : G \to H$ be any typed boundary isomorphism between operands with one term input and the same environment interface. The original unary clause has three serial components: the unary operation edge, the operand, and the single-use environment distributor. Its operation edge has one output and the operand has one input. Both associated index maps are necessarily the identity on a singleton, so the operation's identity isomorphism glues with

f. This constructs the prefix isomorphism with no extra hypothesis.

Lean: UnaryBoundaryRenaming.unary_prefix_compatible

Lean: UnaryBoundaryRenaming.unaryPrefixIso

The output map f_O preserves boundary wire types by the extracted output label law. Apply the preceding single-use extension theorem to this very bijection, obtaining the forest automorphism. The prefix's output action is f_O, by the serial output law; the forest's input action is f_O, by the extension theorem. Thus their compatibility is proved. Applying serial gluing again gives an isomorphism between the two literal unary translations. Its external output action remains exactly f_O, because the forest's output action is f_O as well. This theorem applies at arbitrary allowance and to arbitrary operands admitting f, including operands with nested boxes. It does not itself construct the separate box-body renaming correspondence needed for a lambda.

Lean: UnaryBoundaryRenaming.unaryForestIso

Lean: UnaryBoundaryRenaming.unary_forest_compatible

Lean: UnaryBoundaryRenaming.unary_boundaryIso

Lean: UnaryBoundaryRenaming.unary_boundaryIso_output

76.4 Induction over the original syntax

The unary-fragment predicate is an indexed predicate on the existing `Term`: atomic terms belong to it, and applying any original unary operation preserves membership. It is not a new term datatype or a replacement translation. Transporting a graph along equality of allowances commutes with the original unary clause; eliminate that equality to prove the statement. Now induct on fragment membership. The leaf case is the previous atomic renaming theorem. For a unary node, expose the original translation and renaming clauses, move the allowance equality through the unary constructor, and apply the constructor theorem to the induction hypothesis. The result has only the fragment-membership and bijection hypotheses, with no machine, stability, or template premise.

Lean: UnaryFragmentRenaming.UnaryFragment

Lean: UnaryFragmentRenaming.castDepth_unary

Lean: UnaryFragmentRenaming.unaryFragment_rename

A list of unary operators can be folded around an original term. Induction on the list shows that folding around an atomic term belongs to the fragment, and the fragment theorem then proves renaming for arbitrarily long chains. This supplies an unbounded syntactic family, rather than a fixed set of examples.

Lean: UnaryFragmentRenaming.unaryChain

Lean: UnaryFragmentRenaming.unaryChain_fragment

Lean: UnaryFragmentRenaming.unaryChain_rename

The three public example constructions use the constructor theorem directly. For `ref` 1, use the original constant automorphism with both ambient resource bijections. For dereferencing an atom, use the original atom-renaming isomorphism, retaining its instance edge. For dereferencing a variable, use the original selected-variable isomorphism. All three have

arbitrary finite variable and atom scopes and preserve the original terms, labels, and sharing graphs. They establish representation correspondence, not behavioural equivalence between allocation and a constant result.

Lean: `UnaryFragmentRenaming.referenceOne_boundaryRename`

Lean: `UnaryFragmentRenaming.dereferenceAtom_boundaryRename`

Lean: `UnaryFragmentRenaming.dereferenceVariable_boundaryRename`

Updated remaining work. Unary operations are removed from the remaining bijective translation-renaming cases. Application, binary operations, and lambda boxes remain, together with non-bijective renaming and graph-level substitution. The concrete copying UAM, its representation laws, the eleven template families, semantic weakening, and Figure 22's observational-equivalence chain are still required. The full build and all three axiom audits pass for the present milestone without `sorryAx` or project-specific axioms.

As of 25 October 2026, the estimated completion is approximately **45% by mathematical effort**, with a broad plausible range of **30–55%**. This supersedes the earlier estimate after the serial-only milestone. The increase reflects a completed constructor and an unbounded fragment theorem; it does not count infrastructure declarations as completed machine or template proofs.

The application and binary obligations recorded at this milestone are subsequently discharged in Section 77; Appendix B records the current remaining tasks.

77 Renaming all lambda-free terms and padded binary clauses

The main theorem of this milestone proves bijective environment renaming for every lambda-free term of the original syntax, at arbitrary finite variable and atom scopes. Application, all unary operations, and all four binary operations may be nested without restriction. Repeated variable or atom occurrences are allowed: both two-use distributor forests and the original middle-block exchange remain in the graph. A named distinct-list environment corollary uses exactly the existing environment and translation definitions.

The construction also proves application and binary congruence for supplied operand isomorphisms at arbitrary, possibly unequal allowances. Thus it is usable with boxed operands once their own boundary correspondence is established. It does not prove lambda-box renaming, an operational allocation/read/write rule, or Proposition 12.2. The theorem at the end is stated on the original `Translation.term`; no replacement translation is introduced.

77.1 Coordinates of the original operand shuffle

The left and right operand tags are represented by Booleans, with use numbers zero and one. A component index pairs such a tag with an original environment position. The grouped position is obtained by applying the existing pair-boundary bijection and transporting through the existing interface-length equality. Every grouped position has this form: invert the pair-boundary bijection after transporting back along the length equality, and split its component tag.

Lean: `PairBoundaryCoordinates.use`

Lean: `PairBoundaryCoordinates.component`

Lean: `PairBoundaryCoordinates.index`

Lean: `PairBoundaryCoordinates.index_surjective`

The original shuffle is the inverse of the original four-block ordering followed by the shuffled ordering. Evaluating it at an original component therefore cancels the original ordering's inverse. Apply this cancellation in the four cases (left/right operand, term/atom resource). Term resource i in use u reaches grouped term position $i + ku$. Atom resource a reaches grouped atom position $a + hu$, after both term blocks; its absolute position is $2k + a + hu$. For atoms, the separate concatenation identity checks that $k + a$ is exactly the right-block injection, by equality of finite-index values. These calculations retain the original order rather than replacing the shuffle by a counting law.

Lean: `PairBoundaryCoordinates.shuffle_component`

Lean: `PairBoundaryCoordinates.term_index`

Lean: `PairBoundaryCoordinates.atom_join`

Lean: `PairBoundaryCoordinates.atom_index`

77.2 Matching the multi-use environment forests

Tensor the original automorphisms of $D_{k,m}$ and $D_{h,m}$. This constructs an automorphism of the actual environment distributor for every use count m and allowance. Its outputs carry the resource actions ρ and α. Its input actions preserve the use number: $i + ku$ maps to $\rho(i) + ku$, and $a + hu$ maps to $\alpha(a) + hu$. The existing finite-grid index supplies the bounds even for arbitrary m; the original distributor action and tensor block laws prove the ordered-port equations. In particular, zero resources and zero uses need no nonemptiness assumption.

Lean: `MultiUseBoundaryRenaming.iso`

Lean: `MultiUseBoundaryRenaming.output_term`

Lean: `MultiUseBoundaryRenaming.output_atom`

Lean: `MultiUseBoundaryRenaming.input_term`

Lean: `MultiUseBoundaryRenaming.input_atom`

A typed environment permutation π splits into its term and atom bijections by the previous section. Apply the multi-use construction to that pair. Splitting an arbitrary output position into the two resource blocks proves that the output action is exactly π. For two uses, the coordinate laws above identify the input action with applying π independently inside each original operand, then performing the original pair shuffle. This is the precise bridge between the two operand graphs and their sharing forests.

Lean: `MultiUseBoundaryRenaming.ofTyped`

Lean: `MultiUseBoundaryRenaming.ofTyped_output`

Lean: `MultiUseBoundaryRenaming.ofTyped_pair_input`

77.3 The application and binary constructor proofs

A typed boundary isomorphism fixes a singleton input index: both its image and the original index are zero. Assemble two operand isomorphisms by the existing parallel theorem using the original pair boundary; transport the resulting output interface along the same equality as the translation. This defines the raw and final pair isomorphisms. Their two argument inputs are individually fixed, as follows by the singleton law and the two component insertion equations. If the operand output maps agree, both shuffled output blocks carry that common map; prove this by the component output equation in each of the two cases.

Lean: `PairBoundaryRenaming.input_singleton`

Lean: `PairBoundaryRenaming.rawIso`

Lean: `PairBoundaryRenaming.iso`

Lean: `PairBoundaryRenaming.iso_input`

Lean: `PairBoundaryRenaming.iso_output`

A small head index selects either the original application edge or one of the original binary-operation edges. In both cases the selected graph is the existing primitive graph with one term input and two ordered term outputs. It is an abbreviation for these original graphs, not

an additional language operation. The identity of this head matches the fixed two argument inputs of the pair isomorphism, so serial gluing yields the prefix isomorphism.

Lean: BinaryBoundaryRenaming.Head

Lean: BinaryBoundaryRenaming.graph

Lean: BinaryBoundaryRenaming.prefix_compatible

Lean: BinaryBoundaryRenaming.prefixIso

Choose the actual two-use forest automorphism from the left operand's typed output bijection. To prove compatibility, express every intermediate position as a component and an environment position using surjectivity of the pair index. The prefix output law, the pair output law, and the two-use input law all give the same grouped position. Thus agreement between operand resource actions proves the entire matching obligation. The clause abbreviation is literally the original serial expression, and gluing this prefix and forest gives its boundary isomorphism. The external output action equals the original left operand action because the final forest realises that action exactly.

Lean: BinaryBoundaryRenaming.forestIso

Lean: BinaryBoundaryRenaming.forest_compatible

Lean: BinaryBoundaryRenaming.clause

Lean: BinaryBoundaryRenaming.iso

Lean: BinaryBoundaryRenaming.iso_output

Specialise the head to application or to each binary operation. This gives public constructor isomorphisms for the existing translation functions, together with their exact external output equations. The only compatibility hypothesis at this constructor level is equality of the two operand output actions; below, the syntax induction proves it from a common resource specification.

Lean: BinaryBoundaryRenaming.application_boundaryIso

Lean: BinaryBoundaryRenaming.binary_boundaryIso

Lean: BinaryBoundaryRenaming.application_boundaryIso_output

Lean: BinaryBoundaryRenaming.binary_boundaryIso_output

77.4 The resource-action induction invariant

For resource bijections ρ and α, a resource-action certificate says that an isomorphism maps variable output i to $\rho(i)$ and atom output $k + a$ to $k + \alpha(a)$. Two such certificates imply equality of their complete output index maps: split an arbitrary environment index at k, apply the relevant certificate equation, and use injectivity of the output-port constructor. This is why a mere existence theorem for each operand is too weak for induction.

Lean: ResourceBoundaryAction.ResourceAction

Lean: ResourceBoundaryAction.agree

If a constructor's external action is the operand's action, compose that output equation with the operand certificate. This proves a general certificate transfer lemma. Apply it to the

existing unary output theorem and to the new application and binary output theorems, using the agreement just proved for the two operands. Consequently the same specified resource action persists through every non-binding constructor.

Lean: ResourceBoundaryAction.of_output

Lean: ResourceBoundaryAction.unary

Lean: ResourceBoundaryAction.application

Lean: ResourceBoundaryAction.binary

The atomic isomorphisms must satisfy that stronger specification too. Extract the selected-resource output index from its existing explicit port equation. Tensor it with the unused atom forest to prove the variable clause's two resource equations. For the instance prefix, insertion commutation at an output says that its external output map is exactly the operand output map; checking the allowance cases proves this for the original prefix. Combine it with the selected-atom action and unused variable forest to prove the atom clause. For constants, the primitive has no outputs, so the inner tensor's outputs all come from the variable weakening forest and the outer tensor adds exactly the atom block. Tensor output laws and the empty-output arithmetic prove both constant equations. All existing interface casts are retained throughout.

Lean: AtomicBoundaryActions.selected_outputIndex

Lean: AtomicBoundaryActions.variableTerm_action

Lean: AtomicBoundaryActions.iso_output

Lean: AtomicBoundaryActions.atom_action

Lean: AtomicBoundaryActions.constant_action

77.5 Induction on every lambda-free term

The lambda-free predicate is indexed by the original term. It includes variables, atoms, constants, and is closed under every unary and binary operation and application. It has no lambda constructor. Ordinary syntactic renaming preserves membership by induction, including non-injective renamings; this membership fact alone is not a graph substitution or non-injective graph-renaming theorem. A second induction shows the translation allowance is zero: leaves have depth zero, unary depth is unchanged, and the maximum of two zeros is zero.

Lean: LambdaFreeRenaming.LambdaFree

Lean: LambdaFreeRenaming.rename

Lean: LambdaFreeRenaming.depth_zero

For the induction only, expose this zero allowance by equality transport of the already defined translated graph. This auxiliary graph is the original translation with its index rewritten, not a simplification of its carriers or edges. Equality transport commutes with unary translation. For application and binary translation, eliminate the two depth-zero equalities: their maximum is zero and both original pads compute to the zero pad. This proves the two general zero-allowance

identities and their specialisations to actual translated terms.

Lean: LambdaFreeRenaming.graph

Lean: LambdaFreeRenaming.graph_unary

Lean: LambdaFreeRenaming.application_zero

Lean: LambdaFreeRenaming.binary_zero

Lean: LambdaFreeRenaming.graph_application

Lean: LambdaFreeRenaming.graph_binary

The strong induction theorem constructs an isomorphism together with its resource-action certificate. Leaves use the newly certified atomic isomorphisms. A unary node uses unary constructor congruence and certificate preservation. At application or binary nodes, obtain both certified operand isomorphisms, derive agreement from the certificates, apply constructor congruence, and carry the certificate forward. Thus the agreement premise is discharged inside the proof, at every node and for arbitrarily repeated resources.

Finally, eliminating the two zero-allowance equalities converts the induction's isomorphism back to the original translated graphs. The theorem `Translation.lambdaFree_rename` has only the original term, its lambda-free certificate, and the two finite resource bijections as parameters. It has no operational, stability, robustness, or operand-agreement assumption. The only target equality transport is the pre-existing syntactic depth-renaming identity, just as in the earlier atomic endpoint.

Lean: LambdaFreeRenaming.rename_with_action

Lean: LambdaFreeRenaming.uncast

Lean: LambdaFreeRenaming.lambdaFree_rename

77.6 Unequal allowances and supplied boxed operands

Recursive label equivalence survives one step of the original padding. Primitive labels retain equality. For boxes, their existing body isomorphism is padded by the previously proved recursive-isomorphism theorem, preserving binder metadata. This yields a padded boundary isomorphism with the exact same outer port and edge bijections. Its external output action is unchanged. Iteration gives any finite padding, with the output equation proved by induction on the number of steps. These are structural representation results, not execution-padding invariance.

Lean: BoundaryPaddingRenaming.label_equiv

Lean: BoundaryPaddingRenaming.raise

Lean: BoundaryPaddingRenaming.raise_output

Lean: BoundaryPaddingRenaming.pad

Lean: BoundaryPaddingRenaming.pad_output

Equality transport of the allowance also preserves the isomorphism and its output action; eliminate the equality to prove both facts. Compose iterated padding and equality transport to obtain the original `padTo` operation, whose output law is obtained by composing the two

preceding laws. Substituting that equation into the term and atom certificate fields proves resource-action preservation under padding.

Lean: BoundaryPaddingRenaming.castDepth

Lean: BoundaryPaddingRenaming.castDepth_output

Lean: BoundaryIso.padTo

Lean: BoundaryPaddingRenaming.padTo_output

Lean: ResourceAction.padTo

For operands at depths d and e, pad their supplied certified isomorphisms independently to $\max(d, e)$. Their resource certificates still agree, so the application and binary constructor theorems apply at exactly the common allowance chosen by Figure 16. No equal-depth premise is added, and no box is removed. These two constructors are ready for operands containing boxes, provided their own resource-renaming isomorphisms are established separately.

Lean: BoundaryPaddingRenaming.padded_application_boundaryIso

Lean: BoundaryPaddingRenaming.padded_binary_boundaryIso

77.7 Concrete instances, rejected mismatch, and remaining scope

The callback invocation $f()$ is an application of a variable to the original unit constant, so its lambda-free certificate immediately gives the renaming theorem. Self-application of a variable gives a regression with the same resource in both operands. Assignment of an allocated reference combines the original atom, reference-creation, constant and assignment clauses; the same endpoint proves its renaming. A further family combines dereferencing a variable with an atom and is parameterised by every binary operation, covering addition, subtraction, assignment and atom equality without separate assumptions. These are translation theorems, not termination or store-behaviour assertions.

Lean: LambdaFreeRenamingExamples.callbackInvocation_rename

Lean: LambdaFreeRenamingExamples.repeatedVariable_rename

Lean: LambdaFreeRenamingExamples.assignmentAllocation_rename

Lean: LambdaFreeRenamingExamples.everyBinary_rename

A concrete two-variable, two-atom term applies a dereferenced variable to an atom-equality expression. Its membership proof follows the syntax, and swapping both resource sorts instantiates the general theorem nontrivially. For the negative regression, compare two original constant-graph automorphisms: one swaps two variable resources and one fixes them. Their certified images of variable output zero differ. Equality of their whole output actions would imply $1 = 0$. This rejects the tempting but invalid claim that well-typed operand isomorphisms automatically match.

Lean: LambdaFreeRenamingExamples.twoSortExample

Lean: LambdaFreeRenamingExamples.twoSortExample_free

Lean: LambdaFreeRenamingExamples.twoSortExample_rename

Lean: LambdaFreeRenamingExamples.mismatched_operand_actions

Identity isomorphisms satisfy the identity resource certificate by reflexivity. Further regressions

use the proved nontrivial atom and variable permutations, first padded to allowances one and three, then combined by the common-allowance application and binary theorems. A genuine translated lambda operand at depth one can also be combined with a depth-zero constant, using the lambda operand's identity isomorphism. This last instance tests boxed composition, not arbitrary lambda renaming. Finally the named-environment corollary specialises the lambda-free theorem to the lengths of an existing distinct-list environment; the environment translation wrapper is definitionally the original translation.

Lean: `BinaryRenamingRegression.refl`

Lean: `BinaryRenamingRegression.unequalAllowanceRenaming`

Lean: `BinaryRenamingRegression.unequalAllowanceBinaryRenaming`

Lean: `BinaryRenamingRegression.boxedOperandApplication`

Lean: `BinaryRenamingRegression.lambdaFree_namedEnvironment`

Completion boundary and estimate. Application and binary operations are removed from the remaining bijective translation-renaming cases. The missing constructor is lambda, including the correspondence between a box's external free-resource interface and its body's ordered boundary. The existing recursive label relation fixes body boundaries; it cannot silently serve as a proof of arbitrary boundary permutation inside a box. Establish that correspondence while preserving the original terms and Figure 16 translation. Non-bijective graph renaming, graph substitution, the concrete copying UAM, its execution representation laws, all eleven robustness template families, semantic weakening, and Figure 22's equivalence chain remain.

As of 25 October 2026, estimated completion of the requested Proposition 12.2 formalisation is approximately **50% by mathematical effort**, with a broad plausible range of **35–60%**. This estimate supersedes the earlier unary-fragment estimate. It reflects a complete non-binding syntax induction and unequal-allowance constructor proofs; it does not count these as completed machine or robustness-template proofs. Proposition 12.2 itself is not yet proved.

Subsequent milestone. Section 78 resolves the structural lambda case by recording BPerm steps explicitly and proves the full syntax induction. The earlier lambda-free result remains an ordered boundary-isomorphism theorem. The stronger syntax coverage uses a different, explicit derivation conclusion; its semantic interpretation is not asserted here.

78 Full translation renaming through box permutation

This milestone closes the structural bijective-renaming induction for every constructor of the original term language, including arbitrarily nested lambda boxes and operands at unequal allowances. The theorem is about the actual Figure 16 translation, with its original distributors, shuffle and boxes. Its conclusion is a finite derivation generated by the box-permutation rule and structural congruence, followed by an ordered boundary isomorphism with a proved resource action. It does not assert observational equivalence.

This distinction is mathematically necessary. An ordered isomorphism of two single-box graphs fixes each outer output individually. A nontrivial permutation of their free resources therefore requires the structural box-permutation pre-template of Figure 21, not an enlargement of the existing isomorphism notion. The new derivation type records this use explicitly. Its semantic interpretation still requires the copying machine, BPerm robustness and execution congruence. No machine or robustness hypothesis is hidden in the structural theorem.

78.1 Typed permutations and the bound-prefix offset

A typed permutation of an output list is a bijection of its positions preserving each wire label. Identity is immediate. Inversion follows by applying the forward typing equation at the inverse position; composition follows by transitivity of the two typing equations. The underlying finite-position bijections remain explicit rather than being selected by choice.

Lean: BoxPermutationPair.TypedPermutation

Lean: BoxPermutationPair.refl

Lean: BoxPermutationPair.symm

Lean: BoxPermutationPair.trans

Prefix extension acts as the identity below n and sends position $n + j$ to $n + p(j)$. The bound and free evaluation lemmas follow by the two branches of this definition. For bijectivity, split a position at n and use the original inverse laws in the free branch. For typing, the prefix is homogeneous in term wires and the suffix equation is exactly the supplied typing equation. Empty prefixes and empty free interfaces require no special assumptions.

Lean: BoxPermutationPair.prefixMap

Lean: BoxPermutationPair.prefixMap_bound

Lean: BoxPermutationPair.prefixMap_free

Lean: BoxPermutationPair.extend

An equality cast changes only the interface index; its numerical-position lemma follows by equality elimination. The combined term/atom permutation is read from the already proved distributor boundary action. The two block lemmas extract its action on term and atom indices from the corresponding port equations.

Lean: BoxPermutationPair.cast

Lean: BoxPermutationPair.environment

Lean: BoxPermutationPair.environment_term

Lean: BoxPermutationPair.environment_atom

Lean: BoxPermutationPair.cast_val

Splitting the body interface exposes n bound variables followed by the whole free interface. The inner permutation is the prefix extension transported along that equality. A box-permutation pair records the outer and inner typed bijections, pointwise fixation of bound outputs, and the equation

$$p_{\mathrm{body}}(n + j) = n + p_{\mathrm{free}}(j).$$

The bound/free index injections retain the offset explicitly. Evaluation proves that every typed outer permutation gives a pair. Conversely, splitting an arbitrary body index at n proves that a pair's inner action is uniquely specified by its outer action. Inverse pairs follow by applying the inverse body map to the two original equations. Composite pairs follow by substituting the first pair's equations into the second pair's equations.

Lean: BoxPermutationPair.interface_split

Lean: BoxPermutationPair.inner

Lean: BoxPermutationPair.boundIndex

Lean: BoxPermutationPair.freeIndex

Lean: BoxPermutationPair.Pair

Lean: BoxPermutationPair.inner_bound

Lean: BoxPermutationPair.inner_free

Lean: BoxPermutationPair.pair

Lean: BoxPermutationPair.Pair.unique

Lean: BoxPermutationPair.Pair.symm

Lean: BoxPermutationPair.Pair.trans

Printed indexing discrepancy. PDF Definition 12.7(2) (HTML Section 12.2) prints $\rho(i) = \rho'(i - n)$ for free positions. Taken literally, its right side omits the bound-prefix offset. With one bound and one free variable, both body positions would map to the bound position, contradicting injectivity. The checked regression below certifies this obstruction. We use the offset-preserving equation above, equivalently $\rho(i) = n + \rho'(i - n)$ in the paper's one-based coordinates. This explicit indexing correction changes neither the terms nor their translation.

78.2 Actual interface permutation and compensation

The interface operation fixes internal ports and input ports and moves an old output i to output $p(i)$. Its inverse uses p^{-1}. It preserves port roles by case analysis and wire types by the permutation's typing equation. In the paper's convention, which reorders the boundary list, this forward-position operation is $\Pi_{\mathrm{id}}^{p^{-1}}$. Keeping track of the inverse avoids reversing the compensation outside a box.

Lean: `InterfacePermutation.ports`

Lean: `InterfacePermutation.input_iff`

Lean: `InterfacePermutation.output_iff`

Lean: `InterfacePermutation.label`

The permuted graph has exactly the same internal carrier, edge carrier, finite enumerations and labels. Both incidence lists are mapped by the port bijection in their existing order. Injectivity preserves no-duplicates and incidence uniqueness. Pulling a port back along the inverse proves coverage; the type equation proves the two typed-incidence laws. The resulting boundary isomorphism is the port bijection with identity on edges, and its input and output actions compute respectively to identity and p. Primitive-label restrictions are unchanged definitionally; binding-freedom and stability follow from the previously proved boundary-isomorphism invariance theorems.

Lean: `InterfacePermutation.net`

Lean: `InterfacePermutation.iso`

Lean: `InterfacePermutation.iso_input`

Lean: `InterfacePermutation.iso_output`

Lean: `InterfacePermutation.allLabels_iff`

Lean: `InterfacePermutation.bindingFree_iff`

Lean: `InterfacePermutation.stable_iff`

Suppose $f : G \to H$ fixes inputs and has output action p. Compose it with interface permutation by p^{-1} on H. The composite fixes both boundaries, so the existing conversion theorem yields a genuine boundary-fixing recursive isomorphism $G \cong \Pi_{\mathrm{id}}^{p} H$. Applying this to the interface operation itself proves the round-trip law. Applying the same permutation to both sides of a fixed-boundary isomorphism follows by conjugating its port map. Finally, permuting the compensated isomorphism and cancelling the round trip factors f through actual interface permutation of G. All these results preserve the original ordered notion of recursive isomorphism.

Lean: `InterfacePermutation.compensate`

Lean: `InterfacePermutation.roundtrip`

Lean: `InterfacePermutation.congr`

Lean: `BoxPermutationRule.realize`

78.3 The concrete BPerm graph pair and its structural safety

For a body G and a certified pair $(p_{\mathrm{body}}, p_{\mathrm{free}})$, the literal Figure 21 rule relates a box containing $\Pi_{\mathrm{id}}^{p_{\mathrm{body}}} G$ to $\Pi_{\mathrm{id}}^{p_{\mathrm{free}}}(\mathrm{box}\, G)$. In our forward-position convention both displayed permutations therefore use inverse bijections. The raw relation has exactly this constructor, with the same outer interfaces. Its isomorphism closure allows a boundary-fixing recursive isomorphism before and after that one raw rule.

A useful compensated normal form starts from the original box and ends at the box of the forward-permuted body, with inverse free permutation outside. The function `right` constructs this graph; the normal-form theorem below *derives* the pair from the literal rule. Neither relation asserts observational equivalence or execution.

Lean: BoxPermutationRule.right

Lean: BoxPermutationRule.Step

Lean: BoxPermutationRule.UpToIso

Every recursively checked primitive-label restriction is preserved because the only new outer label is the same box and the body's labels have not changed. If a body isomorphism has the pair's prescribed output action, compensation identifies its source with the inverse-permuted target. Apply box congruence to that fixed-boundary isomorphism, then apply the literal paper rule to the target body. This places the required pair in the rule's isomorphism closure without assuming the desired rule instance. Specialising to the actual interface-operation isomorphism derives the compensated normal form.

Lean: BoxPermutationRule.allLabels_right

Lean: BoxPermutationRule.of_bodyIso

Lean: BoxPermutationRule.normalized

For comparison, take an ordered isomorphism between two original box corollas. Apply its target-incidence equation to position i in the ordered output list. The result says that output i maps to output i. Hence a nontrivial free permutation cannot be implemented by such an isomorphism, regardless of the two body graphs. This rules out the tempting but incorrect shortcut of treating BPerm as ordinary graph renaming.

Lean: BoxPermutationRule.box_output_fixed

Lean: BoxPermutationRule.nontrivial_not_ordered

Binding-freedom of a single box is equivalent to binding-freedom of its body: the outer level contains no hole, while focus exclusion and inner binding paths are exactly those of the body. Combine this with interface-permutation invariance to obtain the right graph's safety equivalence. Case analysis on a raw rule proves preservation of all primitive restrictions and binding-freedom. The same properties extend across the two boundary-fixing isomorphisms. These are structural safety theorems, not machine input-safety or robustness.

Lean: BoxPermutationSafety.bindingFree_box_iff

Lean: BoxPermutationSafety.bindingFree_right

Lean: BoxPermutationSafety.Step.allLabels

Lean: BoxPermutationSafety.Step.bindingFree

Lean: BoxPermutationSafety.UpToIso.bindingFree

Lean: BoxPermutationSafety.UpToIso.allLabels

78.4 The original lambda interface and abstraction shell

Lift a variable bijection under a binder using the existing capture-free `Term.lift`: zero is fixed and successor positions follow the free bijection. The inverse equations follow by zero/successor case analysis. Combine this lifted bijection with the atom bijection and apply precisely the interface cast already used by `lambdaBox`. The resulting lambda pair satisfies the bound/free conditions: split free positions into term and atom blocks and use their exact distributor actions. This proves the agreement between the scoped binder convention and the box's ordered body boundary.

Lean: LambdaPermutationPair.lift

Lean: LambdaPermutationPair.lambdaBodyPermutation

Lean: LambdaPermutationPair.lambdaPair

A resource-action certificate determines the whole output permutation: split an arbitrary position into its term or atom block and apply the respective certificate field. Equality transport carries this equation through the lambda body cast. The general body-isomorphism rule then produces a BPerm step for the original lambda box, with the inverse outer permutation explicitly present.

Lean: LambdaPermutationPair.outputs

Lean: LambdaPermutationPair.cast_output_action

Lean: LambdaPermutationPair.lambda_of_action

The abstraction shell is the literal lambda operation, followed by a supplied box graph, followed by the mandatory single-use distributors. Its singleton thunk input fixes compatibility with the lambda edge. Build the prefix isomorphism; use its typed free-output action to choose the existing single-use forest automorphism. The matching equation proves serial compatibility. Composing the two isomorphisms gives shell congruence, and the final port equation proves that the shell retains exactly the supplied box's resource action.

Lean: AbstractionShell.graph

Lean: AbstractionShell.prefix_compatible

Lean: AbstractionShell.prefixIso

Lean: AbstractionShell.forestIso

Lean: AbstractionShell.forest_compatible

Lean: AbstractionShell.iso

Lean: AbstractionShell.iso_output

78.5 Finite structural derivations and their precise scope

A derivation is an inductively generated finite proof tree. Its generators are boundary-fixing recursive isomorphism and the concrete BPerm rule. It is closed under symmetry, transitivity, serial and parallel composition, tensor, enclosing a box, equality transport of interfaces, and structural depth raising. These are explicit syntactic congruence constructors. In particular, the raising constructor is not a claim of execution-padding invariance, and the rule constructor is not a proof of BPerm robustness. There is no constructor for arbitrary pairs of terms, no constructor assuming renaming, and no Proposition 12.2 constructor.

Lean: BoxPermutationDerivation.Derivation

Reflexivity uses identity isomorphism. A rule up to isomorphism expands into two isomorphism generators and one raw rule. Depth equality is eliminated; finite padding follows by induction on its number of raises, and common-allowance padding adds the existing equality cast. Shell, abstraction and unary congruence are derived by serial composition, retaining each fixed component. Pair congruence uses the original parallel boundary and its original output-interface cast; application and binary congruence prefix the original operation and append the original two-use distributors. Thus the derived clauses preserve Figure 16 literally.

Lean: BoxPermutationDerivation.refl

Lean: BoxPermutationDerivation.of_upToIso

Lean: BoxPermutationDerivation.depthCast

Lean: BoxPermutationDerivation.pad

Lean: BoxPermutationDerivation.padTo

Lean: BoxPermutationDerivation.shell

Lean: BoxPermutationDerivation.abstraction

Lean: BoxPermutationDerivation.unary

Lean: BoxPermutationDerivation.pair

Lean: BoxPermutationDerivation.application

Lean: BoxPermutationDerivation.binary

78.6 Full syntax induction with a resource-action invariant

A renaming witness consists of an intermediate graph M, a finite derivation $G \Rightarrow M$, an ordered boundary isomorphism $M \to H$, and its exact variable/atom resource-action certificate. The datatype permits heterogeneous allowance indices at its boundary, but its constructor requires those indices to coincide; consequently every witness proves equality of the allowances. The intermediate graph is constructed in the proof, not supplied by the caller. An ordinary certified boundary isomorphism gives a witness with a reflexive derivation.

Lean: FullTranslationRenaming.Renaming

Lean: FullTranslationRenaming.of_iso

Lean: FullTranslationRenaming.depth_eq

Unary congruence combines the derived syntactic clause with the existing unary isomorphism

and its action law. Padding combines derivation padding with proved isomorphism padding and certificate preservation. For application and binary operations, first match the two resource actions using their certificates, then use the existing constructor isomorphism. The padded variants eliminate the allowance equalities supplied by the two witnesses and pad both operands to exactly the original maximum. Thus neither equal original operand depths nor a separate matching premise is needed.

Lean: `FullTranslationRenaming.unary`

Lean: `FullTranslationRenaming.padTo`

Lean: `FullTranslationRenaming.application`

Lean: `FullTranslationRenaming.binary`

Lean: `FullTranslationRenaming.padded_application`

Lean: `FullTranslationRenaming.padded_binary`

The abstraction case is the essential new step. Lift the body's existing derivation through the original abstraction constructor. Its remaining body isomorphism supplies the lambda BPerm instance proved above. Lift that single rule, with its two isomorphisms, through the abstraction shell. The resulting box is the target box with inverse free-interface permutation. Invert that interface operation and use shell congruence, which also moves the original distributors. Its output equation gives the required free variable/atom action. The new witness therefore has exactly the required shape for another enclosing lambda; arbitrary nesting follows without a new assumption.

Lean: `FullTranslationRenaming.abstraction`

The final isomorphism of any witness can be compensated, yielding a finite derivation from G to the explicitly boundary-compensated target. The public translation theorem is proved by induction on the original term: leaves use the certified atomic isomorphisms; unary, application and binary nodes use the constructor results; lambda uses the capture-free lifted variable bijection and the new abstraction result. All arities and all finite variable/atom environments are quantified. The conclusion names the original translated term and the translation of its original syntactic renaming, with no replacement translation.

Lean: `FullTranslationRenaming.derivation`

Lean: `FullTranslationRenaming.term_rename_via_boxPermutation`

78.7 Invariants, regressions and remaining semantic obligations

Serial expansion contains exactly its two components' labels; inspect the Boolean component tag in each direction. Induction on structural derivations then proves preservation of every recursively checked primitive-label predicate. Isomorphisms and raw rules use their existing invariance results; composition uses component conjunctions; parallel assembly uses its component tag; boxing uses the body predicate; casts eliminate equalities; raising uses structural padding invariance. Focus exclusion, hole exclusion and membership in the full example language are immediate specialisations.

Lean: `BoxPermutationInvariants.allLabels_iff`

Lean: `BoxPermutationInvariants.allLabels`

Lean: BoxPermutationInvariants.focusFree

Lean: BoxPermutationInvariants.holeFree

Lean: BoxPermutationInvariants.language

The public derivation corollary exposes the final allowance equality and compensated target. Two further corollaries apply the full theorem to the exact left and right programs of Proposition 12.2; they prove renaming of each program, not equivalence between those two programs. The nested regression uses two binders above an application and nontrivially swaps both free resource sorts. A constant beside that term tests unequal allowances. The named-environment corollary simply uses the original distinct-list environment wrapper.

Lean: FullRenamingExamples.termRenamingDerivation

Lean: FullRenamingExamples.privateCell_fullRenaming

Lean: FullRenamingExamples.constantResult_fullRenaming

Lean: FullRenamingExamples.nestedRenamingTerm

Lean: FullRenamingExamples.nestedRenaming

Lean: FullRenamingExamples.unequalDepthRenaming

Lean: FullRenamingExamples.full_namedEnvironment

The negative regressions check three separate errors. Swapping a bound position with a free position contradicts the bound-fixation condition. Swapping two free outputs of an ordered box contradicts the corolla output theorem. Finally, the printed unshifted pair equation at one bound and one free variable forces a collision in a two-element bijection, exposing the indexing discrepancy rather than silently repairing it.

Lean: FullRenamingExamples.captured_swap_rejected

Lean: FullRenamingExamples.box_swap_not_iso

Lean: FullRenamingExamples.unshifted_box_pair_impossible

Completion boundary and estimate. As of 26 October 2026, the structural renaming induction covers all syntax, including lambda boxes. The raw BPerm pair and its structural safety laws are also complete. Interpreting these derivations as contextual equivalences remains open: it requires BPerm robustness, the concrete copying UAM and the corresponding execution and congruence laws. Non-bijective graph renaming, graph substitution, the other template families, semantic weakening and Figure 22's final chain also remain. Appendix B lists only unfinished obligations. The estimated mathematical completion is now roughly **50–55%**, with a broad plausible range of **35–65%**. This is an effort estimate, not a count of declarations or a claim that half of the machine rules have been implemented. Proposition 12.2 is still unproved.

79 Concrete search for the copying UAM

This milestone defines the intrinsic search interactions on the copying graph hierarchy and proves their operational laws. It covers the operation-search rules of Figure 9 and the additional term-contraction and instance interactions of Figure 18 (PDF p. 37). It uses the complete example operation alphabet. Copying itself (Figure 17), substitution of stable arguments, and the compute rules, including Figure 19 store updates, remain separate obligations.

The representation is a token mode and a wire of an actual typed hypernet. The graph remains fixed during a search transition. No copying transition is encoded as a linear compute step. A rewrite token suspends this phase; it is not a successful value and does not mean the full copying machine is stuck. An eventual concrete specimen instance must connect these coordinates with focused graph states and combine them with copy and compute transitions.

79.1 Incidence rules and determinism

The argument count is zero for constants and lambda, one for unary operations, and two for application and binary operations. In particular, lambda's thunk output is not an evaluated argument. Primitive labels are injective at every depth: at depth zero project from the lifted label; at successor depth use injectivity of the primitive summand. A position is a pair (m, v) of mode and vertex.

Lean: `Copying.Operation.arguments`

Lean: `Copying.primitiveLabel_injective`

Lean: `Search.Position`

Entry at an operation with no evaluated arguments returns its finishing mode at the same wire; otherwise it searches target zero. Entry at either input of a term contraction produces rewrite mode at that input. Entry at an instance produces backtrack mode at the same wire. Atom contractions, weakening, stores, focus edges and holes have no entry instruction. On return from argument i, the resume function searches argument $i + 1$ if it exists among the evaluated arguments; otherwise it uses the operation's finishing mode. The optional result makes an absent target an absent transition.

Lean: `Search.enter`

Lean: `Search.resume`

The transition relation requires an actual incident edge. An entry witness gives its exact primitive label, the current vertex in its source list, and the entry equation. A return witness gives an operation edge with singleton source, an index strictly below its argument count, the current vertex at that target index, and the resume equation. Thus neither a label guess nor an arbitrary adjacency suffices. Both entry and resume commute with every vertex map: optional list lookup commutes with mapping, and all other branches retain the same mode and mapped vertex. Injectivity is unnecessary for these two laws.

Lean: `Search.Step`

Lean: `Search.enter_map`

Lean: `Search.resume_map`

For determinism, compare two entry witnesses using uniqueness of the edge sourcing a given vertex; label injectivity then identifies their instructions. For two return witnesses, target

uniqueness identifies the edge, singleton sources identify the root, and the duplicate-free target list identifies the argument index. The two instruction equations give equal successors. Entry and return cannot overlap because their source modes differ. The result-uniqueness lemma allows explicitly equal source positions; determinism is its reflexive case.

There is no search step out of rewrite mode. Backtrack at an external input also has no step, since an input cannot be an edge target. Finally, two finite search runs from the same source to normal endpoints have equal endpoints and equal lengths, by the generic deterministic trace theorem.

Lean: `Search.result_unique`

Lean: `Search.deterministic`

Lean: `Search.no_rewrite`

Lean: `Search.final_no_step`

Lean: `Search.terminal_run_unique`

79.2 Exact transport across graph representations

A search transport consists of bijections on vertices and edges, preservation and reflection of each primitive label, exact preservation of the ordered source and target lists, and preservation and reflection of input status. It may relate different depth allowances. No box-body equation is required by this interface because search does not enter boxes. It is a sufficient interface for search only, not a replacement for recursive graph isomorphism. Its position bijection preserves modes. Its inverse uses the inverse carrier bijections, the reversed primitive equivalence, and mapped-list inverse laws.

Lean: `Search.Transport`

Lean: `Search.Transport.positions`

Lean: `Search.Transport.symm`

Transport a step by mapping its incident edge and vertices. List membership, indexed lookup and singleton sources are preserved, and the entry/resume map lemmas supply the result equation. Applying the inverse transport proves reflection. Induction on a finite trace gives correspondence at exactly the same number of steps. Normality follows in both directions by transporting any hypothetical outgoing step back to the original graph.

Lean: `Search.Transport.step`

Lean: `Search.Transport.step_iff`

Lean: `Search.Transport.trace_iff`

Lean: `Search.Transport.terminal_iff`

Recursive label equivalence preserves and reflects equality with each primitive label: primitive cases are literal equality and box cases cannot be primitives. Hence a typed boundary isomorphism supplies a concrete search transport using its carrier bijections and incidence laws. This proves search invariance under the existing recursive isomorphism discipline without

identifying box permutation with ordered isomorphism.

Lean: Search.primitive_equiv

Lean: Search.ofBoundaryIso

79.3 Value recognition and wire typing

An entry or resume result in search mode belongs to the interacting edge's target list, by the successful optional lookup. Consequently no step produces search mode at an external input. For a backtrack entry result, case analysis on the primitive and operation proves that its vertex is the unchanged root and its label is a stable primitive. Conversely, a return witness has an argument index and therefore an active operation; its resume instruction cannot produce backtrack mode.

Lean: Search.enter_search_mem

Lean: Search.resume_search_mem

Lean: Search.no_initial_target

Lean: Search.enter_backtrack

Lean: Search.argument_active

Lean: Search.resume_not_backtrack

It follows that every transition producing backtrack mode identifies an actual passive-operation or instance edge. Its source is exactly the singleton containing the returned vertex: the typed source length is one, and source membership identifies that unique element. This is operational recognition of a stable primitive, not an assumed machine law. An interface equality cast preserves every shallow label predicate by equality elimination. Applied to the primitive corolla and the already proved stability criterion, this shows that the recognised primitive, isolated as its own corolla with the required input interface, is a stable graph. It does not claim that an arbitrary surrounding graph is stable.

Lean: Search.backtrack_stable_edge

Lean: Search.labels_cast

Lean: Search.recognized_primitive_stable

Typing is checked against the actual signatures. Successful entry has only term sources; given a term root, its result wire is term. Every evaluated argument index selects a term wire in the operation output signature. Resume therefore also preserves term type. These finite signature facts are proved by cases on the complete operation and primitive alphabets. For an actual step, map its instruction equation along the graph's vertex-label function and use the source/target typing equations. Both endpoint vertices then have term type. In particular, the instance's atom output and lambda's thunk output cannot become searched arguments.

Lean: Search.enter_input_term

Lean: Search.enter_output_term

Lean: Search.argument_type

Lean: Search.resume_output_term

Lean: Search.endpoints_term

79.4 Padding and shallow context insertion

One recursive raise supplies an actual search transport: outer vertex and edge carriers are unchanged, primitive labels remain the same primitive, ordered incidence is unchanged, and input status is unchanged. This argument uses the definition of padding, not determinism. Identity transports and composition are built from the corresponding bijections; the primitive, incidence and input equations compose. Iterate this construction for padding by n, transport across a depth-index equality by equality elimination, and compose the two for padding to an arbitrary common allowance. Step reflection and exact trace correspondence follow from the transport theorems.

Lean: Search.padding

Lean: Search.Transport.refl

Lean: Search.Transport.trans

Lean: Search.pad

Lean: Search.castDepth

Lean: Search.padTo

Lean: Search.padding_step_iff

Lean: Search.padTo_trace_iff

A successful entry edge cannot be a hole: label injectivity would identify its primitive with a hole, whose entry instruction is absent. When filling a shallow hole, a host interaction therefore retains its edge; return interactions also retain their operation edge. Map its vertices along the host attachment map. The plugging incidence equations and the entry/resume map laws prove the same transition in the filled graph. For an interaction inside the filling, use the inserted edge and inserted-vertex map instead. Neither map is assumed to be injective. Induction transports whole host or inserted traces without changing their lengths. These are preservation laws for shallow plugging; reflection, nested specimen classification and full-machine locality remain additional obligations.

Lean: Search.search_ne_hole

Lean: Search.plug_host

Lean: Search.plug_insert

Lean: Search.plug_host_trace

Lean: Search.plug_insert_trace

79.5 The phase machine and exact observations

Successful phase completion means backtrack mode at an external input. The phase machine has exactly the search relation as its step relation and this completion predicate as finality. Final normality is the input obstruction proved above; determinism is the concrete search theorem. Generic counted machine runs and graph traces coincide by induction on their constructors.

A transport preserves completion because it preserves mode and input status. Combining this with exact trace transport gives exact machine-run correspondence. For existential successful runs, an arbitrary target endpoint is pulled back through the inverse position bijection; conversely map the original endpoint forward. Thus successful runs exist at cost n on one side exactly when they do on the other. Instantiating the transport by arbitrary padding proves the corresponding operational padding theorem for this phase. No full copying execution instance is manufactured from it.

Lean: `Search.Completed`

Lean: `Search.machine`

Lean: `Search.machine_deterministic`

Lean: `Search.steps_iff_trace`

Lean: `Search.Transport.completed_iff`

Lean: `Search.Transport.machine_steps_iff`

Lean: `Search.Transport.completes_at_iff`

Lean: `Search.padding_completion_iff`

79.6 Regressions on the original translated graphs

The term-contraction and instance rules specialise the entry constructor with the exact Figure 18 instructions. A no-entry lemma rules out a competing witness: source uniqueness identifies its edge and primitive-label injectivity identifies its instruction, contradicting absence of an entry result. It excludes search at atom contractions and stores. Concrete corolla regressions cover both ordered term-contraction inputs and the instance input at every depth, using their explicit source lists.

Lean: `Search.term_contraction`

Lean: `Search.instance_return`

Lean: `Search.no_entry`

Lean: `Search.atom_contraction_no_search`

Lean: `Search.store_no_search`

Lean: `Search.contraction_ports`

Lean: `Search.instance_corolla`

For any translated abstraction body, the outer lambda edge is an actual edge of the two serial assemblies. Its source is the external input; its zero argument count returns backtrack there.

The original box and single-use distributors remain in place. This applies to the unchanged right-hand side of Proposition 12.2 for every environment size (k, h). Its one-step trace ends at a normal input, so deterministic terminal-run uniqueness proves that every successfully completed search run from this initial state has exactly one step and that same endpoint. This says nothing yet about callbacks or repeated use of the returned closure.

Lean: `Search.abstraction_returns`

Lean: `Search.constantResult_search`

Lean: `Search.constantResult_exact_cost`

For an ordered multi-argument test, take the literal Figure 16 translation of $1 + 2$ in the empty environment. The graph definition retains all distributors. Its named head, left constant and right constant edges are the corresponding finite assembly coordinates; argument i is target i of the head, rather than a separately invented vertex.

Lean: `Search.additionGraph`

Lean: `Search.additionHead`

Lean: `Search.additionLeft`

Lean: `Search.additionRight`

Lean: `Search.additionArgument`

The five checked transitions are: enter the head and search the left constant; return from that constant; resume at the head and search the right constant; return from it; finish the head in rewrite mode at the root. Each proof gives the actual edge, primitive label, ordered source and target positions, and reduced instruction equation. The trace concatenates these five witnesses. Since rewrite mode has no outgoing search step, every normal search run from the same initial state has this endpoint and length five. The padding trace theorem transports this concrete run to every larger depth allowance. No arithmetic computation or store mutation is claimed by the test.

Lean: `Search.addition_enter`

Lean: `Search.addition_left`

Lean: `Search.addition_next`

Lean: `Search.addition_right`

Lean: `Search.addition_finish`

Lean: `Search.addition_trace`

Lean: `Search.addition_exact_cost`

Lean: `Search.addition_padded`

The full build checks 298 jobs. All three axiom audits pass: six abstract, 477 graph and 428 copying declarations, with only the standard logical axioms `propext`, `Classical.choice` and `Quot.sound` where used. Appendix B removes the completed search-definition obligation while retaining the copy, compute, refocusing and full specimen-interface tasks. Estimated completion of Proposition 12.2 is about 55% by mathematical effort, with a broad 40–65%

uncertainty range (26 October 2026). This is a judgement about the remaining mathematics, not a ratio of declarations or pages; the template robustness proofs still carry substantial uncertainty.

80 Reference allocation with fresh store identity

The first concrete store computation is now formalized: reference creation, Figure 19(a) of the paper (PDF p. 38). The local rule retains its stable-value premise and the canonical atom tree $D^{\diamond}_{1,1}$. Its actual graph construction has proved freshness, preservation of the argument, exact store resolution, renaming compatibility, and forward padding preservation. A search/allocation subsystem executes the unchanged Figure 16 translation of `ref` n in five transitions for every natural literal n.

This is an operational extension beyond the preceding search-only milestone. It is still a subsystem: it omits copying, dereference, assignment, atom equality, and the other compute rules. Its allocation transitions are rooted local instances, optionally followed by an unchanged serial continuation. A complete contextual matching relation, focused-graph correspondence, determinism and refocusing for the combined machine remain to be proved. No observational execution instance or Proposition 12.2 theorem is asserted here.

80.1 The literal allocation pair and its stable premise

For a stable graph $G : \star \Rightarrow O$, the redex is serial composition of the `ref` corolla with G. The allocated prefix is serial composition of an instance, $D^{\diamond}_{1,1}$, and a store. It has type $\star \Rightarrow \star$. Appending G gives the reduct with the same external interface as the redex. The canonical tree is the original tree constructor; its weakening and contraction edges are not removed as an optimisation. All carriers and incidence equations are those of actual finite typed graphs.

The local rule is generated by this pair with a proof of stability of G. Its inversion theorem states exactly that a related pair has such a stable witness, with equalities to the constructed redex and reduct. The forward proof eliminates the rule constructor; the converse substitutes the two equalities and applies that constructor. There is no premise assuming the resulting programs equivalent or the machine correct.

Lean: `Allocation.before`

Lean: `Allocation.cellPrefix`

Lean: `Allocation.after`

Lean: `Allocation.Rule`

Lean: `Allocation.rule_iff`

80.2 Fresh allocation and exact store resolution

The instance, contraction, weakening and store are explicit edge coordinates in the nested serial assembly. Every original edge of G is injected into the other component of the outer serial assembly. Injectivity follows from the tagged carrier representation. The fresh store is in the prefix component, so it cannot equal any retained argument edge. Original labels are unchanged, and the designated instance and store have their required primitive labels. Freshness is therefore a property of disjoint graph occurrences, independent of any numerical choice of heap addresses.

Lean: `Allocation.instanceEdge`

Lean: `Allocation.contraction`

Lean: `Allocation.weakening`

Lean: `Allocation.store`

Lean: `Allocation.oldEdge`

Lean: `Allocation.oldEdge_injective`

Lean: `Allocation.store_fresh`

Lean: `Allocation.old_label`

Lean: `Allocation.instance_label`

Lean: `Allocation.store_label`

The instance target is the input of the canonical atom contraction. The contraction target is the new store input. These two adjacency witnesses, together with their exact primitive labels, give a two-hop atom trace. Its endpoint has the store label, so it is a successful resolution. The previously proved functional atom-tracing theorem implies that any successful trace starting at this instance has that same store and exactly two hops. After allocation resets focus to search, the instance rule returns backtrack at the external input in one step. This last result is about search, whereas the two-hop trace is the separate store-matching relation; their costs must not be added together as machine transitions.

Lean: `Allocation.instance_hop`

Lean: `Allocation.contraction_hop`

Lean: `Allocation.resolves`

Lean: `Allocation.resolves_exact`

Lean: `Allocation.returns`

80.3 Renaming and forward padding preservation

A recursive isomorphism between two argument graphs extends through the redex by identity on its `ref` head, and through the reduct by identity on its allocated prefix. Stability transports along the same argument isomorphism, yielding the corresponding raw allocation rule. Thus the stable premise is preserved, rather than silently discarded by a representation change.

One recursive padding step leaves the canonical atom tree unchanged except for its allowance index. The prefix padding comparison follows by applying serial-padding comparison twice, then the primitive and canonical-tree padding equalities. The redex comparison uses one serial-padding comparison and the primitive equality. The reduct comparison uses serial padding and the prefix comparison. Stability under padding supplies a valid raw allocation at the higher allowance.

Lean: `Allocation.before_congr`

Lean: `Allocation.after_congr`

Lean: `Allocation.renamed_rule`

Lean: `Allocation.raise_tree`

Lean: Allocation.prefix_raise

Lean: Allocation.before_raise

Lean: Allocation.after_raise

Lean: Allocation.raised_rule

An allocation up to isomorphism is witnessed by a stable argument and recursive isomorphisms from its two given graphs to the literal redex and reduct. Every raw rule supplies identity isomorphisms. Further source and target renamings compose inversely with the witnesses, proving representation congruence. To pad such a rule, pad the argument and both witness isomorphisms, then compose with the redex and reduct padding comparisons. Induction gives preservation under any finite number of raises.

These are forward operational padding results for allocation pairs. They do not yet reflect all possible matches from a higher allowance, or prove the full execution-padding interface. In particular, no padding property is inferred solely from determinism.

Lean: Allocation.UpToIso

Lean: Allocation.rule_upToIso

Lean: Allocation.upToIso_congr

Lean: Allocation.upToIso_raise

Lean: Allocation.upToIso_pad

80.4 Preserved contents and non-aliasing in larger graphs

An atom hop inside an inserted graph remains a hop after shallow plugging: map its shared vertex along the inserted-vertex map and use the mapped source and target memberships. Labels are unchanged. Mapping a finite trace proves resolution at the same hop count. Comparing that known resolution with any other successful trace in the filled host, functional terminal-trace uniqueness forces the same inserted store and count. It follows that the inserted reference cannot resolve to a retained host edge, because the inserted and retained edge tags differ. No acyclicity of the host is assumed; the claim applies equally when the host contains cyclic store structures. It is a resolution-locality result, not a completed full-machine specimen law.

Lean: StoreTrace.hop_insert

Lean: StoreTrace.resolves_insert

Lean: StoreTrace.resolves_insert_exact

Lean: StoreTrace.inserted_not_host

The old-vertex map is the right attachment map of serial composition. Every old edge retains its source and target lists mapped along this function. The new store's target list is precisely the singleton containing the mapped old input. Thus allocation keeps the argument graph and connects the store to that argument, rather than to an unspecified value. Exact resolution and freshness exclude successful lookup at any old argument edge. Two independently allocated

graphs placed in parallel also have distinct store occurrences, because their parallel-component tags differ.

Lean: `Allocation.oldVertex`

Lean: `Allocation.old_source`

Lean: `Allocation.old_target`

Lean: `Allocation.stored_input`

Lean: `Allocation.not_old_store`

Lean: `Allocation.parallel_stores_distinct`

80.5 Safety and the scope of stability

For any primitive predicate P, the allocation prefix satisfies P whenever P holds of instance, atom weakening, atom contraction and store. Prove this by serial assembly of the primitive corollas and case analysis on the canonical tree's edge label. Appending an argument satisfying P preserves that global property. Specialising to exclusion of focus labels and exclusion of hole labels proves preservation of focus freedom and hole freedom. A stable argument supplies both properties, so its allocated graph has both.

The whole allocated graph is not stable in the paper's sense: its designated store edge would have to have a stable primitive label, but stores are not stable primitives. Injectivity of primitive labels turns this into a contradiction. The rule's stability condition concerns its preserved argument, not the heap containing that argument. This distinction prevents an incorrect whole-graph stability invariant from being assumed for future store rules.

Lean: `Allocation.prefix_labels`

Lean: `Allocation.after_labels`

Lean: `Allocation.after_focusFree`

Lean: `Allocation.after_holeFree`

Lean: `Allocation.stable_allocation_safe`

Lean: `Allocation.after_not_stable`

80.6 A graph-changing subsystem and the original translated execution

A state is a dependent pair of an actual graph and a token coordinate in that graph. The subsystem transition relation has three constructors: an intrinsic search step on a fixed graph; a rooted allocation of a stable argument from rewrite mode to search mode; and that allocation with an unchanged serial continuation. Both allocation constructors keep the stable premise. The continuation constructor retains its whole graph, including the original translation's single-use distributors.

Successful finality is backtrack mode at an external input. Search has no step there by its proved input obstruction; allocation requires rewrite mode, so cannot apply. These cases prove final normality and supply a genuine abstract-machine instance for the subsystem. Determinism of this graph-changing relation is not established by this construction and is not

assumed in the regression executions below. An allocation followed by any serial continuation returns its new instance in one search step: use the actual retained instance edge and its singleton source at the overall input.

Lean: `Allocation.State`

Lean: `Allocation.Step`

Lean: `Allocation.completed_normal`

Lean: `Allocation.machine`

Lean: `Allocation.returns_suffix`

The constant argument is the unchanged translation of the natural literal n in the empty environment. Its two zero-resource distributor components have no edges; the remaining constant edge is stable. Eliminating those empty components proves the global stable-label criterion. The reference graph is the unchanged translation of `ref` n. The allocated graph replaces its local reference prefix while retaining the same single-use environment distributors. Head and value edges are their actual finite assembly coordinates, and the argument vertex is the head's target port.

Lean: `Allocation.constantArgument`

Lean: `Allocation.constantArgument_stable`

Lean: `Allocation.referenceGraph`

Lean: `Allocation.allocatedGraph`

Lean: `Allocation.referenceHead`

Lean: `Allocation.referenceValue`

Lean: `Allocation.referenceArgument`

The first search interaction enters the reference operation and searches its argument. The constant returns backtrack at that argument vertex. Returning through the unary operation then changes to rewrite mode at the root. Each proof uses the actual edge, label and ordered incidence list. Append the stable allocation-with-continuation step and the instance-return step to obtain an exactly five-transition machine run. Its endpoint satisfies successful finality. This is existence of the displayed five-step execution, not a theorem that every subsystem execution has that length.

The use of empty environments here is a concrete semantic regression. It does not restrict the general local allocation theorem, and it does not replace the required arbitrary-environment Proposition 12.2 endpoint.

Lean: `Allocation.reference_enter`

Lean: `Allocation.reference_value`

Lean: `Allocation.reference_ready`

Lean: `Allocation.reference_run`

Lean: `Allocation.reference_completes`

80.7 Checking the returned reference and its stored literal

An atom hop in the left component of serial composition is preserved by mapping its shared vertex through the left attachment map. Labels are unchanged; induction transports exact resolution traces. This lifts the local allocation resolution to the actual post-execution graph with its untouched distributors. The allocated instance and store names refer to these retained left-component occurrences. The resulting reference resolves to that store in two atom hops; any alternative successful resolution agrees on both store and count.

Lean: `StoreTrace.hop_serial_left`

Lean: `StoreTrace.resolves_serial_left`

Lean: `Allocation.allocatedInstance`

Lean: `Allocation.allocatedStore`

Lean: `Allocation.allocated_resolves`

Lean: `Allocation.allocated_resolves_exact`

The allocated value occurrence is the original constant edge retained through both serial assemblies. Its label is the literal n, and the target list of the resolved store equals the source list of this value edge, by the actual incidence definitions. Instantiating the execution and resolution results at $n = 1$ gives the requested example's allocation subexpression: a five-transition run, a fresh reference resolving to its store, and that store attached to the original literal one. Final normality rules out further subsystem transitions after the reference has returned.

This does not yet run the enclosing let, copy or invoke the lambda, or execute a dereference. Those operations and their template proof obligations remain on the route to Proposition 12.2.

Lean: `Allocation.allocatedValue`

Lean: `Allocation.allocated_value_label`

Lean: `Allocation.allocated_store_value`

Lean: `Allocation.ref_one`

Lean: `Allocation.reference_final_normal`

Validation checks all 304 build jobs and all three axiom audits: six abstract, 477 graph and 460 copying declarations. Only standard Lean logical axioms occur; there are no project-specific axioms or unfinished proofs. The estimated completion of Proposition 12.2 is roughly 55–60% by mathematical effort, with a broad 40–65% uncertainty range (26 October 2026). This is a judgement about the remaining mathematics, not a declaration count. The largest unknowns remain copying/matching, combined-machine laws, and the Table 3 template robustness proofs.

81 Dereferencing through arbitrary atom trees

Figure 19(b) of the paper (PDF p. 38) now has a concrete local dereference rule for an arbitrary atom contraction tree with $h + 1$ inputs, for every $h \geq 0$. One input belongs to the instance being read; the other h inputs remain available to other references. Both rule sides are actual finite typed hypernets, not a heap lookup substituted for the paper's graph rule.

The computation keeps the store and the atom tree, weakens the consumed instance, and inserts $D^{\star}_{1,2}$ between the store and its contents. Search then requests copying at that term distributor. It does not yet return the stored value. Exact store resolution, preservation of other reference inputs, renaming, forward padding, safety, and a five-transition local search/read execution are proved.

The complete allocation/application/read execution of Proposition 12.2 is still pending. In particular, combining the local allocation and read rules with arbitrary focused contexts, proving global rule matching and determinism, and supplying the ensuing copy transitions remain necessary. The local read execution below does not claim to be a run of the original translated entire program. The terms and all Figure 16 translation clauses remain unchanged.

81.1 Finite wiring frames with exposed reference inputs

The common input list is one term wire followed by h atom wires; the output is one term wire. A named tree hole expects $h + 1$ atom inputs and one atom output. A second, differently named hole expects two term inputs and one term output. The first will receive the preserved atom tree, the second the new term distributor. The redex edge tags are reader, instance, tree and store; the reduct edge tags are discard, instance, tree, store and share. These are finite enumeration types, not semantic operations.

Lean: `Dereference.inputs`

Lean: `Dereference.treeHole`

Lean: `Dereference.shareHole`

Lean: `Dereference.BeforeEdge`

Lean: `Dereference.AfterEdge`

The boundary-index helper embeds an index of $h + 1$ into the length-indexed input interface without changing its numerical value. The other-reference list consists of input positions $i + 1$, in order, for $0 \leq i < h$. Its no-duplicates proof composes injectivity of the input constructor with injectivity of successor. Membership in that list is equivalent to being one of these successor-indexed inputs, by the finite-range map definition.

Lean: `Dereference.inputPort`

Lean: `Dereference.otherInputs`

Lean: `Dereference.otherInputs_nodup`

Lean: `Dereference.mem_otherInputs`

In the redex frame, the reader consumes term input zero and produces the instance's term input. The instance produces the first atom-tree input; the remaining atom-tree inputs are exactly the external successor positions. The tree output enters the store, whose term output

is the sole external output. The frame has three internal vertices and four edges. Their label, source and target functions explicitly encode this wiring.

The net constructor proves source/target lists duplicate-free, uniqueness of the incident edge at each side of a vertex, coverage of all non-boundary positions, and equality of incidence types with each primitive signature. Finite edge case analysis proves uniqueness. Coverage splits internal positions into their three cases and input positions into zero or successor. Typing of the variable-length atom list follows from its constant atom labels. These are proved fields of the actual graph, not external well-formedness assumptions.

Lean: `Dereference.beforeLabel`

Lean: `Dereference.beforeSource`

Lean: `Dereference.beforeTarget`

Lean: `Dereference.beforeFrame`

The reduct replaces the reader by term weakening feeding the old instance. The instance, atom tree and store remain connected in the same order. The store's term output and external term input zero feed the two inputs of the new share hole, in that order. Its output is the external term output. This frame has four internal vertices and five edges. Its typing and incidence proofs repeat the explicit finite checks, with the store-result wire and share hole as the additional cases. The other reference inputs are still attached to the old atom-tree positions.

Lean: `Dereference.afterLabel`

Lean: `Dereference.afterSource`

Lean: `Dereference.afterTarget`

Lean: `Dereference.afterFrame`

81.2 Filling the frames with the actual trees

Given a contraction tree T of atom type with $h+1$ inputs, the redex filling puts its actual net in the tree slot and a primitive corolla at each other edge. The reduct filling puts the same T in its tree slot, the original canonical tree $D^{\star}_{1,2}$ in the share slot, and the designated primitive corollas at the other edges. Simultaneous expansion removes the holes and connects these components by the proved frame incidences. Relabelling primitives into allowance d yields both copying hypernets. In particular, no term weakening or contraction in the canonical tree is omitted.

The raw local rule consists of these two constructed graphs, for any T and d. Its inversion theorem recovers a tree witness and equalities with the two sides by elimination of the rule constructor; the converse substitutes those equalities and constructs the rule. Unlike allocation, dereferencing does not replace or evaluate its external stored contents, so this local fragment has no stable-value parameter. The subsequent copying operation has its own copyability obligation.

Lean: `Dereference.beforeFill`

Lean: `Dereference.afterFill`

Lean: `Dereference.before`

Lean: Dereference.after

Lean: Dereference.Rule

Lean: Dereference.rule_iff

The instance and store occurrences on each side have explicit expanded edge tags. The tree injections retain every original tree edge under its component tag. Their label equation is literal preservation of the old tree label, lifted to the copying alphabet. One allowance raise of either rule side equals constructing that same side at allowance $d + 1$; this follows by cases on the allowance, since every edge is primitive. The rule therefore survives a raise by rewriting these two equalities.

The first search step on the reduct meets the second contraction of $D^{\star}_{1,2}$ at external term input zero. The exact source membership and term-contraction label give a rewrite token at that same vertex. This is the required copy request.

Lean: Dereference.beforeInstance

Lean: Dereference.afterInstance

Lean: Dereference.beforeStore

Lean: Dereference.afterStore

Lean: Dereference.beforeTree

Lean: Dereference.afterTree

Lean: Dereference.tree_labels

Lean: Dereference.before_raise

Lean: Dereference.after_raise

Lean: Dereference.rule_raise

Lean: Dereference.requests_copy

81.3 Preservation of resolved store identity and trace length

Each side has the actual expansion attachment map from tree vertices into the surrounding fragment. If two tree edges are adjacent, map their shared vertex through this attachment. Mapped target/source membership proves adjacency in the fragment. The tree's weakening-or-contraction label condition makes this an atom hop. The argument applies separately to the redex and reduct with identical original tree edges.

The instance enters whichever tree edge consumes input zero: its target and that inserted input are the same frame vertex. At the other end, the unique tree root targets the store's input. Root-output membership supplies the shared vertex, and the root's permitted label supplies the atom-hop condition. Thus the two boundary hops are proved from the actual graph construction.

Lean: Dereference.beforeTreeVertex

Lean: Dereference.afterTreeVertex

Lean: Dereference.tree_hop_before

Lean: Dereference.tree_hop_after

Lean: Dereference.enters_before

Lean: Dereference.enters_after

Lean: Dereference.root_before

Lean: Dereference.root_after

Appending one relation step to an n-step trace gives an $(n+1)$-step trace, by induction on the original trace. The arbitrary-tree theorem supplies an edge consuming input zero and an n-step trace from that edge to its root. Map this trace into each rule side, append the root-to-store hop, and prepend the instance-to-tree hop. Both sides resolve to their designated store in $n+2$ atom hops.

This proves existence without choosing a canonical tree presentation. Given any proposed successful resolution at cost m on either side, deterministic atom tracing and terminal-store uniqueness identify m with the common cost above. Consequently the two sides admit successful resolution at exactly the same counts. The same uniqueness theorem excludes every other endpoint store. These counts measure atom-trace witnesses, not UAM execution steps. This does not yet prove uniqueness of all possible context matches in a whole machine state, nor identify non-isomorphic tree presentations as graphs.

Lean: Dereference.trace_snoc

Lean: Dereference.resolves_both

Lean: Dereference.resolution_count_iff

Lean: Dereference.store_unique_before

Lean: Dereference.store_unique_after

81.4 The local execution and its pending copy

The reader's argument vertex is the first internal frame vertex. Entering the actual reader edge searches this vertex; entering the actual instance returns backtrack there. Returning through the unary reader then changes to rewrite mode at external term input zero. All three search witnesses carry their exact edge labels and ordered incidence positions.

Lean: Dereference.argument

Lean: Dereference.search_argument

Lean: Dereference.instance_returns

Lean: Dereference.ready

A state pairs an actual graph at the local read interface with a token coordinate in that graph. The subsystem relation consists of intrinsic search and the raw read computation from rewrite mode on the redex to search mode on the reduct. Successful finality remains backtrack at an external input. Such states have no search successor by the input obstruction and no read successor because the read constructor requires rewrite mode. This proves final normality for the subsystem machine.

Concatenate the three search transitions, the single read transition, and the proved term-contraction search transition. The result is a five-step execution ending in rewrite mode on the reduct. Its finality predicate is false, because rewrite and backtrack modes differ. No claim of combined-machine determinism or value return is hidden in this execution theorem.

Lean: Dereference.State

Lean: Dereference.Step

Lean: Dereference.final_normal

Lean: Dereference.machine

Lean: Dereference.read_run

Lean: Dereference.read_pending_copy

Two further incidence equalities check the intended read action: the old instance source is now the target of the discard weakening; and the new term-contraction source list consists of the store output followed by the external result input. For each other reference $i < h$, the tree's input $i + 1$ attaches to external atom position $i + 1$ on both sides. Unfold the expansion's input attachment and the ordered finite-range list to prove these equations. Thus reading neither reconnects another reference to a different input nor consumes it.

Lean: Dereference.instance_discarded

Lean: Dereference.shared_store_result

Lean: Dereference.before_other_input

Lean: Dereference.after_other_input

81.5 Carrier renaming, safety, and retained contents

An equality-labelled isomorphism of primitive nets lifts to a recursive copying isomorphism at every allowance. Keep its carrier bijections and ordered incidence equations; label equality followed by reflexivity of recursive label equivalence proves the label field. To rename a matched tree, assemble its net isomorphism at the tree component and identity isomorphisms everywhere else. Simultaneous-expansion congruence supplies the redex and reduct isomorphisms, and the lifting construction promotes them to allowance d. Both maps send the designated store occurrence to the designated store occurrence by definition. Iterating the raw one-step padding theorem proves forward padding of local rules by any finite allowance increment.

Lean: Dereference.liftIso

Lean: Dereference.beforeBaseIso

Lean: Dereference.afterBaseIso

Lean: Dereference.before_rename

Lean: Dereference.after_rename

Lean: Dereference.rename_store

Lean: Dereference.rule_pad

A primitive label satisfies a recursively lifted predicate precisely when the primitive itself satisfies it, by allowance cases. Hence the redex has any global predicate true of reader,

instance, store, atom weakening and atom contraction. The reduct has any global predicate true of instance, store and both sorts of weakening and contraction. Case analysis on the expanded edge component proves these claims; tree components use the contraction-tree label condition or the canonical-tree label definition. Specialise to exclusion of focus labels and hole labels to prove both rule sides focus-free and hole-free.

Lean: Dereference.label_primitive

Lean: Dereference.before_labels

Lean: Dereference.after_labels

Lean: Dereference.focusFree_sides

Lean: Dereference.holeFree_sides

Appending any graph $G : \star \Rightarrow O$ to the reduct gives the read fragment with actual contents. Each edge in G retains its label through the right serial injection. The output of the new term distributor is attached exactly to the old input of G, by the actual serial incidence equation. No assumption that G is acyclic or already evaluated is used here. The known local resolution trace persists in the left component of serial assembly; comparing it with any successful trace in the complete graph forces that trace to end at the retained store. These are contents and lookup laws, not a completed context-closure theorem for read computations.

Lean: Dereference.withContent

Lean: Dereference.content_label

Lean: Dereference.content_attachment

Lean: Dereference.content_store_unique

81.6 Canonical and noncanonical regression instances

For the canonical atom tree with one input, its root is contraction edge one, as witnessed by its output incidence and uniqueness of the output producer. The instance enters this same edge, so the empty internal tree trace plus the two boundary hops gives resolution in exactly two hops. Resolution-count preservation gives the same result after reading.

For a noncanonical example, use the weakening-free binary contraction tree. Its sole edge is both entry and root, giving two-hop resolution with one additional exposed reference. The other-reference theorem keeps that reference at external atom position one, and the general execution theorem supplies the five-step read trace for this noncanonical match.

Lean: Dereference.canonical_resolves_before

Lean: Dereference.canonical_resolves_after

Lean: Dereference.binary_resolves_before

Lean: Dereference.binary_other_reference

Lean: Dereference.binary_read_run

Finally attach the original empty-environment translation of literal one to the canonical reduct. Its named retained value edge still has the constant one label, and its source equals the term distributor's output target list. The canonical term distributor has all three expected edges:

term weakening and both term contractions. These equalities are checked on the actual graph construction. They establish preserved contents and the real copy handoff; they do not supply the pending copying execution or a complete execution of !(`ref` 1) from its term translation.

Lean: `Dereference.storedOne`

Lean: `Dereference.storedOneValue`

Lean: `Dereference.stored_one_label`

Lean: `Dereference.stored_one_attachment`

Lean: `Dereference.canonical_copy_edges`

Validation checks all 311 build jobs. The three axiom audits check six abstract, 477 graph and 489 copying declarations and use only standard Lean logical axioms; no unfinished proofs or project-specific axioms occur. Estimated completion of Proposition 12.2 is about 60% by mathematical effort, with broad uncertainty of 45–70% (26 October 2026). This judgement concerns the remaining mathematics. Copying, integration and matching of the complete machine, and the template robustness proofs remain substantial obligations.

82 The literal copying rule and preservation of shared stores

Figure 17 (PDF p. 37) now has a concrete finite-graph implementation, for an arbitrary term contraction tree C with $n+1$ inputs and an arbitrary copyable hypernet $H : \star \Rightarrow \star^k \otimes \diamond^h$. The selected occurrence is input zero; the other n inputs keep their order. The rewrite duplicates H, retains C behind the second copy, weakens the consumed input of C, and inserts the paper's two output distributors. Neither the proposition's terms nor the translation clauses are changed.

This milestone proves typing and finiteness by constructing actual nets, recursive safety, disjointness of copied edge identities, preservation of literal box contents, carrier-renaming laws and forward padding laws. Local copy-and-return executions and shared-reference resolution are checked. It does not yet prove independence from different contraction-tree matches in one ambient graph, arbitrary focused-context matching, or determinism and refocusing of the complete copying UAM. Those remain in Appendix B.1.

82.1 Retaining the matched contraction tree

The residual tree is the original matched tree with its first input closed by a term weakening. Its interface is $\star^n \Rightarrow \star$. The construction uses a two-edge frame: a weakening and a tree hole of signature $\star^{n+1} \Rightarrow \star$. One internal term vertex connects the weakening output to input zero of the hole. The remaining hole inputs are the external inputs in increasing order; its output is the external output. The label and incidence functions specify this wiring directly.

Lean: `CopyRule.treeHole`

Lean: `CopyRule.residualLabel`

Lean: `CopyRule.residualSource`

Lean: `CopyRule.residualTarget`

The frame is a finite typed net with a Boolean edge carrier and one internal vertex. Its constructor checks each source and target for duplicate freedom, unique incidence, complete coverage and agreement with the primitive signature. The two edge cases discharge uniqueness; coverage splits vertices into the single internal vertex, external inputs and external output. The ordered finite input range gives both the typing equation and input injectivity.

Lean: `CopyRule.residualFrame`

Filling the tree slot with C and the other slot with the weakening corolla produces the residual graph. Relabelling primitive graphs into allowance d produces both the original tree and the residual tree at that allowance; it does not alter any graph incidence. The two edge injections distinguish the new weakening from every original tree edge. Their label theorem follows by unfolding expansion. Increasing the allowance commutes literally with both primitive-only constructions, by the two depth cases.

Lean: `CopyRule.residualFill`

Lean: `CopyRule.tree`

Lean: `CopyRule.residual`

Lean: `CopyRule.residualWeakening`

Lean: `CopyRule.residualTreeEdge`

Lean: CopyRule.residual_labels

Lean: CopyRule.tree_raise

Lean: CopyRule.residual_raise

The tree-vertex map is the expansion insertion map. The selected input is exactly the weakening's target, not an erased wire. Original input $i+1$ becomes external input i; this follows by indexing the explicit finite input list. Every original source and target list is mapped entrywise by the same insertion map. Thus the entire ordered incidence of C survives, not merely its number of leaves or its output type.

Lean: CopyRule.residualTreeVertex

Lean: CopyRule.selected_input_weakened

Lean: CopyRule.residual_input_order

Lean: CopyRule.residual_sources

Lean: CopyRule.residual_targets

82.2 The two rule graphs and their side condition

Write $R(C)$ for the residual tree. Before copying the graph is $C;H$. After copying it is

$$(H \otimes (R(C);H));\chi;(D^{\star}_{k,2} \otimes D^{\diamond}_{h,2}),$$

where χ is the middle-block exchange of Figure 17. Each copy of H has term outputs followed by atom outputs; the exchange groups the two term blocks before the two atom blocks. It uses the same ordered output bijection as the original binary translation. The external input order is the active input followed by the n residual inputs. Interface equality transport changes only length indices and preserves edge and internal carriers. The full canonical distributors, including their weakening edges, remain.

Lean: CopyRule.pairBoundary

Lean: CopyRule.copies

Lean: CopyRule.before

Lean: CopyRule.after

The intrinsic rule relates these two concrete graphs only when H is copyable in Definition 10.2's existing formalisation. This includes instances, constants, active operations and lambda payloads. It does not replace copyability by stability. The rule-characterisation theorem says precisely that any rule witness supplies such a tree and copyable payload, with the two graph equalities. Its proof eliminates or constructs the sole rule constructor. The transition is separate from the linear compute relation.

Lean: CopyRule.Rule

Lean: CopyRule.rule_iff

The left and right edge injections use different Boolean component tags. Their labels are exactly the original labels, proved by unfolding the constructors at zero and successor allowance.

The injections have disjoint images: equality would force the distinct Boolean tags to be equal. In particular a box label, including its complete body, is preserved exactly.

Lean: CopyRule.leftEdge

Lean: CopyRule.rightEdge

Lean: CopyRule.copy_labels

Lean: CopyRule.copies_disjoint

82.3 Safety throughout nested boxes

Let P be a primitive-label predicate. If P holds on term weakening and contraction, it holds throughout the tree and its residual. For the original tree this is the tree's label classification; for the residual, split an edge into the inserted weakening or an original tree edge. If H also satisfies P recursively, serial assembly proves safety of the redex. For the reduct, require P on weakening and contraction of both sorts. Split parallel components, use residual safety for the second component, then use the distributor and serial-assembly label laws. No assumption that box bodies are passive is introduced.

Lean: CopyRule.tree_labels

Lean: CopyRule.residual_allLabels

Lean: CopyRule.before_allLabels

Lean: CopyRule.after_allLabels

Taking P to exclude focus labels proves focus freedom of the reduct; taking it to exclude hole labels proves hole freedom. These are recursive properties, so they also exclude forbidden labels inside copied boxes. The frame's temporary tree hole has been filled and does not remain.

Lean: CopyRule.focusFree_after

Lean: CopyRule.holeFree_after

82.4 Local copy execution

A local state consists of a concrete graph at the common rule interface and a focus mode with a vertex. Its transition relation contains intrinsic search and the copy transition from rewrite focus at input zero to search focus at that same external input on the reduct. Finality means backtracking at an external input. A final state has no transition: search is impossible by search finality, while copying requires a different mode. This defines a search/copy subsystem with a proved final-normal law. It is not a claim that the entire copying UAM, including all compute rules and contexts, is finished.

Lean: CopyRule.State

Lean: CopyRule.Step

Lean: CopyRule.final_normal

Lean: CopyRule.machine

For an instance payload, the active copy has an instance edge at input zero, so its next search transition backtracks there. For any constant payload, the same argument uses the

zero-argument passive-operation search rule. The incidence calculation unfolds the parallel boundary and serial insertion; the active input remains exactly position zero. Composing one intrinsic copy transition and this return proves a two-step local execution in each case. These runs begin at rewrite focus, rather than claiming an unproved whole-program path from the original source term.

Lean: CopyRule.instance_returns

Lean: CopyRule.constant_returns

Lean: CopyRule.instance_run

Lean: CopyRule.constant_run

82.5 References share their original cell

To test reference copying, attach a single original store to the atom output of the copy reduct for an instance payload. This test is at allowance zero and allows any residual term tree. The two designated instance edges are nested left/right injections; the designated store is the right serial component. The atom distributor has the actual canonical two-use tree: a weakening, a lower contraction and its root contraction. Its edge selector chooses these three occurrences. Both instance labels remain instance labels, and the store label remains a store label.

Lean: CopyRule.sharedStore

Lean: CopyRule.sharedLeft

Lean: CopyRule.sharedRight

Lean: CopyRule.sharedCell

Lean: CopyRule.sharedContraction

Lean: CopyRule.shared_instance_labels

Lean: CopyRule.shared_cell_label

Both instance outputs enter the same lower atom contraction, at its first and second input respectively. That contraction feeds the root contraction, which feeds the single original store. Each hop proof supplies the actual shared target/source vertex and checks its ordered incidence membership. Composing these hops gives exactly three atom-routing steps from either copy to the same store. Store-trace uniqueness then rules out any different successful destination or hop count. The store is shared; copying has not allocated a new cell for either occurrence.

Lean: CopyRule.shared_left_hop

Lean: CopyRule.shared_right_hop

Lean: CopyRule.shared_root_hop

Lean: CopyRule.shared_store_hop

Lean: CopyRule.shared_resolves

Lean: CopyRule.shared_resolves_unique

82.6 Renaming and allowance transport

Boundary equality reindexing preserves a deep isomorphism, by eliminating the two interface equalities. It commutes with raising by the same argument. An ordered isomorphism between two matched tree carriers lifts to the copying hierarchy. For residual trees, use componentwise expansion isomorphism on the two-edge frame, with identity on the weakening. Parallel component isomorphisms then transport the two copies, and serial composition transports both complete rule sides. The payload is allowed a deep isomorphism, which also renames recursively inside boxes.

Lean: CopyRule.reboundaryIso

Lean: CopyRule.raise_reboundary

Lean: CopyRule.tree_congr

Lean: CopyRule.residual_congr

Lean: CopyRule.copies_congr

Lean: CopyRule.before_congr

Lean: CopyRule.after_congr

Raising commutes with the parallel constructor and boundary reindexing. The residual serial component commutes up to the already proved serial padding isomorphism; the residual primitive graph itself raises literally. These facts prove padding for the paired copies. Serial padding gives the redex law, and the paired-copy law followed by distributor padding gives the reduct law. These are explicit operational construction proofs, not an inference from determinism.

Lean: CopyRule.copies_raise

Lean: CopyRule.before_raise

Lean: CopyRule.after_raise

The representation closure of a copy pair consists of a tree and a copyable payload, together with deep isomorphisms from the supplied two graphs to the literal redex and reduct. Every raw rule supplies such a witness. Independent renamings of its two sides preserve this closure by inverse composition. Raising transports the two witnesses using the preceding padding laws and the proved preservation of copyability. Induction gives every finite padding amount. This is forward preservation; no new converse execution theorem is being claimed for the combined UAM.

Lean: CopyRule.UpToIso

Lean: CopyRule.rule_upToIso

Lean: CopyRule.upToIso_congr

Lean: CopyRule.upToIso_raise

Lean: CopyRule.upToIso_pad

82.7 Operational regressions and remaining obligations

The lambda regression applies the rule to an arbitrary original lambda payload, with no stability restriction on its body. For every box-labelled edge, both copied labels equal the original box label by transitivity with the general label theorem. This includes nested boxes without translating or normalising their contents. A separate application regression checks that active application heads also satisfy copyability.

Lean: `CopyRule.lambda_rule`

Lean: `CopyRule.lambda_box_preserved`

Lean: `CopyRule.application_rule`

The weakening-free binary tree supplies a noncanonical matched tree. Its residual contains the new weakening even though the original tree had none. Copying a reference with this match gives two distinct instance edges that resolve to the same cell in three hops, by disjointness and the shared resolution theorem. These checks do not yet constitute repeated closure application or execution of a divergent callback in the complete language; those require the missing application rule and machine integration.

Lean: `CopyRule.binary_residual_weakening`

Lean: `CopyRule.binary_shared_reference`

The build checks 318 jobs. The axiom audits check 6 abstract, 477 graph and 528 copying results and use only the standard Lean logical axioms. Estimated completion of Proposition 12.2 is about 63% by mathematical effort, with broad uncertainty of 48–73% (26 October 2026). The copying construction is now proved, but matching independence, the remaining compute rules, full-machine integration and the eleven template robustness proofs remain substantial. Appendix B.1 removes the completed local copy construction from its pending list while retaining these exact obligations.

83 Stable assignment with preserved store identity

Figure 19(c) (PDF p. 38) now has a concrete local assignment rule. The matched atom contraction tree is arbitrary, the new value is an arbitrary stable hypernet, and all other atom-reference inputs remain available. The rule updates the selected store's term connection, weakens the consumed instance and old contents, and returns unit. It neither allocates a fresh store nor deletes the old heap. This matters for Proposition 12.2 because its arbitrary callback belongs to the full language and may use assignment, even though the displayed terms contain no assignment themselves.

The development proves exact preservation of store-resolution costs, ordered boundary preservation, safety, renaming and recursive padding. Seven-step local executions are checked, including a payload using the unchanged Figure 16 literal translation. The local rule is not yet integrated into arbitrary focused-context matching or the complete copying UAM. In particular, no whole-program assignment execution or callback equivalence is claimed here. The proposition's terms and translation remain unchanged.

83.1 Interfaces and finite assignment frames

Let $C : \diamond^{h+1} \Rightarrow \diamond$ be the matched atom tree and $G : \star \Rightarrow O$ the new-value graph. The fragment's input list is one term input followed by h atom inputs. Its output list is one term output, for the old contents, followed by O, preserving the new value's external interface. The assigning instance supplies input zero of C; the other inputs of C remain external. Connections from the value's atom outputs back into that tree can be made by a surrounding graph; these fragment interfaces expose those cuts rather than assuming the references are distinct cells. Such arbitrary ambient matching remains a separate integration obligation.

The frame contains a tree hole and a value hole with exactly the signatures of C and G. Its redex edge tags are assignment, instance, tree, store and value. The reduct adds separate tags for discarding the instance, discarding the old contents, and returning unit, while retaining instance, tree, store and value tags.

Lean: `Assignment.inputs`

Lean: `Assignment.outputs`

Lean: `Assignment.treeHole`

Lean: `Assignment.valueHole`

Lean: `Assignment.BeforeEdge`

Lean: `Assignment.AfterEdge`

Input positions are embedded by their numerical indices. The other-reference list is the increasing list of successor positions. Value outputs are the successor-indexed output positions, leaving output zero for the old contents. Both lists are duplicate-free by injectivity of the relevant port constructor and successor. Their membership lemmas follow from finite-range mapping: a vertex belongs precisely when it has the corresponding indexed form.

Lean: `Assignment.inputPort`

Lean: `Assignment.otherInputs`

Lean: `Assignment.outputPort`

Lean: `Assignment.valueOutputs`

Lean: `Assignment.otherInputs_nodup`

Lean: `Assignment.mem_otherInputs`

Lean: `Assignment.valueOutputs_nodup`

Lean: `Assignment.mem_valueOutputs`

The redex has four internal vertices. Two term vertices are the assignment arguments: the first feeds the instance, the second feeds G. The instance produces an atom vertex feeding the tree's first input; the other atom vertex connects the tree root to the store. The store's term output is external output zero. The value hole's outputs occupy the remaining external positions. The label, source and target functions specify every one of these incidences in order.

The finite-net constructor checks unique incidence, complete coverage, duplicate-free incidence lists and signature typing. Internal coverage is four-way finite case analysis. External coverage splits an index into zero or successor. Uniqueness separates the five edge tags and uses disjointness of zero and successor output positions. Mapping each boundary occurrence to its type gives the required input/output signature lists.

Lean: `Assignment.beforeLabel`

Lean: `Assignment.beforeSource`

Lean: `Assignment.beforeTarget`

Lean: `Assignment.beforeFrame`

In the reduct, the tree and instance keep their atom wiring. A new term weakening feeds the consumed instance. The store's output now feeds the new value at the second term vertex. A separate term weakening feeds old output zero. Unit consumes the external term input and has no outputs. The same four internal vertices suffice, and all O outputs keep their positions. The seven-edge constructor proves the same finite typing and incidence obligations, using the same zero/successor distinctions.

Lean: `Assignment.afterLabel`

Lean: `Assignment.afterSource`

Lean: `Assignment.afterTarget`

Lean: `Assignment.afterFrame`

83.2 The stable compute rule

Relabelling the primitive frames into allowance d produces two patterns. The filling functions replace the tree hole by C at that allowance and the value hole by G. Every other frame edge is filled by its original primitive corolla. Simultaneous expansion gives actual finite typed redex and reduct hypernets; no holes remain from the pattern itself. This uses G literally, without simplifying distributors inside a translated value.

Lean: `Assignment.beforePattern`

Lean: `Assignment.afterPattern`

Lean: `Assignment.beforeFill`

Lean: `Assignment.afterFill`

Lean: `Assignment.before`

Lean: `Assignment.after`

The local rule requires the existing full stability predicate on G, including its label, output-type and input-reachability conditions. No stability obligation is assumed of the entire heap. Rule inversion exposes exactly a matched tree, a stable value, and the equalities to these two concrete graph constructions. The proof is elimination and construction of the single assignment-rule constructor.

Lean: `Assignment.Rule`

Lean: `Assignment.rule_iff`

Designated edge injections locate the original instance, store and each new-value edge on both sides. Their labels are preserved exactly, by unfolding expansion at zero and successor allowance. In particular, the store occurrence remains a store and each value edge keeps its original operation or box label.

Lean: `Assignment.beforeInstance`

Lean: `Assignment.afterInstance`

Lean: `Assignment.beforeStore`

Lean: `Assignment.afterStore`

Lean: `Assignment.beforeValue`

Lean: `Assignment.afterValue`

Lean: `Assignment.value_labels`

Lean: `Assignment.store_labels`

83.3 Exact preservation of store resolution

The tree-edge injections and vertex insertion maps locate the preserved atom tree inside either rule graph. Every tree label is its original primitive label lifted to the allowance. An adjacent pair of tree edges therefore yields an atom-routing hop on either side: the tree's label classification establishes a forwarding edge, and mapping the shared vertex proves adjacency in the expanded net.

Lean: `Assignment.beforeTree`

Lean: `Assignment.afterTree`

Lean: `Assignment.beforeTreeVertex`

Lean: `Assignment.afterTreeVertex`

Lean: `Assignment.tree_labels`

Lean: `Assignment.tree_hop_before`

Lean: `Assignment.tree_hop_after`

The instance enters input zero of the same tree on both sides. Given an edge incident to that input, its adjacency with the instance follows from the explicit frame attachment. At the other end, the tree's root output is attached to the designated store input. These entry and exit lemmas hold for arbitrary contraction-tree presentations, not only canonical ones.

Lean: `Assignment.enters_before`

Lean: `Assignment.enters_after`

Lean: `Assignment.root_before`

Lean: `Assignment.root_after`

The contraction-tree reachability theorem supplies an input-zero edge and a path of some length m from that edge to the root. Map this path through the two tree embeddings, prepend the instance-entry hop and append the root-to-store hop. This gives resolution to the designated store on both sides in exactly $m + 2$ hops. The general uniqueness theorem for store traces then proves equality of the successful resolution counts in both directions. It also proves that any successful resolution from the instance must end at the designated store, before or after assignment. Stability of the new value is not needed for these routing lemmas; the rule itself still requires it.

Lean: `Assignment.resolves_both`

Lean: `Assignment.resolution_count_iff`

Lean: `Assignment.store_unique_before`

Lean: `Assignment.store_unique_after`

83.4 What changes and what remains attached

The value-vertex insertion map places G in the reduct. The store target is exactly the image of G's term input. Every source and target list of G is mapped entrywise by this insertion, and output i of G remains external output $i + 1$. These equations follow from the expansion definition and the frame's ordered value-output list. They prove reconnection of the actual value graph, rather than an abstract update of a separate heap map.

Lean: `Assignment.afterValueVertex`

Lean: `Assignment.updated_content`

Lean: `Assignment.value_sources`

Lean: `Assignment.value_targets`

Lean: `Assignment.value_output_order`

Separate edge selectors identify the two new weakenings. The first has term-weakening label and feeds external old-content output zero. The other has the same label and feeds precisely the consumed instance's input. Neither weakening is erased. For each other reference input $i + 1$, the tree-vertex map before and after assignment is the same external input position. Thus the update changes a term connection while preserving the atom routing and other reference attachments.

Lean: `Assignment.discardOld`

Lean: `Assignment.discardInstance`

Lean: `Assignment.old_content_weakened`

Lean: `Assignment.instance_weakened`

Lean: `Assignment.other_references_preserved`

An arbitrary continuation may be attached to all fragment outputs, including the old contents at output zero. Serial composition preserves every edge label of that continuation. The already established successful atom trace also survives this attachment, so store-trace uniqueness prevents the updated instance from resolving to a different surrounding cell. This argument imposes no acyclicity condition on the continuation. It is a structural and routing preservation result, not yet a full contextual execution theorem for assignment.

Lean: `Assignment.withContents`

Lean: `Assignment.old_contents_retained`

Lean: `Assignment.attached_store_unique`

83.5 Search, update and unit return

The two argument coordinates are the first and second internal term vertices. Search first enters the instance argument. The instance returns by backtracking; the assignment head then resumes at the second, new-value argument. Once that argument returns, backtracking through assignment requests rewrite focus at the external term input. After the assignment rewrite resets search there, the unit edge returns immediately. These five individual interaction lemmas check the actual ordered argument positions.

Lean: `Assignment.instanceArgument`

Lean: `Assignment.valueArgument`

Lean: `Assignment.search_instance`

Lean: `Assignment.instance_returns`

Lean: `Assignment.search_value`

Lean: `Assignment.ready`

Lean: `Assignment.unit_returns`

A state keeps a concrete graph and its focus coordinate together. The local transition relation contains intrinsic search and the stable assignment compute rule. Finality is backtracking at an external input. Such a state cannot search further, by the established search theorem, and cannot assign, because assignment requires rewrite mode. This proves final normality for the subsystem machine. It does not assert global determinism of overlapping matches or supply the remaining language rules.

Lean: `Assignment.State`

Lean: `Assignment.Step`

Lean: `Assignment.final_normal`

Lean: `Assignment.machine`

Any constant corolla returns at the new-value argument in one search step. For such a value the complete local run has seven transitions: enter the instance, return from it, move to the value, return from it, request rewrite, perform assignment, and return unit. The run uses the proved stability of the constant, so no unproved operational premise is hidden in this example.

Lean: `Assignment.constant_returns`

Lean: `Assignment.constant_run`

83.6 Renaming, padding and safety

An ordered isomorphism of matched atom trees and a deep isomorphism of value graphs induce deep isomorphisms on both rule sides. Simultaneous expansion assembles identity isomorphisms on the primitive corollas, the lifted tree isomorphism in the tree slot, and the supplied value isomorphism in the value slot. Boundary order is unchanged.

Lean: `Assignment.before_congr`

Lean: `Assignment.after_congr`

Padding has an explicit graph-isomorphism proof for each side. The maps keep frame vertices fixed and map each component edge or internal vertex within its original component tag. Their inverses use the same tags. Labels agree under the recursive padding map. Incidence follows by composing list maps; the host-vertex lemma checks that padding commutes with attachment to each frame vertex. Empty weakening sources and the empty unit target are handled directly. The uniform relabelling presentation of padding is then identified with the existing recursive raise operation, yielding deep isomorphisms for the original assignment graphs.

Lean: `Assignment.before_raise_graph`

Lean: `Assignment.before_raise`

Lean: `Assignment.after_raise_graph`

Lean: `Assignment.after_raise`

Assignment up to representation isomorphism retains a witness tree and a stable value, together with deep isomorphisms to the literal redex and reduct. A raw rule gives this immediately. Raising transports the witnesses using the preceding two graph laws and the proved preservation of stability. Induction gives forward preservation for every finite padding amount. No determinism hypothesis is used to infer padding invariance.

Lean: `Assignment.UpToIso`

Lean: `Assignment.rule_upToIso`

Lean: `Assignment.upToIso_raise`

Lean: `Assignment.upToIso_pad`

For any primitive-label predicate P, redex safety follows from safety of G, classification of the tree's labels, and P on assignment, instance and store. Reduct safety additionally requires term weakening and unit. The proof splits the finite frame components and applies simultaneous expansion's label law. Excluding focus and hole labels gives both properties on both sides whenever G is stable. Conversely, the updated whole heap is not stable: its designated store label violates the stable-label condition. This prevents mistakenly treating the entire post-update graph as the stable argument required by a later compute rule.

Lean: `Assignment.before_labels`

Lean: `Assignment.after_labels`

Lean: `Assignment.stable_assignment_safe`

Lean: `Assignment.after_not_stable`

83.7 Translated payloads and concrete update regressions

The literal-edge selector identifies the natural-number edge in the original closed Figure 16 constant translation, including its retained empty distributors. That edge proves a one-step return at the value argument. Substituting this return and the already proved stability of the translated literal gives the same seven-step local assignment execution. Only the payload is claimed to be this exact translated term; the whole local rule fragment is not being presented as a completed translation of a source assignment context.

Lean: `Assignment.literalEdge`

Lean: `Assignment.translated_value_returns`

Lean: `Assignment.translated_value_run`

The updated-heap regression attaches the original translated old literal to the reduct's old-content output, while the new payload is the original translated new literal. Edge selectors locate the updated store, new literal, old literal and old-content weakening. Label equations establish which numerical value each edge represents. The store target equals the new literal's source, and the weakening target equals the old literal's source. The two literal occurrences are disjoint because they belong to different serial components, even when their numerical values coincide.

Lean: `Assignment.updatedHeap`

Lean: `Assignment.updatedCell`

Lean: `Assignment.updatedValue`

Lean: `Assignment.discardedValue`

Lean: `Assignment.oldWeakening`

Lean: `Assignment.updated_literal`

Lean: `Assignment.discarded_literal`

Lean: `Assignment.old_new_disjoint`

Instantiating with the weakening-free binary atom tree tests a noncanonical match with another exposed reference. Updating literal one to literal two has exactly the store-to-two and weakening-to-one connections just proved. This verifies concrete mutation while retaining the discarded graph for later garbage-collection reasoning.

Lean: `Assignment.one_to_two`

Validation checks 326 build jobs and the three axiom audits: 6 abstract, 477 graph and 576 copying results. Only standard Lean logical axioms occur. Estimated completion of Proposition 12.2 is about 65% by mathematical effort, with broad uncertainty of 50–75% (26 October 2026). Assignment construction is removed from Appendix B.1's pending compute rules. Atom equality, arithmetic, application, full-machine integration and template robustness remain, as do the final Figure 22 chain and semantic weakening argument.

84 Atom equality by resolved store identity

Both Figure 19(d) rules (PDF p. 38) now have concrete finite-graph implementations. Two instances attached to one atom contraction tree and store compare true. Instances attached to two separate trees and stores compare false. Both instances are weakened after the comparison, while all trees, stores and store-content interfaces remain. The trees and the numbers of other references are arbitrary.

The decisive semantic check is identity of the resolved store, not equality of stored contents. Successful lookups determine a unique boolean even in cyclic hosts, and inserting a fragment into a surrounding graph preserves that boolean. Each of the two local rules has a checked seven-step search/compute/return execution. Renaming, forward padding and recursive label safety are proved. These results complete the local constructions for all four Figure 19 store operations.

They do not yet prove determinism of the combined copying UAM, arbitrary focused-context matching, or the robustness of the templates. The lookup outcome uniqueness theorem below is narrower than uniqueness of an entire graph rewrite. Proposition 12.2 and its Figure 16 translation remain unchanged, and the full proposition is still unfinished.

84.1 The common-store frame

The common-store fragment has one external term input, h other atom inputs, and one term output for the contents of the shared store. Its tree slot has $h + 2$ atom inputs: the first two come from the two instances; the rest are the other references. The redex edge tags are comparison, left instance, right instance, tree and store. The reduct replaces the comparison by a true constant and adds a separate weakening for each instance.

Lean: `AtomEquality.Same.inputs`

Lean: `AtomEquality.Same.treeHole`

Lean: `AtomEquality.Same.BeforeEdge`

Lean: `AtomEquality.Same.AfterEdge`

The external input index is unchanged by the boundary embedding. The other-reference list consists of successor positions in increasing order. Injectivity of successor and the input constructor proves that this list has no duplicates; finite-range membership characterises its elements.

Lean: `AtomEquality.Same.inputPort`

Lean: `AtomEquality.Same.otherInputs`

Lean: `AtomEquality.Same.otherInputs_nodup`

Lean: `AtomEquality.Same.mem_otherInputs`

Five internal vertices carry the wiring: two term argument inputs, two atom outputs of the instances, and the atom connection between tree root and store. The comparison's ordered outputs feed the two term vertices. The tree's first two inputs receive the two atom vertices; its root feeds the store; the store's term output is the external output. Label, source and target functions spell out this graph. The constructor proves finiteness, complete vertex coverage,

unique incidence, duplicate-free lists and signature typing by finite case analysis and the indexed other-input list.

Lean: AtomEquality.Same.beforeLabel

Lean: AtomEquality.Same.beforeSource

Lean: AtomEquality.Same.beforeTarget

Lean: AtomEquality.Same.beforeFrame

On the reduct side, each consumed instance receives a term weakening at its original term input. The true constant consumes the external term input and has no output. The atom tree and store wiring are identical to the redex wiring. Seven finite edge cases establish the same coverage, uniqueness and typing obligations; no tree or store edge is discarded.

Lean: AtomEquality.Same.afterLabel

Lean: AtomEquality.Same.afterSource

Lean: AtomEquality.Same.afterTarget

Lean: AtomEquality.Same.afterFrame

84.2 The distinct-store frame

The distinct-store fragment has one external term input, $n + m$ other atom inputs, and two term outputs for the two stores' contents. Each tree slot has one selected instance input and its own other references: n on the left, m on the right. The two tree holes have different names. The edge types explicitly contain two store occurrences, in addition to two trees and two instances. Including these stores is essential: two separate reference paths alone would not prove that their eventual stores differ.

Lean: AtomEquality.Different.inputs

Lean: AtomEquality.Different.treeHole

Lean: AtomEquality.Different.BeforeEdge

Lean: AtomEquality.Different.AfterEdge

The left other-reference block occupies positions 1 through n; the right block occupies positions $n+1$ through $n+m$. The finite lists use these offsets explicitly. Their duplicate-freedom proofs reduce equality of port indices to equality of the corresponding finite indices. Membership lemmas expose the index witnesses, allowing the frame proof to show that the two blocks cannot overlap.

Lean: AtomEquality.Different.inputPort

Lean: AtomEquality.Different.leftInputs

Lean: AtomEquality.Different.rightInputs

Lean: AtomEquality.Different.leftInputs_nodup

Lean: AtomEquality.Different.rightInputs_nodup

Lean: AtomEquality.Different.mem_leftInputs

Lean: AtomEquality.Different.mem_rightInputs

The redex has two internal term vertices and four internal atom vertices. The two instance outputs enter their respective tree inputs; each tree root feeds its own store; the stores feed external term outputs zero and one. The other-reference blocks attach to the corresponding trees. The net proof checks coverage by separating external input zero, the left block and the right block. Source uniqueness uses the disjoint index intervals; target uniqueness separates the two store outputs. The remaining finite cases establish typing and all incidence requirements.

Lean: AtomEquality.Different.beforeLabel

Lean: AtomEquality.Different.beforeSource

Lean: AtomEquality.Different.beforeTarget

Lean: AtomEquality.Different.beforeFrame

The reduct retains those six internal vertices, both trees and both stores. It weakens each consumed instance and places a false constant at the external term input. The nine-edge frame proves the same interface and typing properties. The two output positions remain distinct, so the store identities cannot be merged merely by choosing equal contents.

Lean: AtomEquality.Different.afterLabel

Lean: AtomEquality.Different.afterSource

Lean: AtomEquality.Different.afterTarget

Lean: AtomEquality.Different.afterFrame

84.3 Literal rule graphs and depth allowances

For the common-store case, the filling functions put the supplied arbitrary atom contraction tree in its slot and fill every other edge with its original primitive corolla. Expansion followed by primitive relabelling produces the two actual hypernets at every allowance d. The rule has one constructor relating exactly those graphs. It requires an actual contraction tree, without any assumed machine determinism or observational equivalence.

Lean: AtomEquality.Same.beforeFill

Lean: AtomEquality.Same.before

Lean: AtomEquality.Same.afterFill

Lean: AtomEquality.Same.after

Lean: AtomEquality.Same.Rule

The designated left instance, right instance and store are explicit edge injections on each side. They provide the coordinates used by all lookup and execution proofs. Raising the allowance preserves each rule graph literally because all of its labels are primitive and the graph wiring is unchanged. Rule preservation follows by rewriting these equalities; induction gives every finite padding amount.

Lean: AtomEquality.Same.beforeLeft

Lean: AtomEquality.Same.beforeRight

Lean: AtomEquality.Same.beforeStore

Lean: AtomEquality.Same.afterLeft

Lean: AtomEquality.Same.afterRight

Lean: AtomEquality.Same.afterStore

Lean: AtomEquality.Same.before_raise

Lean: AtomEquality.Same.after_raise

Lean: AtomEquality.Same.rule_raise

Lean: AtomEquality.Same.rule_pad

The distinct-store filling functions put the two arbitrary trees into separate slots, retaining the other primitive corollas. The rule likewise relates the literal redex and reduct. Separate edge injections identify both instances and both stores before and after the rewrite. Its primitive construction has the same literal raising equalities and finite-padding preservation, again proved directly from the constructors.

Lean: AtomEquality.Different.beforeFill

Lean: AtomEquality.Different.before

Lean: AtomEquality.Different.afterFill

Lean: AtomEquality.Different.after

Lean: AtomEquality.Different.Rule

Lean: AtomEquality.Different.beforeLeft

Lean: AtomEquality.Different.beforeRight

Lean: AtomEquality.Different.beforeStore

Lean: AtomEquality.Different.beforeStoreRight

Lean: AtomEquality.Different.afterLeft

Lean: AtomEquality.Different.afterRight

Lean: AtomEquality.Different.afterStore

Lean: AtomEquality.Different.afterStoreRight

Lean: AtomEquality.Different.before_raise

Lean: AtomEquality.Different.after_raise

Lean: AtomEquality.Different.rule_raise

Lean: AtomEquality.Different.rule_pad

84.4 Resolving both references before and after comparison

The common path lemma concerns one contraction tree embedded into two actual hypernets. Suppose its adjacency steps, selected input attachment and root-to-store attachment are preserved in both embeddings, and the two designated destinations have store labels. Tree reachability supplies a path of length q from the selected input edge to the root. Map that same path into both graphs, prepend the instance hop and append the store hop. Both resolutions therefore have exactly $q + 2$ hops. The concrete applications below discharge every embedding premise; the local rules do not assume these routing properties.

Lean: AtomEquality.tree_resolution_pair

In the common-store case, the left and right tree-edge injections use the same tree slot, but the entry inputs are zero and one respectively. The vertex maps are expansion insertion maps. Adjacent tree edges become atom hops because their labels are weakening or contraction and their shared vertex is preserved. The instance-target calculation proves entry, and the tree-root output calculation proves exit to the shared store.

Lean: AtomEquality.Same.beforeLeftTree

Lean: AtomEquality.Same.beforeLeftVertex

Lean: AtomEquality.Same.afterLeftTree

Lean: AtomEquality.Same.afterLeftVertex

Lean: AtomEquality.Same.resolves_left

Lean: AtomEquality.Same.beforeRightTree

Lean: AtomEquality.Same.beforeRightVertex

Lean: AtomEquality.Same.afterRightTree

Lean: AtomEquality.Same.afterRightVertex

Lean: AtomEquality.Same.resolves_right

The resulting existence theorems preserve each reference's own exact resolution count across the rewrite. They do not assert that the left and right references have equal counts. For any proposed count, uniqueness of successful store traces identifies it with the constructed count; this proves the two exact-count equivalences.

Lean: AtomEquality.Same.left_resolution_count_iff

Lean: AtomEquality.Same.right_resolution_count_iff

In the distinct-store case, the left edge and vertex maps use the first tree slot and the right maps use the second. Each selected entry is input zero of its own tree. The two root outputs attach to their own designated stores. The same adjacency, entry and exit calculations discharge the common path lemma for each reference separately.

Lean: AtomEquality.Different.beforeLeftTree

Lean: AtomEquality.Different.beforeLeftVertex

Lean: AtomEquality.Different.afterLeftTree

Lean: AtomEquality.Different.afterLeftVertex

Lean: AtomEquality.Different.resolves_left

Lean: AtomEquality.Different.beforeRightTree

Lean: AtomEquality.Different.beforeRightVertex

Lean: AtomEquality.Different.afterRightTree

Lean: AtomEquality.Different.afterRightVertex

Lean: AtomEquality.Different.resolves_right

The resulting existence theorems preserve each reference's own exact resolution count across the rewrite. They do not assert that the left and right references have equal counts. For any proposed count, uniqueness of successful store traces identifies it with the constructed count; this proves the two exact-count equivalences.

Lean: AtomEquality.Different.left_resolution_count_iff

Lean: AtomEquality.Different.right_resolution_count_iff

84.5 Store identity determines a unique boolean

For a graph G and selected edges a, b, a justified comparison outcome r consists of successful store traces $a \rightsquigarrow s$ and $b \rightsquigarrow t$ with their individual lengths, together with $r = \mathsf{true}$ if and only if $s = t$. The definition compares the destination edges themselves. It never inspects their term outputs or stored contents.

Two such witnesses must have the same left store and the same right store, by uniqueness of successful store resolution. Substituting these identities leaves the same truth condition for both booleans, so Boolean case analysis proves their equality. This result needs no global acyclicity and no assumption that the full UAM is deterministic.

Lean: AtomEquality.Outcome

Lean: AtomEquality.outcome_unique

Inserting a matched fragment into any correctly typed hole preserves both successful traces with their exact lengths. Store occurrences are mapped by the inserted-edge injection, which preserves and reflects equality. Consequently the comparison outcome is preserved. Applying outcome uniqueness in the resulting host graph excludes any contrary successful outcome obtained by a different routing witness. This proves locality of the identity decision; it does not by itself prove contextual closure or uniqueness of all graph rewrites.

Lean: AtomEquality.outcome_insert

Lean: AtomEquality.inserted_outcome_unique

In the common-store construction, the two resolution theorems have the same destination edge, on both rule sides. They therefore justify true. A false outcome is impossible by uniqueness of the comparison result.

Lean: AtomEquality.Same.true_before

Lean: AtomEquality.Same.true_after

Lean: AtomEquality.Same.cannot_return_false

In the distinct-store construction, equality of the designated stores would force two distinct frame tags to be equal. This contradiction proves store distinctness before and after the rule. Together with the two successful resolutions it justifies false on both sides. A true outcome is therefore impossible, regardless of the contents attached to the stores.

Lean: AtomEquality.Different.stores_distinct_before

Lean: AtomEquality.Different.stores_distinct_after

Lean: AtomEquality.Different.false_before

Lean: AtomEquality.Different.false_after

Lean: AtomEquality.Different.cannot_return_true

84.6 The search and equality subsystem

A state consists of its actual graph and token position. One common local transition relation contains intrinsic search and both equality rewrites, with the interfaces fixed by their respective fragments. Each rewrite starts in rewrite mode at the external term input and resets search at that input in the reduct. Finality is backtracking at an external input. The search finality theorem excludes another search transition there; both equality constructors require a different focus mode. Thus final normality is proved for this subsystem machine.

Lean: AtomEquality.State

Lean: AtomEquality.Step

Lean: AtomEquality.final_normal

Lean: AtomEquality.machine

For the common-store case, the argument coordinates are the two internal term vertices. Search enters the left instance, returns from it, resumes at the right instance, and returns from it. Backtracking through the comparison head then requests rewriting. The result-edge selector identifies the literal true constant in the reduct, and its label theorem checks that value directly. Searching that constant returns at the external input in one step. Composing the five pre-compute search steps, the equality rewrite and the final return gives exactly seven transitions. These are local fragment executions; a complete execution of a translated source equality expression in an arbitrary context is not claimed here.

Lean: AtomEquality.Same.leftArgument

Lean: AtomEquality.Same.rightArgument

Lean: AtomEquality.Same.search_left

Lean: AtomEquality.Same.left_returns

Lean: AtomEquality.Same.right_returns

Lean: AtomEquality.Same.search_right

Lean: AtomEquality.Same.ready

Lean: AtomEquality.Same.resultEdge

Lean: AtomEquality.Same.result_label

Lean: AtomEquality.Same.result_returns

Lean: AtomEquality.Same.comparison_run

For the distinct-store case, the argument coordinates are the two internal term vertices. Search enters the left instance, returns from it, resumes at the right instance, and returns from it. Backtracking through the comparison head then requests rewriting. The result-edge selector identifies the literal false constant in the reduct, and its label theorem checks that value directly. Searching that constant returns at the external input in one step. Composing the five pre-compute search steps, the equality rewrite and the final return gives exactly seven transitions. These are local fragment executions; a complete execution of a translated source equality expression in an arbitrary context is not claimed here.

Lean: AtomEquality.Different.leftArgument

Lean: AtomEquality.Different.rightArgument

Lean: AtomEquality.Different.search_left

Lean: AtomEquality.Different.left_returns

Lean: AtomEquality.Different.right_returns

Lean: AtomEquality.Different.search_right

Lean: AtomEquality.Different.ready

Lean: AtomEquality.Different.resultEdge

Lean: AtomEquality.Different.result_label

Lean: AtomEquality.Different.result_returns

Lean: AtomEquality.Different.comparison_run

84.7 Representation and boundary preservation

For the common-store construction, ordered tree-carrier isomorphisms extend to both rule sides. Componentwise expansion uses the supplied isomorphism in each tree slot and identity on every primitive corolla. Primitive relabelling lifts that isomorphism to the required allowance. The two weakening selectors then prove explicitly that each consumed instance is fed by a term weakening at its original source.

Lean: AtomEquality.Same.before_congr

Lean: AtomEquality.Same.after_congr

Lean: AtomEquality.Same.discardLeft

Lean: AtomEquality.Same.left_weakened

Lean: AtomEquality.Same.discardRight

Lean: AtomEquality.Same.right_weakened

The single store target remains external output zero. Every other tree input $i + 2$ remains

external input $i+1$ before and after comparison, as shown by indexing the two-prefix source list. The stored content and all other-reference attachments therefore retain their ordered boundary positions.

Lean: AtomEquality.Same.store_boundary

Lean: AtomEquality.Same.other_references

For the distinct-store construction, ordered tree-carrier isomorphisms extend to both rule sides. Componentwise expansion uses the supplied isomorphism in each tree slot and identity on every primitive corolla. Primitive relabelling lifts that isomorphism to the required allowance. The two weakening selectors then prove explicitly that each consumed instance is fed by a term weakening at its original source.

Lean: AtomEquality.Different.before_congr

Lean: AtomEquality.Different.after_congr

Lean: AtomEquality.Different.discardLeft

Lean: AtomEquality.Different.left_weakened

Lean: AtomEquality.Different.discardRight

Lean: AtomEquality.Different.right_weakened

The two store targets remain external outputs zero and one respectively. Other left-tree input $i+1$ remains input $i+1$; other right-tree input $j+1$ remains input $n+j+1$. The finite-range source lists prove these four before/after equalities without permuting reference blocks.

Lean: AtomEquality.Different.store_boundaries

Lean: AtomEquality.Different.other_left_references

Lean: AtomEquality.Different.other_right_references

84.8 Safety and stored-content regressions

Recursive label safety for the common-store fragments follows by classifying the finite frame edges. Tree labels satisfy the chosen predicate whenever atom weakening and contraction do; the other edges require the appropriate comparison/result operation, instances, stores and term weakenings. Specialising the predicate to exclude focus labels, or hole labels, proves both rule sides focus-free and hole-free at every allowance. The temporary tree slots have been filled by actual graphs.

Lean: AtomEquality.Same.before_labels

Lean: AtomEquality.Same.after_labels

Lean: AtomEquality.Same.focusFree_sides

Lean: AtomEquality.Same.holeFree_sides

Recursive label safety for the distinct-store fragments follows by classifying the finite frame edges. Tree labels satisfy the chosen predicate whenever atom weakening and contraction do; the other edges require the appropriate comparison/result operation, instances, stores and term weakenings. Specialising the predicate to exclude focus labels, or hole labels, proves both

rule sides focus-free and hole-free at every allowance. The temporary tree slots have been filled by actual graphs.

Lean: AtomEquality.Different.before_labels

Lean: AtomEquality.Different.after_labels

Lean: AtomEquality.Different.focusFree_sides

Lean: AtomEquality.Different.holeFree_sides

Serially attaching arbitrary contents preserves a justified outcome. Both successful traces lift into the left serial component; equality of store edges is preserved and reflected by its injection. The common-store case thus compares true for arbitrary contents. A weakening-free binary tree supplies a checked noncanonical seven-step true execution.

Lean: AtomEquality.outcome_serial_left

Lean: AtomEquality.Same.arbitrary_contents

Lean: AtomEquality.Same.noncanonical_run

The distinct-store case compares false for arbitrary two-input contents. No surrounding content graph can identify the designated stores in a successful comparison: a hypothetical true outcome contradicts outcome uniqueness. The theorem does not require the contents to terminate or the surrounding graph to be acyclic. It is a routing theorem, not an assertion that arbitrary callbacks have already been executed in the full language.

Lean: AtomEquality.Different.arbitrary_contents

Lean: AtomEquality.Different.contents_cannot_identify_stores

The final regression attaches two copies of the unchanged Figure 16 translation of the same natural numeral to the two stores. The heap and literal-edge selectors retain their actual distributors and component tags. Both literal labels denote the same numeral, and explicit incidence equalities connect the respective store targets to those literal sources. Nevertheless the reference comparison is false, by the arbitrary-contents theorem. This checks that equality has not been replaced by comparison of stored values.

Lean: AtomEquality.Different.identicalContents

Lean: AtomEquality.Different.equalValueHeap

Lean: AtomEquality.Different.equalValueLeft

Lean: AtomEquality.Different.equalValueRight

Lean: AtomEquality.Different.identical_literal_labels

Lean: AtomEquality.Different.identical_literal_attachments

Lean: AtomEquality.Different.identical_contents_compare_false

The full build checks 335 jobs. All three axiom audits pass: 6 abstract, 477 graph and 652 copying results, using only standard Lean logical axioms. Estimated completion of Proposition 12.2 is about 67% by mathematical effort, with broad uncertainty of 52–77% (26 October 2026). All four store operations now have local concrete rules. Arithmetic and application remain to be constructed, followed by the combined-machine laws, template robustness, semantic weakening and Figure 22 equivalence chain. Appendix B.1 retains those obligations and removes atom-equality construction from pending work.

85 Stable micro-beta application and exact opening costs

The local application rule of Figure 10(b) (PDF p. 18), with the stability condition imposed in PDF Definition 12.1, is now constructed for the copying alphabet. It removes the application and lambda heads, opens the lambda box, and attaches the stable argument to the body's first, bound output. Free variable and atom outputs retain their original order. Neither the body nor the argument is copied or substituted syntactically by this rule.

This is a central prerequisite for Proposition 12.2: its let and sequencing sugar and the callback all use application. The construction accepts every finite body environment and every correctly typed stable argument. It proves six transitions to the opened body, and $6 + n$ transitions when followed by n steps of body search. It does not assume that the body terminates.

The proposition's terms and the Figure 16 translation are unchanged. The regression bodies below are literal translations, including their weakening and contraction distributors. The local redex contains the lambda head and box; surrounding translation distributors belong to the eventual matching context. Executing every complete translated application in arbitrary contexts still requires full-machine integration. The present construction also does not prove the micro-beta template's robustness.

85.1 Connecting the bound output without changing free outputs

Let F and O be arbitrary ordered wire interfaces. A body fragment has interface $\star \Rightarrow \star \otimes F$, while its argument has interface $\star \Rightarrow O$. Their bound-output splice has interface $\star \Rightarrow F \otimes O$. Its two temporary holes name these fragments separately. The first external output block uses indices $i < |F|$; the second uses $|F| + j$ for $j < |O|$. Finite-range lists enumerate both blocks.

Lean: `Application.spliceHole`

Lean: `Application.freePort`

Lean: `Application.valuePort`

Lean: `Application.freePorts`

Lean: `Application.valuePorts`

The membership lemmas recover the corresponding index witness. Duplicate-freedom follows from injectivity of each index embedding: the first preserves the index and the second adds a fixed offset. These facts supply coverage and uniqueness obligations for the actual net.

Lean: `Application.mem_freePorts`

Lean: `Application.mem_valuePorts`

Lean: `Application.freePorts_nodup`

Lean: `Application.valuePorts_nodup`

There is exactly one internal frame vertex. The body consumes the external term input; its bound output targets this internal vertex, which is consumed by the argument. The body targets the entire first external block after its bound output, and the argument targets the second block. The net proof checks all incidence laws. The two output blocks cannot collide because one index is below $|F|$ and the other is at least $|F|$. Every output belongs to one block

by the same case split. Typing follows by indexing the left and right parts of the concatenated interface. No free identity wire is introduced.

Lean: `Application.spliceSource`

Lean: `Application.spliceTarget`

Lean: `Application.spliceFrame`

The primitive filling chooses the body or argument according to the hole tag. Relabelling the frame embeds it at the required copying allowance; the typed filling accounts for that alphabet. Simultaneous expansion then removes both holes and retains the actual component graphs. These four constructors separate the finite wiring proof from its alphabet-dependent instantiation.

Lean: `Application.spliceFill`

Lean: `Application.splicePattern`

Lean: `Application.spliceFilling`

Lean: `Application.splice`

85.2 The literal application rule

For a body $B : \star \Rightarrow \star^{k+1} \otimes \diamond^{h}$ at allowance d, the closure is the lambda corolla followed by the original lambda box. The application redex at allowance $d+1$ is the application corolla followed by the ordered parallel pair of this closure and an argument G. Opening the body raises its representation allowance to $d+1$ and exposes its first term output as the bound output. This preserves every edge and recursively pads each nested box; it performs no term substitution. Splicing that body with G gives the reduct.

Lean: `Application.closure`

Lean: `Application.before`

Lean: `Application.openBody`

Lean: `Application.after`

The rule has one constructor, with the full predicate $\mathsf{Stable}(G)$ as a premise. This is the paper's operational side condition, rather than an assumed observational equivalence or determinism property. Inversion of the constructor gives an equivalent existential description by the actual body, argument, stability proof and the two graph equalities. Conversely, those witnesses construct the rule.

Lean: `Application.Rule`

Lean: `Application.rule_iff`

The argument and opened-body edge injections use distinct component tags. Each injection preserves its entire label, including recursively typed box bodies. Injectivity follows from equality of dependent pairs with a fixed tag. Equality between a body edge and an argument edge would instead force the two Boolean tags to agree, which is impossible. Thus opening

retains both components without identifying, duplicating or relabelling the argument's reference occurrences.

Lean: Application.argumentEdge

Lean: Application.bodyEdge

Lean: Application.argument_label

Lean: Application.body_label

Lean: Application.argument_injective

Lean: Application.body_injective

Lean: Application.components_disjoint

85.3 Boundary and search locality

A stable primitive is an instance or passive operation. Case analysis on the complete operation alphabet shows that entering it immediately returns the token, independently of its outputs. Stability of a graph supplies an actual edge at the unique input, with such a label. This constructs its one-step return; existence is derived from the graph laws, not assumed as a machine property.

Lean: Search.enter_stable

Lean: Search.stable_return

A generic search-transport lemma covers any simultaneously expanded component. The interacting edge acquires its component tag. Source membership, ordered target lookup and the singleton source equation transport through the insertion map. The already-proved commutation of entry and backtracking with vertex maps then reconstructs the same search step. This argument works for arbitrary typed attachment maps, without postulating their injectivity.

Lean: Search.expand_insert

The two vertex maps are the actual body and argument insertion maps. The body input remains the external input. Body output zero and argument input zero become the same internal vertex. Every body output $i + 1$ becomes external output i, and every argument output j becomes external output $|F| + j$. These are explicit equalities of vertices, proved by indexing the frame's incidence lists. They include every free variable and atom output.

Lean: Application.bodyVertex

Lean: Application.argumentVertex

Lean: Application.body_input

Lean: Application.bound_connection

Lean: Application.free_outputs

Lean: Application.argument_outputs

Specialising generic expansion transport to the two tags preserves body and argument search respectively. Mapping each step of a finite body trace gives a trace of precisely the same length

after attachment. This is search locality; it does not yet assert locality for all copying and compute transitions of the complete machine.

Lean: Application.body_search

Lean: Application.argument_search

Lean: Application.body_trace

85.4 Recursive renaming and padding

Componentwise expansion extends body and argument isomorphisms to the splice. Ordered parallel assembly does the same for a tensor. A recursive body isomorphism extends through the original lambda-box cast and the lambda corolla, giving closure congruence. Tensor and serial congruence then prove redex congruence. Padding the body isomorphism and applying splice congruence prove reduct congruence. Stability transports across the argument isomorphism, so the renamed graph pair satisfies the raw rule.

Lean: Application.splice_congr

Lean: Application.tensor_congr

Lean: Application.closure_congr

Lean: Application.before_congr

Lean: Application.after_congr

Lean: Application.renamed_rule

Padding the splice is proved by an explicit finite-carrier isomorphism. It retains the frame vertex and each component tag, and uses the corresponding padded component vertex or edge. Inverse laws and labels hold by cases. For source and target lists, insertion commutes with that vertex map; composition of list maps proves the incidence equations. This yields a recursive isomorphism between the padded splice and the splice of padded components. It is an independently proved representation law, not a consequence of determinism.

Lean: Application.splice_raise_graph

Lean: Application.splice_raise

Existing serial, tensor and lambda-box padding laws establish closure and redex padding. The interface equality cast in the opened body commutes with raising, giving a literal equality there. Splice padding gives the reduct isomorphism. The stability-padding equivalence supplies the stable argument premise for the rule at the next allowance.

Lean: Application.closure_raise

Lean: Application.before_raise

Lean: Application.openBody_raise

Lean: Application.after_raise

Lean: Application.raised_rule

The rule up to recursive isomorphism records an actual body and stable argument, together with isomorphisms from both supplied sides to their raw rule graphs. Every raw rule supplies

reflexive isomorphisms. Renaming either side composes inverse isomorphisms with those witnesses. Raising both sides raises the body and argument witnesses, transports stability, and composes the newly proved redex and reduct padding isomorphisms. This establishes the operational representation closure required for later matching.

Lean: Application.UpToIso

Lean: Application.rule_upToIso

Lean: Application.upToIso_congr

Lean: Application.upToIso_raise

85.5 Exact execution costs

The local machine carries an actual graph and a token coordinate. Its steps are intrinsic search or a stable micro-beta rewrite at the external input. The rewrite resets search at the reduct's external input. A final state is backtracking at that external input. Search cannot continue there, and micro-beta requires rewrite focus, so final normality is proved. This machine is the application subsystem, not the complete copying UAM.

Lean: Application.State

Lean: Application.Step

Lean: Application.final_normal

Lean: Application.machine

Explicit selectors identify the application edge, lambda edge and original argument edges. The two argument coordinates are the ordered internal vertices created by serial composition. Search first enters the function, returns immediately from its lambda head, and then enters the argument. The stable-graph theorem supplies the argument's actual passive edge and return. Backtracking from the second argument requests rewriting at the application root.

Lean: Application.head

Lean: Application.lambdaEdge

Lean: Application.valueEdge

Lean: Application.argument

Lean: Application.search_function

Lean: Application.function_returns

Lean: Application.search_argument

Lean: Application.argument_returns

Lean: Application.ready

Composing these five transitions gives the pre-compute search trace. Search determinism and normality at rewrite focus imply that every normal search-only run from this initial state has exactly that endpoint and length. Appending the micro-beta compute step gives six transitions

to search at the opened body. There is no termination assumption on the body: in particular, search has not entered its box before this step.

Lean: `Application.search_trace`

Lean: `Application.search_exact`

Lean: `Application.opening_run`

Induction on a finite body-search trace lifts it to subsystem machine steps through body attachment, preserving its length. Appending those steps to the six-transition opening run gives the quantitative $6 + n$ theorem. If the body trace returns to its input in backtrack mode, that input maps to the external input of the reduct, so the resulting endpoint is final. These statements only compose body *search* steps; they do not silently claim execution of further copying or compute requests.

Lean: `Application.body_run`

Lean: `Application.opening_then_body`

Lean: `Application.returning_body`

85.6 Recursive safety and call-by-value restrictions

Any recursive primitive-label predicate satisfied by the body and lambda head holds in the closure. Requiring it also of the argument and application head proves it on the redex. The reduct needs only the body and argument hypotheses: padding and casting preserve the body's predicate, and expansion takes every label from one of the two components. Specialising to absence of focus and hole labels gives both safety properties on both sides, using the corresponding consequences of argument stability.

Lean: `Application.closure_labels`

Lean: `Application.before_labels`

Lean: `Application.after_labels`

Lean: `Application.safe_sides`

An active-operation corolla cannot be stable. If it were, its actual edge label would equal a stable primitive label. Injectivity of primitive labelling identifies that primitive with the active operation, forcing its kind to be passive as well as active, a contradiction. This proves a concrete negative side-condition check rather than admitting unevaluated arguments.

Lean: `Application.active_not_stable`

85.7 Unchanged-translation regressions and remaining work

Three actual Figure 16 translations provide the bodies: a constant with one unused bound variable, that bound variable itself, and a nested lambda whose own bound variable is used. Their original distributors remain. After opening, the constant returns at its literal edge; the variable enters its retained term contraction and requests copying; the nested lambda returns at its own lambda edge without opening the inner box. Each proof identifies the actual edge and checks its source incidence and search result.

Lean: `Application.Examples.constantBody`

Lean: `Application.Examples.variableBody`

Lean: `Application.Examples.nestedBody`

Lean: `Application.Examples.constant_body_returns`

Lean: `Application.Examples.variable_body_requests_copy`

Lean: `Application.Examples.nested_body_returns`

The body-attachment law transports each of those transitions to the corresponding reduct, for an arbitrary argument of the appropriate interface. Appending it to the general opening execution gives seven-step runs: the constant and nested lambda return, whereas the variable stops at a copy request. This last regression checks that micro-beta has not been replaced by immediate syntactic substitution.

Lean: `Application.Examples.constant_after_returns`

Lean: `Application.Examples.variable_after_requests_copy`

Lean: `Application.Examples.nested_after_returns`

Lean: `Application.Examples.constant_run`

Lean: `Application.Examples.variable_run`

Lean: `Application.Examples.nested_run`

Instantiating the constant-body run with an instance corolla checks a reference argument and retains its atom output. Conversely, the active application corolla fails stability even for an unused parameter. These checks do not establish the eventual contextual equivalence in the presence of divergent callbacks or repeated closure use; those require the combined machine and robustness results.

Lean: `Application.Examples.reference_argument_run`

Lean: `Application.Examples.unused_argument_still_must_be_stable`

The complete build checks 342 jobs, and all three axiom audits pass: 6 abstract, 477 graph and 748 copying declarations, using only standard Lean logical axioms. Appendix B.1 removes local application construction from the unfinished compute-rule list. Arithmetic construction, full context matching, combined determinism and refocusing, template robustness, semantic weakening and the Figure 22 chain remain. The mathematical-effort estimate is about 70%, with broad uncertainty of 55–80% (26 October 2026); this is a judgement about remaining proof difficulty, not a declaration-count percentage.

86 A combined contextual copying subsystem

The previously separate allocation, dereference, assignment, atom-equality, application and copying machines now execute within one transition system. Every proved local run embeds with precisely its original cost. Graph-changing transitions can occur in shallow typed contexts, including an argument of an active operation. A concrete allowance-indexed execution instance transports all finite runs under recursive graph isomorphism. Its execution-isomorphism law, previously an abstract premise, is proved; the existing graph-context congruence theorem therefore applies to this instance without that premise.

This is a subsystem milestone on the route to Proposition 12.2. The original terms, Figure 16 translation, stable-argument premises and copyability premises are unchanged. Section 87 adds arithmetic and forward exact-cost padding to this same machine. Matching independence, deterministic behaviour up to representation, refocusing, focused-graph correspondence, reverse padding transport and specimen locality are not proved for this combined system. The named representation relation admits isomorphic presentations of each rewrite; this is an explicit semantic closure, not a theorem asserting unique raw graph matches. The final proposition and all template-robustness obligations remain open.

86.1 Local rules and shallow matching

A local rule records two actual finite hypernets with the same ordered interface and one active coordinate on each side. Its constructors are reference creation, dereference, assignment, same-atom equality, different-atom equality, micro-beta and copying, together with the unary and binary arithmetic constructors added in Section 87. Creation, assignment and micro-beta retain stability of the supplied value; copying retains copyability and its arbitrary residual contraction tree. The two equality constructors retain their separate finite graphs. The coordinates are their original term inputs. No constructor assumes termination or the desired equivalence.

Lean: `Combined.Local`

The rewrite relation is the least relation generated from these local pairs by three further constructors. Recursive isomorphisms may rename both sides and their active vertices independently. Typed replacement may put both sides in the same selected edge slot of a host graph. Simultaneous expansion may change one selected component while every other filling remains equal. The selected host edge is erased by replacement: its label supplies the slot's signature and is not asserted to be an observable hole label. These constructors give explicit witnesses for a match; they do not implement a unique-match algorithm.

All insertions take place at the outer graph level. There is no constructor that evaluates inside an unopened box. A recursive isomorphism may rename vertices inside a box, but it preserves the box hierarchy and does not move the token into its body. Box opening is the actual micro-beta rule. Later binding-free template arguments must still justify their own restrictions; shallow closure alone does not prove template robustness.

Lean: `Combined.Rewrite`

At fixed allowance d and interfaces I, O, a state is a dependent pair of an actual hypernet and a search position on that graph. A transition is either the existing intrinsic search relation on one unchanged graph, or a rewrite from rewrite focus at the source coordinate to search focus at the target coordinate. Finality is backtracking at an external input. Search cannot continue there, and a rewrite requires a different focus mode; case analysis therefore proves

final normality and supplies a machine.

Lean: Combined.State

Lean: Combined.Step

Lean: Combined.final_normal

Lean: Combined.machine

Every local rule becomes a transition by the local-match constructor followed by the rewrite-step constructor. Conversely, rewrite focus has no intrinsic search successor, by the previously checked search theorem. Thus copying and compute are explicit graph-changing transitions; neither has been put into the older linear search or compute parameter.

Lean: Combined.local_step

Lean: Combined.rewrite_not_search

86.2 State isomorphisms and exact costs

A state isomorphism consists of a recursive graph isomorphism, equality of focus modes and equality of the target token coordinate with the image of the source coordinate. Reflexivity uses identity bijections. Symmetry uses the inverse vertex map. Transitivity composes graph isomorphisms and vertex maps; the port-map composition equation follows by the three cases of a port. These maps fix ordered external inputs and outputs. Consequently a token is at an input before transport exactly when it is at an input after transport, proving equivalence of finality.

Lean: Combined.StateIso

Lean: Combined.StateIso.refl

Lean: Combined.StateIso.symm

Lean: Combined.StateIso.trans

Lean: Combined.StateIso.final_iff

One-step transport is proved by cases on the transition. Search uses the established incidence transport through the induced boundary isomorphism. For a rewrite, the explicit renaming closure changes its source presentation; the old target itself can be retained, with the reflexive target isomorphism. In both cases there is one target transition and an isomorphism relating endpoints. This proof uses the chosen rewrite relation's representation closure; it does not prove that arbitrary redex decompositions are unique.

Lean: Combined.step_transport

Induction on a run transports one step and then the remainder using the newly obtained endpoint isomorphism. No stuttering steps are inserted, so the length stays exactly n. Successful termination at cost n means existence of a final endpoint after n transitions. Run transport and finality transport prove the forward implication; the inverse state isomorphism proves the converse. No determinism assumption is used.

Lean: Combined.run_transport

Lean: Combined.TerminatesIn

Lean: Combined.terminatesIn_iso

86.3 Changing graphs inside an active context

Plugging a state inserts both its graph and its token into the same typed host slot. The two mode components agree by construction. An intrinsic search step lifts by the established insertion theorem; a rewrite lifts by the corresponding rewrite constructor. Induction extends this to all finite runs with the exact same length. This theorem concerns transitions, not preservation of finality at an arbitrarily attached boundary.

Lean: `Combined.plugState`

Lean: `Combined.step_plug`

Lean: `Combined.run_plug`

Serial state insertion separately handles a changing left component followed by a fixed continuation, and a fixed left component followed by a changing right component. Each construction maps the token with the actual serial insertion map. A left external input remains an external input; the proof computes the source port of the left frame hole. One-step lifting uses search transport or simultaneous rewrite expansion with the other component unchanged. The unchanged-component proof is a case split on the two hole tags. Run lifting then follows by induction.

Lean: `Combined.serialLeft`

Lean: `Combined.serialRight`

Lean: `Combined.left_input`

Lean: `Combined.step_serialLeft`

Lean: `Combined.step_serialRight`

Lean: `Combined.run_serialLeft`

Lean: `Combined.run_serialRight`

A right-component input becomes an internal connection of the serial frame. Backtracking there is therefore not final for the whole graph. The proof computes the right hole's source port and contradicts its alleged external-input status. A left-component input instead remains external, so backtracking there is final. These two results prevent local completion from being incorrectly promoted to program termination.

Lean: `Combined.right_return_not_final`

Lean: `Combined.left_return_final`

86.4 An unbounded execution instance and exact representation

The unbounded state type is the dependent union over all natural allowances of closed-interface states. A transition lifts a fixed-allowance transition; it does not add padding steps. Finality is inherited from that component, and normality follows by inversion of a lifted step. This defines a concrete machine and a concrete execution instance. Initialization retains the supplied closed graph literally and places search at its unique external term input.

Here "unbounded" means there is no global upper bound on permitted finite allowances. It does not mean that different padded representations have been identified. Unlike the completed linear development, this subsystem is not yet quotiented by padding; forward exact-cost transport is now proved in Section 87, while reflection of runs from the larger allowance remains unfinished.

Lean: `Combined.UnboundedState`

Lean: `Combined.UnboundedStep`

Lean: `Combined.unbounded_final_normal`

Lean: `Combined.unboundedMachine`

Lean: `Combined.execution`

Each transition preserves its allowance by construction. Transitivity of equality proves the same for a run. Conversely, every fixed-allowance run lifts one transition at a time. Reflection is proved by induction on an unbounded run: inversion of its first lifted transition identifies the source allowance and state, and the induction hypothesis reflects the remaining run. The dependent endpoint equality records the resulting bounded state. For endpoints already presented at the same allowance, eliminating that equality gives an exact equivalence of the two finite-run judgements at length n. Thus the wrapper introduces neither extra runs nor extra costs.

Lean: `Combined.step_depth`

Lean: `Combined.run_depth`

Lean: `Combined.run_unbounded`

Lean: `Combined.run_bounded`

Lean: `Combined.run_unbounded_iff`

The representation relation on unbounded states lifts a state isomorphism at the same allowance. It is symmetric, and related states have equivalent finality. Inverting this relation and an unbounded step reduces one-step transport to the fixed-allowance theorem. Induction transports unbounded runs with their exact lengths and related endpoints.

Lean: `Combined.Represents`

Lean: `Combined.represents_symm`

Lean: `Combined.represents_final`

Lean: `Combined.unbounded_step_transport`

Lean: `Combined.unbounded_run_transport`

For any cost preorder Q, a successful source run of length n transports to a successful target run of the same length. Reflexivity $Q(n, n)$ supplies the quantitative comparison. Applying

this reasoning in both directions proves machine equivalence for every cost preorder, including exact equality of costs. Initial states of isomorphic closed graphs have equal search modes and the same external input, so they are related. This discharges the execution-isomorphism interface law for this concrete subsystem.

Lean: `Combined.represents_refines`

Lean: `Combined.represents_equiv`

Lean: `Combined.execution_respectsIso`

The existing graph-context congruence theorem now specializes to the concrete execution. Its proof uses the previously established composition of observation contexts and equality up to graph isomorphism; the new execution-isomorphism theorem supplies the formerly abstract premise. The result preserves any already proved contextual equivalence under an admissible graph context, with the universal cost relation. It does not assert that any new program pair is contextually equivalent, or discharge BPerm's operational robustness. It also does not remove the independent padding premise from the observation-reflection theorem.

Lean: `Combined.equiv_context`

86.5 Every existing local execution embeds

Each old machine step is mapped to the corresponding combined step. For allocation, ordinary search and the raw rule map directly. Its additional suffix-allocation constructor is handled by left serial lifting, using the proved equality for the external input. Induction on a complete allocation run preserves its length and literal endpoint graphs.

Lean: `Combined.allocation_step`

Lean: `Combined.allocation_run`

Dereference and assignment use their existing intrinsic search constructors or their corresponding local-rule constructors. Assignment's stability witness is retained. In each case induction on a run yields a combined run with exactly the same endpoints and cost.

Lean: `Combined.dereference_step`

Lean: `Combined.dereference_run`

Lean: `Combined.assignment_step`

Lean: `Combined.assignment_run`

The atom-equality embedding handles both same-identity and distinct- identity rules, retaining their contraction trees. Application retains the original box body, argument and stability witness. Copying retains the original residual tree, value and copyability witness. Each one-step case analysis extends by run induction, again without changing any costs. These embeddings are inclusions; they do not assert that the richer combined system has only the runs available in an isolated local machine.

Lean: `Combined.equality_step`

Lean: `Combined.equality_run`

Lean: `Combined.application_step`

Lean: Combined.application_run

Lean: Combined.copying_step

Lean: Combined.copying_run

86.6 Term-wire typing throughout execution

Both active coordinates of every local rule are term typed, by the literal interface definitions. Induction on a rewrite preserves this property: recursive isomorphisms preserve vertex labels; replacement and expansion insertion preserve the labels of their inserted coordinates. Case analysis then combines rewrite typing with the established intrinsic search endpoint theorem. Thus both endpoints of every combined transition are on term wires, including coordinates that have become internal.

Lean: Combined.local_endpoints_term

Lean: Combined.rewrite_endpoints_term

Lean: Combined.step_endpoints_term

A run preserves a term-typed token. The zero-length case uses its initial typing hypothesis; in the nonempty case the first step supplies typing at the intermediate state and induction handles the rest. Inversion of a lifted transition gives the unbounded one-step theorem. Finally, reflect any run from a closed program into its initial allowance and use the run theorem. The initial input has type term by the closed-program interface. This is an operational token invariant, not yet the stronger preservation of all stable/copyable graph predicates required by the paper.

Lean: Combined.run_term

Lean: Combined.unbounded_step_term

Lean: Combined.reachable_term

86.7 Concrete execution regressions

The reference program is the actual Figure 16 translation of `ref` n, with its distributors retained. Its safety proofs are the translation's existing focus-free and hole-free theorems. Embedding the previous allocation run and lifting the allowance gives a five-step successful execution. The final state explicitly witnesses termination. Any recursively isomorphic closed representation has exactly the same successful costs, by the concrete representation-equivalence theorem.

Lean: Combined.Examples.referenceProgram

Lean: Combined.Examples.reference_run

Lean: Combined.Examples.reference_terminates

Lean: Combined.Examples.reference_representation

A second regression places that unchanged translated allocation in the argument of an outer reference-operation corolla. The two serial graphs record its source and its allocated inner reduct, and the coordinate selector records the argument's input as inserted in that host. This

is a concrete assembled context test; no equality with the complete Figure 16 translation of a nested reference expression is asserted.

Lean: Combined.Examples.outerRef

Lean: Combined.Examples.nested

Lean: Combined.Examples.nestedAllocated

Lean: Combined.Examples.innerInput

The outer corolla first searches its argument. The general right serial run theorem performs the five-step allocation inside that argument. The returned token is then handled by the outer corolla's backtrack rule, which requests the outer rewrite. Concatenating these traces gives seven transitions. The intermediate returned argument is not final because its input is internal; the seven-step endpoint is not final because its mode is rewrite. Thus the integration checks actual graph change in a context and subsequent host search, without claiming the next allocation has run.

Lean: Combined.Examples.enter_inner

Lean: Combined.Examples.return_to_outer

Lean: Combined.Examples.nested_run

Lean: Combined.Examples.inner_return_not_final

Lean: Combined.Examples.nested_endpoint_not_final

The application regression uses a literal translated constant body, including its unused-parameter distributor, with a stable unit argument. The local rule's safety theorem supplies the closed-program certificates. Embedding its seven-step opening-and-return run gives an unbounded run and successful termination. This is the concrete application redex with an original translated body, not a claim that the entire translated application has already been matched through all surrounding distributors.

Lean: Combined.Examples.applicationProgram

Lean: Combined.Examples.application_run

Lean: Combined.Examples.application_terminates

At this integration milestone, the full build checked 349 jobs and the audits passed: 6 abstract, 477 graph and 830 copying declarations, with only standard Lean logical axioms. Appendix B.1 removes the completed non-arithmetic context integration and its execution-isomorphism premise, while retaining the full-machine extensions and independent padding law. No whole remaining package is declared complete. Estimated progress by mathematical effort is about 73%, with broad uncertainty of 58–83% (26 October 2026). The change reflects operational integration and concrete execution laws, not the number of new declarations; template robustness remains a substantial source of uncertainty.

87 Arithmetic and exact forward execution padding

All three arithmetic operations now have actual local graph rules and checked executions in the combined machine. Addition and subtraction evaluate both numeric arguments in order, perform one rewrite and return the numeric result in seven transitions. Unary minus takes five transitions. This completes the local rule families for the existing example alphabet; it does not prove unique matching, determinism or refocusing of the complete copying UAM.

The same milestone proves forward padding of every combined transition and every finite run, including their actual contextual matches. A run of length n gives a run of exactly length n after an unused allowance layer is added. Successful termination is preserved under arbitrarily many layers, and when both a closing context and its inserted program are padded. This proves one direction of the padding obligation. The converse, not supplied by these forward constructions, is proved in Section 88. Neither direction is inferred from determinism.

The terms of Proposition 12.2 and the Figure 16 translation are unchanged. The natural-number arithmetic convention below is explicit; no new numeric constant datatype or altered term translation is introduced.

87.1 Arithmetic meanings and literal local rules

The existing constants carry natural numbers. Binary arithmetic codes select addition or subtraction and map to the corresponding original operation label. Their evaluation functions are $m + n$ and $m \dot{-} n$, where $\dot{-}$ denotes natural subtraction truncated at zero. Unary minus is $0 \dot{-} n$. The project author clarified the intended equation $-x = 0 - x$ during this milestone; keeping the existing natural-number domain therefore makes its result zero for every n. This last fact is proved with Lean's natural-subtraction theorem, rather than built into the syntax. Figure 10(a) explicitly depicts binary arithmetic; the unary rule uses this clarified interpretation of Figure 13's unary operation.

Lean: `Arithmetic.BinaryCode`

Lean: `Arithmetic.BinaryCode.symbol`

Lean: `Arithmetic.BinaryCode.eval`

Lean: `Arithmetic.neg`

Lean: `Arithmetic.neg_eq_zero`

A number is the original constant corolla, with one term input and no outputs. The unary redex is a unary-minus corolla serially connected to that number. The binary redex is the chosen arithmetic corolla connected to the ordered tensor of two number corollas. Their reducts are single number corollas with the computed result. Thus the binary construction is the three-edge to one-edge rule of Figure 10(a). The rule relation has one unary constructor and one constructor parameterized by the two binary codes. It does not accept Boolean, unit, function or reference operands as numbers. Their separate language operations and search behaviour remain present.

Lean: `Arithmetic.number`

Lean: `Arithmetic.unaryBefore`

Lean: `Arithmetic.unaryAfter`

Lean: `Arithmetic.binaryBefore`

Lean: Arithmetic.binaryAfter

Lean: Arithmetic.Rule

A padded number corolla is literally the same primitive corolla at the next allowance. The established serial-padding isomorphism therefore gives unary-redex padding. Tensor padding and serial padding give binary-redex padding. Each result-padding equation is the number equation specialized to the computed result. The isomorphism form of number padding is obtained by rewriting that equality and taking the identity isomorphism. These facts supply the new arithmetic cases of combined operational padding below.

Lean: Arithmetic.number_raise

Lean: Arithmetic.unaryBefore_raise

Lean: Arithmetic.unaryAfter_raise

Lean: Arithmetic.binaryBefore_raise

Lean: Arithmetic.binaryAfter_raise

Lean: Arithmetic.number_raise_iso

Searching a number corolla returns immediately at the same external input. The proof supplies its actual edge, source membership and the primitive entry equation. A number is a stable constant, whose established stability consequences prove absence of focus and hole labels. This certifies its safety independently of any termination equivalence.

Lean: Arithmetic.number_returns

Lean: Arithmetic.safe_number

87.2 Ordered search and concrete computation

The unary head and numeric-value selectors name the two actual edges; the argument selector names their internal connection. Entry searches that connection. The numeric edge returns the token, and backtracking through the unary head requests rewriting at the external input. Each proof checks incidence and the relevant entry or resume function, separately for allowance zero and positive allowances where the primitive carrier presentations differ.

Lean: Arithmetic.unaryHead

Lean: Arithmetic.unaryValue

Lean: Arithmetic.unaryArgument

Lean: Arithmetic.unary_enter

Lean: Arithmetic.unary_value_returns

Lean: Arithmetic.unary_ready

The binary selectors name the operation head, either operand edge and either ordered internal argument coordinate. The five search steps are: enter the left argument, return from its number, resume into the right argument, return from its number, and request rewriting at the

root. The argument indices are zero and one, and their bounds are proved from the operation's arity. These are ordered incidence proofs, not an evaluation oracle.

Lean: `Arithmetic.binaryHead`

Lean: `Arithmetic.binaryValue`

Lean: `Arithmetic.binaryArgument`

Lean: `Arithmetic.binary_enter`

Lean: `Arithmetic.binary_left_returns`

Lean: `Arithmetic.binary_search_right`

Lean: `Arithmetic.binary_right_returns`

Lean: `Arithmetic.binary_ready`

Inversion of an arithmetic rule chooses its actual combined local constructor. The existing local-match constructor then supplies a combined rewrite from rewrite focus to search focus on the result. Appending that transition and the one-step number return to the three or five search steps gives the five- and seven-step executions. No standalone arithmetic execution parameter is assumed. Since these rules are in the original combined relation, its already-proved contextual lifting, recursive-isomorphism transport, finite-run representation and token-typing theorems now cover arithmetic too.

Lean: `Arithmetic.rule_step`

Lean: `Arithmetic.unary_run`

Lean: `Arithmetic.binary_run`

The input graphs satisfy the translation language's label predicate: every primitive is an operation, and serial and tensor composition preserve that predicate. Consequently the established language safety theorems yield focus-free and hole-free certificates. Evaluating the actual finite edge enumerations proves that a binary redex has three edges and its result one, while a unary redex has two edges and its result one. The proofs hold for all allowances and all natural operands.

Lean: `Arithmetic.unary_language`

Lean: `Arithmetic.binary_language`

Lean: `Arithmetic.binary_edge_counts`

Lean: `Arithmetic.unary_edge_counts`

87.3 Padding contextual attachments without moving the token

Uniform padding keeps each outer vertex and edge carrier and relabels its edges, recursively padding box bodies. For a simultaneously expanded host, the padded filling of an edge is the padded component with interface equality casts supplied by the label-padding laws. Those casts are necessary because the hole signature is indexed by the host edge's label. They change no port index or wire type.

Lean: `Padding.expandFilling`

The expansion attachment equation says that inserting the appropriately cast component port into the padded host reaches exactly the original vertex. For an internal port both sides are the same tagged component vertex. For an input or output, both sides use the same host source or target position. Case analysis on ports and the incidence-index definitions proves the equation. The expansion-padding isomorphism uses identity vertex and edge bijections; labels agree, and its source and target equations follow by composing list maps and applying that attachment equation to each port.

Lean: `Padding.expand_insert`

Lean: `Padding.expand_graph`

Single-slot replacement has the analogous attachment equation. The host embedding is unchanged and a cast inserted input or output reaches the same host boundary incidence. The corresponding padding isomorphism again uses identity outer bijections. Host-edge incidence commutes with the host embedding; inserted-edge incidence commutes by the attachment equation. The source filler requires no gratuitous boundary cast, which lets this isomorphism compare the actual graphs used by the rewrite constructor.

Lean: `Padding.plug_insert`

Lean: `Padding.plug_graph`

Uniform padding also supplies a concrete search-transport structure. Both port and edge bijections are identities. Case analysis on the allowance and original label proves that an edge has a specified primitive label before padding exactly when it has that same primitive label afterwards. Source and target lists are unchanged, and external-input status is unchanged. The previous search-transport theorem can therefore be reused without modifying intrinsic search.

Lean: `Padding.search`

87.4 Every combined rewrite and run pads forward

First, replacing ordered interfaces by equal lists transports a rewrite and casts its active coordinates. After eliminating the interface equalities, the proof constructs identity-carrier isomorphisms to the reboundaried graphs and invokes the existing representation closure. A separate helper observes that graph isomorphisms fix each indexed external input. Another helper expresses a previously proved rule-side padding isomorphism using uniform padding instead of the equivalent original raising function.

Lean: `Combined.rewrite_reboundary`

Lean: `Combined.rewrite_input_iso`

Lean: `Combined.graphIsoOfRaise`

Every local rule pads forward. For allocation, assignment, micro-beta and copying, the proof uses their actual redex and reduct padding isomorphisms. Stability transports by the proved stability-padding equivalence; copyability transports by its proved padding theorem. Dereference and the two atom-equality cases agree directly with their next-allowance primitive graphs. The unary and binary arithmetic cases use the new redex and number padding laws. All active ports are external term inputs at this local stage, so their coordinates agree by the input-isomorphism helper. No side condition is dropped or replaced by a machine-determinism assumption.

Lean: `Combined.local_padding`

A recursive graph isomorphism lifts through uniform padding with its original outer bijections. The recursive label-equivalence padding theorem supplies its only new field; all incidence and internal-label laws are unchanged. Induction on a contextual rewrite now handles every constructor. The local case is the theorem above. The renaming case uses padded isomorphisms. For replacement, cast the padded component's interfaces, rewrite in the padded host, and rename through the replacement-padding isomorphisms; the attachment equations identify the actual token coordinates. For simultaneous expansion, all unselected components remain equal after padding, and the corresponding expansion isomorphisms finish the same argument. This establishes forward transport for arbitrary finite shallow matches.

Lean: `Combined.graphIso`

Lean: `Combined.rewrite_padding`

Padding a state increases its allowance and pads its actual graph, while retaining its mode and port. Finality is therefore equivalent before and after this state transformation. A combined step lifts by cases: intrinsic search uses the uniform search transport, and a graph-changing step uses rewrite padding. Induction lifts every run to its literal padded endpoints, with unchanged length. A successful run retains its final endpoint, giving preservation of successful termination at each exact cost.

Lean: `Combined.padState`

Lean: `Combined.padState_final`

Lean: `Combined.step_padding`

Lean: `Combined.run_padding`

Lean: `Combined.terminatesIn_padding`

87.5 Unbounded programs and padded observations

The unbounded-state padding function adds one to its allowance and applies the fixed-allowance state transformation. Finality remains equivalent. Inverting an unbounded step and applying fixed-allowance step padding gives one lifted transition. Run induction gives the same number of unbounded transitions. For any cost preorder Q, successful source runs therefore witness refinement to the padded state using $Q(n, n)$. This is a forward refinement theorem, not a two-sided equivalence.

Lean: `Combined.padUnbounded`

Lean: `Combined.padUnbounded_final`

Lean: `Combined.unbounded_step_padding`

Lean: `Combined.unbounded_run_padding`

Lean: `Combined.padding_refines`

Padding an initialized closed program agrees with initializing its original padded program object. The proof applies equality congruence to the graph-padding equality, with the search input fixed explicitly. Induction on the number of added layers composes one-step refinements; no additional machine transitions are introduced by this proof. Successful termination then follows for every finite padding by applying the exact-cost refinement to the original successful run.

Lean: `Combined.padding_initial`

Lean: `Combined.initial_pad_refines`

Lean: `Combined.terminates_pad`

Initial states of recursively isomorphic closed graphs have the same search mode and external input, so the previously proved representation relation gives equivalence for every cost preorder, including exact cost. To pad a closed observation, first pad the already closed program using the new forward refinement. The established context-filling padding isomorphism then identifies that padded program with the result of closing the padded fragment in the padded context. Composing with exact isomorphism invariance proves forward preservation of all successful observation costs.

Lean: `Combined.initial_iso`

Lean: `Combined.close_pad_refines`

The converse needs an additional proof: an arbitrary run from a padded graph may use a match presented at the larger allowance, and that match must be reflected to the smaller representation. None of the forward constructors provides this inversion automatically. Until reflection is proved, the execution does not satisfy `RespectsPadding`; the cofinal observation criterion and general semantic weakening cannot be concluded from this milestone alone.

87.6 Arithmetic and representation regressions

The unary and binary local redexes define actual closed-program objects, using the language safety certificates proved above. Their finite runs lift to the unbounded machine and give explicit five- and seven-transition termination witnesses. These program objects denote the literal local redex graphs. They are not advertised as the complete Figure 16 translations of arithmetic expressions with surrounding environment distributors.

Lean: `Arithmetic.Examples.unaryProgram`

Lean: `Arithmetic.Examples.binaryProgram`

Lean: `Arithmetic.Examples.unary_terminates`

Lean: `Arithmetic.Examples.binary_terminates`

The unary-result theorem substitutes the proved zero result into its actual endpoint state. The subtraction-underflow theorem does the same for every pair $m \leq n$, using natural subtraction. Endpoint-state congruence is used because the state contains a graph and a vertex on that

graph. Addition and subtraction with a zero right operand both return n in seven steps. All these regressions quantify over arbitrary natural numbers and allowances.

Lean: `Arithmetic.Examples.unary_result`

Lean: `Arithmetic.Examples.subtraction_underflow`

Lean: `Arithmetic.Examples.addition_zero`

Lean: `Arithmetic.Examples.subtraction_zero`

A contextual regression connects an outer unary-minus corolla to a binary arithmetic redex. Its argument selector is the actual right-component insertion coordinate. The outer head enters that argument in one search step. The right-serial run-lifting theorem executes all seven inner transitions, including its graph-changing arithmetic rule. Backtracking then resumes the outer head, which performs its own rewrite and numeric return. The resulting trace has $1+7+1+1+1 = 11$ transitions. This checks interaction of two real rewrites across a contextual boundary, not merely an isolated compute step.

Lean: `Arithmetic.Examples.nested`

Lean: `Arithmetic.Examples.innerInput`

Lean: `Arithmetic.Examples.nested_enter`

Lean: `Arithmetic.Examples.nested_run`

The original translated reference program still terminates after any finite padding, by the new general termination theorem. The application regression with a literal translated constant body retains a successful run of exactly seven transitions after any padding. Every binary arithmetic redex also retains its exact seven-transition cost at arbitrarily higher allowances. These proofs use the exact-cost refinement directly and eliminate its equality of run lengths; they do not claim uniqueness of all combined executions.

Lean: `Arithmetic.Examples.translated_reference_pad`

Lean: `Arithmetic.Examples.application_pad_cost`

Lean: `Arithmetic.Examples.binary_pad_cost`

The full build checks 355 jobs. The three audits pass for 6 abstract, 477 graph and 912 copying declarations, with only standard Lean logical axioms. At this milestone the estimate was about 75% of mathematical effort, broadly 60–85%. Section 88 completes the converse padding argument, gives the updated estimate and records its new verification results. Appendix B.1 retains only the unfinished tasks.

88 Exact padding reflection and contextual congruence

The concrete combined execution now satisfies both representation laws: recursive-isomorphism invariance and padding invariance. Every finite run from a padded state reflects to a run of exactly the same length at the original allowance. Its endpoint is recursively isomorphic to the padding of the smaller endpoint. Together with the preceding section's forward transport, this establishes equality of all successful execution costs. No determinism, refocusing or matching-independence assumption is used.

This is a substantial machine obligation, not the final template theorem. The abstract cofinal-allowance and full term-context-congruence results can now be instantiated with this concrete execution without representation-law premises. The full copying-machine correspondence, matching independence, refocusing, specimen laws and eleven template families still require proofs. In particular, this section does not assert equivalence of the two programs of Proposition 12.2. Their terms and their Figure 16 translations are unchanged.

The dependency chain is: typed truncation and its two round trips; reflection of stability and copyability; descent of every primitive local match; descent of contextual matches; exact finite-run reflection; and the unbounded observation and congruence consequences. The depth invariant is proved simultaneously with operational descent, rather than assumed.

88.1 Typed truncation and the representable fragment

Write P_d for padding from allowance d to $d+1$, and L_d for the new auxiliary truncation in the reverse direction. Truncation preserves every outer vertex, edge, wire label and ordered incidence list. It preserves primitive labels. At a positive target allowance, it preserves a box's bound, free-variable and free-atom counts and recursively truncates its body. At target allowance zero there is no box constructor. To make the auxiliary function total, an excessive box is replaced by a typed hole with input $[\mathsf{thunk}(b)]$, output the original free interface, and a fixed auxiliary name. Such a replacement is used only in the structural proof. It is neither an execution step nor a clause of term translation.

Totality matters because a context's replaced slot disappears during plugging: its label need not be representable even if the filled graph is. The proofs below exclude excessive boxes from actual representable source and target graphs. No semantic result is claimed for truncating an arbitrary excessive box without that restriction.

Lean: `Lowering.slot`

Lean: `Lowering.lower`

Lean: `Lowering.label`

The input and output signature equations follow by cases on the target allowance and on whether the label is primitive or boxed. In the excessive case, the placeholder has exactly the box's old signature by construction. Uniform graph truncation is therefore the existing typed relabelling operation; its graph laws, incidence coverage and finite enumerations are inherited from that operation. The uniform presentation and recursive presentation are equal by cases on allowance and labels. The uniform form makes unchanged outer carriers available definitionally in token proofs.

Lean: `Lowering.label_input`

Lean: `Lowering.label_output`

Lean: `Lowering.graph`

Lean: Lowering.graph_eq_lower

A label at allowance $d+1$ *fits* at allowance d when it is primitive, or when $d > 0$ and every label in its boxed body fits one layer lower. At target allowance zero a box never fits. A graph fits when all its outer labels do. Thus this is a recursive depth bound on the actual graph, not an assumption that its syntactic allowance is minimal.

Representability is invariant under recursive isomorphism. At the bottom layer equivalent labels have the same primitive/box case. At higher layers, related boxes have identical metadata and a body isomorphism; transport each body edge using its edge bijection and apply induction. The graph version uses the outer edge bijection, including its inverse for the forward universal quantifier. Every padded label fits by induction on allowance: a padded box has a padded body. Quantifying this label result gives both the uniform and original graph-padding certificates.

Lean: Lowering.FitsLabel

Lean: Lowering.Fits

Lean: Lowering.fitsLabel_equiv

Lean: Lowering.fits_iso_iff

Lean: Lowering.fits_padding_label

Lean: Lowering.fits_graph

Lean: Lowering.fits_raise

88.2 The two round trips and reflection of isomorphism

Truncation preserves recursive label equivalence even before restricting to representable graphs. Related primitives remain identical. Related boxes have the same metadata; at target zero their typed placeholders are identical, while at a higher target the induction hypothesis transports the body isomorphism. Consequently graph truncation transports an isomorphism using exactly its old vertex and edge bijections. Ordered incidence is unchanged.

Lean: Lowering.label_equiv

Lean: Lowering.graphIso

Lowering a padded label recovers the original label up to recursive isomorphism. The primitive case is immediate. In the box case, apply induction to each body label and use identity outer vertex and edge bijections. The resulting graph round trip $L_d(P_d(G)) \cong G$ also fixes every outer port. For the converse, $P_d(L_d(H)) \cong H$, a representability certificate rules out the excessive-box case and supplies the induction hypotheses for every nested body. Its outer bijections are again identities. These proofs establish that the auxiliary placeholder does not alter representable graphs.

Lean: Lowering.label_roundtrip

Lean: Lowering.roundtrip

Lean: Lowering.label_recover

Lean: Lowering.recover

Representability exactly characterizes the isomorphism-closed image of padding:

$$\mathsf{Fits}(H) \iff \exists G \text{ at allowance } d,\ P_d(G) \cong H.$$

For the forward implication choose $G = L_d(H)$ and use recovery. For the reverse implication, padded graphs fit and representability transports through the specified isomorphism. Padding is also faithful on recursive isomorphism: from $P_d(G) \cong P_d(H)$, lower the isomorphism and compose with the two left round trips to obtain $G \cong H$. The converse uses the established padding congruence. No quotient or new graph equality is introduced.

Lean: `Lowering.fits_iff_image`

Lean: `Lowering.reflectIso`

Lean: `Lowering.padding_iso_iff`

88.3 Assembly, erased slots and search coordinates

Truncation commutes with actual serial assembly up to an isomorphism that retains each component tag and the shared interface vertices. Its source and target equations compose the two incidence maps; cases on an internal, input or output port establish that its attachment is unchanged. This is the same explicit finite serial construction used by the original translation.

Lean: `Lowering.serial_graph`

For simultaneous expansion, truncate each filling and cast its boundary to the signature of the truncated host label. The signature equalities proved above justify these casts. Inserting such a cast port in the truncated host reaches exactly the old vertex. Internal ports retain their component tag; inputs and outputs retain their source or target index. The expansion isomorphism has identity outer bijections, and its two incidence equations follow by mapping this attachment identity over each edge's port list. Single-slot plugging has the same attachment argument. Retained host edges use their unchanged host embedding; inserted edges use the attachment identity.

Lean: `Lowering.expandFilling`

Lean: `Lowering.expand_insert`

Lean: `Lowering.expand_graph`

Lean: `Lowering.plug_insert`

Lean: `Lowering.plug_graph`

A primitive corolla truncates to the same primitive corolla at the smaller allowance, by cases on allowance. Equality casts of the external interface commute with truncation by eliminating those equalities. Representability is unchanged by either the translation's interface casts or the graph constructor's reboundary operation: neither changes edge labels. Serial assembly fits exactly when both components fit, by inspecting its Boolean-tagged edge carrier. Tensor composition has the same equivalence, using the general tensor label characterization.

Lean: `Lowering.primitive`

Lean: `Lowering.castInterfaces`

Lean: `Lowering.fits_castInterfaces`

Lean: Lowering.fits_reboundary

Lean: Lowering.fits_serial

Lean: Lowering.fits_tensor

A plugged graph $C[e := G]$ fits exactly when G fits and every edge of C other than e has a fitting label. There is deliberately no condition on the erased label of e. The proof splits the replacement's edge carrier into retained host edges and inserted edges. Likewise, simultaneous expansion fits exactly when every filling fits; every skeleton edge disappears. This avoids an unjustified restriction on the existing contextual-match relation.

Lean: Lowering.fits_plug

Lean: Lowering.fits_expand

A box over an allowance-zero body cannot fit one layer lower: apply the proposed certificate to its unique edge to obtain false. A higher box fits exactly when its body fits, again by inspecting the unique edge. Truncating a higher box acts recursively on its body. The lambda-box equation also preserves the existing bound-first interface convention, using the interface-cast law. Combining these equations with serial assembly and primitive truncation gives the isomorphism for complete lambda payloads.

Lean: Lowering.box_not_fits

Lean: Lowering.fits_box

Lean: Lowering.box

Lean: Lowering.lambdaBox

Lean: Lowering.lambdaPayload

On fitting labels, having a specified primitive label is equivalent before and after truncation. At allowance zero the only problematic case, a box turning into a hole, is excluded by the certificate; at higher allowances boxes remain boxes. For any predicate on primitives, the recursive all-labels predicate is therefore equivalent before and after truncation. The proof induces through box bodies and uses their fitting certificates. Its graph version quantifies over unchanged edge carriers. In particular, no focus, hole or non-language label is introduced into a fitting language graph.

Intrinsic search transport uses identity port and edge bijections, this primitive-recognition equivalence, and unchanged source and target lists. External-input status is unchanged. The existing exact search-transport theorem therefore applies in both directions, with the actual token coordinate.

Lean: Lowering.primitive_iff

Lean: Lowering.labelAll_iff

Lean: Lowering.allLabels_iff

Lean: Lowering.search

88.4 Reflection of the operational side conditions

Stable graphs fit at every smaller positive-to-predecessor allowance: each stable edge is a primitive instance or passive operation, never a box. Truncation preserves its primitive label and stability classification. The existing theorem deriving the full stability predicate from these labels recovers the output and reachability clauses as well. Conversely, for a fitting graph, stable labels after truncation reflect to the original primitive labels, so the same argument proves stability of the original graph. Stability has not been weakened to an informal value test.

Lean: `Stable.fits`

Lean: `Stable.lower`

Lean: `Stable.lower_iff`

A lambda payload at allowance one genuinely requires its box layer: serial representability would imply representability of that box, contradicting the bottom-layer box obstruction. At a higher allowance, a lambda payload fits exactly when its body does; the lambda head always fits, and the box/interface lemmas identify its body's condition.

To lower any copyable graph, induct on its copyability derivation, retaining an equality that its allowance is a successor. The instance, constant, application, unary and binary cases truncate to their original primitive corollas. In the lambda case, target allowance zero is impossible by the obstruction. Otherwise the lambda-payload isomorphism identifies the result with a lambda over the truncated body, which is copyable by definition. No stability premise is imposed on that body. In the renaming case, pull representability back through the isomorphism, use induction, and transport copyability through the lowered isomorphism.

Lean: `Copyable.lambda_not_fits`

Lean: `Copyable.lambda_fits_iff`

Lean: `Copyable.lower`

For a fitting graph, copyability of its lowering also implies original copyability: pad the copyable lowering using the established forward theorem and transport it along recovery. Thus copyability is equivalent on that fragment. Specializing to $P_d(G)$ and using the left round trip proves

$$\mathsf{Copyable}(P_d(G)) \iff \mathsf{Copyable}(G).$$

Induction yields the result for any finite number of layers. Equality transport in the depth index is eliminated directly; combining it with repeated padding gives the exact pad-to-allowance equivalence. Extra allowance therefore cannot make a previously non-copyable payload eligible for the copy rule.

Lean: `Copyable.lower_iff`

Lean: `Copyable.raise_iff`

Lean: `Copyable.pad_iff`

Lean: `Copyable.castDepth_iff`

Lean: `Copyable.padTo_iff`

88.5 Every local match descends

The recovery isomorphism is first restated using the original recursive raising function. If a specified padded graph is isomorphic to a higher graph, lowering that isomorphism and composing with the inverse left round trip identifies the smaller graph with the lowering of the higher graph.

Every local rule uses an ordered external input as its active port, which is fixed by any boundary-preserving graph isomorphism. Consequently a known smaller input-rooted rewrite transports through specified padded source and target isomorphisms to a rewrite between their lowerings. The target isomorphism also certifies representability of the higher target, because all padded graphs fit. This helper supplies both conclusions required by operational induction.

Lean: `Lowering.recover_raise`

Lean: `Lowering.fromRaised`

Lean: `Combined.lower_input_rule`

Suppose an actual primitive local rule takes (G, v) to (H, w) at allowance $d+1$ and G fits at d. Then H fits at d and the actual contextual rewrite relation at allowance d relates $(L_d(G), v)$ to $(L_d(H), w)$. The proof covers every constructor, as follows.

1. Allocation and assignment lower their stable argument. Stable graphs fit automatically. Their existing redex/reduct padding isomorphisms, followed by argument recovery, compare the smaller local rule to the given higher one. Assignment retains exactly the matched atom tree.

2. Dereference and both atom-equality cases have entirely primitive local graphs. Their original padding equalities directly supply the smaller rule, with the same matched atom trees and indexed active input.

3. Micro-beta extracts representability of the closure from the source's serial and tensor decomposition. At target zero its lambda box contradicts representability. Otherwise the lambda-body certificate and stable-argument certificate recover both smaller operands. The established before/after padding and congruence isomorphisms compare the two actual beta rules.

4. Copying extracts the payload certificate from the source's serial decomposition. Copyability lowers by the new converse theorem; the existing copy-construction congruence and padding isomorphisms retain the same contraction-tree presentation on both sides. This is not yet a proof of independence between different possible tree matches.

5. Unary and binary arithmetic use their established redex-padding and result-number isomorphisms. Their operands, operation codes and numeric results are unchanged.

All cases use the preceding input-rooted helper. No local rule is added to make this proof succeed, and no source or target relation is redefined.

Lean: `Combined.local_lowering`

88.6 Contextual matches and exact finite runs

Induct on the existing contextual rewrite derivation. A local step is the preceding theorem. For recursive renaming, pull source representability back, apply induction, and push target representability and the lowered rewrite forward. For plugging, the exact representability characterization provides both the inserted-source certificate and the unchanged surviving-host certificates. Induction supplies the inserted-target certificate. The new attachment equations and plugging isomorphisms transport the lower rewrite at precisely the original inserted token coordinates. For simultaneous expansion, every source filling fits. Only the selected filling changes; induction handles it and equality handles the others. The expansion attachment law then transports the rewrite. This covers arbitrary nested derivations of the shallow contextual relation, not just canonical local redexes.

Lean: `Combined.rewrite_lowering`

A lowered state consists of the truncated graph and the same mode and vertex. Its finality condition is literally unchanged: backtracking at an external input. A search step lowers by the concrete search transport. A rewrite step lowers by contextual match descent, which also proves that its target still fits. Induction on finite runs therefore proves both endpoint representability and a run of exactly the original length between the lowered endpoints. The induction carries the source certificate through each step; no global preservation assertion is left as a hypothesis.

Lean: `Combined.lowerState`

Lean: `Combined.lowerState_final`

Lean: `Combined.step_lowering`

Lean: `Combined.run_lowering`

The two graph round trips lift to state isomorphisms because their outer bijections are identities and thus fix even internal token vertices. An arbitrary state isomorphism also pads, using the already-proved graph padding transport while retaining its mode and vertex equations.

Lean: `Combined.lower_pad_state`

Lean: `Combined.pad_lower_state`

Lean: `Combined.padStateIso`

Let s have allowance d, and suppose $P_d(s) \longrightarrow^n t$. Lower the entire run using the initial padded graph's certificate. This gives a run from $L_d(P_d(s))$ to $L_d(t)$ and proves that t fits. Transport its starting state through the left round trip to s, obtaining a run $s \longrightarrow^n u$ and a state isomorphism $L_d(t) \cong u$. Pad that isomorphism, reverse it, and compose with recovery of t to obtain $P_d(u) \cong t$. Hence

$$P_d(s) \longrightarrow^n t \quad \Longrightarrow \quad \exists u,\ s \longrightarrow^n u \ \wedge\ P_d(u) \cong t.$$

There is no assumption that a rewrite match or a successor is unique. Finality transports through the endpoint isomorphism. Combining reflection with forward padding proves equivalence of successful termination at each exact cost n. The result includes zero-step runs.

Lean: `Combined.run_reflect_padding`

Lean: `Combined.terminatesIn_padding_iff`

88.7 Unbounded executions and observations

An unbounded run retains its original allowance. The existing bounded-run extraction theorem therefore turns a run from a padded unbounded state into a fixed-allowance run. Reflect that run by the preceding theorem and reinsert it into the unbounded machine. Its endpoint representation relation is obtained from the reflected endpoint state isomorphism.

For any cost preorder Q, every successful higher run now has a successful lower run at the identical cost, which is related to itself by reflexivity of Q. This proves reverse quantitative refinement. Pairing it with forward refinement gives padding equivalence for every Q, not only the universal termination preorder. Induction and transitivity give initial-program equivalence under any finite padding. Specializing to one layer and the universal preorder discharges the concrete `RespectsPadding` interface.

Lean: `Combined.unbounded_run_reflect_padding`

Lean: `Combined.padding_refines_reverse`

Lean: `Combined.padding_equiv`

Lean: `Combined.initial_pad_equiv`

Lean: `Combined.execution_respectsPadding`

The successful-cost spectrum of a closed program is the set of natural numbers n for which its initial state has an n-step run to a final state. This set is equal before and after any finite padding. To prove either inclusion, apply the corresponding refinement with the equality cost order and eliminate the resulting equality of costs. Existentially quantifying the cost gives termination equivalence. Negating that equivalence also preserves nontermination; no classification of divergent executions is needed.

Lean: `Combined.initial_cost_pad_iff`

Lean: `Combined.terminates_pad_iff`

For a closing context and an open fragment, first apply initial padding equivalence to the original closed filling. The established context-padding isomorphism identifies its padded graph with the filling of the padded context by the padded fragment. Concrete initial-isomorphism equivalence then gives exact-cost equivalence of those closed observations. The general observation reflection theorem can now be instantiated: contextual equivalence at a larger allowance reflects to the original allowance with both execution representation premises discharged.

Lean: `Combined.close_pad_equiv`

Lean: `Combined.equiv_of_pad`

The cofinal-allowance theorem now applies to the concrete combined execution: observational equivalence of terms is equivalent to contextual equivalence at arbitrarily large common adequate allowances. The existing proof includes all original certified closing contexts by padding them, and now both needed representation laws are established for the actual machine. Full congruence now applies to every original annotated term context, including terms with different minimum translation depths. The proof supplies concrete isomorphism and padding invariance to the general context theorem. Its premise is the terms' observational equivalence; the equivalence of Proposition 12.2 remains to be established.

Lean: `Combined.observationalEquiv_iff_cofinal`

Lean: Combined.observationalEquiv_context

88.8 Regressions, scope and remaining work

The bottom-layer regression rejects a lambda payload as representable without its only box layer. The positive nested-payload regression allows an arbitrary body and any finite extra allowance, deriving copyability from the new equivalence. The erased-slot regression allows a nonrepresentable box as a host slot when a fitting graph replaces it: the host has only one edge, so there are no surviving host labels to check. The unfilled host still fails representability. These distinguish actual box depth from irrelevant erased skeleton labels.

Lean: Lowering.Examples.lambda_needs_layer

Lean: Lowering.Examples.nested_payload_copyable

Lean: Lowering.Examples.erased_box_slot_fits

For every scoped term, padding followed by truncation satisfies the original translation-language predicate. This uses recursive all-labels reflection and the existing translation and padding safety proofs. For each of the two unchanged Proposition 12.2 terms, every certified closing context at its original minimum allowance and every finite padding preserve all successful costs. The environments are arbitrary variable and atom counts; there is no empty-environment restriction. The two assertions compare each program with its own padded representation, not with the other program. A final regression preserves nontermination of every closed contextual filling under padding. Its universal quantification includes admissible contexts with divergent callbacks or repeated closure use, but does not independently prove the behaviour of a particular such callback.

Lean: Lowering.Examples.translated_roundtrip_language

Lean: Lowering.Examples.privateCell_observation_padding

Lean: Lowering.Examples.constantResult_observation_padding

Lean: Lowering.Examples.nontermination_padding

All 89 new declarations are included in the copying axiom audit. The full build checks 362 jobs; all three audits pass with 6 abstract, 477 graph and 1001 copying declarations, using only `propext`, `Classical.choice` and `Quot.sound`. There are no project axioms or unfinished proofs in this milestone.

Appendix B.1 no longer lists padding reflection or its observation and congruence consequences as unfinished. Matching independence, deterministic behaviour up to representation, refocusing, focused-graph correspondence, specimen locality and template robustness remain. The completed invariance law concerns the concrete combined relation as defined; correspondence with the paper must still be maintained if that relation needs correction.

At this milestone the estimate for Proposition 12.2 was about 78% of the mathematical effort, with a broad plausible range of 60–88% (26 October 2026). The change from 75% reflects completion of a substantive semantic interface and its congruence consequences. This is a judgement about remaining proof difficulty, not a count of declarations or completed to-do bullets. The eleven-template argument and final Figure 22 chain retain substantial uncertainty.

The subsequent selection and runtime-safety milestone, with its updated estimate, is recorded in Section 89.

89 Canonical rule selection and recursive runtime safety

Every contextual rewrite of the concrete combined machine now has a unique *recognized rule choice*. Independently chosen decompositions of one source graph agree on the operation family, ordered arithmetic operands, atom-comparison result and copy-head primitive. More precisely, arbitrary copy matches at the same wire select the same payload head *edge*; read and write matches select the same store edge. These are properties of the actual host graph, with no global acyclicity assumption. The proof permits different matched contraction trees, different local interfaces and different representation allowances. It does not assume an isomorphism between the chosen matches.

This completes the source-selection part of matching independence. It does *not* yet identify the graphs produced by independently chosen rewrites: in particular, the residual tree, copied payload, distributors and surviving host must still be compared by a boundary-respecting recursive isomorphism. The theorem records a copy's head occurrence, not a new proof that the entire copied box forest is unique. These remaining successor-graph obligations are retained in Appendix B.1, along with refocusing and the specimen laws.

A second result proves recursive label safety through every complete finite run. The source translation alphabet excludes stores; the runtime alphabet adds stores because reference allocation creates them. Neither hidden focus edges nor unfilled holes can appear, even in duplicated or opened box bodies. The original rewrite relation, stability and copyability premises, proposition terms, and Figure 16 translations are unchanged.

The dependency chain is: incidence-preserving occurrences; canonical heads and ordered arguments; deterministic term-sharing and atom-store resolution; unique rule recognition; then preservation of recursive labels through all local rules, their context closure and finite runs. These proofs supply premises needed by the paper's copying-machine determinism argument in Section 10.3, rather than replacing that argument with an assumed interface.

89.1 Occurrences inside arbitrary hosts

An occurrence $f : G \hookrightarrow H$ consists of a map of ports and an injective map of edge occurrences. Mapping the ordered source and target lists of an edge must give exactly the corresponding host lists. A primitive label p in G must remain p in H, interpreted at the host's depth allowance. No global injectivity of ports is assumed: attachment may identify boundary ports. Edge injectivity is essential for preserving distinct store identities. This interface describes an occurrence; it is not an assertion that an arbitrary non-injective variable renaming preserves program meaning. It also does not compare the contents of arbitrary box labels, so later uses are restricted to primitive recognition and incidence tracing.

Lean: `MatchEmbedding`

The identity occurrence uses identity maps. Composition composes both maps; edge injectivity composes, primitive-label preservation is used twice, and mapping an incidence list twice equals mapping the composite function. A previously proved search transport supplies an occurrence by forgetting its inverse and boundary classification. A recursive graph isomorphism gives such a search transport, hence an occurrence.

Lean: `MatchEmbedding.refl`

Lean: `MatchEmbedding.comp`

Lean: `MatchEmbedding.ofTransport`

Lean: MatchEmbedding.ofDeepIso

Shallow plugging maps inserted edges to the right summand and ports by the actual attachment map. Simultaneous expansion maps a filler edge a to the dependent pair (e, a) for its slot e. Both edge maps are injective; their incidence and primitive-label laws are the definitions of plugging and expansion. The left and right occurrences in serial assembly use its existing edge and port maps; their edge images are the two distinct slot tags. These constructions require no safety predicate on the erased slot label.

Lean: MatchEmbedding.plug

Lean: MatchEmbedding.expand

Lean: MatchEmbedding.serialLeft

Lean: MatchEmbedding.serialRight

Membership in a source or target list survives mapping because membership survives list mapping. An adjacency witness is a shared port in a target and a source; mapping that witness preserves adjacency. These facts deliberately use forward incidence preservation, not a false assumption that the attachment map is globally injective on ports.

Lean: MatchEmbedding.source_mem

Lean: MatchEmbedding.target_mem

Lean: MatchEmbedding.adjacent

89.2 Canonical active heads and ordered arguments

$\mathsf{HeadAt}(G, v, e, p)$ says that edge e has primitive label p and consumes wire v. If two heads consume the same wire, source-incidence uniqueness forces their edges to be equal. Their labels are then equal, and injectivity of primitive labelling forces the primitives to be equal. Transporting a head uses primitive preservation and source membership preservation.

Lean: MatchEmbedding.HeadAt

Lean: MatchEmbedding.head_unique

Lean: MatchEmbedding.head

$\mathsf{Argument}(G, e, i, a)$ says that the ith target of edge e exists and is consumed by edge a. Two witnesses use the same port because the ordered list has one value at index i; source-incidence uniqueness then equates the consuming edges. Mapping a list preserves the value at an existing index, so occurrences preserve this ordered argument relation. This rules out changing operand order while presenting the same host match.

Lean: MatchEmbedding.Argument

Lean: MatchEmbedding.argument_unique

Lean: MatchEmbedding.argument

89.3 Term-sharing resolution

The forward labels for a copy trace are term contraction and term weakening. A payload head is an instance or operation primitive. A hop follows an adjacency whose departing edge has a forward label. Each forward edge has exactly one term target, by its signature. Consequently two hops from that edge agree on the next edge: their shared ports coincide, and source uniqueness identifies the next consumer. A payload head is terminal for this relation because neither an instance nor an operation is a sharing edge. The weakening case is admitted in intermediate tree reasoning, although a weakening cannot consume an externally selected input.

Lean: `CopyTrace.ForwardLabel`

Lean: `CopyTrace.PayloadHead`

Lean: `CopyTrace.Hop`

Lean: `CopyTrace.forward_output`

Lean: `CopyTrace.hop_functional`

Lean: `CopyTrace.head_terminal`

A resolution of wire v to payload edge e, primitive p and count n chooses its first consuming edge and an exact n-hop trace ending at a p-labelled payload head. Two successful resolutions first agree on their starting edge by source uniqueness. The functional-terminal-trace theorem then gives the same endpoint and exact length. Primitive-label injectivity also gives the same p. A surrounding cycle causes no problem: the theorem compares successful traces and does not claim every host trace terminates.

Lean: `CopyTrace.Resolves`

Lean: `CopyTrace.resolves_unique`

An occurrence maps sharing labels and adjacency, hence each hop, and maps an entire exact trace without changing its length. Combining this transport with uniqueness in the larger host proves that attachment cannot redirect an already resolved request or supply a different successful hop count. The auxiliary last-hop lemma appends one hop to a trace and increments its length; its proof is induction on the trace.

Lean: `CopyTrace.hop_map`

Lean: `CopyTrace.resolves_map`

Lean: `CopyTrace.resolves_map_exact`

Lean: `CopyTrace.snoc`

89.4 Every actual copy match resolves to its payload

A copyable graph has one term input, which source coverage assigns to an edge. Its proved outer-label property permits only instances, operations or boxes. Typing excludes a box at this input, since a box consumes a thunk wire. Thus the edge is an instance/operation head. No condition on the behaviour or stability of a lambda body is added to copyability.

Lean: `Copyable.head_exists`

In the literal copying redex, the tree's output and the payload's input are the same serial-assembly wire. Each tree adjacency becomes a sharing hop through the left occurrence. The root targets that shared wire, so it has one last hop to the payload head in the right occurrence. The arbitrary-tree termination theorem supplies, for every selected input, a trace from its consumer to the root. Mapping that trace and appending the last hop produces resolution to the attached payload, with count $m + 1$ for a tree trace of length m.

Lean: `CopyRule.tree_payload_wire`

Lean: `CopyRule.before_tree_hop`

Lean: `CopyRule.before_root_hop`

Lean: `CopyRule.before_resolves`

The consumer of a tree input must be a contraction: a weakening's source list is empty. This proves the active head required for copy recognition. If a particular payload head was specified in advance, head uniqueness identifies it with the head chosen by the existence proof, giving the same resolution theorem for that specified edge.

Lean: `CopyRule.before_active_head`

Lean: `CopyRule.before_resolves_head`

Now place two independently chosen redexes into one host, with the same image of their selected input. Their trees may have different input counts and need not be isomorphic. Their payloads may be presented at different allowances. Resolve each selected input in its own redex, map both traces into the host, and apply host resolution uniqueness. The images of the payload heads are the same edge and have the same primitive. This is the precise copy-selection independence established here; it does not equate the two rewritten graphs.

Lean: `CopyRule.match_payload_unique`

89.5 Read and write destinations

Atom hops and successful store-resolution traces map through the same occurrence interface. Unlike copy traces, their forward labels also include instance edges, and their terminal edge is a store. The proofs map the exact trace and preserve the store label; previously proved store-resolution uniqueness supplies its canonical endpoint and count.

Lean: `StoreTrace.hop_map`

Lean: `StoreTrace.resolves_map`

A resolved store operand at wire v specifies an operation o, an argument index i, a store edge s and a hop count. It witnesses the operation head, the edge consuming its ith argument, and a store-resolution trace from that edge. Two witnesses agree on the head, then on the ordered argument edge, then on the store and count. Occurrences preserve all three witnesses.

Lean: `StoreTrace.Operand`

Lean: `StoreTrace.operand_unique`

Lean: `StoreTrace.operand_map`

For each concrete dereference redex, its reader's argument is the displayed instance and its already proved trace resolves to the retained store. For assignment, argument zero is the displayed instance resolving to the destination store; the new stable value remains argument one. These witnesses are built from the actual local incidence lists and existing tree-resolution proofs. Given two independent embeddings of read redexes, or two of write redexes, at the same host token, transport the operand witnesses and apply uniqueness. Both matches select the same store occurrence. This result does not yet compare the entire written-value graphs or the successor heaps.

Lean: `Dereference.before_operand`

Lean: `Dereference.match_store_unique`

Lean: `Assignment.before_operand`

Lean: `Assignment.match_store_unique`

89.6 Reference-equality branch independence

A comparison decision b at v consists of resolved stores s, t for the ordered arguments zero and one, together with

$$s = t \quad \Longleftrightarrow \quad b = \mathsf{true}.$$

Two decisions agree on each store by operand uniqueness, hence agree on b. Mapping a decision uses edge injectivity in the reverse implication $f(s) = f(t) \Rightarrow s = t$. This is why mere forward incidence preservation would not suffice to exclude a false aliasing match.

Lean: `AtomEquality.Decision`

Lean: `AtomEquality.decision_unique`

Lean: `AtomEquality.decision_map`

The one-store local rule has decision true: both existing atom traces end at its same store. The two-store rule has decision false: the store edges have distinct constructor tags in the actual frame. The operation head and both ordered argument incidences are checked directly. If both redex shapes could embed at the same host token, transported decisions would force true to equal false. Thus the branches are disjoint under independently chosen contexts and contraction trees, not just under a renaming of one match.

Lean: `AtomEquality.Same.before_decision`

Lean: `AtomEquality.Different.before_decision`

Lean: `AtomEquality.same_different_disjoint`

89.7 Arithmetic operand independence

A number argument witnesses the edge consuming argument i and requires its label to be the actual natural-number constant n. The argument edge is unique; equality of its primitive labels implies equality of the numbers. Occurrences preserve this witness and its index.

Lean: `Arithmetic.NumberArgument`

Lean: `Arithmetic.numberArgument_unique`

Lean: `Arithmetic.numberArgument_map`

The local unary head is numeric negation and its argument is the specified number edge. The binary head is the specified addition/subtraction operation; its two number edges occur at indices zero and one respectively. All these facts are checked from the original serial/tensor incidence lists, with no evaluation or term simplification. The operation-code-to-symbol function is injective by its two cases. Therefore independently embedded binary redexes at one token agree on the operation code, the left number and the right number. The arithmetic convention remains $-n = 0 - n$ on natural numbers.

Lean: `Arithmetic.unary_head`

Lean: `Arithmetic.unary_operand`

Lean: `Arithmetic.binary_head`

Lean: `Arithmetic.binary_left_operand`

Lean: `Arithmetic.binary_right_operand`

Lean: `Arithmetic.BinaryCode.symbol_injective`

Lean: `Arithmetic.binary_match_unique`

89.8 Unique selection for the full contextual relation

A rule choice records allocation, read, assignment, beta, a Boolean comparison result, a copy-head primitive, a unary numeric operand, or a binary operation code with both numbers. Its head function gives the corresponding active operation or term contraction. Comparison choices with different answers, copy choices with different payload labels, and arithmetic choices with different numbers deliberately share a head; incidence and trace facts must distinguish those cases.

Lean: `Combined.Choice`

Lean: `Combined.Choice.head`

Recognition of a choice means that the source wire has its required head, plus the appropriate comparison decision, copy resolution, or ordered number arguments. Allocation, read, assignment and beta need no additional data in this particular record. Their finer store-selection results were proved separately above. Recognition only inspects the source graph; it neither changes the operational semantics nor asserts that a recognized choice has an applicable rewrite. For example, it does not remove beta's stable-argument premise.

Lean: `Combined.Recognizes`

Recognition transports through occurrences by the head, argument and trace transport lemmas. For uniqueness, head uniqueness first excludes differing operation families. The remaining

cases use comparison-decision uniqueness, copy-resolution uniqueness, or number-argument uniqueness. Binary-symbol injectivity handles addition versus subtraction. These exhaust the choice constructors.

Lean: `Combined.recognizes_map`

Lean: `Combined.recognizes_unique`

Every local rewrite has a recognition witness. The fixed operation heads are read directly from the local frames. Equality uses the true/false decision lemmas, copying uses the arbitrary-tree head and resolution theorems, and arithmetic uses the two ordered operand witnesses. The original local-rule proof supplies its existing stability or copyability premise.

Induction on an actual contextual rewrite then lifts recognition through recursive renaming, shallow plugging and simultaneous expansion. The unchanged fillers need not be inspected for this source property. Combining existence with uniqueness gives the milestone theorem:

$$G@v \longrightarrow H@w \quad \Longrightarrow \quad \exists! c,\ \mathsf{Recognizes}(G, v, c).$$

This statement applies to every derivation of the existing rewrite relation. Uniqueness of c must not be confused with uniqueness of (H, w) up to representation; the latter still needs the successor-coherence argument.

Lean: `Combined.local_recognizes`

Lean: `Combined.rewrite_recognizes`

Lean: `Combined.rewrite_unique_choice`

Every choice head is active or a term contraction, by cases on the choice. Stable graphs contain only instances and passive operation heads, so they support no contextual rewrite at any wire. More generally, an embedded stable payload cannot be rewritten at the image of its input, even if its host is cyclic or contains other redexes: source coverage supplies its passive head, whose image conflicts with every recognized rewrite head. Finally, a rewrite cannot be based at an external output, because its recognized head would consume that output, contradicting graph incidence coverage.

Lean: `Combined.choice_head_active`

Lean: `Combined.stable_no_rewrite`

Lean: `Combined.rewrite_not_output`

Lean: `Combined.stable_occurrence_no_rewrite`

For a full search transport, apply occurrence transport in both directions; the inverse port map cancels the forward map. This proves recognition preservation and reflection. Instantiating with arbitrary finite padding shows that the chosen operation, copy-head label, comparison answer and numeric operands do not change with unused representation allowance.

Lean: `Combined.recognizes_transport_iff`

Lean: `Combined.recognizes_pad_iff`

89.9 Recursive label preservation through computation

A primitive predicate P admits the local rules when it includes every operation, instance, store, weakening and contraction label. This is a sufficient closure condition for the new labels introduced by computation. It need not admit focus or hole labels. Safety is the existing recursive predicate $\mathsf{AllLabels}(P, G)$, so it inspects every box body as well as outer labels.

Lean: `RuleSafety.Admits`

Safety of a serial composite implies safety of each component by evaluating the predicate on its corresponding edge occurrence and using label equality. For beta, source safety additionally gives safety of the chosen lambda body: inspect its actual box edge, unfold the recursive label predicate and undo the provably equal interface cast. Inspecting the argument occurrence gives its complete recursive safety independently of the separate stability premise.

Lean: `RuleSafety.serial_left`

Lean: `RuleSafety.serial_right`

Lean: `RuleSafety.application_body`

Lean: `RuleSafety.application_argument`

Local preservation is proved by cases on all nine rule constructors. Allocation extracts safety of the old value and adds only admitted prefix labels. Read and equality use their existing all-label lemmas. Assignment extracts safety of the value occurrence and applies its after-graph lemma. Beta extracts the body and argument predicates and uses preservation when opening the box. Copying extracts payload safety from the serial source and preserves it in both copies and the environment distributors. Arithmetic returns an admitted numeric constant.

For context closure, recursive isomorphisms transport the predicate. Plugging splits each target edge into a surviving host edge, certified by source safety, or an inserted edge, certified by induction. Simultaneous expansion uses induction at the rewritten slot and the explicit equality of all other fillers. No safety assumption on an erased host slot is needed.

Lean: `RuleSafety.local_allLabels`

Lean: `RuleSafety.rewrite_allLabels`

89.10 Runtime alphabet and reachable-state guarantees

The runtime predicate is the union of the original translation alphabet with the store primitive. A runtime graph satisfies this predicate at every depth. All five admissibility fields follow by cases on their constructor. Every translated-language graph is a runtime graph: prove this by induction on the box allowance, injecting primitive membership into the union and recursively handling boxed bodies.

The predicates excluding every focus label and excluding every hole label also admit all local rules, by constructor disjointness. Applying the generic rewrite theorem gives preservation of runtime membership, focus freedom and hole freedom separately. This proves a runtime invariant; it does not incorrectly claim that allocation preserves the smaller translation alphabet.

Lean: `RuleSafety.RuntimePrimitive`

Lean: `RuleSafety.RuntimeGraph`

Lean: RuleSafety.admits_runtime

Lean: RuleSafety.language_runtime

Lean: RuleSafety.admits_focusFree

Lean: RuleSafety.admits_holeFree

Lean: Combined.rewrite_runtime

Lean: Combined.rewrite_focusFree

Lean: Combined.rewrite_holeFree

A combined search step leaves the graph unchanged; a rewrite step uses the proved label-preservation theorem. Induction on exact finite machine runs therefore preserves any admitted recursive predicate. For an unbounded run, use the already proved exact bounded-run representation, then apply that induction at the fixed allowance. A certified closed program supplies initial focus freedom and hole freedom, giving both at every reachable state. If its initial graph is additionally in the translation alphabet, the same argument gives runtime-alphabet membership throughout. These conclusions hold for every finite prefix, irrespective of whether the execution eventually terminates.

Lean: Combined.step_allLabels

Lean: Combined.run_allLabels

Lean: Combined.reachable_safe

Lean: Combined.reachable_runtime

89.11 Regression consequences, audit and remaining effort

The arithmetic regression excludes two matches whose unequal operands were swapped: binary-match uniqueness would force the two numbers equal. An allocation regression proves that its after-graph really leaves the translation alphabet by inspecting the newly created store; another proves that it nevertheless remains in the runtime alphabet. A numeric value embedded in any host cannot support a rewrite at its input, by the stable occurrence theorem.

Lean: SelectionExamples.unequal_operands_not_swappable

Lean: SelectionExamples.allocation_not_translation_alphabet

Lean: SelectionExamples.allocation_runtime

Lean: SelectionExamples.embedded_number_no_rewrite

For each unchanged Proposition 12.2 term, every finite run in every certified closing context remains focus-free and hole-free. The quantification covers arbitrary variable/atom environments and finite prefixes arising from admissible divergent callbacks or repeated closure use. These are safety invariants, not new equivalence or termination proofs for such callbacks; the Figure 22 template argument is still required.

Lean: SelectionExamples.privateCell_runs_safe

Lean: SelectionExamples.constantResult_runs_safe

All 103 new declarations are included in the axiom audit. The complete build passes 369 jobs. The three audits check 6 abstract, 477 graph and 1104 copying declarations and use only

`propext`, `Classical.choice` and `Quot.sound`. No unfinished proof or project-specific axiom is introduced.

Appendix B.1 removes source-selection independence and recursive label safety from the unfinished obligations, while retaining successor-graph coherence, refocusing, focused-state correspondence, specimen locality, semantic renaming/substitution, all required template robustness proofs and final assembly. None of the four broad remaining packages is claimed wholly closed.

The 80% mathematical-effort estimate originally recorded at this milestone was too optimistic and is withdrawn. Section 90 records the corrected low-confidence assessment of 50–65%, alongside the remaining major proof obligations. The completed selection and safety proofs remain valid; their size did not justify the earlier estimate.

90 Allocation successor coherence and exact costs

The allocation branch of matching independence is now complete for the actual combined rewrite relation. If two rewrites start at the same ref request in the same host graph, their successor graphs are recursively isomorphic and the isomorphism carries one resumed token to the other. The matches may choose different retained hosts, stable argument subgraphs, context decompositions and output representations. No agreement of these choices is assumed. The theorem covers every finite interface and depth allowance, including arbitrary typed context insertion and simultaneous expansion.

This is stronger than source selection: the entire result is compared, including the fresh instance, canonical atom tree, store, retained graph and token. It completes the allocation case of the copying-machine determinism obligation. Section 91 subsequently completes both reference-equality branches. The remaining successor families, refocusing and the concrete specimen interface remain open. Neither the Proposition 12.2 terms nor Figure 16's translation is modified.

The construction follows the allocation rule in PDF Figure 19(a). Its source is a ref corolla followed by a stable argument A; its target replaces that corolla by the allocated-cell prefix and keeps A. The proof turns this observation into a canonical replacement of one edge of the entire source host. It uses the previously proved substitution associativity and unit laws, then inducts over the actual context-closed rewrite derivation. The original stability premise is retained: a ref head alone need not be reducible.

The dependency chain is therefore source-head uniqueness, state-level replacement congruences, canonical local allocation, induction over actual matches, successor coherence, then exact successful-cost consequences. This discharges a concrete machine obligation needed before the paper's robustness theorem can be instantiated; it is not yet the reference-creation template's robustness proof.

90.1 States and replacement congruences

A state consists of a graph, a focus mode and a vertex. When a graph is changed by equality, transporting its vertex along that equality gives the same dependent state. Equality of states supplies a state isomorphism by the identity construction. Both facts follow by eliminating the equality proof; they handle representation indices without dropping the token coordinate.

Lean: `Combined.StateIso.cast_state`

Lean: `Combined.StateIso.ofEq`

A state isomorphism between two fillers lifts through plugging into a fixed host edge. The existing graph congruence gives the graph isomorphism; its inserted-port equation, followed by the original token equation, gives the new token equation. For simultaneous expansion, assign the supplied graph isomorphism to the changed component and identity isomorphisms to all other components. The assembly isomorphism preserves the selected token by its component-insertion equation. These statements do not assume that attachment is injective on boundary ports.

Lean: `Combined.StateIso.plug`

Lean: `Combined.StateIso.expand`

Changing equal interface lists preserves a primitive-head witness. Labels are unchanged and the source incidence list is mapped by the boundary cast; source membership therefore survives

by membership in a mapped list.

Lean: `Combined.head_reboundary`

For a host G, selected edge e, fitting replacement A and old token v, define the *replacement state*

$$R(G, e, A, v) = \big(G[e := A],\ \text{search},\ \iota_G(v)\big),$$

where ι_G is the actual retained-host port map. The old wire is retained, including when it is a boundary wire or participates in cyclic host structure. This definition records where search resumes; it does not itself assert that the operational relation can perform the replacement.

Lean: `Combined.Replacement.state`

Replacing an edge of a plugged filler can be performed either after plugging or inside the filler before plugging. The graph isomorphism is substitution associativity; its middle-component equation carries the old filler token to the same attached wire. In a simultaneous expansion, regroup the replacement into its selected component and leave every other component unchanged. The expansion/replacement interchange isomorphism provides the graph map and its component equation provides the token map. The cast in the formal statement only identifies the selected component of the updated family with the explicitly replaced filler.

Lean: `Combined.Replacement.plug`

Lean: `Combined.Replacement.expand`

Recursive renaming of the host transports replacement to the renamed edge. The replacement's interface is reindexed by the equalities forced by the related edge labels. Host-plug congruence gives the graph isomorphism, and its retained-host equation proves the token equation. Finally, if a changed component has an independently chosen isomorphic output, compose expansion interchange with expansion congruence. Equality transport removes the selected family-component cast before applying that component isomorphism.

Lean: `Combined.Replacement.rename`

Lean: `Combined.Replacement.expand_result`

90.2 Canonical allocation in a source host

Two successive changes of interface lists equal the single change by their composite equalities. Eliminating the four equality proofs reduces the claim to the identity-boundary law. In an allocation source, the selected head is the left serial component's sole ref edge. Its primitive label and source membership follow directly from the ref corolla and serial incidence.

Lean: `reboundary_comp`

Lean: `Allocation.head`

Lean: `Allocation.head_at`

Suppose e has label ref in G. Its interfaces are then both the singleton term interface. The *allocation filler* is the existing allocated-cell prefix, reindexed by these forced interface equalities. It contains an instance edge, the original canonical atom-sharing tree and a store edge. The *canonical allocation successor* is

$$C(G, e, v) = R(G, e, \text{cellPrefix}, v).$$

Thus the construction depends only on the source host, its selected edge and its focused wire. It does not inspect a chosen decomposition into argument and retained context, nor simplify any distributor.

Lean: `Allocation.filler`

Lean: `Allocation.canonical`

Canonical allocation commutes with recursive host renaming. Apply the replacement-renaming isomorphism and combine the successive interface reindexings; proof irrelevance identifies the resulting equality witnesses. For a local source ref; A, serial replacement interchange first moves the replacement inside the left ref corolla. The corolla substitution unit then identifies that replaced component with the allocated-cell prefix. Serial congruence leaves A fixed. All these isomorphisms fix the external input, so the token equation follows as well.

Lean: `Allocation.canonical_rename`

Lean: `Allocation.canonical_local`

90.3 Independently chosen contextual matches

Every actual rewrite already has a recognized primitive head in its changed component. Map that head into a surrounding occurrence. If the host token has primitive head e, source uniqueness identifies the mapped component head with e, and primitive-label injectivity identifies their labels. Consequently the component contains the same primitive head. This reflection argument uses the actual rewrite witness; it does not claim that arbitrary occurrences reflect all source information, and needs no injective port map.

Lean: `Combined.rewrite_head_reflect`

For a local rule whose source head is ref, inspect the actual local-rule constructor. Allocation has the canonical result just proved. Dereference, assignment, both equality branches, application, term copying and arithmetic have different primitive heads, contradicting head uniqueness. The allocation constructor still carries its original stable-argument proof. Thus the local case contains neither an assumed determinism law nor an extra reduction rule.

Lean: `Combined.local_allocation_canonical`

Now induct over the actual contextual rewrite derivation. The local case is the preceding result. In a renaming case, reflect the ref head to the original source, apply induction, and compose the inverse canonical-source renaming with the inductive result and target renaming. In a plugging case, reflect the head into the changed filler, apply induction, and compose replacement associativity with state congruence. In an expansion case, reflect into the selected component and use the proved expansion-result interchange. In each case the head's edge equality identifies the canonical replacement of the whole source with the one used in the induction.

Lean: `Combined.rewrite_allocation_canonical`

Successor coherence at allocation means that any two actual rewrites from the same ref request produce state-isomorphic successors. Both successors are isomorphic to $C(G, e, v)$; invert the first isomorphism and compose it with the second. This compares complete output graphs and resumed token positions, not merely labels or observable answers.

Lean: `Combined.allocation_rewrite_deterministic`

The corresponding full-machine step cannot be a search step: search has no transition from

rewrite mode. Therefore every full step at the request has the canonical successor, and any two such steps have isomorphic successors. Moreover, an admitted allocation can step directly to the canonical representative: rename the target of its actual rewrite by the inverse canonical-result isomorphism. Its token equation gives the required resumed coordinate. The existence premise here is an actual transition, not a hypothetical stability or correctness axiom.

Lean: `Combined.allocation_step_canonical`

Lean: `Combined.allocation_step_deterministic`

Lean: `Combined.allocation_step_to_canonical`

90.4 Exact costs and the unbounded machine

Write $T_n(s)$ for existence of a successful run of exactly n combined transitions from s. Given an actual allocation step $s \to t$,

$$T_{n+1}(s) \quad \Longleftrightarrow \quad T_n(t).$$

For the forward direction, split off the successful run's first step. Its successor is state-isomorphic to t, and the previously proved run transport carries the tail at the same cost. Conversely, prepend the given allocation step. This argument places no restriction on the later computation; in particular it does not assume termination of a callback.

Lean: `Combined.allocation_cost_iff`

An unbounded step retains its finite representation allowance. Inverting its wrapper reduces two allocation steps to the bounded result above, then re-wrapping the state isomorphism gives the existing representation relation. That relation preserves every successful cost. Thus independently chosen unbounded allocation successors refine one another for every cost preorder, including exact equality. This remains an allocation theorem, not global unbounded-machine determinism.

Lean: `Combined.allocation_unbounded_deterministic`

Lean: `Combined.allocation_unbounded_equiv`

An actual search step has a unique full-machine successor: another search step agrees by intrinsic search determinism, while a rewrite alternative would require rewrite mode, from which the given search step cannot start. A final state has precisely cost zero, because finality excludes any first transition. Its zero-step run supplies the converse.

Lean: `Combined.search_successor_unique`

Lean: `Combined.final_cost_iff`

More generally, a given step $s \to t$ whose alternative successors are all state-isomorphic to t satisfies

$$T_n(s) \quad \Longleftrightarrow \quad \exists m \in \mathbb{N}\ (n = m + 1 \ \wedge \ T_m(t)).$$

A successful zero-step run at s would contradict its outgoing step. For a positive run, remove its first step and transport its tail by successor coherence. The reverse direction prepends the given step. Search uniqueness and proved allocation coherence discharge this intermediate

theorem's local coherence hypothesis in their respective specializations. The public search and allocation cost laws have no assumed determinism premise.

Lean: `Combined.coherent_step_cost_iff`

Lean: `Combined.search_cost_iff`

Lean: `Combined.allocation_cost_spectrum`

90.5 Regressions on the original translation and arbitrary hosts

The literal translated term `ref` n retains all Figure 16 single-use distributors, including its empty-environment forests. Its ref head has the required label and input incidence by definition. The actual allocation transition is obtained by lifting the existing stable-argument allocation through the unchanged serial distributor continuation. Applying successor coherence compares every other full-machine successor with this constructed result.

Lean: `AllocationCoherenceExamples.reference_head`

Lean: `AllocationCoherenceExamples.reference_allocate`

Lean: `AllocationCoherenceExamples.reference_successor`

For every natural n and cost k, the original translation of `ref` n satisfies

$$T_k([\![\texttt{ref } n]\!], \text{search}, \text{input}_0) \quad \Longleftrightarrow \quad k = 5.$$

There are three search transitions (enter ref, return the numeral, request allocation), one allocation transition and one instance-return transition. Apply the exact cost laws successively, then finality's zero-cost law. The resulting five successor equations force $k = 5$; the reverse direction supplies those five costs. This proves uniqueness of successful cost under the full combined relation, beyond the earlier exhibited five-step run.

Lean: `AllocationCoherenceExamples.reference_exact_cost`

Exact bounded/unbounded run correspondence gives the same cost characterization for the unbounded initializer. The already proved padding reflection and preservation then give the same characterization at every larger allowance. In particular this covers the literal `ref` 1 subexpression used in Proposition 12.2. It does not claim the proposition's entire closure is observationally equivalent yet.

Lean: `AllocationCoherenceExamples.reference_unbounded_exact_cost`

Lean: `AllocationCoherenceExamples.reference_padded_exact_cost`

Finally, insert an allocation source with any stable argument into any typed host edge whose input interface is a single term. Reindex its boundary by the supplied interface equality and lift the actual local allocation step. Map the ref-head witness through the same attachment. The general coherence theorem shows that every alternative successor agrees with the host filled by the allocation result. No acyclicity, global boundary injectivity or common matching decomposition is required.

Lean: `AllocationCoherenceExamples.host_allocation_successor`

All 39 new declarations are audited. The full build passes 373 jobs and the three audits inspect 6 abstract, 477 graph and 1143 copying declarations, with only `propext`, `Classical.choice` and `Quot.sound`. The standard check script now includes the copying audit as well.

Appendix B.1 removes allocation from the unfinished successor-coherence families. Its four broad packages remain open: in particular copying needs comparison of complete payloads, residual trees, distributors and retained hosts, and all eleven template families still need the required robustness arguments before Figure 22 can be assembled.

The earlier estimate of 80% completion is withdrawn. A cautious current assessment is **50–65% of the mathematical effort**, with low confidence (26 October 2026). That is not a measured percentage, a count of declarations, or a claim that the remaining work is routine. Source selection and allocation coherence are substantial established prerequisites; full copying-machine laws, the dependent template proofs and the final observational argument can still account for a large part of the work. This milestone does not justify a numerically precise increase within that range.

91 Reference-equality successor coherence and exact costs

Both reference-equality branches now have complete successor-state coherence for the actual combined machine. Two independently chosen contextual matches at the same comparison request produce recursively isomorphic whole graphs, and the isomorphism preserves the resumed token. This closes both comparison cases of Appendix B.1's machine obligation. It does not close the whole machine package. Section 92 subsequently completes assignment, and Section 93 completes dereference. Section 94 completes arithmetic. Sections 95 and 96 complete application and copying successor comparison; refocusing and the specimen laws remain open.

The operational rule is still the paper's Figure 19(d), discussed in HTML Section 10.1. The two outcomes depend on whether the argument instances lead to the same store edge. The new proof compares the complete reducts, extending the earlier uniqueness of that Boolean decision. All original edges outside the comparison head are retained, including instances, contraction trees, stores and their attached contents. The replacement introduced below is a proof construction; it is not a new rule or a change to the language.

The dependency chain is: unique source head and store decision; explicit local replacement isomorphisms; state-level substitution congruences from Section 90; induction over actual rewrites; bounded and unbounded successor coherence; exact successful-cost laws. Neither Proposition 12.2's terms nor Figure 16's translation changes. The four broad remaining packages stay open. The cautious estimate remains 50–65% of mathematical effort, with low confidence: closing these two cases does not establish the dependent template argument.

91.1 A result with two explicit weakenings

The finite-index helpers say that a one-element type contains only zero, and that every element of the three-element type is zero, one or two. Their proofs use the bounds carried by each finite index. They support exhaustive incidence checks and inverse-map proofs below.

Lean: `DiscardResult.fin_one`

Lean: `DiscardResult.fin_three`

For a constant c, the *result-and-discard graph* D_c has interface $\star \to \star \otimes \star$, no internal vertices and three edges. Edge zero is the constant c, with the sole input as its source and no targets. Edges one and two are term weakenings, with no sources and with the first and second output, respectively, as their targets. The label, source and target functions specify precisely these three cases. The graph therefore returns a value while explicitly retaining the two old argument wires.

Lean: `DiscardResult.label`

Lean: `DiscardResult.source`

Lean: `DiscardResult.target`

Enumerating the three edges proves well-formedness. Incidence lists are empty or singleton; only the constant uses the input and each weakening uses its own output. This gives uniqueness and coverage. Labels have exactly the specified interfaces, proving typing. Both carriers are finite. Primitive relabelling interprets D_c at any depth allowance and preserves these properties.

Lean: `DiscardResult.base`

Lean: DiscardResult.net

91.2 The canonical equality successor

Suppose e in G is labelled by atom equality, v is the request wire, and b is a Boolean. The *canonical equality successor* is

$$E(G, e, v, b) = R(G, e, D_{\mathrm{bool}(b)}, v),$$

where R is the replacement state of Section 90. The filler first transports $D_{\mathrm{bool}(b)}$ along the interface equalities implied by the head label. Then E replaces only e, resumes search at the retained image of v, and keeps every other edge occurrence. Defining this graph requires no termination or stability assumption about the stored contents. Its agreement with actual rewriting is a separate theorem below.

Lean: AtomEquality.filler

Lean: AtomEquality.canonical

A recursive isomorphism $f : G \cong H$ carries the canonical successor to $E(H, f(e), f(v), b)$. Primitive labels are preserved by recursive isomorphism, so the renamed head has the same operation. Apply the proved replacement renaming construction. The two boundary casts on the filler compose to the single cast required in H; proof irrelevance identifies their equality witnesses. The state isomorphism transports both the graph and token.

Lean: AtomEquality.canonical_rename

91.3 The same-store local rule

The local head is the unique edge in the comparison component. Its label is atom equality and its source is the request input, proving the head predicate. The abbreviation `replaced` names the graph component of $E(G, e, v, \mathrm{true})$ for this local source, with an arbitrary matched contraction tree. No canonical presentation of that tree is required.

Lean: AtomEquality.Same.head

Lean: AtomEquality.Same.head_at

Lean: AtomEquality.Same.replaced

The forward internal-vertex map retains each frame vertex and each contraction-tree vertex, while removing the extra sum tags introduced by one-edge plugging. The inverse restores these tags. Primitive corollas and D_c have empty internal carriers, so all other cases are impossible. Case analysis proves the two composites are identities, yielding the internal-vertex bijection.

Lean: AtomEquality.Same.vertexTo

Lean: AtomEquality.Same.vertexFrom

Lean: AtomEquality.Same.vertices

For edges, retain each left-instance, right-instance, tree and store occurrence. The old comparison edge is excluded by the subtype used for retained host edges. Send the three new edges to the result, left weakening and right weakening components of the original target. The inverse sends those components back to indices zero, one and two and restores every retained

edge. Exhausting the finite indices proves both inverse laws; in the excluded-head case the retained-edge inequality gives a contradiction.

Lean: `AtomEquality.Same.edgeTo`

Lean: `AtomEquality.Same.edgeFrom`

Lean: `AtomEquality.Same.edges`

The frame-port equation follows by splitting a port into input, internal vertex or output. The tree-port equation then splits the tree port: an internal port follows directly from the vertex map, while input and output ports reduce to the frame equation at the corresponding tree attachment. This checks the whole interface of an arbitrary tree, not only its root.

Lean: `AtomEquality.Same.map_frame`

Lean: `AtomEquality.Same.map_tree`

Assemble the two bijections into a graph isomorphism. Internal wire labels and retained edge labels agree by construction. Primitive edge incidences reduce directly; tree incidences are lists of inserted ports, so mapping each entry uses the tree-port equation. The three fresh edges have exactly the sources and targets of the target result and weakenings. Finally, extend the graph isomorphism to a state isomorphism: both tokens have search mode and the same external input. This proves canonicality of the actual same-store local rule, with its complete retained tree and store.

Lean: `AtomEquality.Same.replacement_iso`

Lean: `AtomEquality.Same.canonical_local`

91.4 The distinct-store local rule

The distinct-store construction follows the same method with two arbitrary trees and two distinct store components. Its unique comparison head again satisfies the head predicate. Here `replaced` is the graph component of $E(G, e, v, \text{false})$. These definitions select the original local rule's head and Boolean; they do not merge stores with equal contents.

Lean: `AtomEquality.Different.head`

Lean: `AtomEquality.Different.head_at`

Lean: `AtomEquality.Different.replaced`

The vertex maps retain frame vertices and the separately tagged vertices of both trees. All other internal cases are empty. Restoring and deleting these tags proves the inverse laws. The edge maps retain both instances, both trees and both store occurrences. Only the comparison head is replaced; the three new indices map to result and argument weakenings. Exhaustive finite-index cases prove the edge maps inverse, while the retained-head case is excluded by its inequality proof.

Lean: `AtomEquality.Different.vertexTo`

Lean: `AtomEquality.Different.vertexFrom`

Lean: `AtomEquality.Different.vertices`

Lean: `AtomEquality.Different.edgeTo`

Lean: AtomEquality.Different.edgeFrom

Lean: AtomEquality.Different.edges

The frame-port equation again fixes inputs, outputs and retained frame vertices. Separate left-tree and right-tree equations reduce boundary ports to that frame equation and internal ports to their own tagged vertex map. Thus neither tree can be confused with the other, even when their labels or presentations coincide.

Lean: AtomEquality.Different.map_frame

Lean: AtomEquality.Different.map_tree

Lean: AtomEquality.Different.map_right_tree

The full graph isomorphism checks labels and incidence lists using both tree-port equations, and checks the three replacement edges directly. It extends to the actual successor state because the resumed input token is fixed. The statement compares complete successors, so retaining the second store and all its attachments is part of the conclusion.

Lean: AtomEquality.Different.replacement_iso

Lean: AtomEquality.Different.canonical_local

91.5 Independence of every actual contextual match

An actual rewrite at an atom-equality head supplies a store-identity decision. Apply the already proved recognition theorem to that rewrite; source-head uniqueness forces its recognised operation to be atom equality. The other operation constructors, including each arithmetic code, are incompatible. The remaining comparison recognition contains the required Boolean decision. No decision premise is added to the final coherence theorem.

Lean: Combined.rewrite_equality_decision

To reflect a comparison into the changed component of a context, first reflect the unique head using the actual inner rewrite and its match embedding. Obtain an inner decision from that rewrite, transport it to the host, and compare it with the given host decision. Uniqueness of store resolution makes the Booleans equal. Thus the reflected head, its edge image and the decision are all justified without assuming globally injective boundary attachment.

Lean: Combined.rewrite_equality_reflect

For a local rewrite, compare its distinguished head with the specified head. All non-comparison constructors contradict the operation label. In the same-store and distinct-store constructors, head uniqueness identifies the edge and decision uniqueness identifies the Boolean. The explicit local state isomorphism from the preceding subsections then applies.

Lean: Combined.local_equality_canonical

Induct over the actual contextual rewrite derivation. The local case is settled above. In the renaming case, reflect the head and decision, apply the induction hypothesis, and compose with canonical renaming and the supplied renaming of the actual output. For plugging, reflect into the filler, use the induction hypothesis there, and compose replacement associativity with the lifted state isomorphism. For simultaneous expansion, reflect into the selected component; replacement/expansion interchange and equality of the other components yield the complete result. Every constructor of the actual rewrite relation is covered.

Lean: Combined.rewrite_equality_canonical

Equality successor coherence says that any two actual rewrites from the same graph and request vertex, at an atom-equality head, have recursively isomorphic successor states. Derive the Boolean from the first rewrite; canonicality sends both successors to the same $E(G, e, v, b)$. Invert one isomorphism and compose with the other. This allows independently chosen hosts, tree presentations, context decompositions and output representations. It assumes neither machine determinism nor agreement of those choices.

Lean: `Combined.equality_rewrite_deterministic`

In the full step relation, a source in rewrite mode cannot take a search step. Eliminating those cases reduces step coherence to rewrite coherence. For the unbounded allowance-indexed machine, each step lifts a bounded step at the same allowance, so bounded coherence gives represented successor states. The existing transport of executions along representation then gives successor equivalence for every cost preorder, including exact equality. This is equivalence of the successor executions; it is not yet the contextual equivalence asserted by Proposition 12.2.

Lean: `Combined.equality_step_deterministic`

Lean: `Combined.equality_unbounded_deterministic`

Lean: `Combined.equality_unbounded_equiv`

91.6 Exact costs and arbitrary hosts

For an admitted comparison step $s \to t$, successor coherence gives

$$\mathsf{TerminatesIn}(n, s) \iff \exists m,\ n = m + 1 \ \wedge\ \mathsf{TerminatesIn}(m, t).$$

A zero-step successful run would make s final, contradicting its step. For a positive run, compare its first successor with t by coherence and transport the remaining run at the same cost. Conversely, prepend the chosen comparison step. The generic cost lemma of the allocation milestone encapsulates this argument.

Lean: `Combined.equality_cost_spectrum`

Serial attachment of a graph to a continuation preserves every actual search interaction, with the token carried by the left insertion map. This follows from the proved simultaneous-expansion search law, selecting the left component of the serial pattern.

Lean: `Search.serial_left`

Attach arbitrary store contents to either comparison fragment. Map its head into the serial composition and lift the original local compute step. Full step coherence shows that every competing successor is isomorphic to this explicit successor. More generally, place the fragment in an arbitrary typed host edge, with equality casts for the matching interfaces. Head transport and contextual step transport give the same conclusion. There is no hypothesis that the inserted input remains an external input, so the host may resume execution after the comparison returns.

Lean: `AtomEquality.Same.contents_successor`

Lean: `AtomEquality.Different.contents_successor`

Lean: `AtomEquality.Same.host_successor`

Lean: `AtomEquality.Different.host_successor`

For serially attached contents, the only successful comparison cost is seven. There are five preparatory search interactions: enter the left argument, return from its instance, move to

the right argument, return from its instance, and request comparison. There is then one compute step and one return from the Boolean constant. The final input is external in this serial composition. Apply search-cost inversion five times, the comparison cost law once, and search-cost inversion once more; the final state has only zero successful cost. Finite arithmetic gives exactly seven and also constructs the run of that length. No stability, termination or acyclicity assumption about the stored contents is used. The statement excludes every alternative successful cost under the full combined relation, rather than merely exhibiting a seven-step local run.

Lean: `AtomEquality.Same.contents_exact_cost`

Lean: `AtomEquality.Different.contents_exact_cost`

91.7 Unchanged translated contents, nested boxes and padding

Successful costs in the unbounded machine are precisely the successful costs of a fixed bounded representative. Reflect an unbounded finite run to its original allowance, preserving the endpoint and finality; conversely, lift a bounded run. This general lemma is used in both regression families.

Lean: `Combined.successful_unbounded_iff`

The first closed regression graph attaches the unchanged translation of an arbitrary closed term t to a same-store comparison with a noncanonical binary atom tree. The second attaches two unchanged copies of that same translation to two distinct stores. It uses a tensor of the translated contents and one canonical one-input atom tree per store. Serial and tensor label-preservation laws, together with the existing translation theorems, prove focus freedom and hole freedom, so these are valid closed programs. The comparison redexes are explicitly constructed local graph scenarios; no claim identifies them with the whole translation of a surface comparison expression.

Lean: `EqualityCoherenceExamples.sharedContents`

Lean: `EqualityCoherenceExamples.distinctContents`

Each regression has exactly the successful cost seven in the unbounded machine after any finite amount of depth padding. First use exact padding reflection, then the bounded/unbounded successful-cost correspondence, then the appropriate arbitrary-contents theorem. The serial input map fixes the external input, giving the original initialization state. Separately, map the local store decision into the serial graph: the shared-store result is true, and the distinct-store result is false despite identical contents.

Lean: `EqualityCoherenceExamples.shared_exact_cost`

Lean: `EqualityCoherenceExamples.distinct_exact_cost`

Lean: `EqualityCoherenceExamples.shared_decision`

Lean: `EqualityCoherenceExamples.distinct_decision`

Specialise both cost results to the unchanged translation of $\Omega = (\lambda x.xx)(\lambda x.xx)$. Comparison terminates without executing the stored application or opening its boxes; this does not prove Ω's divergence. The arbitrary-term results also cover deeper nested boxes.

Lean: `EqualityCoherenceExamples.omega_contents`

92 Assignment successor coherence and exact costs

The assignment case of Appendix B.1's successor-coherence obligation is now proved for the actual context-closed combined machine. Any two writes from the same request produce recursively isomorphic complete successor graphs, with matching resumed token positions. This extends the earlier uniqueness of the selected store to uniqueness of the whole update. Allocation and both reference-equality branches were completed previously. Section 93 subsequently completes dereference and unifies all store-operation comparisons. Section 94 completes arithmetic; Sections 95 and 96 complete application and copying successor comparison.

The operational rule remains Figure 19(c), described in HTML Section 10.1. It traces the reference through an arbitrary atom contraction tree, reconnects the existing store to the stable new value, weakens the old contents and the consumed instance, and returns unit. The paper explicitly allows the new value to refer to the same store, potentially forming a cycle. The new contextual proof assumes neither acyclicity of the surrounding graph nor global injectivity of its boundary attachments.

The central proof construction exchanges two target endpoints, then replaces only the assignment head by unit and two weakenings. It is an auxiliary normal form for comparing the original reducts, not an additional transition rule. Neither Proposition 12.2's terms nor Figure 16's translation changes. The stable-value premise is retained in every actual local assignment step. The structural comparison is more general than that premise, which is useful for transporting it through arbitrary contexts.

The dependency chain is: unique assignment match; typed target exchange; commutation with renaming, plugging and expansion; an explicit isomorphism to Figure 19(c); induction over actual rewrites; successor coherence and exact successful-cost laws. Four broad Appendix B.1 packages remain open. The mathematical-effort estimate remains 50–65%, with low confidence, since the remaining machine laws and the eleven-template argument are substantial.

92.1 Target exchange as a typed graph construction

For elements a, b, x of a type, let $\tau_{a,b}(x)$ be b if $x = a$, a if $x = b$ but $x \neq a$, and x otherwise. The left, right and other-element lemmas state these three actions. The right equation also covers $a = b$, by first splitting that equality. Applying the exchange twice returns x: split $x = a$, $x = b$, and the remaining case, and use those equations. Thus the exchange supplies both directions of a bijection, even when the selected elements coincide.

Lean: `TargetSwap.exchange`

Lean: `TargetSwap.exchange_left`

Lean: `TargetSwap.exchange_right`

Lean: `TargetSwap.exchange_other`

Lean: `TargetSwap.exchange_involutive`

Lean: `TargetSwap.permutation`

For a map f, exchange commutes with f at x provided $f(x) = f(a)$ implies $x = a$ and $f(x) = f(b)$ implies $x = b$. These two restricted injectivity conditions suffice: split the same three cases, and in the remaining case use the conditions to exclude both image equalities. This formulation is crucial because an inserted graph's entire boundary map need not be injective.

Lean: `TargetSwap.exchange_map`

More generally, let p be a type-preserving permutation of a graph's ports that preserves the predicate of being an external input. The *target reindexing* of G retains its internal and edge carriers, labels and all source lists, and replaces each ordered target list T_e by $p(T_e)$. Finiteness and source laws are inherited. Injectivity of p gives target non-repetition and uniqueness. For coverage of a non-input port v, use the original coverage of $p^{-1}(v)$ and map that incidence forward. Preservation of input status supplies its side condition. Type preservation of p proves target typing. Identity reindexing is equality of graphs, by the identity list-map law and proof irrelevance.

Lean: `TargetSwap.reindex`

Lean: `TargetSwap.reindex_refl`

Exchanging two non-input ports preserves input classification: both selected ports remain non-inputs and all others are fixed. If their wire labels agree, the same three-way case split proves preservation of every port's label. The *target-exchange graph* $S_{a,b}(G)$ instantiates reindexing with these proofs. It moves only target incidences; it is not a renaming of both ends of each wire.

Lean: `TargetSwap.input_iff`

Lean: `TargetSwap.typed`

Lean: `TargetSwap.net`

92.2 Transport through arbitrary attachments

A recursive isomorphism $f : G \cong H$ induces $S_{a,b}(G) \cong S_{f(a),f(b)}(H)$ using the same carrier bijections and label proofs. Sources are unchanged. For targets, compose the original target correspondence with the exchange-map lemma; the port bijection supplies its two injectivity conditions. Recursive box labels remain related by the original isomorphism.

Lean: `TargetSwap.rename`

A target of a retained host edge cannot equal the image of a non-input port of an inserted component. Such an equality would make that port an output of the component attached to a target of the replaced host edge. Target uniqueness in the original host would then identify the retained edge with the replaced edge, a contradiction. Also, target exchange inside the component does not change its attachment map: on internal, input and output ports this follows directly from the definition of plugging.

Lean: `TargetSwap.host_target_ne`

Lean: `TargetSwap.insert_plug`

Consequently, exchanging the two inserted ports in $C[G]$ is isomorphic to $C[S_{a,b}(G)]$, with identity carrier bijections. Retained host targets avoid both selected ports by the preceding lemma. Inserted targets satisfy the exchange-map equation, using injectivity restricted to non-input ports; every actual target is a non-input port. Sources agree by the attachment-map lemma. This proves commutation with arbitrary typed plugging without a global boundary-injectivity or acyclicity premise.

Lean: `TargetSwap.plug`

For simultaneous expansion by a family F, choose one component c. The component permutation exchanges a, b in $F(c)$ and is the identity in every other component. Its input and

typing laws follow by splitting whether the component is c. Reindexing each $F(e)$ with this permutation defines a family P with exactly the original carrier types. At c it is the exchanged graph, and elsewhere it equals $F(e)$ by identity reindexing.

Lean: `TargetSwap.componentPermutation`

Lean: `TargetSwap.component_input`

Lean: `TargetSwap.component_typed`

Lean: `TargetSwap.family`

Lean: `TargetSwap.family_same`

Lean: `TargetSwap.family_other`

The insertion maps of F and P agree on every port, by the three port cases. Exchanging the selected inserted endpoints in the expanded graph is therefore isomorphic to expanding P, again by identity carrier bijections. For targets within c, apply restricted injectivity and the exchange-map lemma. Targets in another component cannot hit the selected ports: uniqueness of the component owning a non-input target would force that component to be c. Those targets are fixed. Source and label equations are inherited.

Lean: `TargetSwap.insert_family`

Lean: `TargetSwap.expand`

92.3 Unique write data determined by the source

An *assignment match* at request port v records an assignment head e, a store s, an instance wire i, new-value and old-value ports n, o, and a store-trace length k. Its evidence says: v is a source of the assignment head, s has the store label, the ordered target lists are $T_e = [i, n]$ and $T_s = [o]$, and operand zero resolves through the instance and atom tree to s in k contractions. It records source facts, not a choice of result or a replacement for the stable-value condition.

Lean: `Assignment.Match`

Any incidence-preserving match embedding transports all these data and proofs. Ordered target lists transport componentwise; store resolution uses the previously proved operand transport theorem. Two matches at the same request are equal. Source-head uniqueness gives the same head; uniqueness of operand resolution gives the same store and trace length. Equality of the two ordered head target lists identifies i, n, and equality of the singleton store target lists identifies o. The remaining fields are proofs, so proof irrelevance completes equality of the records.

Lean: `Assignment.Match.map`

Lean: `Assignment.Match.unique`

Both n and o occur as targets, hence neither is an external input. Target typing of the assignment and store labels shows both carry term wires.

Lean: `Assignment.Match.new_not_input`

Lean: `Assignment.Match.old_not_input`

Lean: `Assignment.Match.values_typed`

These facts define the swapped graph $S_{n,o}(G)$ by the typed non-input target-exchange construction.

Lean: `Assignment.Match.swapped`

The assignment head and store are distinct because their primitive labels differ. The instance wire differs from n by non-repetition of the head's target list, and differs from o by target uniqueness for the distinct head and store.

Lean: `Assignment.Match.head_ne_store`

Lean: `Assignment.Match.instance_ne_new`

Lean: `Assignment.Match.instance_ne_old`

The exchanged head now targets $[i, o]$, and the same store now targets $[n]$. The exchange equations and the two instance inequalities prove these formulas. Every other edge retains its full ordered target list: if one of its targets were n or o, target uniqueness would identify it with the head or store. The two indexed head-target lemmas extract i and o from $[i, o]$. They provide the precise attachment ports used by replacement.

Lean: `Assignment.Match.swapped_head_target`

Lean: `Assignment.Match.swapped_store_target`

Lean: `Assignment.Match.swapped_other_target`

Lean: `Assignment.Match.swapped_first`

Lean: `Assignment.Match.swapped_second`

The *canonical assignment successor* of a match m is

$$W(G, m, v) = R(S_{n,o}(G), e, D_{()}, v),$$

where $D_{()}$ is the previously defined unit-and-two-weakenings graph and R is the replacement state. Its filler is transported to the head's interface using the assignment label. Search resumes at the retained image of v. The two weakenings now attach to i and o, whereas the original store attaches to n. No new store is allocated; the old contents persist behind their weakening.

Lean: `Assignment.Match.filler`

Lean: `Assignment.Match.canonical`

Every original local assignment source supplies this match for any atom contraction tree and any new-value graph. The head and store are their actual expanded edge occurrences; the instance and new-value wires are the first two frame ports and the old-value wire is the first output. Labels and ordered targets are the defining incidence equations. The existing store-resolution theorem supplies a finite trace and its length; classical choice selects its witness.

Uniqueness ensures no later result depends on this selection. No stability premise is needed merely to describe these source incidences.

Lean: `Assignment.localMatch`

92.4 Canonical updates commute with context formation

Renaming commutes with the canonical successor: first transport the target exchange through the recursive isomorphism, then apply replacement renaming. The filler boundary casts compose to the cast for the renamed match. The graph isomorphism carries the old request port to the new one, giving a state isomorphism, not only a graph comparison.

Lean: `Assignment.Match.canonical_rename`

Plugging commutes with the canonical successor by combining target-exchange locality with the previously proved replacement-plugging isomorphism. The attachment-map equation identifies the resumed token. Thus applying the canonical construction to the embedded match gives the same state, up to recursive isomorphism, as inserting its canonical successor into the host.

Lean: `Assignment.Match.canonical_plug`

For simultaneous expansion, suppose F and K agree away from c, and $W(F(c), m, v)$ is isomorphic as a state to the specified result in $K(c)$. First commute the target exchange into the selected component, giving the family P. Its selected member is isomorphic to the swapped $F(c)$ and its other members equal those of K. Replacement's expansion theorem then inserts the given component result. Equality of insertion maps identifies the token. This version permits the component successor to have an independent carrier representation, as actual contextual rewrites do.

Lean: `Assignment.Match.canonical_expand_result`

92.5 Explicit comparison with the original local reduct

The abbreviation `replaced` denotes the graph of the canonical successor for the original local source. The forward internal-vertex map removes the additional sum tag introduced by head replacement, keeping all four frame vertices and every tree and value-component vertex. The inverse restores that tag. Primitive components and the replacement filler have no internal vertices, so their cases are empty. Case analysis proves both composites are identities and packages the maps as a bijection.

Lean: `Assignment.Replacement.replaced`

Lean: `Assignment.Replacement.vertexTo`

Lean: `Assignment.Replacement.vertexFrom`

Lean: `Assignment.Replacement.vertices`

The edge map retains each instance, tree, store and value occurrence. The removed assignment occurrence is impossible in the retained-edge subtype. The three filler edges are sent respectively to the unit, discarded-instance weakening and discarded-old-value weakening of the original reduct. The inverse restores these three finite indices and the retained-edge tags. Enumeration

of the three indices and the one-edge primitive components proves the two inverse laws, giving the edge bijection.

Lean: `Assignment.Replacement.edgeTo`

Lean: `Assignment.Replacement.edgeFrom`

Lean: `Assignment.Replacement.edges`

The vertex bijection carries every retained frame port to its corresponding port in the original reduct. For tree vertices, internal ports are immediate and boundary ports reduce to that frame equation. The new definition `beforeValueVertex` is the actual insertion map of the original value component, including its boundary. The analogous value-map lemma has the same proof by internal/input/output cases. Finally, every retained non-head, non-store edge has its original targets mapped into the replacement host, because the target exchange fixes its target list.

Lean: `Assignment.Replacement.map_frame`

Lean: `Assignment.Replacement.map_tree`

Lean: `Assignment.Replacement.beforeValueVertex`

Lean: `Assignment.Replacement.map_value`

Lean: `Assignment.Replacement.retained_target`

These bijections define a complete graph isomorphism to Figure 19(c)'s original reduct. Vertex and edge labels are preserved by cases. Tree and value source lists use their insertion-map lemmas, and the other sources are immediate. For targets, retained instance, tree and value edges use the unchanged-target lemma; the store uses its new singleton target $[n]$. The unit has no targets. The two weakenings use the indexed exchanged-head ports i, o. Thus both ordered incidence relations agree for every edge, not just at the store. The request input is fixed by the vertex map, so this lifts to a state isomorphism with the original resumed search state.

This structural isomorphism allows any value graph. Actual operational application still requires that graph to be stable. In this language a stable local payload contains only passive primitive edges; box bodies can be attached beyond its thunk outputs in the host. The proof retains the entire payload and all recursively labelled surrounding components.

Lean: `Assignment.Replacement.iso`

Lean: `Assignment.Replacement.canonical_local`

92.6 From local comparison to every actual rewrite

For a local transition with an assignment match, inspect its rule constructor. Every non-assignment constructor has a different head label, contradicting source-head uniqueness. In the assignment case, match uniqueness identifies the given match with the original local match, and the explicit state isomorphism applies. A parallel inversion proves that an actual local step at an assignment head supplies a match. This is obtained from the rule, rather than assumed as a matching oracle.

Lean: `Combined.local_assignment_canonical`

Lean: `Combined.local_assignment_match`

Every actual contextual assignment rewrite supplies a source match. Induct on the rewrite derivation. At a local step use local inversion. For renaming, plugging or simultaneous expansion, reflect the distinguished head into the rewritten component, apply induction, and transport the match out through the corresponding incidence-preserving embedding.

Lean: `Combined.rewrite_assignment_match`

The same induction compares every actual successor with the source match's canonical successor. At each contextual constructor, reflect the head and extract the inner match. Its transported match equals the supplied outer match by uniqueness. Apply induction in the component and then the canonical renaming, plugging or expansion theorem. Compose with the result renaming where the rewrite explicitly permits an independently represented target. No common contextual decomposition of two rewrites is assumed.

Lean: `Combined.rewrite_assignment_canonical`

Assignment successor coherence says that two actual rewrites from the same assignment request have recursively isomorphic complete successors, preserving mode and token. Extract a match from the first rewrite, compare both results with its canonical successor, and compose one comparison's inverse with the other. The full one-step result follows because intrinsic search cannot start in rewrite mode; hence both steps must be rewrites. This is a theorem for one operation family, not an assumption of global machine determinism.

Lean: `Combined.assignment_rewrite_deterministic`

Lean: `Combined.assignment_step_deterministic`

92.7 Quantitative consequences and unbounded execution

If s is an assignment request and $s \to t$ is any actual step, then

$$s \text{ terminates in } n \quad \iff \quad \exists m,\ n = m + 1 \ \wedge\ t \text{ terminates in } m.$$

The general coherent-step cost theorem applies because every alternative successor is state-isomorphic to t. Isomorphisms preserve exact finite runs and finality. Thus the statement concerns all successful costs, not merely one chosen run.

Lean: `Combined.assignment_cost_spectrum`

For closed unbounded states, each step is a lift of a bounded step. Destruct the two lifts, use bounded successor coherence, and lift the resulting state isomorphism to the proved

representation relation. Representation invariance then gives observational equivalence of the successors for every cost preorder, including equality of costs. No finite-depth cutoff is added.

Lean: `Combined.assignment_unbounded_deterministic`

Lean: `Combined.assignment_unbounded_equiv`

A stable new value returns at its actual inserted input in one intrinsic search step. This transports the existing stable-input return theorem through the local simultaneous expansion. With arbitrary contents attached to the old-value output and the new value's remaining outputs, every actual write successor agrees with the original updated graph and resumed token. Apply full assignment-step coherence to the actual serially embedded local write; its head is transported by the serial match embedding.

Lean: `Assignment.value_returns`

Lean: `Assignment.contents_successor`

The entire local write, with these arbitrary attached contents, has exactly the successful cost seven. Five search steps visit the instance, return from it, visit the stable new value, return from it, and request assignment. One compute step updates the store, and one search step returns unit at the original input. Search determinism supplies an exact cost equivalence at each search step; assignment coherence supplies it at the compute step. The resulting state is final, whose only successful cost is zero. Substituting these seven equivalences proves both existence of the seven-step run and exclusion of every other successful cost. No evaluation of the old contents or the attached thunk bodies is required.

Lean: `Assignment.contents_exact_cost`

The arbitrary-host theorem inserts the original source and target into any single typed hole, transporting their interfaces along the specified equalities. Compare an arbitrary actual step with the explicitly plugged local write using assignment-step coherence. This preserves the complete host and permits the resumed request to be an internal wire; it does not assume that every such host is final or terminates in seven steps.

Lean: `Assignment.host_successor`

92.8 Unchanged-translation regression scenarios

The closed program `storedTerm` places the unchanged Figure 16 translation of an arbitrary closed term t in the old cell and supplies a stable constant as the new value. It uses the depth of that translation; serial composition and the previously proved safety of both components establish focus and hole freedom. The exact-cost theorem gives successful cost seven for every finite padding. Its proof first reflects padded runs using the established exact padding theorem, identifies bounded and unbounded successful runs, then instantiates the arbitrary-contents cost law. These are heap-state scenarios, not an identification with the translation of an entire surface-language assignment expression.

Lean: `AssignmentCoherenceExamples.storedTerm`

Lean: `AssignmentCoherenceExamples.storedTerm_exact_cost`

The program `translatedLiterals` retains the full original translations of both the old and new natural-number literals, including their distributors. Safety follows from their proved

stability. The same padding and bounded/unbounded correspondences prove that its complete set of successful costs is precisely $\{7\}$ for any pair of literals.

Lean: `AssignmentCoherenceExamples.translatedLiterals`

Lean: `AssignmentCoherenceExamples.translatedLiterals_exact_cost`

Specializing the first scenario to the literal Ω translation proves that overwriting this old data terminates in exactly seven steps at every allowance. It does not assert a separate divergence theorem for Ω. Finally, every actual successor of the translated-literal write is isomorphic to the explicit updated heap, by the arbitrary-contents successor theorem. The earlier attachment equations identify the existing store with the new literal and the old weakening with the old literal; the two retained occurrences remain distinct. These checks concern the full combined machine, not the earlier restricted local assignment machine.

Lean: `AssignmentCoherenceExamples.overwrite_omega`

Lean: `AssignmentCoherenceExamples.translatedLiterals_successor`

93 Dereference coherence and the complete store interface

Every actual contextual read now has a unique complete successor up to recursive state isomorphism. Together with the preceding allocation, assignment and equality results, this proves successor coherence for all four store-operation heads, including both equality branches. The proof uses the original Figure 19(b) rule, described in HTML Section 10.1. It retains the existing store and atom tree, consumes the instance by weakening, and connects the store and request to the exact distributor $D^{\mathrm{term}}_{1,2}$, including its weakening and two contractions.

The key quantitative conclusion is a continuation law. With arbitrary contents attached, a read succeeds in n transitions if and only if $n = 5 + r$ and the resulting copying request succeeds in r transitions. In particular, the five-step read prefix is not a completed evaluation. This strengthens the earlier chosen local trace to a statement about every successful execution of the full combined machine, including all contextual matching choices.

The dependency chain is: unique source read match; a finite read graft; typed target exchange; commutation with contexts; explicit isomorphism to the original reduct; induction over actual rewrites; successor coherence; exact cost decomposition. The graft is solely a proof construction. It does not add a transition or change the terms of Proposition 12.2, the translation of Figure 16, or any stability condition in the operational rules.

This completes the store-operation part of Appendix B.1's successor obligation. Section 94 subsequently completes arithmetic. Sections 95 and 96 complete application and copying successor comparison; the complete machine's refocusing and specimen laws remain open. The four broad packages stay open. The mathematical-effort estimate remains roughly 50–65%, with low confidence: these proofs do not complete the template argument or the final observational equivalence.

93.1 A finite graft for the exact distributor

The *read graft* is a graph with one term input q, one term output i, three fresh internal term ports a, b, c and four edges. Its first two edges are weakenings and its last two are contractions. The ordered source and target lists are

edge	sources	targets
0	[]	$[i]$
1	[]	$[b]$
2	$[b, c]$	$[a]$
3	$[a, q]$	$[c]$.

Edge zero will discard the consumed instance. Edges one to three are the literal $D^{\mathrm{term}}_{1,2}$ pattern, temporarily joining its first input to its output at a. That temporary feedback is removed by target exchange with the store-content port before the graph is compared with a read result. No claim of copyability or stability is made for the intermediate graft.

Lean: `ReadStub.label`

Lean: `ReadStub.source`

Lean: `ReadStub.target`

Every index of the four-element edge type is one of $0, 1, 2, 3$, by its finite bound. Enumerating these cases verifies absence of repeated ports in each list and uniqueness of each source and target incidence. For coverage, q, a, b, c each occur once as a source and i, a, b, c each occur once as a target; i is not a source and q is not a target. All labels are term, so the listed arities

also verify typing. These facts construct the finite primitive graph. Relabelling its primitive edges at any depth d gives the corresponding hypernet, with the same carriers and incidences.

Lean: `ReadStub.fin_four`

Lean: `ReadStub.base`

Lean: `ReadStub.net`

93.2 State-level transport of target exchange

A non-input port remains a non-input under any incidence-preserving match embedding: target coverage supplies an edge targeting it, the embedding transports that incidence, and no target in the receiving graph can be an external input. This argument applies even when boundary attachment maps are not globally injective.

Lean: `TargetSwap.embedded_not_input`

The state-level target exchange $S_{a,b}(G, v)$ consists of the exchanged graph and the search token at the unchanged port v. Only target incidences are exchanged, so moving the token by the exchange would be incorrect. If two graphs are equal, transporting their ports along that equality preserves both the non-input conditions and equality of endpoint labels. Eliminating the equality reduces each assertion to its original proof. The same elimination gives the state isomorphism back to the original exchanged state.

Lean: `TargetSwap.state`

Lean: `TargetSwap.cast_not_input`

Lean: `TargetSwap.cast_typed`

Lean: `TargetSwap.state_cast`

Suppose a recursive graph isomorphism maps a, b, v to a', b', w. The earlier target-exchange renaming theorem supplies an isomorphism of the exchanged graphs, and the given token equation supplies the state isomorphism. For plugging, use the earlier exchange-plugging graph isomorphism. Its carrier map is the identity and its insertion-map equation identifies the token. Thus exchanging the embedded endpoints in the whole graph agrees with inserting the exchanged component state.

Lean: `TargetSwap.state_congr`

Lean: `TargetSwap.state_plug`

For simultaneous expansion, let F and K agree except at the selected component. Assume that exchanging the selected endpoints in $F(e)$ gives a state isomorphic to the specified state in $K(e)$. First commute exchange through the expansion, obtaining the family P which exchanges only $F(e)$. At e, the family definition identifies $P(e)$ with that exchanged graph; elsewhere P agrees with F, hence with K. Apply expansion congruence to the given component state isomorphism. The insertion-map equation tracks the token through both comparisons. Other components may contain arbitrary recursive boxes and cycles.

Lean: `TargetSwap.state_expand_result`

93.3 Unique source data and the canonical read successor

A *dereference match* at a port v records a dereference head e, a store s, its content port o, and an atom-trace length k. Its evidence says that v is a source of e, s has the store label and singleton target $[o]$, and operand zero of the dereference resolves to s in k contractions. This is a description of source incidences and resolution, not an assumed result or matching oracle.

Lean: `Dereference.Match`

Match embeddings transport the head, store and content port, preserving the trace length. Ordered target transport preserves $[o]$, and operand transport preserves store resolution. Matches at the same source request are unique: source-head uniqueness identifies the heads; uniqueness of operand resolution identifies the stores and trace lengths; the singleton target equations identify the content ports. Proof irrelevance identifies the remaining evidence. Because o is a store target, it is not an input; the store's target-typing law gives it type term.

Lean: `Dereference.Match.map`

Lean: `Dereference.Match.unique`

Lean: `Dereference.Match.contents_not_input`

Lean: `Dereference.Match.contents_typed`

The match's filler is the read graft transported to the head's exact interface using its primitive label. Replacing just e by this filler defines the grafted graph. Its fresh port is the graft's internal port zero, and its retained port is the host image of o. The fresh port is internal by construction. The retained port is non-input by the original property and the three cases of the host port map. Both ports have type term: this is immediate for the graft port, and follows from preservation of host labels for the content port. Thus target exchange is well typed.

Lean: `Dereference.Match.filler`

Lean: `Dereference.Match.grafted`

Lean: `Dereference.Match.fresh`

Lean: `Dereference.Match.retained`

Lean: `Dereference.Match.fresh_not_input`

Lean: `Dereference.Match.retained_not_input`

Lean: `Dereference.Match.endpoints_typed`

The *canonical read successor* is the search state obtained by exchanging the targets at those fresh and retained ports in the grafted graph, retaining the old request token. The store, previously targeting o, now targets a; the distributor root, previously targeting a, now targets o. Its ordered inputs therefore connect the store and resumed request to the original stored contents. The separate weakening connects to the consumed instance.

Lean: `Dereference.Match.canonical`

Every original local read source supplies a match. Its head and store are their explicit expanded occurrences; the content port is the source graph's single term output. The earlier finite store-resolution theorem supplies a trace length and operand proof. Classical choice selects this witness, while match uniqueness proves that its choice cannot change the canonical successor.

Lean: `Dereference.localMatch`

93.4 Commutation with renaming and contexts

A recursive isomorphism of source graphs induces an isomorphism of their grafted graphs. Use the carrier bijections supplied by host plugging. On retained edges, labels and incidence lists obey the original isomorphism. On the four inserted edges, labels are identical; the insertion-map equation, applied portwise to each ordered list, proves both incidence equations. Boundary casts are checked separately on internal, input and output ports. The fresh port maps to the fresh port, and host-port naturality transports both the content port and the token. State-level exchange congruence now proves renaming compatibility of the canonical successor.

Lean: `Dereference.Match.graft_rename`

Lean: `Dereference.Match.canonical_rename`

For plugging, associativity of replacement first identifies grafting at the embedded head with inserting the grafted component. Its middle-component port equation identifies the retained content port and request token, and the fresh graft port maps directly. The two embedded endpoints remain non-inputs, and insertion preserves their labels. State-level exchange congruence followed by exchange-plugging gives the desired canonical-state comparison for any typed host.

Lean: `Dereference.Match.canonical_plug`

For simultaneous expansion, replace the head only in the selected component of a family F, giving a family P. Its selected graph equals the grafted component, so equality transport supplies its endpoint conditions and token. The expansion-plugging isomorphism compares whole-graph grafting with expansion by P; its inserted-port and component-port equations identify the fresh endpoint, retained endpoint and token. Apply exchange congruence, then the state-level exchange-expansion theorem with the given component successor comparison. This allows the new family K to use an independently represented successor, as the actual rewrite relation requires.

Lean: `Dereference.Match.canonical_expand_result`

93.5 The explicit isomorphism to Figure 19(b)

The abbreviation `replaced` names the graph of the canonical successor for the original local source. Its forward internal-vertex map retains the three old frame vertices and every atom-tree vertex. The graft's fresh vertex becomes the fourth frame vertex of the original reduct, and its other two vertices become the distributor's two internal vertices.

Lean: `Dereference.Replacement.replaced`

Lean: `Dereference.Replacement.vertexTo`

The inverse separates these cases and restores the replacement tags. Primitive components have no internal vertices. Enumerating the finite frame and graft indices proves that the two maps are inverse bijections.

Lean: Dereference.Replacement.vertexFrom

Lean: Dereference.Replacement.vertices

The edge map retains the instance, store and every atom-tree occurrence. The deleted head is excluded by the retained-edge subtype. Graft edge zero becomes the discarded-instance weakening; graft edges one, two and three become respectively the distributor's weakening and its two contractions. The inverse restores the finite indices and retained-edge tags. Enumerating the four graft indices and the one-edge primitive components proves both inverse equations and gives the edge bijection.

Lean: Dereference.Replacement.edgeTo

Lean: Dereference.Replacement.edgeFrom

Lean: Dereference.Replacement.edges

The vertex bijection preserves every atom-tree port's attachment. Internal tree ports and the single output follow directly from the definitions; for inputs, split the first input from the remaining atom-interface inputs and reduce the respective frame attachments. Moreover, target exchange fixes every such tree port: retained host ports cannot be the fresh graft port, and none of the tree's attached internal, input or output ports is the external content port. These observations also cover noncanonical atom trees with any number of additional references at their boundary.

Lean: Dereference.Replacement.map_tree

Lean: Dereference.Replacement.exchange_tree

These bijections preserve all labels and ordered incidence lists, giving an isomorphism of complete graphs. Tree sources use the attachment equation; tree targets additionally use the fact that exchange fixes them. The instance target is unchanged. The store target moves from the old content port to the fresh port. In the graft, the distributor root target makes the opposite move, whereas the discarded-instance weakening, distributor weakening and inner contraction targets are fixed. All remaining source and label cases are the displayed finite incidences. The request input is fixed, so this graph isomorphism lifts to a state isomorphism with the original resumed search state. This establishes correspondence with the unchanged reduct, not merely agreement at the store endpoint.

Lean: Dereference.Replacement.iso

Lean: Dereference.Replacement.canonical_local

93.6 Coherence of every actual contextual read

For a local step admitting a dereference match, inspect the rule constructor. Every non-read constructor has a different active head label, contradicting source-head uniqueness. In the read case, uniqueness identifies the given match with the original local match, and the explicit isomorphism applies. The parallel local inversion theorem extracts a match from any actual local step at a dereference head, using the same constructor analysis.

Lean: `Combined.local_dereference_canonical`

Lean: `Combined.local_dereference_match`

Inducting over actual rewrite derivations extends match extraction to all contextual reads. At a local step use inversion. Under recursive renaming, plugging or simultaneous expansion, reflect the active head into the rewritten component, extract its match by induction, and transport it outward through the corresponding match embedding. No assumption of unique decomposition or a pre-existing deterministic matcher is used.

Lean: `Combined.rewrite_dereference_match`

A second induction compares each actual read successor with the canonical successor of its source match. At a contextual constructor, extract the inner match and identify its outward transport with the supplied match by uniqueness. Apply induction inside the component and then the appropriate canonical renaming, plugging or expansion theorem. In the renaming case, compose with the separately specified target isomorphism. Thus independent contextual decompositions are compared through one source-determined state.

Lean: `Combined.rewrite_dereference_canonical`

Given two reads at the same request, extract a match from the first and compare both targets with its canonical successor. Composing one comparison's inverse with the other gives a complete recursive state isomorphism between the targets. For full-machine steps in rewrite mode, intrinsic search is impossible, so both steps must be rewrites and the same theorem applies. The generic coherent-step cost law then shows that one actual read step consumes exactly one transition and leaves the same full spectrum of future successful costs, independently of matching choices.

Lean: `Combined.dereference_rewrite_deterministic`

Lean: `Combined.dereference_step_deterministic`

Lean: `Combined.dereference_cost_spectrum`

An unbounded step is a lift of a bounded step. Destructing two such lifts and applying bounded coherence gives the proved representation relation between their successors. Representation invariance gives observational equivalence at every cost preorder, including equality of costs. These statements impose no finite-depth cutoff and no global determinism premise.

Lean: `Combined.dereference_unbounded_deterministic`

Lean: `Combined.dereference_unbounded_equiv`

93.7 All store operations under one theorem

The predicate `StoreHead` holds when the selected request is an allocation, dereference, assignment or atom-equality head. Its four constructors retain the actual head-incidence evidence. Equality includes both the same-cell and distinct-cell operational rules. It asserts the kind of head, not that every such source admits a rewrite.

Lean: `Combined.StoreHead`

Store successor coherence says that any two actual rewrites from one of these heads have recursively isomorphic complete successors, including the resumed token. Case analysis on `StoreHead` applies the four independently proved family theorems. The full-machine step theorem follows by the same case analysis. This closes the store-operation portion of successor coherence while keeping existence and stable-payload conditions in the original operational rules.

Lean: `Combined.store_rewrite_deterministic`

Lean: `Combined.store_step_deterministic`

The generic coherent-step theorem now yields a uniform cost law: if $s \to t$ is any actual store computation, then s terminates in n exactly when $n = 1 + r$ and t terminates in r. Finally, destructing unbounded lifts transfers coherence to represented states, and representation invariance gives successor equivalence for every cost preorder. These are genuine consequences of the proved store comparisons; they do not supply the still then-missing application, arithmetic or copying comparisons.

Lean: `Combined.store_cost_spectrum`

Lean: `Combined.store_unbounded_deterministic`

Lean: `Combined.store_unbounded_equiv`

93.8 Exact cost of the read prefix with arbitrary contents

Attach any graph H to the local source's content output. Transport the read head through serial composition and compare an arbitrary actual step with the serially embedded original local read. Dereference coherence shows that every successor is isomorphic to the explicit result retaining H. Neither termination, stability nor acyclicity of H is assumed.

Lean: `Dereference.contents_successor`

Let s be this composed graph with a search token at its request, and let p be the composed original reduct with a rewrite token at that request. The *read cost handoff* is

$$s \text{ terminates in } n \iff \exists r,\ n = r + 5 \ \wedge\ p \text{ terminates in } r.$$

Its five transitions are: search the instance argument; return from the instance; request dereference; perform the store-preserving read rewrite; request copying at the new distributor. Apply the exact search-cost theorem at the four search transitions and the dereference cost theorem at the computation. Eliminating the five intermediate natural numbers gives the displayed equation; conversely, inserting $r+4, r+3, r+2, r+1, r$ constructs all five predecessor costs. This is a full-machine equivalence for all successful costs, not an existence-only local execution.

Lean: `Dereference.contents_cost_handoff`

The five-step prefix itself always exists: embed the previously proved local read run into the combined machine, then transport it through serial attachment. No successful execution can have cost at most five. If it did, the handoff equation would force the residual cost to be zero. A zero-step successful run would make p final, but its mode is rewrite, whereas finality requires backtrack at an input. This proves that the mandatory pending copy has not been erased, even for literal data.

Lean: `Dereference.contents_prefix`

Lean: `Dereference.contents_no_early_success`

Successor coherence also holds after inserting the read into any typed host hole, transporting interfaces along their equalities. Transport the head through rebounding and plugging, then compare an arbitrary step with the explicitly plugged original local read. This permits internal resumed request wires. The fixed five-step cost statement above concerns the serial content attachment; an arbitrary surrounding host can perform further work.

Lean: `Dereference.host_successor`

93.9 Unchanged translated data and depth padding

The closed program `storedTerm` places the literal Figure 16 translation of any closed term t at the read source's content output. Its depth is the translation depth of t; focus and hole freedom follow from the safety of the read source, translation safety and serial composition. The state `pending` is the explicit original read reduct with the same translation attached and its token in rewrite mode. These are heap-state regressions; no identification with a whole surface-language read translation is asserted.

Lean: `DereferenceCoherenceExamples.storedTerm`

Lean: `DereferenceCoherenceExamples.pending`

For every finite depth padding and every proposed successful cost, the unbounded initial execution satisfies exactly the read cost handoff to `pending`. First use the proved padding-invariance theorem to reflect successful costs to the original allowance. Then use the bounded/unbounded successful-run correspondence and instantiate the arbitrary-contents handoff. Lifting the serial prefix also proves an actual five-step unbounded run to the specified pending state. The same zero-residual-cost contradiction proves that no padding permits a successful cost at most five.

Lean: `DereferenceCoherenceExamples.storedTerm_cost_handoff`

Lean: `DereferenceCoherenceExamples.storedTerm_prefix`

Lean: `DereferenceCoherenceExamples.storedTerm_no_early_success`

Specializing the general cost law to a natural-number literal retains its complete translation and copying continuation. Specializing it to the literal self-application term Ω retains its nested boxes as stored data; this proves no separate divergence or termination result for Ω. Finally, arbitrary-successor coherence specializes to every translated payload, preserving the entire attached graph and resumed token under every actual contextual read.

Lean: `DereferenceCoherenceExamples.literal_cost_handoff`

Lean: `DereferenceCoherenceExamples.omega_cost_handoff`

Lean: `DereferenceCoherenceExamples.storedTerm_successor`

94 Arithmetic coherence and the remaining machine cases

The original arithmetic rules now have complete successor-state coherence under every constructor of the actual contextual rewrite relation. This covers unary natural negation, addition and truncated subtraction. A second result exhausts the computation alphabet: any pair of successors is recursively isomorphic, or the original state requests application or copying. At that milestone these were the two remaining computation families for successor coherence. Sections 95 and 96 subsequently complete application and copying, establishing complete successor determinism up to representation. Refocusing and robustness remain separate obligations.

Arithmetic differs from the completed store comparisons because it removes both operand edges and their internal wires. The proof records the surviving occurrences in the original source, derives this record from every actual rewrite, and compares two results through their common source. No deletion certificate is added to the operational premises. The binary rule remains Figure 10(a), and unary negation remains the agreed natural-number operation $-n = 0 - n = 0$. The terms of Proposition 12.2, Figure 16's translation, and the stability conditions of other operations are unchanged.

Every successful unary redex execution has exactly five transitions, and every successful binary redex execution has exactly seven, even after arbitrary finite depth padding. These statements quantify over all successful runs of the full machine. A binary redex attached below an arbitrary host returns control in seven transitions, after which the host's own successful cost is retained. The nested arithmetic regression has exactly eleven successful transitions. These are explicit graph redexes, not simplified replacements for complete surface-term translations.

The dependency chain is: unique source recognition; survivor certificates; renaming and context transport; certificates for the original local rules; induction over actual rewrites; whole-state coherence; exact cost laws; and exhaustive classification of the remaining heads. Appendix B.1 removes arithmetic from its outstanding successor comparisons. Its four broad work packages remain open, and the mathematical-effort estimate stays at roughly 50–65%, with low confidence.

94.1 Surviving vertices and edges

A head has *internal arithmetic targets* when each port in its target list is an internal vertex of the source graph. This property prevents arithmetic deletion from removing a host boundary wire after insertion. It is proved for the original redexes and transported through all rewrite contexts; it does not assume global acyclicity or injectivity of boundary attachment.

Lean: `Arithmetic.InternalTargets`

Fix a source graph G, its arithmetic head e, and the numeric result r. A *source-relative deletion certificate* for a successor H consists of injective maps $j_V : V_H \to V_G$ on internal vertices and $j_E : E_H \to E_G$ on edges, with exact images

$$\operatorname{im}(j_V) = \{x \mid \operatorname{internal}(x) \notin \operatorname{tgt}_G(e)\},$$
$$\operatorname{im}(j_E) = \{a \mid \neg \operatorname{Adjacent}_G(e, a)\}.$$

Here adjacency means that a target of e is a source of a. The port map $\widehat{j_V}$ fixes external inputs and outputs and acts by j_V internally. Internal labels and every ordered source list are preserved. An edge mapped to e has numeric label r and empty target list. Every other edge has a recursively equivalent label and the same ordered targets after mapping by $\widehat{j_V}$. Thus the certificate describes the entire successor, including all retained boxes; it is stronger than

identifying the numeric result alone. The local redex proofs below show that the head survives and precisely its numeric operands and operand wires disappear.

Lean: Arithmetic.Reduction

Two injective maps with the same image determine a bijection between their domains: send an element to the unique element with the same image. Classical choice selects the preimage; injectivity proves both inverse equations. The commuting equation says that applying the second embedding after this bijection recovers the first embedding. Applying this construction separately to j_V and j_E gives the vertex and edge bijections between two certified successors. Their commuting equations follow directly; the port equation adds the input and output cases, where both maps are the identity.

Lean: Arithmetic.Reduction.sameRange

Lean: Arithmetic.Reduction.sameRange_commute

Lean: Arithmetic.Reduction.vertexBijection

Lean: Arithmetic.Reduction.edgeBijection

Lean: Arithmetic.Reduction.vertex_commute

Lean: Arithmetic.Reduction.edge_commute

Lean: Arithmetic.Reduction.port_commute

These bijections form a recursive graph isomorphism. Internal labels agree through their common source labels. At the arithmetic head both edge labels are the same numeric constant; elsewhere compose the two recursive label comparisons through G. For incidences, map both candidate lists into G, use their certificate equations and the commuting equations, then cancel the injective port map. The head's two target lists are both empty. If the two resumed tokens also map to the same source port, the port commuting equation and injectivity identify them. Both states are in search mode, so this upgrades the graph isomorphism to a complete state isomorphism.

Lean: Arithmetic.Reduction.iso

Lean: Arithmetic.Reduction.stateIso

94.2 Renaming the source and successor

Port mapping respects composition of internal-vertex maps, by the three port constructors. Mapping an entire list respects the same composition, by list-map composition and the pointwise port equation. These elementary laws control all incidence equations under renaming.

Lean: Arithmetic.Reduction.port_comp

Lean: Arithmetic.Reduction.map_ports_comp

To re-present the successor by an isomorphism $K \cong H$, compose its carrier bijections with H's maps into G. Surjectivity leaves the images unchanged, and composition preserves injectivity. Recursive label comparison transports the numeric head label exactly and all other labels relationally. The incidence laws compose; an empty mapped target list implies the original list

is empty. Thus the certificate is preserved under arbitrary recursive successor renaming.

Lean: `Arithmetic.Reduction.targetRename`

To re-present the source by $f : G \cong N$, compose the certificate maps with f and replace the selected head by $f(e)$. Target membership reflects through the inverse vertex bijection, and adjacency reflects through the inverse edge bijection. These facts transfer the exact-image conditions. Labels and incidences then compose with the corresponding isomorphism laws. The two port-transport lemmas state the resulting composition equations explicitly, so the resumed token can subsequently be transported without rebuilding this argument.

Lean: `Arithmetic.Reduction.sourceRename`

Lean: `Arithmetic.Reduction.targetRename_port`

Lean: `Arithmetic.Reduction.sourceRename_port`

94.3 Deleting an arithmetic redex inside a plugged graph

Let $C[h := G]$ be a typed plugging. An inserted component port can equal an internal vertex of that component only if it was already that internal vertex: inputs and outputs attach to host ports, whose carrier tags are disjoint from component-internal tags. When the selected head has internal targets, its plugged target vertices are exactly the inserted original target vertices. It therefore cannot be adjacent to a retained host edge, and adjacency to an inserted edge is equivalent to adjacency inside G. The forward direction reflects the shared internal port using the preceding disjointness fact; the reverse direction uses incidence transport. Internal-target closure itself survives plugging because insertion preserves component-internal vertices.

Lean: `Arithmetic.Reduction.inserted_internal`

Lean: `Arithmetic.Reduction.plug_target`

Lean: `Arithmetic.Reduction.plug_adjacent_host`

Lean: `Arithmetic.Reduction.plug_adjacent_inner`

Lean: `Arithmetic.Reduction.plug_closed`

The whole successor's vertex map is the identity on host vertices and the component certificate map on inserted vertices. Mapping an inserted port this way agrees with first mapping it in the component and then inserting it into the original plugged graph. Internal ports are immediate; boundary ports follow by considering the host port to which they attach. The host-port equation is simply the identity in each of the three port cases.

Lean: `Arithmetic.Reduction.plug_port`

Lean: `Arithmetic.Reduction.plug_host`

Define the whole edge map likewise: retain each host edge and use the component edge map inside the hole. Disjoint carrier tags and component injectivity prove injectivity. The preceding target and adjacency lemmas give the exact vertex and edge images: every host occurrence survives, and the component removes precisely its original operand occurrences. Host labels are unchanged; component labels use the original certificate. Ordered incidence lists commute by the two port equations. This constructs a deletion certificate for the whole plugging, without imposing a common host decomposition on two independently chosen arithmetic matches.

Lean: `Arithmetic.Reduction.plug`

94.4 Simultaneous expansion and dependent component carriers

Suppose two filling families F and K agree at every host edge except h, where a certificate compares $F(h)$ with $K(h)$. Equality of two hypernets induces a recursive isomorphism in the reverse direction by eliminating the equality. For each component, define its vertex and edge maps to be the certificate maps when its index is h, and the equality-induced isomorphism maps otherwise. At h these definitions reduce to the certificate maps. The whole expansion maps preserve the host vertices and every component index, using these maps within the corresponding dependent component carriers.

Lean: `Arithmetic.Reduction.equalIso`

Lean: `Arithmetic.Reduction.fiberVertices`

Lean: `Arithmetic.Reduction.fiberEdges`

Lean: `Arithmetic.Reduction.fiber_vertices_same`

Lean: `Arithmetic.Reduction.fiber_edges_same`

Lean: `Arithmetic.Reduction.expansionVertices`

Lean: `Arithmetic.Reduction.expansionEdges`

Mapping a port inserted from any component commutes with inserting its component image. Internal ports retain their component index; boundary ports attach to the same host port on both sides. A port inserted at index e can be an internal vertex tagged by e only when it was that internal vertex. The selected head's expanded targets are therefore exactly its own tagged internal targets. Adjacency within the selected component reflects and preserves adjacency there. Adjacency to another component is impossible: internal ports have different tags, and boundary attachments lie in the host. In particular the selected head still has only internal targets.

Lean: `Arithmetic.Reduction.expansion_port`

Lean: `Arithmetic.Reduction.expansion_internal`

Lean: `Arithmetic.Reduction.expansion_target`

Lean: `Arithmetic.Reduction.expansion_adjacent_same`

Lean: `Arithmetic.Reduction.expansion_adjacent_other`

Lean: `Arithmetic.Reduction.expansion_closed`

These facts construct the expansion certificate. Each component map is injective, by the given certificate at h and by an isomorphism elsewhere. Preserved indices give injectivity of the whole maps. In the selected component the image conditions are exactly the certificate conditions; every other component is covered by the inverse isomorphism, and every host vertex survives. No other component is adjacent to the selected head. Internal labels, recursively equivalent edge labels, and incidences follow component by component; the port commutation law handles attachment. This proof also accounts for unchanged components containing arbitrary boxes or cycles.

Lean: `Arithmetic.Reduction.expand`

94.5 Certificates for the original numeric rules

A two-element index is either zero or one, by its finite bound. Enumerating the serial redex carriers shows that every unary internal vertex is its sole operand wire and every binary internal vertex is one of its two ordered operand wires. Primitive numeric edges have no internal vertices of their own. Thus all internal vertices occur in the head's target list, and every head target is internal. Enumerating the two unary edges, or the three binary edges, shows that the head is adjacent exactly to the other edges. The head is not adjacent to itself, because its source is the external input.

Lean: `Arithmetic.fin_two`

Lean: `Arithmetic.unary_internal_deleted`

Lean: `Arithmetic.binary_internal_deleted`

Lean: `Arithmetic.unary_internal_targets`

Lean: `Arithmetic.binary_internal_targets`

Lean: `Arithmetic.unary_adjacent_iff`

Lean: `Arithmetic.binary_adjacent_iff`

A general local construction now suffices. Suppose a closed redex has all internal vertices targeted by its head, the head is adjacent precisely to all other edges, and its source list is the singleton external input. The result corolla has no internal vertices, so its vertex map is the empty map. Map its unique edge to the old head. The vertex image is empty by the first hypothesis; the edge image is exactly the head by the second. The singleton source and empty target list satisfy the incidence requirements, and the result label is numeric by construction. Applying this construction to unary and binary redexes gives the original arithmetic rules' certificates, with results $0 - n$ and $\mathrm{eval}(o, m, n)$ respectively.

Lean: `Arithmetic.result_reduction`

Lean: `Arithmetic.unary_reduction`

Lean: `Arithmetic.binary_reduction`

94.6 Recognized arithmetic requests and actual contextual rewrites

An *arithmetic request* is either unary negation with operand n, or a binary code o with ordered operands m, n. Its recognized machine choice has the same data; its result evaluates the indicated natural-number operation. The map to machine choices is injective, because the constructors and their fields distinguish the requests.

Lean: `Arithmetic.Request`

Lean: `Arithmetic.Request.choice`

Lean: `Arithmetic.Request.result`

Lean: `Arithmetic.Request.choice_injective`

A complete arithmetic certificate additionally specifies the source head, recognition of its primitive label at the source token, internal-target closure, a deletion certificate, and the

equation mapping the resumed successor token back to the original source token. Internal-target closure is invariant under recursive isomorphism: reflect target membership, use internality in the source, then map the internal vertex forward.

Lean: `Arithmetic.Certificate`

Lean: `Arithmetic.internalTargets_rename`

For renaming, transport the head by the match embedding, rename the source and then the successor of the deletion certificate, and cancel the successor isomorphism with its inverse in the token equation. For plugging, use the plugged deletion certificate and head, together with the inserted-port commutation law. For expansion, use the expanded certificate and the component-port equation at h. In each case the resumed token is tracked through the same carrier map as the graph. The original unary and binary rules supply complete certificates using the explicit local reductions; both tokens are the external input, so their token equation is reflexive.

Lean: `Arithmetic.Certificate.rename`

Lean: `Arithmetic.Certificate.plug`

Lean: `Arithmetic.Certificate.expand`

Lean: `Arithmetic.unary_certificate`

Lean: `Arithmetic.binary_certificate`

Recognition can be reflected into the component of an actual rewrite even though a general match embedding need not be surjective. The component's actual rewrite has some recognized choice by the earlier selection theorem. Transport that choice to the host and compare it with the specified host recognition. Uniqueness of source recognition identifies the choices, leaving the required component recognition. The actual-rewrite premise is essential to this argument.

Lean: `Combined.rewrite_recognition_reflect`

For a local rewrite recognized as arithmetic, inspect the original rule constructor. In the unary and binary cases, recognition uniqueness identifies the requested operation and ordered operands, so the corresponding local certificate applies. Every store, application or copying constructor has a different source-head label. Head uniqueness therefore excludes it; binary codes are split into addition and subtraction where needed to distinguish assignment and atom equality. No existing stability or copyability premise is erased. Inducting over the actual rewrite relation now derives certificates for every arithmetic computation: the local case is just proved, while renaming, plugging and expansion use recognition reflection and certificate transport.

Lean: `Combined.local_arithmetic_certificate`

Lean: `Combined.rewrite_arithmetic_certificate`

Take any two actual arithmetic rewrites from the same source request. Their derived certificates have the same head by source-head uniqueness and the same result by the fixed recognized request. The survivor-image theorem therefore gives a recursive graph isomorphism; their token equations give the state isomorphism. This is *arithmetic successor coherence*. It compares complete successors produced by arbitrary contextual matches, not only the selected redex fragments. A combined transition from rewrite mode cannot be a search transition, so the

result immediately covers the full combined step relation.

Lean: Combined.arithmetic_rewrite_deterministic

Lean: Combined.arithmetic_step_deterministic

An arithmetic transition consumes exactly one unit of successful cost. For any chosen actual successor t, a successful source cost n is equivalent to $n = 1 + k$ and a successful cost k from t. Any competing first successor is isomorphic to t, and the previously proved execution-isomorphism law transports its remaining run without changing cost. Lifting the state isomorphism proves representation equivalence of arbitrary unbounded successors; representation equivalence then yields observational equivalence at every cost preorder. These results do not assume global machine determinism.

Lean: Combined.arithmetic_cost_spectrum

Lean: Combined.arithmetic_unbounded_deterministic

Lean: Combined.arithmetic_unbounded_equiv

94.7 An exhaustive description of the remaining coherence cases

The historical predicate *higher-order head* records either an application edge or a term-contraction edge at the current token. At the arithmetic milestone these were the two families awaiting complete successor comparison; application is subsequently discharged in Section 95; the predicate does not assert that either family is nondeterministic.

Lean: Combined.HigherOrderHead

For two actual rewrites, obtain the recognized choice of the first one and inspect all eight choice constructors. Allocation, read, assignment and either comparison branch use the completed store-coherence theorems. Unary and binary arithmetic use the new theorem. The application and copy cases supply the corresponding higher-order head. Thus every rewrite pair is coherent or has one of these two heads. For full machine steps, a search first step has its unique literal successor, and mixed search/rewrite steps are impossible. The remaining case is the rewrite classification. Consequently any possibly unresolved pair must also have rewrite mode.

Lean: Combined.rewrite_coherent_or_higherOrder

Lean: Combined.step_coherent_or_higherOrder

If the source is not a rewrite request at one of these heads, eliminate the second alternative to obtain successor coherence outright. Applying the one-step cost theorem gives the corresponding exact cost decomposition. The same exhaustive disjunction lifts to the unbounded machine: fixed-depth coherent successors become representation-equivalent successors, and the head/mode alternative is unchanged. This identified the remaining proof obligation at the arithmetic milestone. Section 95 further reduces it to copying.

Lean: Combined.step_deterministic_away_higherOrder

Lean: Combined.cost_spectrum_away_higherOrder

Lean: Combined.unbounded_coherent_or_higherOrder

94.8 Exact costs, arbitrary hosts and regressions

Search in the right component of a serial graph is a genuine search step in the whole graph, with its token inserted at the actual shared interface. This follows from simultaneous-expansion search transport at the right-hand component of the two-edge serial pattern.

Lean: `Search.serial_right`

Unary evaluation takes three search transitions to reach its rewrite request, one arithmetic transition, and one return from the resulting number. Apply search cost decomposition at each search step and arithmetic cost decomposition at the compute step. The final external-input backtrack state has exactly zero remaining successful cost. Adding these equations gives $n = 5$; conversely the explicit remaining costs $4, 3, 2, 1, 0$ reconstruct a successful run. Binary evaluation has five search transitions before rewriting, then the same compute and return, giving $n = 7$ in both directions. These statements concern every successful run of the full machine.

Lean: `Arithmetic.unary_cost_iff`

Lean: `Arithmetic.binary_cost_iff`

For an arbitrary typed host hole, first cast the local redex boundary to the hole's signature and plug the original arithmetic transition into that hole. Its source head is still arithmetic by boundary transport and the match embedding. Head uniqueness excludes both higher-order alternatives, so the classification theorem compares any actual successor with this chosen one. This proves unary and binary host-successor coherence while retaining the entire host and its recursive boxes.

Lean: `Arithmetic.unary_host_successor`

Lean: `Arithmetic.binary_host_successor`

Exact padding reflection and exact finite/unbounded-run correspondence transfer the local five- and seven-step characterizations to every padded closed arithmetic redex program. The underflow regression combines the previously constructed subtraction result 0 for $m \leq n$ with the new all-run cost characterization. The negation regression similarly combines its exact result 0 with the five-step spectrum. Neither operation is replaced by an observationally assumed constant; the existing rule and execution proofs supply the stated results.

Lean: `Arithmetic.Examples.unary_pad_cost_iff`

Lean: `Arithmetic.Examples.binary_pad_cost_iff`

Lean: `Arithmetic.Examples.underflow_result_and_cost`

Lean: `Arithmetic.Examples.negation_result_and_cost`

The *arithmetic cost handoff* retains an arbitrary host C above a binary redex. A successful cost n at the redex's inserted input is equivalent to $n = 7 + k$ and a successful cost k from the inserted backtrack state of its numeric result. Transport the five search steps and final numeric return through serial composition, and use the embedded recognition and contextual arithmetic coherence at the computation step. Summing the seven one-step cost equations proves the forward implication; choosing the seven successive remaining costs proves the reverse implication. The returned token is internal to the host, so it is not mistaken for a final whole-program state. No termination assumption is made about the host continuation.

Lean: `Arithmetic.Examples.binary_host_handoff`

Finally consider unary negation applied to a binary arithmetic redex. One search step enters the inner redex. Its seven-step handoff changes the inner graph into the numeric result before the outer operation resumes. There remain one outer readiness step, one unary computation and one numeric return. The final external-input backtrack has cost zero. Composing these cost equivalences proves that the nested expression succeeds at exactly $1+7+1+1+1 = 11$ transitions. This regression checks actual graph change between the two arithmetic computations, not merely concatenation of abstract numbers.

Lean: `Arithmetic.Examples.nested_cost_iff`

95 Application coherence and the remaining copying case

Application now has complete successor-state coherence in the original combined copying machine. Two actual rewrites at the same application request produce recursively isomorphic complete graphs and corresponding resumed tokens. The proof includes arbitrary renaming, plugging and simultaneous expansion, and permits the two matches to describe the actual boxed body by different recursively isomorphic graphs. Together with the preceding store and arithmetic results, this milestone isolated copying as the sole remaining family for successor coherence. Section 96 now resolves that case and proves complete successor determinism up to representation. Refocusing, specimen locality and the template argument remain open.

The operational rule and its stability premise remain unchanged. Opening a lambda exposes its full padded body, preserves its ordered variable and atom attachments, and connects the first bound variable to the stable argument. It does not simplify the Figure 16 distributors. In particular, an original translated variable requests copying after beta; it does not immediately become its argument. The two terms in Proposition 12.2 are unchanged.

The quantitative result applies to every successful run of the full machine: application contributes exactly six transitions before the actual opened-body state. The same law holds under an arbitrary serial host. Constant and nested-lambda bodies have exactly seven successful transitions, while a variable body has seven plus the successful cost of its remaining copy request. No body-termination assumption occurs in the six-step law.

The proof dependency chain is: unique source frame and box body; an explicit presentation of all surviving and newly exposed occurrences; a realization of that presentation by the original beta reduct; transport through every rewrite constructor; comparison of independently represented bodies; whole-state coherence; exact costs; and the exhaustive copying-only classification. The presentations and certificates below are mathematical proof data derived from actual rewrites. They add no premise to the transition relation.

Appendix B.1 removes application from its outstanding successor comparisons. Its four broad work packages remain open. The current assessment after the subsequent copying milestone is in Section 96; the dependent template and final observational arguments are still unresolved.

95.1 Presenting the complete result

A *partial incidence presentation* P consists of ambient sets V_P and E_P, vertex and edge labels, predicates selecting the retained vertices and edges, and ordered source and target lists in the ambient port space. The ambient data need not itself be a well-formed graph: incidence lists for deleted occurrences are harmless proof data. This allows one space to name both the old source occurrences and all newly exposed body occurrences.

Lean: `Opening.Presentation`

A *realization* of P by a genuine hypernet H consists of injections $j_V : V_H \to V_P$ and $j_E : E_H \to E_P$ whose images are exactly the selected occurrences. It preserves internal wire labels, relates edge labels by the recursive relation `LabelEquiv`, and satisfies

$$\widehat{j}_V(\mathrm{src}_H(e)) = \mathrm{src}_P(j_E(e)), \qquad \widehat{j}_V(\mathrm{tgt}_H(e)) = \mathrm{tgt}_P(j_E(e)).$$

Port extension fixes the external input and output indices, and acts by j_V on internal vertices. The equations are equalities of ordered lists.

Lean: `Opening.Realization`

Two realizations of the same presentation determine bijections between their vertex sets and between their edge sets. Send an occurrence to the unique occurrence with the same ambient

image; exact image equality gives existence, and injectivity gives uniqueness and the inverse equations. The vertex and edge commuting lemmas express equality of these images. The port commuting lemma follows by cases on internal, input and output ports.

Lean: `Opening.Realization.vertexBijection`

Lean: `Opening.Realization.edgeBijection`

Lean: `Opening.Realization.vertex_commute`

Lean: `Opening.Realization.edge_commute`

Lean: `Opening.Realization.port_commute`

These bijections constitute a recursive graph isomorphism. Labels agree through their common ambient labels; recursive edge-label comparisons compose by symmetry and transitivity. To compare source or target lists, embed both lists in the presentation, use the realization equations and commuting laws, and cancel the injective port map. If the resumed tokens have the same ambient image, the vertex bijection also maps one token to the other; with both modes set to search this gives a state isomorphism.

Lean: `Opening.Realization.iso`

Lean: `Opening.Realization.stateIso`

A realization can be pulled back along any recursive isomorphism $K \cong H$. Compose its injections with the isomorphism's bijections. The inverse maps supply all required preimages, and composing the label and incidence laws proves that K realizes the same presentation.

Lean: `Opening.Realization.rename`

95.2 The source frame and its actual body

An *application source match* records three source edges: the application a, its lambda ℓ, and the lambda's box b. It also records internal function and thunk wires f, t, and the argument attachment q. The application head is at the requested token v, and the ordered incidences are

$$\mathrm{tgt}(a) = [f, q], \quad \mathrm{src}(\ell) = [f], \quad \mathrm{tgt}(\ell) = [t], \quad \mathrm{src}(b) = [t].$$

The lambda label is the actual primitive lambda. The box label is recursively related to the actual body B, with one bound term variable, k free term variables and h free atoms. Equality casts reconcile $k + 1$ with $1 + k$; they do not permute the ordered ports. Stability is required by the actual local transition, not duplicated in this auxiliary source-shape record.

Lean: `Application.Match`

Two matches of the same source and requested port have identical frame data. Unique head selection identifies their application edges. Comparing the two ordered target lists identifies the function wire and argument port. Unique source ownership identifies the lambda edge at that function wire; its target identifies the thunk wire, and source ownership again identifies the box edge. The actual label of this unique box then relates the two candidate bodies. The relation `BoxRelated` fixes their binder/interface metadata and supplies a recursive isomorphism of the full bodies.

Lean: `Application.Match.shape_unique`

Lean: `Application.Match.bodies_related`

The box signature proves that the list $q :: \mathrm{tgt}(b)$ has length $1 + |\Gamma| + |\Delta|$. This is exactly the

opened body's output interface. The body-port map sends a body internal vertex to a new occurrence, its unique input to the old request port v, and output i to entry i of this list. Thus output zero reaches the argument; the remaining outputs retain the original box's ordered free term and atom attachments.

Lean: `Application.Match.output_length`

Lean: `Application.Match.bodyPort`

The presentation for a match has ambient sets

$$V_G + V_{\mathrm{open}(B)}, \qquad E_G + E_{\mathrm{open}(B)}.$$

It retains every old vertex except f, t, every old edge except a, ℓ, b, and every newly exposed body vertex and edge. Old occurrences retain their labels and ordered incidence lists; body occurrences use their actual labels and the body-port map just defined. Here $\mathrm{open}(B)$ is precisely the existing depth-raised, interface-cast body used by the original reduct.

Lean: `Application.Match.presentation`

95.3 The original beta reduct realizes the presentation

The local source match selects the application, lambda and box in the existing `before` graph. Its fields reduce directly to the source construction. The local realization maps the reduct's argument-connection wire to the original argument wire, its body internals to new occurrences, and its argument internals to the corresponding old source occurrences. The edge map similarly sends body edges to new occurrences and argument edges to old occurrences.

Lean: `Application.localMatch`

Lean: `Application.openingVertices`

Lean: `Application.openingEdges`

The vertex and edge maps are injective by disjointness of the tagged summands and injectivity within each summand. Their exact-image lemmas are stronger than injectivity: every old occurrence except the two frame wires and three frame edges has a preimage, and every body occurrence has a preimage. The proofs enumerate the finite frame indices and the host/body summands; no assumption about the body's size, termination or absence of nested boxes is used.

Lean: `Application.opening_vertices_injective`

Lean: `Application.opening_edges_injective`

Lean: `Application.opening_vertex_range`

Lean: `Application.opening_edge_range`

The serial-output lemma states that an output port of a right serial component remains the corresponding output of the serial composite. Applying it through the local construction proves that the actual box's target list is the free environment ports, in order. The body-port comparison then checks internal, input and output cases; at output zero it obtains the argument connection, and at successor indices it obtains the free environment port. The argument-port

comparison follows the same cases using the existing argument embedding and right-serial output law.

Lean: `Application.right_serial_output`

Lean: `Application.local_box_targets`

Lean: `Application.opening_body_port`

Lean: `Application.opening_argument_port`

The complete local realization combines the injections and exact images. For a body edge, the source and target equations follow by mapping the body-port comparison over its ordered lists. For an argument edge, use the argument-port comparison instead. Labels are exactly the construction's labels, including every nested body box. This proves that the original `after` graph realizes the full source-relative presentation.

Lean: `Application.opening_realization`

95.4 Occurrences and changes of presentation

An occurrence map consists of maps on ports, internal vertices and edges. The latter two maps are injective, internal ports map to their indicated internal occurrences, wire labels are preserved, edge labels are recursively related, and source and target lists commute with the port map. It does not require arbitrary boundary attachments to be injective.

Lean: `Opening.Occurrence`

Forgetting the additional recursive-label and internal-vertex information gives the existing primitive matching embedding. A recursive isomorphism gives an occurrence map by its bijections. Plugging gives one by the selected component summand; simultaneous expansion gives one by tagging each component occurrence with the selected edge index. Their label and incidence laws are those of the actual graph constructions.

Lean: `Opening.Occurrence.matchEmbedding`

Lean: `Opening.Occurrence.ofIso`

Lean: `Opening.Occurrence.plug`

Lean: `Opening.Occurrence.expand`

Mapping a source match along an occurrence map preserves all frame equations. Primitive head and lambda labels use the matching embedding, while the box comparison uses the full recursive label relation. The ambient origin-port map sends old occurrences through the occurrence map and keeps new body occurrences fixed; external ports follow the occurrence's boundary attachment. It commutes with embedding an old port. The body-port compatibility lemma holds internally and at the body input by definition, and at each output by mapping the ordered argument-plus-box-target list.

Lean: `Application.Match.map`

Lean: `Application.Match.originPort`

Lean: `Application.Match.origin_old`

Lean: `Application.Match.bodyPort_map`

An isomorphism of presentations consists of ambient vertex and edge bijections preserving the

retention predicates, labels and ordered incidence lists. Composing a realization with these bijections transports it to the new presentation. Inverses prove the exact-image conditions; the remaining conditions follow by composition.

Lean: `Opening.Presentation.Iso`

Lean: `Opening.Realization.transport`

For a recursively renamed source, the origin-port map is the port extension of the sum of its vertex bijection with the identity on new body vertices. This gives the source-renaming presentation isomorphism. Old label and incidence equations come from the source isomorphism; new body incidences use the body-port compatibility lemma.

Lean: `Application.Match.origin_iso`

Lean: `Application.Match.renamePresentation`

For two matches of one source, a recursive isomorphism between their opened bodies also induces a presentation isomorphism. It is the identity on old occurrences and the body isomorphism on new ones. The body-port comparison uses unique frame and argument data: internal ports follow the body bijection, inputs agree at v, and outputs use the identical ordered attachment list. Retention predicates agree because the removed frame occurrences are unique. Labels and ordered incidences then follow from the body isomorphism.

Lean: `Application.Match.bodyPort_congr`

Lean: `Application.Match.bodyPresentation`

95.5 Complete certificates and independent bodies

An *opening certificate* for $G, v \longrightarrow H, w$ records a source match, a realization of its presentation by H, and the token equation $\widehat{j}_V(w) = \mathrm{inl}(v)$. Thus it determines the full graph and where execution resumes. The original local rule supplies such a certificate using the local realization and its input-token equation.

Lean: `Application.OpeningCertificate`

Lean: `Application.local_certificate`

Equal interface casts can be removed from a recursive graph isomorphism: eliminate the interface equalities, after which the original isomorphism has the required type. To rename both sides of a certificate, transport its presentation along the source isomorphism and pull back its realization along the inverse successor isomorphism. Port-map composition and the successor inverse equation prove the renamed token equation.

Lean: `Application.cast_iso_reflect`

Lean: `Application.OpeningCertificate.rename`

Two opening certificates for the same source request are coherent even when their body graphs and interface metadata are supplied independently. The unique source box first identifies the metadata and relates the cast bodies. Remove the casts, raise the resulting recursive isomorphism through depth padding, and restore the opened-body interface cast. This gives the required isomorphism between the actual opened bodies. Transport one realization through its induced presentation isomorphism, then compare the two realizations of one presentation. Their token equations agree on the old source port, yielding the complete state isomorphism.

Lean: `Application.OpeningCertificate.coherent`

95.6 Transport through arbitrary plugging

The auxiliary shift map sends an old component occurrence to the old component summand of the host, and keeps a new body occurrence new: $A + B \to (C + A) + B$. It is injective. Its old- and new-image equivalences identify preimages in the corresponding summands, and its disjointness lemma excludes old host occurrences from its image. All four proofs split the sum constructors.

Lean: `Opening.shift`

Lean: `Opening.shift_injective`

Lean: `Opening.shift_old`

Lean: `Opening.shift_new`

Lean: `Opening.shift_not_host`

The plugged vertex and edge injections keep old host occurrences in their host summands and shift the selected component's certified images. The injectivity lemmas use host/component disjointness, shift injectivity and the original certificate's injectivity. The range lemmas split host, old component and new body occurrences: the host is retained in full, and the component and body cases reduce to the original exact-image equations.

Lean: `Application.OpeningCertificate.plugVertices`

Lean: `Application.OpeningCertificate.plugEdges`

Lean: `Application.OpeningCertificate.plug_vertices_injective`

Lean: `Application.OpeningCertificate.plug_edges_injective`

Lean: `Application.OpeningCertificate.plug_vertex_range`

Lean: `Application.OpeningCertificate.plug_edge_range`

The host-port lemma preserves the same host attachment under the new injection. The component-port lemma commutes with insertion and the ambient origin-port map. Both follow by port cases: internal component ports shift, while boundary ports use the host-port law. The source and target presentation lemmas treat old edges using the occurrence's list equations and new edges using the body-port compatibility law.

Lean: `Application.OpeningCertificate.plug_host_port`

Lean: `Application.OpeningCertificate.plug_component_port`

Lean: `Application.OpeningCertificate.plug_source`

Lean: `Application.OpeningCertificate.plug_target`

The plugged realization uses these maps and range laws. Host edge labels and incidences are retained; component labels come from the original realization, and their incidences follow by composing the component-port and presentation list equations. The resulting opening certificate uses the mapped source match; its token equation is the component-port equation applied to the original token equation. Thus arbitrary typed plugging preserves complete opening certificates.

Lean: `Application.OpeningCertificate.plugRealization`

Lean: `Application.OpeningCertificate.plug`

95.7 Transport through simultaneous expansion

The expansion proof first handles dependent occurrence sets. Fix component sets F_e, K_e, a selected index h, a map $K_h \to F_h + N$, and bijections $K_e \cong F_e$ for $e \neq h$. The tag map sends an old selected occurrence to its pair (h, x) and leaves a new occurrence in N. It is injective by disjointness and injectivity of dependent-pair construction.

Lean: `Opening.Fiber.tag`

Lean: `Opening.Fiber.tag_injective`

The lifted map on $\sum_e K_e$ applies the selected map and tagging at h, and the given bijection at every other index. Its same-index and other-index lemmas expose these two clauses. A selected tag cannot be an old occurrence at another index: equality of dependent pairs would equate their indices. The lifted map is injective by four cases on whether the two indices equal h. The two mixed cases use this disjointness; the selected case uses the selected injection; the other case uses index equality followed by the corresponding bijection's injectivity.

Lean: `Opening.Fiber.lift`

Lean: `Opening.Fiber.lift_same`

Lean: `Opening.Fiber.lift_other`

Lean: `Opening.Fiber.tag_not_other`

Lean: `Opening.Fiber.lift_injective`

The range at an old selected occurrence is exactly the range of the selected map at that old occurrence. The same is true for a new occurrence. In each forward direction, an index other than h is ruled out by its tag; the reverse direction supplies index h. Every old non-selected occurrence has a preimage obtained using its component bijection's inverse.

Lean: `Opening.Fiber.range_same`

Lean: `Opening.Fiber.range_extra`

Lean: `Opening.Fiber.range_other`

For actual expansion, the unchanged-family premise supplies a recursive isomorphism $K_e \cong F_e$ at each non-selected index. Apply the dependent construction separately to vertices and edges, using the opening realization at the selected index. Whole-graph vertices retain the outer host summand and shift the component fibers; whole-graph edges are just the edge fibers. Their injectivity follows from the dependent injection lemma and host/fiber disjointness.

Lean: `Application.OpeningCertificate.otherIso`

Lean: `Application.OpeningCertificate.fiberVertices`

Lean: `Application.OpeningCertificate.fiberEdges`

Lean: `Application.OpeningCertificate.expansionVertices`

Lean: `Application.OpeningCertificate.expansionEdges`

Lean: `Application.OpeningCertificate.expansion_vertices_injective`

Lean: `Application.OpeningCertificate.expansion_edges_injective`

The expanded range lemmas keep every outer host vertex and every occurrence of a non-selected component. At the selected component they reduce to its certificate's exact images; new body

occurrences use the new-range lemma. Distinct component indices ensure that none of the unchanged occurrences is mistaken for a deleted application, lambda, box, function wire or thunk wire.

Lean: Application.OpeningCertificate.expansion_vertex_range

Lean: Application.OpeningCertificate.expansion_edge_range

The host-port equation fixes host attachments. The selected-component port equation uses the certificate and its ambient origin map; the other-component port equation uses the corresponding recursive isomorphism. Internal cases use the appropriate dependent-map clause, and boundary cases reduce to the host-port equation. The expanded presentation's source and target laws then follow for old selected edges from the occurrence equations and for new body edges from body-port compatibility.

Lean: Application.OpeningCertificate.expansion_host_port

Lean: Application.OpeningCertificate.expansion_selected_port

Lean: Application.OpeningCertificate.expansion_other_port

Lean: Application.OpeningCertificate.expansion_source

Lean: Application.OpeningCertificate.expansion_target

The expanded realization combines these injections, images and port laws. At a selected edge it uses the original realization's recursive label and incidence equations. At every other edge it uses the unchanged component's recursive isomorphism. Its token equation follows from the selected port law and the original certificate. This constructs a certificate for every actual simultaneous-expansion rewrite, preserving all other component contents.

Lean: Application.OpeningCertificate.expansionRealization

Lean: Application.OpeningCertificate.expand

95.8 Derivation from the unchanged transition relation

An opening witness hides the actual opened box's depth, interface counts and body graph while retaining its full certificate. Two witnesses for one source request are coherent by unpacking their certificates and applying the independent-body comparison. This existential packaging permits induction over the machine's depth-indexed rewrite relation.

Lean: Application.OpeningWitness

Lean: Application.OpeningWitness.coherent

For an actual local transition recognized as application, the beta case supplies the original local certificate. Every other local constructor is excluded by unique head recognition: its primitive label differs from application. Stability is still present in the beta constructor. Induction over an arbitrary actual rewrite then produces its witness. The local case is just proved. For renaming, plugging and expansion, recognition reflects into the selected component, the induction hypothesis produces its witness, and the corresponding certificate transport constructs the result.

Lean: Combined.local_application_certificate

Lean: Combined.rewrite_application_certificate

Application successor coherence states: if a source request is at an application head and has two actual rewrite successors, those complete search states are recursively isomorphic. Derive a witness from each rewrite and compare them. For full machine steps, a search transition from rewrite mode is impossible, so both steps reduce to the rewrite theorem. No global determinism assumption, template law or body-termination assumption is used.

Lean: `Combined.application_rewrite_deterministic`

Lean: `Combined.application_step_deterministic`

For a chosen actual application step $s \to t$, the successful-cost spectrum satisfies $T_n(s) \iff \exists m\,(n = m + 1 \wedge T_m(t))$. A successful zero-step run cannot start at a state with a transition. Any nonempty successful run has a first successor isomorphic to t, and execution-isomorphism invariance transports its remaining cost; the reverse direction prepends the chosen step. Unbounded application steps lift finite-depth steps, so their successors are related by the existing representation relation. Representation equivalence then gives their machine equivalence for every cost order.

Lean: `Combined.application_cost_spectrum`

Lean: `Combined.application_unbounded_deterministic`

Lean: `Combined.application_unbounded_equiv`

95.9 The historical reduction to the copying case

The *copying coherence frontier* is an exhaustive theorem about the existing machine. Two actual rewrites either have recursively isomorphic successors or share a term-contraction head. Start from the earlier exhaustive classification into coherent, application or copying cases, and discharge the application case by the new theorem. For full steps, search successors are unique, rewrite/search overlap is impossible, and the rewrite classification applies otherwise.

Lean: `Combined.rewrite_coherent_or_copying`

Lean: `Combined.step_coherent_or_copying`

Consequently, excluding a rewrite-mode term-contraction request suffices for successor coherence and for the exact one-step successful-cost decomposition. The two corollaries resolve the copying alternative and apply the existing coherent-step cost lemma. The unbounded classification follows by unpacking both lifted steps and lifting the resulting state isomorphism. None of these results alone resolves the copying alternative; Section 96 supplies its complete proof.

Lean: `Combined.step_deterministic_away_copying`

Lean: `Combined.cost_spectrum_away_copying`

Lean: `Combined.unbounded_coherent_or_copying`

95.10 Exact application costs and host continuations

An *application cost handoff* is the exact law

$$T_n(\mathrm{before}(B,G),\mathsf{search},0) \iff \exists r\ (n = r+6\ \wedge\ T_r(\mathrm{after}(B,G),\mathsf{search},0)),$$

where G satisfies the original stability premise. Apply the exact search-cost law to the five pre-compute transitions, then application coherence to the beta transition. Collect the six successor equations by natural-number arithmetic. Conversely, prepend these six steps to any successful reduct run. Existentially forgetting the exact cost gives termination equivalence between the initial and opened-body states. In particular, failure of the reduct to terminate cannot be hidden by application.

Lean: `Application.opening_cost_iff`

Lean: `Application.opening_terminates_iff`

For an arbitrary typed plugged host, reboundary and plug the original beta step. The mapped source head remains application, so the step-coherence theorem compares every possible successor with this actual plugged reduct. For an arbitrary serial host, transport the five search steps and beta transition to their actual host attachments. Apply the same cost decomposition to obtain six plus the successful cost of the hosted reduct. The continuation may compute further, diverge or get stuck; its success is not presumed.

Lean: `Application.host_successor`

Lean: `Application.host_handoff`

95.11 Regressions with unchanged translated bodies

A translated constant with one unused parameter returns in one search step after opening. A translated nested lambda likewise returns in one search step without opening its inner box. Applying the six-step handoff and the exact zero-cost law at the final state proves that both initial applications have successful cost exactly seven, for every stable argument with the required interface. These are all-run statements in the full combined machine.

Lean: `Application.Examples.constant_cost_iff`

Lean: `Application.Examples.nested_cost_iff`

The translated variable retains its original $D_{1,1}$ distributor. Its next search transition after beta is a copy request, so its successful spectrum is seven plus that request's spectrum. If the total were at most seven, the remaining cost would be zero. A zero-step run cannot be successful at rewrite mode, proving the no-early-success regression. This deliberately leaves the actual remaining copying computation to the copying development.

Lean: `Application.Examples.variable_cost_iff`

Lean: `Application.Examples.variable_no_early_success`

A reference argument retains its atom attachment and satisfies the existing stable-instance lemma, so the constant-body result specializes to successful cost seven. Finally, the existing closed constant-body application program has this same exact cost in the unbounded machine after every finite depth padding: use the already proved initial-cost padding equivalence and exact finite/unbounded successful-run correspondence, then the constant cost theorem.

Concrete divergent callbacks and repeated closure use still require the remaining copying and template development; these results do not claim those end-to-end regressions.

Lean: `Application.Examples.reference_argument_cost_iff`

Lean: `Application.Examples.constant_pad_cost_iff`

96 Copying coherence and complete successor determinism

Copying now completes successor coherence for the original combined machine. For any two transitions $s \to t$ and $s \to u$, the entire successor states t and u are recursively isomorphic, including their token positions. The result covers independently chosen contraction-tree boundaries, all copyable payload families, recursive renaming, plugging, and simultaneous expansion. It lifts to the unbounded machine's representation relation. No determinism or certificate premise is added to the transition relation.

The construction leaves Proposition 12.2's terms, the Figure 16 translation, the original copyability condition, and every stability condition unchanged. An isomorphic normal presentation of a payload is used only inside the proof. Lambda bodies remain complete recursive graphs; their contents are neither executed nor replaced by an interface summary when copying.

Copying successor coherence means that all actual copying rewrites at one source request produce recursively isomorphic complete successor states. The key observation is that the literal reduct retains every original vertex and edge. It changes the selected source attachment to a fresh weakening wire and redirects the payload's ordered output attachments through the original distributors. One fresh payload copy and the distributor machinery are then added. Thus the result can be described relative to the entire source graph, without making the chosen tree boundary part of the description.

The proof does not assert that two independently chosen contraction trees are isomorphic or even have the same number of external inputs. Instead it proves that each literal local reduct, and then each contextual reduct, realizes the same source-relative incidence description, up to recursive payload isomorphism. This distinguishes the result from the earlier transport theorem, which assumed a supplied isomorphism of tree presentations.

The dependency chain is: payload normal forms and rigidity; unique active source frame and payload endpoint; a presentation retaining the whole source; a certificate for the literal reduct; transport through every constructor of `Rewrite`; comparison of two independently derived certificates; and complete successor determinism. Exact-cost consequences follow from the already proved transport of executions and finality along recursive isomorphism.

Appendix B.1 removes copying successor coherence and unbounded determinism up to representation from its outstanding obligations. Section 97 subsequently proves refocusing for copying and allocation. Refocusing for the remaining operations, specimen locality, the focused-graph correspondence, template arguments, semantic weakening and Figure 22 remain open. The estimated completion is roughly 55–70% of mathematical effort, with low confidence. This range is a judgement about the remaining proofs, not a fraction of declarations or completed list entries; Proposition 12.2 is still unproved.

96.1 Normal presentations of copyable payloads

A *copying payload normal form* is either a single primitive payload or the literal lambda edge followed by its boxed body. The datatype of plain codes distinguishes instance, constant, application, unary and binary operations. It retains the precise constant or operation parameter. The variable-interface sizes are respectively $0, 0, 2, 1, 2$; only an instance has an atom output, with atom-interface size one. The primitive map selects the original label, and the graph map constructs its original one-edge corolla at depth d. These definitions do not restrict copying to passive operations.

Lean: `CopyRule.PlainCode`

Lean: `CopyRule.PlainCode.variables`

Lean: CopyRule.PlainCode.atoms

Lean: CopyRule.PlainCode.primitive

Lean: CopyRule.PlainCode.net

The primitive map is injective: case analysis on two codes reduces label equality to equality of their constructors and parameters. No plain code has the lambda primitive. These facts will exclude a plain/lambda ambiguity at a shared payload head.

Lean: CopyRule.PlainCode.primitive_injective

Lean: CopyRule.PlainCode.not_lambda

The distinguished plain edge is the sole edge of its corolla. Its label and source incidence prove that it is the head at input zero. Its source list is exactly that singleton; its target list enumerates all environment outputs in order. The proofs inspect the depth representation and primitive constructor; none rearranges the boundary.

Lean: CopyRule.PlainCode.edge

Lean: CopyRule.PlainCode.head

Lean: CopyRule.PlainCode.source

Lean: CopyRule.PlainCode.target

The indexed family of normal forms records the graph just described, including the full body parameter in its lambda constructor. Every graph satisfying the existing `Copyable` predicate is recursively isomorphic to one of these forms. Induction on copyability is direct in its six generating cases; in a renaming case, compose the inverse supplied renaming with the induction hypothesis. The existence of a normal form is therefore derived from the original rule premise.

Lean: CopyRule.NormalForm

Lean: CopyRule.normal_form

For a normal form, the head is its primitive edge. The tail is that same edge in a plain payload and the box edge in a lambda payload. The recorded primitive is the plain label or lambda. The head is incident to input zero, its entire source list is that singleton, and the tail targets enumerate the environment outputs. For lambda, the latter equation follows through the right serial insertion of the box. Every normal form satisfies the original copyability predicate, by its corresponding constructor.

Lean: CopyRule.NormalForm.head

Lean: CopyRule.NormalForm.tail

Lean: CopyRule.NormalForm.primitive

Lean: CopyRule.NormalForm.head_at

Lean: CopyRule.NormalForm.head_source

Lean: CopyRule.NormalForm.tail_target

Lean: CopyRule.NormalForm.copyable

96.2 Rigidity of two payload occurrences

Occurrence embeddings compose by composing their port, internal-vertex and edge maps. Injection and label preservation compose, and the ordered incidence equations follow from the composition law for list mapping. The right component of a serial graph has its canonical occurrence embedding: internal vertices and edges receive the right-component tag, while boundary ports are attached by serial insertion. These maps are injective on internal vertices and edges; boundary ports are allowed to be attached by the context.

Lean: `Opening.Occurrence.comp`

Lean: `Opening.Occurrence.serialRight`

The relation `PayloadRelated` simultaneously asserts equality of the variable and atom interface counts and recursive isomorphism of the complete payloads. Its constructor uses the same counts on both sides, so elimination also identifies the dependent interface types.

Lean: `CopyRule.PayloadRelated`

A literal lambda payload has one internal thunk wire between its head and box tail. The head's target and tail's source are both the singleton containing this wire. The definition and equations follow directly from serial composition.

Lean: `CopyRule.NormalForm.lambdaWire`

Lean: `CopyRule.NormalForm.lambda_head_target`

Lean: `CopyRule.NormalForm.lambda_tail_source`

Suppose two normal payload occurrences in G have the same head edge. Their primitive labels agree. If both are plain, primitive injectivity identifies the codes and the payloads; the tail equality is the given head equality. A mixed plain/lambda case is impossible. If both are lambdas, the head's singleton target identifies the image of the thunk wire. Uniqueness of the edge consuming that wire identifies the box tails. Their recursive box-label relation then supplies equal interface counts and an isomorphism of the entire bodies. Serial composition with the common lambda corolla yields the payload isomorphism. This proves both `PayloadRelated` and equality of the actual tail-edge images, without assuming an agreement between the two body descriptions.

Lean: `CopyRule.NormalForm.related`

96.3 The uniquely determined source data

The selected input can be the first or second source of its binary contraction. The list `selectedSources` records this order with a Boolean: false means $[v, u]$, and true means $[u, v]$.

A *copying successor match* consists of a normal payload occurrence in G, the active contraction edge, this source-position Boolean, the other source port, and a proved copy-resolution trace from the selected port v to the payload head. The active edge has the term-contraction label and exactly the recorded source list. The match records no particular contraction-tree boundary. Its tail is the image of the normal payload's tail.

Lean: `CopyRule.selectedSources`

Lean: `CopyRule.SuccessorMatch`

Lean: `CopyRule.SuccessorMatch.tail`

The active-edge label and source list establish its head-at-v property. Source-list duplicate freedom proves that v and the other source differ. For two matches, source uniqueness first identifies the active edge. Equality of its two ordered source lists then identifies the Boolean and the other source. Opposite Boolean cases would repeat v, contradicting duplicate freedom.

Lean: `CopyRule.SuccessorMatch.head_at`

Lean: `CopyRule.SuccessorMatch.distinct`

Lean: `CopyRule.SuccessorMatch.frame_unique`

The two copy-resolution traces have the same source port, so the previously proved resolution uniqueness identifies their payload heads. Payload rigidity then identifies interface counts, relates the complete payloads recursively, and identifies their actual tails. The tail target list has exactly the environment length, since occurrence embeddings preserve incidence lists and the normal tail enumerates those outputs.

Lean: `CopyRule.SuccessorMatch.payload_unique`

Lean: `CopyRule.SuccessorMatch.output_length`

Any occurrence embedding of G into a larger graph transports this match. Compose the payload occurrence, map the active edge and source ports, preserve the Boolean, and transport the resolution trace. Source-list equality follows by mapping the recorded ordered pair; primitive preservation supplies the label.

Lean: `CopyRule.SuccessorMatch.map`

96.4 A presentation retaining the complete source graph

The output-index map is precisely the existing pair-boundary permutation, with the existing interface equality cast. It sends an output of either copy to the matching position in the two term blocks followed by the two atom blocks. No new permutation convention is introduced.

Lean: `CopyRule.copyOutput`

The fresh internal vertices comprise a second payload's internal vertices, one weakening wire, the ordered output-link wires of both copies, and the internal vertices of the original environment distributor with multiplicity two. The fresh edges comprise the second payload's edges, one weakening edge, and that distributor's edges. These are disjoint sums, so each kind has a separate tag.

Lean: `CopyRule.AddedVertex`

Lean: `CopyRule.AddedEdge`

The weakening and output-link ports name their respective fresh wires. A fresh payload port maps internal vertices into the clone, its sole input to the old selected source v, and its outputs to the left-copy links. A distributor port maps internal vertices into its fresh component, inputs to the ordered links, and outputs to the original tail's ordered target list. The proved length equation justifies every target-list index.

Lean: `CopyRule.SuccessorMatch.weakPort`

Lean: `CopyRule.SuccessorMatch.linkPort`

Lean: `CopyRule.SuccessorMatch.copyPort`

Lean: CopyRule.SuccessorMatch.distributorPort

The *source-relative copying presentation* has ambient carriers

$$V_G + V_{\text{added}}, \qquad E_G + E_{\text{added}}.$$

Every retention predicate is true. Old labels are retained; fresh labels come from the payload, weakening and original distributor. On old edges, sources are unchanged except that the selected source of the active contraction is replaced by the fresh weakening wire, in the same ordered position. Targets are unchanged except that the tail's outputs now enter the right-copy links. The fresh copy receives v, the weakening has empty source and its fresh wire as target, and the distributor connects the links to the old output attachments. All these equations concern entire ordered incidence lists.

Lean: CopyRule.SuccessorMatch.presentation

A *complete copying certificate* records a source match, a realization of its presentation by the actual successor graph, and the equation sending the resumed token to the old image of v. A realization preserves labels recursively, has exactly the indicated image, and preserves source and target lists. The certificate is proof data to be constructed from every actual rewrite; it is not an additional rule premise.

Lean: CopyRule.SuccessorCertificate

96.5 Carrier bijections for the literal reduct

The vertex map from the literal reduct to the presentation sends the residual tree and retained payload to their original source occurrences. It sends the active copy to the fresh payload summand, the residual weakening wire to the fresh weakening summand, the serial joining wires to the output-link summand, and the distributor vertices to their own summand. The inverse reconstructs the nested serial and parallel tags from each of these alternatives.

Lean: CopyRule.successorVertices

Lean: CopyRule.successorVertexInverse

The edge map follows the same assignment: retained tree and retained payload edges go to the old graph, while active-copy, weakening and distributor edges go to their fresh summands. Its inverse reconstructs the original nested edge tags. The vertex and edge maps are bijections. Both inverse equations are proved by cases on depth and the disjoint-sum/component tags; the output-link case respects the existing interface casts.

Lean: CopyRule.successorEdges

Lean: CopyRule.successorEdgeInverse

Lean: CopyRule.successorVertexBijection

Lean: CopyRule.successorEdgeBijection

A two-element source list containing the selected port can be written as one of the two ordered forms $[v, u]$ and $[u, v]$. Inspect its two entries and the membership proof to select the Boolean and other port. The local match uses this fact on the actual active contraction supplied by the tree. The payload occurs as the right serial component, and the existing trace theorem

proves resolution through the tree to its head.

Lean: CopyRule.selected_sources_exists

Lean: CopyRule.localSuccessorMatch

Every target of a non-tail edge of a normal payload is internal. There is no such edge in a plain payload; in a lambda payload the only non-tail edge is the head and its only target is the thunk wire. This small fact is essential when showing that the retained copy changes only the tail's external outputs.

Lean: CopyRule.NormalForm.nonTail_targets_internal

96.6 Ports and ordered incidences of the literal construction

A left serial output and the matching right serial input become the same internal joining wire, indexed by that interface position. These equations are proved from the serial pattern at each depth. They are boundary equations, not a graph-rewriting simplification.

Lean: CopyRule.serial_left_output

Lean: CopyRule.serial_right_input

The pair insertion maps a port into the original parallel pair with its original boundary cast. The active insertion takes the left payload into the outer serial composition. The retained insertion first enters the right payload after the residual tree, then the right parallel component, and then the outer composition. The tree insertion similarly passes through the residual frame, retained serial component and outer pair. These are the literal nested attachment maps of the existing copying rule.

Lean: CopyRule.pairInsert

Lean: CopyRule.activeInsert

Lean: CopyRule.retainedInsert

Lean: CopyRule.treeInsert

Pair outputs have the previously specified pair-boundary indices. The active copy's input is external input zero. The retained residual input i becomes external input $i + 1$, because input zero has been assigned to the active copy. Unfolding the boundary permutation proves the equations, with finite-index equality established by equality of their natural-number values.

Lean: CopyRule.pair_output

Lean: CopyRule.pair_left_input

Lean: CopyRule.pair_right_input

After the vertex bijection, the active insertion agrees with the presentation's fresh-copy port map. Internal ports are immediate; the input case uses the left-input equation, and outputs use the pair-output and serial-joining laws. Retained payload internal and input ports map to their original source occurrences. Its outputs instead map to the right-copy links. These last ports are precisely the attachments that will be reconnected by the distributor.

Lean: CopyRule.active_port

Lean: CopyRule.retained_internal

Lean: CopyRule.retained_input

Lean: CopyRule.retained_output

The residual tree's internal and output ports map to their old source occurrences. Its selected input zero maps to the new weakening wire. The residual-frame input $i+1$ maps to residual external input i, and the full outer construction restores it to source external input $i+1$. Thus all unselected tree inputs preserve their original order. The proofs unfold the residual frame and use the pair-input equations.

Lean: CopyRule.tree_internal

Lean: CopyRule.tree_output

Lean: CopyRule.tree_zero

Lean: CopyRule.residual_succ

Lean: CopyRule.tree_succ

The diversion map sends exactly v to the weakening wire and every other source port to its old occurrence. Mapping an old edge's source list by this function gives the presentation source list. For the active edge this is its ordered pair and distinctness theorem; for any other edge, incidence uniqueness prevents it from containing v. The tree attachment map agrees with this diversion: inspect internal, output, selected-input and other-input cases.

Lean: CopyRule.divertedPort

Lean: CopyRule.SuccessorMatch.source_divert

Lean: CopyRule.tree_port

The local tail targets are the original external outputs in order, since the payload is the right serial component. Therefore the literal distributor attachment, after the vertex bijection, agrees with the presentation's attachment map: internal vertices and inputs are direct, and outputs use this tail-target equation at the same finite index.

Lean: CopyRule.local_tail_targets

Lean: CopyRule.distributor_port

The retained tree-edge name follows its residual-frame, serial and parallel tags. The added-weakening edge name follows the same enclosing tags but selects the residual weakening instead.

Lean: CopyRule.keptTreeEdge

Lean: CopyRule.addedWeakening

The active copy's source and target lists are the original payload lists mapped by active insertion. The retained copy has the analogous equations with retained insertion. The retained tree has them with tree insertion. Each proof composes the literal list maps in the construction, so ordering and repeated use of boundary attachments are preserved exactly. The new weakening has empty source and the singleton residual image of the selected tree input as target.

Lean: CopyRule.active_source

Lean: CopyRule.active_target

Lean: CopyRule.retained_source

Lean: CopyRule.retained_target

Lean: CopyRule.kept_tree_source

Lean: CopyRule.kept_tree_target

Lean: CopyRule.added_weakening_source

Lean: CopyRule.added_weakening_target

96.7 The complete local certificate

The local match's tail is the right serial image of the payload tail. No payload edge is the active contraction: the established outer-label classification of copyable graphs excludes term-contraction labels there. These facts separate the old-tree and old-payload cases of the incidence proof.

Lean: CopyRule.local_tail_eq

Lean: CopyRule.local_active_not_payload

For every ambient edge, take its inverse image in the actual reduct and map its entire source list by the vertex bijection. The result is exactly the presentation source list. Old tree edges use diversion; old payload edges use retained insertion and cannot be active; new payload edges use active insertion; weakening and distributor edges use their attachment equations. The target proof has the same cases, but distinguishes the payload tail from other payload edges. Tail targets become right-copy links, and non-tail targets remain internal. This proves both incidence laws without discarding any residual tree or host occurrence.

Lean: CopyRule.local_source

Lean: CopyRule.local_target

The local realization combines the two carrier bijections, exact image properties, wire labels, recursively related edge labels, and the two ordered incidence theorems. Exact image is surjectivity because all retention predicates are true. The local certificate uses this realization and the local source match. Its token equation holds because input zero maps to its old external port. The entire original `after` graph is thus certified.

Lean: CopyRule.localRealization

Lean: CopyRule.local_certificate

96.8 Transport through occurrences and recursive renaming

For an occurrence $f : G \hookrightarrow N$, the origin-port map sends old ports through f and fixes each kind of newly added port. The origin-edge map similarly sends old edges through f and fixes fresh edges. On old ports, this agrees with ordinary occurrence attachment followed by old-source inclusion. These definitions allow the source presentation to move with a context without changing its fresh component.

Lean: `CopyRule.SuccessorMatch.originPort`

Lean: `CopyRule.SuccessorMatch.originEdge`

Lean: `CopyRule.SuccessorMatch.origin_old`

Fresh-copy attachments commute with the origin-port map by cases on the payload port. Distributor attachments commute too: their output case uses the occurrence's target-list equation, list indexing through a map, and the proved output length. This argument permits boundary attachment by the host.

Lean: `CopyRule.SuccessorMatch.copyPort_map`

Lean: `CopyRule.SuccessorMatch.distributorPort_map`

Every source and target list of the presentation commutes with these origin maps. For old edges, injectivity of the occurrence edge map preserves the active-edge and tail-edge tests. For fresh payload and distributor edges use the attachment equations. The weakening case is immediate. Thus the complete incidence description is natural under occurrences, not just under bijections.

Lean: `CopyRule.SuccessorMatch.source_map`

Lean: `CopyRule.SuccessorMatch.target_map`

For recursive isomorphism, the origin-port map is the port extension of the old-vertex bijection plus identity on fresh vertices. This gives an isomorphism of presentations, with the corresponding sum bijection on edges. Wire labels, recursive edge labels and ordered incidences are preserved by the preceding laws. A certificate can therefore be renamed on both source and result: transport its presentation, rename its realization, and compose the port equations for its token.

Lean: `CopyRule.SuccessorMatch.origin_iso`

Lean: `CopyRule.SuccessorMatch.renamePresentation`

Lean: `CopyRule.SuccessorCertificate.rename`

96.9 Transport through a single plugged context

The plugged vertex map retains host vertices as old host vertices; on an inserted successor it uses the local realization and shifts its old tags into the plugged source graph, leaving the fresh tags fresh. The edge map retains all non-hole host edges and treats inserted edges analogously. Disjoint host, old-component and fresh-component tags, together with local injection, prove both maps injective.

Lean: CopyRule.SuccessorCertificate.plugVertices

Lean: CopyRule.SuccessorCertificate.plugEdges

Lean: CopyRule.SuccessorCertificate.plug_vertices_injective

Lean: CopyRule.SuccessorCertificate.plug_edges_injective

Both maps have the entire ambient presentation as their image. For a host occurrence choose that occurrence itself; for an old or fresh component occurrence choose a preimage under the local realization. The omitted hole is absent on both sides of plugging. Hence the image equations agree with the presentation's universally true retention predicates.

Lean: CopyRule.SuccessorCertificate.plug_vertex_range

Lean: CopyRule.SuccessorCertificate.plug_edge_range

Host ports commute with old-source inclusion by cases on their port kind. Inserted component ports commute with the local realization followed by the origin-port map; boundary cases reduce to host attachments, and internal cases follow the disjoint tags. The source and target equations for inserted ambient edges then follow from presentation naturality under the plugging occurrence.

Lean: CopyRule.SuccessorCertificate.plug_host_port

Lean: CopyRule.SuccessorCertificate.plug_component_port

Lean: CopyRule.SuccessorCertificate.plug_source

Lean: CopyRule.SuccessorCertificate.plug_target

The plugged realization uses these maps, injection, image and port laws. For a surviving host edge its original incidence lists are untouched; it cannot be the active edge or tail because those belong to the inserted component. For an inserted edge, use local realization followed by occurrence naturality. Labels are preserved in the same two cases. The resulting plugged certificate uses the transported source match and maps its resumed token through the host attachment. This proves closure for the actual plugging constructor.

Lean: CopyRule.SuccessorCertificate.plugRealization

Lean: CopyRule.SuccessorCertificate.plug

96.10 Comparison of independently described payloads

A recursive payload isomorphism acts on fresh payload vertices and edges; it fixes the weakening, link and distributor summands. Its port extension commutes with fresh-copy attachments, and with distributor attachments when the two actual tail edges agree. The latter hypothesis ensures that both presentations reconnect to the same ordered host targets.

Lean: `CopyRule.SuccessorMatch.addedVertices`

Lean: `CopyRule.SuccessorMatch.addedEdges`

Lean: `CopyRule.SuccessorMatch.copyPort_congr`

Lean: `CopyRule.SuccessorMatch.distributorPort_congr`

The presentation comparison is identity on the complete old graph and the just-defined bijection on the fresh part. Match-frame uniqueness identifies the active edge, source position and other port. Tail equality identifies the redirected old target list. The payload isomorphism checks cloned labels and incidence lists recursively; the distributor and weakening components are unchanged. This constructs an isomorphism of the entire presentations.

Lean: `CopyRule.SuccessorMatch.payloadPresentation`

For two certificates at one source request, payload uniqueness supplies matching interface counts, the recursive payload isomorphism and tail equality. Transport the first realization along the presentation comparison. It and the second realization now have exactly the same ambient image, so the general realization-comparison theorem constructs their recursive graph isomorphism. Both token equations name the old selected port, which the comparison fixes. The result is a state isomorphism, not merely agreement of payload heads.

Lean: `CopyRule.SuccessorCertificate.coherent`

96.11 Transport through simultaneous expansion

The existing dependent-fiber map uses a selected map at the changed filling and bijections at every other filling. If the selected map is surjective, so is the whole map into old fibers plus fresh occurrences. An old selected fiber uses the selected preimage; another old fiber uses its bijection inverse; a fresh occurrence also uses the selected preimage. This general lemma is used for both vertices and edges.

Lean: `Opening.Fiber.surjective`

For an unchanged filling, equality of source and target fillings supplies a recursive identity-transport isomorphism in the required direction. The vertex and edge fiber maps use the certificate at the selected hole and these isomorphisms elsewhere. The full expansion vertex map additionally retains host vertices; the expansion edge map is the edge fiber map, since expansion replaces every host edge by its filling.

Lean: `CopyRule.SuccessorCertificate.otherIso`

Lean: `CopyRule.SuccessorCertificate.fiberVertices`

Lean: `CopyRule.SuccessorCertificate.fiberEdges`

Lean: `CopyRule.SuccessorCertificate.expansionVertices`

Lean: `CopyRule.SuccessorCertificate.expansionEdges`

Fiber injection proves expansion-edge injection. Expansion-vertex injection also distinguishes the retained host summand from all fiber and fresh summands. Fiber surjectivity and local exact images prove the corresponding whole-image properties; host vertices again supply their own preimages. Every ambient occurrence is therefore represented exactly once.

Lean: CopyRule.SuccessorCertificate.expansion_vertices_injective

Lean: CopyRule.SuccessorCertificate.expansion_edges_injective

Lean: CopyRule.SuccessorCertificate.expansion_vertex_range

Lean: CopyRule.SuccessorCertificate.expansion_edge_range

There are three port equations. Host ports commute with old inclusion. Ports in the selected successor filling commute with its realization and source occurrence. Ports in any other filling commute with that filling's unchanged isomorphism and then its old inclusion. Internal cases follow the fiber maps; input and output cases reduce to the common host attachment.

Lean: CopyRule.SuccessorCertificate.expansion_host_port

Lean: CopyRule.SuccessorCertificate.expansion_selected_port

Lean: CopyRule.SuccessorCertificate.expansion_other_port

Source and target lists of the selected ambient component commute with its fiber tag by occurrence naturality. For a different fiber, sources and targets remain the original filling lists attached to the host. The active and tail edge tests cannot select those edges: equality would identify two distinct fiber tags. These four equations separate changed and unchanged fillings in the whole incidence proof.

Lean: CopyRule.SuccessorCertificate.expansion_source

Lean: CopyRule.SuccessorCertificate.expansion_target

Lean: CopyRule.SuccessorCertificate.outside_source

Lean: CopyRule.SuccessorCertificate.outside_target

The expansion realization combines these maps and image laws. At the selected fiber, labels and incidences follow the local realization and selected-port law; at another fiber, they follow its equality-transport isomorphism and other-port law. The expansion certificate adds the transported source match and token equation. No assumptions about the other fillings' computations are introduced; their equality is exactly the existing expansion constructor's premise.

Lean: CopyRule.SuccessorCertificate.expansionRealization

Lean: CopyRule.SuccessorCertificate.expand

96.12 From actual rewrites to complete machine determinism

The proposition `SuccessorWitness` existentially hides the payload counts and graph while retaining a complete certificate. Any two witnesses at the same source request give isomorphic successor states by certificate comparison. For an actual local copy rule, normalize its copyable payload, construct the normal local certificate, and transport it back along the proved isomorphisms of the literal before and after constructions. Thus an arbitrary original copyable payload, not just its normal presentation, obtains a witness.

Lean: `CopyRule.SuccessorWitness`

Lean: `CopyRule.SuccessorWitness.coherent`

Lean: `CopyRule.local_witness`

An actual rewrite already supplies a recognized head. If an occurrence embedding exhibits a head at the mapped request, unique source incidence identifies its primitive with that recognized head. This reflects existence of the requested primitive head back to the source. For a local rewrite whose head is a term contraction, all non-copy constructors contradict their own proved operation heads. The remaining copy constructor supplies the local witness just established.

Lean: `Combined.rewrite_head_exists_reflect`

Lean: `Combined.local_copy_certificate`

Induction on the original rewrite derivation now gives a copying witness for every rewrite at a term-contraction head. The local case uses the local certificate theorem; recursive renaming, plugging and expansion reflect the head to the induction hypothesis and apply the corresponding certificate transport. Consequently any two actual copying rewrites at that request have recursively isomorphic whole results and corresponding search tokens.

Lean: `Combined.rewrite_copy_certificate`

Lean: `Combined.copying_rewrite_deterministic`

Complete successor determinism up to representation asserts

$$s \to t \ \wedge \ s \to u \quad \Longrightarrow \quad \mathrm{Nonempty}(\mathrm{StateIso}(t, u)).$$

The earlier exhaustive rewrite classification either already provides the successor isomorphism or identifies a common copying head. The new copying comparison resolves that final alternative, proving rewrite determinism. For full steps, two search successors are literally equal, search and rewrite cannot overlap in focus mode, and two rewrites use the completed theorem. This is an unconditional theorem about the existing transition relation; it makes no template, refocusing or termination assumption.

Lean: `Combined.rewrite_deterministic`

Lean: `Combined.step_deterministic`

For any actual step $s \to t$, successful termination in n steps from s is equivalent to successful termination in m steps from t, with $n = m + 1$. The forward direction compares the first step of the successful run with the chosen step, then transports its remaining run along the successor isomorphism. The reverse direction prefixes the chosen step. Existing final-normality and representation-transport lemmas justify the zero-length and finality cases.

Lean: `Combined.cost_spectrum`

Every unbounded step is a lift of one finite-depth step. Two such steps from the same

unbounded state have the same depth and interfaces, so the completed finite theorem lifts to `Represents` between their successors. Previously proved representation invariance then gives observational state equivalence for every cost order Q, in particular exact costs. Literal equality of named graph carriers is not claimed; recursive representation is the conclusion.

Lean: `Combined.unbounded_deterministic`

Lean: `Combined.unbounded_successors_equiv`

96.13 Exact costs of copying and of arbitrary finite prefixes

An *exact prefix cost law* says that for any actual finite run $s \longrightarrow^r t$,

$$\mathrm{TerminatesIn}(n, s) \iff \exists m\ (n = m + r\ \wedge \mathrm{TerminatesIn}(m, t)).$$

Induct on the given prefix. The empty prefix is immediate; the successor case uses the one-step spectrum theorem and adds its one transition to the inductive cost. Apply this law to a successful run ending in a final state. Final normality forces any successful continuation there to have length zero, so every successful run from the same source has the same length. This uniqueness concerns the entire combined machine, including copying.

Lean: `Combined.run_cost_spectrum`

Lean: `Combined.successful_cost_unique`

Every selected input of a genuine contraction tree requests copying in one search transition: use its proved active term-contraction head and the existing search-entry rule. The actual local copying step then gives the exact one-transition cost equation from rewrite mode to the literal reduct in search mode. Composing search entry with this equation gives two transitions from the original search state to the reduct's continuation. These equations also hold if no successful continuation exists.

Lean: `CopyRule.requests_copy`

Lean: `CopyRule.copying_cost_iff`

Lean: `CopyRule.initial_cost_iff`

Under an arbitrary serial host, transport this two-transition run using the existing context law and apply the general prefix-cost theorem. The continuation is the actual reduct attached at the same host position. In particular an internal return is not silently declared final; its remaining host execution is still counted.

Lean: `CopyRule.host_handoff`

The active fresh lambda returns in one search step without examining its body. Its copied head label is lambda; its source list is the original head's singleton mapped by active insertion, hence the selected input. The original lambda entry rule returns there. This argument applies to arbitrary body graphs and arbitrary nesting, independently of whether a later application of the body would terminate.

Lean: `CopyRule.lambda_returns`

If the actual copied payload has this immediate return transition, then its initial successful cost is exactly three: one search transition, one copying transition, and the return. The exact two-step continuation law reduces the claim to the return law and zero cost at the final

external input. Constants and instances use their existing return theorems; lambdas use the new theorem. The resulting three cost equivalences quantify over every successful run of the full machine and every matched contraction tree.

Lean: `CopyRule.passive_cost_iff`

Lean: `CopyRule.constant_cost_iff`

Lean: `CopyRule.instance_cost_iff`

Lean: `CopyRule.lambda_cost_iff`

96.14 Semantic regression scenarios and the remaining boundary

The closed constant-copying program uses the existing one-input canonical contraction tree and original constant corolla. Its focus- and hole-freedom come from the established tree/payload label-safety theorem. Every successful run of this program in the unbounded machine, at every finite padding, costs exactly three. Exact padding reflection reduces the assertion to its original depth, exact finite-run representation reduces it to the combined finite machine, and the constant cost theorem finishes the proof. This graph is a copying-rule regression; no claim identifies it with a new term translation.

Lean: `CopyRule.Examples.constantProgram`

Lean: `CopyRule.Examples.constant_pad_cost_iff`

Copying a lambda whose body is the unchanged translated nested-lambda body also has exact cost three. This is an instance of the arbitrary-body theorem; it ensures that copying and returning the outer function do not execute either body. Separately, the two distinct copied reference instances in the existing shared-store graph must resolve to the same cell, with three trace hops each. Compare any proposed resolutions with the previously constructed traces by store-resolution uniqueness. Thus the coherence argument remains compatible with reference aliasing, rather than treating copying as store allocation.

Lean: `CopyRule.Examples.nested_lambda_cost_iff`

Lean: `CopyRule.Examples.shared_reference_unique`

These regressions prove copying costs, preservation of nested bodies and store identity, and compatibility with the separately proved depth-padding semantics. They do not constitute the Figure 22 contextual-equivalence proof, a refocusing theorem, or robustness of the copying template. In particular, repeated closure applications and divergent callbacks still have to be handled in the general specimen and template arguments. The present milestone supplies the unconditional successor-determinism law those arguments require.

97 Refocusing for copying and allocation

A rooted state is reachable by search alone from the input of its own underlying graph. Refocusing requires that a machine transition preserve this property, even when the transition changes that graph. We now prove it for every actual copying rewrite and every actual reference allocation, in arbitrary surrounding graphs. The copying result includes an exact search-prefix length law. The proof uses the complete successor certificates of Section 96, not an assumed noninterference property of copying.

Intrinsic copying refocusing is preservation of search rootedness by the search and copying branches of the existing combined relation. This partition is exhaustive together with the operation-headed compute branch. Allocation is then discharged within the latter branch. Full refocusing is proved equivalent to refocusing for precisely the remaining operation-headed rewrites other than reference creation, both at finite allowance and for the unbounded machine. This is a reduction of outstanding obligations, not a proof that all operations refocus.

The paper correspondence is the copying branch of PDF Lemma 10.4 (HTML Lemma 9, Section 10.3) and the allocation branch of PDF Lemma 12.4 (HTML Lemma 13, Section 12.1). The prerequisites are operation-path reachability, PDF Lemma C.4 (HTML Lemma 23), and the earlier-search-visit property, PDF Lemma C.5 (HTML Lemma 24), in Appendix C. Our proof specializes the prefix-replay argument behind PDF Lemma C.8 (HTML Lemma 27): copying uses its complete retained-occurrence description; allocation uses replacement of a single operation edge. We do not yet claim the full stationary-transition theorem for arbitrary copying graph fragments.

The dependency map for this milestone is:

1. Search rootedness implies operation-only reachability. A term-sharing edge consequently blocks search from the retained copyable payload.
2. The original copy certificate provides retained vertices, edges and ordered incidences in the complete reduct. The reachable prefix avoids the only changed incidences, so it replays exactly.
3. Search history records earlier visits to operation sources, including when control returns from an evaluated argument. It excludes revisiting a source before an eventual rewrite normal form.
4. This history replays the prefix preceding a replaced operation head. The already established allocation representation theorem identifies that replacement with every actual allocation successor.
5. Recursive representation and padding preserve rootedness. The remaining compute families are an explicit premise only in the full-run corollaries.

The terms of Proposition 12.2, Figure 16's translation, the transition relation, and all original stability and copyability conditions are unchanged. The focused-graph correspondence, specimen locality, remaining operation refocusing, templates and final equivalence chain are still open. The estimate remains 55–70% of mathematical effort, with low confidence; no percentage is inferred from this section's 102 declarations.

97.1 Rootedness and operation paths

For a graph G, an external input i, and a coordinate state p, write RootedFrom(G, i, p) when

$$\exists n \in \mathbb{N}, \qquad (?, \mathrm{in}_i) \xrightarrow{\mathrm{search}, n}_G p.$$

The graph is fixed throughout this witness. In particular, a past execution from a different graph is not a rootedness witness for the current state.

Lean: `Search.RootedFrom`

Operation reachability is generated by external inputs and by moving from the unique source of an operation edge to one of its targets. The operation-source lemma derives the singleton source list from the signature and a membership hypothesis. This reachability predicate is an invariant implied by rootedness, not an alternative definition of rootedness.

Lean: `Search.OperationReachable`

Lean: `Search.operation_source`

If a reachable vertex is a target of an edge with singleton source, that source is reachable. An input cannot be an edge target; otherwise the last operation in the reachability witness is the given incoming edge by target uniqueness. The same argument shows that a target of a non-operation edge cannot be operation reachable. Neither result assumes the surrounding graph is acyclic.

Lean: `Search.OperationReachable.parent`

Lean: `Search.OperationReachable.not_nonoperation_target`

Every search interaction preserves operation reachability. Entering an argument extends an operation path. Returning from an argument recovers its operation parent, then either stays there or descends to the next argument. Interactions which only change mode keep the same vertex; primitives with no search interaction give impossible cases. Induction extends this statement to runs, and starting at an input proves it for every rooted token.

Lean: `Search.step_reachable`

Lean: `Search.run_reachable`

Lean: `Search.rooted_reachable`

The initial search state is rooted by the zero-length run. Appending one search transition preserves rootedness. A recursive graph isomorphism preserves and reflects it by the existing exact search-run transport, which fixes each external input.

Lean: `Search.rooted_initial`

Lean: `Search.rooted_step`

Lean: `Search.rooted_iso`

For the closed interface $[\star] \Rightarrow []$, `Rooted` chooses the sole input. `UnboundedRooted` applies this predicate at the allowance stored in an unbounded state. A state isomorphism maps the graph and token together, so rootedness is invariant; every initial closed program is rooted. The existing unbounded representation relation inherits the same equivalence by its finite representative.

Lean: `Combined.Rooted`

Lean: `Combined.UnboundedRooted`

Lean: `Combined.rooted_iso`

Lean: `Combined.initial_rooted`

Lean: `Combined.represents_rooted`

97.2 The retained payload is inaccessible to search

A nonempty copy-resolution trace has a final sharing hop: induction on its tail either returns the first hop or the final hop of the shorter trace. A resolution from an active term contraction cannot have length zero, because its terminal primitive would then also be that contraction, contradicting uniqueness of the outgoing head. Thus it has a predecessor sharing edge.

Lean: `CopyTrace.last_hop`

Lean: `CopyTrace.resolves_predecessor`

A *copying search barrier* is the obstruction supplied by a sharing edge immediately before the payload. Forwarding labels are contractions or weakenings, never operations. The final hop targets the payload's singleton source, so the non-operation-target lemma excludes operation reachability there. This argument uses the actual incidence relation in the whole host.

Lean: `CopyTrace.forward_not_operation`

Lean: `CopyTrace.resolved_source_unreachable`

A successor match includes an occurrence of the complete payload and its resolution trace. Mapping its singleton head source into the host therefore proves that the payload input is unreachable by search. For a primitive payload its tail is its head; for a lambda payload its tail is a box, which cannot satisfy the primitive-label premise. Consequently no reachable primitive search interaction can use the retained tail's source. This is the specific fact needed when copying changes the tail's targets.

Lean: `CopyRule.SuccessorMatch.payload_input_unreachable`

Lean: `CopyRule.SuccessorMatch.primitive_tail_eq_head`

Lean: `CopyRule.SuccessorMatch.tail_source_unreachable`

97.3 Retained occurrences inside the actual reduct

A *retained occurrence* identifies an old source vertex or edge inside the complete copying successor. The certificate's realization covers every old occurrence. Choose a preimage for each old internal vertex and edge, and extend the vertex map to ports while fixing the external interface. These choices use ordinary classical choice; they do not assert that the whole old graph is an unchanged subgraph.

Lean: `CopyRule.SuccessorCertificate.retainedVertex`

Lean: `CopyRule.SuccessorCertificate.retainedEdge`

Lean: `CopyRule.SuccessorCertificate.retainedPort`

The choice specifications say that the chosen vertex and edge realize the corresponding old occurrence. The port equation follows by its three cases: internal, input and output. Applying the realization to two equal retained ports gives equality of their old occurrences, hence injectivity of the port map.

Lean: `CopyRule.SuccessorCertificate.retained_vertex`

Lean: `CopyRule.SuccessorCertificate.retained_edge`

Lean: `CopyRule.SuccessorCertificate.retained_port`

Lean: `CopyRule.SuccessorCertificate.retained_port_injective`

Every retained primitive has the same label, by the certificate's recursive label comparison. Every source list other than that of the active contraction is the old ordered source list mapped through retained ports. Every target list other than that of the payload tail obeys the analogous law. Apply the injective realization map to both sides and use the source-relative presentation equations. The two exceptions are essential: copying really changes those attachments. Finally, the certificate's token equation identifies the retained request vertex with the resumed vertex.

Lean: `CopyRule.SuccessorCertificate.retained_label`

Lean: `CopyRule.SuccessorCertificate.retained_source`

Lean: `CopyRule.SuccessorCertificate.retained_target`

Lean: `CopyRule.SuccessorCertificate.retained_token`

97.4 Exact replay of the copying prefix

Pure search has no outgoing transition from rewrite mode. Therefore a search run ending outside rewrite mode cannot start in it: normality would force its endpoint to equal its source. A separate induction splits any run of length $n + 1$ into a prefix of length n and its last interaction.

Lean: `Search.run_source_not_rewrite`

Lean: `Search.last_step`

An entering interaction which produces rewrite mode puts the token at the vertex it was given; so does an operation-resume interaction. This follows by inspecting the actual `enter` and `resume` equations. If the resulting vertex has a term-contraction head, the resume case is

impossible: it would give the same vertex an operation head. Hence the immediate predecessor of a copying request is search at that same vertex.

Lean: `Search.enter_rewrite_vertex`

Lean: `Search.resume_rewrite_vertex`

Lean: `Search.contraction_predecessor`

The zero-length run cannot end in rewrite mode. Splitting a rooted copying request at its last interaction and applying the predecessor result gives $n = m + 1$ and an exact m-step run ending at search on the request vertex. The last step is precisely the copying trigger.

Lean: `Search.copying_prefix`

Take a reachable search interaction whose successor is not in rewrite mode. It cannot use the active contraction: entering that head would request a rewrite. Nor can it use the payload tail: the search barrier excludes its source, and in a backtracking case the reachable argument has a reachable parent. All required ordered incidences are therefore preserved by the retained maps. The existing naturality equations for entering and resuming then construct the same interaction in the actual successor. Induction replays a whole prefix; its intermediate states cannot be in rewrite mode because its endpoint is not.

Lean: `CopyRule.SuccessorCertificate.replay_step`

Lean: `CopyRule.SuccessorCertificate.replay_prefix`

The *copying refocusing cost* is an exact comparison between searches on two different graphs. If a search on G reaches the request in n steps, then, for an actual copy successor K, there is an m such that

$$n = m + 1, \qquad (?, \mathrm{in}_i) \xrightarrow{\mathrm{search}, m}_K (?, w).$$

Replay the pre-trigger prefix and use the retained-token equation. Forgetting m gives rootedness of the successor. This statement counts search steps; it does not identify the graph-changing copy transition with a search step.

Lean: `CopyRule.SuccessorCertificate.rooted_cost`

Lean: `CopyRule.SuccessorCertificate.rooted`

97.5 Discharging the intrinsic machine cases

The public copying theorems take an actual contextual rewrite at a term-contraction head. The existing extraction theorem supplies its complete successor certificate; the preceding results then give rootedness and the exact prefix-length law. No certificate or noninterference premise is left for the caller to prove.

Lean: `Combined.copying_rooted`

Lean: `Combined.copying_rooted_cost`

The intrinsic relation consists of unchanged search transitions and actual rewrites at term-contraction heads. The compute relation consists of actual rewrites at operation heads. Both embed directly into the original combined relation. They cannot both start at the same state: search has no rewrite source, and a contraction head cannot also be an operation head.

Canonical source recognition shows that every combined step belongs to one of these two relations, so this is an exhaustive partition of the existing machine.

Lean: `Combined.IntrinsicStep`

Lean: `Combined.ComputeStep`

Lean: `Combined.IntrinsicStep.combined`

Lean: `Combined.ComputeStep.combined`

Lean: `Combined.intrinsic_compute_disjoint`

Lean: `Combined.step_iff_intrinsic_or_compute`

Refocusing for a relation means that a rooted source has a rooted successor. The intrinsic relation refocuses: the search case appends a search step, and the copy case uses the newly proved theorem. Hence the whole finite machine is refocusing if and only if its compute relation is refocusing. The forward implication restricts a full-machine proof; the reverse implication uses the exhaustive partition.

Lean: `Combined.Refocusing`

Lean: `Combined.intrinsic_refocusing`

Lean: `Combined.refocusing_iff_compute`

Lift each of the two relations to the unbounded state by keeping the same finite allowance for a step. Their disjunction is again exactly the actual unbounded relation. Unbounded refocusing is preservation of the unbounded rooted predicate. The intrinsic theorem lifts without additional premises, and the full unbounded obligation reduces equivalently to the compute obligation. No global upper bound on future box depth is asserted.

Lean: `Combined.UnboundedIntrinsicStep`

Lean: `Combined.UnboundedComputeStep`

Lean: `Combined.unbounded_step_iff_intrinsic_or_compute`

Lean: `Combined.UnboundedRefocusing`

Lean: `Combined.unbounded_intrinsic_refocusing`

Lean: `Combined.unbounded_refocusing_iff_compute`

Once compute refocusing is supplied, induction over an actual finite run preserves rootedness at its endpoint. The compute-refocusing hypothesis is explicit and still unresolved in general; this corollary is not presented as an unconditional full-machine theorem.

Lean: `Combined.run_rooted`

97.6 Exact search history, including backtracking

A *search visit* to v in a run ending at p with length n is an exact factorization $n = a + b$ into a run from the input to $(?, v)$ followed by a run to p. *Search history stacks* are operation paths from the chosen input whose operation sources satisfy a predicate P recording earlier visits. They are logical witnesses reconstructed from a run, not an added runtime stack.

Lean: `Search.Visit`

Lean: `Search.HistoryStack`

Replacing P by a pointwise consequence Q preserves a history stack by induction. From a stack at an edge target, recover its parent stack and that parent's predicate: inputs are not targets, and uniqueness of the incoming edge identifies the final operation. A recorded visit extends across one additional step by extending its suffix; the current search vertex is visited with suffix length zero.

Lean: `Search.HistoryStack.mono`

Lean: `Search.HistoryStack.parent`

Lean: `Search.Visit.step`

Lean: `Search.Visit.current`

A full history consists of a stack annotated by exact visits in the current run, together with a visit to its current token vertex. For one search step, extend every recorded suffix. Entering a fresh argument adds its operation source to the stack; resuming recovers that source from the parent theorem, then either returns to it or records the next argument. Mode-only changes retain the vertex's earlier visit. This proves history preservation for every search rule, including backtracking.

Lean: `Search.History`

Lean: `Search.history_step`

Induction extends history across a run, adding its length to the existing history length. The initial input has the empty stack and its zero-length visit, so every rooted run carries a full history. In particular, if its final mode is not search, the visit to its final vertex has a strictly positive suffix: a zero suffix would identify the endpoint with search mode. This proves the exact-run version of PDF Lemma C.5.

Lean: `Search.history_run`

Lean: `Search.history`

Lean: `Search.revisit`

97.7 Single operation replacement and allocation

A prior search visit cannot recur with a positive intervening run before an eventual rewrite endpoint. Otherwise concatenate the intervening cycle with the suffix to obtain two search runs from the same state to a rewrite normal form with unequal lengths. Determinism and normality imply equal lengths, contradicting positivity. Only the already proved determinism of search is used here.

Lean: `Search.Visit.no_earlier_cycle`

Replace one edge e by an arbitrary graph A of the same typed interface. A search interaction which does not use e transports along the retained host-port map. Its other edge is retained with the same primitive label and mapped ordered source and target lists; the search equations commute with this map. The premise specifies both ways an interaction can use an edge: entering one of its sources or returning at an ordered target.

Lean: `Search.replace_step`

Consider a run prefix ending at search on the source v of e, followed by a run to a rewrite state. No earlier interaction in that prefix uses e. An entering interaction would already be a visit to v; a return at a target of e would imply such an earlier visit by search history. Either contradicts the no-earlier-cycle result. Induction therefore transports the entire prefix through replacement, with exactly the same length.

Lean: `Search.replace_prefix`

Head replacement refocusing says that if a rewrite request is rooted at the source of an operation, replacing just that edge by any graph with its interface leaves search at the retained source rooted. The earlier-visit theorem supplies the pre-head prefix; the preceding replay theorem transports it. This special theorem needs no one-way premise on the replacement graph. It does not authorize deletion or rewiring of other host edges, and therefore does not by itself cover all compute rules.

Lean: `Search.replacement_rooted`

Recursive state isomorphism preserves rootedness from any specified input, not just a closed program's input. The graph transport carries the witness, and the state-isomorphism equation identifies its endpoint token. For an actual allocation rewrite, the earlier canonical-successor theorem supplies an isomorphism from replacement of the original reference head by its fresh cell prefix to the actual successor. Apply head replacement refocusing and then this isomorphism. The allocation rule still requires the same stable argument as before; the proof never removes that premise.

Lean: `Combined.StateIso.rootedFrom`

Lean: `Combined.allocation_rooted`

The remaining compute relation contains actual rewrites at operation heads whose operation is not reference creation. It embeds in the compute relation. To prove full refocusing from refocusing of this remaining relation, partition the operation cases by equality with reference creation: use the allocation theorem in that case and the supplied premise otherwise. Conversely, any full-machine proof restricts to the remaining relation. Thus this reduction is an equivalence, not merely a sufficient-condition theorem.

Lean: `Combined.RemainingComputeStep`

Lean: `Combined.RemainingComputeStep.compute`

Lean: Combined.refocusing_iff_remaining_compute

Lift the remaining relation to unbounded states and repeat the same argument at each step's finite allowance. Finally, induction preserves rootedness along any finite combined run once refocusing of this remaining relation has been proved. The explicit premise still covers dereference, assignment, equality, application and arithmetic; the statement assumes neither their truth nor termination of their continuations.

Lean: Combined.UnboundedRemainingComputeStep

Lean: Combined.unbounded_refocusing_iff_remaining_compute

Lean: Combined.run_rooted_of_remaining

97.8 Representation laws and semantic regressions

A search transport between graphs with one term input must map that input to itself: it preserves the input predicate, and there is only one input index. Exact transport and reflection of runs therefore preserve and reflect rootedness. Applying this result to the already proved concrete padding transport establishes rootedness invariance at every larger allowance. This uses concrete run transport, not an inference from determinism alone.

Lean: Search.Transport.sole_input

Lean: Search.Transport.rooted_iff

Lean: Search.rooted_padTo_iff

The retained payload input is not rooted in any focus mode or from any external input, since every rooted token would be operation reachable. For closed states, any combined successor of a rooted contraction request is rooted: a search successor from rewrite mode is impossible, leaving the actual copying theorem.

Lean: CopyRule.SuccessorMatch.payload_input_not_rooted

Lean: Combined.copying_step_rooted

The test graph uses a canonical one-input contraction tree and a primitive reference-allocation operation as an enclosing host. The two payload graphs are the original `CopyRule.before` and `CopyRule.after` applied to a constant. The host first enters its argument, where the copying request is now an internal token position. Entering the host followed by the copy trigger gives an exact two-step rootedness witness.

Lean: Combined.RefocusingExamples.copyTree

Lean: Combined.RefocusingExamples.copyHost

Lean: Combined.RefocusingExamples.copiedConstant

Lean: Combined.RefocusingExamples.copiedConstantAfter

Lean: Combined.RefocusingExamples.host_enters_copy

Lean: Combined.RefocusingExamples.host_copy_request_rooted

The actual copying step under this host preserves rootedness by the general theorem. The returned payload at the host's internal argument is not a final state; the outer reference

operation is still pending. This regression checks that refocusing and finality are kept distinct at an internal token, rather than testing only a copy request at the external root.

Lean: `Combined.RefocusingExamples.host_copy_result_rooted`

Lean: `Combined.RefocusingExamples.host_copy_return_not_final`

For the unchanged translation of `ref` n, the existing entering, value and resume interactions give a three-step search witness to the allocation request. Every actual combined successor is rooted by allocation refocusing; no particular representative of the allocated graph is assumed. The unchanged translated addition example has its established five-step search witness, and its rootedness holds under every finite padding by the proved padding equivalence.

Lean: `Combined.RefocusingExamples.translated_reference_request_rooted`

Lean: `Combined.RefocusingExamples.translated_reference_result_rooted`

Lean: `Combined.RefocusingExamples.translated_addition_request_rooted`

Lean: `Combined.RefocusingExamples.translated_addition_rooted_padding`

All 102 declarations above are included in the axiom audit. The full build passes 446 jobs, and 2305 axiom checks pass: 6 abstract, 477 graph and 1822 copying declarations. Only `propext`, `Classical.choice` and `Quot.sound` occur. Appendix B.1 removes the discharged copying and allocation refocusing sub-obligations; its remaining operation, specimen, template and final-assembly obligations remain explicit.

98 Proof checking

98.1 The axiom audits

The file `Audit.lean` contains six `#print axioms` commands. They inspect counting-simulation soundness, the forward and converse simulation constructions, the two contextual sufficiency theorems, and the zero-step converse regression theorem. These diagnostic commands introduce no new definitions or proofs.

```
Lean: Audit: abstract theorem dependencies
```

The graph audit inspects shallow plugging, host and inserted path equivalence, plugging congruence, exact trace uniqueness, typed operation-path uniqueness, finite deep enumeration, unique holes under boxing, deep plugging and its other-label locality, and the self-loop and deep-replacement regression proofs. Twelve further commands audit occurrence-coordinate observation preservation, exact hole inventories, sharp freshness and consumption, isomorphism invariance of contexts, deep site provenance, deep context closure, and the new reuse and collision regressions.

```
Lean: Graph.Audit: concrete theorem dependencies
```

```
Lean: Graph.Audit: context calculus dependencies
```

Nine further commands audit the exact deep inventory, both versions of sharp context closure, hole-freeness, selected-label consumption, certified deep plugging, and the deep reuse and collision regressions.

```
Lean: Graph.Audit: exact deep inventory dependencies
```

Eleven further commands audit recursive label transitivity, operation-path invariance, occurrence embeddings, hole and context invariance, deep congruence, quotient plugging and context validity, and the strict-extension regressions.

```
Lean: Graph.Audit: recursive isomorphism dependencies
```

Ten further commands audit shallow host congruence, site transport and its specification, simultaneous congruence, quotient invariance, retained holes, freshness, result validity, and the moved-address and wrong-address regressions.

```
Lean: Graph.Audit: site transport dependencies
```

Sixteen further commands inspect shallow associativity, both corolla units, deep associativity, quotient composition, composed retained inventories, sharp composite context closure and its certified constructor, three-stage coherence, deep restoration, quotient identity, and the self-loop, repair, and mixed-depth regressions.

```
Lean: Graph.Audit: composition and unit dependencies
```

Six further commands inspect global focus-free substitution, focussed shallow and deep substitution, finite-filling boundary agreement, the initiality regression, and rejection of a boxed focus.

```
Lean: Graph.Audit: concrete focus dependencies
```

Eight further commands inspect operation-view existence, search determinism, search boundary laws, shallow filling preservation, and the thunk, value, and variable regressions.

```
Lean: Graph.Audit: search rule dependencies
```

Ten further commands inspect explicit focus insertion and certification, both incidence-recovery

laws, hole freedom, term-typing preservation, initial attachment, search endpoint realization, and the interior and value regressions.

```
Lean: Graph.Audit:  focus realization dependencies
```

Twelve further commands inspect general erasure and its invariants, the reinsertion isomorphism and focus endpoints, arbitrary-state recovery, state search determinism and boundary laws, and the execution/interior regressions.

```
Lean: Graph.Audit:  erasure and state search dependencies
```

Fifteen further commands inspect insertion–erasure cancellation, coordinate transport and realization, erasure naturality, source invariance, quotient search determinism and boundaries, and the concrete quotient regressions.

```
Lean: Graph.Audit:  round trips and quotient search dependencies
```

Fourteen further commands inspect recursive focus freedom, coordinate search, insertion and erasure, recursive state search and quotient determinism, the quotient projection and lifting laws, exact-length executions, and regressions.

```
Lean: Graph.Audit:  recursive state search and execution dependencies
```

Sixteen further commands inspect canonical roots, search preservation of rootedness, operation reachability, insertion of rooted paths, the concrete path obstruction, refocusing, and the positive and negative state examples.

```
Lean: Graph.Audit:  rooted-state and path-obstruction dependencies
```

Sixteen further commands inspect operation-source typing, one-wayness and recursive renaming, path extraction and host reflection through fillings, one-way filling closure, the concrete Lemma 26(2) obstruction, its paired single-hole corollary, and the positive and negative examples.

```
Lean: Graph.Audit:  one-way filling dependencies
```

Twenty-five further commands inspect simultaneous expansion incidence, local path extraction, paired path reflection, focussed expansion, its identity isomorphism and hole freedom, the concrete shallow specimen bridges, and the two-hole and alternating-side regressions.

```
Lean: Graph.Audit:  simultaneous shallow specimen dependencies
```

Thirteen further audit commands cover the peeling and identity-copy isomorphisms, certified execution correctness, order independence both explicitly and in the recursive quotient, finite realization, and the two-hole schedule examples. All use only the standard logical axioms below. The no-internal-vertex and identity-copy isomorphism proofs do not require classical choice.

```
Lean: Graph.Audit:  sequential expansion dependencies
```

Nineteen further audit commands inspect shallow and deep independent interchange, residual selected-label preservation, pointed renaming invariants, quotient substitution and freshness, sharp quotient context validity, and the positive and negative pointed-address examples. These depend only on the standard logical axioms listed below. The pointed host invariant and the correctly renamed example do not require classical choice.

```
Lean: Graph.Audit:  deep interchange and pointed quotient dependencies
```

Nineteen further commands audit recursive filling focus freedom, retained focus copies, the deep specimen path and boundary laws, the outer realization bridge, and the deep closed-state

example, including the impossibility of closing its context by the old shallow rules. All use only the standard logical axioms below; there are no new project axioms or admitted proof terms.

Lean: `Graph.Audit: recursive deep specimen dependencies`

Twenty-one further commands audit search transport through recursive fillings, exact execution lengths, boundary reflection, invariant-region reflection, paired actual-state executions and rootedness, and the positive deep-box and negative hole-boundary examples. These use only the standard logical axioms; there are no new project axioms or omitted proof terms.

Lean: `Graph.Audit: recursive filling search dependencies`

Twenty-one further commands audit erasure–expansion commutation and its coordinate equation, the focussed-state bridge, boundary classification, non-exiting rooted specimens, synchronized settled traversal, and the deep traversal and non-rootedness regressions. They use only the standard logical axioms; no project axiom or omitted proof is introduced.

Lean: `Graph.Audit: focussed specimen traversal dependencies`

In the pinned snapshot, counting-simulation soundness, finite deep enumeration, unique holes under boxing, occurrence-coordinate observation preservation, isomorphism invariance of unique holes, and uniqueness of the two-hole example report only `propext` and `Quot.sound`. Recursive label transitivity, recursive operation-path invariance, the nonexistence of a literal-label isomorphism between the example boxes, and their equality as quotient classes likewise report only these two axioms. Existence of transported sites also reports only these two axioms; the chosen transport and substitution results additionally use classical choice. All audited composition and unit results report these three standard axioms. The focus results do too, except the boxed-focus rejection, which uses only `propext` and `Quot.sound`. The search audits use only `propext` and `Quot.sound`, except shallow filling preservation, which also uses classical choice. The new realization audits use all three standard axioms, except term-typing preservation, which uses only propositional extensionality and quotient soundness. All new general-erasure and state-search audit entries use those three standard axioms. The round-trip and quotient-search audits do too, except coordinate search renaming, which uses only `propext` and `Quot.sound`. The recursive-search audits use all three axioms, except label and graph focus-freedom invariance, coordinate search transport, and the renamed-box focus-freedom example, which use only `propext` and `Quot.sound`. The rooted-state audit entries use all three axioms, except coordinate operation-reachability preservation and the incoming-trace operation theorem, which use only `propext` and `Quot.sound`. In the one-way audit group, source typing, the input-type lemma, the reachability characterization, recursive renaming, and the four basic corolla checks use only `propext` and `Quot.sound`. The extraction, reflection, filling-closure, obstruction, and filled-state example entries also use classical choice. In the simultaneous-expansion group, the six incidence checks, identity isomorphism, hole freedom, and the two alternating-side premise checks use only `propext` and `Quot.sound`; the other entries also use classical choice. These are standard Lean logical axioms: propositional extensionality, quotient soundness, and classical choice, respectively. Neither audit reports `sorryAx` or a project-specific axiom.

Absence of project-specific axioms does not discharge a theorem's hypotheses. Specimen laws and initial lifting remain parameters of the abstract sufficiency results. They do not appear as named global axioms in the report. In contrast, the concrete graph constructions explicitly prove the invariant fields needed to construct their results. Neither fact supplies the missing bridge to a concrete UAM.

98.2 Reproducing the Lean and document checks

The snapshot pins Lean `4.35.0` in `lean-toolchain` and uses only the bundled `Std`. With that toolchain available, run

```
lake build
lake env lean Hypernets/Audit.lean
lake env lean Hypernets/Graph/Audit.lean
```

The script `scripts/check.sh` runs the build and both audits. Run it using `bash` with the script path as its argument. It selects the workspace-local macOS ARM compiler if present, and otherwise uses `lake` from the environment. The audits are checked separately from the root import.

The additional local-search audit checks both adjacency directions, canonical rewiring and its uniqueness characterization, both state-level correspondences, determinism, and actual variable and value successors. Only the standard logical axioms occur; no project axiom or `sorryAx` is used.

Lean: `Graph.Audit: independent incidence-search correspondence`

The search-adequacy audit checks incoming subdivision, both fragment-plugging isomorphisms, internal orientation exchange, rule attachment and factorization, explicit simple-context opening and closing, both final correspondence theorems, arbitrary-context soundness, and the boundary and self-loop regressions. No project-specific axiom or admitted proof occurs.

Lean: `Graph.Audit: finite-pattern simple-context search adequacy`

The commentary's source checker recursively scans all Lean modules under `Hypernets/`, including nested graph modules, attributed declarations, noncomputable definitions, abbreviations, and anonymous examples. It verifies that each declaration has a source-fragment link, all linked ranges are valid, and each working source file equals the version at the pinned commit. Diagnostic audit ranges are linked separately. Run it with

```
python3 docs/mathematical-commentary/check_sources.py
```

This syntactic coverage check complements the mathematical exposition; it does not itself check the correctness of English explanations. Source links are immutable GitHub commit links, so future edits cannot move the referenced proof.

From the commentary project directory, `make` checks the links, compiles with `latexmk`, and exports the PDF to the repository's `output/pdf/` directory. The committed PDF can therefore be read without a TeX installation, and its editable sources and reproduction instructions are included alongside it.

Appendix B records completion of the selected general-theorem obligations.

The concrete UAM audit checks the actual fragment constructors, substitution mode and adjacency theorems, transition exclusion, final normality, and non-vacuous substitution and compute executions.

Lean: `Concrete UAM axiom audit`

The tuple and concrete-predicate audit checks both compatibility directions, relatedness, provenance, exact inventory, closed entries, specimen reconstruction, and the rooted consequences of the full quasi-specimen and robustness clauses.

Lean: `Original-hole tuple and concrete safety audit`

Set presentations and the representation obstruction. The audit also checks the two outer round trips, recursive annotated round trips, isomorphism transport and reflection, arbitrary-depth promotion coherence, finite-family uniformization, and the negative binder-erasure theorem.

Lean: `Graph.Audit: set presentations and binder-erasure obstruction`

Actual-content freshening and representation equivalence. The audit checks naming bijections, freshening isomorphisms, actual-content signature consistency, both recursive round trips, isomorphism reflection, quotient inverse laws, relation transport, arbitrary depth coherence, and the repaired counterexample and context-property regressions.

Lean: `Graph.Audit: actual-content representation equivalence`

The finite-union audit also checks surjectivity of interpretation, arbitrary depth coherence of relation pullbacks, and coherent, renaming-invariant layered pre-templates.

Lean: `Graph.Audit: global pre-template coherence`

Search history and the full UAM structural theorem. The audit checks chronological revisiting, actual-state and coordinate rootedness correspondence, stationary replay, focus insertion through plugging, ordinary search reflection, concrete substitution rootedness and uniqueness, the two full-UAM equivalences, and non-vacuous execution regressions.

Lean: `Graph.Audit: full-UAM structural theorem`

Constructed concrete specimen interface. The audit checks insertion and erasure inventories, allowed membership, insertion–expansion commutation, actual relocated specimens, full-UAM context traversal, initial witnesses, the constructed abstract interface, safety and robustness bridges, and the nested-hole execution regressions.

Lean: `Graph.Audit: concrete specimen interface`

Full class machine and concrete forward simulation. The audit checks substitution uniqueness across source classes, full class determinism and refocusing, exact run projection, fixed-source search reflection, the class specimen system, class safety/robustness bridges, concrete counting simulation and refinement, behaviour closure, and actual substitution, compute, and nested-hole regressions.

Lean: `Graph.Audit: full class machine and concrete simulation`

Concrete converse integration. The audit checks comparison reversal, converse robustness bridges, the reverse counting simulation, both refinements, and constructed initial equivalence.

Lean: `Graph.Audit: concrete converse integration`

Independent observations and contextual sufficiency. The audit checks initial endpoint representation, independent recursive pairing, finite exact tuples, constructed initial lifting, both contextual clauses, single-fragment specialisation, and a non-vacuous nested-hole regression.

Lean: `Graph.Audit: concrete contextual theorem`

Unbounded theorem audit. The final audit checks the concrete substitution padding proof, full one-step and exact-run adequacy, global determinism and refocusing, finite-source search locality, the constructed specimen system, both semantic bridges, initial lifting and the final two-clause theorem. Its only logical axioms are Lean's standard `propext`, `Classical.choice` and `Quot.sound`.

```
Lean: Graph.Audit:  complete unbounded theorem
```

98.3 Copying-foundation audit

Eight additional commands inspect substitution composition, operation-path uniqueness, the canonical tree's graph invariants and acyclicity, closure under nested replacement, depth invariance, recursive-isomorphism invariance, and the environment-instantiation regression. All report only standard Lean axioms among propositional extensionality, classical choice and quotient soundness. There is no final Proposition 12.2 theorem to audit yet.

```
Lean: Copying.Audit:  completed prerequisite dependencies
```

Twelve further audit commands cover parallel assembly and path separation, the concrete distributor and its path and label invariants, serial assembly and interface connection, and the ordered-input and atom-interface checks. All report only standard Lean axioms.

```
Lean: Copying.Audit:  distributors and graph assembly
```

Fourteen further commands audit finite termination, arbitrary-tree root tracing and renaming, atom-successor classification, store-resolution uniqueness, store equality, renaming and depth padding, and the weakening-free and cyclic-store regressions. All report only standard Lean axioms.

```
Lean: Copying.Audit:  contraction trees and store tracing
```

Twelve further commands inspect ordered tensor construction, selected-resource wiring, all five translation-clause alphabet proofs, global focus and hole freedom, and the reference-one, bound-variable and single-use-distributor regressions. All report only standard Lean axioms.

```
Lean: Copying.Audit:  literal translation clauses
```

Fifteen further commands check the typed middle-block permutation, both binary clauses, padding counts and depth renaming, the full recursive translation and its global alphabet/focus/hole properties, and the actual Proposition 12.2 graph regressions. All report only standard Lean axioms.

```
Lean: Copying.Audit:  complete recursive translation
```

Thirteen further commands inspect full stability construction, absence of internal vertices and adjacency, unique-edge existence, recursive renaming, copyable outer labels, and the translated-payload and unchanged right-program factorisation results. All report only standard Lean axioms.

```
Lean: Copying.Audit:  stability and copyability
```

Fourteen further commands check positive-path correspondence, absence of self-loops, recursive sourced-hole closure, recursive-isomorphism invariance, term/hole regressions, and the finite zero-input closure counterexample. All report only standard Lean axioms.

```
Lean: Copying.Audit:  binding-free closure and its boundary
```

Twelve further commands audit sole-hole filling, derivation of the source condition, binding-free filling, syntax and graph-constructor composition, exact reconstruction of the original

translation, the concrete observation interface, universal-cost termination equivalence, and the original-program context checks. All use only standard Lean axioms.

Lean: Copying.Audit: contexts and observations

Thirteen further commands audit expansion adjacency reflection and crossings, padding invariance, parallel and directed serial closure, abstraction and full context-translation certificates, and positive and negative regressions. All report only standard Lean axioms.

Lean: Copying.Audit: binding-free context translation

Sixteen further commands audit exact hole occurrence extraction and transport, padding invariance, the complete context certificate, recursive replacement and associativity, sole-hole composition, graph-context congruence with its explicit execution hypothesis, and positive and negative regressions. All use only standard Lean axioms; auditing the conditional congruence theorem does not discharge its execution hypothesis.

Lean: Copying.Audit: certified occurrences and graph congruence

Nine further commands audit the generic expansion–replacement isomorphism, component-wise isomorphism assembly, both serial replacement laws, serial congruence, unary replacement, its explicit sole-hole address, and actual unary graph and term filling. All use only standard Lean axioms.

Lean: Copying.Audit: actual unary term filling

Thirteen further commands audit box-body interchange on both sides of serial assembly, arbitrary-depth serial filling and sole-hole preservation, boxing, the full abstraction clause, nested unary filling, both general nested-context correspondence theorems, and the mixed two-binder regression. All report only standard Lean axioms.

Lean: Copying.Audit: certified nested-context filling

Sixteen further commands audit generic ordered parallel replacement and congruence, deep parallel filling, both application and binary argument laws, sole-hole certificates, one-step and iterated padding compatibility, padding congruence, and the two combined regressions. All report only standard Lean axioms; no copying-machine assumption is introduced by these structural laws.

Lean: Copying.Audit: parallel filling and padding

Thirteen further commands audit exact padding addition, extension to a common allowance with its sole-hole certificate, scoped application/binary congruence, unrestricted constructor congruence, the general graph and term filling theorems, certified packaging, and the mixed-context regressions. All use only standard Lean axioms. The structural theorems have no machine-side premises; they do not assert observational congruence.

Lean: Copying.Audit: unrestricted context filling

Fourteen further commands audit exact padding transitivity, padded closing contexts, iterated execution transfer, observational reflection and isomorphism, the translated-program coherence law, the cofinal criterion, and the negative regression including determinism. All use only standard Lean axioms. The semantic transfer results retain their explicit execution hypotheses; the counterexample confirms that padding invariance is not implicit in them.

Lean: Copying.Audit: common allowances and observation transfer

Thirteen further commands audit padded graph-context filling and congruence, program-padding coherence, scoped isomorphism at a common allowance, equal-depth observational

term-context congruence, exact context depths and their bounds, and the unequal minimal depths of the original proposition terms. All use only standard Lean axioms. The semantic congruence theorem retains its execution-isomorphism, equal-depth and input-equivalence premises.

Lean: Copying.Audit: observational context congruence at equal depths

Eighteen further commands audit one-step padding through the literal assembly and translation constructors, specified-allowance constructor laws, widening and composition of common-allowance isomorphisms, their preservation by all contexts, the direct argument-padding correspondence, unrestricted semantic congruence, and the unchanged proposition-program regression. All use only standard Lean axioms. General semantic congruence retains execution isomorphism, execution padding and input-equivalence premises; it has no equal-depth premise.

Lean: Copying.Audit: unrestricted observational context congruence

Ten further commands audit stability-label padding, one-step and iterated stability equivalence, depth transport, lambda-payload padding isomorphism, and copyability preservation at arbitrary adequate allowances. All use only standard Lean axioms and have no execution premises.

Lean: Copying.Audit: stability and copyability under padding

Six further commands audit certified open-context composition, raw and open-program filling associativity, exact composed-context depths, and the host and filling correspondence for separately translated contexts. All use only standard Lean axioms; none assumes execution or template robustness.

Lean: Copying.Audit: actual translated context composition

Twelve further commands audit resource retagging, the permuted distributor boundary and its exact positions, carrier isomorphism and its arbitrary-depth lifting, and the combined environment and single-use permutation laws. All use only standard Lean axioms, with no execution or template premises.

Lean: Copying.Audit: distributor resource permutations

Ten further commands audit identity, inversion, composition and adjacency for typed boundary isomorphisms, embedding the boundary-fixing interface, and the explicit all-port resource automorphism of the original distributor. They have no execution or template premises and use only standard Lean axioms.

Lean: Copying.Audit: typed boundary isomorphisms

Twenty-one further commands audit selected-resource coordinates and ordered incidences, actual selected-resource renaming and its exact output action, parallel and instance-prefix transport, and all three atomic translation cases. The atomic-case endpoint has no machine or template premises. All audited results use only standard Lean axioms.

Lean: Copying.Audit: literal atomic translation renaming

Twenty-six further commands audit compatible serial gluing, the necessity of matching shared actions, recursive structural safety, recovery of the original boundary-fixing isomorphism, and actual distributor compositions and mismatch regressions. No execution invariance is inferred from these structural laws.

Lean: Copying.Audit: compatible serial renaming and safety

Fourteen further commands audit typed environment decomposition, literal single-use extension,

the complete unary constructor theorem, the unbounded unary-fragment endpoint, and allocation and dereference instances.

Lean: Copying.Audit: original unary translation fragment

Thirty-two further commands audit the original operand shuffle, two-use forest matching, exact atomic actions, the full lambda-free renaming endpoint, unequal-depth application and binary constructors, and positive and negative regressions. The full build and all three audits pass without unfinished proofs or project-specific axioms.

Lean: Copying.Audit: complete lambda-free translation renaming

Forty-one further commands audit typed box pairs, actual interface permutation, compensation, the concrete BPerm pair, its structural safety, the full nested-lambda renaming induction, and positive and negative regressions. The full build passes 289 jobs; the three audits report only standard Lean logical axioms.

Lean: Copying.Audit: full translation renaming via BPerm

The concrete copying-search milestone audits operational determinism, value recognition, typing, isomorphism and padding invariance, plugging preservation, exact phase costs, and traces in the unchanged translation. The full build checks 298 jobs; the abstract, graph and copying audits respectively check 6, 477 and 428 declarations and report only standard logical axioms.

Lean: Copying.Audit: concrete search phase

The reference-allocation milestone adds the concrete local graph rule, freshness and exact store-resolution proofs, argument preservation, representation laws and executions of the unchanged reference translation. The full build now checks 304 jobs; the audits check 6 abstract, 477 graph and 460 copying declarations with standard logical axioms only.

Lean: Copying.Audit: concrete reference allocation

The general dereference milestone audits the arbitrary-tree read rule, resolution and reference preservation, copying handoff, representation laws, safety, and canonical/noncanonical regressions. The full build checks 311 jobs; the audits check 6 abstract, 477 graph and 489 copying declarations, with only standard logical axioms.

Lean: Copying.Audit: general local dereference

The literal copy-rule milestone audits residual wiring, disjoint copies, recursive safety, exact local returns, shared-store resolution, representation laws and lambda/application regressions. The full build checks 318 jobs; the audits check 6 abstract, 477 graph and 528 copying results.

Lean: Copying.Audit: literal copying and shared stores

The stable assignment milestone audits exact store routing, retained old contents, recursive safety, seven-step local executions and representation laws. The full build checks 326 jobs; the audits check 6 abstract, 477 graph and 576 copying results, with only standard logical axioms.

Lean: Copying.Audit: stable assignment and store preservation

The atom-equality milestone audits both graph rules, store-identity outcome uniqueness and context preservation, exact seven-step runs, safety and representation laws. The build checks 335 jobs; the audits check 6 abstract, 477 graph and 652 copying results, with only standard logical axioms.

Lean: Copying.Audit: atom equality and reference identity

The stable micro-beta milestone audits the actual bound-output splice, box opening, exact

boundary and label preservation, recursive representation laws, six-step opening and subsequent body-search costs, and original-body regressions. The full build checks 342 jobs; the audits check 6 abstract, 477 graph and 748 copying declarations with only standard logical axioms.

```
Lean: Copying.Audit:  stable micro-beta application
```

The combined subsystem milestone audits shallow context closure, all local-machine embeddings, exact isomorphism transport, term-token typing, finite-run reflection, the concrete execution instance and contextual congruence, and contextual allocation regressions. The full build checks 349 jobs; the audits check 6 abstract, 477 graph and 830 copying declarations with only standard logical axioms.

```
Lean: Copying.Audit:  combined contextual subsystem
```

The arithmetic and forward-padding milestone audits all numeric local rule families, their exact combined executions and contextual regression, attachment padding laws, complete combined run preservation and padded closed observations. The full build checks 355 jobs; the audits check 6 abstract, 477 graph and 912 copying declarations, using only standard Lean logical axioms.

```
Lean: Copying.Audit:  arithmetic and forward execution padding
```

The exact padding-reflection milestone audits typed truncation and its two round trips, isomorphism and copyability reflection, descent of every local and contextual match, preservation of representability, exact finite-run reflection, unbounded padding equivalence and concrete full term-context congruence. All 89 new declarations are audited. The full build checks 362 jobs; the audits check 6 abstract, 477 graph and 1001 copying declarations, with only standard Lean logical axioms.

```
Lean: Copying.Audit:  exact padding reflection and concrete congruence
```

The canonical-selection milestone audits all 103 new declarations: occurrence maps, copy-head and store-destination independence, comparison-branch exclusion, ordered numeric matching, the unique-choice theorem for arbitrary contextual rewrites, stable-input inertness, recognition padding reflection, recursive runtime safety and the regression consequences. The complete build passes 369 jobs; the audits check 6 abstract, 477 graph and 1104 copying declarations with only the three standard logical axioms listed above.

```
Lean: Copying.Audit:  match selection and runtime safety
```

The allocation-coherence milestone audits all 39 new declarations: replacement state congruences, canonical allocation for arbitrary contextual matches, bounded and unbounded successor coherence, exact cost laws, literal-translation regressions and arbitrary-host successor comparison. The full build passes 373 jobs. The three audits inspect 6 abstract, 477 graph and 1143 copying declarations, using only the standard logical axioms listed above. The standard check script now runs all three audits.

```
Lean: Copying.Audit:  allocation coherence and exact costs
```

The reference-equality milestone audits all 61 new declarations, covering both local replacement isomorphisms, complete contextual successor coherence, exact costs, arbitrary-host regressions and unchanged translated store contents. The full build passes 379 jobs. The audits now inspect 6 abstract, 477 graph and 1204 copying declarations, with only `propext`, `Classical.choice` and `Quot.sound`; no project-specific axioms or `sorryAx` occur.

```
Lean: Copying.Audit:  reference-equality coherence and exact costs
```

The assignment milestone audits all 78 new declarations: typed target exchange, its locality

and expansion laws, unique write matches, canonical successor transport, the original-reduct isomorphism, complete contextual coherence, exact costs and unchanged-translation regressions. The full build passes 388 jobs. All three audits pass: 6 abstract, 477 graph and 1282 copying declarations (1765 total), using only `propext`, `Classical.choice` and `Quot.sound`. No project-specific axiom or `sorryAx` occurs.

`Lean: Copying.Audit: assignment coherence and exact costs`

The dereference and complete-store milestone audits all 71 new declarations: the finite graft, state-level exchange transport, unique read matches, canonical contextual successors, the original-reduct isomorphism, complete store-operation coherence, exact read cost handoff and unchanged-translation regressions. The full build passes 397 jobs. All three audits pass: 6 abstract, 477 graph and 1353 copying declarations (1836 total). The only dependencies are `propext`, `Classical.choice` and `Quot.sound`; there are no project-specific axioms or `sorryAx` dependencies.

`Lean: Copying.Audit: dereference and complete store coherence`

The arithmetic-coherence milestone audits all 85 new declarations: survivor certificates, recursive renaming and contextual transport, derivation from the original rules, whole-state coherence, exhaustive classification of remaining heads, exact costs and host/padding regressions. The full build passes 406 jobs. The three audits check 6 abstract, 477 graph and 1438 copying declarations (1921 total), with only `propext`, `Classical.choice` and `Quot.sound`. No project-specific axiom or `sorryAx` occurs.

`Lean: Copying.Audit: arithmetic coherence and remaining machine cases`

The application-coherence milestone audits all 115 new declarations: partial incidence presentations and their realizations, actual box-opening certificates, recursive renaming and all context transports, independent-body comparison, complete successor coherence, the copying-only classification, and exact full-machine cost regressions. The full build passes 419 jobs. The three audits check 6 abstract, 477 graph and 1553 copying declarations (2036 total), with only `propext`, `Classical.choice` and `Quot.sound`. No project-specific axiom or `sorryAx` occurs.

`Lean: Copying.Audit: application coherence and copying frontier`

The copying-coherence milestone audits all 167 new declarations: payload normal forms and rigidity, complete source-relative presentations and their literal realizations, every contextual transport, independent-certificate comparison, full-machine successor determinism up to representation, exact prefix costs, unique successful-run lengths, and copying/padding/aliasing regressions. The full build passes 436 jobs. All three audits pass: 6 abstract, 477 graph and 1720 copying declarations (2203 total). Their only axiom dependencies are `propext`, `Classical.choice` and `Quot.sound`; no project-specific axiom or `sorryAx` occurs.

`Lean: Copying.Audit: complete copying and machine successor coherence`

The refocusing milestone audits all 102 new declarations: rooted search, operation reachability, retained-payload barriers, exact copying-prefix replay, the intrinsic/extrinsic partition, exact search history, single-head replacement, contextual allocation refocusing, remaining-operation reductions, padding laws and regressions. The full build passes 446 jobs. The audits contain 6 abstract, 477 graph and 1822 copying declarations (2305 total), all passing with only `propext`, `Classical.choice` and `Quot.sound`. No project-specific axiom or `sorryAx` occurs.

`Lean: Copying.Audit: intrinsic and allocation refocusing`

References

[1] Dan R. Ghica, Koko Muroya, and Todd Waugh Ambridge. *A robust graph-based approach to observational equivalence*. Logical Methods in Computer Science 21(2:8), 2025. Reference version: arXiv:1907.01257v5.

[2] *Hypernets in Lean 4*. Repository at revision `dbb4d4c`.

A Technical terms

This appendix is alphabetical. Click a term or its page number to jump to its definition in this PDF. Related terms may share a definition. Descriptions of abstract interface predicates state what the interface supplies; they do not claim that a concrete UAM interpretation has already been constructed.

Actual translated-context composition p. 390

Certified addressed plugging of two separately translated contexts, with required padding, is recursively isomorphic to the direct translated composite.

Actual-content label p. 235

An actual named box body with a typing proof; its external counts are read from the name of its sole input rather than stored as separate label fields.

Actual-state refocusing p. 255

Preservation of the rooted predicate by a relation on named states, interpreted on recursive state classes.

Addressed context composition p. 350

The site obtained by inserting an inner context at an outer site and following the inner hole in the resulting graph.

Adjacency p. 54

Edges e, f are adjacent when a vertex occurs in both the target list of e and the source list of f.

Admissible behaviour p. 214

A relation on actual states certified to contain only pairs of compute shape; determinism is a separate property.

Aligned substitution entry p. 217

One original hole name and its two type-correct replacement graphs, recording their shared depth and ordered interface.

Allocation successor coherence p. 511

Any two actual rewrites at the same ref request produce recursively isomorphic graphs with matching resumed token positions.

Allocation up to isomorphism p. 434

A stable allocation pair with independently supplied recursive graph isomorphisms to its literal source and target; proved preserved by forward padding.

Allowance-indexed subsystem execution p. 479

The dependent union of fixed-allowance combined machines, with exact finite-run reflection; exact padding equivalence is now proved for this concrete execution.

Allowed context p. 34

A context satisfying the supplied predicate Allowed in an observation interface.

Alphabet p. 60

A type of edge labels equipped with a signature assigning their input and output wire types.

Annotated set hierarchy p. 227

A finite-depth graph-labelled hierarchy using arbitrary vertex carriers while retaining the bound and free counts on each box label.

Annotated term context p. 340

A lambda-context whose hole and outer scopes are separately indexed, retaining variables bound around the hole.

Application cost handoff p. 556

Exactly six transitions precede the actual opened-body continuation, both locally and under an arbitrary serial host.

Application opening certificate p. 551

A source match, a realization by the whole successor graph, and the exact resumed-token equation; derived from the original rewrite relation.

Application source match p. 548

The actual application, lambda and box frame, its two internal wires, argument attachment and recursively identified body.

Application successor coherence p. 555

Any two actual contextual application rewrites of one source request produce recursively isomorphic entire successor states.

Arbitrary contraction tree p. 318

A finite acyclic contraction/weakening graph with one output; connection to its unique root is derived.

Arithmetic cost handoff p. 545

Binary arithmetic attached below an arbitrary host contributes exactly seven steps before the host continuation resumes.

Arithmetic deletion certificate p. 538

Injective maps of surviving vertices and edges into the original source, with exact images and full label/incidence laws determining the whole result.

Arithmetic request p. 542

A unary negation operand or a binary operation with ordered numeric operands, determining both recognition and the numeric result.

Arithmetic successor coherence p. 543

Any two actual contextual arithmetic rewrites of the same source request produce recursively isomorphic complete states, including the resumed token.

Assignment interface p. 451

One term input and the other atom-reference inputs, with one old-content term output followed by the new value outputs.

Assignment match p. 523

Source data identifying the assignment head, unique resolved store and ordered instance, new-value and old-value ports.

Assignment resolution preservation p. 454

Exact preservation of the atom-routing count to the selected store before and after its term connection is updated.

Assignment rule p. 453

The stable Figure 19(c) update reconnecting the original store to the new value, weakening the instance and old contents, and returning unit.

Assignment subsystem p. 456

Intrinsic search plus the stable local assignment rule, with final normality and checked seven-step literal-value executions.

Assignment successor coherence p. 527

Any two actual writes at the same request have recursively isomorphic complete successor graphs with matching resumed token positions.

Atom-comparison decision p. 503

A Boolean fixed by whether the two ordered arguments resolve to the same store occurrence.

Atom-store resolution p. 319

An exact sequence of atom hops ending at a store, without following its term output.

Atomic translation renaming p. 396

Bijective resource renaming for the original variable, atom and constant translation clauses, preserving their actual graph wiring.

Backtracking mode p. 35

The abstract focus mode backtrack, one of the three constructors of the focus datatype.

Behaviour closure under isomorphism p. 269

Extrinsic rules closed under independent recursive endpoint renaming; this leaves full class transitions unchanged.

Behaviour invariance under isomorphism p. 265

Preservation of extrinsic transitions by independent recursive renaming of both endpoints.

Bijection p. 47

A pair of forward and inverse functions satisfying both inverse equations.

Binding-free copying graph p. 336

A focus-free graph with no positive path from a contraction, instance, box or hole source to a hole source, at any depth.

Binding-free translation certificate p. 344

The theorem that every annotated lambda-context has a binding-free graph translation at every finite hole allowance.

Binding-freedom under padding p. 344

Preservation and reflection of the full recursive binding-free condition when unused box allowance is increased.

Blocked input p. 252

A term input from which no operation edge can leave; this suffices to exclude operation paths to outputs.

Body consistency p. 230

The condition that box edges with literally equal contents have equal external binder metadata; needed by naive binder erasure.

Bound-output splice p. 469

A two-component graph connecting body output zero to the argument input while retaining separate ordered free-body and argument output blocks.

Boundary carrier actions p. 401

The separately extracted bijections of internal vertices, input positions, and output positions of a typed boundary isomorphism.

Boundary compensation p. 419

Undo a proved output action by inverse interface permutation to recover a boundary-fixing recursive isomorphism.

Boundary finality p. 108

Backtracking mode with the focus source at an input; on a state this is the final-state characterization of Proof F.4.

Boundary initiality p. 108

Search mode with the focus source at an input; preserved through all certified fillings.

Boundary normalization p. 224

The bijective change from an arbitrary vertex carrier to disjoint internal, input-index, and output-index carriers.

Boundary padding renaming p. 414

The proved preservation of typed boundary isomorphisms and their exact resource actions under the original recursive depth padding.

Boundary port p. 48

An input or output constructor carrying an index in the corresponding ordered boundary list; input and output constructors are disjoint.

Boundary transport p. 94

Reindexing boundary ports along equal ordered interfaces while preserving internal and edge carriers.

Boundary-list isomorphism p. 233

An isomorphism preserving ordered input and output lists, allowing interface indices to have different presentations before transport.

Box p. 60

A label storing bound/free counts n, m and a body of type $* \to *^{n+m}$; its outer edge has type $T^n(*) \to *^m$.

Box context filling p. 359

The addressed-filling law that retains an enclosing box and replaces only its selected body fragment.

Box-label replacement p. 64

Changing the body of one selected box edge while retaining its binding metadata and all outer incidence data.

Box-permutation pair p. 417

Typed body and outer permutations agreeing on free resources and fixing every bound output, with the bound-prefix offset explicit.

Canonical allocation successor p. 510

The source host with its selected ref edge replaced by the original cell prefix, resuming search at the retained old wire.

Canonical assignment successor p. 524

A proof construction that exchanges the new and old value targets, then replaces the assignment head by unit and two weakenings.

Canonical equality successor p. 516

The complete source graph with its comparison head replaced by a Boolean and two weakenings, resuming at the retained request wire.

Canonical read successor p. 532

The source-determined read result formed by grafting at the head and exchanging the fresh feedback target with the retained content target.

Canonical replacement state p. 510

A one-edge replacement paired with search mode and the retained-host image of the old token vertex.

Canonical root p. 144

The initial state class obtained by erasing a state's focus and attaching a search focus at its unique input.

Certified composed plugging p. 103

A context constructor using pairwise disjoint retained and final inventories, without requiring intermediate validity.

Certified deep fresh plugging p. 84

The context constructor that packages addressed replacement with a proof of exact site freshness, permitting reuse of consumed labels.

Certified filling execution p. 167

A sequence of prescribed shallow fillings whose residual family consists of identity copies when it stops.

Child occurrence p. 69

An occurrence inside a box body. Primitive labels have no child occurrences.

Class input safety p. 267

The three unrestricted input-safety alternatives evaluated on the full class machine.

Class quasi-specimen p. 267

An unconditional concrete specimen witness with matching endpoint modes and conditional rooted comparisons on classes.

Class robustness p. 268

The unrooted, stuck-tail, or matching quasi-specimen alternatives for remote rewrite successors on the class machine.

Class-valued specimen system p. 266

The proved concrete locality interface whose endpoints are recursive state-isomorphism classes.

Cofinal observation allowances p. 377

An arbitrarily large collection of common adequate depths; equivalence there reflects to every adequate depth under the execution representation laws.

Combined copying subsystem p. 476

The local copying, store, application and arithmetic rules together with intrinsic search and explicit shallow contextual rewrite closure.

Combined rule choice p. 504

Source matching data: rule family, comparison answer, copy-head primitive, or ordered arithmetic numbers.

Combined source recognition p. 504

A predicate reading rule-choice data from the source graph; all actual rewrites have a unique recognized choice.

Combined successor determinism p. 570

Any two successors in the full original combined machine are recursively isomorphic states; the unbounded version concludes representation equivalence.

Common-allowance isomorphism p. 384

Existence of an adequate common depth at which the two original graphs, padded by the original operation, are recursively isomorphic.

Common-depth equivalence p. 284

Two finite presentations become equal after possibly different numbers of paddings.

Common-store equality fragment p. 459

Two instances feeding one arbitrary atom contraction tree and store; comparison returns true and weakens both instances.

Compatible provenance p. 72

The condition that no hole label in the replacement occurs at a retained host origin of the provenance map.

Compatible serial renaming p. 402

Agreement of the left output and right input permutations on every shared interface position, ensuring the original serial insertions commute.

Complement restriction p. 95

The bijection between the complements of corresponding selected elements, used for surviving host edges.

Complete specimen tuple p. 220

A finite tuple with pointwise related closed replacements, distinct names, and exact coverage of a recursive specimen's original holes.

Complete term translation p. 329

Recursive construction of Figure 16 hypernets for every scoped term, retaining the prescribed sharing graphs.

Composed site p. 101

The address of an inner selected occurrence after inserting its host at an outer site.

Compute shape p. 214

Rewrite mode entering an active operation at the source, and search mode at the target; the effect itself is supplied by behaviour.

Concrete entering focus p. 193

A focus whose target is a source of a shallow hole-labelled edge in its original context.

Concrete exiting focus p. 193

A focus whose source is a target of a shallow hole-labelled edge in its original context.

Concrete input-safety p. 222

The paper's three stuck, settled-specimen, or non-rewrite quasi-specimen alternatives for every entering search specimen.

Concrete observation interface p. 276

Independent context filling and initialisation, without template-dependent observation data.

Concrete output-closure p. 161

The requirement that each related pair in a concrete pre-template has at least one one-way member.

Concrete pre-template p. 161

An interface-indexed relation on actual hypernets, with proofs that its related members are globally focus-free.

Concrete quasi-specimen p. 221

A mode-compatible specimen witness that exists unconditionally and supplies rooted comparisons when both endpoint states are rooted.

Concrete recursive specimen p. 178

A unique-hole focussed context with recursive related fillings, an allowed erasure, and actual hole-free filled states.

Concrete refocusing p. 150

Preservation of concrete rootedness by each transition of a relation on recursive state classes.

Concrete robustness p. 222

The paper's unrooted, stuck-tail, or matching quasi-specimen alternatives for non-entering rewrite specimens.

Concrete rootedness p. 145

Existence of a finite search-only run from the canonical initial attachment to the state's recursive class.

Concrete state p. 110

A globally focussed, hole-free hypernet of type $* \to \epsilon$, with search, substitution, and extrinsic compute transitions.

Concrete substitution transition p. 212

Application of the substitution fragments at the same shallow hole of a focus-free simple context.

Concrete template p. 223

An input-safe layered pre-template satisfying output closure at every available depth.

Concrete term-context congruence p. 497

Every original term context preserves observational equivalence for the concrete combined execution, with its representation laws proved and no equal-depth restriction.

Concrete typed net p. 49

A finite labelled hypergraph with ordered boundaries and certified distinctness, incidence uniqueness, coverage, and signature typing.

Constructed concrete specimen system p. 262

The abstract locality interface instantiated with actual recursive specimens and proved full-UAM graph laws.

Constructor padding law p. 382

A proved equality or recursive isomorphism commuting depth padding with an unchanged Figure 16 graph constructor.

Context p. 63

A hypernet in which each complete typed hole label occurs at most once over all nesting depths.

Context depth function p. 381

The exact outer allowance selected by the original context interpreter from a hole allowance, using successors and fixed-sibling maxima.

Context-only traversal p. 35

The specimen-interface law supplying paired context steps for a rooted, settled specimen whenever either endpoint can step.

Context-prefix replay p. 248

Extraction of a pre-rewrite search prefix in the unchanged context and its transport to a new filling.

Contextual equivalence p. 34

Mutual contextual refinement: both directions hold for every pair of observations in the same allowed context.

Contextual refinement p. 34

The requirement that observations of two fragments in each common allowed context produce states related by the chosen state refinement.

Converse class quasi-specimen p. 270

A concrete quasi witness whose comparisons enter the original right endpoint and leave the original left endpoint.

Converse class robustness p. 271

Robustness for steps from the original right endpoint, with matching runs from the left and converse quasi witnesses.

Coordinate characterization of state search p. 132

Search from a realized state is equivalent to a coordinate step on its original graph followed by a focussed endpoint isomorphism.

Coordinate search machine p. 183

Search on mode–vertex pairs, with successful return at an input; defined also for open contexts.

Copy resolution p. 501

An exact term-sharing trace from a selected wire to an instance or operation payload head.

Copy rule p. 446

The intrinsic Figure 17 transformation duplicating a copyable payload, retaining its weakened residual tree and inserting both ordered output distributors.

Copy subsystem p. 447

Intrinsic search plus the concrete copy transition; it has checked local return executions but is not the complete copying UAM.

Copy-head independence p. 502

Independently chosen tree/payload matches at one wire select the same host payload-head edge, without assuming the trees isomorphic.

Copyability reflection p. 494

The proved converse padding law: extra representation allowance cannot make a previously non-copyable payload copyable.

Copyable payload p. 334

The instance or operation-with-boxes shape of Definition 10.2, before adding sharing distributors.

Copying canonical tree p. 309

The ordered right-spine graph of Definition 10.1, with one weakening and one contraction per input.

Copying closing observation p. 342

Initialization of the actual graph obtained by filling a concrete binding-free, single-hole closing context.

Copying coherence frontier p. 555

The historical exhaustive classification isolating term-contraction requests; the complete copying comparison now discharges that remaining case.

Copying global label predicate p. 308

A primitive-label property required on every edge at every box depth.

Copying intrinsic refocusing p. 573

Preservation of search rootedness by the original search and copying transitions, proved through retained-occurrence prefix replay.

Copying operation reachability p. 574

Reachability from an external input using only operation edges; a necessary invariant of rooted search, not a definition of rootedness.

Copying payload normal form p. 558

An original primitive corolla or literal lambda with its complete boxed body; every copyable payload is recursively isomorphic to such a graph.

Copying refocusing cost p. 577

The exact law that the new graph reaches its resumed copy token in one fewer search step than the old graph needed to reach the rewrite request.

Copying replacement closure p. 308

Closure of a graph class under well-typed replacement at any selected edge, including inside boxes.

Copying retained occurrence p. 576

An old source vertex or edge identified inside the actual reduct through its successor certificate, with precise exceptions to preserved incidences.

Copying runtime alphabet p. 506

The translation alphabet extended with stores, closed under all combined executions at every recursive depth.

Copying scoped term p. 302

A term with separate finite variable and atom indices; lambda extends only the variable scope.

Copying search barrier p. 575

A sharing edge before the retained payload excludes operation reachability there and prevents the rooted search prefix from using changed payload incidences.

Copying search completion p. 429

Backtrack mode at an external input; the successful finality predicate of the search-only phase machine.

Copying search phase p. 425

Intrinsic token traversal on the copying graph hierarchy; rewrite mode suspends the phase for pending copy or compute transitions.

Copying search rootedness p. 574

Existence of a pure-search run from a specified external input on the current graph of the state.

Copying search transport p. 426

Carrier bijections preserving primitive labels, ordered incidence and input status, which preserve and reflect search runs at exact costs.

Copying search value recognition p. 427

The theorem that producing backtrack mode identifies an actual passive-operation or instance edge and its stable primitive corolla.

Copying source presentation p. 562

A complete incidence description retaining every old source occurrence and adding one clone, weakening, output links and the original distributors.

Copying stable graph p. 332

The Definition 10.3 fragment with instance/passive edges, atom/thunk outputs, and reachability from its single input.

Copying successor certificate p. 562

A source match, realization of its whole-result presentation, and resumed-token equation, derived from an actual copying rewrite.

Copying successor coherence p. 558

Independent copying rewrites at one request have recursively isomorphic complete successor graphs and corresponding tokens.

Copying successor match p. 560

A normal payload occurrence, active ordered contraction sources and a copy-resolution trace, independent of a chosen tree boundary.

Copying wire p. 305

A term, atom, or binder-indexed thunk wire; only the first two admit sharing edges.

Corolla p. 50

A graph with one operation edge, no internal vertices, and precisely that operation's ordered input and output boundaries.

Corolla attachment p. 204

The map from a selected edge's corolla inputs and outputs to its ordered host source and target ports; it need not be globally injective.

Corolla copy p. 158

A one-edge filling retaining an original label and its complete ordered input and output boundaries.

Corresponding sites p. 96

Addresses related by edge bijections at each enclosing level, with the same local replacement interface.

Cost converse p. 30

The relation Q^{-1} defined by $a\,Q^{-1}\,b$ exactly when $b\,Q\,a$.

Cost preorder p. 29

A reflexive and transitive relation on natural-number execution counts; additional algebraic properties are imposed separately.

Counted execution p. 27

The relation $s \longrightarrow^{n} t$, recording exactly n machine transitions, including the zero-step execution from a state to itself.

Counting simulation p. 32

A relation preserving finality and matching steps using related execution counts and an up-to relation, with an explicit stuck-execution alternative.

Cumulative local padding p. 370

The padding accumulated at a hole through successive common-depth joins; its increment is the selected host branch's depth deficit.

Deep address transport p. 89

The map obtained by converting a deep address to fibre coordinates, applying a hole embedding, and converting back to a target address.

Deep freshness p. 83

Disjointness between the replacement hole inventory and the host holes retained along the selected address; it is necessary and sufficient for composing two contexts.

Deep occurrence p. 62

An address that records an enclosing box edge and recursively an occurrence in its body, distinguishing equal bodies at different locations.

Deep plugging p. 64

Typed replacement at an addressed site, recursively descending through boxes and rebuilding the selected enclosing labels.

Deep retained hole p. 82

A host hole under an unselected edge at some level of the address, as defined recursively without reference to the replacement graph.

Depth bound p. 61

The index bounding permitted box nesting in a hypernet; a graph need not use all of the available depth.

Depth class embedding p. 281

The injective map of recursive focussed state classes induced by one padding.

Depth inventory isomorphism p. 280

A bijection of recursive addresses across depths preserving their optional hole labels.

Depth padding p. 279

Increase the allowed depth while retaining outer graph incidence and recursively padding box bodies.

Depth promotion p. 62

Raising a hypernet's available depth bound by relabelling primitives and recursively promoting box bodies, without changing outer incidence.

Depth promotion p. 229

Increasing an allowed nesting depth while preserving incidence and recursively promoting box contents.

Depth representability p. 491

Recursive certificate that every actual nested box fits one allowance lower; primitive labels always fit.

Depth truncation p. 490

Auxiliary typed lowering that preserves primitive labels and incidence; excessive boxes become typed placeholders only outside the certified representable fragment.

Depth-independent contextual refinement p. 298

Contextual refinement for every finite representative and original context, evaluated on the unbounded machine.

Depth-independent graph class p. 240

A finite-depth actual-content graph modulo recursive isomorphism and depth promotion.

Dereference interface p. 438

One term input followed by the other atom references, and one term output; the reading instance supplies the first atom-tree input internally.

Dereference match p. 532

Source incidence and trace evidence identifying a read head, its uniquely resolved store, the stored-content port and the trace length.

Dereference resolution preservation p. 440

The theorem that a matched atom tree resolves to its designated store before and after reading at exactly the same atom-trace counts.

Dereference rule p. 439

The Figure 19(b) graph rewrite retaining the atom tree and store, weakening the consumed instance, and inserting the actual two-use term distributor.

Determinism p. 27

The property that any two one-step successors of the same state are equal.

Determinism modulo isomorphism p. 255

Uniqueness of transition targets up to recursive focus-preserving graph isomorphism.

Disjoint hole inventories p. 71

The condition that no complete typed hole label occurs in both graphs, including occurrences inside boxes.

Distinct-store equality fragment p. 460

Two instances feeding separate arbitrary atom trees and explicit distinct stores; comparison returns false while preserving both stores.

Distributor p. 313

A forest of canonical sharing trees whose input for resource i at use j occurs at position $i + kj$.

Distributor resource permutation p. 392

A bijective reordering applied consistently to every use and output of a literal distributor, preserving its graph up to carrier renaming.

Distributor serial renaming p. 404

The proved automorphism of the original many-use distributor followed by its single-use forest, using the same resource permutation in both.

Edge p. 49

An element of a net's finite edge carrier, carrying an operation label and ordered source and target vertex lists.

Edge hole inventory p. 80

The hole labels observed in one edge fibre, including all descendants of its label.

Empty graph p. 50

The net with no internal vertices, no edges, and empty input and output boundaries.

Entering specimen p. 35

A specimen satisfying the interface's supplied entering predicate. Its concrete interpretation by hole incidence is not yet instantiated.

Equal-depth term-context congruence p. 380

Preservation of all-allowances observational equivalence by every original term context, for equally deep input translations and an isomorphism-invariant execution.

Equality subsystem p. 465

Intrinsic search plus the two concrete atom-equality rewrites, with final normality and exact seven-step local executions.

Equality successor coherence p. 519

Any two actual atom-equality rewrites from the same request have recursively isomorphic whole successors and matching token positions.

Erased-slot depth condition p. 493

Plugging requires representability of the filler and surviving host labels, with no condition on the label of the erased slot.

Erasure–expansion commutation p. 189

An explicit isomorphism between erasing a retained expanded focus and expanding the erased context, preserving the joined coordinate.

Erasure-based state search p. 128

Search between actual states, defined by a coordinate step on the erased source and a focus-preserving isomorphism to the realized target.

Exact cost order p. 30

The preorder E with $a\,E\,b$ exactly when $a = b$, requiring identical successful execution counts in refinement.

Exact graph trace p. 317

A finite sequence of relation steps indexed by its length; this length is not machine execution cost.

Exact prefix cost law p. 571

A given finite prefix contributes exactly its length to every successful run, even when its successor has no successful continuation.

Execution isomorphism invariance p. 351

The requirement that recursively isomorphic closed graph presentations agree on successful termination.

Execution padding invariance p. 376

The explicit requirement that padding an admissible closed program once preserves its successful termination observation.

Execution padding reflection p. 496

Every run from a padded state has a run of identical length at the original allowance, with an isomorphic padded endpoint.

Expansion–replacement interchange p. 353

The isomorphism between replacing a component edge after finite expansion and replacing it in that component before expansion.

Explicit-hole context search p. 209

Finite interaction fragments plugged into the same explicit simple context; proved equivalent to erasure-based state search.

Extracted state coordinate p. 126

The original mode and joined term vertex in the generally erased underlying graph of an arbitrary closed state.

Fibre embedding p. 89

An injective map between label fibres that preserves the complete hole observation at every position.

Fibre provenance p. 75

An injective, hole-observation-preserving map from a changed label's occurrence fibre to retained old fibre addresses or replacement occurrences.

Final state p. 27

A state satisfying a machine's successful-finality predicate; the machine laws require that it has no outgoing transition.

Finite enumeration p. 47

A finite list containing every element of a carrier, together with a completeness proof; the list may contain repetitions.

Finite filling sequence p. 109

A finite sequence of focus-free addressed replacements avoiding the transported focus, preserving its boundary status.

Finite-depth stage p. 284

A depth paired with a recursive state class at that depth.

Focus erasure p. 122

Deleting the unique focus edge and its internal target, contracting the target onto the source while proving all graph invariants.

Focus insertion p. 116

Splitting a non-output term vertex and connecting its old image to a fresh internal vertex by a new focus edge.

Focus mode p. 35

One of search, backtrack, or rewrite; this datatype alone does not specify a graph position.

Focus round trip p. 124

Erasure followed by reinsertion, with an explicit isomorphism restoring original names, the focus edge, and its endpoints.

Focus-avoiding site p. 109

A shallow site different from the focus edge, or any site strictly inside a box.

Focus-extended filling family p. 259

The original filling family on old edges, extended by a retained corolla for the new focus.

Focus-free hypernet p. 107

A graph with no focus labels at any depth, including inside every boxed body.

Focussed hypernet p. 107

A hypernet with exactly one global focus, shallow, self-acyclic, and not exposed.

Focussed isomorphism p. 126

A graph isomorphism preserving the distinguished focus edge and mode, hence its endpoints and boundary observations.

Focussed recursive isomorphism p. 136

An isomorphism permitting independent names inside boxes while preserving the distinguished focus edge and its mode.

Focussed-context coordinate p. 191

The mode and joined term vertex of a focussed context after erasure; the context may still contain holes.

Forward execution padding p. 487

Every actual combined run lifts to a larger representation allowance with the same endpoints under padding and exactly the same cost; converse run reflection is separate.

Fragment p. 34

An element of the family F_i in an observation interface, matched with contexts of the same interface index i.

Fresh allocation store p. 432

The new store occurrence in the allocated prefix, proved distinct from every retained argument edge; resolved inserted references also cannot alias retained host stores.

Fresh naming p. 232

A bijective assignment of names consisting of two chosen prefix coordinates and a distinct enumeration index.

Full translation renaming p. 417

Bijective renaming of every original term by a finite BPerm derivation and a certified final boundary isomorphism; semantic interpretation remains open.

Full UAM p. 214

The concrete machine whose steps are search, substitution, or admissible extrinsic behaviour, with proved final normality.

Full UAM class transition p. 265

Existence of representatives of the source and target classes making one actual UAM step.

Fully quotiented substitution p. 174

Substitution whose arguments are a host-with-site class and a replacement class, and whose result is a recursive graph class.

General term-context congruence p. 386

Preservation of the existing observational equivalence by every original annotated context, under explicit execution representation laws and without equal input depths.

Global behaviour p. 290

An extrinsic relation on unbounded states, restricted by actual compute shape at some finite presentation.

Global context set p. 299

A set of interpreted depth-independent context classes, inducing automatically invariant finite membership.

Global converse robustness p. 297

Robustness with the actual two specimen filling vectors exchanged.

Global determinism p. 292

Uniqueness of unbounded successor states for the extrinsic relation; proved to extend to the full UAM.

Global input safety p. 296

The three concrete safety alternatives, with execution, stuckness and comparisons on the unbounded full UAM.

Global observation p. 297

Independent filling and initial attachment in a finite original context, followed by projection to the union.

Global pre-template p. 241

A relation on depth-independent graph classes whose related typed representatives are focus-free.

Global quasi-specimen p. 295

An unconditional concrete specimen and mode agreement with conditional rooted comparisons on the unbounded machine.

Global refocusing p. 294

Preservation of global rootedness by extrinsic transitions, proved to extend to full executions.

Global robustness p. 296

The concrete robustness alternatives quantified over every global rewrite successor of a specimen endpoint.

Global rootedness p. 293

Search reachability from an initial state on the union, equivalent to finite rootedness of every representative.

Global specimen p. 295

An actual finite-depth context with paired fillings, whose endpoints are evaluated on the unbounded machine.

Global UAM step p. 291

An intrinsic transition or a transition of the unbounded extrinsic behaviour.

Graph signature p. 48

The assignment of an ordered input-label list and output-label list to each operation label.

Graph-body package p. 235

A named body graph and its actual output count, determined by its own ordered interface.

Head replacement refocusing p. 580

Replacing one operation edge by a graph with the same interface preserves rooted search at its retained source port.

Higher-order head p. 544

An application or term-contraction head used in a historical two-family classification; both families now have complete successor coherence.

Hole boundary p. 185

A searching coordinate at an original shallow hole source, or a backtracking coordinate at one of its targets.

Hole embedding p. 89

An injective map of graph occurrences preserving hole labels and the absence of a hole; it need not itself preserve incidence.

Hole inventory p. 71

The collection of complete hole labels h for which some shallow or deep occurrence observes h.

Hole label p. 60

A natural-number name together with input and output wire-type lists. Equality compares the complete typed label, not just its name.

Hole lookup p. 63

The recursive function returning the typed hole label at an occurrence, or no label when that occurrence is not a hole.

Hole presence p. 71

The predicate $\mathsf{HasHole}(G, h)$, witnessed by an occurrence whose hole lookup returns h.

Hole-free graph p. 71

A graph having no hole label at any occurrence or nesting depth.

Host embedding p. 51

The map retaining host vertices during shallow replacement, tagging internal names and fixing each external boundary position.

Hypernet p. 61

A typed net over a finite-depth alphabet whose labels may include boxes with bodies over the preceding alphabet.

Image of padding p. 492

The recursively isomorphic copies of padded graphs, proved to be exactly the graphs satisfying depth representability.

Improvement cost order p. 30

The preorder I with $a\,I\,b$ exactly when $b \leq a$; the target successful execution takes no more steps than the source.

Incidence coverage p. 49

The net laws saying that exactly non-output vertices occur in some source list and exactly non-input vertices occur in some target list.

Incidence search p. 200

An original-graph local interaction together with explicit target rewiring; proved equivalent to erasure-based state search.

Incidence uniqueness p. 49

The net laws allowing a vertex to occur in the sources of at most one edge and in the targets of at most one edge.

Incoming incidence trace p. 147

A reverse path recording each target and source incidence, without restricting edge labels.

Incoming-side focus subdivision p. 201

Inserting a focus before a non-input term port by splitting target incidences; permits a focus at an output boundary.

Independent filling p. 274

Template-free recursive substitution that never revisits inserted fragments.

Independent sites p. 171

Addresses that diverge at distinct edges, possibly inside a shared chain of boxes, so neither selects an ancestor of the other.

Initial focus attachment p. 120

Constructing a genuine initial state by inserting search mode at the sole input of a focus-free, hole-free graph.

Initial graph observation p. 275

The recursive state class obtained by initialising a closed focus-free filled graph.

Initial lifting p. 34

The contextual-lifting hypothesis placing initial observations of related fragments into the specified lifted state relation.

Initial observation specimen p. 275

An allowed initial specimen constructed from an unfocussed paired filling.

Input-safety p. 37

The abstract condition on rooted, entering search specimens: a stuck alternative, a cost-related execution to a settled specimen, or a quasi-specimen alternative.

Insertion map p. 51

The map tagging replacement internal vertices freshly and identifying its boundary positions with the selected host edge's indexed ports.

Insertion–erasure cancellation p. 127

Erasing an inserted focus recovers the original graph up to isomorphism and restores its insertion position.

Interchange result p. 172

The two residual addresses after independent substitutions together with a recursive isomorphism comparing the final graphs.

Interface p. 49

For a concrete net, the ordered input and output wire-label lists; matching lists enforce type-correct replacement.

Interface permutation p. 419

Move boundary output positions by a typed bijection while retaining carriers, labels and ordered incidences.

Internal arithmetic targets p. 538

The arithmetic head targets only internal operand wires, so deleting them cannot remove host boundary wires after insertion.

Internal vertex p. 48

A vertex from the internal carrier, tagged separately from all input and output boundary positions.

Isomorphism p. 57

Bijections of internal vertices and edges preserving labels and ordered incidence lists while fixing the external boundary positions.

Isomorphism class p. 92

A hypernet modulo recursive isomorphism at a fixed depth bound and ordered interface.

Isomorphism-invariant allowed contexts p. 258

An allowed-context predicate preserved by literal-label graph isomorphisms.

Lambda-free translation renaming p. 410

The proved bijective renaming theorem for every original term without lambda constructors, including unrestricted nesting of application, unary and binary operations.

Layered pre-template p. 177

A choice of an interface-indexed focus-free relation at each depth bound; compatibility is not automatic, but holds for the interpretation of a global pre-template.

Lifted state relation p. 36

The relation witnessed by a remote specimen with rooted endpoints equal to the two states being compared.

Literal translation clauses p. 323

Graph constructors following Figure 16, retaining canonical sharing, weakening, and box interfaces.

Local interaction p. 197

A variable trigger, operation entry, or ordered argument return inspected directly in the original focussed graph.

Local rewiring p. 199

Carrier bijections preserving non-focus labels and target incidences, with explicit equations for moving the focus and redirecting sources.

Local-match lowering p. 495

Descent of every actual local rule to the smaller allowance while preserving its stability/copyability premise and proving target representability.

Machine p. 27

A transition relation on states, a successful-finality predicate, and a proof that final states cannot step.

Match embedding p. 499

An edge-injective occurrence preserving primitive labels and ordered incidence; attachment may identify boundary ports.

Matched argument p. 500

The consumer of a specified ordered target of an operation edge; incidence uniqueness fixes the consumer.

Matched head p. 500

The primitive edge consuming a selected wire; both the edge and its primitive are unique in the host.

Micro-beta opening cost p. 473

Six local search/compute transitions reach the opened body; a following body-search trace of length n gives $6 + n$ subsystem transitions.

Middle-block permutation p. 327

The typed exchange $(A, B, C, D) \mapsto (A, C, B, D)$ used before two-use environment distributors.

Multi-use boundary action p. 411

A resource permutation repeated independently at every use of the original environment distributors.

Named graph p. 232

A set graph whose vertices are elements of a finite subset of a common space of names, including its boundary vertices.

Named tuple application p. 220

A tuple-filling certificate modulo simultaneous permutation of its aligned entries; this is parallel application, not a sequential schedule.

Natural arithmetic convention p. 483

Addition and truncated subtraction on the existing natural constants; unary minus is zero minus its operand, hence zero.

Non-self-loop hole p. 55

A selected edge whose source and target lists are disjoint; this condition permits reflection of inserted paths.

Normal state p. 27

A state with no outgoing transition, whether or not it is successfully final.

Numeric arithmetic redex p. 483

An arithmetic operation corolla attached to one or two ordered numeric corollas, rewritten to its computed number corolla.

Numeric operand match p. 504

An ordered argument whose consuming edge is labelled by an actual natural-number constant.

Observation coverage p. 273

Every original hole, including inside original boxes, has a declared slot.

Observation interface p. 34

Typed fragment and context families, an allowed-context predicate, and a possibly partial, nonfunctional relation producing observed states.

Observation slots p. 273

Typed depth-and-hole positions supplied by an independent fragment vector.

Occurrence p. 62

An individual shallow edge or an addressed edge inside boxes; occurrences distinguish positions even when their labels agree.

Occurrence coordinates p. 69

A dependent pair consisting of an outer edge and an address in the occurrence fibre of that edge's label.

Occurrence fibre p. 69

The disjoint sum of the edge's own shallow occurrence and all child occurrences determined by its label.

Occurrence observation p. 69

For a fibre address, the hole lookup obtained either from its shallow label or recursively from its child address.

One-way hypernet p. 152

A hypernet with no operation path from a term input to a term output; paths to thunk outputs are permitted.

Opened search slot p. 208

A selected edge relabelled by a typed hole, giving exactly one shallow hole when the original host is hole-free.

Opening presentation p. 547

Ambient old-source and new-body occurrences, with retention predicates and ordered incidence data describing a complete beta result.

Opening realization p. 547

Injective maps from a genuine graph onto exactly the retained presentation occurrences, preserving wire labels, recursive edge labels and ordered incidences.

Operation kind p. 111

The active/passive classification; finishing produces rewrite/backtrack mode respectively.

Operation path p. 59

A forward, endpoint-sensitive path defined by a reverse operation trace with its edge list reversed; empty paths are permitted.

Operation specification p. 60

The assignment of a term-argument count and an ordered list of thunk binder counts to each operation.

Operation stack p. 244

A proof record of enclosing operation arguments, carrying chronological search visits to their sources.

Operation step p. 58

A step through an edge selected by the operation predicate, from a vertex in its source list to one in its target list.

Operation trace p. 59

A reverse traversal recording each actual operation edge, starting at a target vertex and continuing from that edge's source.

Operation-reachable vertex p. 146

A vertex reached from an input by a path consisting only of actual operation edges.

Opposite specimen system p. 39

The specimen system obtained by swapping its left and right endpoint maps while retaining modes and entering and rootedness predicates.

Ordered parallel assembly p. 312

Disjoint graph assembly with type-preserving bijections specifying the external port order.

Ordered tensor p. 322

Binary graph juxtaposition whose interfaces concatenate the left and right port lists.

Original selected fragment p. 105

The corolla carrying a site's selected label, including the whole body when the label is a box.

Overhead cost order p. 30

The preorder H with $a\,H\,b$ exactly when $a \leq b$; the target successful execution takes at least as many steps as the source.

Padded occurrence p. 367

The same addressed hole after increasing both its host and local depth allowances, with replacement padded by the same number of steps.

Padding attachment law p. 486

Padding a host and its inserted component preserves the actual attached token coordinate after the necessary interface equality casts.

Padding of copying side conditions p. 388

Stability is preserved and reflected by extra unused allowance; copyability is preserved, including arbitrary lambda bodies.

Padding-compatible relation family p. 286

A family of transition relations for which every transition is preserved by padding.

Padding-invariant predicate p. 285

A predicate whose truth is preserved and reflected by one padding step.

Pair boundary coordinates p. 410

The exact correspondence between an operand resource position and its position after the original four-block exchange.

Paired label p. 274

A local certificate for retained, related-hole, or recursively rebuilt-box replacements.

Paired output-closure p. 160

An edge-wise condition: retain both copies or have a one-way filling on at least one side, with the side allowed to vary.

Parallel argument filling p. 363

Replacement through the exact ordered parallel boundary, retaining the selected component and every ordered port attachment.

Path p. 54

A nonempty edge list whose consecutive edges are adjacent. Repeated edges are allowed; this predicate does not specify endpoint vertices.

Pattern matching p. 206

An isomorphism between a plugged finite focus fragment and an actual state, equivalent to focussed coordinate realization.

Plugging site p. 64

An address selecting an edge, possibly inside boxes, indexed by the required local depth bound and replacement interface.

Pointed hypernet p. 173

A typed host graph together with a selected occurrence of a fixed local depth and interface.

Pointed isomorphism class p. 173

A host-with-site pair modulo the equivalence generated by structural recursive site renamings.

Pointed renaming p. 173

A finite zigzag of structural site correspondences, with identity, reversal, and composition.

Port p. 48

A vertex represented as an internal name, an indexed input, or an indexed output, with the three constructors kept disjoint.

Port joining and movement p. 198

Original-port functions that join the old focus endpoints and redirect sources at the newly selected port.

Primitive label p. 60

An operation, variable, focus-mode label, or typed hole; a primitive label has no box body.

Primitive signature p. 60

The typed port assignment for primitives: one term input for non-holes, operation outputs determined by arities, and stored interfaces for holes.

Provenance certificate p. 72

An injective map assigning every result occurrence a retained host or inserted replacement origin with the same hole observation.

Quasi-specimen p. 36

The abstract relation requiring a non-rewrite specimen witness, with prescribed endpoint relations, whenever the compared states are rooted.

Quotient context validity p. 93

The unique-hole predicate on recursive isomorphism classes, justified by invariance under a change of representative.

Quotient freshness p. 174

The condition that the retained-hole inventory of a pointed class is disjoint from the replacement class hole inventory.

Quotient plugging p. 93

For a fixed concrete host and site, the well-defined operation sending a replacement isomorphism class to the class of the plugged graph.

Quotient search p. 134

Erasure-based search descended to focus-preserving literal-label state isomorphism classes; it has unique successor classes.

Read cost handoff p. 536

A full-machine read succeeds in a given cost exactly when that cost is five plus a successful cost of its mandatory copying continuation.

Read graft p. 530

A finite proof construction retaining the exact read distributor and a temporary feedback wire, reconnected to the existing store before comparison with the original reduct.

Reasonable triple p. 29

Three cost preorders satisfying the addition, decrease, inclusion, and sandwich conditions used by counting-simulation soundness.

Reassociation bijections p. 99

Carrier bijections that change the sum tags of host, middle, and final replacement vertices and edges.

Recursive edge certificate p. 177

Evidence that an original edge is retained, filled at a related hole, or rebuilt from recursively filled box bodies.

Recursive isomorphism p. 87

Carrier bijections preserving incidence and primitive labels, with corresponding box bodies recursively isomorphic and all binder metadata preserved.

Recursive label equivalence p. 87

Equality of primitives, or matching box metadata with recursively isomorphic bodies; primitive and box labels are never equivalent.

Recursive paired filling p. 177

A finite paired expansion construction that descends through original box bodies and supplies actual left and right graphs.

Recursive state class p. 140

A state modulo focussed recursive isomorphism, including independent renaming at every nested box depth.

Recursive state search p. 139

Coordinate search after erasure, allowing a focussed recursive isomorphism at the realized target.

Reference allocation rule p. 432

The stable-argument graph rewrite of Figure 19(a), introducing an instance, the canonical atom tree, and a fresh store attached to the preserved argument.

Reindexing p. 56

Renaming a net's internal and edge carriers along bijections, transporting labels and incidence while preserving boundary order.

Relabelling p. 60

Changing operation labels along a signature-preserving map, leaving the graph's carriers, vertex labels, and incidence unchanged.

Related boxes p. 87

Boxes with the same bound/free metadata and bodies satisfying a specified relation at that exact interface.

Related observation vectors p. 274

Pointwise pre-template membership at declared typed slots.

Relation pullback p. 241

The relation on typed graphs obtained by interpreting both arguments as depth-independent graph classes.

Relational isomorphism p. 86

Boundary-preserving carrier bijections preserving ordered incidence and vertex labels, with corresponding edge labels related by a supplied relation.

Relational plugging congruence p. 91

The construction extending a replacement isomorphism for a reflexive label relation by identity on the surviving host.

Relocated context p. 258

The unique-hole context obtained by erasing the old focus and inserting a new one at a typed coordinate.

Relocated specimen p. 260

An allowed recursive specimen rebuilt at a new coordinate, with certified fillings and closed actual endpoints.

Remote coordinate p. 193

A coordinate which is not in rewrite mode at a hole source.

Remote specimen p. 36

A specimen whose mode is not rewrite or which is not entering; only an entering rewrite focus is excluded.

Renaming witness p. 422

An intermediate graph reached by a finite structural derivation, with an isomorphism to the target and its exact resource action.

Replacement carriers p. 51

The disjoint union of host and replacement internal names, and the union of surviving host edges with inserted replacement edges.

Representation class p. 238

An actual-content hypernet modulo recursive type-preserving graph isomorphism; these classes are equivalent to the existing typed graph classes.

Residual contraction tree p. 445

The matched term contraction tree with input zero closed by an actual weakening and all other input positions retained in order.

Residual filling family p. 165

The assignments left after one filling: original assignments on retained edges, identity corollas on inserted edges.

Resource boundary action p. 412

A certificate specifying exactly how variable and atom output positions move, ensuring independently proved operand isomorphisms agree.

Result-and-discard graph p. 515

A constant at the sole input and two term weakenings at the two outputs; a proof construction used to compare actual equality successors.

Retained hole label p. 73

A label observed below a surviving host edge, excluding the selected removed edge and all of its descendants.

Reverse operation traversal p. 58

The machine traversing selected operation edges backwards, taking boundary inputs as successful final states.

Rewrite mode p. 35

The abstract focus mode rewrite, used in the robustness premise for a remote specimen.

Robustness p. 37

The abstract condition requiring a rooted, remote rewrite specimen's source step to lead to stuckness or a cost-related quasi-specimen match.

Rooted state p. 35

A state satisfying the specimen interface's supplied predicate Root, which its laws require to be preserved by transitions.

Rule-label admissibility p. 506

A primitive predicate containing every operation, instance, store, weakening and contraction introduced by the rules.

Scoped graph isomorphism p. 360

Equality of the finite depth indices followed by recursive graph isomorphism; this does not identify different padding allowances.

Search and allocation subsystem p. 435

The actual graph-changing machine combining intrinsic search with rooted stable-argument allocation, optionally retaining a serial continuation; the other UAM rules remain pending.

Search and read subsystem p. 441

Intrinsic search plus the local dereference computation; its five-step read execution ends at a pending copy request, not a returned value.

Search interaction fragment p. 202

A finite graph containing a variable or operation corolla and one focus at a typed input or output boundary.

Search mode p. 35

The abstract focus mode search, used with entering specimens in the input-safety premise.

Search operation view p. 111

A certified decomposition of an actual operation edge into its singleton source, ordered argument prefix, and remaining thunk targets.

Search origins p. 185

The condition that each operation or variable in a filling belongs to a retained copy or an original hole.

Search outcome p. 112

Entry selects the first argument or finishes; return selects the next argument or finishes after the last.

Search position p. 112

A mode and outer vertex of the underlying graph, realized by insertion of an explicit focus edge.

Search restart p. 245

Replacement of the current focus mode by search at the same underlying vertex.

Search step p. 112

The coordinate relation implementing the five rules of Fig. 9(a–e), with no substitution or compute transitions.

Search visit p. 244

An exact chronological split of an initial execution at a search-mode coordinate.

Search visit (copying) p. 579

An exact factorization of an input-rooted search run through search mode at a specified vertex, recording its earlier occurrence in the run.

Search-only machine p. 142

A machine on state classes containing search transitions and boundary finality, but no substitution or computation transitions.

Search-preserving family p. 182

A simultaneous family retaining literal corolla copies of every original operation and variable edge.

Selected hole label p. 84

The partial observation of the selected edge itself after following a site address; selecting a box returns no hole label.

Selected-resource coordinates p. 396

Explicit edge and vertex bijections describing every weakening, the selected contraction and all ordered incidences of the literal selected-resource clause.

Serial assembly p. 315

Composition by expanding a two-hole pattern whose shared interface becomes internal vertices.

Serial path direction p. 345

The property that a path can pass from the left serial component to the right, but cannot return.

Set presentation p. 224

A finite labelled graph with an arbitrary vertex carrier and injective, disjoint, ordered boundary maps.

Settled specimen p. 36

A specimen in backtracking mode, or in search mode without entering; every settled specimen is remote.

Shallow occurrence p. 62

The occurrence of an edge at the current graph layer, without descending into a box body.

Shallow plugging p. 54

Replacing one outer edge by a graph with its exact signature interface, gluing corresponding boundary ports and retaining the outer interface.

Shallow rewrite match p. 476

A local graph rewrite embedded by typed replacement or expansion, up to recursive representation isomorphism, without evaluating inside unopened boxes.

Shallow specimen p. 161

A focussed unique-hole context with two simultaneous shallow filling families, typed relation witnesses, allowed erasure, and hole-free resulting graphs.

Shallow substitution run p. 167

A finite sequence of ordinary addressed substitutions at shallow edges, with its step count.

Shared-store copy regression p. 448

A concrete copied reference whose two distinct instance edges resolve through the atom distributor to one original store at the same exact cost.

Sharp plugging freshness p. 73

The requirement that no retained host hole label occurs in the replacement; labels confined to the removed region may be reused.

Simple search context p. 209

A globally focus-free graph with one shallow hole edge and no other hole occurrences, including inside boxes.

Simultaneous expansion p. 156

Replacement of every shallow host edge by a graph of its interface, with disjoint tagged internal carriers; corollas retain unchanged edges.

Single observation slot p. 277

The singleton interface representing one original graph replacement.

Single-hole occurrence inventory p. 348

A recursive assertion of exactly one specified hole occurrence, with every other label hole-free.

Site interface transport p. 96

Equality transport of the interface indices of an address, leaving its selected occurrence unchanged.

Site transport p. 97

A classically chosen corresponding address in a recursively isomorphic host, with unchanged local depth and interface.

Slot relabelling p. 207

Changing only the label of a selected edge while retaining its ordered interface; proved immaterial after plugging.

Slot-context search p. 206

Application of a finite corolla interaction in one discarded-edge slot, with both actual state endpoints matched by plugging.

Sole-hole certificate p. 340

An addressed hole occurrence together with recursive absence of every other hole.

Source port p. 49

A source-list vertex selected by a finite index in the edge label's input signature; the arity equation justifies the lookup.

Sourced hole p. 337

An addressed hole with at least one input port; the proved binding-free plugging theorem applies at these occurrences.

Specified-label copy p. 176

A graph with one edge of a specified label and full ordered boundary incidence, permitting an updated box body.

Specimen p. 35

An element of the abstract specimen type, equipped with left and right state endpoints, a mode, and an entering predicate.

Specimen system p. 35

The interface packaging specimens, endpoints, modes and predicates, with rootedness preservation, final agreement, and context-traversal laws.

Stable micro-beta rule p. 470

The Figure 10(b) application rewrite with the copying-language stability premise: remove application and lambda heads, open the box, and connect the stable argument to the bound output.

State equivalence p. 31

Mutual state refinement for the same cost preorder; it compares successful termination and execution counts in both directions.

State isomorphism p. 477

A recursive graph isomorphism preserving the token mode and mapping its actual vertex; it transports combined finite runs with exact costs.

State isomorphism class p. 134

A class of states identified by focus-preserving isomorphisms that preserve full edge labels literally.

State quotient projection p. 141

The surjective map from literal-label state classes to recursive state classes; search executions project and lift with exact lengths.

State refinement p. 31

The requirement that each successful source execution has a successful target execution with counts related by the chosen cost preorder.

Stationary replay conditions p. 247

One-way old fillings and immediate rewrite at other inputs, together with proved context search transport.

Stationary rewrite p. 251

A context-preserving change from rewrite to search at the same designated input, satisfying the one-way and other-input conditions.

Store operand match p. 503

An operation-head and ordered-argument witness together with an exact trace to its actual store edge.

Store successor coherence p. 536

Any two actual computations at an allocation, read, write or atom-equality request have recursively isomorphic complete successor states.

Store-identity comparison outcome p. 464

A boolean justified by two successful store traces: true exactly when their destination store edges coincide, independently of their contents.

Structural BPerm derivation p. 422

A finite syntactic proof tree generated by the concrete BPerm rule, recursive isomorphism and explicit graph congruence operations.

Stuck state p. 27

A normal state that is not successfully final; it cannot step and does not count as successful termination.

Subdivision vertex exchange p. 204

The bijection exchanging an old internal attachment with the fresh focus vertex, identifying the two subdivision orientations.

Substitution fragments p. 211

The actual graphs consisting of a rewrite focus followed by a variable and a computation, and a search focus followed by the same computation.

Substitution presentation p. 253

An erased base graph with its selected variable and proved source and resumed-target isomorphisms.

Substitution unit p. 103

A selected one-edge corolla: filling it returns the inserted graph, while restoring an edge's own corolla returns its host, up to isomorphism.

Successful-cost spectrum p. 497

All natural lengths of successful executions from a program initializer; this set is unchanged by finite padding.

Target port p. 49

A target-list vertex selected by a finite index in the edge label's output signature.

Target-exchange graph p. 522

A graph with two equally typed non-input target endpoints exchanged, retaining all carriers, labels and source incidences.

Term wire p. 60

The wire type $*$, distinguished from a thunk wire and used for ordinary operation inputs and term arguments.

Thunk wire p. 60

The wire type $T^n(*)$, carrying the number n of variables bound by a corresponding box.

Translation depth padding p. 328

Increasing a finite graph's box allowance without adding graph structure, to assemble subterms at a common depth.

Translation language graph p. 324

A hypernet containing only operation, contraction, weakening, and instance primitives, recursively inside boxes.

Tuple filling p. 217

Recursive simultaneous application that visits only original edges and box bodies and records the paired replacements at original holes.

Typed boundary isomorphism p. 394

A bijection on graph ports and edges preserving wire types, input/output roles, recursive edge labels and ordered incidences, while allowing boundary positions to move.

Typed search position p. 119

A mode and term-labelled vertex of a graph of type $* \to \epsilon$, realizable as an explicit focussed state under focus and hole freedom.

Typed single-use extension p. 407

The automorphism of the unchanged single-use environment distributors realising any type-preserving environment permutation on both boundaries.

Unary boundary congruence p. 407

The proved extension of any typed operand boundary isomorphism through the original unary operation and its single-use distributors.

Unary operations p. 58

The condition that every selected operation edge has a source list of length one; no one-output condition is imposed.

Unary term-context filling p. 356

The proved correspondence between actual filling of a translated unary context and the original translation of the filled term.

Unary translation fragment p. 406

The original atomic terms closed under arbitrarily many original unary operations; its bijective translation-renaming theorem is proved.

Unbounded run p. 286

An exactly counted finite run on the union carrier, with each transition witnessed at finite depth.

Unbounded state p. 285

An equivalence class of finite-depth stages under common-depth equivalence.

Uniform box family p. 229

A finite family of separately bounded box contents promoted to a common maximum depth.

Unique holes p. 63

The property that any two occurrences observing the same complete typed hole label are equal, at every nesting depth.

Uniqueness away from an edge p. 81

Uniqueness of hole occurrences whose outer edges differ from one selected edge, with no restriction on the selected fibre.

Universal cost order p. 30

The preorder U relating every pair of natural-number counts, so refinement ignores quantitative cost.

Unrestricted context filling p. 372

A single certified address in every original term-context translation that reconstructs each filled translation using the appropriately padded argument.

Up-to closure p. 32

The relation obtained by placing a candidate relation between a left auxiliary state refinement and a right auxiliary state refinement.

Variable prefix p. 211

A new unary variable edge placed before the sole input of a bound computation, preserving its original edges and boxes.

Vertex label p. 49

The internal vertex's assigned label, or the wire label at the indexed position in the relevant boundary list.

Vertex name p. 232

A triple of natural numbers used as an ordinary vertex identifier in the freshened presentation.

Wire type p. 60

Either the term type $*$ or a thunk type $T^n(*)$ with its binding count.

Zero-input closure counterexample p. 338

Two binding-free graphs whose plugging at a zero-input hole creates a forbidden instance-to-hole path.

B What remains for the paper's concrete theorem

The unbounded contextual sufficiency theorem in the concrete encoding is proved: Section 52 discharges the full-UAM depth-transfer obligation and proves both implications. No unfinished Lean proof obligation remains in that construction. The following paper-correspondence audits remain; they qualify the earlier claim that the paper's theorem was fully settled.

1. **Context closure and observations.** Compare the paper's contextual refinement with `PaperRefines` and the concrete observation construction. Explain the absence of plugging closure from the final Lean premises, and establish the paper's plugging-closed instance with matching observation quantification, or correct the encoding if necessary. See Section 1.5.
2. **Substitution stability and refocusing.** Audit the correspondence between Figure 9(f), Lemma 7.6, and the formal substitution/rootedness argument. Determine why the formal theorem needs no explicit stability premise and whether the encoded rule and rootedness justify that difference. See Section 1.6.

Output closure, input safety, robustness, deterministic and refocusing behaviour, and a reasonable cost triple remain semantic hypotheses on an application. The copying UAM, language translations, and application-specific equivalences were outside the selected general-theorem milestone; Proposition 12.2 is now an explicit extension target.

B.1 Remaining work for Proposition 12.2

Completed prerequisites are recorded in Sections 53–97. Execution-padding invariance, term-context congruence, runtime safety and source selection are proved. Complete successor coherence now covers every store and arithmetic operation, application, and copying. The full finite and unbounded machines have proved successor determinism up to recursive representation, together with exact prefix costs and unique successful-run lengths. The original copying construction, all copyability and stability premises, and the unchanged variable-body copy request are retained. Search, contextual copying and allocation now preserve search rootedness. Exact search history and prefix replay are proved, as are rootedness representation and padding laws. Completed tasks are omitted from the list below. The four remaining tasks are:

1. Prove any further stability/copyability invariants required by the specimen and template arguments beyond recursive label safety and stable-input inertness. Give the completed structural renaming derivations their contextual-equivalence interpretation, using proved BPerm robustness and execution congruence rather than identifying box permutations with ordered isomorphisms. Prove non-bijective renaming and graph-level substitution laws. Preserve the terms of Proposition 12.2 and the translation of Figure 16.
2. Prove refocusing for the remaining operation families: dereference, assignment, equality, application and arithmetic. Establish specimen locality for the complete unbounded machine and integrate the proved successor determinism up to representation into the concrete specimen-system instance. Prove correspondence with the paper's focused graph representation and preserve the completed execution-isomorphism and exact padding-invariance laws through any necessary semantic corrections. Retain the zero-input qualification of Lemma E.1.
3. Prove all eleven template families' safety and robustness in Table 3's dependency order, including the indirect copying and garbage-collection dependencies. BPerm's concrete graph pair and structural binding-freedom are proved; its operational input-safety and forward/converse robustness remain.
4. Prove the semantic weakening laws, assemble Figure 22's equivalence chain, and obtain Proposition 12.2 for arbitrary distinct-list environments.

Completed syntactic weakening and substitution identities do not discharge the semantic weakening item. No claim of completion is made until the final proposition passes the build and axiom audit.